

无线电干扰源的快速自动定位与清除模型

摘要

针对有界测向误差下无线电干扰源的快速定位与清除,本文以角楔交会描述位置不确定性,将检测点选择、多源航路调度和定向覆盖逐步结合,建立从单源定位到全域无残留判断的模型与算法。

针对问题一,基于示向度误差有界的特点,本文建立了**有界区间角楔交会模型**,把每个检测点的示向度按误差界张成角楔,干扰源必在各角楔交会的公共部分中。由于形成的定位区域为凸多边形,直径归结为顶点集上的最远点对。使用程序模拟构造的五组算例直径在 10.59 ~ 25.64 m 之间,而以该直径为直径的圆并不总能覆盖定位区域,五组算例中有两组失败,越界顶点到圆心距离超过半径约 10.6%。

针对问题二,只掌握一个检测点处的示向度、源点距离未知,无法先定位再布点,故建立**以期望定位区域直径为准则的布点模型**:由两探测束的交会几何写出定位区域直径,并对源点位置与第二次测向的误差求取期望,得到只依赖第二个检测点相对第一个检测点位置 (a, b) 的二维价值函数 $E[D](a, b)$,使用理论方法求解与数值模拟方法求解获得结果吻合。我们选择一个合理阈值 60m,则**第二检测点和候选区域即为相对位置满足 $E[D] < 60m$ 的区域 $C = \{(a, b) | E[D](a, b) < 60\}$** ,具体区域形状见下正文。

针对问题三,建立**联合覆盖、定位与清除模型**,以保守定位控制清除风险,以连续覆盖判据验证任务完成,并逐步引入共用航路、沿途定位和逐频道检测调度。在 120 组场景开展五策略配对实验,另作 80 次单因素消融。随机 50 组中, *joint* 全部清除,完整退出时间均值 3067 s,较初代减少 42.6%。结果表明,联合规划能同时减少行程与检测成本。

针对问题四,建立**连续位置与朝向覆盖模型**,以带空间余量的角区间并集证明无残留,结合保守交会、双侧补测及光学清除覆盖,形成 21 站紧凑布局与信息航路策略。离线 60 组的 779 个源全部清除并认证;随机主集完整退出均值为 5819.1 s,较 25 站联合策略降低 11.25%,完整退出每源均值为 461.3 s。

所建模型以保守几何约束提供定位和退出依据,以期望估价及航路启发式降低成本;其效率受先验分布与候选范围影响,尚无全局最优保证。

关键词: 角楔交会 期望布点 联合航路 连续覆盖

一、问题背景与重述

1.1 问题背景

随着无线电技术的普及和广泛应用，频谱管理中的干扰源定位与清除问题日益突出。干扰不仅影响正常通信，还可能危及公共安全。传统清除方式通常受环境遮挡、低功率发射等影响，精准定位耗时长，且危险环境下人工作业风险高，因此研发机器狗自动定位清除系统代替人工执行任务具有现实需求。但任务中通常面临着目标区域范围较大，干扰源数量、频道、位置、有效接收半径及类型未知，测向存在误差，移动、检测、切换频道和清除过程存在耗时，定向干扰源的发射方向无法辨识等困难。如何在信息不完全且时间受限的条件下高效完成任务，是该自动定位清除系统亟待解决的问题。

1.2 问题重述

考虑到上述实际情况中面临的困难，本文需依次解决以下四个问题：

1. 已知若干检测点的位置及干扰源关于这些检测点的示向度，给出用交会定位法计算可能位置区域（多边形）直径的算法，并回答以该直径为直径的圆能否覆盖此区域。
2. 已知在某一个检测点处测得的示向度，给出第二个检测点的选择策略，并指出该点可取在什么范围（候选区域）之内。
3. 区域内存在多个干扰源，它们占用互不相同的频道且总个数未知，机器狗从给定起点出发自动定位并逐个清除。要求给出策略与算法，经模拟器演练检验后给出清除比例与平均定位清除时间，并据以完成正式测试。
4. 区域内除全向干扰源外还混有定向干扰源，其个数与发射方向均未知，其余条件与问题三相同。要求给出相应的检测与清除策略及算法。

二、问题分析

四个子问题层层递进。问题一是单次交会定位的几何基础；问题二则将该几何模型进一步用作检测点位置的选择依据，形成单源定位策略；问题三在严格的时间预算下，把单源定位能力嵌入“测向、收敛、清除”的在线决策；问题四基于问题三模型的基础之上，再额外引入定向源方向诊断对决策的影响。

2.1 问题一的分析

本问不宜假设误差分布，而应利用题目给出的误差界，采用最坏情形模型：每个检测点的示向度对应一个由“测量方向 $\pm 1^\circ$ ”张成的角楔。多个检测点的角楔求交，即得干扰源的可能定位区域。由于角楔是凸集，其交仍为凸集（有界时即为凸多边形），因此区域直径可转化为凸多边形顶点间的最远点对问题。求解时，需要先判断交会区域是否有界，再检验以该直径为直径的圆能否覆盖定位区域。整体求解思路可概括为：由误差界构造单点角楔，通过多检测点角楔交会确定凸多边形定位区域，进而将区域直径计算归结为顶点间的最远点对问题；同时辅以有界性判定与圆覆盖检验。

2.2 问题二的分析

问题的关键在于第二个检测点无法由单次观测唯一确定：只掌握 S_1 处的示向度，源点的距离信息完全缺失，故既不能先定位再布点，也不存在一个位置在每次试验中都最好。本文改以“布点后的定位效果之期望”反向衡量布点位置，沿用问题一中“定位区域直径”这一判据，取其对第二次检测点示向度 θ_2 的测量不确定性与源点位置的不确定性的期望为关键判据，既不依赖某一次定位的运气，也无需先知道源点的距离。又因直径的期望只由两次探测的交会几何决定，它只依赖第二个检测点相对第一个检测点的位置，与绝对坐标无关。于是本问归结为以第二检测点对第一检测点的相对位置为决策变量、以 $E[D](a, b)$ 为价值函数的二维寻优问题，其等值线内部有界域即第二个检测点的候选区域。

2.3 问题三的分析

源数未知使“当前目标清完”与“任务完成”并不等价。本问将保守角楔定位、逐频道无信号覆盖与清除航路统一建模，经历局部选点、系统扫圈、共用航路、沿途定位及联合扫描调度五代改进，以完整退出时间和清除比例共同评价，并用连续覆盖证书判断何时停止。

2.4 问题四的分析

定向源可能因测点位于发射背面而无信号，因而问题三的距离圆盘排除规则不再适用。本问把未知发射朝向纳入搜索状态，以整方格与连续角区间的覆盖建立无源证书；定位阶段结合角楔外包、短横向基线与光学清除覆盖。算法由格网扫描发展为有证据的联合搜索，再通过压缩站位、改进目标估计和信息排路降低完整退出成本，从而兼顾发现效率与无残留的判断依据。

三、模型假设

为便于建模求解，在合理范围内作如下假设：

1. 示向度误差有界， $|\Delta\theta_i| \leq \varepsilon$ （本题 $\varepsilon = 1^\circ$ ），且同一检测点的误差在观测期间固定不变，只有更换地点才呈现统计规律；故采用有界区间模型（最坏情形），所得区域保证包含真值。
2. 检测点坐标与示向度读数精确已知；示向度以 x 轴正向逆时针为正，取值于 $[0^\circ, 360^\circ)$ 。
3. 干扰源位置在观测与清除期间固定不变，各源频道互不相同且不随时间变化。
4. 测向机与干扰源位于同一平面，区域内无障碍遮挡。
5. 全向源的覆盖角度为 360° ，定向源为定向方向两侧各 90° （含边界），覆盖范围内信号均匀辐射、场强仅随距离衰减。
6. 机器狗沿直线以 5 m/s 匀速行进，忽略转向与加减速耗时；移动中不可检测，单次检测 5 s ，任意两频道切换 1 s 。
7. 与干扰源距离不超过 20 m 时，光学定位（ 3 s ）与激光清除（ 2 s ）必定成功且与覆盖角度无关；距离不超过 5 m 且处于覆盖角度内时无法获得示向度，可直接定位并清除。

四、符号说明

本文使用的主要符号见表 1，未列出的符号在首次出现处说明。

表 1 主要符号说明

符号	含义	单位
$S_i = (x_i, y_i), \theta_i$	第 i 个检测点的坐标与示向度	m, °
$T = (x_T, y_T)$	干扰源（待定位目标）的真实位置	m
$\Delta(\alpha, \beta)$	两方位角之差，归一到 $(-180^\circ, 180^\circ]$	°
ε	示向度误差界（本题 1° ）	°
$\mathcal{R}, V$	定位区域及其顶点集	无
D	定位区域直径	m
M	直径圆圆心（直径中点）	m
A, B	直径端点	m
$S_1, S_2 = (a, b)$	第一、第二个检测点及其相对位置	m
θ_2	第二个检测点处的示向度	°
φ	候选区域极角（自 S_1 的示向度方向量起）	°
x_{max}	干扰源最大可被有效接收距离	m
$E[D](a, b)$	定位区域直径的期望（第二个检测点的价值函数）	m

五、模型建立与求解

5.1 问题一：示向度交会定位模型

5.1.1 模型建立

设第 i 个检测点为 $S_i = (x_i, y_i)$ ，测得的示向度为 θ_i （ x 轴正向逆时针，取值 $[0^\circ, 360^\circ)$ ），误差界为 $\varepsilon = 1^\circ$ 。干扰源则落在以 S_i 为顶点、关于示向度方向对称张开 2ε 的角楔之内（为避免方位角相减在跨 0° 时出错，记两角之差 $\Delta(\alpha, \beta) = ((\alpha - \beta + 180^\circ) \bmod 360^\circ) - 180^\circ$ ）：

$$\mathcal{W}_i = \{S_i\} \cup \{T \neq S_i : |\Delta(\beta(S_i, T), \theta_i)| \leq \varepsilon\}, \quad (1)$$

其中 $\beta(S, T)$ 为自检测点 S 指向干扰源 T 的方位角。 m 次测向给出的定位区域即各角楔之交，如图 1 所示（以 $m = 3$ 为例）。每个角楔可表示为两个闭半平面之交；各角楔对干扰源的约束同时成立，故定位区域为 $\mathcal{R} = \bigcap_{i=1}^m \mathcal{W}_i$ ，它是 $2m$ 个半平面之交，为凸多边形区域，可能为空、无界，或退化为点、线段。非空有界时其顶点数不超过 $2m$ ，且每个顶点必是某两条边界直线的交点。记该区域的顶点集为 V 。

若各示向度方向过于接近， $\mathcal{R}$ 沿公共方向无限延伸，非空定位区域的直径为 $+\infty$ ，故在确认非空后须判定有界性。由于无界是无穷远处的性质，判据只涉及方向：角楔是顶点在 S_i 的锥，若某方向同时落在各角楔张开的方向区间内，则区域内任一点沿该方向平移任意远都不会离开区域，区域因而无界；反之，若区域有界，取其中趋于无穷的一列点单位化，由单位圆的紧性可取收敛子列，而自各 S_i 指向该点列的方位角都趋于

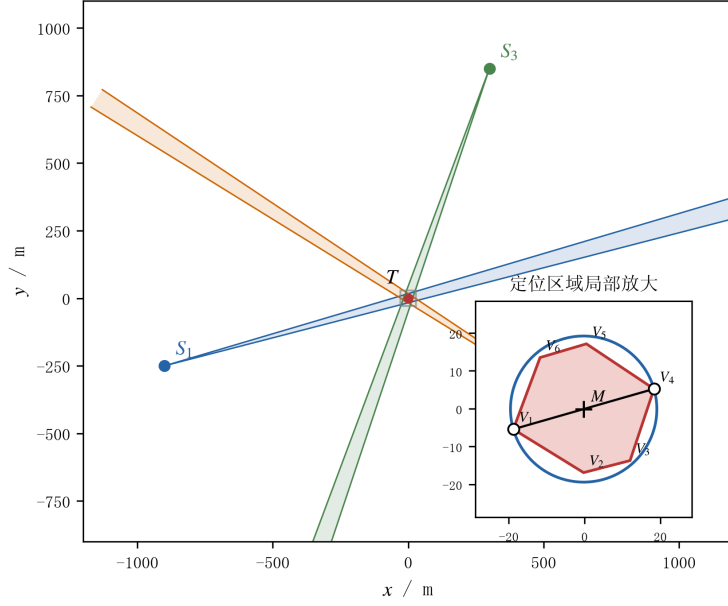


图1 多检测点示向度交会得到的定位区域; 右下角的局部放大图中标出交点 V_i 和直径中点 M

该极限方向，角楔又是闭集，故极限方向必落在每个区间内。于是对于非空 $\mathcal{R}$ ，无界当且仅当各方位角区间 $\Theta_i = [\theta_i - \varepsilon, \theta_i + \varepsilon]$ （跨 0° 按圆周区间处理）存在公共交集，当 $m = 2$ 时即两次测向的方向夹角不超过 2ε 。该判据只需 $O(m)$ 次角度比较，且不依赖检测点坐标。

对已判定为有界的区域，凸多边形上任意两点的最大距离必由两个顶点达到。故求直径不必在连续区域上作最优化，只需在顶点集 V 上比较点对距离：

$$D = \max_{P, Q \in \mathcal{R}} \|P - Q\| = \max_{P, Q \in V} \|P - Q\|. \quad (2)$$

设使式 (2) 取到最大值的两个顶点为 A, B ，则直径圆圆心为 $M = (A + B)/2$ 、半径为 $r = D/2$ 。

覆盖与否由圆盘的凸性直接判定： $\mathcal{R} \subseteq$ 该直径圆当且仅当其全部顶点含于圆内，即

$$\max_{P \in V} \|P - M\| \leq D/2. \quad (3)$$

条件不满足时，必有顶点 P 使 $\angle APB < 90^\circ$ 而落在圆外。注意该直径圆的圆心固定为 M ，与圆心可自由选取的最小包围圆不同，故式 (3) 需逐个顶点检验。注意到最远点对可能不唯一，但覆盖判定与取法无关：若某一对最远点 A, B 确定的圆已覆盖 $\mathcal{R}$ ，则 $\mathcal{R}$ 含于半径 $D/2$ 的圆内，而 $\mathcal{R}$ 内任意两点距离不超过 D ，故任何其他最远点对都必对应于该圆上的一条直径，其确定的圆与原圆重合；若某一对不能覆盖，则任何一对也都不能。因此下文任取一对最远点构造直径圆即可。

5.1.2 模型求解算法

算法输入为 m 组“检测点坐标与示向度”及误差界 ε ，输出为顶点集 V 、直径 D 、直径端点 A, B 、圆心 M 及覆盖判定，流程如图 2，分四步。

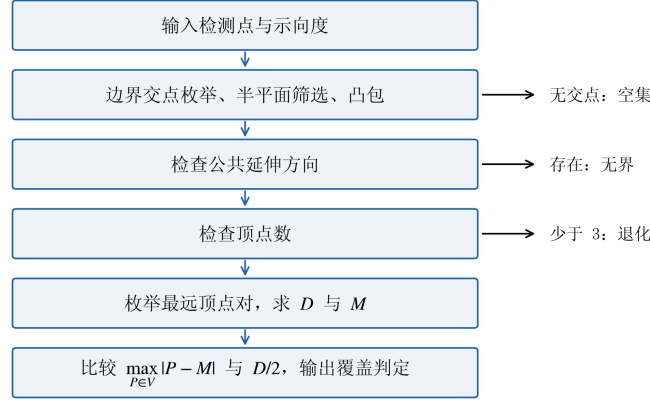


图 2 问题一求解算法流程

第一步（区域构造与非空检验）：区域的每个顶点都是两条边界直线的交点，故算法枚举每个角楔两条边界射线所在直线（共 $2m$ 条）的两两交点，再用全部 $2m$ 个半平面判据逐一筛选；同一角楔两条边界线的交点即检测点 S_i ，它同样作为候选点保留。随后对候选点集求凸包（包含全部候选点的最小凸多边形，用 Andrew 单调链实现），剔除重复点与近似共线点，得到逆时针排列的顶点表 V 。若候选点集为空，说明各角楔无公共交集（测量数据自相矛盾），算法返回状态 `empty`（无可行定位区域）并终止该例计算。

第二步（有界性与退化检验）：对非空交集，检查 m 个方位角区间 $[\theta_i - \varepsilon, \theta_i + \varepsilon]$ 是否存在公共方向。若存在，返回 `unbounded`；若不存在且 $|V| < 3$ ，返回 `degenerate`，表示点或线段。其余情形得到有界凸多边形，继续计算直径。

第三步（求直径）：按式 (2) 枚举 V 中的顶点对取最大距离，对应两点记为 A, B ，圆心取 $M = (A + B)/2$ ；对本文算例规模直接枚举即可，计算量为 $O(|V|^2)$ 。程序在顶点较多时使用旋转卡壳算法，其距离比较次数随顶点数线性增长；另以两法交叉计算检查直径的一致性。区域构造需枚举 $O(m^2)$ 个交点，每个交点检验 $2m$ 个约束，故直接实现的总体计算量为 $O(m^3)$ 。

第四步（覆盖判定）：按式 (3) 计算 $\max_{P \in V} \|P - M\|$ ，判定条件为

$$\max_{P \in V} \|P - M\| \leq D/2 + \tau, \quad (4)$$

其中绝对容差 $\tau = 10^{-9} \text{ m}$ 用于容忍相切构型的浮点误差，避免相切构型被误判为不覆盖；判定不覆盖时，输出取到该最大值的顶点作为越界位置。

为保证结果不依赖具体实现，程序另有一套独立等价的区域构造算法：自一个大正方形出发，用 Sutherland-Hodgman 算法逐个半平面裁剪多边形。在五组算例上的顶点最大偏差小于 10^{-9} m ，直径枚举与旋转卡壳的结果一致，状态名与原因说明见附录 B 的 `analyse()`。

5.1.3 求解结果与分析

使用程序构造五组良态算例（检测点分布在干扰源四周，方位张角 $180^\circ \sim 249^\circ$ 、最小方位间隔不小于 25° 、与干扰源距离 $160 \sim 1300 \text{ m}$ ），结果见表 2；五组算例的定位区域直径均在 $10.59 \sim 25.64 \text{ m}$ 之间，即两次以上交会已能把候选范围从测向距离的百米

量级收缩到数十米以内。表中同时给出覆盖判定，其两种结果的对照见图 3：左图（算例 A_1 ）的定位区域全部顶点都落在直径圆内或圆周上，最远顶点到圆心 M 的距离恰为 $D/2$ （比值 1.0000），此时直径圆就是该区域的最小包围圆；右图（算例 A_2 ）中，橙色虚线标出的顶点 P 落在圆外， $|PM| = 10.29 \text{ m}$ 比 $D/2 = 9.31 \text{ m}$ 大 0.98 m （比值 1.1055），直径圆覆盖判定为“否”。

表 2 问题一算例的计算结果

算例	m	顶点数	$D \text{ (m)}$	$D/2 \text{ (m)}$	$\max \ P - M\ \text{ (m)}$	比值	覆盖
A_1	6	6	17.64	8.82	8.82	1.0000	是
A_2	5	5	18.61	9.31	10.29	1.1055	否
A_3	4	5	25.64	12.82	13.75	1.0722	否
A_4	5	4	21.12	10.56	10.56	1.0000	是
A_5	6	5	10.59	5.30	5.30	1.0000	是

可见五组算例中有两组（ A_2 、 A_3 ）的直径圆无法覆盖定位区域，越界顶点到 M 的距离分别比 $D/2$ 大 0.98 m 与 0.93 m ，超出 10.55% 与 7.22%；这两组算例的区域均为斜五边形，由于 A_2 中两个顶点距离较近，故可以近似为四边形，而越界顶点所对直径形成的张角小于 90° （图 3 右图中圈出的顶点张角为 84.2° ），因而落在圆外。

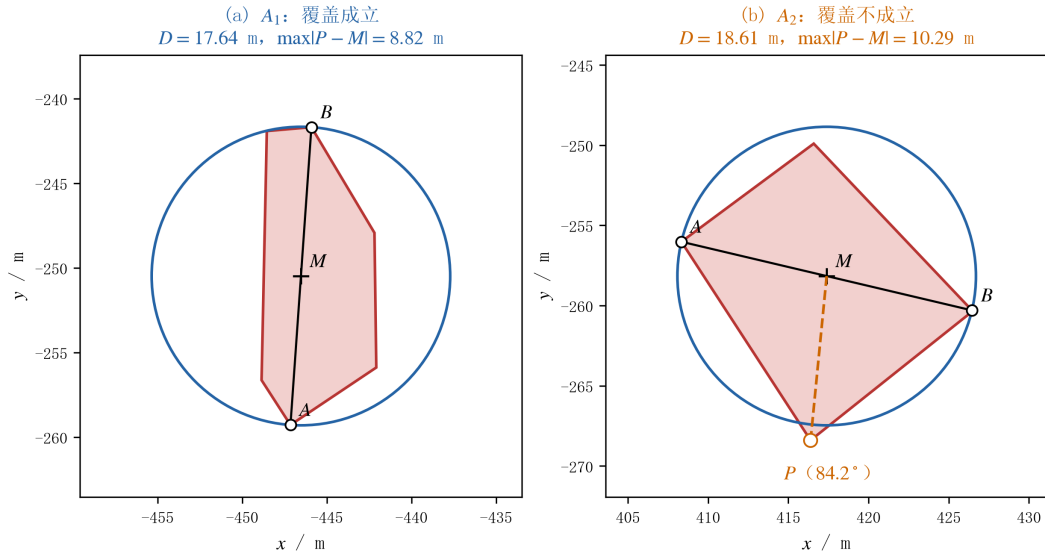


图 3 直径圆覆盖判定的两种结果：(a) 覆盖成立；(b) 覆盖不成立

为检验覆盖失败是否仅为个别现象，本文以固定随机种子作四组几何扫描（表 3）。叠加测向误差后，3 ~ 6 个检测点的覆盖失败率由 3.25% 升至 28.60%、最坏时直径圆需放大约 69%。测向误差可能改变多边形形状并提高覆盖失败比例，但误差不一致并不必然导致失败。上述比例针对本文规定的构型采样分布，不能解释为所有实际布点的普遍失败概率。其中两点扫描用于检验纯角楔几何性质，未按有效接收半径筛选检测点。本节数值的数据清单与复现命令见附录 A。

表 3 四组随机扫描的覆盖失败统计（固定种子，理想/实际构型逐例配对）

构型	检测点数	抽样数	有界有效数	失败数	失败率	最坏比值	需放大约
理想	2	20000	20000	231	1.16%	1.0336	3.4%
理想	3 ~ 6	4000	3472	113	3.25%	1.0304	3.0%
实际	2	20000	19996	246	1.23%	1.0346	3.5%
实际	3 ~ 6	4000	3472	993	28.60%	1.6876	68.8%

注：理想构型指各示向度精确指向源点；实际构型指在上一批构型上叠加与算例生成同分布的 $\pm\epsilon$ 测向误差；失败率以有界有效数为分

母，“需放大约”为最坏比值减 1。两组 3 ~ 6 点扫描各有 528 组未满足方位间隔要求而被筛除，不计入有界有效数。

5.2 问题二：第二检测点的选择模型

5.2.1 模型建立

记号与概率描述

S_1 置于原点，其示向度方向取为 x 轴正方向； S_2 置于 (a, b) 。已知 S_1 的示向度后，源点 T 可能位置 (x_T, y_T) 服从一个条件概率分布

$$T \sim P_T(T), \quad (5)$$

而给定源点位置与 S_2 的位置， θ_2 的读数由于误差亦不能确定，近似服从

$$\theta_2 \sim P_\theta(\theta_2 | T, S_2). \quad (6)$$

位形参数 (θ_2, a, b) 一经确定，定位区域直径便由几何关系唯一确定给出：

$$D = D(\theta_2, a, b). \quad (7)$$

于是 D 服从由几何参量 (a, b) 控制的概率分布，第二个检测点的选择问题即化为求如下二维价值函数：

$$E[D](a, b) = \iint d^2 \vec{r}_T P_T(T | S_1) \int d\theta_2 D(\theta_2, a, b) P_\theta(\theta_2 | T, S_2), \quad (8)$$

$E[D](a, b)$ 越小，在该点放置第二个检测点所得定位平均效果越好。这即为问题二求解的关键衡量指标。

5.2.2 理论求解

(a) 理论推导

式 (8) 含内外两次共计三重概率积分，直接使用不便，故作以下五条近似：

1. S_2 距 S_1 的距离 R 不太小。该近似的合理性并非天然成立： S_2 距 S_1 过近时两视线的交角趋于零，所得定位区域必然很大，这是我们不希望的，故 R 较小的区域本来就不在选择范围之内，而本问关心的待选区域附近该近似较好。
2. 忽略源点纵坐标，认为其恰好落在 x 轴上，其位置只由一个参数 x_T 描述。当 R 不小时源点纵坐标 y_T 是小量，将分布对 y_T 作展开：

$$P_\theta(\theta_2 | x_T, y_T, a, b) \sim P_\theta(\theta_2 | x_T, 0, a, b) + \frac{d}{dy_T} P_\theta \Big|_{y_T=0} y_T + o(y_T) \quad (9)$$

而在第一个概率积分中区间关于 $y_T = 0$ 对称，一阶项相互抵消，故 y_T 对 $E[D]$ 的影响是高阶小量。

3. 考虑 y_T 小量近似后所得零阶项 $P_\theta(\theta_2 | x_T, 0, a, b)$ 在扇形区间内积分，可认为先验权重满足 $P_T(x_T | S_1) \propto x_T$ ，归一化后得

$$P_T(x_T | S_1) = \frac{2x_T}{x_{max}^2}, \quad x_T \in [0, x_{max}], \quad (10)$$

其中 x_{max} 为干扰源的最大可被有效接收距离，在本问中可直接取为 1500m。

4. 与第 2 条同理，由于检测点二实际测得方向角在理论值 $\theta_{2,ideal}$ 左右 1° 区间内均匀分布，偏离值为小量认为其分布近似 $\theta_2 \sim \delta(\theta_2 - \theta_{2,ideal})$ ，即等效直接认为第二次探测所得方位正对源点。
5. 在计算第二检测点的定位区域直径期望时忽略探测半径的影响，而将第二检测点的定位区域角楔延伸到足够长进行计算。这条近似在第二检测点距离第一检测点定位区域不太远时成立。因为此时整个第一检测点的定位区域完全包含在第二检测点的定位区域内，第二检测点的探测半径对定位区域直径的影响可以忽略。

五条近似把价值函数精简为单个一维积分：

$$E[D](a, b) = \int_0^{x_{max}} dx_T P_T(x_T | S_1) D(x_T, a, b). \quad (11)$$

(b) 定位区域直径的解析推导

近似取定 θ_2 正对源点后，两条探测视线在源点处的交角 α 满足

$$\sin \alpha = \frac{|b|}{r_{2T}}, \quad r_{2T} = \sqrt{(x_T - a)^2 + b^2}, \quad (12)$$

图 4 为一个特定情况下的两个检测点示意图，并取源点 T 、第二检测点 S_2 ；图中蓝、黄两个区域分别为两次探测各自的定位区域， $\angle S_1 T S_2 = \alpha$ 为两视线在源点处的交角，右上角为两区域相交所得的定位区域 $ABCD$ 的局部放大图。所得定位区域为局部图中的红色近似平行四边形 $ABCD$ ：该区域的直径即对角线 $|BD|$ 。

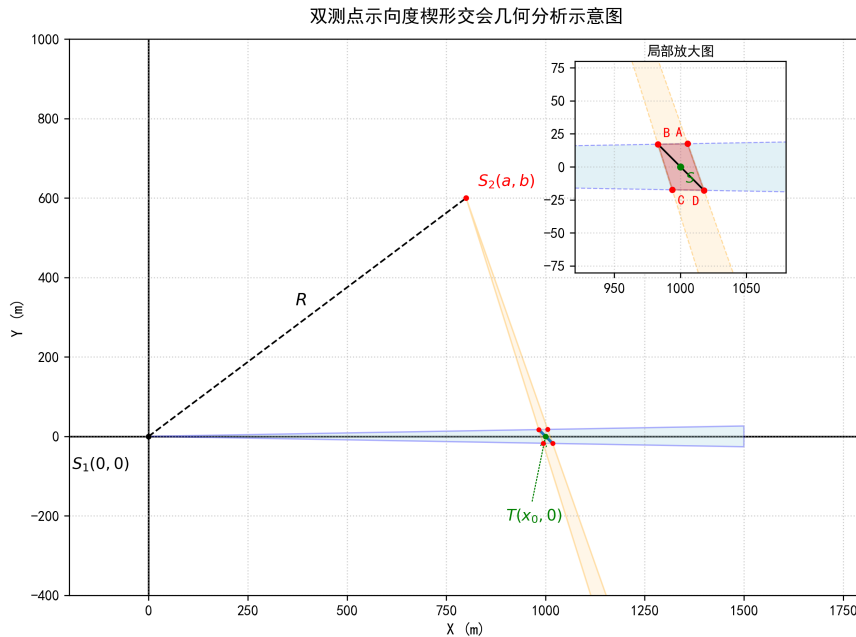


图 4 两探测束交会产生定位区域示意图

本文探测束张角极小 ($\Delta\theta = 2\varepsilon = 2^\circ \approx 0.0349 \text{ rad} \ll 1$), 故在源点处两次探测束的横向宽度分别为

$$w_1 = x_T \Delta\theta, \quad w_2 = r_{2T} \Delta\theta = \sqrt{(x_T - a)^2 + b^2} \Delta\theta. \quad (13)$$

由式 (12) 及 $(x_T - a)/r_{2T}$ 的符号, 交角还满足

$$\cos \alpha = \frac{x_T - a}{r_{2T}} = \frac{x_T - a}{\sqrt{(x_T - a)^2 + b^2}}. \quad (14)$$

其对角线长度由两组对边的宽度与夹角 α 完全确定:

$$D(x_T, a, b) = \frac{\sqrt{w_1^2 + w_2^2 + 2w_1w_2 \cos \alpha}}{\sin \alpha}. \quad (15)$$

该式有直接的几何解释: 下标 1、2 的两项分别对应两次探测各自的束宽, 交叉项 $2w_1w_2 \cos \alpha$ 则来自两组对边之间的夹角。

把式 (13)、(12) 与 (14) 代入式 (15), 即可消去 w_1, w_2, α , 将直径完全显式化为 a, b, x_T 的函数。分子先提出公因子 $r_{2T}\Delta\theta$, 被开方式化为 $x_T^2 + r_{2T}^2 + 2x_T(x_T - a) = 4x_T^2 - 4ax_T + a^2 + b^2$, 于是

$$\begin{aligned} D(x_T, a, b) &= \frac{\Delta\theta r_{2T}}{|b|} \sqrt{x_T^2 + r_{2T}^2 + 2x_T(x_T - a)} \\ &= \frac{\Delta\theta}{|b|} \sqrt{(x_T - a)^2 + b^2} \sqrt{(2x_T - a)^2 + b^2}, \end{aligned} \quad (16)$$

其中第一个根式即式 (12) 中的 $r_{2T} = |S_2T|$, 第二个根式是源点到点 $(a/2, -b/2)$ 之距离的两倍。定位区域直径由这两个几何量的乘积与 $|b|$ 之比给出。

式 (16) 的合理性可由两个极限情形检验: 当第二个检测点正落在第一个视线方向附近 ($a \rightarrow x_T, b \rightarrow 0$) 时交角趋于 90° , 此时 $D \rightarrow \Delta\theta x_T = w_1$, 恰为定位区域沿垂向的宽度; 当 $b \rightarrow 0$ 且 $x_T \neq a$ 时两视线共线、 $\sin \alpha \rightarrow 0$, $D \rightarrow \infty$, 与式 (12) 的退化判据一致。

代入 $\Delta\theta = 2^\circ = \pi/90 \text{ rad}$, 得直径函数的解析表达式

$$D(x_T, a, b) = \frac{\pi}{90|b|} \sqrt{(x_T - a)^2 + b^2} \sqrt{(2x_T - a)^2 + b^2}. \quad (17)$$

再代回式 (11) 并利用式 (10), 即得可直接用于数值积分与优化的目标函数

$$E[D](a, b) = \frac{\pi}{45 x_{max}^2 |b|} \int_0^{x_{max}} x_T \sqrt{[(x_T - a)^2 + b^2][(2x_T - a)^2 + b^2]} dx_T. \quad (18)$$

引入无量纲变量 $u = x_T/x_{max} \in [0, 1]$ 与 $\hat{a} = a/x_{max}$ 、 $\hat{b} = b/x_{max}$, 式 (18) 还可写成

$$E[D](a, b) = \frac{\pi x_{max}}{45 |\hat{b}|} \int_0^1 u \sqrt{[(u - \hat{a})^2 + \hat{b}^2][(2u - \hat{a})^2 + \hat{b}^2]} du, \quad (19)$$

即 $E[D]$ 只依赖 x_{max} 与无量纲位形 $(\hat{a}, \hat{b})$, 与绝对尺度无关。

(c) 价值函数的闭合解

价值函数的不定积分核为

$$\int x_T \sqrt{[(x_T - a)^2 + b^2][(2x_T - a)^2 + b^2]} dx_T. \quad (20)$$

作平移 $t = x_T - a/2$ (即 $x_T = t + a/2$)，两个二次因子化为关于 t 的偶函数

$$(x_T - a)^2 + b^2 = \left(t - \frac{a}{2}\right)^2 + b^2, \quad (2x_T - a)^2 + b^2 = 4t^2 + b^2, \quad (21)$$

即根号内是 t 的**四次多项式** (偶四次)。因此式 (20) 的原函数属于椭圆积分类型，不能由初等函数显示表示。

我们使用 Mathematica 软件对理论公式 (18) 进行了计算绘图，计算结果见 5.2.4 节。

5.2.3 数值方法求解

为了与由理论近似解析式得到的 $E[D](a, b)$ 交叉验证，我们使用 Python 程序对该分布作了数值求解。算法输入为已知示向度方向 (用于建立坐标系)、有效接收半径 x_{max} 与误差界 ε ；输出为定位区域直径期望的分布 $E[D](a, b)$ 最优相对位形 (a^*, b^*) 、最小期望直径 $E[D]_{\min}$ 与候选区域，分四步。

第一步建立标准坐标系： S_1 置于原点，其示向度方向取为 x 轴正向， $S_2 = (a, b)$ ；于是几何量 $|b|$ 、 r_{2T} 、 α 都可用 a, b, x_T 写出。

第二步 D 的精确计算：对给定 (a, b, x_T) ，作 S_1 的两条边界射线 (与 x 轴成 $\pm\varepsilon$) 与 S_2 的两条边界射线 (绕 “ $S_2 \rightarrow T$ ” 方向成 $\pm\varepsilon$)。四条直线两两求交得 6 个候选点，再用角楔定义筛出真正落在两个楔形内的顶点 (良态下恰为 4 个，即四边形 $ABCD$)，取

$$D(x_T, a, b) = \max_{P, Q \in \{A, B, C, D\}} |PQ|; \quad (22)$$

两条边界线平行 (两视线共线) 时交会区域无界，该源位样本直接舍弃。这一步用的是四边形的严格几何，与前文解析式的平行四边形近似无关。

第三步求期望。源位采样取在 x 轴正半轴上，上限为有效接收半径上界，在 $[0, x_{max}]$ 上取 n 个分点，以 $w_k \propto x_T^{(k)}$ (即式 (10)) 加权。要注意只有 “第二个检测点也能收到该频道信号” 的源位样本才是可用的：对每个样本还须满足 $|S_2 G_k| \leq x_{max}$ 。把不满足该条件或无界的样本剔除后，在**剩余样本**上重新归一，得

$$E[D](a, b) = \frac{\sum_{k \in \mathcal{K}(a, b)} w_k D(x_T^{(k)}, a, b)}{\sum_{k \in \mathcal{K}(a, b)} w_k}, \quad w_k \propto x_T^{(k)}, \quad (23)$$

其中 $\mathcal{K}(a, b)$ 为可用样本集合。式 (23) 是条件期望 $E[D | \text{两次探测均有效}]$ ，而非对所有源位的无条件平均；这一区别在 S_2 远离 S_1 时不可忽略，故本文一律采用式 (23)。

第四步寻优：网格取以 x_{max} 为半径的靶区圆域、步长 30 m，剔除 $|y| \leq 60$ m 的近轴退化带 (该带内两视线近简并、交会区域被拉成极窄长条甚至无界)，其余格点逐点代入式 (23)，取最小值点即 (a^*, b^*) 。为避免近轴期望值发散区域干扰绘图颜色映射，程序中把 $E[D]$ 超过全体可用格点中位数 10 倍的格点标注为不可用带，不参与寻优。

5.2.4 求解结果与分析

我们取 $x_{max} = 1500$ m、 $\varepsilon = 1^\circ$ ，通过上述两套相互独立的计算方法分别得到结果，并在最后据此确定第二检测点的待选区域。

(a) 理论解计算结果

按照 5.2.2 节分析，我们使用 Mathematica 软件进行计算，将式 (19) 的闭式被积函数，把式 (17) 的直径代入对源位的加权积分，在 15 m 步长的矩形网格上直接作高密度

求积，计算网格覆盖 $a \in [-500, 1795]$ m、 $b \in [-1800, 1800]$ m。图 5(a) 给出 $E[D](a, b)$ 在所在区域内的函数图像。其中 $a < -500$ 区域不在计算处理范围内，绘制为白色。而接近 x 轴部分 $E[D]$ 发散，超出绘图范围与机器精度限制，亦绘制为白色。

(b) 数值模拟计算结果

我们按 5.2.3 节的算法，使用 Python 程序，对每个格点由两个角楔的严格几何求出交会四边形直径，再对源位加权求积绘制了 $E[D](a, b)$ 图像；网格为以 x_{max} 为半径的靶区圆域、步长 30 m。由于在数值模拟中并未采取理论推导时的近似，故绘图区域限制更大。 $|S_2 S_1| > x_{max}$ 或 $|S_2 G| > x_{max}$ 导致两次探测不可能同时有效接收的不可行区，与两视线近似共线、 $E[D]$ 超出色标上限而被截断的区域均绘制为白色

图 5(b) 给出同一价值函数由数值模拟得到的分布。与 (a) 相比，两者的整体形态、极小值位置与色带走向完全一致，极小值为 $E[D]_{\min} = 57.34$ m，位于 $(1080, \pm 660)$ m。

(c) 结果交叉对比

图 5 中的两幅图绘制在参数与颜色映射完全相同的两个坐标系中。直观可见在二者定义域重叠范围内，计算结果的整体形态吻合度极高。

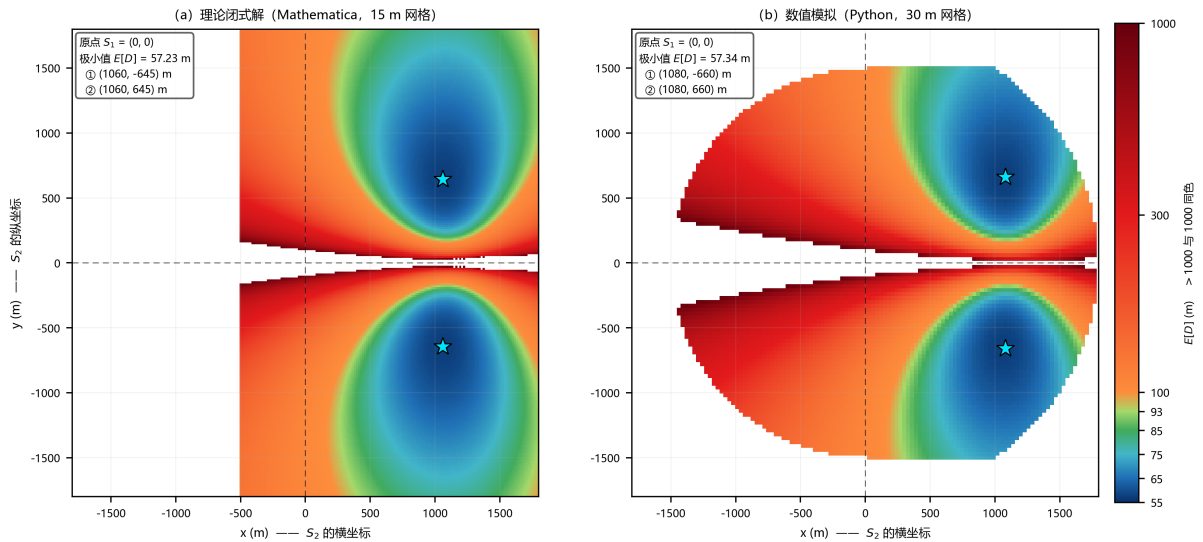


图 5 期望定位区域直径的分布：理论闭式解与数值模拟的并排对比。(a) Mathematica 按解析式在 15 m 网格上求积；(b) Python 按交会四边形严格几何在 30 m 网格上求解。两图的极小值位置与色带走向一致

为进一步定量分析，我们把 MMA 的 15 m 网格结果双线性插值到数值模拟的 30 m 格点上，对两者直接逐点对比。表 4 列出若干代表性位置的结果。在最优带 ($E[D] \leq 60$ m 区域) 内 374 个可对比点上，两者的相对偏差中位数为 0.14%、最大不超过 0.28%；放宽到 $E[D] \leq 100$ m 的 3302 个点上，相对偏差中位数为 0.18%、最大不超过 0.51%。两条完全独立的求解路线在关心的取值范围内完美吻合，说明 5.2.2 节的解析推导、5.2.3 节的四边形几何与其数值自洽自洽性。

在两种计算结果中观察 $E[D]$ 图像性质，其取值从 57 m 一直延伸到色标上限 10^3 m 以上，且在极小值附近平缓而在 x 轴附近发散，故图中采用非线性色标：把 55 ~ 100 m 一段拉开、100 m 以上压缩，提升问题更关心的低值区的区分度。图像关于 $b = 0$ (S_1 的

表 4 理论闭式解与数值模拟在代表性第二检测点位置的对比

a (m)	b (m)	理论闭式解 $E[D]$ (m)	数值模拟 $E[D]$ (m)	相对偏差
1080	660	57.256	57.336	+0.14%
1050	660	57.260	57.351	+0.16%
1110	750	57.784	57.849	+0.11%
1200	690	58.932	58.963	+0.05%
1200	600	59.194	59.222	+0.05%
1200	900	60.441	60.475	+0.06%
900	900	61.038	61.154	+0.19%
1350	750	64.434	64.406	-0.04%
900	1200	65.973	66.066	+0.14%
1410	600	69.941	69.856	-0.12%
1500	900	72.758	72.684	-0.10%
600	600	78.228	78.499	+0.35%
300	900	94.310	94.561	+0.27%

视线方向)严格上下对称,极小值成对出现,这是由体系对称性所决定的。 $E[D] \leq 60m$ 的最优带为极小点周围的一小片区域,在此范围内靠近和远离检测点一的探测区间,都会使得最终探测区间的直径期望增大。这是由于当检测点二距离检测点一的定位区域很近时,观测角夹角正弦 $\sin\alpha$ 平均值小,使得最终定位区域重叠极大,直径快速发散。当检测点二距离检测点一的定位区域很远时,检测点二的定位区域在干扰源点处展宽已经变得较大,使得定位区域直径增大。

(d) 待选区域确定

问题二要求给出的是第二检测点的可选范围。综合上文计算结果,本文选取 $E[D] \leq 60\text{ m}$ 的区域作为第二检测点的待选区域:该阈值比极小值高约 5%,既保证定位效果(期望直径接近销毁半径数量级),又给现场留出足够的位形余量。具体范围见图 6所示。两套计算给出的同一区域的边界几乎重合。实际布点时建议取区域中心附近的 $(a, b) \approx (0.71 x_{max}, 0.43 x_{max}) = (1065, 645)\text{ m}$,此时 $E[D]$ 接近极小值,在期望意义下最利于确认干扰源位置。

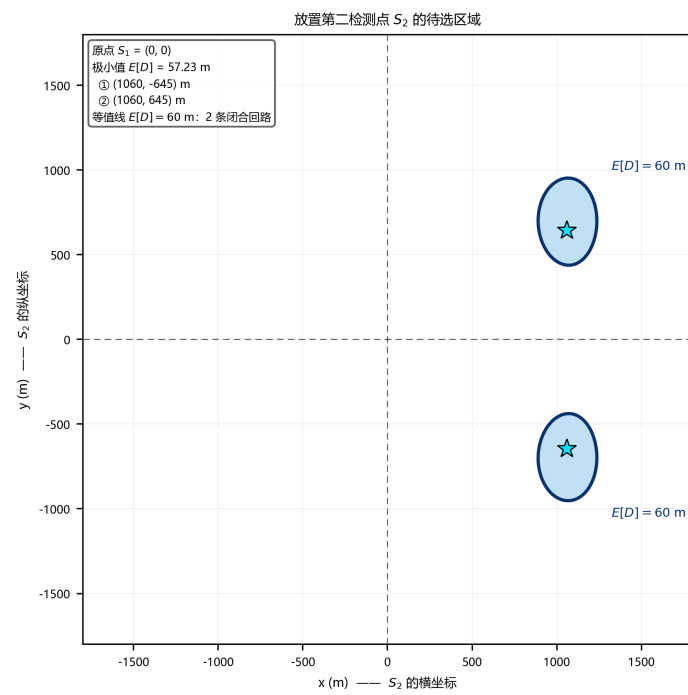


图 6 $E[D] \leq 60 \text{ m}$ 等值线围出的第二检测点待选区域

5.3 问题三：多源定位与清除策略

5.3.1 模型建立

问题三的源数未知，而每个频道至多对应一个静止全向源，因此，完成任务不仅要求清除已发现目标，还须排除其余频道存在残留源的可能。为此，先建立统一时间成本，再分别描述目标定位与区域排除的约束。

(a) 任务成本与评价指标

设靶区 $\Omega = \{x : \|x\| \leq 1800\}$ ，频道集合 $C = \{1, \dots, 20\}$ ，源接收半径 $\rho_c \in [1000, 1500]$ m。机器人从原点出发，速度为 5 m/s；测量耗时 5 s，切频耗时 1 s。清除需先进行 3 s 光学定位，成功时另需 2 s。令 $L = \sum_k \|s_k - s_{k-1}\|$ 为相邻动作终点间的总行程， N 为对应动作次数，则完整退出时间为

$$T_{\text{exit}} = \frac{L}{5} + 5N_{\text{measure}} + N_{\text{switch}} + 5N_{\text{clear,ok}} + 3N_{\text{clear,fail}}. \quad (24)$$

测向机初始频道为 1，清除不改变频道；进入和退出操作不增加上述时间。以 T_{exit} 为主要指标，同时报告清除比例及各次实验的 $T_{\text{exit}}/N_{\text{clear,ok}}$ ，避免将漏清造成的提前退出视为效率提高。末次清除时刻记为 T_{last} ，两者之差表示验证其余频道所需的时间。

(b) 观测约束与保守定位区域

将收到示向度的测量称为正观测，无信号测量称为负观测；近场观测则表明源距测点不超过 5 m，可原地清除并确认结果。正观测用于定位，负观测用于排除，两类信息共同构成逐频道状态。

沿用问题一的角楔交会方法，令 $u(\theta) = (\cos \theta, \sin \theta)$ ， $\text{cross}(a, b) = a_x b_y - a_y b_x$ 。对测站 s_i 和示向度 θ_i ，取含舍入余量的误差半宽 $\eta = (1.005 + 10^{-9})^\circ$ ，得到

$$W_i = \{x : \text{cross}(u(\theta_i - \eta), x - s_i) \geq 0, \text{cross}(u(\theta_i + \eta), x - s_i) \leq 0\}. \quad (25)$$

记 $B(s, r)$ 为以 s 为中心、半径为 r 的闭圆盘， I_c^+ 为频道 c 的正观测索引。结合靶区及接收范围，有

$$P_c = \Omega \cap \bigcap_{i \in I_c^+} (B(s_i, 1500) \cap W_i) \subseteq \tilde{P}_c. \quad (26)$$

其中 $\tilde{P}_c$ 是包含所有相容源位的保守外包：以每圆 32 条外切半平面代替圆域，再与角楔求交。相对估计中心 $\hat{x}_c$ ，令 r_c 为其到外包多边形各顶点的最大距离，即有 $P_c \subseteq B(\hat{x}_c, r_c)$ 。当 $r_c \leq 20$ m 时，到中心清除具有保证；否则仍须补测或试探。负观测可修正点估计，但不用于缩小这一凸外包。

(c) 连续覆盖与任务完成判据

定位解决已发现目标的位置不确定性；对未发现频道，还需判断是否存在未被检测的区域。用边长 h 的闭方格覆盖靶区，保留所有与圆域相交的单元，包括中心位于圆外的边界单元。单元内点到中心 q 的距离至多为 $h/\sqrt{2}$ ，故取

$$r_{\text{cert}} = 1000 - \frac{h}{\sqrt{2}} - \epsilon, \quad h = 50 \text{ m}, \quad \epsilon = 10^{-6} \text{ m}. \quad (27)$$

若频道 c 在 s 处无信号且 $\|q - s\| \leq r_{\text{cert}}$ ，则单元内任一点 x 满足

$$\|x - s\| \leq \|x - q\| + \|q - s\| \leq h/\sqrt{2} + r_{\text{cert}} < 1000 \leq \rho_c. \quad (28)$$

该单元若有源，应能接收，与负观测矛盾，因而可整格排除。记 U_c 为尚未排除的单元中心集合， C_{pending} 、 C_{unheard} 分别为已发现未清除、尚未发现的频道集合，则

$$C_{\text{pending}} = \emptyset, \quad \forall c \in C_{\text{unheard}}, U_c = \emptyset \quad (29)$$

构成连续覆盖证书。它保证判据成立时无残留；其成立依赖静止全向源、接收半径下界及无信号观测的可靠性。实际排除只能依据该频道的负观测，经过测站本身不产生排除信息。

为据此选择测站，令 $U = \bigcup_{c \in C_{\text{unheard}}} U_c$ 。候选站 p 的潜在覆盖收益定义为

$$g(p) = |U \cap B(p, r_{\text{cert}})|. \quad (30)$$

它衡量一次停留可能排除的空间范围；具体频道的检测次数仍需在航路规划中计入。

5.3.2 算法实践

在上述成本与约束下，策略由局部定位逐步扩展为覆盖、清除和检测调度的联合规划。表 5 概括各代机制，以下按其所解决的问题展开。

表 5 五代策略的主要机制与局限

策略	主要机制	局限
field	期望场选点、目标航路排序	未发现频道搜索耗时较长
sweep	利用接收下界系统扫圈	搜索与清除路径重复
tour	覆盖与清除共用航路	固定骨架、点网格判据
adaptive	沿途定位、动态覆盖、连续证书	清除终点偏离规划路点
joint	联合选择航路、测站与检测时机	有限候选的启发式优化

(a) 第一代：期望场选点 field

利用问题二的期望直径场评价补测位置。将各方位对应的场旋转、平移并汇总为 $\bar{E}[D](g)$ ，在当前位置 s 选取使下式最小的候选点 g ：

$$F(g) = \bar{E}[D](g)/1500 + \lambda \|g - s\|/3600, \quad \lambda = 1.5. \quad (31)$$

两项分别反映定位不确定性和移动代价，缺失的场值取 1500 m。目标清除顺序先由最近邻确定，再用 2-opt 交换两条边以缩短航路。该策略侧重已知方位的定位收益，对未发现源仍需额外搜索。

(b) 第二代：系统覆盖 sweep

为提高未发现源的检测机会，原点扫描后，在半径 1100 m 的圆周上设置八个候选测站，利用接收半径下界进行系统搜索。覆盖扩大后再集中清除目标，但两个阶段可能重复经过同一区域，由此产生航路合并的需求。

(c) 第三代：共用航路 tour

以半径 1000 m 的八点环带作为覆盖骨架，将目标 p 插入航路边 (a, b) 时，新增距离为

$$\Delta L(p) = \|a - p\| + \|p - b\| - \|a - b\|; \quad \text{末尾插入时为 } \|a - p\|. \quad (32)$$

按最小增量插入目标，再以保持骨架角向顺序的 2-opt 改善航路。搜索与清除由此合并，但固定测站和离散覆盖判据仍限制搜索完备性。

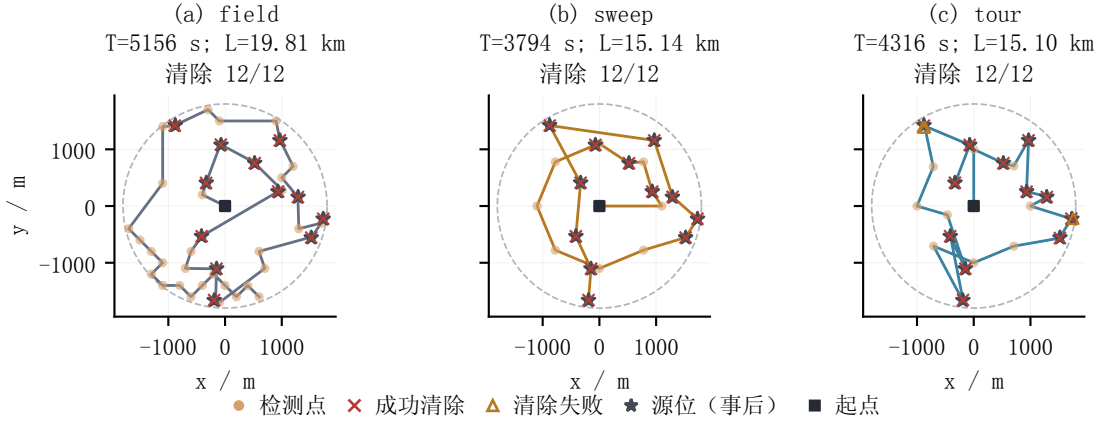


图 7 随机场景第一组的前三代轨迹；源位仅用于实验后分析

图 7 中三者均清除 12 源，*tour* 耗时 4316 s，高于 *sweep* 的 3794 s，说明航路合并的收益还受测站和目标分布影响。

(d) 第四代：沿途定位与动态覆盖 adaptive

为摆脱固定骨架，将测站选择与实时观测相结合，并采用式 (29) 判断任务完成。只有一条示向度的目标也纳入清除航路，在接近过程中补测；未发现频道则按式 (30) 的覆盖收益动态选站。

沿途补测首先需要有效的横向基线。原点检测到的频道集合记为 C_{heard} ，在 72 个等分方向中选择

$$\phi^* = \arg \max_{\phi} \sum_{c \in C_{\text{heard}}} \sin^2(\phi - \theta_c), \quad s_1 = 400 u(\phi^*). \quad (33)$$

横向分量越大，测向交会越不易近共线，从而一次移动可改善多个目标的定位。随后，对仅有一条方位的目标，沿估计中心 $\hat{x}_c$ 接近，并沿视线单位法向 n 构造两个补测点：

$$s_{\pm} = s + 0.55(\hat{x}_c - s) \pm \ell n, \quad \ell = \min\{100, \max\{35, 0.25\|\hat{x}_c - s\|\}\}. \quad (34)$$

选取经补测点到估计中心总距离较小的一侧。取得多条方位后继续逼近，必要时侧移；失去信号时向最近正观测位置回退补测。因此，实际清除终点可能偏离原规划路点，其检测收益需要重新评估。

(e) 第五代：航路与检测调度联合优化 joint

针对这一偏移，进一步同时选择覆盖站、航路顺序和检测时机。先排列所有已发现未清除目标，再从候选站中按下式贪心选择并插入：

$$p^* = \arg \max_p \frac{g(p)^\alpha}{100 + \max(0, \Delta L(p)) + 30\beta \max(1, |C_{\text{unheard}}|)}. \quad (35)$$

其中 α 调节覆盖收益偏好， β 调节检测成本； $30 = 5(5 + 1)$ 将检测和切频的 6 s 折算为行程，100 m 为代价下限。已有目标点亦可承担检测任务，其插入增量为零。选中后仅从规划用集合中扣除覆盖单元，实际排除仍须等待负观测。

adaptive 使用 $(\alpha, \beta) = (2, 0.3)$; *joint* 比较五组权重生成的航路, 具体参数见附录。为计入同站检测不同频道的成本, 对航路 $\pi = (p_1, \dots, p_m)$ 逐站更新各频道的剩余集合 $U_c^{(j)}$, 以

$$\hat{T}(\pi) = L(\pi)/5 + 6 \sum_{j,c} \mathbf{1}\{j \text{ 承担检测任务}, U_c^{(j-1)} \cap B(p_j, r_{\text{cert}}) \neq \emptyset\} \quad (36)$$

比较方案, 其中 $\mathbf{1}$ 为条件成立时取 1、否则取 0 的指示函数。该式近似切频成本并省略后续定位、清除费用, 仅用于规划; 实际总时间按式 (24) 计算。

选站后还可减少绕行。将仅承担检测任务的站称为覆盖站; 先删除冗余站, 再将剩余站 p_0 向航路投影点 z 移动。设 Q 为仅由该站覆盖的必要单元中心, $v = z - p_0$, 则 $p(t) = p_0 + tv$ 必须满足

$$\|p_0 - q + tv\|^2 \leq r_{\text{cert}}^2, \quad q \in Q, \quad 0 \leq t \leq 1. \quad (37)$$

令 $a = \|v\|^2$ 、 $b_q = 2(p_0 - q)^\top v$ 、 $c_q = \|p_0 - q\|^2 - r_{\text{cert}}^2 \leq 0$, 展开为二次不等式。初始点可行, 故 $a > 0$ 时最大允许步长为

$$t_{\max} = \min \left\{ 1, \min_{q \in Q} \frac{-b_q + \sqrt{b_q^2 - 4ac_q}}{2a} \right\}. \quad (38)$$

实际步长留 10^{-8} 余量, 仅接受缩短航路的移动。若 $a = 0$ 则不移动; 若 Q 为空, 则删除覆盖站或取消目标点的检测任务。图 8 将这一约束与连续覆盖判据一并表示。

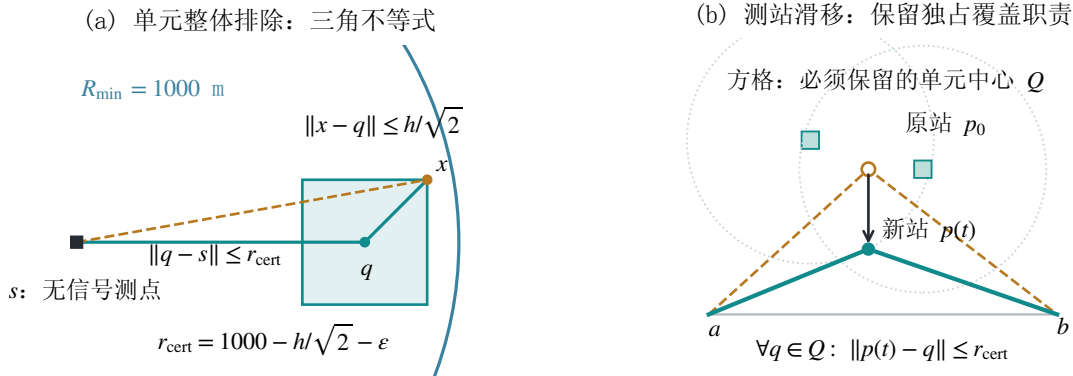


图 8 单元整体排除与保留独占覆盖单元的测站移动

执行时, 每轮仅访问首站, 再按新观测重规划。目标站先定位清除; 若承担覆盖任务, 再以实际终点评估未发现频道的检测收益。覆盖站直接逐频道测量。至少两条方位且 $r_c \leq 400$ m 时允许试探中心, 失败后原地补测并继续定位; 只有 $r_c \leq 20$ m 时清除才有保证。

对沿途可选补测, 还需兼顾接收可能性。沿首条中心射线抽样并结合负观测, 估计接收概率 $\hat{P}_{\text{recv}}$ (附录 4.1)。仅在该概率不低于 0.5, 且移动基线与已有视线的锐角不小于 25° 时补测; 这一近似只筛选可选测量, 不删除待清除目标。最近邻、2-opt 和贪心选点均为启发式方法, 不保证全局最优。

图 9 中, *joint* 行程为 11.00 km, 略高于 *adaptive* 的 10.91 km, 但检测次数由 186 降至 136, 完整时间由 3357 s 降至 3072 s, 表明检测调度也能带来独立于行程缩短的收益。

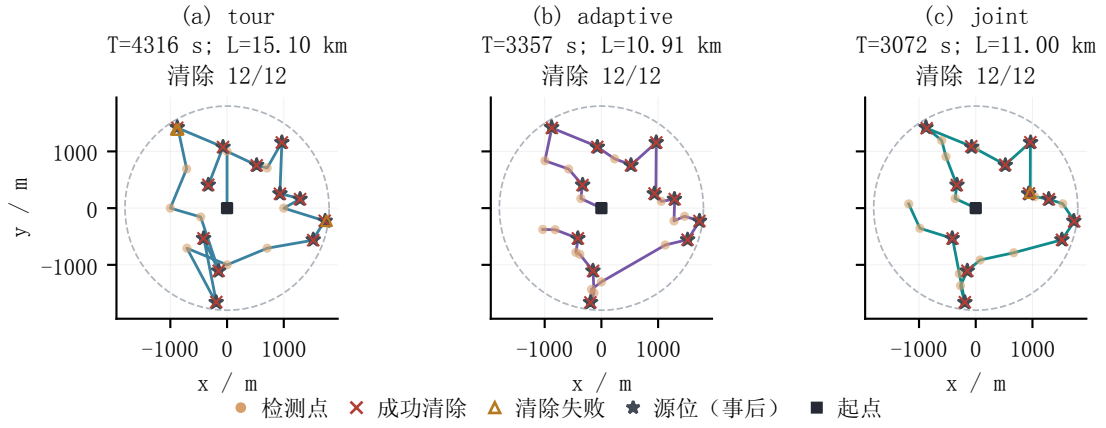


图9 同组场景下固定航路、沿途定位与联合调度的轨迹对比

5.3.3 数据分析

(a) 实验设计与总体表现

设置随机、外圈、中心聚集、非平滑固定位置误差及最小接收半径五类场景，分别为 50、20、10、20、20 组。五种策略使用各自默认参数，在相同源位、接收半径和误差场下独立运行，共 600 次；分布与种子见附录。所有轨迹图均取随机集第一组，统计包含未全清实验；P90 表示完整时间的 90% 分位数。

表6 随机 50 组场景的清除结果与完整时间

策略	清除比例	全清组数	$\bar{T}_{\text{exit}}/\text{s}$	P90/s	每源耗时/s
field	0.995	47/50	5347	5820	422
sweep	0.986	43/50	4186	4610	334
tour	1.000	50/50	4028	4674	317
adaptive	1.000	50/50	3348	3602	264
joint	1.000	50/50	3067	3367	241

清除比例按总源数加权，每源耗时为各组比值的算术平均。全部 120 组中，*tour*、*adaptive* 和 *joint* 均全清，*field*、*sweep* 分别全清 114、92 组。随机集上，*joint* 较 *field* 的完整时间均值降低 42.6%，配对 bootstrap 95% 区间为 [41.1, 44.1]%。行程由 21.04 km 降至 11.17 km，检测由 180.5 次降至 127.4 次。图 10 还显示，末次清除后仍存在搜索验证成本，因而完整退出时间能更全面地评价策略。

(b) 场景差异与机制消融

图 11 中，*joint* 在五类场景的平均耗时均最低。最小接收半径组的 *sweep* 仅 7/20 组全清，说明固定扫圈仍有漏清风险；中心聚集组的 *adaptive* 耗时 2573.5 s，略高于 *tour* 的 2529.6 s，表明改进收益具有场景依赖性。在随机前 20 组分别取消清除后扫描、多方案规划、测站精修和概率筛选，新增 80 次实验。相对完整 *joint*，平均时间分别增加 0.15%、1.78%、1.93%、1.12%，各组均全清。第一项的 95% 配对区间跨零，尚不足以确认其速度收益；其余三项区间为正，支持相应机制的有效性。

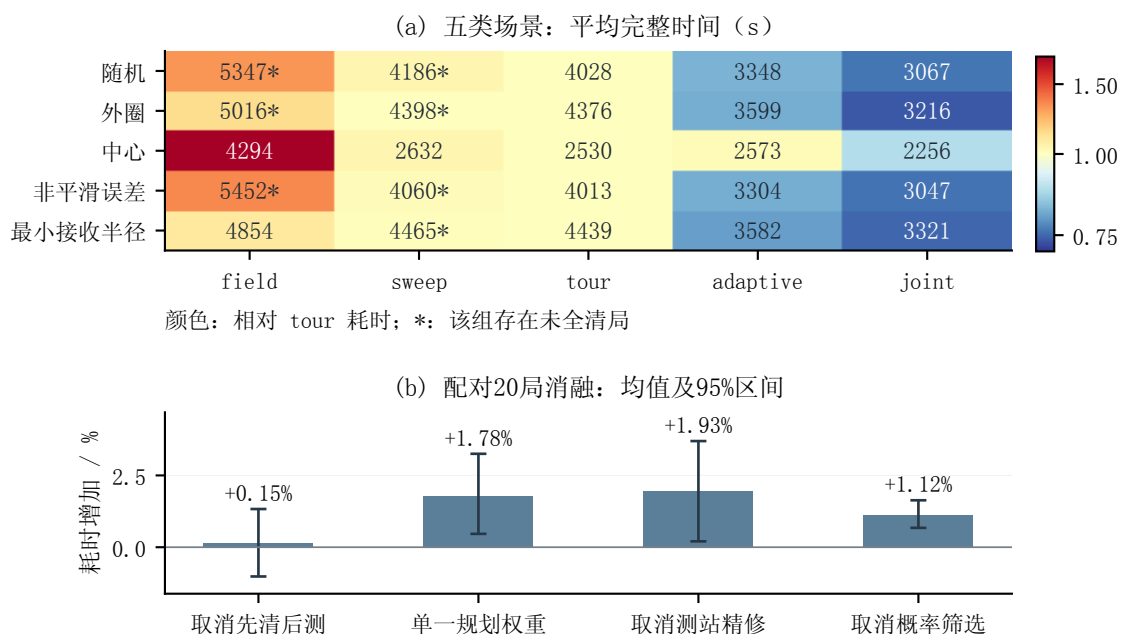
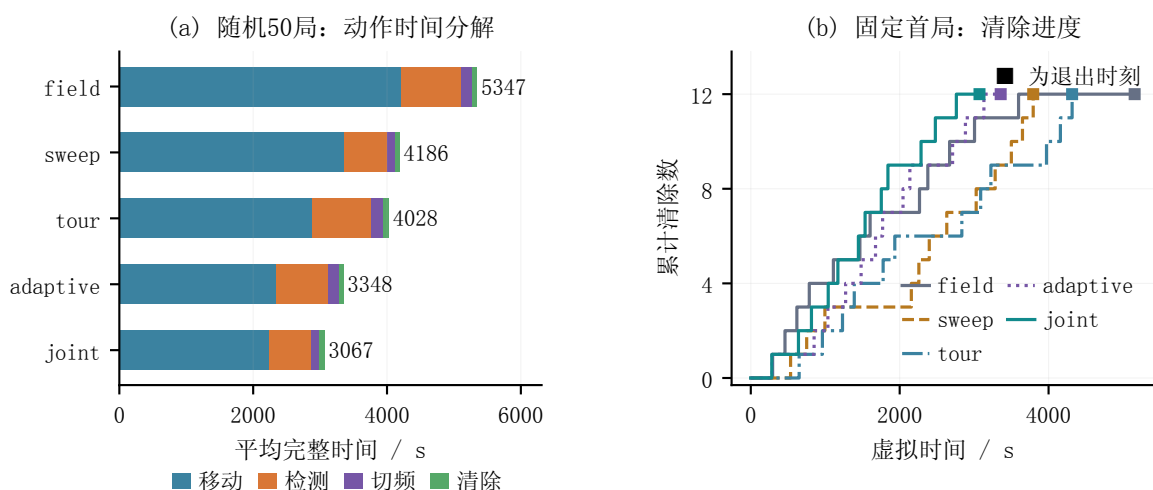


图 11 跨场景时间比较与四项单因素消融的配对区间

由此可见，保守定位与连续覆盖为任务完成提供依据，共用航路和逐频道调度则降低完整成本。上述结论限于所测离线分布，不能推及任意误差场下的快速全清；计算时间和补充统计见附录，正式测试结果见表 7。

表 7 问题三正式测试结果（待补）

测试案例编码	清除干扰源个数	平均定位清除时间	程序运行时间
测试 1 案例编码（待补）	待补	待补	待补
测试 2 案例编码（待补）	待补	待补	待补
测试 3 案例编码（待补）	待补	待补	待补

5.4 问题四：含定向源时的定位与清除模型

5.4.1 模型建立

问题四允许干扰源仅向一个半平面发射。因而，问题三中“近处无信号即可排除该区域”的规则失效：机器狗可能已接近源，却始终位于其背面。为此，将未知发射朝向纳入搜索状态，分别建立发现、定位与退出的证据，再优化取得这些证据的时间成本。

(a) 定向接收模型与任务成本

设源位置为 $p \in \Omega$ ，接收半径为 $r \in [1000, 1500]$ m，物理发射方向为单位向量 $d(\phi) = (\cos \phi, \sin \phi)$ 。在测站 q 能接收该定向源，当且仅当

$$H(q; p, r, \phi) = \mathbf{1}\{\|q - p\| \leq r, (q - p)^\top d(\phi) \geq 0\} = 1. \quad (39)$$

全向源去掉第二个条件。因此，无信号意味着“距离超出接收半径”或“测站位于发射背面”，不能单独推出 $\|q - p\| > 1000$ m。发射方向是待排除的隐变量；实现不必先准确辨别全向、定向类型，便可对两者统一搜索。

动作成本沿用 (24)，优化完整退出时间 T_{exit} 。同时记录成功清除数 n_{clear} 、末次清除时刻 T_{last} 及

$$\bar{t}_{\text{clear}} = T_{\text{last}}/n_{\text{clear}}, \quad \bar{t}_{\text{exit}} = T_{\text{exit}}/n_{\text{clear}}. \quad (40)$$

前者为末次清除口径，后者为完整任务口径，计入末次清除后的残留搜索；本文主要比较完整任务时间，另报告末次清除口径作为补充。 $n_{\text{clear}} = 0$ 时两者不定义，仍保留该局的清除数与总时间。清除比例与时间必须共同报告，避免把遗漏目标带来的提前退出误判为提速。

(b) 从三角格网到任意朝向覆盖

先考虑在三角格网顶点检测。若 p 位于三角形 $v_1 v_2 v_3$ 内，存在 $\lambda_i \geq 0$ 、 $\sum_i \lambda_i = 1$ ，使

$$p = \sum_i \lambda_i v_i \implies \sum_i \lambda_i (v_i - p)^\top d = 0. \quad (41)$$

所以至少一个顶点处于源的发射半平面。对于边长 a 的等边三角形，内部任一点到任一顶点的距离不超过 a ，故完整三角剖分配合 $a \leq 1000$ m 是一个充分条件。这里 $a/\sqrt{3}$ 仅是最近格点的覆盖半径，最近格点未必位于发射正面，不能据此放宽定向覆盖条件。

有限格网还存在边界问题：靶区边缘朝外发射的源可能看不到任何域内测站。于是需要外侧站位，但“补一圈”以及只检查径向朝内、朝外两种情况，仍不足以证明覆盖所有位置与方向。更一般地，对候选源位置 p ，1000 m 内的测站须从各侧围住 p ，即其相对向量的凸包包含原点，才能保证任意发射半平面均有可接收测站。连续区域的计算判据如下。

(c) 连续位置与朝向的无源证书

用闭方格覆盖靶区，保留所有与 Ω 相交的边界格。设格中心为 c 、边长为 h ，格内位置 p 满足 $\|p - c\| \leq \rho = h/\sqrt{2}$ 。若某频道在 q 实际无信号，且

$$\|q - c\| + \rho \leq 1000, \quad (q - c)^\top d \geq \rho, \quad (42)$$

则该格内每个位置均满足

$$\begin{aligned} \|q - p\| &\leq \|q - c\| + \|p - c\| \leq 1000, \\ (q - p)^\top d &= (q - c)^\top d - (p - c)^\top d \geq \rho - \|p - c\| \geq 0. \end{aligned} \quad (43)$$

假如该格中存在朝向 d 的源，便应能接收，与无信号矛盾。因此，此次负观测排除的是整个方格内的一段连续朝向，而不只是格中心。

令 $a_q = \arg(q - c)$ 、 $D_q = \|q - c\|$ 。对满足距离条件且 $D_q > \rho$ 的测站，第二个不等式等价于排除圆弧

$$I_q(c) = \left[a_q - \arccos \frac{\rho}{D_q}, a_q + \arccos \frac{\rho}{D_q} \right] \pmod{2\pi}. \quad (44)$$

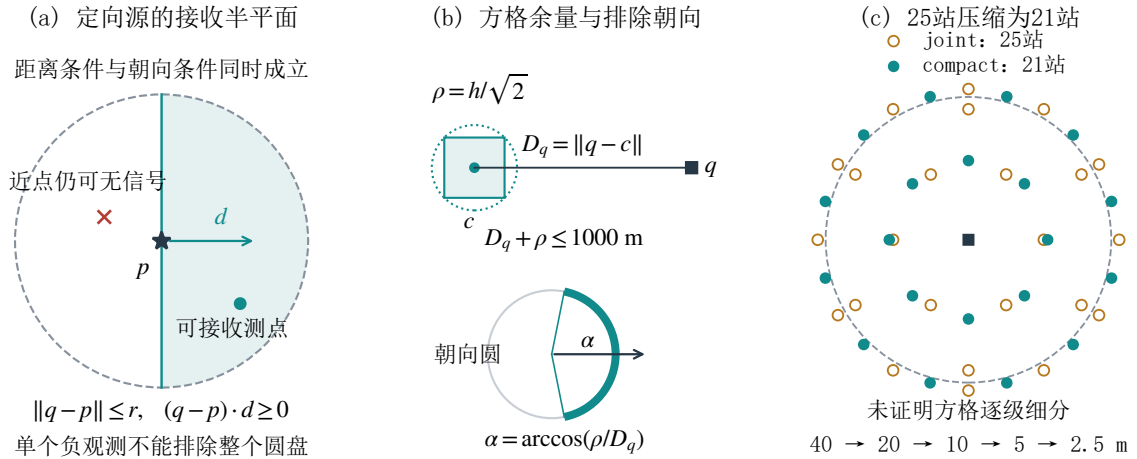
其余测站令 $I_q(c) = \emptyset$ 。记频道 j 的实际无信号测站集合为 Q_j^- ，将跨过零角的圆弧拆分、按起点排序合并，若每个相交方格都有

$$\bigcup_{q \in Q_j^-} I_q(c) = [0, 2\pi), \quad (45)$$

则该频道在靶区内不可能还有任意朝向的定向源；全向源更易接收，也随之排除。最终完成判据为

$$\forall j \in \{1, \dots, 20\}, \quad \text{频道 } j \text{ 已成功清除} \quad \vee \quad (\text{无待清正观测且持有连续无源证书}). \quad (46)$$

计划测站只能验证布局可行，最终判据必须使用该频道实际获得的负观测。证书成立说明无残留；预算耗尽而证书未成立时，程序退出并标记未完成。



负观测只排除带余量证明的“位置一朝向”状态；布局认证后仍须逐频道实际检测。

图 12 定向接收、带方格余量的朝向排除与测站布局压缩；方格图为几何示意

5.4.2 算法实践

在上述模型下，算法由格网启发式演进到有证据的联合搜索，再压缩布局、改进估价。表 8 中的三版均保留可运行实现，性能比较使用相同场景重新计算。

表 8 问题四三版策略的递进关系

策略	主要机制	进一步需要解决的问题
grid	三角格网补外环、按环推进、发现后清除	径向覆盖启发式缺少任意朝向证书
joint	25 站连续覆盖、搜索清除共用航路、保守定位	站位较多，纯距离排序忽略检测收益
compact	21 站自适应证书、后验中心、信息代价排序	有限候选局部优化，未证明全局最优

(a) 由覆盖可行性反求紧凑站位

第一版采用边长 1000 m 的域内格网及外环，借鉴问题三按环推进、顺路清除的组织方式。第二版取消径向朝向假设，用 (45) 核验“原点、950 m 六点环、 $950\sqrt{3}$ m 六点错位环、1900 m 十二点环”，共 25 站，并将待扫描站和待清目标合并排序。

在有可靠退出条件后，第三版对同心环的半径、点数和相位进行候选搜索，以站数和开放航程衡量成本，以连续证书约束可行性，最终采用

$$S = \{(0, 0)\} \cup \{998 d(k\pi/4) : k = 0, \dots, 7\} \cup \{1867 d(\pi/12 + k\pi/6) : k = 0, \dots, 11\}. \quad (47)$$

外环超过靶区半径是为了检测边缘朝外源，题设允许机器狗在靶区外移动。该布局含原点共 21 站，较第二版减少 16%；它是已找到的可行方案，并未证明站数最少。

紧凑布局的几何余量较小，固定粗方格可能无法认证。故从 $h = 40$ m 开始，仅将未证明方格四等分，逐层重复圆弧并集检验；最细允许到 0.625 m，本布局在 $40 \rightarrow 20 \rightarrow 10 \rightarrow 5 \rightarrow 2.5$ m 即全部通过。各层未证明格数为 348, 380, 252, 172, 0；细分后格数增多不意味着覆盖退化，每次接受仍然证明整个方格。这样既减少站数，又保留连续区域的保证。

图 12 展示两版布局的相对位置；完整细分计数列于附录。

(b) 短横向基线与合法负观测约束

发现源后，设 I_j^+ 为有示向度的正观测索引， W_i 为对应角楔， $\mathcal{H}_j$ 为已由合法双侧负观测获得的半平面约束集合，构造

$$P_j = \Omega \cap \bigcap_{i \in I_j^+} (B(s_i, 1500) \cap W_i) \cap \bigcap_{H \in \mathcal{H}_j} H \subseteq \tilde{P}_j. \quad (48)$$

其中 $\tilde{P}_j$ 将圆域各用 32 条外切半平面近似，仍包含真源；角楔半宽为 $(1.005 + 10^{-8})^\circ$ ，其中 0.005° 覆盖读数保留两位小数的舍入误差。它延续问题一的保守定位，但不因单次失去信号而删去距离圆盘。

从最后一个正观测点 s 出发，令示向度单位向量为 u 、法向为 n ，估计中心沿射线的非负投影距离为 a ，构造

$$q_{\pm} = s + tu \pm \ell n, \quad t = 0.6a, \quad \ell = \min\{100, \max(22, 0.3 \max(a, 1))\} \text{ m}. \quad (49)$$

横向基线改善交会角；先测较近一侧，若无信号再测另一侧。必要时将 t, ℓ 同时缩为 $1/2, 1/4$ ，处理越过源或跨出发射半平面的情形。

双侧无信号还可能产生严格的纵向上界。设角误差半宽为 η ，当

$$t \tan \eta \leq \ell, \quad t \geq \ell(\sec \eta + \tan \eta), \quad (50)$$

任意楔内远端位置 $p = s + xu + yn$ ， $x \geq t$ 、 $|y| \leq x \tan \eta$ ，其与 s 的连线经过横截线段 q_-q_+ 。因为 s 曾接收，至少一个端点与 s 同属该源的发射半平面。又有

$$\begin{aligned} \|p - q_{\pm}\|^2 - \|p - s\|^2 &= t^2 + \ell^2 - 2tx \mp 2\ell y \\ &\leq \ell^2 - t^2 + 2t\ell \tan \eta \leq 0, \end{aligned} \quad (51)$$

故该端点也在源的实际接收半径内。如果两端实际均无信号，便可反证排除 $x \geq t$ ，加入 $u^\top(p - s) \leq t$ 的保守截断。程序留数值余量；条件未满足时不加入该约束。

取得至少两条方位后允许试清除中心；若包围半径不超过 20 m 则清除有保证，否则只是试探，失败后继续补测。最多 8 轮逼近后，对外包多边形采用主轴旋转的方格清除点覆盖，网格间距 $b = 28$ m。由于

$$\max_p \min_{z \in b\mathbb{Z}^2} \|p - z\| = b/\sqrt{2} = 19.799 \dots < 20 \text{ m}, \quad (52)$$

保留所有距多边形不超过 20 m 的格点，逐个试清即可在有限步骤内命中。此保证依赖真源仍在外包内及预算足够；它利用光学清除与发射朝向无关的题设，而不要求始终收到信号。

(c) 用后验中心改善单方位目标估价

只有一条方位时，几何中心不能体现先前在哪些位置无信号。为改善清除节点的位置估计，在首条中心射线上取 150 个距离点 $p_k = s + t_k u$ ，接收半径取 9 点，发射方向取与正观测相容的 36 点，全向先验权重取 $\pi_o = 0.5$ 。令 L_o, L_d 表示候选全向、定向状态与实际正负回复相容，则

$$\begin{aligned} w_k &= t_k \mathbf{1}_{\Omega}(p_k) \left[\frac{\pi_o}{9} \sum_r L_o(p_k, r) + \frac{1 - \pi_o}{9 \cdot 36} \sum_{r, \phi} L_d(p_k, r, \phi) \right], \\ \hat{p}_j &= \frac{\sum_k w_k p_k}{\sum_k w_k}, \quad R_j = \max_{v \in \text{vert}(\tilde{P}_j)} \|v - \hat{p}_j\|. \end{aligned} \quad (53)$$

t_k 来自面积先验的极坐标权重。该计算是中心射线上的离散后验近似，仅改变规划中心；完整保守多边形仍被保留，并重算 R_j 。权重和为零时回退到几何中心。因此，先验不准确会影响效率，却不会改变无源证书的成立条件。

图 13 中，双侧负观测只在 (50) 成立时截断远端；中图为固定观测构造的估价示例，右图说明清除网格为何能覆盖保守定位区域。

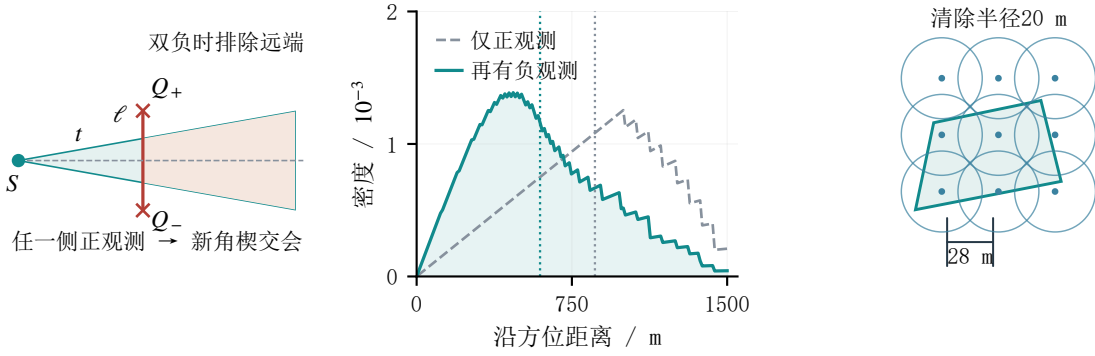
(d) 将检测收益写入开放航路目标

将未访问扫描站与待清目标中心共同组成节点集，从实际位置出发求开放航路 π 。单纯最短距离可能把信息量大的扫描站排到后面，使更多频道在后续各站重复检测。为此，独立生成 1024 个固定参考状态，源位按面积均匀、朝向均匀、接收半径均匀；令

(a) 双侧短基线与双负截断

(b) 单方位后验用于估价

(c) 清除圆盘覆盖兜底



(a) 误差角示意放大10倍；实际 $\epsilon = 1.005^\circ$ 。(b) 负测点为(550, 180) m，竖线为均值。
后验均值不缩小保守可行域；兜底依靠清除半径，不依赖信号接收方向。

图 13 双侧补测、单方位后验中心与清除覆盖；左图误差角放大 10 倍，实际半宽约 1.005°

F_k 为前 k 个航路节点及既有扫描点能够接收的参考状态比例，只有扫描节点更新该比例。用

$$J(\pi) = L(\pi) + \frac{30U}{b(F_0)} \sum_{k:\pi_k \text{ 为扫描站}} b(F_{k-1}), \quad b(F) = 0.35 + 0.65(1 - F) \quad (54)$$

评价方案， U 为当前未发现频道数， $30 = 5 \times (5 + 1)$ 将一次测量及切频折算为行程。常数 0.35 为扫描代价下限；参考覆盖比例不是该局真实发现概率，该式也不代替逐动作的实际计时。

求解时，以保留上一轮顺序、最小代价插入新节点的航路热启动，再加至多 5 个最近邻起点；每个候选至多作 4 轮 2-opt 片段翻转和单点移位，按完整目标择优。每轮只执行首个节点，随后用新观测和实际清除落点重规划。扫描站检测所有未发现频道，并为满足距离、基线条件的单方位目标补第二条方位；清除点不默认全频道补扫。严格档保留全部必要扫描站，最终仍逐频道核验 (46)。

每轮至多有 20 个剩余扫描站和 20 个目标节点。设节点数为 n ，每种邻域含 $O(n^2)$ 个候选，完整目标重算需 $O(n)$ ，故至多 $K = 6$ 个起点、 $I = 4$ 轮的搜索为 $O(KIn^3)$ ；参考状态的接收掩码可缓存。证书对每个方格合并 m 个测点的圆弧需 $O(m \log m)$ ，只细分未证明格可减少计算量。两者均适合本题有限规模，计算开销另见附录。

5.4.3 数据分析

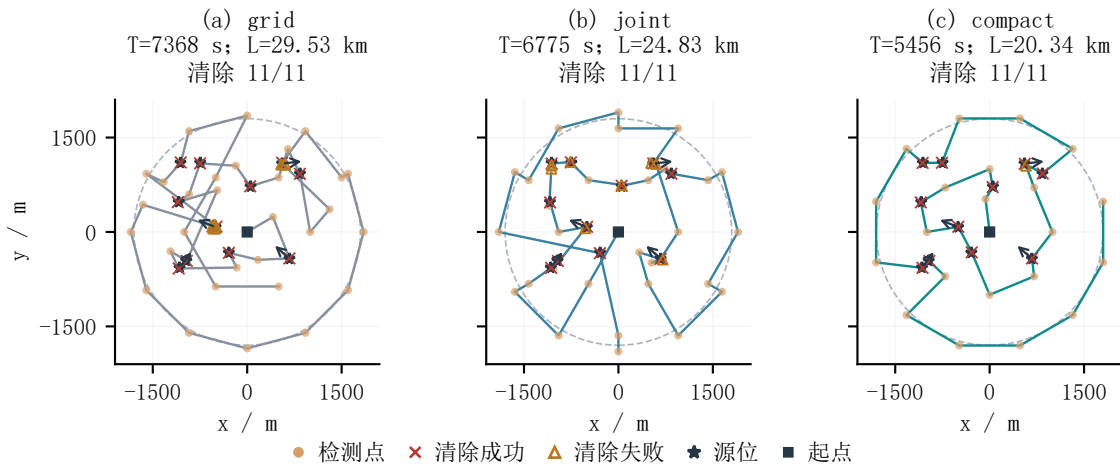
(a) 统一实验与版本比较

采用 30 组随机混合场景，源数为 10 ~ 16，位置按圆域面积均匀，约一半源为随机朝向的定向源；另设最小接收半径、贴边朝外和全向三类压力场景，各 10 组。三版策略在相同源位、接收半径和固定位置误差下独立运行，共 180 次。主集前 10 组增加两项消融和三个速度档，共计 230 次；种子、参数及逐动作审计见附录 F。所有数值均来自此次离线重跑。

表 9 随机混合 30 组的清除结果与完整退出成本

策略	清除比例	全清组数	时间均值/s	P90/s	每源/s	行程/km
grid	100.00%	30/30	8426.1	9392.4	666.7	34.42
joint	100.00%	30/30	6556.9	6944.0	520.1	24.17
compact	100.00%	30/30	5819.1	6131.5	461.3	22.08

每源列为逐局 $T_{\text{exit}}/n_{\text{clear}}$ 的算术平均，清除比例按总源数加权。相较同样具有连续证书的 *joint*, *compact* 的随机集完整时间均值降低 11.25%，95% 配对 **bootstrap** 区间为 [9.14, 13.39]%；其末次清除口径的每源时间为 423.7 s。两个每源口径的差额反映末次清除后继续排除未知频道的成本。



同一预定首局；源位及箭头均为退出后真值，箭头表示可接收半平面的中心方向。

图 14 随机混合第一组的三版实际轨迹；源位及朝向仅在实验后用于绘图

图 14 以同一场景展示轨迹差异。布局压缩、定位估计及航路优化共同改变实际行程与动作次数，不能把整版提速全部归因于减少四个测站。

由图 15 可见，*compact* 同时减少移动和检测支出。与 *joint* 相比，随机主集行程从 24.17 km 降至 22.08 km，检测从 275.5 次降至 223.0 次，说明提速并非仅靠缩短几何航程。

(b) 压力场景、机制消融与覆盖取舍

表 10 压力场景下的全清组数及完整时间均值

场景	<i>grid</i>		<i>joint</i>		<i>compact</i>	
	全清	时间/s	全清	时间/s	全清	时间/s
最小接收半径	10/10	8604.9	10/10	7535.2	10/10	6612.2
贴边朝外	4/10	13185.4	10/10	8225.0	10/10	7603.1
全向	10/10	7437.9	10/10	6013.8	10/10	5328.0

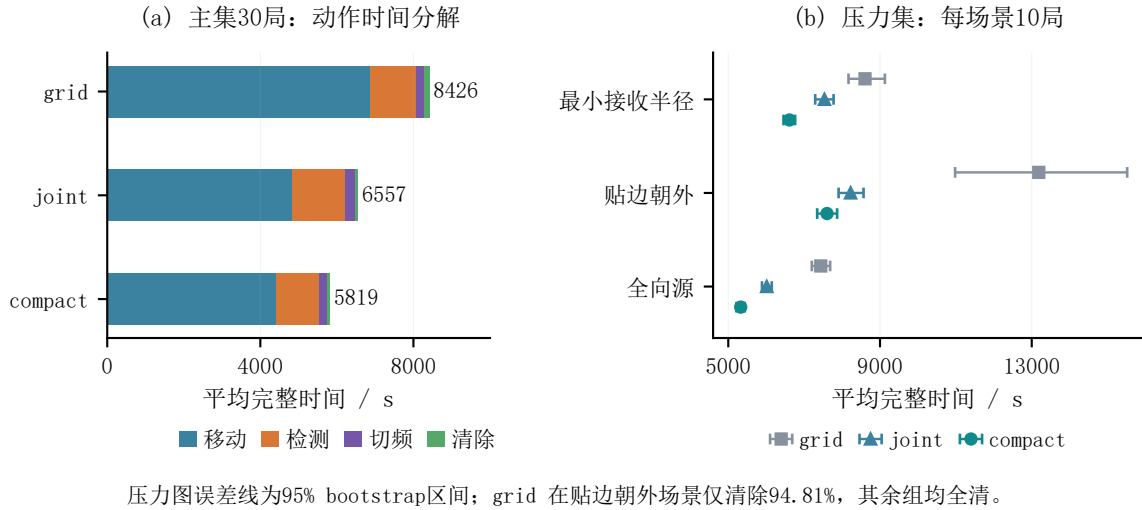


图 15 主集平均动作时间分解与压力场景完整时间的 95% bootstrap 区间

贴边朝外组专门检验域外测站的作用，最小半径组固定接收半径为 1000 m，以检验覆盖余量。全部 60 组中，*compact* 清除 779/779 个源，60 组取得完整证书。经验全清结果验证了所测场景的表现；任意朝向的可靠退出仍由 (43) 和 (46) 支持，不能由有限样本零失败代替证明。

分别关闭后验中心和信息代价，主集前 10 组的完整时间相对完整 *compact* 变化为 +7.97% 和 +2.59%。图 16 的配对区间显示，后验中心的收益得到支持，信息代价对应的区间跨零，尚不足以单独确认稳定收益。消融仅改变对应开关，几何证书与清除兜底保持一致。

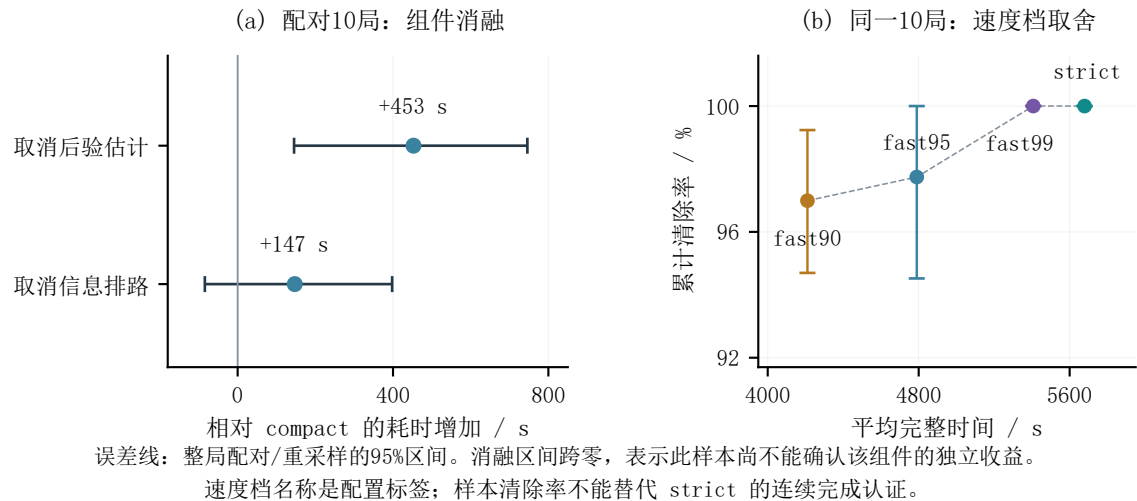


图 16 两项单因素消融的时间变化区间及稀疏速度档的清除率与时间取舍

速度档减少扫描站而提前结束搜索，仍清除已发现目标，却不要求全域证书。*fast99/fast95/fast90* 是配置名称，不能代表当局真实清除比例或未来全清保证；源分布未知时应选严格档。本节为离线仿真，正式模拟器三次测试数据尚未取得。

六、模型评价与改进

四问形成了“位置约束、检测点选择、联合航路、无残留认证”的递进体系。评价时，既考察定位与退出判据是否可靠，也考察取得这些证据需要付出多少时间；以下结论依据前文推导及统一离线实验。

6.1 参数敏感性与稳健性

误差界影响定位尺度。固定检测点与示向度读数，记误差半宽为 ε 时的角楔交集为 $\mathcal{R}(\varepsilon)$ 。由角楔随误差界增大而扩张，有

$$\varepsilon_1 \leq \varepsilon_2 \implies \mathcal{R}(\varepsilon_1) \subseteq \mathcal{R}(\varepsilon_2), \quad D(\varepsilon_1) \leq D(\varepsilon_2). \quad (55)$$

直径关系针对非空有界交集；误差界继续增大还可能使区域无界。因此，低估误差可能破坏真值包含性，高估误差则可能增加补测成本。问题二的小角近似也表明，远离共线且固定几何与先验时，定位尺度随误差界近似同比增长；近共线布点会放大误差，不能将这一近似推广到全部候选位置。

网格精度影响认证的保守程度。问题三由 (27) 得到有效认证半径 $1000 - h/\sqrt{2} - \varepsilon$ ；问题四由 (42) 和 (44) 可见，增大格宽 h 会收紧距离条件并缩短可排除朝向区间。细网格更易取得证书，但增加运算量。Q4 的 21 站布局经未证明方格逐级细分，在 2.5 m 通过核验，说明自适应精度可减小几何离散的保守损失；它不意味着该分辨率对所有布局均足够。

模块收益具有场景依赖性。既有消融中，Q3 取消测站精修后平均增时 1.93%，Q4 关闭后验中心后增时 +7.97%，对应配对区间均为正；但 Q3 清除后扫描、Q4 信息代价的区间跨零（图 (11) 和 (16)）。这支持保留有效机制，也表明不能把每个新增模块都视为稳定提速来源；上述开关实验尚不能确定连续参数的最优取值。

6.2 模型优点

一是定位与完成判据具有可验证依据。问题一直接利用误差界构造可行域，揭示直径圆可能不能包住定位区域；后续采用保守外包及包围半径判断清除条件。问题三、四进一步以实际负观测构造连续覆盖证书，使“已知目标清完”与“区域内无残留”得到明确区分。

二是将定位精度与完整任务成本统一考虑。问题二以期望直径选择补测位置，问题三、四将测站、清除目标及检测调度纳入航路。Q3 随机集最终策略较初代、Q4 紧凑策略较上一严格版的完整时间平均缩短 42.6% 和 11.25%；两项比较各自使用相同场景配对，支持联合规划降低移动与检测开销的作用。

三是具备较完整的数值验证链。几何计算采用独立方法交叉核验，策略比较保留失败场景，并报告完整退出时间、清除比例及配对区间。Q3 最终策略在 120 组场景全清，Q4 最终严格版在 60 组场景清除全部 779 个源并认证；逐动作记录和固定种子使结果能够复核。

6.3 模型局限

首先，可靠性依赖观测模型成立。保守定位及无源证书要求源静止、接收半径下界有效、误差有界且接收区内无漏报。遮挡、异常测向或源移动均可能破坏这些条件。证书成立能保证无残留，但尚未证明算法在任意合法场景和有限预算内都能取得证书；离线样本全清也不等于未来零失败风险。

其次，估价模型存在近似与先验偏差。问题二的面积先验、小角近似及接收可达性处理限制其适用范围，解析值与数值值在已比较位置吻合，不能替代全域误差界。Q4将单方位位置近似为中心射线，并采用固定全向比例和参考状态估价；源分布变化时，规划中心和检测收益可能失准。问题二的 60 m 阈值是期望性能标准，也不保证每次定位直径均低于该值。

最后，求解效率与最优性仍有局限。贪心选点、有限候选及局部航路交换不能保证全局最优，21 站也未证明为最少站数。更强的搜索和更细的认证会增加计算开销，光学网格兜底在定位区域较大时可能耗时较多。本地运行时间未计入真实接口通信、网络抖动等影响，正式测试记录尚待补充。

6.4 改进方向

改进应分别对应上述局限。其一，在位置估价中比较不同空间分布、接收半径及全向比例先验，并对近共线、不同交会角分支和接收截断重新核验几何近似；以信息充分的补测位置降低先验依赖，同时保留保守外包与连续证书。

其二，以同一完整时间目标协调测站选择、补测时机和清除顺序，通过小规模精确求解建立比较基准，再评估局部搜索的改进空间；对覆盖边界采用自适应细分，并设置计算预算，平衡虚拟任务耗时与实际运算时间。

其三，在独立场景中扩大误差场、定向比例和边缘源分布的验证范围，报告超时、漏清及接口失败；若引入漏报或移动源，应重建相应的观测与置信判据，经校准后再用于退出决策，不能直接沿用现有确定性无源证书。

参考文献

- [1] 作者. 无线电测向与定位技术 [M]. 出版社, 年份.
- [2] 作者. 论文题目 [J]. 期刊名, 年份, 卷 (期): 页码.
- [3] 全国大学生数学建模竞赛组委会. 全国大学生数学建模竞赛论文格式规范（2026 年修订稿）.

附录 A 支撑材料文件列表

支撑材料按问题分别存放，包含计算与绘图程序、输入算例和结果数据。以下路径相对于附件根目录 B_locator/。

- 1. Q1/p1_intersection.py: 问题一角楔交会、直径和覆盖判定。
- 2. Q1/p1_experiments.py: 生成五组算例、四组固定种子扫描并写出汇总结果。
- 3. Q1/make_figures.py: 重绘问题一角楔示意、算法流程和覆盖对比三幅插图。
- 4. Q1/cases/: 五组输入 CSV 及真值核验文件; Q1/results/: 逐例结果和汇总 JSON。

5. Q2/pysimulation/src/p2_grid_expectation.py: 问题二数值模拟源程序, 交会四边形直径、加权期望、寻优与候选区域 (附录 C)。
6. Q2/MMAcode/expected_d mathematica_ver.nb: 理论闭式解的 Mathematica 程序, 导出 15 m 网格上的 $E[D]$ 与最优点。
7. Q2/pysimulation/out/p2_grid/p2_grid_map.csv、p2_grid_report.json: 数值模拟逐点结果与汇总统计 (本文数值来源)。
8. Q2/make_q2_paper_figures.py: 问题二正文插图生成脚本 (由上述两份原始数据出图)。
9. figures/p2-mma-vs-pysim.png: 理论闭式解与数值模拟的并列对比 (图 5)。
10. figures/p2-contour60.png: $E[D] \leq 60$ m 等值线围出的待选区域 (图 6)。
11. figures/p2-wedge-geometry.png: 两探测束交会与定位区域直径构造 (图 4)。
12. Q2/Q2modeling.md 等: 问题二建模手稿、参考热力图与 Mathematica 校验脚本。

问题一程序依赖 Python 3、NumPy 和 Matplotlib, 在附件根目录依次运行以下命令, 可生成五组算例、四组固定种子扫描及正文插图。

Listing 1: 问题一数据与图形的复现命令

```
python Q1/p1_experiments.py
python Q1/make_figures.py
```

附录 B 问题一完整源程序

以下代码直接从对应支撑材料文件载入, 各程序按所列文件名保存并保持目录结构即可运行。

2.1 角楔交会、直径与覆盖判定

Listing 2: Q1/p1_intersection.py

```
# -*- coding: utf-8 -*-
"""问题一几何求解。坐标单位 m, 示向度为从正东逆时针量取的角度。

命令行: python p1_intersection.py cases/p1_case01.csv
全部实验: python p1_experiments.py
"""

from __future__ import annotations
import csv
import json
import math
import os
from pathlib import Path

import numpy as np

BEARING_ERR = 1.0      # 示向度误差界 (度)
ANG_TOL = 1e-9         # 角度判据容差 (度)
DIST_TOL = 1e-6        # 半平面判据容差 (米)
COVER_TOL = 1e-9       # 覆盖判据容差 (米)
CLIP = 1e7             # 裁剪参考实现的外框半边长 (米)
DIAM_BRUTE_LIMIT = 128 # 顶点数不超过该值时直径用暴力枚举
```

```

def bearing(p, q):
    """p 处指向 q 的方位角 (度, [0,360)) """
    return math.degrees(math.atan2(q[1] - p[1], q[0] - p[0])) % 360.0

def ang_diff(a, b):
    """a-b 归一化到 [-180,180)。"""
    return (a - b + 180.0) % 360.0 - 180.0

def wedge_halfplanes(p, theta, err=BEARING_ERR):
    """角楔 W = {X : |ang(X-p) - theta| <= err} 的两个半平面 a · X + b <= 0"""
    p = np.asarray(p, float)
    tm = math.radians((theta - err) % 360.0)
    tp = math.radians((theta + err) % 360.0)
    dm = np.array([math.cos(tm), math.sin(tm)])
    dp = np.array([math.cos(tp), math.sin(tp)])
    # cross(dm, X-p) >= 0  <=>  a1 · X + b1 <= 0
    a1 = np.array([dm[1], -dm[0]])
    b1 = float(-dm[1] * p[0] + dm[0] * p[1])
    # cross(dp, X-p) <= 0  <=>  a2 · X + b2 <= 0
    a2 = np.array([-dp[1], dp[0]])
    b2 = float(dp[1] * p[0] - dp[0] * p[1])
    return [(a1, b1), (a2, b2)]

def region_is_unbounded(svds, err=BEARING_ERR):
    """非空区域的公共延伸方向; 无公共方向时返回 None, 复杂度 O(m)。"""
    if len(svds) == 0:
        return None
    ref = float(svds[0])
    lo, hi = -err, err
    for theta in svds[1:]:
        center = ang_diff(theta, ref)
        lo, hi = max(lo, center - err), min(hi, center + err)
        if lo > hi + ANG_TOL:
            return None
    return (ref + (lo + hi) / 2.0) % 360.0

def _line_intersection(P, u, Q, v):
    """直线 P + t · u 与 Q + s · v 的交点, 平行时返回 None"""
    det = u[0] * (-v[1]) - (-v[0]) * u[1]
    if abs(det) < 1e-14:
        return None
    r = Q - P
    t = (r[0] * (-v[1]) - (-v[0]) * r[1]) / det
    return P + t * u

def convex_hull(P):
    """Andrew monotone chain, 逆时针, 去掉共线点"""
    def half(pts):
        st = []
        for p in pts:
            while len(st) >= 2 and _cross2(st[-1] - st[-2], p - st[-2]) <= 0:
                st.pop()
            st.append(p)
        return st
    P = P[np.lexsort((P[:, 1], P[:, 0]))]

```

```

    lo = half(P)
    hi = half(P[:-1])
    return np.array(lo[:-1] + hi[:-1])

def _cross2(u, v):
    return float(u[0] * v[1] - u[1] * v[0])

def _dedup_hull(cand):
    """候选顶点去重 + 凸包, 返回 (P, "empty"|"degenerate"|"polygon")"""
    if len(cand) == 0:
        return np.zeros((0, 2)), "empty"
    P = np.array(cand, float)
    P = P[np.lexsort((P[:, 1], P[:, 0]))]
    ded = [P[0]]
    for v in P[1:]:
        if np.linalg.norm(v - ded[-1]) > 1e-7:
            ded.append(v)
    P = np.array(ded)
    if len(P) < 3:
        return P, "degenerate"
    H = convex_hull(P)
    if len(H) < 3:
        return H, "degenerate"
    return H, "polygon"

def region_by_vertices(pts, svds, err=BEARING_ERR, tol=DIST_TOL):
    """主算法: 枚举 2m 条边界直线的两两交点, 用全部半平面判据筛选后求凸包。

    R 是凸集, 其任一极点必是两条起作用约束边界直线的交点, 故枚举 + 筛选是精确的;
    半平面判据在处理"边界直线过检测点"的情形时仍良定义, 检测点本身也可作顶点保留。
    """
    pts, svds = np.asarray(pts, float), np.asarray(svds, float)
    if pts.ndim != 2 or pts.shape[1] != 2 or len(pts) != len(svds) or len(pts) == 0:
        raise ValueError("需输入等长、非空的二维检测点和示向度数组")
    if not np.isfinite(pts).all() or not np.isfinite(svds).all() or not 0 < err < 90:
        raise ValueError("坐标和示向度须为有限数, 误差界须在 (0,90) 度内")
    H, lines = [], []
    for p, th in zip(pts, svds):
        H += wedge_halfplanes(p, th, err)
        for s in (-1.0, 1.0):
            a = math.radians((th + s * err) % 360.0)
            lines.append((np.asarray(p, float), np.array([math.cos(a), math.sin(a)])))
    cand = []
    n = len(lines)
    for i in range(n):
        for j in range(i + 1, n):
            X = _line_intersection(lines[i][0], lines[i][1], lines[j][0], lines[j][1])
            if X is None or not np.all(np.isfinite(X)):
                continue
            if all(float(a @ X + b) <= tol for (a, b) in H):
                cand.append(X)
    return _dedup_hull(cand)

def clip_halfplane(poly, a, b, tol=DIST_TOL):
    """Sutherland-Hodgman: 保留 {X : a * X + b <= 0} 的部分"""
    if not poly:
        return []
    out = []

```



```

n = len(poly)
for i in range(n):
    P = poly[i]
    Q = poly[(i + 1) % n]
    fp = float(a @ P + b)
    fq = float(a @ Q + b)
    if fp <= tol:
        out.append(P)
    if (fp < -tol and fq > tol) or (fp > tol and fq < -tol):
        t = fp / (fp - fq)
        out.append(P + t * (Q - P))
ded = []
for v in out:
    if not ded or np.linalg.norm(v - ded[-1]) > 1e-9:
        ded.append(v)
if len(ded) > 1 and np.linalg.norm(ded[0] - ded[-1]) < 1e-9:
    ded.pop()
return ded

def region_by_clipping(pts, svds, err=BEARING_ERR, box=CLIP, tol=DIST_TOL):
    """有界算例的裁剪校验：若碰到人为外框则要求增大 box。"""
    poly = [np.array([-box, -box], float), np.array([box, -box], float),
            np.array([box, box], float), np.array([-box, box], float)]
    for p, th in zip(pts, svds):
        for (a, b) in wedge_halfplanes(p, th, err):
            poly = clip_halfplane(poly, a, b, tol)
            if not poly:
                return np.zeros((0, 2)), "empty"
    P = np.array(poly, float)
    if len(P) < 3:
        return P, "degenerate"
    if float(np.max(np.abs(P))) >= box - tol:
        raise ValueError("区域碰到校验外框：请增大 box 或先检查有界性")
    H = convex_hull(P)
    return (H if len(H) >= 3 else P), "polygon"

def region_from_bearings(pts, svds, err=BEARING_ERR):
    """主入口：返回（顶点数组，status），status 取 empty / unbounded / degenerate / bounded。"""
    P, st = region_by_vertices(pts, svds, err)
    if st == "empty":
        return P, "empty"
    rec = region_is_unbounded(list(svds), err) if len(svds) else None
    if rec is not None:
        return P, "unbounded"
    if st == "degenerate":
        return P, "degenerate"
    return P, "bounded"

def _diameter_brute(P):
    n = len(P)
    best, pair = -1.0, (P[0], P[1])
    for i in range(n - 1):
        d = np.linalg.norm(P[i + 1:] - P[i], axis=1)
        k = int(np.argmax(d))
        if float(d[k]) > best:
            best, pair = float(d[k]), (P[i], P[i + 1 + k])
    return best, pair

```

```

def rotating_calipers(P):
    """凸多边形直径 (旋转卡壳,  $O(n)$ )。P 需为逆时针凸序 (允许共线点)。"""
    n = len(P)
    if n < 3:
        return diameter(P, brute_limit=2)
    k = 1
    best, pair = -1.0, (P[0], P[1])
    for i in range(n):
        i2 = (i + 1) % n
        e = P[i2] - P[i]
        while True:
            k2 = (k + 1) % n
            if abs(_cross2(e, P[k2] - P[i])) > abs(_cross2(e, P[k] - P[i])):
                k = k2
            else:
                break
        for j in (k, (k + 1) % n):
            d = float(np.linalg.norm(P[i] - P[j]))
            if d > best:
                best, pair = d, (P[i], P[j])
            d = float(np.linalg.norm(P[i2] - P[j]))
            if d > best:
                best, pair = d, (P[i2], P[j])
    return best, pair

def diameter(P, brute_limit=DIAM_BRUTE_LIMIT):
    """凸多边形直径, 返回 (D, (A, B))"""
    n = len(P)
    if n == 0:
        return 0.0, None
    if n == 1:
        return 0.0, (P[0], P[0])
    if n == 2:
        return float(np.linalg.norm(P[0] - P[1])), (P[0], P[1])
    if n <= brute_limit:
        return _diameter_brute(P)
    return rotating_calipers(P)

def min_enclosing_circle(P):
    """最小包围圆 (center, radius); 顶点数很少, 直接枚举 1/2/3 点候选圆。"""
    P = np.asarray(P, float)
    n = len(P)
    if n == 0:
        return None, 0.0
    if n == 1:
        return P[0].copy(), 0.0
    cand = [(P[i].copy(), 0.0) for i in range(n)]
    for i in range(n):
        for j in range(i + 1, n):
            c = 0.5 * (P[i] + P[j])
            cand.append((c, float(np.linalg.norm(P[i] - c))))
    for i in range(n):
        for j in range(i + 1, n):
            for k in range(j + 1, n):
                A, B, C = P[i], P[j], P[k]
                d = 2.0 * (A[0] * (B[1] - C[1]) + B[0] * (C[1] - A[1])
                           + C[0] * (A[1] - B[1]))
                if abs(d) < 1e-12:
                    continue
                aa, bb, cc = float(A @ A), float(B @ B), float(C @ C)

```

```

        ux = (aa * (B[1] - C[1]) + bb * (C[1] - A[1]) + cc * (A[1] - B[1])) / d
        uy = (aa * (C[0] - B[0]) + bb * (A[0] - C[0]) + cc * (B[0] - A[0])) / d
        c = np.array([ux, uy])
        cand.append((c, float(np.linalg.norm(A - c))))

    best = None
    for c, r in cand:
        if all(float(np.linalg.norm(P[i] - c)) <= r + 1e-9 for i in range(n)):
            if best is None or r < best[1]:
                best = (c, r)
    return best

def region_area(P):
    n = len(P)
    if n < 3:
        return 0.0
    return abs(0.5 * float(np.sum(P[:, 0] * np.roll(P[:, 1], -1)
                                   - np.roll(P[:, 0], -1) * P[:, 1])))

def analyse(pts, svds, err=BEARING_ERR):
    """给定检测点与示向度，输出定位区域的全套几何量（长度量按 4 位小数落盘）。"""
    P, status = region_from_bearings(pts, svds, err)
    base = {"n_bearings": len(pts), "status": status}
    if status == "empty":
        base["reason"] = "各角楔无公共交集，测量数据与误差界不相容"
        return base
    if status == "unbounded":
        rec = region_is_unbounded(list(svds), err)
        base["reason"] = "非空角楔交集存在公共延伸方向，直径为无穷大"
        base["recession_direction_deg"] = None if rec is None else round(float(rec), 4)
        base["n_vertices"] = int(len(P))
        base["vertices"] = [[round(float(v[0]), 4), round(float(v[1]), 4)] for v in P]
        return base
    if status == "degenerate" or len(P) < 3:
        base["status"] = "degenerate"
        base["reason"] = "公共交集退化为线段或点"
        base["vertices"] = [[round(float(v[0]), 4), round(float(v[1]), 4)] for v in P]
        return base

    D, (A, B) = diameter(P)
    c = 0.5 * (A + B)
    maxr = float(max(np.linalg.norm(v - c) for v in P))
    worst = P[int(np.argmax(np.linalg.norm(P - c, axis=1)))]
    mc, mr = min_enclosing_circle(P)
    base.update({
        "n_vertices": int(len(P)),
        "vertices": [[round(float(v[0]), 4), round(float(v[1]), 4)] for v in P],
        "diameter_m": round(D, 4),
        "diameter_endpoints": [[round(float(A[0]), 4), round(float(A[1]), 4)],
                               [round(float(B[0]), 4), round(float(B[1]), 4)]],
        "circle_center": [round(float(c[0]), 4), round(float(c[1]), 4)],
        "circle_radius_m": round(D / 2.0, 4),
        "max_vertex_dist_m": round(maxr, 4),
        "diameter_circle_covers": bool(maxr <= D / 2.0 + COVER_TOL),
        "farthest_vertex": [round(float(x), 4) for x in worst],
        "min_enclosing_center": [round(float(mc[0]), 4), round(float(mc[1]), 4)],
        "min_enclosing_radius_m": round(float(mr), 4),
        "min_circle_over_half_diameter": round(float(mr) / (D / 2.0), 6),
        "area_m2": round(region_area(P), 4),
    })
    return base

```

```

def run_case_csv(path, truth_path=None, err=BEARING_ERR):
    """读单个算例 CSV (列含 x_m,y_m,svd_deg) 与可选真值 JSON, 返回 analyse 结果。"""
    with open(path, encoding="utf-8") as f:
        rows = list(csv.DictReader(f))
    pts = [(float(r["x_m"]), float(r["y_m"])) for r in rows]
    svds = [float(r["svd_deg"]) for r in rows]
    res = analyse(pts, svds, err)
    res["n_bearings"] = len(rows)
    if truth_path and os.path.exists(truth_path):
        with open(truth_path, encoding="utf-8") as f:
            truth = json.load(f)
        G = truth["source_xy"]
        res["truth_xy"] = G
        if "circle_center" in res:
            res["center_error_vs_truth_m"] = round(math.dist(res["circle_center"], G), 3)
        if "min_enclosing_center" in res:
            res["mincircle_error_vs_truth_m"] = round(
                math.dist(res["min_enclosing_center"], G), 3)
        # 真值是否满足全部约束 (等价于在半平面内)
        H = []
        for p, th in zip(pts, svds):
            H += wedge_halfplanes(p, th, err)
        res["truth_inside_region"] = bool(all(a @ np.asarray(G, float) + b <= 1e-6
                                             for (a, b) in H))

    return res

if __name__ == "__main__":
    import argparse
    parser = argparse.ArgumentParser(description=__doc__)
    parser.add_argument("csv", nargs="?", default="cases/p1_case01.csv")
    parser.add_argument("--error", type=float, default=BEARING_ERR)
    args = parser.parse_args()
    path = Path(args.csv)
    if not path.is_absolute() and not path.exists():
        path = Path(__file__).resolve().parent / path
    print(json.dumps(run_case_csv(path, path.with_suffix(".truth.json"), args.error),
                    ensure_ascii=False, indent=2))

```

2.2 算例生成、随机扫描与几何校验

Listing 3: Q1/p1_experiments.py

```

# -*- coding: utf-8 -*-
"""问题一: 固定种子生成五组算例, 求解并统计四组几何扫描。

运行 python p1_experiments.py: 数据写入 cases/ 和 results/。
加 --skip-sweep 只计算算例, 使用独立结果名, 避免覆盖完整扫描。
"""
from __future__ import annotations
import argparse
import csv
import json
import math
import os
import random
import sys

```

```

import numpy as np

sys.path.insert(0, os.path.dirname(os.path.abspath(__file__)))
import p1_intersection as P

R_ARENA = 1800.0 # 目标区域半径 (m)
R_RECV_MIN, R_RECV_MAX = 1000.0, 1500.0 # 有效接收距离范围 (m)
NEAR_R = 5.0 # 近场阈值 (m)
DEFAULT_SEED = 20260901
SWEEP_TWO_POINT = {"n_trials": 20000, "seed": 17, "r_max": 1800.0}
SWEEP_WELL_COND = {"n_trials": 4000, "seed": 5, "min_gap": 3.0,
                    "dist": (300.0, 1500.0), "m_choices": (3, 4, 5, 6)}
SWEEP_ERR_SEED_OFFSET = 900001 # 误差随机流与构型随机流分离, 保证理想/实际同构型配对
CASE_FIELDS = ["det_id", "x_m", "y_m", "svd_deg", "true_svd_deg",
               "true_az_err_deg", "dist_m"]
CASE_LABELS = ["A1", "A2", "A3", "A4", "A5"]

def wrap360(a):
    return a % 360.0

def bearing(p, q):
    """p 处指向 q 的方位角 (度)"""
    return wrap360(math.degrees(math.atan2(q[1] - p[1], q[0] - p[0])))

def in_cone(src, p, half_deg):
    """点 p 是否落在以 src 为顶点、沿定向方向的 ±half_deg 锥内 (全向 half=180)"""
    if half_deg >= 180.0:
        return True
    return abs(P.ang_diff(bearing(src["xy"], p), src["dir_deg"])) <= half_deg + 1e-12

def measure(src, p):
    """按题目附录规则返回一次检测结果 (不含示向度误差)"""
    d = math.dist(p, src["xy"])
    if d > src["r_recv"]:
        return "no_signal", None
    if not in_cone(src, p, src["cone_half"]):
        return "no_signal", None
    if d <= NEAR_R:
        return "near", None
    return "direction", bearing(p, src["xy"])

def make_case_p1(rng, n_pts=3, min_az_gap=25.0, min_dist=150.0,
                 max_dist=1300.0, spread=140.0):
    """生成一组"良态"算例: 检测点分布在干扰源四周一定方位张角内, 且有最小间隔与距离。"""
    ang = rng.uniform(0, 2 * math.pi)
    rad = rng.uniform(50, 0.75 * R_ARENA)
    G = (rad * math.cos(ang), rad * math.sin(ang))
    src = {"xy": G, "r_recv": rng.uniform(R_RECV_MIN, R_RECV_MAX),
           "cone_half": 180.0, "dir_deg": None}

    n = max(2, n_pts)
    base = rng.uniform(0, 360)
    span = 2 * spread
    while n * min_az_gap > span:
        min_az_gap *= 0.85
    offs = np.sort(rng.uniform(-spread, spread, size=n))
    for _ in range(200):

```

```

gaps = np.diff(offss)
if len(gaps) == 0 or gaps.min() >= min_az_gap:
    break
offss = np.sort(rng.uniform(-spread, spread, size=n))

pts, azs = [], []
for o in offss:
    a = math.radians((base + o) % 360.0)
    for _try in range(80):
        d = rng.uniform(min_dist, min(max_dist, src["r_recv"]))
        p = (G[0] + d * math.cos(a), G[1] + d * math.sin(a))
        if math.hypot(*p) <= R_ARENA:
            pts.append(p)
            azs.append(bearing(p, G))
            break
if len(pts) < 2:
    return [], {"error": "failed to build well-conditioned case"}

errs = rng.uniform(-P.BEARING_ERR, P.BEARING_ERR, size=len(pts))
rows = []
for i, (p, e) in enumerate(zip(pts, errs), start=1):
    res, svd = measure(src, p)
    if res != "direction":
        continue
    rows.append({
        "det_id": i,
        "x_m": round(p[0], 3),
        "y_m": round(p[1], 3),
        "svd_deg": round(wrap360(svd + e), 2),
        "true_svd_deg": round(svd, 4),
        "true_az_err_deg": round(e, 4),
        "dist_m": round(math.dist(p, G), 3),
    })
truth = {"source_xy": [round(G[0], 3), round(G[1], 3)],
        "r_recv_m": round(src["r_recv"], 3),
        "type": "omnidirectional", "dir_deg": None,
        "bearing_error_bound_deg": P.BEARING_ERR,
        "azimuth_span_deg": round(float(max(azs) - min(azs)), 2),
        "min_azimuth_gap_deg": round(float(np.min(np.diff(np.sort(azs)))), 2),
        "if len(azs) > 1 else None,
        "n_detection_points": len(rows)}
return rows, truth

def write_csv(path, rows, fields):
    with open(path, "w", newline="", encoding="utf-8") as f:
        w = csv.DictWriter(f, fieldnames=fields, lineterminator="\n")
        w.writeheader()
        w.writerows(rows)

def gen_cases(out_dir, seed=DEFAULT_SEED, n_cases=5):
    """生成 n_cases 组算例，返回（文件路径，真值）列表。"""
    rng = np.random.default_rng(seed)
    out = []
    for k in range(n_cases):
        rows, truth = make_case_p1(rng, n_pts=int(rng.integers(3, 7)))
        name = "p1_case%02d" % (k + 1)
        csv_path = os.path.join(out_dir, name + ".csv")
        truth_path = os.path.join(out_dir, name + ".truth.json")
        write_csv(csv_path, rows, CASE_FIELDS)
        with open(truth_path, "w", encoding="utf-8") as f:

```

```

        json.dump(truth, f, ensure_ascii=False, indent=2)
    out.append((csv_path, truth_path))
    return out

def solve_full(pts, svds):
    """返回 dict: status / D / half / max_vertex_dist / ratio / excess / covers. """
    V, st = P.region_from_bearings(pts, svds)
    if st != "bounded":
        return {"status": st}
    D, (A, B) = P.diameter(V)
    M = 0.5 * (A + B)
    dists = np.linalg.norm(V - M, axis=1)
    k = int(np.argmax(dists))
    maxr = float(dists[k])
    half = D / 2.0
    ua, ub = A - V[k], B - V[k]
    na, nb = float(np.linalg.norm(ua)), float(np.linalg.norm(ub))
    # 覆盖成立时最远顶点就是直径端点 (ua 或 ub 为零向量), 该处的张角无定义
    angle = None
    if na > 1e-12 and nb > 1e-12:
        cosang = float(ua @ ub / (na * nb))
        angle = math.degrees(math.acos(max(-1.0, min(1.0, cosang))))
    return {"status": "bounded", "D": float(D), "half": float(half),
            "max_vertex_dist": maxr, "ratio": maxr / half,
            "excess": maxr - half, "covers": bool(maxr <= half + P.COVER_TOL),
            "worst_vertex_angle_deg": angle}

def run_cases(case_paths):
    """逐例求解 (analyse 落盘 4 位小数 + 全精度比值与张角)。"""
    results = []
    for i, (csv_path, truth_path) in enumerate(case_paths):
        res = P.run_case_csv(csv_path, truth_path)
        res["case"] = os.path.basename(csv_path)
        res["label"] = CASE_LABELS[i] if i < len(CASE_LABELS) else "C%d" % (i + 1)
        if res["status"] == "bounded":
            rows = list(csv.DictReader(open(csv_path, encoding="utf-8")))
            pts = [(float(r["x_m"]), float(r["y_m"])) for r in rows]
            svds = [float(r["svd_deg"]) for r in rows]
            full = solve_full(pts, svds)
            res["ratio"] = round(full["ratio"], 6)
            res["excess_m"] = round(full["excess"], 4)
            res["excess_pct"] = round(100.0 * full["excess"] / full["half"], 4)
            ang = full["worst_vertex_angle_deg"]
            res["worst_vertex_angle_deg"] = None if ang is None else round(ang, 2)
        results.append(res)
    return results

def sweep_two_point(n_trials=SWEETWO_POINT["n_trials"], seed=SWEETWO_POINT["seed"],
                    r_max=SWEETWO_POINT["r_max"], with_error=False):
    """两测点构型: 检测点落在半径 r_max 的目标区域内, 方位差 3°~179°, 只统计有界者。

    with_error=False 为理想构型 (示向度精确指向源点); True 为实际构型 (在同一批构型上
    叠加 ±BEARING_ERR 的测向误差)。误差取自独立随机流, 两组构型逐例配对。
    """
    rng = random.Random(seed)
    erng = random.Random(seed + SWEETWO_POINT["err_seed_offset"])
    n_bounded = n_fail = 0
    n_unbounded = n_empty = n_degenerate = n_rejected = 0
    worst = None

```

```

for _ in range(n_trials):
    dth = rng.uniform(3.0, 179.0)
    th1 = rng.uniform(90.0, 270.0)
    th2 = (th1 + dth) % 360.0
    r1, r2 = rng.uniform(150.0, r_max), rng.uniform(150.0, r_max)
    A = (r1 * math.cos(math.radians(th1 + 180.0)),
         r1 * math.sin(math.radians(th1 + 180.0)))
    B = (r2 * math.cos(math.radians(th2 + 180.0)),
         r2 * math.sin(math.radians(th2 + 180.0)))
    if math.hypot(*A) > r_max or math.hypot(*B) > r_max:
        n_rejected += 1
        continue
    if with_error:
        # 测得示向度 = 真方位角 + 误差
        th1 = (th1 + erng.uniform(-P.BEARING_ERR, P.BEARING_ERR)) % 360.0
        th2 = (th2 + erng.uniform(-P.BEARING_ERR, P.BEARING_ERR)) % 360.0
    st = solve_full([A, B], [th1, th2])
    if st["status"] != "bounded":
        if st["status"] == "unbounded":
            n_unbounded += 1
        elif st["status"] == "empty":
            n_empty += 1
        elif st["status"] == "degenerate":
            n_degenerate += 1
        continue
    n_bounded += 1
    if worst is None or st["ratio"] > worst["ratio"]:
        worst = {"S1": [A[0], A[1], th1],
                 "S2": [B[0], B[1], th2],
                 "D_m": round(st["D"], 4), "max_vertex_dist_m": round(st["max_vertex_dist"], 4),
                 "ratio": round(st["ratio"], 6), "covers": st["covers"]}
    if not st["covers"]:
        n_fail += 1
return {"n_trials": n_trials, "seed": seed, "r_max_m": r_max,
        "with_error": bool(with_error),
        "bearing_error_deg": P.BEARING_ERR if with_error else 0.0,
        "n_bounded": n_bounded, "n_fail": n_fail,
        "n_unbounded": n_unbounded, "n_empty": n_empty, "n_degenerate": n_degenerate,
        "n_rejected_geometry": n_rejected,
        "fail_pct": 100.0 * n_fail / max(n_bounded, 1),
        "worst_ratio": None if worst is None else worst["ratio"],
        "worst_config": worst}

def sweep_well_conditioned(n_trials=SWEET_WELL_COND["n_trials"],
                           seed=SWEET_WELL_COND["seed"],
                           min_gap=SWEET_WELL_COND["min_gap"],
                           dist=SWEET_WELL_COND["dist"],
                           m_choices=SWEET_WELL_COND["m_choices"],
                           with_error=False):
    """3-6 个检测点的良态构型：方位角最小间隔 > min_gap, 检测点到原点距离在 dist 内。

    with_error 的含义同 sweep_two_point: True 时在同一批构型上叠加 ±BEARING_ERR 测向误差。
    """
    rng = random.Random(seed)
    erng = random.Random(seed + SWEET_ERR_SEED_OFFSET)
    n_bounded = n_fail = 0
    n_unbounded = n_empty = n_degenerate = n_rejected = 0
    per_m = {str(m): {"bounded": 0, "fail": 0} for m in m_choices}
    worst = None
    for _ in range(n_trials):
        m = rng.choice(list(m_choices))
        ang = sorted(rng.uniform(0.0, 360.0) for _ in range(m))

```



```

gaps = [min((angs[i] - angs[j]) % 360.0, (angs[j] - angs[i]) % 360.0)
         for i in range(m) for j in range(i + 1, m)]
if min(gaps) <= min_gap:
    n_rejected += 1
    continue
pts, svds = [], []
for t in angs:
    rr = rng.uniform(dist[0], dist[1])
    pts.append((rr * math.cos(math.radians(t)), rr * math.sin(math.radians(t))))
    s = t + 180.0
    if with_error:
        # 测得示向度 = 真方位角 + 误差
        s += erng.uniform(-P.BEARING_ERR, P.BEARING_ERR)
    svds.append(s % 360.0)
st = solve_full(pts, svds)
if st["status"] != "bounded":
    if st["status"] == "unbounded":
        n_unbounded += 1
    elif st["status"] == "empty":
        n_empty += 1
    elif st["status"] == "degenerate":
        n_degenerate += 1
    continue
n_bounded += 1
per_m[str(m)]["bounded"] += 1
if worst is None or st["ratio"] > worst["ratio"]:
    worst = {"m": m, "pts": [list(p) for p in pts],
             "svds_deg": list(svds),
             "D_m": round(st["D"], 4), "max_vertex_dist_m": round(st["max_vertex_dist"], 4),
             "ratio": round(st["ratio"], 6), "covers": st["covers"]}
if not st["covers"]:
    n_fail += 1
    per_m[str(m)]["fail"] += 1
for v in per_m.values():
    v["fail_pct"] = round(100.0 * v["fail"] / max(v["bounded"], 1), 4)
return {"n_trials": n_trials, "seed": seed, "min_gap_deg": min_gap,
        "dist_range_m": list(dist), "m_choices": list(m_choices),
        "with_error": bool(with_error),
        "bearing_error_deg": P.BEARING_ERR if with_error else 0.0,
        "n_bounded": n_bounded, "n_fail": n_fail,
        "n_unbounded": n_unbounded, "n_empty": n_empty, "n_degenerate": n_degenerate,
        "n_rejected_geometry": n_rejected,
        "fail_pct": 100.0 * n_fail / max(n_bounded, 1),
        "worst_ratio": None if worst is None else worst["ratio"],
        "worst_config": worst, "per_m": per_m}

```

```

SWEEP_PARAM_KEYS = ("n_trials", "seed", "r_max_m", "min_gap_deg", "dist_range_m",
                    "m_choices", "with_error", "bearing_error_deg")

```

```

def build_summary(case_paths, results, sweeps, seed):
    ok = [r for r in results if r["status"] == "bounded"]
    cases_summary = {
        "n_cases": len(results),
        "n_bounded": len(ok),
        "n_unbounded": sum(1 for r in results if r["status"] == "unbounded"),
        "n_degenerate": sum(1 for r in results if r["status"] == "degenerate"),
        "n_empty": sum(1 for r in results if r["status"] == "empty"),
        "diameter_m_range": [min(r["diameter_m"] for r in ok),
                             max(r["diameter_m"] for r in ok)] if ok else None,
        "n_diameter_circle_misses": sum(1 for r in ok if not r["diameter_circle_covers"]),
        "min_circle_over_half_diameter_max": max(

```

```

        r["min_circle_over_half_diameter"] for r in ok) if ok else None,
        "n_truth_inside_region": sum(1 for r in ok if r.get("truth_inside_region")),
    }
per_case = [{"case": r["case"], "label": r["label"], "n_bearings": r["n_bearings"],
            "n_vertices": r.get("n_vertices"), "diameter_m": r.get("diameter_m"),
            "circle_radius_m": r.get("circle_radius_m"),
            "max_vertex_dist_m": r.get("max_vertex_dist_m"),
            "ratio": r.get("ratio"), "excess_m": r.get("excess_m"),
            "excess_pct": r.get("excess_pct"),
            "coverage": r.get("diameter_circle_covers"),
            "min_circle_over_half_diameter": r.get("min_circle_over_half_diameter"),
            "worst_vertex_angle_deg": r.get("worst_vertex_angle_deg"),
            "center_error_vs_truth_m": r.get("center_error_vs_truth_m"),
            "truth_inside_region": r.get("truth_inside_region")} for r in results]
meta = {"seed": seed, "n_cases": len(results),
        "coverage_tolerance_m": P.COVER_TOL,
        "halfplane_tolerance_m": P.DIST_TOL,
        "arena_radius_m": R_ARENA,
        "bearing_error_bound_deg": P.BEARING_ERR,
        "r_recv_range_m": [R_RECV_MIN, R_RECV_MAX],
        "case_files": ["cases/" + os.path.basename(c) for c, _ in case_paths],
        "sweeps": {k: {p: s[p] for p in SWEEP_PARAM_KEYS if p in s}
                    for k, s in sweeps.items()}}
return {"meta": meta, "cases_summary": cases_summary, "per_case": per_case,
        "sweeps": sweeps}

def empty_sweep(base, with_error):
    """--skip-sweep 时的占位结果：字段与真实扫描一致，仅数值为空。"""
    d = {"n_trials": base["n_trials"], "seed": base["seed"],
        "with_error": bool(with_error),
        "bearing_error_deg": P.BEARING_ERR if with_error else 0.0,
        "n_bounded": 0, "n_fail": 0, "fail_pct": None, "worst_ratio": None,
        "worst_config": None, "n_unbounded": 0, "n_empty": 0, "n_degenerate": 0}
    if "r_max" in base:
        d["r_max_m"] = base["r_max"]
    if "min_gap" in base:
        d["min_gap_deg"] = base["min_gap"]
        d["dist_range_m"] = list(base["dist"])
        d["m_choices"] = list(base["m_choices"])
        d["per_m"] = {}
    return d

def validate_cases(case_paths):
    """复算五组算例的裁剪误差、直径及真值约束，保存正文校验量。"""
    max_vertex_error = max_diameter_error = 0.0
    spans, gaps, distances = [], [], []
    for csv_path, truth_path in case_paths:
        with open(csv_path, encoding="utf-8") as f:
            rows = list(csv.DictReader(f))
        with open(truth_path, encoding="utf-8") as f:
            truth = json.load(f)
        pts = [(float(r["x_m"]), float(r["y_m"])) for r in rows]
        svds = [float(r["svd_deg"]) for r in rows]
        V, status = P.region_from_bearings(pts, svds)
        C, _ = P.region_by_clipping(pts, svds)
        error = max(min(float(np.linalg.norm(v - c)) for c in C) for v in V)
        D, _ = P.diameter(V)
        Dc, _ = P.rotating_calipers(V)
        assert status == "bounded" and len(V) == len(C) and error < 1e-6
        assert abs(D - Dc) < 1e-9

```

```

    assert P.run_case_csv(csv_path, truth_path)["truth_inside_region"]
    max_vertex_error = max(max_vertex_error, error)
    max_diameter_error = max(max_diameter_error, abs(D - Dc))
    spans.append(truth["azimuth_span_deg"])
    gaps.append(truth["min_azimuth_gap_deg"])
    distances.extend(float(row["dist_m"]) for row in rows)
return {"cases_checked": len(case_paths),
        "max_vertex_difference_m": max_vertex_error,
        "max_diameter_difference_m": max_diameter_error,
        "azimuth_span_deg_range": [min(spans), max(spans)],
        "min_azimuth_gap_deg": min(gaps),
        "source_distance_m_range": [min(distances), max(distances)]}

def main(argv=None):
    ap = argparse.ArgumentParser(description="问题一: 算例生成 + 逐例求解 + 四组随机扫描")
    ap.add_argument("--out", default=os.path.dirname(os.path.abspath(__file__)),
                    help="算例与结果的输出目录（默认为本文件所在目录）")
    ap.add_argument("--seed", type=int, default=DEFAULT_SEED, help="算例生成种子")
    ap.add_argument("--cases", type=int, default=5, help="算例组数")
    ap.add_argument("--skip-sweep", action="store_true", help="跳过四组随机扫描")
    args = ap.parse_args(argv)
    if args.cases < 1:
        ap.error("算例数须为正整数")

    out = os.path.abspath(args.out)
    os.makedirs(out, exist_ok=True)
    case_dir = os.path.join(out, "cases")
    os.makedirs(case_dir, exist_ok=True)
    result_dir = os.path.join(out, "results")
    os.makedirs(result_dir, exist_ok=True)
    case_paths = gen_cases(case_dir, args.seed, args.cases)
    results = run_cases(case_paths)

    jl = os.path.join(result_dir, "p1_results.jsonl")
    with open(jl, "w", encoding="utf-8") as f:
        for r in results:
            f.write(json.dumps(r, ensure_ascii=False) + "\n")

    if args.skip_sweep:
        sweeps = {"two_point": empty_sweep(SWEEP_TWO_POINT, False),
                  "two_point_err": empty_sweep(SWEEP_TWO_POINT, True),
                  "well_conditioned": empty_sweep(SWEEP_WELL_COND, False),
                  "well_conditioned_err": empty_sweep(SWEEP_WELL_COND, True)}
    else:
        sweeps = {"two_point": sweep_two_point(),
                  "two_point_err": sweep_two_point(with_error=True),
                  "well_conditioned": sweep_well_conditioned(),
                  "well_conditioned_err": sweep_well_conditioned(with_error=True)}

    summary = build_summary(case_paths, results, sweeps, args.seed)
    summary["validation"] = validate_cases(case_paths)
    summary_name = "p1_cases_only.json" if args.skip_sweep else "p1_summary.json"
    with open(os.path.join(result_dir, summary_name), "w", encoding="utf-8") as f:
        json.dump(summary, f, ensure_ascii=False, indent=2)

    print("== 逐例结果 (%s) == % out)
    for r in results:
        print("%-4s %-16s %-10s D=%-9s D/2=%-9s max|P-M|=%-9s ratio=%-9s cover=%-5s r*/D*2=%-8s" % (
            r["label"], r["case"], r["status"], r.get("diameter_m", "-"),
            r.get("circle_radius_m", "-"), r.get("max_vertex_dist_m", "-"),
            r.get("ratio", "-"), r.get("diameter_circle_covers", "-"),

```

```

        r.get("min_circle_over_half_diameter", "-"))
if not args.skip_sweep:
    print("\n== 随机扫描 (理想构型 / 实际构型: 同构型叠加  $\pm 0.0^\circ$  测向误差) =="
        % P.BEARING_ERR)
    for key, name in (("two_point", "两测点 理想"),
        ("two_point_err", "两测点 实际"),
        ("well_conditioned", "3-6 测点 理想"),
        ("well_conditioned_err", "3-6 测点 实际")):
        s = sweeps[key]
        print("%-14s: trials=%d bounded=%d fail=%d (%.4f%%) worst_ratio=%s "
            "[unbounded=%d empty=%d degenerate=%d rejected=%d]" % (
                name, s["n_trials"], s["n_bounded"], s["n_fail"], s["fail_pct"],
                "-" if s["worst_ratio"] is None else "%.6f" % s["worst_ratio"],
                s["n_unbounded"], s["n_empty"], s["n_degenerate"],
                s["n_rejected_geometry"]))
    print("\n== 论文表 2 行 ==")
    for r in results:
        lab = r["A\textsubscript{%s}" % r["label"].lstrip("AC")]
        print("          %s & %d & %d & %.2f & %.2f & %.2f & %.4f & %s \\\\" % (
            lab, r["n_bearings"], r["n_vertices"], r["diameter_m"],
            r["circle_radius_m"], r["max_vertex_dist_m"], r["ratio"],
            "是" if r["diameter_circle_covers"] else "否"))
    print("\n写出: %s, %s, cases/p1_case*.csv, cases/p1_case*.truth.json"
        % (os.path.join(result_dir, summary_name), j1))
    return 0

if __name__ == "__main__":
    sys.exit(main())

```

2.3 问题一绘图

Listing 4: Q1/make_figures.py

```

"""由算例和几何求解器重绘问题一的三幅正文图。"""
from pathlib import Path
import csv
import os
import tempfile

os.environ.setdefault("MPLCONFIGDIR", str(Path(tempfile.gettempdir()) / "b-locator-mpl"))
import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt
from matplotlib import font_manager
from matplotlib.patches import Circle, FancyBboxPatch, Polygon
import numpy as np

import p1_intersection as P
import p1_experiments as E

HERE = Path(__file__).resolve().parent
PAPER = HERE.parent / "paper"
OUT = PAPER / "figures"
BLUE, RED, ORANGE = "#2463a5", "#b73432", "#cc6500"

def setup():
    font = PAPER / "fonts" / "simsun.ttc"
    if font.exists():

```

```

        font_manager.fontManager.addfont(font)
        plt.rcParams["font.family"] = font_manager.FontProperties(fname=font).get_name()
    plt.rcParams.update({"font.size": 11, "mathtext.fontset": "stix",
                        "axes.unicode_minus": False, "savefig.dpi": 300})
    OUT.mkdir(parents=True, exist_ok=True)

def save(fig, name):
    fig.savefig(OUT / name, bbox_inches="tight", facecolor="white")
    plt.close(fig)

def draw_region(ax, vertices, labels=True):
    D, (A, B) = P.diameter(vertices)
    M = (A + B) / 2
    ax.add_patch(Polygon(vertices, facecolor="#f2d0ce", edgecolor=RED, lw=1.5))
    ax.add_patch(Circle(M, D / 2, fill=False, color=BLUE, lw=1.5))
    ax.plot([A[0], B[0]], [A[1], B[1]], "k-", lw=1.2)
    ax.scatter(*np.array([A, B]).T, facecolor="white", edgecolor="black", zorder=5)
    ax.plot(*M, "k+", ms=9, mew=1.2)
    if labels:
        for name, point in (("A", A), ("B", B), ("M", M)):
            ax.annotate(f"${name}$", point, xytext=(5, 5), textcoords="offset points")
    ax.set_aspect("equal")
    ax.set_xlim(M[0] - 0.74 * D, M[0] + 0.74 * D)
    ax.set_ylim(M[1] - 0.74 * D, M[1] + 0.74 * D)
    return D, A, B, M

def coverage():
    fig, axes = plt.subplots(1, 2, figsize=(8.2, 4.2), constrained_layout=True)
    for i, ax in enumerate(axes, 1):
        with (HERE / "cases" / f"pi_case{i:02d}.csv").open(encoding="utf-8") as f:
            rows = list(csv.DictReader(f))
            points = [(float(r["x_m"]), float(r["y_m"])) for r in rows]
            bearings = [float(r["svd_deg"]) for r in rows]
            vertices, _ = P.region_from_bearings(points, bearings)
            D, A, B, M = draw_region(ax, vertices)
            full = E.solve_full(points, bearings)
            status = "覆盖成立" if full["covers"] else "覆盖不成立"
            ax.set_title(f"({'ab'[i-1]}) $A_{i}$: {status}\n"
                        f"$D={D:.2f}$ m, $\max |P-M|={full['max\_vertex\_dist']:.2f}$ m",
                        fontsize=11, color=BLUE if full["covers"] else ORANGE)
            if not full["covers"]:
                worst = vertices[np.argmax(np.linalg.norm(vertices - M, axis=1))]
                ax.plot([M[0], worst[0]], [M[1], worst[1]], "--", color=ORANGE)
                ax.scatter(*worst, s=55, facecolor="white", edgecolor=ORANGE, zorder=6)
                ax.annotate(f"$P$ ($\{full['worst\_vertex\_angle\_deg']:.1f\}^\circ$)", worst,
                            xytext=(-15, -22), textcoords="offset points", color=ORANGE)
                ax.set_xlabel("$x$ / m")
                ax.set_ylabel("$y$ / m")
                ax.tick_params(labelsize=9)
            save(fig, "pi-coverage-contrast.png")

def flowchart():
    fig, ax = plt.subplots(figsize=(7.6, 4.4))
    ax.set(xlim=(0, 10), ylim=(0, 6.8))
    ax.axis("off")
    steps = [(6.2, "输入检测点与示向度"),
             (5.05, "边界交点枚举、半平面筛选、凸包"),
             (3.9, "检查公共延伸方向"),

```

```

        (2.75, "检查顶点数"),
        (1.6, "枚举最远顶点对, 求  $SD$  与  $MS$ "),
        (0.45,  $r$ "比较  $\max_{P \in V} |P-M|$  与  $D/2$ , 输出覆盖判定")]
for y, label in steps:
    ax.add_patch(FancyBboxPatch((0.6, y - 0.36), 6.4, 0.72,
        boxstyle="round,pad=0.05", facecolor="#edf3f8", edgecolor=BLUE))
    ax.text(3.8, y, label, ha="center", va="center", fontsize=11)
for (y1, _), (y2, _) in zip(steps, steps[1:]):
    ax.annotate("", (3.8, y2 + 0.39), (3.8, y1 - 0.39),
        arrowprops={"arrowstyle": "->", "color": BLUE})
for y, label in [(5.05, "无交点: 空集"), (3.9, "存在: 无界"), (2.75, "少于 3: 退化")]:
    ax.annotate("", (7.9, y), (7.07, y), arrowprops={"arrowstyle": "->"})
    ax.text(8.05, y, label, ha="left", va="center", fontsize=10)
save(fig, "pi-algorithm-flow.png")

def wedge_diagram():
    T = np.array([0.0, 0.0])
    sensors = np.array([[-900., -250.], [700., -450.], [300., 850.]])
    bearings = [P.bearing(s, T) for s in sensors]
    vertices, _ = P.region_from_bearings(sensors, bearings)
    fig, ax = plt.subplots(figsize=(6.3, 5.7))
    ax.set(xlim=(-1200, 1200), ylim=(-900, 1100), xlabel=" $x$  / m", ylabel=" $y$  / m")
    ax.set_aspect("equal")
    for i, (s, theta, color) in enumerate(zip(sensors, bearings, [BLUE, ORANGE, "#49864e"]), 1):
        angles = np.deg2rad([theta - P.BEARING_ERR, theta + P.BEARING_ERR])
        ends = s + 2200 * np.column_stack([np.cos(angles), np.sin(angles)])
        ax.add_patch(Polygon(np.vstack([s, ends]), facecolor=color, alpha=0.15))
    for end in ends:
        ax.plot([s[0], end[0]], [s[1], end[1]], color=color, lw=0.8)
    ax.scatter(*s, color=color, s=30, zorder=4)
    ax.annotate(f" $SS_{\{i\}}$ ", s, xytext=(6, 6), textcoords="offset points", color=color)
    ax.scatter(*T, color=RED, s=18, zorder=6)
    ax.annotate(f" $TT$ ", T, xytext=(-12, 8), textcoords="offset points")
    ax.add_patch(Polygon(vertices, facecolor=RED, edgecolor=RED))
    inset = ax.inset_axes([0.57, 0.045, 0.40, 0.40])
    draw_region(inset, vertices, labels=False)
    for i, v in enumerate(vertices, 1):
        inset.annotate(f" $VV_{\{i\}}$ ", v, xytext=(3, 4), textcoords="offset points", fontsize=8)
    _, (A, B) = P.diameter(vertices)
    inset.annotate(f" $MM$ ", (A + B) / 2, xytext=(4, 4), textcoords="offset points", fontsize=9)
    inset.set_title("定位区域局部放大", fontsize=10)
    inset.tick_params(labelsize=7)
    ax.indicate_inset_zoom(inset, edgecolor="gray", alpha=0.65)
    ax.tick_params(labelsize=9)
    save(fig, "multi_sensor_wedge_diagram.png")

def main():
    setup()
    coverage()
    flowchart()
    wedge_diagram()
    print(f"问题一正文图已生成: {OUT}")

if __name__ == "__main__":
    main()

```

附录 C 问题二主要源程序

问题二的完整源程序（支撑材料 Q2/src/q2_solution.py）见下，仅保留与正式式 (11)–(18) 直接对应的部分：交会四边形顶点、四边形直径与加权期望；寻优与等值区域统计略去，完整脚本见支撑材料。

Listing 5: 问题二：期望定位区域直径与最优点、候选区域（Python 3）

```
import math
import numpy as np

X_MAX, ALPHA_DEG = 1500.0, 1.0          # 有效接收半径；示向度误差界
ALPHA = math.radians(ALPHA_DEG)         # 探测束半张角

def quad_vertices(x_t, a, b, alpha=ALPHA):
    """S1 的两条边界射线（与 x 轴成 +-alpha）与 S2 的两条边界射线
    （绕 S2->S 方向成 +-alpha）两两求交，得交会四边形 ABCD 的四个顶点。"""
    s2 = np.array([a, b])
    psi = math.atan2(-b, x_t - a)        # S 相对 S2 的方位角
    pts = []
    for t1 in (alpha, -alpha):
        u1 = np.array([math.cos(t1), math.sin(t1)])
        for t2 in (psi + alpha, psi - alpha):
            u2 = np.array([math.cos(t2), math.sin(t2)])
            den = u1[0] * u2[1] - u1[1] * u2[0]
            if abs(den) < 1e-14:           # 两射线平行：定位区域无界
                return None
            s = (s2[0] * u2[1] - s2[1] * u2[0]) / den
            pts.append(s * u1)
    return np.array(pts)

def quad_diameter(x_t, a, b, alpha=ALPHA):
    """定位区域直径 D = 四边形顶点两两距离的最大值。"""
    p = quad_vertices(x_t, a, b, alpha)
    if p is None:
        return math.inf
    d = p[:, None, :] - p[None, :, :]
    return float(np.sqrt((d ** 2).sum(-1)).max())

def dbar(a, b, n=800, alpha=ALPHA):
    """期望定位区域直径：按  $P_T(x_T|S1) = 2 x_T / x_{\max}^2$  加权求和。"""
    if abs(b) < 1e-9:                     # 两点与源点共线，区域无界
        return math.inf
    x_t = X_MAX * (np.arange(n) + 0.5) / n
    w = 2.0 * x_t / X_MAX ** 2
    d = np.array([quad_diameter(float(x), a, b, alpha) for x in x_t])
    if not np.isfinite(d).all():
        return math.inf
    return float((w * d).sum() / w.sum())

if __name__ == "__main__":
    # 自检：粗网格上的最优相对位形
    best = min(((dbar(x * 1.0, y * 1.0), x, y)
                for x in range(950, 1251, 25) for y in range(500, 901, 25)),
               key=lambda t: t[0])
    print("Dbar_min = %.2f m at (a,b) = (%.0f, %.0f) m" % best)
```

附录 D 问题三参数、补测概率与复现材料

4.1 补测的中心射线近似

设首个有信号测站为 s_0 ，示向度为 θ_0 。沿 $x(t) = s_0 + t u(\theta_0)$ 在 $5 \sim 1500$ m 上等距取 151 个样本。用极坐标面积元中的 t 为位置权重，并采用接收半径在 $[1000, 1500]$ 上均匀的建模近似。正观测要求 $\rho \geq t$ ，历史负观测 s_j 要求 $\rho < \|x(t) - s_j\|$ ，因此

$$\ell(t) = \max(1000, t), \quad H(t) = \min\{1500, \min_{j \in I^-} \|x(t) - s_j\|\}.$$

I^- 是负观测索引；为空时内层最小值取正无穷。令 $[a]_+ = \max(a, 0)$ ，分母把可能的接收半径区间长度求和，分子进一步要求能在候选站 p 接收：

$$\hat{P}_{\text{recv}}(p) = \frac{\sum_t t \mathbf{1}_{\Omega}(x(t)) [H(t) - \max\{\ell(t), \|x(t) - p\|\}]_+}{\sum_t t \mathbf{1}_{\Omega}(x(t)) [H(t) - \ell(t)]_+}. \quad (56)$$

分母为零时不施加此概率门槛，交由后续几何筛选。它是中心射线上的离散近似，不是完整角楔上的贝叶斯后验。单方位点估计另取 301 个射线样本，剔除靶区外及历史负观测 1000 m 圆盘内样本，作 t 加权平均；该操作不用于缩小保守外包。

4.2 算法参数与实验设置

表 11 问题三算法参数

参数	默认取值与含义
角楔半宽、圆外包	$(1.005 + 10^{-9})^\circ$ ，每圆 32 条切线
清除及近场半径	20 m、5 m；近场仍等待成功清除回复
field 距离系数	$\lambda = 1.5$ ，100 m 行动候选网格
sweep / tour 主圈	(1100 m, 8) / (1000 m, 8)
adaptive 公共基线	400 m，72 个候选方向
沿途逼近	接近比例 0.55，侧移上限 100 m，最多 12 步
连续证书	$h = 50$ m， $r_{\text{cert}} = 964.644660 \dots$ m
joint 覆盖候选	1050、1200、1350、1500 m 四圈，各 72 方向
贪心覆盖 / 规划候选	最多 30 次； (α, β) 取 (1, 0.3)、(1, 1)、(2, 0.3)、(2, 1)、(3, 0.3)
测站精修	删除冗余站、保留独占格移动；最多 3 轮
中心试探	至少两条方位， $r_c \leq 400$ m；失败后原地补测
可选补测	概率门槛 0.5、交会基线锐角至少 25°
清除后扫描	after_service_scan='duty'，实际终点更新
小规模精确排序	不超过 3 个路点枚举；其余 NN+2-opt
离线虚拟时钟上限	360000 s；程序墙钟耗时另记，非虚拟成本

各策略使用自身默认配置；最终两代的有效覆盖半径为证书半径。构造性自检不参与性能统计。

4.3 补充统计

随机场景的源数均匀取 10 ~ 16，源位按圆域面积均匀分布；外圈和中心组均固定 12 源，其半径分别在 1300 ~ 1800 m 和 0 ~ 260 m 内均匀取值。接收半径通常在 [1000, 1500] m 内均匀分布，最小接收组固定为 1000 m；非平滑组采用有界位置哈希误差，同地误差固定且不超过 1°。按正文场景顺序编号 $s = 0, \dots, 4$ ，第 i 组种子为 $2026091200 + 100000s + 97i$ ， i 从 0 起计。

下表时间单位为 s，行程为 km；每源列为逐组比值平均，计算列为策略运行的本地实测耗时，不含场景生成与数据分析。

表 12 全部场景的完整成本与清除结果

场景/策略	清除比例	全清	时间均值	P90	每源	行程	检测	计算
random/field	0.995	47/50	5347.3	5820.3	422.2	21.04	180.5	0.040
random/sweep	0.986	43/50	4186.4	4609.7	333.9	16.78	128.1	0.046
random/tour	1.000	50/50	4028.2	4673.7	316.9	14.41	177.1	0.148
random/adaptive	1.000	50/50	3347.9	3601.6	263.6	11.72	156.7	0.228
random/joint	1.000	50/50	3066.7	3366.7	241.1	11.17	127.4	0.707
annulus/field	0.996	19/20	5015.5	5306.7	419.7	18.68	204.3	0.038
annulus/sweep	0.975	15/20	4397.8	5029.2	377.8	16.93	159.2	0.041
annulus/tour	1.000	20/20	4376.3	4774.1	364.7	15.19	210.3	0.125
annulus/adaptive	1.000	20/20	3598.7	3945.9	299.9	12.52	172.8	0.195
annulus/joint	1.000	20/20	3215.8	3455.7	268.0	11.43	145.6	0.472
center/field	1.000	10/10	4293.7	4379.0	357.8	16.05	170.9	0.040
center/sweep	1.000	10/10	2632.3	2772.4	219.4	9.37	116.3	0.055
center/tour	1.000	10/10	2529.6	2759.1	210.8	9.23	100.3	0.110
center/adaptive	1.000	10/10	2573.5	2675.7	214.5	8.23	144.8	0.179
center/joint	1.000	10/10	2256.1	2377.0	188.0	8.01	98.8	0.885
hash/field	0.992	18/20	5451.7	5993.3	425.3	21.53	180.3	0.041
hash/sweep	0.977	17/20	4060.3	4552.1	324.3	16.24	125.0	0.050
hash/tour	1.000	20/20	4012.8	4730.6	308.7	14.73	163.1	0.157
hash/adaptive	1.000	20/20	3303.9	3587.9	256.2	11.67	150.7	0.236
hash/joint	1.000	20/20	3047.0	3261.9	236.6	11.22	121.5	0.734
worstrecv/field	1.000	20/20	4854.4	5105.9	381.0	18.22	191.9	0.035
worstrecv/sweep	0.861	7/20	4465.0	5011.3	410.8	17.16	163.7	0.038
worstrecv/tour	1.000	20/20	4438.8	4987.5	346.5	15.66	204.5	0.163
worstrecv/adaptive	1.000	20/20	3582.2	3811.7	280.9	12.14	182.1	0.215
worstrecv/joint	1.000	20/20	3320.7	3565.5	260.4	11.79	149.9	0.630

四项消融只用随机集前 20 局，完整策略均值 3076.7 s。下表增长率定义为“消融均值/完整策略均值 - 1”，区间是同一组种子重采样的 4000 次配对 bootstrap，未进行多重比较校正；结论限于这组机制比较。

表 13 单因素消融的成本变化及配对区间

消融	时间/s	增长/%	95% 区间/%	行程/km	检测	计算/s
取消清后扫描	3081.2	0.15	[-0.96, 1.32]	11.24	127.4	0.694
单方案规划	3131.3	1.78	[0.48, 3.23]	11.37	131.6	0.237
取消测站精修	3136.1	1.93	[0.17, 3.71]	11.32	134.1	0.314
取消概率门槛	3111.3	1.12	[0.67, 1.66]	11.19	134.2	0.676

表 14 消融子集的清除结果与完整时间分布

策略	清除比例	全清	时间均值/s	P90/s	每源耗时/s
完整 joint	1.000	20/20	3076.7	3424.0	240.3
取消清后扫描	1.000	20/20	3081.2	3333.5	240.4
单方案规划	1.000	20/20	3131.3	3444.7	244.3
取消测站精修	1.000	20/20	3136.1	3415.2	245.2
取消概率门槛	1.000	20/20	3111.3	3461.8	242.9

4.4 文件清单及复现命令

实验数据与逐动作记录保存在 B_locator/Q3/out/paper_q3/，附有代码、期望场及场景参数的校验信息。图表从同一组数据生成，完整复现命令如下。

Listing 6: 从仓库根目录复现问题三

```
python B_locator/Q3/scripts/run_p3_paper.py
python B_locator/Q3/scripts/make_paper_tables.py
python B_locator/Q3/scripts/make_paper_figures.py
python B_locator/Q3/scripts/make_paper_trajectory_atlas.py
python B_locator/Q3/scripts/make_paper_source.py
python B_locator/Q3/src/p3_selftest.py --out /tmp/q3-selftest.txt
python B_locator/Q3/src/p3_adaptive_checks.py
python B_locator/Q3/src/p3_joint_checks.py
bash B_locator/paper/build.sh
```

实验与绘图分别使用 NumPy、Matplotlib，论文以 XeLaTeX 或 Tectonic 编译。

环境与验证说明见 Q3/PAPER_REPRODUCE.md；全部 120 组五策略轨迹见输出目录中的 supplementary_trajectories.pdf。

附录 E 问题三完整源程序

以下列出五代策略及实验所需的完整模块，可执行源文件与校验值见支撑材料。性能比较以正文的统一实验为准。

5.1 src/p1_intersection.py

```
"""B 题问题 1: 示向度交会定位区域、区域直径与直径圆覆盖判定。

模型的严格表述
-----
检测点 :math:`S_i=(x_i,y_i)`，该点测得示向度 :math:`\theta_i`（度，x 轴正向逆时针，[0,360)），
示向度误差有界 :math:`|e_i|\leq\epsilon`（题目附录 2 图 2: 实线与相邻虚线夹角均为 1°）。
真源必在


$$R = \bigcap_i W_i, \quad W_i = \{ X : |\text{ang}(X - S_i) - \theta_i| \leq \epsilon \}$$


 $W_i$  是以  $S_i$  为顶点、张角  $2\epsilon$  的**角楔**，等价于两个半平面之交：


$$W_i = \{ X : a^-_i \cdot X + b^-_i \leq 0 \} \cap \{ X : a^+_i \cdot X + b^+_i \leq 0 \}$$


因此  $R$  是  $2k$  个半平面的交 —— **凸多边形**，顶点数  $\leq 2k$ 。

算法要点（含两处易错点的正确处理）
-----
1. **可行性 / 有界性**（先判空、再判无界，顺序不能反）
   * 空集：两两示向度可以相差  $> 2\epsilon$ ，但两个“错开”的角楔也可能没有公共点；
   * 无界：存在方向  $d$  使  $|\text{ang}(d) - \theta_i| \leq \epsilon$  对所有  $i$  成立
      $\iff$  所有示向度**两两夹角  $\leq 2\epsilon$ **；此时直径  $= +\infty$ ，不能报有限值。
2. **区域顶点**：凸集的顶点必是两条起作用约束边界直线的交点，故枚举  $2k$  条边界直线的
 $O(k^2)$  个交点、用**半平面判据**  $a \cdot X + b \leq 0$  筛选即可（精确）。
   注意：**检测点本身也可能是顶点**（当它落在其它所有楔内时）。旧实现在这里用
“候选点到检测点的距离  $< 1e-9$  则丢弃”处理角度奇异，会把这种顶点误删 —— 见
`region_by_vertices` 的半平面判据。
3. 提供两套**相互独立**的区域算法，供自检交叉验证：
   * `region_by_vertices`：顶点枚举 + 凸包（主算法，精确）；
   * `region_by_clipping`：Sutherland-Hodgman 逐个半平面裁剪大框（参考实现，可判无界）。
4. **直径**：顶点数  $\leq \text{DIAM\_BRUTE\_LIMIT}$  时暴力枚举；否则旋转卡壳，两者在自检中互相校验。
5. **覆盖判定**：以直径  $AB$  为直径的圆  $C$ ，
 $V \in C \iff \angle AVB \geq 90^\circ \text{ (Thales)} \iff |V - c| \leq D/2$  ( $c$  为  $AB$  中点)。
   一般**不能覆盖**；正确做法是用**最小包围圆**（半径  $r \in [D/2, D/\sqrt{3}]$ ，Jung 定理）。
"""

from __future__ import annotations
import csv
import json
import math
import os
import sys

import numpy as np

BEARING_ERR = 1.0      # 示向度误差界（度）
ANG_TOL = 1e-9         # 角度判据容差（度）
DIST_TOL = 1e-6        # 半平面判据容差（米）
CLIP = 1e7             # 裁剪参考实现的外框半边长（米）
DIAM_BRUTE_LIMIT = 128 # 顶点数不超过该值时直径用暴力枚举

# -----
# 基本几何
# -----
```

```

def bearing(p, q):
    """p 处指向 q 的方位角 (度, [0,360)) """
    return math.degrees(math.atan2(q[1] - p[1], q[0] - p[0])) % 360.0

def ang_diff(a, b):
    """a-b 归一化到 (-180,180]"""
    return (a - b + 180.0) % 360.0 - 180.0

def wedge_halfplanes(p, theta, err=BEARING_ERR):
    """角楔 W = {X : |ang(X-p) - theta| <= err} 的两个半平面, 形式 a * X + b <= 0。

    err < 90°, 故 W 是两条边界直线所夹的尖楔。
    """
    p = np.asarray(p, float)
    tm = math.radians((theta - err) % 360.0)
    tp = math.radians((theta + err) % 360.0)
    dm = np.array([math.cos(tm), math.sin(tm)])
    dp = np.array([math.cos(tp), math.sin(tp)])
    # cross(dm, X-p) >= 0  <=>  a1 * X + b1 <= 0
    a1 = np.array([dm[1], -dm[0]])
    b1 = float(-dm[1] * p[0] + dm[0] * p[1])
    # cross(dp, X-p) <= 0  <=>  a2 * X + b2 <= 0
    a2 = np.array([-dp[1], dp[0]])
    b2 = float(dp[1] * p[0] - dp[0] * p[1])
    return [(a1, b1), (a2, b2)]

def region_is_unbounded(svds, err=BEARING_ERR):
    """区域无界判据: 存在方向 d 与所有示向度夹角 <= err。

    返回一个这样的方向 (度), 有界时返回 None。
    仅在区域非空时有意义 (先判空、再判无界)。
    """
    for th in svds:
        for cand in ((th - err) % 360.0, (th + err) % 360.0):
            if all(abs(ang_diff(cand, t)) <= err + ANG_TOL for t in svds):
                return cand
    return None

def _line_intersection(P, u, Q, v):
    """直线 P + t * u 与 Q + s * v 的交点, 平行时返回 None"""
    det = u[0] * (-v[1]) - (-v[0]) * u[1]
    if abs(det) < 1e-14:
        return None
    r = Q - P
    t = (r[0] * (-v[1]) - (-v[0]) * r[1]) / det
    return P + t * u

def convex_hull(P):
    """Andrew monotone chain, 逆时针, 去掉共线点"""
    def half(pts):
        st = []
        for p in pts:
            while len(st) >= 2 and _cross2(st[-1] - st[-2], p - st[-2]) <= 0:
                st.pop()
            st.append(p)
        return st
    P = P[np.lexsort((P[:, 1], P[:, 0]))]

```

```

    lo = half(P)
    hi = half(P[:-1])
    return np.array(lo[:-1] + hi[:-1])

def _cross2(u, v):
    return float(u[0] * v[1] - u[1] * v[0])

def _dedup_hull(cand):
    """候选顶点去重 + 凸包, 返回 (P, "empty"|"degenerate"|"polygon")"""
    if len(cand) == 0:
        return np.zeros((0, 2)), "empty"
    P = np.array(cand, float)
    P = P[np.lexsort((P[:, 1], P[:, 0]))]
    ded = [P[0]]
    for v in P[1:]:
        if np.linalg.norm(v - ded[-1]) > 1e-7:
            ded.append(v)
    P = np.array(ded)
    if len(P) < 3:
        return P, "degenerate"
    H = convex_hull(P)
    if len(H) < 3:
        return H, "degenerate"
    return H, "polygon"

# -----
# 区域算法 1: 顶点枚举 + 凸包 (主算法, 精确)
# -----
def region_by_vertices(pts, svds, err=BEARING_ERR, tol=DIST_TOL):
    """ $R = \cap W_i$  的顶点集合。

     $R$  是凸集, 其任一极点必是两条起作用约束边界直线的交点; 反之, 落在所有半平面内的
    边界直线交点必为极点 (若  $X = (Y+Z)/2$  且两个非平行约束在  $X$  处取等, 则  $Y, Z$  也
    必须取等, 故  $Y = Z = X$ )。因此枚举 + 筛选是精确的。
    """
    if len(pts) == 0:
        return np.zeros((0, 2)), "empty"
    H, lines = [], []
    for p, th in zip(pts, svds):
        H += wedge_halfplanes(p, th, err)
        for s in (-1.0, 1.0):
            a = math.radians((th + s * err) % 360.0)
            lines.append((np.asarray(p, float), np.array([math.cos(a), math.sin(a)])))
    cand = []
    n = len(lines)
    for i in range(n):
        for j in range(i + 1, n):
            X = _line_intersection(lines[i][0], lines[i][1], lines[j][0], lines[j][1])
            if X is None or not np.all(np.isfinite(X)):
                continue
            # 半平面判据: 在楔顶点处也良定义 (无角度奇异), 故检测点可作顶点保留
            if all(float(a @ X + b) <= tol for (a, b) in H):
                cand.append(X)
    return _dedup_hull(cand)

# -----
# 区域算法 2: 半平面裁剪 (独立参考实现, 可直接判无界)
# -----

```

```

def clip_halfplane(poly, a, b, tol=DIST_TOL):
    """Sutherland-Hodgman: 保留  $\{X : a \cdot X + b \leq 0\}$  的部分"""
    if not poly:
        return []
    out = []
    n = len(poly)
    for i in range(n):
        P = poly[i]
        Q = poly[(i + 1) % n]
        fp = float(a @ P + b)
        fq = float(a @ Q + b)
        if fp <= tol:
            out.append(P)
        if (fp < -tol and fq > tol) or (fp > tol and fq < -tol):
            t = fp / (fp - fq)
            out.append(P + t * (Q - P))
    ded = []
    for v in out:
        if not ded or np.linalg.norm(v - ded[-1]) > 1e-9:
            ded.append(v)
    if len(ded) > 1 and np.linalg.norm(ded[0] - ded[-1]) < 1e-9:
        ded.pop()
    return ded

def region_by_clipping(pts, svds, err=BEARING_ERR, box=CLIP, tol=DIST_TOL):
    """从大框出发逐个半平面裁剪。

    返回 (P, status): status  $\in \{\text{"empty"}, \text{"degenerate"}, \text{"polygon"}, \text{"unbounded"}\}$ ;
    多边形触到外框 ( $| \text{坐标} | > \text{box}/2$ ) 即判定区域无界, P 是被截断的多边形。
    """
    poly = [np.array([-box, -box], float), np.array([box, -box], float),
            np.array([box, box], float), np.array([-box, box], float)]
    for p, th in zip(pts, svds):
        for (a, b) in wedge_halfplanes(p, th, err):
            poly = clip_halfplane(poly, a, b, tol)
            if not poly:
                return np.zeros((0, 2)), "empty"
    P = np.array(poly, float)
    if len(P) < 3:
        return P, "degenerate"
    if float(np.max(np.abs(P))) > box * 0.5:
        return P, "unbounded"
    H = convex_hull(P)
    return (H if len(H) >= 3 else P), "polygon"

def region_from_bearings(pts, svds, err=BEARING_ERR):
    """主入口 (兼容旧接口): 返回 (顶点数组, status)。

    status  $\in \{\text{"empty"}, \text{"unbounded"}, \text{"degenerate"}, \text{"bounded"}\}$ 。
    """
    P, st = region_by_vertices(pts, svds, err)
    if st == "empty":
        return P, "empty"
    rec = region_is_unbounded(list(svds), err) if len(svds) else None
    if rec is not None:
        return P, "unbounded"
    if st == "degenerate":
        return P, "degenerate"
    return P, "bounded"

```

```

# -----
# 直径与最小包围圆
# -----

def _diameter_brute(P):
    n = len(P)
    best, pair = -1.0, (P[0], P[1])
    for i in range(n - 1):
        d = np.linalg.norm(P[i + 1:] - P[i], axis=1)
        k = int(np.argmax(d))
        if float(d[k]) > best:
            best, pair = float(d[k]), (P[i], P[i + 1 + k])
    return best, pair

def rotating_calipers(P):
    """凸多边形直径（旋转卡壳，O(n)）。P 需为逆时针凸序（允许共线点）。"""
    n = len(P)
    if n < 3:
        return diameter(P, brute_limit=2)
    k = 1
    best, pair = -1.0, (P[0], P[1])
    for i in range(n):
        i2 = (i + 1) % n
        e = P[i2] - P[i]
        while True:
            k2 = (k + 1) % n
            if abs(_cross2(e, P[k2] - P[i])) > abs(_cross2(e, P[k] - P[i])):
                k = k2
            else:
                break
        for j in (k, (k + 1) % n):
            d = float(np.linalg.norm(P[i] - P[j]))
            if d > best:
                best, pair = d, (P[i], P[j])
            d = float(np.linalg.norm(P[i2] - P[j]))
            if d > best:
                best, pair = d, (P[i2], P[j])
    return best, pair

def diameter(P, brute_limit=DIAM_BRUTE_LIMIT):
    """凸多边形直径，返回 (D, (A, B))"""
    n = len(P)
    if n == 0:
        return 0.0, None
    if n == 1:
        return 0.0, (P[0], P[0])
    if n == 2:
        return float(np.linalg.norm(P[0] - P[1])), (P[0], P[1])
    if n <= brute_limit:
        return _diameter_brute(P)
    return rotating_calipers(P)

def min_enclosing_circle(P):
    """最小包围圆 (center, radius)。顶点数很少，直接枚举 1/2/3 点候选圆。

    理论:  $r^* \in [D/2, D/\sqrt{3}]$  (Jung 定理, 平面)，等号右端仅当区域为等边三角形时取到。
    """
    P = np.asarray(P, float)
    n = len(P)

```

```

if n == 0:
    return None, 0.0
if n == 1:
    return P[0].copy(), 0.0
cands = [(P[i].copy(), 0.0) for i in range(n)]
for i in range(n):
    for j in range(i + 1, n):
        c = 0.5 * (P[i] + P[j])
        cands.append((c, float(np.linalg.norm(P[i] - c))))
for i in range(n):
    for j in range(i + 1, n):
        for k in range(j + 1, n):
            A, B, C = P[i], P[j], P[k]
            d = 2.0 * (A[0] * (B[1] - C[1]) + B[0] * (C[1] - A[1])
                      + C[0] * (A[1] - B[1]))
            if abs(d) < 1e-12:
                continue
            aa, bb, cc = float(A @ A), float(B @ B), float(C @ C)
            ux = (aa * (B[1] - C[1]) + bb * (C[1] - A[1]) + cc * (A[1] - B[1])) / d
            uy = (aa * (C[0] - B[0]) + bb * (A[0] - C[0]) + cc * (B[0] - A[0])) / d
            c = np.array([ux, uy])
            cands.append((c, float(np.linalg.norm(A - c))))
best = None
for c, r in cands:
    if all(float(np.linalg.norm(P[i] - c)) <= r + 1e-9 for i in range(n)):
        if best is None or r < best[1]:
            best = (c, r)
return best

def region_area(P):
    n = len(P)
    if n < 3:
        return 0.0
    return abs(0.5 * float(np.sum(P[:, 0] * np.roll(P[:, 1], -1)
                                   - np.roll(P[:, 0], -1) * P[:, 1])))

# -----
# 汇总
# -----

def analyse(pts, svds, err=BEARING_ERR):
    """给定检测点与示向度，输出定位区域的全套几何量。"""
    P, status = region_from_bearings(pts, svds, err)
    base = {"n_bearings": len(pts), "status": status}
    if status == "empty":
        base["reason"] = ("所有示向度锥无公共交集（数据不一致：两两夹角可以 > 2°， "
                          "但错开的两个楔也可能不相交）")
        return base
    if status == "unbounded":
        rec = region_is_unbounded(list(svds), err)
        base["reason"] = ("示向度锥近似平行（两两夹角 <= 2°），定位区域无界， "
                          "直径 = +inf；必须换检测点重测")
        base["recession_direction_deg"] = None if rec is None else round(float(rec), 4)
        base["n_vertices"] = int(len(P))
        base["vertices"] = [[round(float(v[0]), 4), round(float(v[1]), 4)] for v in P]
        return base
    if status == "degenerate" or len(P) < 3:
        base["status"] = "degenerate"
        base["reason"] = "公共交集退化为线段或点"
        base["vertices"] = [[round(float(v[0]), 4), round(float(v[1]), 4)] for v in P]
        return base

```



```

D, (A, B) = diameter(P)
c = 0.5 * (A + B)
maxr = float(max(np.linalg.norm(v - c) for v in P))
circle = min_enclosing_circle(P)
if circle is None:
    # Very thin polygons far from the origin can lose circumcircle
    # precision. Translation preserves the geometric problem.
    origin = np.mean(P, axis=0)
    local = min_enclosing_circle(P - origin)
    mc = origin + local[0] if local is not None else origin
    mr = float(np.max(np.linalg.norm(P - mc, axis=1)))
else:
    mc, mr = circle
base.update({
    "n_vertices": int(len(P)),
    "vertices": [[round(float(v[0]), 4), round(float(v[1]), 4)] for v in P],
    "diameter_m": round(D, 4),
    "diameter_endpoints": [[round(float(A[0]), 4), round(float(A[1]), 4)],
                           [round(float(B[0]), 4), round(float(B[1]), 4)]],
    "circle_center": [round(float(c[0]), 4), round(float(c[1]), 4)],
    "circle_radius_m": round(D / 2.0, 4),
    "max_vertex_dist_m": round(maxr, 4),
    "diameter_circle_covers": bool(maxr <= D / 2.0 + 1e-9),
    "min_enclosing_center": [round(float(mc[0]), 4), round(float(mc[1]), 4)],
    "min_enclosing_radius_m": round(float(mr), 4),
    "min_circle_over_half_diameter": round(float(mr) / (D / 2.0), 6),
    "area_m2": round(region_area(P), 4),
})
return base

def run_case_csv(path, truth_path=None, err=BEARING_ERR):
    with open(path, encoding="utf-8") as f:
        rows = list(csv.DictReader(f))
    pts = [(float(r["x_m"]), float(r["y_m"])) for r in rows]
    svds = [float(r["svd_deg"]) for r in rows]
    res = analyse(pts, svds, err)
    res["n_bearings"] = len(rows)
    if truth_path and os.path.exists(truth_path):
        with open(truth_path, encoding="utf-8") as f:
            truth = json.load(f)
            G = truth["source_xy"]
            res["truth_xy"] = G
            if "circle_center" in res:
                res["center_error_vs_truth_m"] = round(math.dist(res["circle_center"], G), 3)
            if "min_enclosing_center" in res:
                res["mincircle_error_vs_truth_m"] = round(
                    math.dist(res["min_enclosing_center"], G), 3)
            # 真值是否满足全部约束（等价于在半平面内）
            H = []
            for p, th in zip(pts, svds):
                H += wedge_halfplanes(p, th, err)
            res["truth_inside_region"] = bool(all(a @ np.asarray(G, float) + b <= 1e-6
                                                for (a, b) in H))
    return res

def main():
    d = sys.argv[1] if len(sys.argv) > 1 else os.path.join(
        os.path.dirname(os.path.abspath(__file__)), "..", "data")
    out = []

```

```

for fn in sorted(os.listdir(d)):
    if not (fn.startswith("p1_case") and fn.endswith(".csv")):
        continue
    res = run_case_csv(os.path.join(d, fn),
                       os.path.join(d, fn.replace(".csv", ".truth.json")))
    res["case"] = fn
    out.append(res)
jl = os.path.join(d, "p1_results.jsonl")
with open(jl, "w", encoding="utf-8") as f:
    for r in out:
        f.write(json.dumps(r, ensure_ascii=False) + "\n")

ok = [r for r in out if r["status"] == "bounded"]
unlimited = [r for r in out if r["status"] == "unbounded"]
summary = {
    "n_cases": len(out),
    "n_bounded": len(ok),
    "n_unbounded": len(unlimited),
    "n_degenerate": sum(1 for r in out if r["status"] == "degenerate"),
    "n_empty": sum(1 for r in out if r["status"] == "empty"),
    "diameter_m_range": [min((r["diameter_m"] for r in ok), default=None),
                         max((r["diameter_m"] for r in ok), default=None)],
    "n_diameter_circle_misses": sum(1 for r in ok if not r["diameter_circle_covers"]),
    "min_circle_over_half_diameter_max": max(
        (r["min_circle_over_half_diameter"] for r in ok), default=None),
    "n_truth_inside_region": sum(1 for r in ok if r.get("truth_inside_region")),
}
with open(os.path.join(d, "p1_summary.json"), "w", encoding="utf-8") as f:
    json.dump(summary, f, ensure_ascii=False, indent=2)
for r in out:
    print("%-16s %-10s D=%-9s cover=%-5s n=%-2s r*/D*2=%-8s center_err=%s"
          % (r["case"], r["status"], r.get("diameter_m", "-"),
             r.get("diameter_circle_covers", "-"), r.get("n_bearings", "-"),
             r.get("min_circle_over_half_diameter", "-"),
             r.get("center_error_vs_truth_m", "-")))
print(json.dumps(summary, ensure_ascii=False))

if __name__ == "__main__":
    main()

```

5.2 src/p2_grid_expectation.py

"""B 题问题 2 (模拟): 固定首次探测扇形后, 第二检测点网格上的"交会区域直径期望"热力图。

问题设定 (按需求方给定的简化模型)

1. **固定当前探测得到的扇形区域**: 探测器 (机器狗) 位于原点 $S_1 = (0, 0)$, 探测方向为 x 轴正方向, 示向度误差界 $\epsilon = 1 \text{ deg}$ 。于是第一次探测给出的信息是角楔

$$W_1 = \{ X : |\text{ang}(X - S_1) - \theta_1| \leq \epsilon \}$$

θ_1 取自当前探测数据 (默认 0 deg , 即"探测方向为 x 轴正方向")。

2. **干扰源均匀分布在扇形内, 近似认为都在 x 轴上**: 源位 $G = (t, 0)$, 权重由"扇形内面积均匀"退化得到: 面积元 $dA = r dr d\theta \rightarrow w(t) \propto t$ (---weight area, 默认); 另提供等权 (---weight length) 作对照。
3. **网格**: 把每个网格点当作第二个检测点 S_2 。默认网格框在右侧多留 $0.2R$ ($x_{\text{range}} = (-1800, 2160)$), 因为最优点在右半区, 这样同样点数下右半区更密。注意**可行域仍是靶区域** $R \leq 1800 \text{ m}$, $x > 1800$ 的部分是空框。
4. **交会区域与直径**: 给定源位 G , 两个检测点各得一条带 $\pm\epsilon$ 的示向度锥,

两者之交是凸四边形 $R = W1 \cap W2$ ，其**直径** $D =$ 区域内任意两点距离的最大值。

5. **期望**： $E[D](S2) = \sum_i w_i D(S2, t_i)$ ，只在"可检测且有界"的样本上取平均，同时记录"可检测概率"，避免把不可用点伪装成好点。

6. **热力图**： $E[D](S2)$ 作为网格点 (x, y) 的函数，对数色标。

两套口径（主 / 辅）

* `measured`（主）：两个楔都用**测得值** $\pm 1^\circ$ ，即 $R = W1(\theta_1) \cap W2(\theta_2)$ ，期望只对源位 `t` 取 —— 完全对应"固定当前探测数据"的字面含义。

* `error_avg`（辅）：同一个区域，但期望**同时对 $\pm 1^\circ$ 测量误差取**，即 `E_{t, e1, e2}[D]` —— 更贴近"定位效果"的真实统计含义。

为什么网格必须是真正二维的（重要结论）

若把 `S2` 放在 x 轴上（ $y = 0$ ），则两条示向度近似平行，夹角 $|d\theta| \leq 2\epsilon$ ，交会区域**无界**、直径为 $\propto$ （这正是 `p1_intersection.region_by_vertices` 会报 `unbounded` 的退化几何，等价于"沿示向度方向逼近"）。所以 $y = 0$ 必须在网格上被剔除。更进一步， $|y|$ 很小但未退化的点，交会区域会被拉成极长的细条（直径可达 $10^4 - 10^5$ m），数学上没错但工程上无用，程序用 `usable_threshold = 10 \times` 中位数 标注并掩码。

可检测性门控

第二个检测点要能收到该频道信号，必须 $|S2 \cdot G| \leq$ 有效接收半径上界 1500 m 。不加这个门控会选出几何有效但物理收不到信号的假最优解，所以它是模型的一部分（默认开启，`--no-reach` 可关闭对照）。

正确性验证

直径由三条**互相独立**的实现给出，随机算例最大相对差 $< 1e-13$ ：
`p1_intersection.region_by_vertices`（半平面枚举）、`independent_diameter`（世界坐标直接解边界交点）、`fast_diameters_vec`（角点枚举 + 楔定义筛选）。
`--selfcheck` 另含蒙特卡洛与退化算例（T0 - T10）。

用法

```
python src/p2_grid_expectation.py --selfcheck
python src/p2_grid_expectation.py --step 20 --n-t 500 --out out/p2_grid
python src/p2_grid_expectation.py --theta1 30 --out out/p2_grid_t30
"""
from __future__ import annotations

import argparse
import csv
import json
import math
import os
import sys
import time

import numpy as np

_HERE = os.path.dirname(os.path.abspath(__file__))
if _HERE not in sys.path:
    sys.path.insert(0, _HERE)

from p1_intersection import BEARING_ERR, ang_diff, bearing, region_by_vertices # noqa: E402

# -----
# 物理常数（题目附录 1/2）
# -----
EPS_DEG = BEARING_ERR          # 示向度误差界 1°
```

```

R_ARENA = 1800.0          # 目标区域半径 (m)
R_RECV_MIN = 1000.0       # 有效接收半径下界 (m, 接口不返回)
R_RECV_MAX = 1500.0       # 有效接收半径上界 (m, 取信号的必要条件)
NEAR_R = 5.0             # 近场阈值 (m): 更近则没有示向度

# 默认计算参数
DEF_STEP = 20.0
DEF_X_RANGE = (-R_ARENA, 1.2 * R_ARENA)    # 右侧多留 0.2R (框内留白, 便于看全)
DEF_Y_RANGE = (-R_ARENA, R_ARENA)
DEF_N_T = 500          # x 轴上源位采样数
DEF_N_E = 400          # 误差 MC 次数 (error_avg 口径)
DEFAULT_SEED = 20260913

# -----
# 基本几何
# -----

def point_in_wedge(X, p, theta, err=EPS_DEG, tol=1e-9):
    """X 是否在角楔内 (与楔顶点重合视为在楔内) """
    X = np.asarray(X, float)
    p = np.asarray(p, float)
    if float(np.linalg.norm(X - p)) < tol:
        return True
    return abs(ang_diff(bearing(p, X), theta)) <= err + 1e-12

def _unbounded_bearings(theta1, theta2, err=EPS_DEG):
    """两楔交无界 <=> 存在方向与两条示向度夹角都 <= err。

    对两个示向度, 等价于 ``min(|dth|, 180-|dth|) <= 2*err``。
    """
    d = abs(ang_diff(theta2, theta1))
    return min(d, 180.0 - d) <= 2.0 * err + 1e-12

def _corner_points(Pw, thP, Qw, thQ, eps):
    """两个角楔交的 6 个候选角点, **全部在世界坐标下求解**。

    每个角楔 ``|ang(X - P) - theta| <= eps`` 的两条边界线方向为 ``theta +/- eps``,
    交点由四线两两相交得到 (6 个候选); 顶点判定直接用楔的定义
    (候选点 X 是顶点 <=> 对每个楔都有 ``|ang(X-P) - theta| <= eps`` 或 X 恰在顶点 P)。
    这个判据与坐标系定向、2x2 解法的行列式符号都无关, 因此最稳健。

    返回 ``(corner_world, valid)`, 形状 ``(..., 6, 2)`` 与 ``(..., 6)``。
    """
    Pw = np.asarray(Pw, float)
    Qw = np.asarray(Qw, float)
    thP = np.asarray(thP, float)
    thQ = np.asarray(thQ, float)

    def two_lines(P, th):
        out = []
        for s in (-eps, eps):
            a = np.radians(th + s)
            n = np.stack([np.sin(a), -np.cos(a)], axis=-1)
            out.append((n, np.einsum("...i,...i->...", n, P)))
        return out

    lines = two_lines(Pw, thP) + two_lines(Qw, thQ)

    def in_wedge(X, P, th):
        d = X - P

```

```

        dist = np.hypot(d[... , 0], d[... , 1])
        ang = np.degrees(np.arctan2(d[... , 1], d[... , 0]))
        diff = (ang - th + 180.0) % 360.0 - 180.0
        return (np.abs(diff) <= eps + 1e-9) | (dist <= 1e-9)

corners, valid = [], []
for ii in range(4):
    for jj in range(ii + 1, 4):
        (n1, c1), (n2, c2) = lines[ii], lines[jj]
        det = n1[... , 0] * n2[... , 1] - n1[... , 1] * n2[... , 0]
        safe = np.where(np.abs(det) < 1e-15, 1e-15, det)
        X = np.stack([(c1 * n2[... , 1] - c2 * n1[... , 1]) / safe,
                      (n1[... , 0] * c2 - n2[... , 0] * c1) / safe], axis=-1)
        ok = in_wedge(X, Pw, thP) & in_wedge(X, Qw, thQ)
        corners.append(X)
        valid.append(ok)
return np.stack(corners, axis=-2), np.stack(valid, axis=-1)

def _diam_from_corners(cw, v, shp):
    """从候选角点与其可用性掩码求凸四边形直径（世界坐标下，距离即真实距离）"""
    nP = cw.shape[-2]
    best = np.zeros(shp)
    for ii in range(nP):
        for jj in range(ii + 1, nP):
            d2 = ((cw[... , ii, 0] - cw[... , jj, 0]) ** 2
                  + (cw[... , ii, 1] - cw[... , jj, 1]) ** 2)
            best = np.where(v[... , ii] & v[... , jj], np.maximum(best, d2), best)
    return best, v.sum(axis=-1)

def independent_diameter(t, S2, eps=EPS_DEG):
    """交会区域直径的**独立参考实现**（标量、世界坐标、直接解边界交点）。

    与 ``_corner_points`` 走的是不同代码路径（这里用射线参数式解交点）。
    无界（``min(|dth|, 180-|dth|) <= 2*eps``）返回 ``inf``。
    """
    S2 = np.asarray(S2, float)
    t = float(t)
    th1 = 0.0 # S1 -> G
    th2 = math.degrees(math.atan2(-S2[1], t - S2[0])) % 360.0 # S2 -> G
    if _unbounded_bearings(th1, th2, eps):
        return math.inf

    lines = []
    for apex, th in (((0.0, 0.0), th1), (tuple(S2), th2)):
        for s in (-eps, eps):
            a = math.radians(th + s)
            lines.append((apex, (math.cos(a), math.sin(a))))

    def in_wedge(X, apex, th):
        dx, dy = X[0] - apex[0], X[1] - apex[1]
        if math.hypot(dx, dy) < 1e-9:
            return True
        ad = math.degrees(math.atan2(dy, dx))
        return abs((ad - th + 180.0) % 360.0 - 180.0) <= eps + 1e-9

    verts = []
    for i in range(4):
        for j2 in range(i + 1, 4):
            (P1, u), (P2, v) = lines[i], lines[j2]
            det = u[0] * (-v[1]) - (-v[0]) * u[1]

```

```

        if abs(det) < 1e-14:
            continue
        rv = (P2[0] - P1[0], P2[1] - P1[1])
        s = (rv[0] * (-v[1]) - (-v[0]) * rv[1]) / det
        X = (P1[0] + s * u[0], P1[1] + s * u[1])
        if in_wedge(X, (0.0, 0.0), th1) and in_wedge(X, tuple(S2), th2):
            verts.append(X)
    if len(verts) < 2:
        return math.inf
    return max(math.dist(a, b) for i, a in enumerate(verts) for b in verts[i + 1:])

def fast_diameters_vec(S2s, ts, eps=EPS_DEG):
    """批量快路径: S2s (m,2) × ts (n,) → D (m,n), status (m,n)。

    **源必须固定在 x 轴正半轴 (S1 在原点) **, 这是本题设定。

    status 编码: 0 = 有界多边形, 1 = 空, 2 = 退化, 3 = 无界。
    """
    S2s = np.asarray(S2s, float)
    ts = np.asarray(ts, float).ravel()
    m, n = S2s.shape[0], ts.size
    D = np.full((m, n), np.nan)
    status = np.ones((m, n), dtype=np.int8)
    if m == 0 or n == 0:
        return D, status
    r = np.hypot(S2s[:, 0], S2s[:, 1])
    ok_row = r >= 1e-9
    status[~ok_row, :] = 2
    th2 = np.degrees(np.arctan2(-S2s[:, 1][:, None],
                                ts[None, :] - S2s[:, 0][:, None])) % 360.0
    dth = np.abs((th2 - 0.0 + 180.0) % 360.0 - 180.0)
    sep = np.minimum(dth, 180.0 - dth)
    unb = (sep <= 2.0 * eps + 1e-12) | (~ok_row[:, None])
    status[unb] = np.where(np.broadcast_to(ok_row[:, None], unb.shape), 3, 2)[unb]
    ok = ~unb
    if not np.any(ok):
        return D, status

    S1w = np.zeros((m, n, 2))
    S2w = np.broadcast_to(S2s[:, None, :], (m, n, 2))
    th1g = np.zeros((m, n))
    corner, valid = _corner_points(S1w, th1g, S2w, th2, eps) # 世界坐标
    cw = corner.reshape(m, n, corner.shape[-2], 2)
    v = valid.reshape(m, n, valid.shape[-1])
    best, n_valid = _diam_from_corners(cw, v, (m, n))
    status = np.where(ok & (n_valid < 2), 1, status).astype(np.int8)
    finite = ok & (n_valid >= 2) & np.isfinite(best)
    D = np.where(finite, np.sqrt(np.maximum(best, 0.0)), np.nan)
    status = np.where(finite, 0, status).astype(np.int8)
    return D, status

def fast_diameters(S2, ts, eps=EPS_DEG):
    """`fast_diameters_vec` 的半点版"""
    D, st = fast_diameters_vec(np.asarray(S2, float)[None, :],
                                np.asarray(ts, float).ravel(), eps)
    return D[0], st[0]

def _fast_pair_grid(S1, th1, S2, th2, eps):
    """通用批量快路径: `(...,)` 形状的 S1/th1/S2/th2 → D 与 status。

```

与 ``fast\_diameters\_vec`` 同一套做法，但两个角楔都任意（不要求 S1 在原点、源在 x 轴上）。用于 error_avg 口径。

```

"""
P1 = np.asarray(S1, float)
P2 = np.asarray(S2, float)
th1 = np.asarray(th1, float)
th2 = np.asarray(th2, float)
if th2.ndim > th1.ndim:
    th1 = th1.reshape(th1.shape + (1,) * (th2.ndim - th1.ndim))
shp = np.broadcast_shapes(th1.shape, th2.shape)
th1 = np.broadcast_to(th1, shp)
th2 = np.broadcast_to(th2, shp)
P1 = np.broadcast_to(np.asarray(P1, float)[..., None, :], shp + (2,))
P2 = np.broadcast_to(np.asarray(P2, float)[..., None, :], shp + (2,))

rho = np.linalg.norm(P1 - P2, axis=-1)
sep = np.abs((th2 - th1 + 180.0) % 360.0 - 180.0)
sep = np.minimum(sep, 180.0 - sep)
D = np.full(shp, np.nan)
status = np.ones(shp, dtype=np.int8)
status[sep <= 2.0 * eps + 1e-12] = 3
status[rho < 1e-9] = 2
ok = ~((sep <= 2.0 * eps + 1e-12) | (rho < 1e-9))
if not np.any(ok):
    return D, status
corner, valid = _corner_points(P1, th1, P2, th2, eps)
cw = corner.reshape(shp + (corner.shape[-2], 2))
v = valid.reshape(shp + (valid.shape[-1],))
best, n_valid = _diam_from_corners(cw, v, shp)
status = np.where(ok & (n_valid < 2), 1, status).astype(np.int8)
finite = ok & (n_valid >= 2) & np.isfinite(best)
D = np.where(finite, np.sqrt(np.maximum(best, 0.0)), np.nan)
status = np.where(finite, 0, status).astype(np.int8)
return D, status

# -----
# 源位采样与权重
# -----
def sector_ray_exit(S1, a_deg, R=R_ARENA):
    """从 S1 沿方位角 a 到半径 R 圆域边界的距离"""
    S1 = np.asarray(S1, float)
    u = np.array([math.cos(math.radians(a_deg)), math.sin(math.radians(a_deg))])
    b = float(S1 @ u)
    c = float(S1 @ S1) - R * R
    disc = b * b - c
    return 0.0 if disc <= 0 else float(-b + math.sqrt(disc))

def source_samples(S1, theta1, err=EPS_DEG, n_t=DEF_N_T, r_arena=R_ARENA,
                  r_recv_max=R_RECV_MAX, near=NEAR_R):
    """x 轴上的源位采样（面积均匀权重）。

    来源：干扰源在扇形内面积均匀 -> ``dA = r dr dtheta``，
    对角向积分后 ``w(r) ∝ r``，故 ``w_i ∝ t_i``（归一化后即离散先验）。
    ``t_hi = min(r_recv_max, 沿 theta1 到靶区边界的距离)``。
    """
    S1 = np.asarray(S1, float)
    t_hi = min(r_recv_max, sector_ray_exit(S1, theta1, r_arena))
    t_lo = near
    if t_hi <= t_lo:

```

```

        return np.zeros(0), np.zeros(0), {"t_lo": t_lo, "t_hi": t_hi, "n_t": 0}
    ts = np.linspace(t_lo, t_hi, int(n_t))
    w_area = ts / float(ts.sum())
    info = {"t_lo": float(t_lo), "t_hi": float(t_hi), "n_t": int(n_t),
            "weight": "area",
            "t_peak_note": "w(t)  $\propto$  t (面积均匀退化的严格结果)",
            "sector_half_angle_deg": float(err),
            "sector_full_width_at_t_hi_m": float(2.0 * t_hi * math.tan(math.radians(err)))}
    return ts, w_area, info

def length_weights(ts):
    w = np.ones_like(np.asarray(ts, float))
    return w / float(w.sum())

# -----
# 网格与期望
# -----
def make_grid(step=DEF_STEP, x_range=DEF_X_RANGE, y_range=DEF_Y_RANGE,
              arena_r=R_ARENA, domain="arena"):
    """生成网格点（仅保留靶区圆域内、且不贴 x 轴退化带的点）。

    默认网格框右侧多留 0.2R（`x_range=(-1800, 2160)`）：最优点在右半区
    （约 `x  $\approx$  +1080` m），把点堆在那里比均匀铺满  $\pm 1800$  更划算。
    **注意可行域仍是靶区圆域** `R <= 1800 m`，`x > 1800` 的部分不会进入结果。
    """
    xs = np.arange(x_range[0], x_range[1] + 1e-9, step)
    ys = np.arange(y_range[0], y_range[1] + 1e-9, step)
    DX, DY = np.meshgrid(xs, ys) # shape (ny, nx)
    X, Y = DX.ravel(), DY.ravel()
    keep = (X * X + Y * Y <= arena_r * arena_r) & (np.abs(Y) > 1e-9)
    if domain == "reach":
        d = np.sqrt(np.minimum(X ** 2, (np.abs(X) - R_RECV_MAX) ** 2) + Y ** 2)
        keep &= (d <= R_RECV_MAX)
    return X[keep], Y[keep], xs, ys, DX.shape

def reach_ok(X, Y, ts, r_recv_max=R_RECV_MAX):
    """可检测性门控矩阵：`|S2 - G_i| <= r_recv_max`。

    源在 x 轴上，对固定 (x,y) 关于 t 的最小值在 `t = clip(x, t_lo, t_hi)` 处取到。
    """
    X = np.asarray(X, float)[: , None]
    Y = np.asarray(Y, float)[: , None]
    ts = np.asarray(ts, float)[None, :]
    t_clip = np.clip(X, float(np.min(ts)), float(np.max(ts)))
    return np.sqrt((X - t_clip) ** 2 + Y ** 2) <= r_recv_max + 1e-9

def expected_diameter_measured(X, Y, ts, w, eps=EPS_DEG, reach=True, chunk=4096):
    """主口径：区域用测得值  $\pm$ eps，期望只对源位取。

    返回 `E`（`E[D*1{可检测}]`）、`E_det`（`E[D|可检测]`，主目标）、
    `cov`、`p_det`、`Dmin`、`Dmax`、`n_ok`。
    """
    S2 = np.column_stack([X, Y])
    n = X.size
    E = np.zeros(n)
    E_det = np.full(n, np.nan)
    cov = np.zeros(n)
    p_det = np.zeros(n)

```



```

Dmin = np.full(n, np.nan)
Dmax = np.full(n, np.nan)
n_ok = np.zeros(n, dtype=int)
wsum = float(np.sum(w))
for s in range(0, n, chunk):
    e = min(n, s + chunk)
    D, _ = fast_diameters_vec(S2[s:e], ts, eps)
    m = np.isfinite(D)
    if reach:
        m &= reach_ok(S2[s:e, 0], S2[s:e, 1], ts)
    sw = np.where(m, w[None, :], 0.0).sum(axis=1)
    contrib = (np.where(m, D, 0.0) * w[None, :]).sum(axis=1)
    with np.errstate(invalid="ignore"):
        E[s:e] = contrib
        E_det[s:e] = np.where(sw > 0, contrib / np.maximum(sw, 1e-300), np.nan)
        Dmin[s:e] = np.where(m.any(axis=1),
                               np.min(np.where(m, D, np.inf), axis=1), np.nan)
        Dmax[s:e] = np.where(m.any(axis=1),
                               np.max(np.where(m, D, -np.inf), axis=1), np.nan)
    cov[s:e] = sw
    p_det[s:e] = sw / wsum
    n_ok[s:e] = m.sum(axis=1)
return {"E": E, "E_det": E_det, "cov": cov, "p_det": p_det,
        "Dmin": Dmin, "Dmax": Dmax, "n_ok": n_ok}

```

```

def expected_diameter_error_avg(X, Y, ts, w, eps=EPS_DEG, n_e=DEF_N_E,
                                chunk=1024, seed=DEFAULT_SEED, reach=True):
    """辅助口径：区域仍取  $W1(\theta_1) \cap W2(\theta_2)$ ，但期望同时对  $\pm eps$  误差取**。
```

定义为 $E_{err}(X) = E_{\{e1,e2\}}[E_{\{t|e\}}[D * 1_{\{\text{可检测}\}}]]$ 。

实现用**分块蒙特卡洛**：对每批网格点抽 n_e 组独立误差 $(e1,e2)$ ，
第 k 组只作用到该批第 k 个点上 —— 每个网格点得到 n_e 个独立无偏估计，取均值。
代价是 $n_e \times n_t \times \text{网格点数}$ ，而不是误差笛卡尔积
(后者在 1 万网格点上要十几 GB，实测会被拖死)。

```

"""
S2 = np.column_stack([X, Y])
n = X.size
if n == 0:
    return {"E_err": np.zeros(0), "E_err_det": np.zeros(0),
            "cov_err": np.zeros(0), "n_pairs": 0}
rng = np.random.default_rng(seed)
E_err = np.zeros(n)
E_err_det = np.full(n, np.nan)
cov_err = np.zeros(n)
for s in range(0, n, chunk):
    e = min(n, s + chunk)
    blk = e - s
    E1 = rng.uniform(-eps, eps, blk)
    E2 = rng.uniform(-eps, eps, blk)
    S2b = S2[s:e]
    S1b = np.zeros((blk, 2))
    base2 = np.degrees(np.arctan2(-S2b[:, 1], S2b[:, 0])) % 360.0
    D, _ = _fast_pair_grid(S1b, E1[:, None], S2b, base2 + E2[:, None], eps)
    m = np.isfinite(D)
    if reach:
        m &= reach_ok(S2b[:, 0], S2b[:, 1], ts)
    contrib = (np.where(m, D, 0.0) * w[None, :]).sum(axis=1)
    wsum = np.where(m, w[None, :], 0.0).sum(axis=1)
    with np.errstate(invalid="ignore"):

```

```

        E_err_det[s:e] = np.where(wsum > 0, contrib / np.maximum(wsum, 1e-300), np.nan)
        cov_err[s:e] = wsum
        E_err[s:e] = contrib
    return {"E_err": E_err, "E_err_det": E_err_det, "cov_err": cov_err,
            "n_pairs": int(n_e), "estimator": "逐点蒙特卡洛（无偏）"}

# -----
# 自检
# -----

def selfcheck(verbose=True, seed=7):
    """独立实现交叉校验 + 退化算例 + 期望一致性 (T0 - T10) """
    out = []
    ok_all = True

    def rec(name, ok, detail=""):
        nonlocal ok_all
        ok_all = ok_all and ok
        out.append({"check": name, "ok": bool(ok), "detail": detail})
        if verbose:
            print(f"[{'OK' if ok else 'FAIL'}] {name} {detail}")

    def th2_of(S2, t):
        return float(np.degrees(np.arctan2(-S2[1], t - S2[0])) % 360.0)

    def mc_points(S2, t, n_pt, rng_):
        """在两个楔各自锥内采样，保留落在交会区域内的点（向量化）"""
        th2 = th2_of(S2, t)
        L = rng_.uniform(5.0, 1900.0, n_pt)
        a1 = np.radians(rng_.uniform(-EPS_DEG, EPS_DEG, n_pt))
        P1 = np.stack([L * np.cos(a1), L * np.sin(a1)], axis=1)
        a2 = math.radians(th2) + np.radians(rng_.uniform(-EPS_DEG, EPS_DEG, n_pt))
        P2 = np.asarray(S2)[None, :] + np.stack([L * np.cos(a2), L * np.sin(a2)], axis=1)
        K = np.vstack([P1, P2])

        def inw(P, apex, th):
            d = P - np.asarray(apex)[None, :]
            dist = np.hypot(d[:, 0], d[:, 1])
            angd = np.degrees(np.arctan2(d[:, 1], d[:, 0]))
            return ((np.abs((angd - th + 180.0) % 360.0 - 180.0) <= EPS_DEG + 1e-9)
                    & (dist > 1e-9))

        return K[inw(K, (0.0, 0.0), 0.0) & inw(K, S2, th2)]

    def max_pair_dist(K, blk=512):
        dmax = 0.0
        for s0 in range(0, len(K), blk):
            dmax = max(dmax, float(np.max(
                np.linalg.norm(K[s0:s0 + blk, None, :] - K[None, :, :], axis=2))))
        return dmax

    rng = np.random.default_rng(seed)

    # ---- T0: 顶点个数=4 且每个顶点真在两个楔内 ----
    n0, bad0 = 0, 0
    for _ in range(2500):
        S2 = np.array([rng.uniform(-1500, 1500), rng.uniform(-1400, 1400)])
        if np.hypot(*S2) < 200.0:
            continue
        t = float(rng.uniform(200.0, 1450.0))
        if np.hypot(S2[0] - t, S2[1]) > R_RECV_MAX:
            continue

```

```

th2 = th2_of(S2, t)
if min(abs(ang_diff(th2, 0.0)), 180 - abs(ang_diff(th2, 0.0))) <= 10.0:
    continue
D, _ = fast_diameters(S2, np.array([t]))
if not np.isfinite(D[0]):
    continue
n0 += 1
c, v = _corner_points(np.zeros((1, 1, 2)), np.zeros((1, 1)),
                      S2.reshape(1, 1, 2), np.array([[th2]]), EPS_DEG)
c, v = c[0, 0], v[0, 0]
if int(v.sum()) != 4:
    bad0 += 1
    continue
for k in np.nonzero(v)[0]:
    if not (point_in_wedge(c[k], (0.0, 0.0), 0.0)
           and point_in_wedge(c[k], S2, th2)):
        bad0 += 1
        break
rec("T0 顶点个数=4 且每个顶点真在两个楔内", bad0 <= 2,
    f"检查 {n0} 组, 异常 {bad0} (允许 ≤2 组数值退化)")

# ---- T1: 快路径 vs p1 参考实现 (良态) ----
n_bad, n_cmp, n_skip, max_dd = 0, 0, 0, 0.0
for _ in range(6000):
    S2 = np.array([rng.uniform(-1500, 1500), rng.uniform(-1500, 1500)])
    if np.hypot(*S2) < 1.0:
        continue
    t = float(rng.uniform(30.0, 1500.0))
    th2 = th2_of(S2, t)
    if min(abs(ang_diff(th2, 0.0)), 180 - abs(ang_diff(th2, 0.0))) <= 2 * EPS_DEG + 0.3:
        n_skip += 1
        continue
    D, _ = fast_diameters(S2, np.array([t]))
    P, pst = region_by_vertices([(0.0, 0.0), tuple(S2)], [0.0, th2])
    if pst != "polygon" or len(P) < 3:
        continue
    n_cmp += 1
    Pa = np.asarray(P, float)
    d_ref = max(math.dist(a, b) for i, a in enumerate(Pa) for b in Pa[i + 1:])
    if not np.isfinite(D[0]):
        n_bad += 1
        continue
    rel = abs(float(D[0]) - d_ref) / max(d_ref, 1e-12)
    max_dd = max(max_dd, rel)
    if rel > 1e-7:
        n_bad += 1
rec("T1 快路径 vs p1 参考实现 (直径, 良态)", n_bad == 0,
    f"比较 {n_cmp} 组, 跳过近临界 {n_skip} 组, 最大相对差 {max_dd:.3e}")

# ---- T2: 快路径 vs 独立实现 ----
worst, worst_cfg, n2 = 0.0, None, 0
for _ in range(8000):
    S2 = np.array([rng.uniform(-1500, 1500), rng.uniform(-1400, 1400)])
    if np.hypot(*S2) < 5.0:
        continue
    t = float(rng.uniform(20.0, 1500.0))
    D, _ = fast_diameters(S2, np.array([t]))
    if not np.isfinite(D[0]):
        continue
    dc = independent_diameter(t, S2)
    if not math.isfinite(dc):
        continue

```

```

n2 += 1
rel = abs(float(D[0]) - dc) / max(dc, 1e-12)
if rel > worst:
    worst, worst_cfg = rel, (S2.copy(), t, float(D[0]), dc)
rec("T2 快路径 vs 独立实现 (直接解边界交点)", worst < 1e-9,
    f"{n2} 组, 最大相对差 {worst:.3e}"
    + (" " if worst_cfg is None else
        f" (最差 S2={np.round(worst_cfg[0],3)}, t={worst_cfg[1]:.1f}, "
        f"数值={worst_cfg[2]:.4f}, 独立={worst_cfg[3]:.4f}) "))

# ---- T3: 蒙特卡洛真值——直径 >= 区域内采样点对最大距离 ----
worst3, worst3_cfg, n3 = 0.0, None, 0
for _ in range(40):
    S2 = np.array([rng.uniform(-1200, 1200), rng.uniform(-1200, 1200)])
    if np.hypot(*S2) < 50.0:
        continue
    t = float(rng.uniform(100.0, 1400.0))
    if np.hypot(S2[0] - t, S2[1]) > R_RECV_MAX:
        continue
    th2 = th2_of(S2, t)
    if _unbounded_bearings(0.0, th2):
        continue
    D, _ = fast_diameters(S2, np.array([t]))
    if not np.isfinite(D[0]):
        continue
    K = mc_points(S2, t, 20000, rng)
    if len(K) < 2:
        continue
    dmax = max_pair_dist(K)
    n3 += 1
    over = dmax / float(D[0]) - 1.0
    if over > worst3:
        worst3, worst3_cfg = over, (S2.copy(), t, float(D[0]), dmax)
rec("T3 直径 >= 区域内 MC 采样点对最大距离", worst3 <= 1e-9,
    f"{n3} 组, 最大超出 {worst3:.3e}"
    + (" " if worst3_cfg is None else
        f" (S2={np.round(worst3_cfg[0],2)}, t={worst3_cfg[1]:.1f}, "
        f"数值={worst3_cfg[2]:.4f}, MC={worst3_cfg[3]:.4f}) "))

# ---- T4: MC 逼近性 ----
n4, ratios = 0, []
for _ in range(40):
    S2 = np.array([rng.uniform(-800, 800), rng.uniform(-800, 800)])
    if np.hypot(*S2) < 50.0:
        continue
    t = float(rng.uniform(100.0, 1400.0))
    if np.hypot(S2[0] - t, S2[1]) > R_RECV_MAX:
        continue
    th2 = th2_of(S2, t)
    if _unbounded_bearings(0.0, th2):
        continue
    D, _ = fast_diameters(S2, np.array([t]))
    if not np.isfinite(D[0]):
        continue
    K = mc_points(S2, t, 40000, rng)
    if len(K) < 2:
        continue
    n4 += 1
    ratios.append(max_pair_dist(K) / float(D[0]))
ratios = np.array(ratios)
ok4 = bool(ratios.size and np.all(ratios >= 0.6) and np.all(ratios <= 1.0 + 1e-9))
rec("T4 MC 最大距离 / 数值直径 ∈ [0.6, 1]", ok4,

```

```

f"{n4} 组, 最小比值 {ratios.min() if ratios.size else float('nan'):.4f}, "
f"最大比值 {ratios.max() if ratios.size else float('nan'):.4f}"

# ---- T5: 贴 x 轴退化必须被识别 ----
bad5, det5 = 0, []
for y in (0.0, 1e-6, 1.0):
    S2 = np.array([600.0, y])
    D, _ = fast_diameters(S2, np.array([300.0, 900.0]))
    det5.append(f"y={y}: D={np.round(D, 3).tolist()}")
    if not np.all(np.isnan(D)):
        bad5 += 1
rec("T5 贴 x 轴退化被识别 (无界/退化)", bad5 == 0, " | ".join(det5))

# ---- T6: y=0 且源比 S2 远 => 无界 ----
D, st = fast_diameters(np.array([600.0, 0.0]), np.array([900.0]))
rec("T6 y=0 且源比 S2 远 -> 无界 (必须剔除)",
    np.isnan(D[0]) and int(st[0]) == 3, f"D={D[0]}, status={int(st[0])}")

# ---- T7: 交会角 90° (S2 在源正上方) 时  $D \approx 2 \tan(\epsilon) |S1S2|$  ----
t = 1000.0
t7_bad, t7_worst = [], 0.0
for y in (30.0, 60.0, 120.0, 1000.0):
    S2 = np.array([t, y])
    Dv, _ = fast_diameters(S2, np.array([t]))
    dc = independent_diameter(t, S2)
    target = 2 * math.tan(math.radians(EPS_DEG)) * float(np.hypot(*S2))
    rel = abs(float(Dv[0]) - target) / target
    t7_worst = max(t7_worst, rel)
    if abs(float(Dv[0]) - dc) / dc > 1e-9:
        t7_bad.append(f"y={y}: 数值={float(Dv[0]):.6f} 独立={dc:.6f} 不一致")
    if rel > 2e-3:
        t7_bad.append(f"y={y}: 相对差={rel:.2e}")
if _unbounded_bearings(0.0, 270.0):
    t7_bad.append("gamma=90° 应判定为良态")
rec("T7 交会角=90° 时  $D \approx 2 \tan(\epsilon) |S1S2|$  (实测误差 < 2e-3)", not t7_bad,
    "; ".join(t7_bad) if t7_bad else f"四组, 最大相对差 {t7_worst:.2e}")

# ---- T8: 源位采样权重归一 + 上界 ----
ts, w, info = source_samples((0.0, 0.0), 0.0, n_t=200)
rec("T8 源位采样权重归一 + 上界正确",
    abs(w.sum() - 1.0) < 1e-12 and abs(info["t_hi"] - min(R_RECV_MAX, R_ARENA)) < 1e-9,
    f"t∈[{info['t_lo']:.1f},{info['t_hi']:.1f}],  $\Sigma w=1$ ={w.sum():.2e}")

# ---- T9: E[D]可检测 与逐点加权平均一致 ----
X = np.array([300.0, 447.2, -200.0, 900.0])
Y = np.array([600.0, 800.0, 300.0, 0.0])
tt, ww, _ = source_samples((0.0, 0.0), 0.0, n_t=60)
res9 = expected_diameter_measured(X, Y, tt, ww)
manual = []
for i in range(X.size):
    num = den = 0.0
    for j in range(tt.size):
        D, _ = fast_diameters(np.array([X[i], Y[i]]), np.array([tt[j]]))
        if np.isfinite(D[0]):
            num += ww[j] * float(D[0])
            den += ww[j]
    manual.append(num / den if den > 0 else np.nan)
ok9 = np.allclose(np.array(manual), res9["E_det"], rtol=1e-10, equal_nan=True)
rec("T9 E[D]可检测 与逐点加权平均一致", ok9,
    f"manual={np.round(manual,4).tolist()}, batch={np.round(res9['E_det'],4).tolist()}")

# ---- T10: 网格框扩展的语义 (诚实版) ----

```

```

# 靶区是半径 1800 m 的圆域, 格点是"圆域 ∩ 网格框"; 因此**平移/放大框边界
# 不会改变圆域内的点数** (只改变框内空白的多少)。要真正加密只能缩小 step。
Xa, Ya, xsa, _, _ = make_grid(step=50.0, x_range=(-R_ARENA, R_ARENA))
Xb, Yb, xsb, _, _ = make_grid(step=50.0)
inside = (Xb ** 2 + Yb ** 2) <= R_ARENA ** 2 + 1e-9
same_count = (Xa.size == Xb.size)
rec("T10 网格框扩展不改变圆域内点数 (加密只能靠缩小 step)",
    bool(np.all(inside)) and same_count and xsb[-1] > xsa[-1],
    f"x_max {xsa[-1]:.0f}->{xsb[-1]:.0f} m: 圆域内点数均为 {Xa.size} (不变)",
    f"全部满足 R<=1800; 右半区点数与左半区相同 (格点本已对称)")

if verbose:
    print(f"\n自检总结: {'全部通过' if ok_all else '存在失败项'}")
return {"ok": ok_all, "checks": out}

# -----
# 输出: CSV / JSON
# -----
def _json_default(o):
    """把 numpy 标量/布尔/数组转成可序列化对象 (自检结果里会混进 np.bool_) """
    if isinstance(o, (np.bool_,)):
        return bool(o)
    if isinstance(o, np.integer):
        return int(o)
    if isinstance(o, np.floating):
        return float(o)
    if isinstance(o, np.ndarray):
        return o.tolist()
    raise TypeError(f"Object of type {type(o).__name__} is not JSON serializable")

def _fmt(v, nd=6):
    if v is None:
        return ""
    if isinstance(v, float) and (math.isnan(v) or math.isinf(v)):
        return "nan" if math.isnan(v) else "inf"
    return round(float(v), nd)

def save_outputs(res, outdir, tag=""):
    os.makedirs(outdir, exist_ok=True)
    X, Y = res["X"], res["Y"]
    path = os.path.join(outdir, f"p2_grid_map{tag}.csv")
    thr = res.get("usable_threshold_m", float("inf"))
    with open(path, "w", newline="", encoding="utf-8-sig") as f:
        wr = csv.writer(f)
        wr.writerow(["x_m", "y_m", "r_m", "phi_deg",
                     "E_diam_given_detectable_m", "E_diam_mixed_mm",
                     "p_detectable", "cov_weight", "Dmin_m", "Dmax_m",
                     "E_diam_err_given_detectable_m", "E_diam_err_mixed_m",
                     "cov_err", "n_ok", "usable", "near_axis_unusable_band"])
    for i in range(X.size):
        r = math.hypot(X[i], Y[i])
        phi = math.degrees(math.atan2(Y[i], X[i])) % 360.0
        ed = res["E_det"][i]
        usable = int(np.isfinite(ed) and ed <= thr and res["p_det"][i] > 0)
        wr.writerow([_fmt(X[i], 4), _fmt(Y[i], 4), _fmt(r, 4), _fmt(phi, 4),
                     _fmt(ed), _fmt(res["E"][i]),
                     _fmt(res["p_det"][i]), _fmt(res["cov"][i]),
                     _fmt(res["Dmin"][i]), _fmt(res["Dmax"][i]),
                     _fmt(res["E_err_det"][i]), _fmt(res["E_err"][i]),

```

```

        _fmt(res["cov_err"][i]), int(res["n_ok"][i]), usable,
        int(np.isfinite(ed) and ed > thr)])
rep = {k: v for k, v in res.items()
        if k not in ("X", "Y", "E", "E_det", "cov", "p_det", "Dmin", "Dmax", "n_ok",
                    "E_err", "E_err_det", "cov_err", "usable")}
rep["csv"] = os.path.basename(path)
with open(os.path.join(outdir, f"p2_grid_report{tag}.json"), "w", encoding="utf-8") as f:
    json.dump(rep, f, ensure_ascii=False, indent=2, default=_json_default)
return path

# -----
# 热力图
# -----
def _savefig(fig, path, dpi=150, tries=6):
    """保存图片并重试（Windows 上杀毒/索引偶尔短暂锁住刚写出的 PNG）。"""
    import time as _t
    last = None
    for k in range(tries):
        try:
            fig.savefig(path, dpi=dpi)
            return path
        except OSError as ex:
            last = ex
            _t.sleep(0.5 * (k + 1))
    raise last

def make_figures(res, outdir, tag=""):
    import matplotlib
    matplotlib.use("Agg")
    import matplotlib.pyplot as plt
    from matplotlib.colors import LogNorm
    from matplotlib.ticker import FixedLocator, FuncFormatter

    os.makedirs(outdir, exist_ok=True)
    X, Y, xs, ys, shape = res["X"], res["Y"], res["xs"], res["ys"], res["shape"]
    ny, nx = shape
    figs = []
    t_hi = float(res["source_prior"]["t_hi"])      # 扇形径向上界 = 有效接收半径上界

    def to_grid(vals):
        g = np.full((ny, nx), np.nan)
        g[np.searchsorted(ys, Y), np.searchsorted(xs, X)] = vals
        return g

    XLIM = (-1500.0, 2200.0)      # x 负方向小一点；正方向盖住右侧扩展后的网格框
    YLIM = (-2000.0, 2000.0)

    # 关注区（E ~ 50 - 1500 m）用对数色标，低值段的差异才看得出来
    ZMIN, ZMAX = 50.0, 2000.0
    lnorm = LogNorm(vmin=ZMIN, vmax=ZMAX)
    MAJOR = [50, 70, 100, 150, 200, 300, 500, 700, 1000, 1500, 2000]
    MINOR = [60, 80, 120, 250, 400, 600, 800, 1200]

    ok_use = res.get("usable", np.ones(X.size, bool))
    GX = to_grid(X)
    GY = to_grid(Y)
    Gm = to_grid(np.where(ok_use, res["E_det"], np.nan))
    Ge = to_grid(np.where(ok_use, res["E_err_det"], np.nan))
    Gc = to_grid(res["p_det"])
    th = np.linspace(0, 2 * math.pi, 2000)

```

```

def best_xy(G):
    r, c = np.unravel_index(int(np.nanargmin(G)), G.shape)
    return float(GX[r, c]), float(GY[r, c]), float(G[r, c])

def show(ax, G, cmap, vmin=None, vmax=None, norm=None, ttl=None):
    kw = {"norm": norm} if norm is not None else {"vmin": vmin, "vmax": vmax}
    im = ax.imshow(G, origin="lower", extent=[xs[0], xs[-1], ys[0], ys[-1]],
                  cmap=cmap, interpolation="nearest", aspect="equal", **kw)
    ax.set_xlim(XLIM[0], XLIM[1])
    ax.set_ylim(YLIM[0], YLIM[1])
    ax.set_xlabel("x (m)"); ax.set_ylabel("y (m)")
    if ttl:
        ax.set_title(ttl, fontsize=9)
    return im

def ring(ax, style="w--", lw=1.0, alpha=0.85):
    """只画落在显示范围内的靶区圆弧 (R=1800 m) """
    cx, cy = R_ARENA * np.cos(th), R_ARENA * np.sin(th)
    m = ((cx >= XLIM[0]) & (cx <= XLIM[1]) & (cy >= YLIM[0]) & (cy <= YLIM[1]))
    ax.plot(np.where(m, cx, np.nan), np.where(m, cy, np.nan),
           style, lw=lw, alpha=alpha)

def draw_sector(ax, zorder=6, label=True):
    """画出**首次探测的扇形** (源位先验所在区域), 半透明。

    角楔半张角 ``eps = 1°``, 径向范围 ``[near, t_hi]``. ``eps`` 只有 1°,
    所以这个扇形在整张图上很窄 (1500 m 处全宽仅约 52 m) —— 为让它看得见,
    填充用半透明 + 亮色描边.**图内不放置任何说明文字/框** (会遮挡内容),
    相关信息一律写进图例、标题或 `report.json`。

    注意 ``ax.fill`` 的 label 会按**网格点数**产生成百上千条图例项,
    所以这里不传 label, 图例项改由下面的 ``sector_patch`` 提供。
    """
    t1 = t_hi
    ang_f = np.linspace(-EPS_DEG, EPS_DEG, 400)
    ax.plot(t1 * np.cos(np.radians(ang_f)), t1 * np.sin(np.radians(ang_f)),
           "-", color="darkorange", lw=1.6, alpha=0.95, zorder=zorder)
    n = 240
    rr = np.linspace(NEAR_R, t1, n)
    aa = np.linspace(-EPS_DEG, EPS_DEG, n)
    A, R_ = np.meshgrid(np.radians(aa), rr)
    ax.fill(R_ * np.cos(A), R_ * np.sin(A), color="darkorange", alpha=0.6,
           lw=0, zorder=zorder)
    for s in (-EPS_DEG, EPS_DEG):
        a = math.radians(s)
        ax.plot([NEAR_R * math.cos(a), t1 * math.cos(a)],
               [NEAR_R * math.sin(a), t1 * math.sin(a)],
               "-", color="darkorange", lw=1.6, alpha=0.95, zorder=zorder)

def sector_patch():
    from matplotlib.patches import Patch
    return Patch(fc="darkorange", alpha=0.6, ec="darkorange",
               label=f"1st-detection sector ( $\pm$ {EPS_DEG:g}°,  $r \leq$  {t_hi:.0f} m)")

def set_ticks(cb, size=9):
    """色标刻度: 主/次刻度都带数字, 且**字号统一** (不再大小不一) """
    cb.set_ticks(MAJOR)
    cb.ax.set_yticklabels([f"{t:g}" for t in MAJOR])
    cb.ax.yaxis.set_minor_locator(FixedLocator(MINOR))
    cb.ax.yaxis.set_minor_formatter(FuncFormatter(lambda v, pos: f"{v:g}"))
    cb.ax.tick_params(which="both", labelsz=size, length=3.5)

```



```

def origin_marker(ax, ms=13, label=True):
    """第一检测点（加粗圆点）。返回 artist，便于显式组织图例。"""
    ln, = ax.plot([0], [0], marker="o", ms=ms, mfc="red", mec="k", mew=1.5,
                  ls="none", zorder=9,
                  label=("S1 = origin (detector)" if label else None))
    return ln

def best_marker(ax, G, ms=10, label=True):
    """最优第二检测点及关于 x 轴对称的那一点（图内不加文字，信息进图例）"""
    bx, by, vb = best_xy(G)
    pts = [by, -by] if abs(by) > 1e-9 else [by]
    first = None
    for n_, yy in enumerate(pts):
        ln, = ax.plot([bx], [yy], "wo", ms=ms, mfc="none", mew=2, ls="none",
                      zorder=7,
                      label=("best S2 (symmetric pair)" if (label and n_ == 0) else None))
    if first is None:
        first = ln
    return bx, by, vb, first

# ----- 图 1: 主口径热力图 -----
fig, ax = plt.subplots(figsize=(10.6, 10.6))
im = show(ax, Gm, "viridis_r", norm=lnorm)
fig.subplots_adjust(top=0.88, bottom=0.07, left=0.09, right=0.98)
ax.set_title("B-Problem 2: expected intersection-region diameter vs 2nd detection point",
             fontsize=10, pad=10)
fig.text(0.5, 0.965,
        "log colour scale; measured bearings; source prior: area-uniform inside the"
        " 1st-detection sector;\n"
        f"$\\varepsilon$={res['eps_deg']}°, $\\theta_1$={res['theta1']}°, "
        f"reach gate $|S_2|\\leq\\{int(R\_RCV\_MAX)\\}$ m, arena $R\\leq\\{int(R\_ARENA)\\}$ m, "
        f"grid step {res['step']:g} m ({res['grid_points']} points)",
        ha="center", va="bottom", fontsize=9)
cb = fig.colorbar(im, ax=ax, shrink=0.8, extend="both")
cb.set_label(f"E[intersection diameter | detectable] (m, log scale), "
             f"colourbar range {ZMIN:g} - {ZMAX:g} m")
set_ticks(cb)
ring(ax)
draw_sector(ax)
h_origin = origin_marker(ax)
bx, by, vb, h_best = best_marker(ax, Gm)
ax.legend(handles=[h_origin, h_best, sector_patch()],
          loc="lower left", fontsize=8, framealpha=0.9)
f1 = os.path.join(outdir, f"p2_grid_E_heatmap{tag}.png")
savefig(fig, f1); plt.close(fig); figs.append(f1)

# ----- 图 2: 两口径并排 -----
fig, axes = plt.subplots(1, 2, figsize=(16.4, 9.4))
for ax2, G, ttl in (
    (axes[0], Gm, "(a) measured bearings fixed\n"
     "expectation over source position only"),
    (axes[1], Ge, "(b) error-averaged\n"
     "expectation over source position AND  $\pm 1^\circ$  reading errors")):
    im = show(ax2, G, "viridis_r", norm=lnorm)
    ring(ax2)
    draw_sector(ax2, label=False)
    origin_marker(ax2, ms=11, label=False)
    ibx, iby, ibv, _ = best_marker(ax2, G, ms=8, label=False)
    ax2.set_title(f"{ttl}\nmin E = {ibv:.1f} m at ({ibx:.0f}, {iby:.0f}) m", fontsize=9)
    cb2 = fig.colorbar(im, ax=ax2, shrink=0.75, extend="both")
    cb2.set_label("E[D | detectable] (m, log scale)")

```

```

    set_ticks(cb2, size=8)
f2 = os.path.join(outdir, f"p2_grid_E_heatmap_two{tag}.png")
fig.tight_layout(); _savefig(fig, f2); plt.close(fig); figs.append(f2)

# ----- 图 3: 可检测概率 -----
fig, ax = plt.subplots(figsize=(10.6, 10.0))
im = show(ax, Gc, "magma", 0.0, 1.0,
          ttl="Detectability: probability the 2nd detector can even hear the channel")
cbp = fig.colorbar(im, ax=ax, shrink=0.8)
cbp.set_label("P(|S2G| ≤ 1500 m) under source prior")
cbp.ax.tick_params(which="both", labels=9)
ring(ax, "c--", 1.0, 0.9)
draw_sector(ax)
h_origin = origin_marker(ax)
ax.legend(handles=[h_origin, sector_patch()],
          loc="lower left", fontsize=8, framealpha=0.9)
f3 = os.path.join(outdir, f"p2_grid_coverage{tag}.png")
_savefig(fig, f3); plt.close(fig); figs.append(f3)

# ----- 图 4: E(r) 与 E(phi) 剖面 -----
fig, axes = plt.subplots(1, 2, figsize=(14.0, 5.4))
r_all = np.hypot(X, Y)
phi_all = np.degrees(np.arctan2(Y, X)) % 360.0
for phi0, style in ((90.0, "-"), (80.0, "--"), (70.0, "-."), (45.0, ":")):
    m = np.abs(((phi_all - phi0 + 180) % 360) - 180) < \
        res["step"] / (2 * np.maximum(r_all, 1)) * 1.2
    if m.sum() < 3:
        continue
    o = np.argsort(r_all[m])
    axes[0].plot(r_all[m][o], res["E_det"][m][o], style, lw=1.4, label=f"φ={phi0:.0f}°")
axes[0].set_xlabel("baseline r = |S1S2| (m)"); axes[0].set_ylabel("E[D | detectable] (m)")
axes[0].set_title("radial profile (measured bearings)", fontsize=10)
axes[0].legend(fontsize=8); axes[0].grid(alpha=0.3)
for r0, style in ((200.0, "-"), (400.0, "--"), (800.0, "-."), (1200.0, ":")):
    m = np.abs(r_all - r0) < res["step"] * 1.2
    if m.sum() < 3:
        continue
    o = np.argsort(phi_all[m])
    axes[1].plot(phi_all[m][o], res["E_det"][m][o], style, lw=1.4, label=f"r={r0:.0f} m")
axes[1].set_xlabel("φ = arg(S2) (deg)"); axes[1].set_ylabel("E[D | detectable] (m)")
axes[1].set_title("angular profile: minima near the 90°-crossing geometry", fontsize=10)
axes[1].legend(fontsize=8); axes[1].grid(alpha=0.3)
f4 = os.path.join(outdir, f"p2_grid_profiles{tag}.png")
fig.tight_layout(); _savefig(fig, f4); plt.close(fig); figs.append(f4)

# ----- 图 5: 交会区域随 S2 变化的示意 -----
from p1_intersection import convex_hull
fig, axes = plt.subplots(1, 3, figsize=(15.0, 5.2))
t_demo = 900.0
for ax2, (rr_, pp_) in zip(axes, ((900.0, 90.0), (180.0, 80.0), (900.0, 20.0))):
    S2 = np.array([rr_ * math.cos(math.radians(pp_)), rr_ * math.sin(math.radians(pp_))])
    th2 = float(np.degrees(np.arctan2(-S2[1], t_demo - S2[0])) % 360.0)
    P, stt = region_by_vertices([(0.0, 0.0), tuple(S2)], [0.0, th2])
    D, stf = fast_diameters(S2, np.array([t_demo]))
    for base, colr, anch in ((0.0, "tab:blue", np.zeros(2)), (th2, "tab:orange", S2)):
        for s in (-1, 1):
            a = math.radians(base + s * EPS_DEG)
            L = 2200.0
            ax2.plot([anch[0], anch[0] + L * math.cos(a)],
                    [anch[1], anch[1] + L * math.sin(a)],
                    color=colr, lw=1.0, alpha=0.75)
    if len(P) >= 3 and not _unbounded_bearings(0.0, th2):

```

```

        H = convex_hull(np.asarray(P, float))
        H = np.vstack([H, H[1:]])
        ax2.fill(H[:, 0], H[:, 1], color="crimson", alpha=0.55, zorder=5)
        ax2.plot([0], [0], marker="o", ms=11, mfc="red", mec="k", mew=1.4, zorder=6)
        ax2.plot([S2[0]], [S2[1]], "o", color="tab:orange", ms=7, zorder=6)
        ax2.plot([t_demo], [0], "k^", ms=8, zorder=6)
        ax2.set_xlim(-1500, 2400); ax2.set_ylim(-1400, 2100)
        dv = float(D[0]) if np.isfinite(D[0]) else float("nan")
        ax2.set_title(f"source t=900 m, r={rr:.0f} m,  $\phi$ ={pp:.0f}°\n"
                      f"intersection D = {dv:.1f} m", fontsize=9)
        ax2.set_xlabel("x (m)"); ax2.set_ylabel("y (m)"); ax2.grid(alpha=0.25)
        f5 = os.path.join(outdir, f"p2_grid_region_demo{tag}.png")
        fig.tight_layout(); _savefig(fig, f5); plt.close(fig); figs.append(f5)
    return figs

# -----
# 主流程
# -----
def run(theta1=0.0, step=DEF_STEP, n_t=DEF_N_T, n_e=DEF_N_E, weight="area",
        domain="arena", eps=EPS_DEG, outdir=None, figures=True,
        seed=DEFAULT_SEED, reach=True, checks=True, verbose=True,
        x_range=DEF_X_RANGE, y_range=DEF_Y_RANGE):
    t0 = time.time()
    S1 = (0.0, 0.0)
    ts, w_area, sinfo = source_samples(S1, theta1, eps, n_t)
    w = w_area if weight == "area" else length_weights(ts)
    if ts.size == 0:
        raise SystemExit("源位采样为空: 检查 theta1 与靶区/接收半径设置")
    X, Y, xs, ys, shape = make_grid(step=step, x_range=x_range, y_range=y_range,
                                     arena_r=R_ARENA, domain=domain)
    if verbose:
        print(f"网格点 {X.size} 个 (step={step} m, domain={domain}, "
              f"x∈[{xs[0]:.0f},{xs[-1]:.0f}], y∈[{ys[0]:.0f},{ys[-1]:.0f}]); "
              f"源位 {ts.size} 个 t∈[{ts[0]:.1f},{ts[-1]:.1f}] m, 权重={weight}")
    meas = expected_diameter_measured(X, Y, ts, w, eps, reach=reach)
    err = expected_diameter_error_avg(X, Y, ts, w, eps, n_e=n_e, seed=seed, reach=reach)

    t_nom = float(np.sum(w * ts))
    med = float(np.nanmedian(meas["E_det"]))
    usable_thr = 10.0 * med
    usable = np.isfinite(meas["E_det"]) & (meas["E_det"] <= usable_thr) & (meas["p_det"] > 0)
    E_use = np.where(usable, meas["E_det"], np.nan) if np.any(usable) else meas["E_det"]

    def _pick(a):
        a = np.asarray(a, float)
        if not np.any(np.isfinite(a)):
            return None
        i = int(np.nanargmin(a))
        return {"S2": [float(X[i]), float(Y[i])],
                      "r_m": float(math.hypot(X[i], Y[i])),
                      "phi_deg": float(math.degrees(math.atan2(Y[i], X[i])) % 360.0),
                      "value_m": float(a[i]), "index": i,
                      "p_det": float(meas["p_det"][i]),
                      "E_mixed_m": float(meas["E"][i]),
                      "E_err_det_m": (float(err["E_err_det"][i])
                                     if np.isfinite(err["E_err_det"][i]) else None),
                      "E_err_mixed_m": float(err["E_err"][i])}

    best_det, best_mix = _pick(E_use), _pick(meas["E"])
    n_band = int((np.isfinite(meas["E_det"]) & (meas["E_det"] > usable_thr)).sum())
    res = {

```

```

"model": "P2 grid simulation: E[intersection-region diameter] over 2nd detection point",
"theta1": float(theta1), "eps_deg": float(eps), "weight": weight, "domain": domain,
"step": float(step), "reach_gate": bool(reach),
"grid_x_range": [float(xs[0]), float(xs[-1])],
"grid_y_range": [float(ys[0]), float(ys[-1])],
"source_prior": sinfo, "arena_r": R_ARENA,
"r_recv_range": [R_RECV_MIN, R_RECV_MAX], "near_r": NEAR_R,
"S1": list(S1), "grid_points": int(X.size), "shape": list(shape),
"xs": [float(v) for v in xs], "ys": [float(v) for v in ys],
"usable_threshold_m": float(usable_thr),
"X": X, "Y": Y,
"E": meas["E"], "E_det": meas["E_det"], "cov": meas["cov"], "p_det": meas["p_det"],
"Dmin": meas["Dmin"], "Dmax": meas["Dmax"], "n_ok": meas["n_ok"], "usable": usable,
"E_err": err["E_err"], "E_err_det": err["E_err_det"],
"cov_err": err["cov_err"], "n_err_pairs": err["n_pairs"],
"summary": {
    "objective": "E[D | detectable] ( $|S2G| \leq 1500$  m 门控, 按扇形内面积均匀先验加权)",
    "E_det_min_m": float(np.nanmin(E_use)),
    "E_det_median_m": med,
    "E_det_max_m": float(np.nanmax(meas["E_det"])),
    "E_mixed_min_m": float(np.nanmin(meas["E"])),
    "usable_points": int(usable.sum()),
    "usable_fraction": float(usable.mean()),
    "near_axis_band_points": n_band,
    "best_det": best_det, "best_mixed": best_mix,
    "best_S2": best_det["S2"] if best_det else None,
    "best_r_m": best_det["r_m"] if best_det else None,
    "best_phi_deg": best_det["phi_deg"] if best_det else None,
    "best_r_over_t": (best_det["r_m"] / t_nom) if best_det else None,
    "mean_source_range_m": t_nom,
    "theory_note": ("交会角  $90^\circ$  时 D 取该基线下的最小值  $\approx 2 \tan(\text{eps}) |S1S2|$ ; "
        "一般情形 D 随基线与近简并程度增大"),
    "x_axis_degenerate": ("y $\approx 0$  时两示向度近平行 ( $|\Delta\theta| \leq 2\text{eps}$ )  $\rightarrow$  交会区域无界、直径 $\rightarrow\infty$ , "
        "被剔除; 其邻域 ( $|y|$  小) 直径可达  $10^4\text{--}10^5$  m, 已用 "
        f"usable_threshold={usable_thr:.1f} m 标注为不可用带"),
    "reach_gate": ("  $|S2G| >$  有效接收半径上界 1500 m 的源位样本不可检测, "
        "不计入 E[D]可检测; 混口径 E[D $\cdot 1$ ]可检测] 另列"),
    "grid_note": ("网格框右侧多留 0.2R (x 到 2160 m) 以便右半区更密; "
        "可行域仍为靶区圆域 R $\leq 1800$  m"),
    "runtime_s": None,
},
},
if checks:
    res["checks"] = selfcheck(verbose=verbose)["checks"]
res["summary"]["runtime_s"] = round(time.time() - t0, 3)
if outdir:
    save_outputs(res, outdir)
    if figures:
        try:
            figs = make_figures(res, outdir)
            res["figures"] = [os.path.basename(f) for f in figs]
        except Exception as ex:
            # 图失败不影响数值结果落盘
            res["figures_error"] = f"{type(ex).__name__}: {ex}"
        if verbose:
            print(f"[warn] 出图失败 (数值结果已保存): {ex}")
    save_outputs(res, outdir)
if verbose:
    s = res["summary"]
    b = s["best_det"]
    print(f"\n最优第二检测点 S2* = ({b['S2'][0]:.1f}, {b['S2'][1]:.1f}) m "
        f" (r={b['r_m']:.1f} m,  $\phi$ = $\{b['phi\_deg']:.1f\}^\circ$ ,  $r/t$ = $\{s['best\_r\_over\_t']:.3f\}$ )")
    print(f"E[D]可检测] 最小 =  $\{s['E\_det\_min\_m']:.3f\}$  m; 中位数 =  $\{s['E\_det\_median\_m']:.3f\}$  m; "

```

```

        f"单条基线 2tan(eps)r = {2*math.tan(math.radians(eps))*b['r_m']:.3f} m")
    print(f"该点 P(可检测) = {b['p_det']:.4f}; 混口径 E[D*1] = {b['E_mixed_m']:.3f} m; "
          f"误差口径 E[D|可检测] = {b['E_err_det_m']}")
    print(f"可用点 {s['usable_points']}/{res['grid_points']}")
    f" (近轴不可用带 {s['near_axis_band_points']} 个, 阈值 {usable_thr:.1f} m)"
    print(f"耗时 {s['runtime_s']} s")
    return res

def main(argv=None):
    ap = argparse.ArgumentParser(
        description="B 题问题 2: 第二检测点网格上的交会区域直径期望热力图")
    ap.add_argument("--theta1", type=float, default=0.0,
                    help="首次探测的示向度(度), 默认 0 (x 轴正向)")
    ap.add_argument("--step", type=float, default=DEF_STEP, help="网格步长 (m)")
    ap.add_argument("--n-t", type=int, default=DEF_N_T, help="x 轴源位采样数")
    ap.add_argument("--n-e", type=int, default=DEF_N_E, help="误差 MC 采样数")
    ap.add_argument("--weight", choices=["area", "length"], default="area",
                    help="源位先验权重: area  $\propto t$  (面积均匀, 默认) / length 等权")
    ap.add_argument("--domain", choices=["arena", "reach"], default="arena",
                    help="S2 取值范围: 整个靶区圆域 / 还要满足 |S2G|<=1500")
    ap.add_argument("--x-range", type=float, nargs=2, default=list(DEF_X_RANGE),
                    metavar=("XMIN", "XMAX"), help="网格 x 范围 (m), 默认右侧多留 0.2R")
    ap.add_argument("--y-range", type=float, nargs=2, default=list(DEF_Y_RANGE),
                    metavar=("YMIN", "YMAX"), help="网格 y 范围 (m)")
    ap.add_argument("--out", default=None, help="输出目录")
    ap.add_argument("--no-reach", action="store_true", help="关闭可检测性门控")
    ap.add_argument("--no-figures", action="store_true")
    ap.add_argument("--no-checks", action="store_true", help="跳过自检 (只出结果, 快很多)")
    ap.add_argument("--selfcheck", action="store_true")
    a = ap.parse_args(argv)
    if a.selfcheck:
        r = selfcheck()
        if a.out:
            os.makedirs(a.out, exist_ok=True)
            with open(os.path.join(a.out, "p2_grid_selfcheck.json"), "w",
                      encoding="utf-8") as f:
                json.dump(r, f, ensure_ascii=False, indent=2, default=_json_default)
        return 0 if r["ok"] else 1
    run(theta1=a.theta1, step=a.step, n_t=a.n_t, n_e=a.n_e, weight=a.weight,
        domain=a.domain, outdir=a.out, figures=not a.no_figures,
        checks=not a.no_checks, reach=not a.no_reach,
        x_range=tuple(a.x_range), y_range=tuple(a.y_range))
    return 0

if __name__ == "__main__":
    raise SystemExit(main())

```

5.3 src/p3_arena.py

"""B 题问题 3: 机器狗与模拟器之间的统一接口层。

两个后端对上层暴露**完全相同的接口与返回字段** (字段名与模拟器 JSON 一致), 因此 :mod:`p3\_robot` 里的策略代码在"离线演练"和"真实模拟器"上完全一致:

```

* :class:`MockArena` —— 完全离线复刻附件 1/附件 2 的物理与计时规则。
    用于自检、调参、批量演练 (可以跑几百局, 正式测试只有 3 次机会)。
* :class:`HttpArena` —— 真实模拟器的 HTTP+JSON 客户端, 负责协议里最容易踩的坑:
    串行发送、`request_id` 幂等与重试、`accepted` 与 HTTP 状态双重校验、

```

```

    ``remaining_real_duration_s`` 动态预算、自身行为日志（论文支撑材料需要）。

计时规则（附件 1 表 1 / 附件 2 第 4 节）
-----
* ``/measure``: 移动距离/5 + （频道变化 ? 1 : 0） + 5 秒；只有 ``/measure`` 会切换频道；
* ``/clear``: 移动距离/5 + （成功 5 / 未发现 3）秒；**不切换频道**；
* ``/enter``、``/exit`` 不推进虚拟时钟；
* 示向度误差在  $[-1^\circ, 1^\circ]$  内，**同一地点固定**（本模块用"平滑随机场 + 局地量化项"建模）。
"""

from __future__ import annotations

import json
import math
import os
import socket
import time
import urllib.error
import urllib.request
from dataclasses import dataclass, field as dc_field

import numpy as np

# -----
# 物理常数（题目附录 1/2、模拟器接口说明）
# -----
R_ARENA = 1800.0          # 靶区半径 (m)
V_ROBOT = 5.0             # 移动速度 (m/s)
T_MEASURE = 5.0           # 一次检测耗时 (s)
T_SWITCH = 1.0            # 频道切换耗时 (s)
T_CLEAR_OK = 5.0          # 成功清除耗时 (s): 光学 3 + 激光 2
T_CLEAR_FAIL = 3.0        # 未发现目标时的光学精确定位耗时 (s)
T_CLEAR = T_CLEAR_OK
NEAR_R = 5.0              # 近场阈值 (m): 更近时返回 near, 无示向度
R_CLEAR = 20.0            # 清除半径 (m)
R_RECV_MIN, R_RECV_MAX = 1000.0, 1500.0
BEARING_ERR = 1.0         # 示向度误差界 (度)
CHANNELS = tuple(range(1, 21))
# 程序运行时间（现实时间）上限 (s): 题目附录 3, /enter 响应里由 remaining_real_duration_s 给出
DEF_BUDGET_S = 1200.0
# 虚拟世界限时 (s): 接口说明 1.4/4.5, 默认 360000 (100 小时)。
# **注意**: 这是虚拟时钟的上限，与上面 1200 s 的现实时间预算是两回事。
# 离线 mock 必须用同一个值，否则等于给离线演练加了一条真实测试里不存在的约束。
DEF_VIRTUAL_LIMIT_S = 360000.0
COORD_LIMIT = 2_000_000.0 # 接口允许的坐标绝对值上限 (m)

class ArenaError(RuntimeError):
    """接口层不可恢复错误（网络重试用尽、响应无法解析等）。"""

# -----
# 离线模拟器
# -----
@dataclass
class MockSource:
    channel: int
    x: float
    y: float
    r_recv: float
    cone_half: float = 180.0 # 覆盖半角: 180 = 全向（问题 4 会用到 <180）
    dir_deg: float | None = None
    cleared: bool = False

```

```

@property
def xy(self):
    return (self.x, self.y)

class MockArena:
    """离线复刻的模拟器（问题 3：全向源）。"""

    def __init__(self, seed=0, n_sources=None, budget_s=DEF_VIRTUAL_LIMIT_S,
                  r_arena=R_ARENA, sources=None, error_corr_m=250.0,
                  error_local_w=0.25, verbose=False):
        """`budget_s` = **虚拟时钟上限**（默认 360000 s，与真实模拟器一致）。"""
        self.seed = int(seed)
        self.rng = np.random.default_rng(self.seed)
        self.r_arena = float(r_arena)
        self.budget = float(budget_s)
        self.sources = list(sources) if sources is not None else self._gen_sources(n_sources)
        self._init_error_field(error_corr_m, error_local_w)
        self.pos = (0.0, 0.0)
        self.channel = 1
        self.vtime = 0.0
        self.entered = False
        self.finished = False
        self.verbose = verbose
        self.actions: list[dict] = []
        self.stats = {"n_measure": 0, "n_clear": 0, "n_clear_fail": 0,
                      "travel_m": 0.0, "n_direction": 0, "n_near": 0, "n_no_signal": 0}

    # ----- 场景生成 -----
    def _gen_sources(self, n_sources=None):
        rng = self.rng
        n = int(n_sources) if n_sources else int(rng.integers(10, 17))
        n = max(1, min(n, len(CHANNELS)))
        chans = rng.choice(np.asarray(CHANNELS), size=n, replace=False)
        out = []
        for ch in chans:
            r = self.r_arena * math.sqrt(float(rng.random()))
            a = float(rng.uniform(0, 2 * math.pi))
            out.append(MockSource(channel=int(ch), x=r * math.cos(a), y=r * math.sin(a),
                                  r_recv=float(rng.uniform(R_RECV_MIN, R_RECV_MAX))))
        return out

    def _init_error_field(self, corr_m, local_w):
        """示向度误差场：平滑随机场（模拟电磁环境）+ 局地量化项；量程严格  $\pm 1^\circ$ 。"""
        rng = self.rng
        self._waves = []
        for _ in range(12):
            lam = float(rng.uniform(max(corr_m * 1.5, 120.0), 3.0 * self.r_arena))
            ang = float(rng.uniform(0, 2 * math.pi))
            self._waves.append((2 * math.pi * math.cos(ang) / lam,
                                2 * math.pi * math.sin(ang) / lam,
                                float(rng.uniform(0, 2 * math.pi)),
                                1.0 / lam))
        # 归一化：用靶区内抽样点的最大绝对值做标定，再乘 1.05 并截断到  $\pm 1$ 
        pts = self._sample_disc(4000, rng)
        raw = self._raw_field(pts[:, 0], pts[:, 1])
        self._scale = max(float(np.max(np.abs(raw))) * 1.05, 1e-9)
        self._local_w = float(local_w)
        self._local_seed = int(rng.integers(1, 10 ** 6))

    def _sample_disc(self, n, rng):

```

```

    r = self.r_arena * np.sqrt(rng.random(n))
    a = rng.uniform(0, 2 * math.pi, n)
    return np.stack([r * np.cos(a), r * np.sin(a)], axis=1)

def _raw_field(self, x, y):
    x = np.asarray(x, float)
    y = np.asarray(y, float)
    v = np.zeros_like(x)
    for (kx, ky, ph, a) in self._waves:
        v = v + a * np.sin(kx * x + ky * y + ph)
    return v

def _local_field(self, x, y):
    i = int(math.floor(x / 25.0))
    j = int(math.floor(y / 25.0))
    h = (i * 73856093) ^ (j * 19349663) ^ (self._local_seed * 83492791)
    h &= 0x7FFFFFFF
    return (h % 200001) / 100000.0 - 1.0

def env_error(self, x, y):
    """该地点的示向度误差（度，[-1,1]，同一地点固定）。"""
    v = (float(self._raw_field(x, y)) / self._scale
        + self._local_w * self._local_field(x, y))
    return float(max(-1.0, min(1.0, v)))

# ----- 状态查询 -----
@property
def virtual_time_s(self):
    return self.vtime

def remaining_budget_s(self):
    return max(0.0, self.budget - self.vtime)

def budget_kind(self):
    return "virtual"

def truth(self):
    return {"n_sources": len(self.sources),
            "n_cleared": sum(1 for s in self.sources if s.cleared),
            "sources": [{"channel": s.channel, "x_m": round(s.x, 3),
                          "y_m": round(s.y, 3), "r_recv_m": round(s.r_recv, 3),
                          "cleared": bool(s.cleared)} for s in self.sources]}

# ----- 内部工具 -----
def _src_of(self, channel):
    for s in self.sources:
        if s.channel == int(channel):
            return s
    return None

def _reject(self, reason, **kw):
    r = {"accepted": False, "real_timestamp_ms": int(time.time() * 1000),
        "virtual_time_s": 0.0, "reason": reason}
    r.update(kw)
    return r

def _advance(self, x, y, dt, kind, payload):
    d = math.hypot(x - self.pos[0], y - self.pos[1])
    self.stats["travel_m"] += d
    self.vtime += dt
    self.pos = (float(x), float(y))
    rec = {"kind": kind, "x": float(x), "y": float(y), "dt": float(dt),

```



```

        "dist_m": float(d), "virtual_time_s": float(self.vtime)}
rec.update(payload)
self.actions.append(rec)
if self.verbose:
    print(f"    [mock] {kind} ({x:.0f},{y:.0f}) dt={dt:.1f}s "
          f"t={self.vtime:.1f}s {payload}")
return rec

# ----- 四条指令 -----
def enter(self):
    if self.entered:
        return self._reject("already_entered")
    self.entered = True
    self.actions.append({"kind": "enter", "virtual_time_s": 0.0})
    return {"accepted": True, "real_timestamp_ms": int(time.time() * 1000),
            "virtual_time_s": 0.0, "max_virtual_duration_s": self.budget,
            "max_real_duration_s": self.budget,
            "remaining_real_duration_s": int(self.budget)}

def measure(self, x, y, channel):
    if not self.entered:
        return self._reject("not_entered")
    if self.finished:
        return self._reject("finished")
    x, y, ch = float(x), float(y), int(channel)
    if not (math.isfinite(x) and math.isfinite(y)) \
        or abs(x) > COORD_LIMIT or abs(y) > COORD_LIMIT:
        return self._reject("bad_position")
    if ch not in CHANNELS:
        return self._reject("bad_channel")
    switch = T_SWITCH if ch != self.channel else 0.0
    d = math.hypot(x - self.pos[0], y - self.pos[1])
    dt = d / V_ROBOT + switch + T_MEASURE
    if self.vtime + dt > self.budget + 1e-9:
        self.finished = True
        return self._reject("timeout")
    src = self._src_of(ch)
    svd = None
    if src is None or src.cleared:
        result = "no_signal"
    else:
        dist = math.hypot(x - src.x, y - src.y)
        in_cone = True
        if src.cone_half < 180.0:
            a = math.degrees(math.atan2(src.y - y, src.x - x))
            ref = src.dir_deg if src.dir_deg is not None else 0.0
            in_cone = abs((a - ref + 180.0) % 360.0 - 180.0) <= src.cone_half + 1e-12
        if dist > src.r_recv or not in_cone:
            result = "no_signal"
        elif dist <= NEAR_R:
            result = "near"
        else:
            result = "direction"
            a = math.degrees(math.atan2(src.y - y, src.x - x))
            svd = round((a + self.env_error(x, y)) % 360.0, 2)
    self.channel = ch
    self.stats["n_measure"] += 1
    self.stats["n_" + result] += 1
    self._advance(x, y, dt, "measure",
                  {"channel": ch, "measure_result": result, "svd_deg": svd,
                   "switch_s": switch})
    resp = {"accepted": True, "real_timestamp_ms": int(time.time() * 1000),

```

```

        "virtual_time_s": self.vtime, "measure_result": result}

    if svd is not None:
        resp["svd_deg"] = svd
    return resp

def clear(self, x, y, channel):
    if not self.entered:
        return self._reject("not_entered")
    if self.finished:
        return self._reject("finished")
    x, y, ch = float(x), float(y), int(channel)
    if not (math.isfinite(x) and math.isfinite(y)) \
        or abs(x) > COORD_LIMIT or abs(y) > COORD_LIMIT:
        return self._reject("bad_position")
    if ch not in CHANNELS:
        return self._reject("bad_channel")
    d = math.hypot(x - self.pos[0], y - self.pos[1])
    src = self._src_of(ch)
    ok = bool(src is not None and not src.cleared
              and math.hypot(x - src.x, y - src.y) <= R_CLEAR + 1e-9)
    dt = d / V_ROBOT + (T_CLEAR_OK if ok else T_CLEAR_FAIL)
    if self.vtime + dt > self.budget + 1e-9:
        self.finished = True
        return self._reject("timeout")
    if ok:
        src.cleared = True
        self.stats["n_clear"] += 1
    if not ok:
        self.stats["n_clear_fail"] += 1
    self._advance(x, y, dt, "clear",
                  {"channel": ch, "clear_result": "success" if ok else "no_target_in_range"})
    return {"accepted": True, "real_timestamp_ms": int(time.time() * 1000),
            "virtual_time_s": self.vtime,
            "clear_result": "success" if ok else "no_target_in_range"}

def exit(self):
    self.finished = True
    self.actions.append({"kind": "exit", "virtual_time_s": self.vtime})
    return {"accepted": True, "real_timestamp_ms": int(time.time() * 1000),
            "virtual_time_s": self.vtime, "exit_reason": "user_exit"}

# -----
# 真实模拟器客户端
# -----

class HttpArena:
    """模拟器 HTTP+JSON 客户端（默认 http://127.0.0.1:2026）。"""

    def __init__(self, base_url="http://127.0.0.1:2026", robot_id="",
                  arena_id="default", timeout=5.0, max_retries=4, retry_delay=0.4,
                  log_path=None, budget_s=DEF_BUDGET_S, safety_margin_s=3.0,
                  enter_wait_s=40.0):
        self.base_url = base_url.rstrip("/")
        self.robot_id = robot_id
        self.arena_id = arena_id
        self.timeout = float(timeout)
        self.max_retries = int(max_retries)
        self.retry_delay = float(retry_delay)
        self.log_path = log_path
        self.budget = float(budget_s)
        self.safety_margin_s = float(safety_margin_s)
        self.enter_wait_s = float(enter_wait_s)

```

```

self._n = 0
self.entered = False
self.pos = (0.0, 0.0)
self.channel = 1
self.vtime = 0.0
self.remaining_real0 = float(budget_s)
self._t0 = None
self.actions: list[dict] = []
# 与 MockArena 同名的统计（接口不返回行程，客户端自己按动作位置累加，
# 只统计 accepted=true 的动作，与虚拟时间的推进口径一致）
self.stats = {"n_measure": 0, "n_clear": 0, "n_clear_fail": 0,
               "travel_m": 0.0, "n_direction": 0, "n_near": 0, "n_no_signal": 0}
if self.log_path:
    os.makedirs(os.path.dirname(os.path.abspath(self.log_path)), exist_ok=True)
    with open(self.log_path, "w", encoding="utf-8") as f:
        f.write(json.dumps({"kind": "session", "base_url": self.base_url,
                           "robot_id": self.robot_id, ensure_ascii=False}) + "\n")

# ----- HTTP -----
def _rid(self, tag):
    self._n += 1
    return f"{tag}-{self._n}"

def _base(self, rid):
    return {"arena_id": self.arena_id, "robot_id": self.robot_id, "request_id": rid}

def _post(self, path, payload, tag):
    data = json.dumps(payload, ensure_ascii=False).encode("utf-8")
    last_err = None
    for attempt in range(self.max_retries):
        req = urllib.request.Request(
            self.base_url + path, data=data,
            headers={"Content-Type": "application/json"}, method="POST")
        try:
            with urllib.request.urlopen(req, timeout=self.timeout) as r:
                body = r.read().decode("utf-8")
                resp = json.loads(body)
                self._log({"kind": "resp", "path": path, "attempt": attempt, "resp": resp})
                return resp
        except urllib.error.HTTPError as e:
            body = ""
            try:
                body = e.read().decode("utf-8", "replace")
                resp = json.loads(body)
                self._log({"kind": "http_error", "path": path, "code": e.code,
                           "resp": resp})
                return resp
            except Exception:
                self._log({"kind": "http_error", "path": path, "code": e.code,
                           "body": body[:500]})
                raise ArenaError(f"{path} HTTP {e.code}: {body[:200]}") from e
        except (urllib.error.URLError, socket.timeout, TimeoutError, ConnectionError) as e:
            # 网络超时/中断：按协议重试完全相同的动作（复用 request_id 与内容）
            last_err = e
            self._log({"kind": "net_error", "path": path, "attempt": attempt,
                       "error": repr(e)})
            if attempt + 1 < self.max_retries:
                time.sleep(self.retry_delay * (attempt + 1))
    raise ArenaError(f"{path} 请求失败（已重试 {self.max_retries} 次）: {last_err!r}")

def _log(self, rec):
    rec["wall_s"] = round(time.time() - (self._t0 or time.time()), 3)

```

```

self.actions.append(rec)
if self.log_path:
    with open(self.log_path, "a", encoding="utf-8") as f:
        f.write(json.dumps(rec, ensure_ascii=False) + "\n")

def _new(self, path, extra, tag):
    rid = self._rid(tag)
    payload = self._base(rid)
    payload.update(extra)
    self._log({"kind": "req", "path": path, "payload": payload})
    return self._post(path, payload, tag)

# ----- 状态 -----
@property
def virtual_time_s(self):
    return self.vtime

def remaining_budget_s(self):
    if self._t0 is None:
        return self.remaining_real0
    used = time.monotonic() - self._t0
    return max(0.0, self.remaining_real0 - used - self.safety_margin_s)

def budget_kind(self):
    return "real"

def truth(self):
    return None

# ----- 四条指令 -----
def enter(self):
    """进入靶区。接口未开放时（倒计时未结束 / 测试未开始）会重试同一 request_id。

    协议规定"同一 request_id + 同一内容返回第一次的完整响应"，所以用固定的
    request_id 重试 /enter 是安全的（不会重复进入、也不会推进时钟）。
    """
    rid = self._rid("enter")
    payload = self._base(rid)
    self._log({"kind": "req", "path": "/enter", "payload": payload})
    t0 = time.monotonic()
    last = None
    keep_retries = self.max_retries
    self.max_retries = min(keep_retries, 2)  # 接口未开放时缩短单次重试，尽快进入下一轮
    try:
        while True:
            try:
                resp = self._post("/enter", payload, "enter")
            except ArenaError as e:
                resp, last = None, e
            if resp is not None and resp.get("accepted") is True:
                self.entered = True
                self._t0 = time.monotonic()
                self.remaining_real0 = float(resp.get("remaining_real_duration_s",
                                                         self.budget))
                self.vtime = float(resp.get("virtual_time_s", 0.0))
                return resp
            if resp is not None and resp.get("accepted") is False \
                and resp.get("reason") == "already_entered":
                return resp
            if time.monotonic() - t0 > self.enter_wait_s:
                if resp is not None:
                    return resp

```

```

        raise ArenaError(
            f"/enter 在 {self.enter_wait_s:.0f} s 内一直失败: {last!r}")
        time.sleep(2.0)
    finally:
        self.max_retries = keep_retries

def measure(self, x, y, channel):
    resp = self._new("/measure", {"position": {"x": float(x), "y": float(y)},
                                   "channel": int(channel)}, "measure")
    if resp.get("accepted") is True:
        self.stats["travel_m"] += math.hypot(float(x) - self.pos[0],
                                              float(y) - self.pos[1])
        self.pos = (float(x), float(y))
        self.channel = int(channel)
        self.vtime = float(resp.get("virtual_time_s", self.vtime))
        res = resp.get("measure_result")
        self.stats["n_measure"] += 1
        if res in ("direction", "near", "no_signal"):
            self.stats["n_" + res] += 1
    return resp

def clear(self, x, y, channel):
    resp = self._new("/clear", {"position": {"x": float(x), "y": float(y)},
                                   "channel": int(channel)}, "clear")
    if resp.get("accepted") is True:
        self.stats["travel_m"] += math.hypot(float(x) - self.pos[0],
                                              float(y) - self.pos[1])
        self.pos = (float(x), float(y))
        self.vtime = float(resp.get("virtual_time_s", self.vtime))
        self.stats["n_clear"] += 1
        if resp.get("clear_result") != "success":
            self.stats["n_clear_fail"] += 1
    return resp

def exit(self):
    return self._new("/exit", {}, "exit")

# -----
# 自检
# -----
def selfcheck(verbose=True, seed=20260913):
    """核对 MockArena 是否忠实复刻附件 1/2 的规则。"""
    out = []
    ok_all = True

    def rec(name, ok, detail=""):
        nonlocal ok_all
        ok_all = ok_all and bool(ok)
        out.append({"check": name, "ok": bool(ok), "detail": detail})
        if verbose:
            print(f"[{'OK' if ok else 'FAIL'}] {name} {detail}")

    # A1 附件 1 表 2 的计时算例 ((300,400) 频道1 -> 频道2 -> /clear 未发现 -> 频道2)
    a = MockArena(seed=1, sources=[MockSource(3, 100000.0, 0.0, 1000.0)])
    a.enter()
    r1 = a.measure(300.0, 400.0, 1)
    r2 = a.measure(300.0, 400.0, 2)
    r3 = a.clear(300.0, 0.0, 3)
    r4 = a.measure(300.0, 0.0, 2)
    ok1 = (abs(r1["virtual_time_s"] - 105.0) < 1e-9
           and abs(r2["virtual_time_s"] - 111.0) < 1e-9)

```

```

        and abs(r3["virtual_time_s"] - 194.0) < 1e-9
        and abs(r4["virtual_time_s"] - 199.0) < 1e-9)
rec("A1 计时算例与附件1表2逐条一致 (105/111/194/199 s)", ok1,
    f"{r1['virtual_time_s']}, {r2['virtual_time_s']}, "
    f"{r3['virtual_time_s']}, {r4['virtual_time_s']}")

# A2 /clear 不切换频道, 也不产生切换耗时
b = MockArena(seed=2, sources=[MockSource(1, 0.0, 0.0, 1000.0)])
b.enter()
b.measure(0.0, 100.0, 1)
t0 = b.virtual_time_s
b.clear(0.0, 50.0, 7)
rec("A2 /clear 不切换频道、不产生切换耗时",
    abs(b.virtual_time_s - t0 - (50.0 / V_ROBOT + T_CLEAR_FAIL)) < 1e-9
    and b.channel == 1,
    f"Δt={b.virtual_time_s - t0:.1f}s, 当前频道={b.channel}")

# A3 三种检测结果与清除半径
src = MockSource(5, 0.0, 0.0, 1000.0)
c = MockArena(seed=3, sources=[src])
c.enter()
m_far = c.measure(0.0, 0.0, 5)          # 距离 0 <= 5 m -> near
m_dir = c.measure(0.0, 300.0, 5)        # direction (真方位 270°)
m_no = c.measure(0.0, 1200.0, 5)        # 超出 1000 m -> no_signal
ok3 = (m_far["measure_result"] == "near" and m_dir["measure_result"] == "direction"
    and abs(m_dir["svd_deg"] - 270.0) <= BEARING_ERR + 1e-9
    and m_no["measure_result"] == "no_signal")
rec("A3 near / direction / no_signal 三种结果与真方位一致", ok3,
    f"near={m_far['measure_result']}, dir={m_dir.get('svd_deg')}°, "
    f"no={m_no['measure_result']}")
cl_far = c.clear(0.0, 1000.0, 5)
cl_ok = c.clear(0.0, 20.0, 5)
rec("A4 清除半径 20 m: 远处未发现、20 m 内成功",
    cl_far["clear_result"] == "no_target_in_range"
    and cl_ok["clear_result"] == "success",
    f"{cl_far['clear_result']} / {cl_ok['clear_result']}")
rec("A5 同一干扰源只能清除一次",
    c.clear(0.0, 0.0, 5)["clear_result"] == "no_target_in_range",
    "二次清除返回 no_target_in_range")

# A6 误差界与"同一地点固定"
d = MockArena(seed=4)
e1 = d.env_error(123.0, -456.0)
e2 = d.env_error(123.0, -456.0)
vals = [d.env_error(float(x), float(y)) for x, y in d._sample_disc(500, d.rng)]
rec("A6 示向度误差: |e|<=1° 且同一地点固定",
    abs(e1 - e2) < 1e-12 and max(abs(v) for v in vals) <= 1.0 + 1e-12,
    f"e={e1:.4f}, 重复一致: 500 点最大 |e|={max(abs(v) for v in vals):.4f}")

# A7 真源一定落在"测得方位 ±1°"的楔内 (区间算法成立的前提)
bad = 0
nchk = 0
d2 = MockArena(seed=4, budget_s=1e9)
d2.enter()
for s in d2.sources:
    for _ in range(3):
        aa = float(d2.rng.uniform(0, 2 * math.pi))
        dd = float(d2.rng.uniform(50.0, 0.9 * s.r_recv))
        x, y = s.x + dd * math.cos(aa), s.y + dd * math.sin(aa)
        r = d2.measure(x, y, s.channel)
        if r.get("accepted") is not True or r.get("measure_result") != "direction":
            continue

```

```

        nchk += 1
        true_a = math.degrees(math.atan2(s.y - y, s.x - x)) % 360.0
        if abs((true_a - r["svd_deg"] + 180.0) % 360.0 - 180.0) > BEARING_ERR + 1e-9:
            bad += 1
    rec("A7 真方位与示向度之差恒 <= 1°", bad == 0 and nchk > 0,
        f"{nchk} 次检测, 越界 {bad} 次")

# A8 超时拒动
e = MockArena(seed=5, budget_s=50.0, sources=[MockSource(1, 0.0, 0.0, 1000.0)])
e.enter()
e.measure(0.0, 1000.0, 1)          # 200 + 5 = 205 s > 50 s
rec("A8 超出时间预算的请求 accepted=false 且不推进时钟",
    e.virtual_time_s == 0.0 and e.finished,
    f"vtime={e.virtual_time_s}")

if verbose:
    print(f"\n自检总结: {'全部通过' if ok_all else '存在失败项'}")
return {"ok": ok_all, "checks": out}

if __name__ == "__main__":
    import argparse
    ap = argparse.ArgumentParser(description="问题3 接口层自检")
    ap.add_argument("--selfcheck", action="store_true")
    args = ap.parse_args()
    res = selfcheck()
    raise SystemExit(0 if res["ok"] else 1)

```

5.4 src/p3_expect_field.py

"""B 题问题 3: 机器狗"基于期望的行动"决策场。

本模块只负责问题 3 的**决策层**：把"下一步该去哪里检测"变成一个可以在全局网络上逐格比较的标量场。它严格按下面的思路实现：

1. **全局网格**：在半径 1800 m 的靶区圆域内铺一张规则网格（`grid_step`，默认 100 m），每个格子记录"如果机器狗去那里做下一次检测，这一步有多好"。
2. **第二问的网格模拟结果**：直接复用 `mod:~p2_grid_expectation~` 产出的热力图 `out/p2_grid/p2_grid_map.csv`。该图是在"第一检测点 $S_1=(0,0)$ 、示向度 $\theta_1=0^\circ$ "的基准几何下算出的 $E[D]$ （ D = 交会定位区域直径）。
3. **变换 + 插值**： $E[D]$ 在**刚体变换下不变**（旋转 + 平移保持距离与夹角）。因此真实一次检测 (S, θ) 对应的场，就是把基准图**绕原点旋转 θ 、再平移到 S ：

$$b = R(-\theta) \cdot (g - S) \quad (g \text{ 是全局网格点, } b \text{ 是基准图坐标})$$

$$E_g = E_{\text{base}}(b) \quad (\text{双线性插值})$$

4. **逐次叠加**：把每个"待测频道"的每一次检测都按上式投到全局网格上叠加。
5. **未覆盖的网格取最大值**：基准图里没有数据（靶区外 / 该点不可检测 / 近轴退化带）或数值超过上限的格子一律记 $E_{\text{max}}=1500$ m —— 也就是"最差的位置"。这样它们只有在附近没有更好的格子时才会被选中，天然形成探索行为。
6. **归一化**： $E_{\text{norm}} = E_{\text{mean}} / E_{\text{max}} \in [0,1]$ （0 = 交会几何最好，1 = 最差）。
7. **距离惩罚层**：行动要花时间，所以在归一化场之上再叠加一层**与探测器距离线性增长**的代价：

$$\text{score}(g) = E_{\text{norm}}(g) + \lambda \cdot |g - p_{\text{robot}}| / (2 \cdot R_{\text{arena}})$$

λ 即"距离项系数"，可调（`dist_weight`，默认 0.35； $\lambda=1$ 表示"跑到靶区对面"等价于把定位质量从最好拉到最差）。已访问过的扫描点邻域、以及离当前位置过近的格子

会被直接排除（重复检测不产生新信息）。

8. ****选点****: 取 ``argmin score``。若整张图都被排除（例如一条方位都还没有），退化为"最大最小距离"探索点（离所有已访问点最远的格子）。

用法

```
python src/p3_expect_field.py --selfcheck
python src/p3_expect_field.py --demo --dist-weight 0.35 --out out/p3

"""
from __future__ import annotations

import argparse
import csv
import json
import math
import os
import sys
from dataclasses import dataclass, field as dc_field

import numpy as np

_HERE = os.path.dirname(os.path.abspath(__file__))
if _HERE not in sys.path:
    sys.path.insert(0, _HERE)

from p1_intersection import BEARING_ERR, ang_diff # noqa: E402

# -----
# 物理常数（题目附录 1/2 与模拟器接口说明）
# -----
R_ARENA = 1800.0          # 靶区半径（m）
E_MAX = 1500.0            # 未覆盖网格的默认最大值（m）——即"最差位置"
R_RECV_MIN = 1000.0       # 有效接收半径下界（m）
R_RECV_MAX = 1500.0       # 有效接收半径上界（m）
NEAR_R = 5.0              # 近场阈值（m）
R_CLEAR = 20.0            # 清除半径（m）
DIST_REF = 2.0 * R_ARENA  # 距离惩罚的归一化尺度（靶区直径，m）

DEFAULT_BASE_CSV = os.path.normpath(
    os.path.join(_HERE, "..", "out", "p2_grid", "p2_grid_map.csv"))
DEFAULT_FAMILY_DIR = os.path.normpath(os.path.join(_HERE, "..", "out", "p3_family"))
E_COL_CANDIDATES = ("E_diam_given_detectable_m", "E_diam_mixed_mm",
                    "E_diam_mixed_m")

# -----
# 基准图（第二问的网格模拟结果）
# -----

@dataclass
class BaseMap:
    """问题 2 的期望热力图：``E[j, i]`` 对应 ``(xs[i], ys[j])``，nan = 无数据。"""

    xs: np.ndarray
    ys: np.ndarray
    E: np.ndarray
    source: str = "p2_grid_expectation"
    e_col: str = "E_diam_given_detectable_m"

    # ----- 构造 -----

    @property
    def step(self) -> float:
```



```

        return float(self.xs[1] - self.xs[0])

@property
def extent(self) -> tuple:
    return (float(self.xs[0]), float(self.xs[-1]), float(self.ys[0]), float(self.ys[-1]))

@classmethod
def from_points(cls, X, Y, E, source="synthetic") -> "BaseMap":
    """由散点（规则网格掩码后）构造。"""
    X = np.asarray(X, float)
    Y = np.asarray(Y, float)
    E = np.asarray(E, float)
    xs = np.unique(np.round(X, 6))
    ys = np.unique(np.round(Y, 6))
    # 步长取最小正间隔（避免浮点噪声把网格拆散）
    dx = _grid_step(xs)
    dy = _grid_step(ys)
    xs = np.round(np.arange(xs[0], xs[-1] + 1e-9, dx), 6)
    ys = np.round(np.arange(ys[0], ys[-1] + 1e-9, dy), 6)
    G = np.full((ys.size, xs.size), np.nan)
    ix = np.searchsorted(xs, np.round(X, 6))
    iy = np.searchsorted(ys, np.round(Y, 6))
    ok = (ix < xs.size) & (iy < ys.size)
    G[iy[ok], ix[ok]] = E[ok]
    return cls(xs=x, ys=y, E=G, source=source)

@classmethod
def load_csv(cls, path=DEFAULT_BASE_CSV, e_col=None) -> "BaseMap":
    if not os.path.exists(path):
        raise FileNotFoundError(
            f"找不到第二问的网格模拟结果 {path}。请先运行\n"
            f"python src/p2_grid_expectation.py --step 40 --out out/p2_grid")
    with open(path, encoding="utf-8-sig") as f:
        rows = list(csv.DictReader(f))
    if not rows:
        raise ValueError(f"{path} 为空")
    col = e_col
    if col is None:
        for c in E_COL_CANDIDATES:
            if c in rows[0]:
                col = c
                break
    if col is None:
        raise ValueError(f"{path} 缺少期望值列，现有列: {list(rows[0])}")
    X = np.array([float(r["x_m"]) for r in rows])
    Y = np.array([float(r["y_m"]) for r in rows])
    E = np.array([_to_float(r.get(col)) for r in rows])
    m = cls.from_points(X, Y, E, source=os.path.basename(path))
    m.e_col = col
    return m

@classmethod
def synthetic(cls, step=100.0, R=R_ARENA) -> "BaseMap":
    """自检用解析基准图（形状与真实图同类：近轴退化带 + 斜前方最优翼）。"""
    xs = np.arange(-R, R + 1e-9, step)
    X, Y = np.meshgrid(xs, xs)
    r = np.hypot(X, Y)
    phi = np.abs(np.degrees(np.arctan2(Y, X)))
    E = 60.0 + 900.0 * (1.0 - np.exp(-r / 700.0)) + 700.0 * np.exp(-phi / 12.0)
    E[r > R] = np.nan
    return cls(xs=x, ys=y, E=E, source="synthetic")

```

```

# ----- 查询 -----
def lookup(self, X, Y, e_max=E_MAX):
    """双线性插值查询。返回 ``E, covered``；四点任一无数据即视为未覆盖。

    ``E`` 已按"未覆盖/超过上限一律取 e_max"的规则裁剪，可直接用于叠加。
    """
    X = np.asarray(X, float)
    Y = np.asarray(Y, float)
    shp = np.broadcast(X, Y).shape
    Xf = np.broadcast_to(X, shp).astype(float).ravel()
    Yf = np.broadcast_to(Y, shp).astype(float).ravel()
    x0, y0 = float(self.xs[0]), float(self.ys[0])
    dx, dy = self.step, float(self.ys[1] - self.ys[0])
    nx, ny = self.xs.size, self.ys.size
    fx = (Xf - x0) / dx
    fy = (Yf - y0) / dy
    i = np.floor(fx).astype(int)
    j = np.floor(fy).astype(int)
    inside = (i >= 0) & (i <= nx - 2) & (j >= 0) & (j <= ny - 2)
    ic = np.clip(i, 0, nx - 2)
    jc = np.clip(j, 0, ny - 2)
    tx = np.clip(fx - ic, 0.0, 1.0)
    ty = np.clip(fy - jc, 0.0, 1.0)
    G = self.E
    e00 = G[jc, ic]
    e10 = G[jc, ic + 1]
    e01 = G[jc + 1, ic]
    e11 = G[jc + 1, ic + 1]
    ok = inside & np.isfinite(e00) & np.isfinite(e10) & np.isfinite(e01) & np.isfinite(e11)
    val = (e00 * (1 - tx) * (1 - ty) + e10 * tx * (1 - ty)
           + e01 * (1 - tx) * ty + e11 * tx * ty)
    val = np.where(ok, val, np.nan)
    E_out = np.where(ok, np.minimum(val, e_max), e_max)
    return E_out.reshape(shp), ok.reshape(shp)

# ----- 变换 -----
def transformed_lookup(self, S, theta_deg, X, Y, e_max=E_MAX):
    """把基准图按"检测点 S + 示向度  $\theta$ "旋转平移后查询全局网格点 (X, Y)。

    ``b = R(- $\theta$ ) * (g - S)``：把全局点转回基准图坐标系（基准图的源在 +x 轴方向）。
    E[D] 在刚体变换下不变，故 ``E(g) = E_base(b)``。
    """
    S = np.asarray(S, float)
    c = math.cos(math.radians(theta_deg))
    s = math.sin(math.radians(theta_deg))
    Xf = np.asarray(X, float) - S[0]
    Yf = np.asarray(Y, float) - S[1]
    bx = Xf * c + Yf * s
    by = -Xf * s + Yf * c
    return self.lookup(bx, by, e_max=e_max)

def _grid_step(v):
    d = np.diff(np.asarray(v, float))
    d = d[d > 1e-9]
    return float(np.min(d)) if d.size else 1.0

def _to_float(v):
    if v is None or v == "" or v == "nan":
        return float("nan")
    try:

```

```

        return float(v)
    except (TypeError, ValueError):
        return float("nan")

def save_map_csv(path, X, Y, E, e_col="E_diam_given_detectable_m"):
    os.makedirs(os.path.dirname(os.path.abspath(path)), exist_ok=True)
    with open(path, "w", newline="", encoding="utf-8-sig") as f:
        w = csv.writer(f)
        w.writerow(["x_m", "y_m", e_col])
        for i in range(len(X)):
            v = E[i]
            w.writerow([round(float(X[i]), 4), round(float(Y[i]), 4),
                        "" if not np.isfinite(v) else round(float(v), 6)])]

    return path

# -----
# 基准图族: 按"先验外径 t_hi"参数化
# -----
class BaseMapFamily:
    """一族基准图 ``{t_hi: BaseMap}``。

    **为什么需要它（这一步不做对，整个"变换 + 插值"就不成立）**

    基准图里源位先验是"从检测点沿示向度射线、 $t \in [5, t_{hi}]$ 、权重  $\propto t$ "。
    真实场景中源还必须落在靶区内，所以射线上先验的**外径**  $t_{hi}$  随  $(S, \theta)$ 
    变化（射线出靶区的距离可以只有几百米）。而  $E[D]$  只依赖
    （相对位置  $b, t_{hi}$ ）这一对量，因此把  $t_{hi}$  也当作族参数，旋转 + 平移 + 插值才严格成立。

    查询时按  $t_{hi}$  在族内做线性插值（先把未覆盖格子填成  $e_{max}$ ，再混合）。
    """

    def __init__(self, maps, t_ref=R_RECV_MAX, e_max=E_MAX):
        self.maps = {float(k): v for k, v in sorted(maps.items())}
        self.t_list = np.array(sorted(self.maps.keys()))
        self.t_ref = float(t_ref)
        self.e_max = float(e_max)
        self._cache = {}

    # ----- 构造 -----
    @classmethod
    def build(cls, outdir, t_his=(150.0, 250.0, 350.0, 450.0, 600.0, 800.0, 1000.0,
                                   1250.0, 1500.0), step=40.0, n_t=200, verbose=True):
        """按 p2 的口径重算一族基准图并落盘。"""
        from p2_grid_expectation import ( # 局部导入，避免循环依赖
            expected_diameter_measured, make_grid, source_samples)

        X, Y, xs, ys, shape = make_grid(step=step, arena_r=R_ARENA, domain="arena")
        os.makedirs(outdir, exist_ok=True)
        maps = {}
        files = []
        for T in t_his:
            ts, w, info = source_samples((0.0, 0.0), 0.0, n_t=n_t, r_arena=float(T))
            res = expected_diameter_measured(X, Y, ts, w)
            E = np.asarray(res["E_det"], float)
            m = BaseMap.from_points(X, Y, E, source=f"p3_family_thi{T:g}")
            fn = os.path.join(outdir, f"p2_grid_map_thi{int(round(T)):04d}.csv")
            save_map_csv(fn, X, Y, E)
            maps[float(T)] = m
            files.append(os.path.basename(fn))
        if verbose:

```

```

        fin = E[np.isfinite(E)]
        print(f"    t_hi={T:6.0f} m -> {os.path.basename(fn)} "
              f"有效格 {fin.size}/{E.size}, 中位数 "
              f"{np.median(fin) if fin.size else float('nan'):.1f} m")
    man = {"kind": "p3_base_map_family", "step": float(step), "n_t": int(n_t),
           "arena_r": R_ARENA, "t_hi": [float(t) for t in t_hi], "files": files,
           "note": "每个文件是 S1=(0,0)、 $\theta_1=0$  下, 源位先验  $t \in [5, t_{hi}]$  的 E[D|可检测]}
    with open(os.path.join(outdir, "p2_grid_family.json"), "w", encoding="utf-8") as f:
        json.dump(man, f, ensure_ascii=False, indent=2)
    return cls(maps)

@classmethod
def load_dir(cls, outdir):
    man_path = os.path.join(outdir, "p2_grid_family.json")
    if not os.path.exists(man_path):
        return None
    with open(man_path, encoding="utf-8") as f:
        man = json.load(f)
    maps = {}
    for t, fn in zip(man["t_hi"], man["files"]):
        maps[float(t)] = BaseMap.load_csv(os.path.join(outdir, fn))
    return cls(maps)

@classmethod
def load_or_build(cls, outdir, **kw):
    fam = cls.load_dir(outdir)
    if fam is not None:
        return fam
    return cls.build(outdir, **kw)

# ----- 查询 -----
def select(self, t_hi):
    """返回该 t_hi 对应的（可缓存的）混合基准图。"""
    t = float(min(max(t_hi, self.t_list[0]), self.t_list[-1]))
    key = round(t / 25.0)
    if key in self._cache:
        return self._cache[key]
    tt = key * 25.0
    if tt <= self.t_list[0]:
        m = self.maps[float(self.t_list[0])]
    elif tt >= self.t_list[-1]:
        m = self.maps[float(self.t_list[-1])]
    else:
        i = int(np.searchsorted(self.t_list, tt) - 1)
        T1, T2 = float(self.t_list[i]), float(self.t_list[i + 1])
        a = (tt - T1) / max(T2 - T1, 1e-9)
        m = blend_maps(self.maps[T1], self.maps[T2], a, self.e_max)
    if len(self._cache) < 64:
        self._cache[key] = m
    return m

def rebase(self, S, theta_deg):
    """给出 (S,  $\theta$ ) 检测对应的基准图: 先按射线出靶区距离取 t_hi, 再在族内插值。"""
    S = np.asarray(S, float)
    u = np.array([math.cos(math.radians(theta_deg)), math.sin(math.radians(theta_deg))])
    b = float(S @ u)
    c = float(S @ S) - R_ARENA * R_ARENA
    disc = b * b - c
    exit_d = float(-b + math.sqrt(disc)) if disc > 0 else 0.0
    t_hi = min(R_RECV_MAX, exit_d)
    return self.select(t_hi), t_hi

```

```

def blend_maps(m1: BaseMap, m2: BaseMap, a, e_max=E_MAX) -> BaseMap:
    """把两张同网格基准图按权重 a 线性混合（未覆盖格子先填 e_max，偏保守）。"""
    a = float(min(max(a, 0.0), 1.0))
    E1 = np.where(np.isfinite(m1.E), m1.E, e_max)
    E2 = np.where(np.isfinite(m2.E), m2.E, e_max)
    covered = np.isfinite(m1.E) | np.isfinite(m2.E)
    E = np.where(covered, (1.0 - a) * E1 + a * E2, np.nan)
    return BaseMap(xs=m1.xs, ys=m1.ys, E=E, source=f"blend({m1.source},{m2.source},{a:.2f})")

# -----
# 决策场
# -----

@dataclass
class FieldConfig:
    grid_step: float = 100.0          # 全局网格步长 (m)
    e_max: float = E_MAX               # 未覆盖网格的默认最大值 (m)
    dist_weight: float = 0.35         # 距离项系数  $\lambda$  (可调)
    visited_radius: float = 250.0     # 已访问扫描点的排除半径 (m)
    min_move: float = 60.0            # 离当前位置太近的格子直接排除 (m)
    arena_r: float = R_ARENA
    normalize: str = "mean"           # mean (期望均值归一化) | max

class ExpectField:
    """全局网格上的"期望定位质量 + 距离"标量场。"""

    def __init__(self, base, cfg: FieldConfig | None = None):
        self.base = base
        self.family = base if isinstance(base, BaseMapFamily) else None
        self.cfg = cfg or FieldConfig()
        step = float(self.cfg.grid_step)
        R = float(self.cfg.arena_r)
        xs = np.arange(-R, R + 1e-9, step)
        GX, GY = np.meshgrid(xs, xs)
        m = np.hypot(GX, GY) <= R + 1e-9
        self.gx = GX[m]
        self.gy = GY[m]
        self.terms: list[tuple[np.ndarray, float, tuple]] = []

    # ----- 叠加 -----
    def add_bearing(self, S, theta_deg, weight=1.0, tag=None):
        """叠加一次检测 (S,  $\theta$ ) 给出的期望场（已裁剪：未覆盖 = e_max）。

        有基准图族时先用 (S,  $\theta$ ) 的"射线出靶区距离"选 t_hi，再在族内插值，
        这样"旋转 + 平移 + 插值"才是严格的刚体不变。
        """
        S = np.asarray(S, float)
        bm = self.base
        t_hi = R_RECV_MAX
        if self.family is not None:
            bm, t_hi = self.family.rebase(S, theta_deg)
        E, cov = bm.transformed_lookup(S, theta_deg, self.gx, self.gy,
                                       e_max=self.cfg.e_max)
        self.terms.append((E, float(weight),
                           (tuple(float(v) for v in S), float(theta_deg), tag, float(t_hi))))
        return cov

    def add_channel_bearings(self, bearings):
        """bearings = [(x, y, svd_deg), ...]"""
        for (x, y, th) in bearings:

```

```

        self.add_bearing((x, y), th)

def add_raw(self, values, weight=1.0, tag=None):
    """叠加一层已经算好的、直接定义在全局网格上的值（用于“探索项”）。"""
    v = np.asarray(values, float)
    if v.shape != self.gx.shape:
        v = np.broadcast_to(v, self.gx.shape)
    self.terms.append((v, float(weight), ("raw", tag)))
    return v

def discovery_term(self, meas_pts, min_dist_m, e_max=None):
    """探索项：某频道“在哪些格点上重新检测才有新信息”。

    对“还没听到过”的频道，若新检测点距它所有历史检测点都 < ``min_dist_m``,
    那么能新覆盖到的区域很小（有效接收半径至少 1000 m），几乎不可能听到 —— 记 e_max;
    反之记 0（无惩罚）。这样全局网格就会自动把扫描点推到“能发现新源”的地方。
    """
    e_max = float(self.cfg.e_max if e_max is None else e_max)
    if not meas_pts:
        return np.zeros(self.gx.shape)
    P = np.asarray(meas_pts, float)
    dmin = np.min(np.hypot(self.gx[:, None] - P[None, :, 0],
                          self.gy[:, None] - P[None, :, 1]), axis=1)
    return np.where(dmin >= float(min_dist_m), 0.0, e_max)

# ----- 聚合 / 打分 -----
def aggregate(self):
    """返回 ``(E_mean_m, coverage_count, n_terms)``。

    * 每一项先把未覆盖格子记成 ``e_max``（“最差位置”）；
    * ``normalize='mean'``：对各项取加权平均（即“总和归一化”）；
    * ``coverage_count``：至少有一项覆盖到的格子数。
    """
    n = self.gx.size
    if not self.terms:
        return (np.full(n, float(self.cfg.e_max)), np.zeros(n, dtype=int), 0)
    W = 0.0
    acc = np.zeros(n)
    cov = np.zeros(n, dtype=int)
    covered = np.zeros(n, dtype=bool)
    for (E, w, _tag) in self.terms:
        acc += w * E
        W += w
        covered |= (E < self.cfg.e_max - 1e-9)
        cov += (E < self.cfg.e_max - 1e-9).astype(int)
    E_mean = acc / max(W, 1e-12)
    if self.cfg.normalize == "max" and np.any(covered):
        ref = float(np.max(E_mean))
        E_mean = E_mean * (float(self.cfg.e_max) / max(ref, 1e-12))
        E_mean = np.minimum(E_mean, float(self.cfg.e_max))
    return E_mean, cov, len(self.terms)

def score(self, cur_xy, visited=(), extra_penalty=None, max_dist=None):
    """计算整张网格的 score = 归一化期望 +  $\lambda$  * 距离/靶区直径（排除项 = +inf）。"""
    E_mean, cov, n_terms = self.aggregate()
    e_norm = E_mean / max(float(self.cfg.e_max), 1e-12)
    if extra_penalty is not None:
        e_norm = e_norm + np.asarray(extra_penalty, float)
    p = np.asarray(cur_xy, float)
    d = np.hypot(self.gx - p[0], self.gy - p[1])
    score = e_norm + float(self.cfg.dist_weight) * d / DIST_REF
    mask = d < float(self.cfg.min_move)

```

```

    if max_dist is not None:
        mask |= d > float(max_dist)
    for q in visited:
        q = np.asarray(q, float)
        mask |= np.hypot(self.gx - q[0], self.gy - q[1]) < float(self.cfg.visited_radius)
    score = np.where(mask, np.inf, score)
    return score, {"E_mean_m": E_mean, "coverage": cov, "n_terms": n_terms,
                  "e_norm": e_norm, "dist_m": d, "excluded": mask}

# ----- 选点 -----
def argmin(self, cur_xy, visited=(), extra_penalty=None, max_dist=None):
    if not self.terms:
        return None # 一条方位都没有: 交给探索兜底
    score, info = self.score(cur_xy, visited, extra_penalty, max_dist=max_dist)
    if not np.any(np.isfinite(score)):
        return None
    i = int(np.argmin(np.where(np.isfinite(score), score, np.inf)))
    return self._describe(i, score[i], info)

def fallback_point(self, cur_xy, visited=(), prefer_near=True, max_dist=None):
    """探索兜底: 离所有已访问点"最远"的格子(最大最小距离准则)。

    一条方位都没有、或整张图都被排除时使用; 也用于避免反复在同一片区域打转。
    ``max_dist`` 给定时只在"还走得"的格子里挑(用于时间预算收尾阶段)。
    """
    p = np.asarray(cur_xy, float)
    if visited:
        V = np.asarray(visited, float)
        dmin = np.min(np.hypot(self.gx[:, None] - V[None, :, 0],
                               self.gy[:, None] - V[None, :, 1]), axis=1)
    else:
        dmin = np.hypot(self.gx - p[0], self.gy - p[1])
    dc = np.hypot(self.gx - p[0], self.gy - p[1])
    key = dmin - (1e-6 * dc if prefer_near else 0.0)
    key = np.where(dc < float(self.cfg.min_move), -np.inf, key)
    if max_dist is not None:
        key = np.where(dc > float(max_dist), -np.inf, key)
    if not np.any(np.isfinite(key)):
        key = dmin - 1e-6 * dc # 放宽: 忽略 max_dist/min_move 再挑一次
    i = int(np.argmax(key))
    E_mean, cov, n_terms = self.aggregate()
    info = {"E_mean_m": E_mean, "coverage": cov, "n_terms": n_terms,
           "e_norm": E_mean / max(float(self.cfg.e_max), 1e-12),
           "dist_m": dc, "excluded": np.zeros(self.gx.size, bool)}
    return self._describe(i, float("nan"), info, kind="explore")

def _describe(self, i, score, info, kind="min"):
    return {
        "kind": kind,
        "index": int(i),
        "xy": (float(self.gx[i]), float(self.gy[i])),
        "score": float(score),
        "E_mean_m": float(info["E_mean_m"][i]),
        "E_norm": float(info["e_norm"][i]),
        "dist_m": float(info["dist_m"][i]),
        "dist_penalty": float(self.cfg.dist_weight * info["dist_m"][i] / DIST_REF),
        "n_terms": int(info["n_terms"]),
        "n_cover": int(info["coverage"][i]),
        "covered": bool(info["coverage"][i] > 0),
    }

# ----- 导出 -----

```

```

def save_csv(self, path):
    E_mean, cov, n_terms = self.aggregate()
    score, info = self.score((0.0, 0.0))
    os.makedirs(os.path.dirname(os.path.abspath(path)), exist_ok=True)
    with open(path, "w", newline="", encoding="utf-8-sig") as f:
        w = csv.writer(f)
        w.writerow(["x_m", "y_m", "E_mean_m", "E_norm", "n_cover", "score_at_origin"])
        for k in range(self.gx.size):
            w.writerow([round(float(self.gx[k]), 3), round(float(self.gy[k]), 3),
                        round(float(E_mean[k]), 4), round(float(E_mean[k]) / self.cfg.e_max, 6),
                        int(cov[k]),
                        "" if not np.isfinite(score[k]) else round(float(score[k]), 6)])

    return path

# -----
# 精确期望（独立实现）：任意  $S_1/\theta_1$  下，候选第二检测点的  $E[D]$ 
# -----
def ray_source_samples(S, theta_deg, err=BEARING_ERR, n_t=200, r_arena=R_ARENA,
                      r_recv_max=R_RECV_MAX, near=NEAR_R):
    """源位先验：在  $S$  沿  $\theta$  的射线上、靶区内、可被有效接收的范围内按面积均匀采样。

    与 :func:`p2_grid_expectation.source_samples` 同一先验，但支持任意  $S$ （不只是原点）。
    返回 ``ts, w, t_hi``：``ts`` 为源到  $S$  的距离采样。
    """
    S = np.asarray(S, float)
    u = np.array([math.cos(math.radians(theta_deg)), math.sin(math.radians(theta_deg))])
    b = float(S @ u)
    c = float(S @ S) - r_arena * r_arena
    disc = b * b - c
    exit_d = float(-b + math.sqrt(disc)) if disc > 0 else 0.0
    t_hi = min(r_recv_max, exit_d)
    if t_hi <= near:
        return np.zeros(0), np.zeros(0), 0.0
    ts = np.linspace(near, t_hi, int(n_t))
    w = ts.copy()
    return ts, w / float(w.sum()), t_hi

def exact_expected_diameter(S, theta_deg, S2, ts, w, err=BEARING_ERR,
                            r_recv_max=R_RECV_MAX, per_sample_gate=False):
    """独立实现（用于自检）：直接枚举源位样本，用精确的两楔交会算  $E[D]$ 。

    与基准图完全同构（源位先验 = 射线上的面积均匀），但**不做任何插值/变换**，
    因此可以用来校验 "旋转平移 + 双线性插值" 的正确性。

    ``per_sample_gate=False``（默认，与 :mod:`p2_grid_expectation` 一致）：
    "可检测"门控按**候选点**判定 —— 只要该点能收到先验中某个源位的信号就保留全部样本；
    置 True 则逐样本判定 ``|S2 - G| <= 有效接收半径上界``。
    """
    from p2_grid_expectation import _fast_pair_grid # 局部导入，避免循环依赖

    S = np.asarray(S, float)
    S2 = np.atleast_2d(np.asarray(S2, float))
    ts = np.asarray(ts, float)
    w = np.asarray(w, float)
    m, n = S2.shape[0], ts.size
    if n == 0:
        return np.full(m, np.nan)
    u = np.array([math.cos(math.radians(theta_deg)), math.sin(math.radians(theta_deg))])
    G = S[None, :] + ts[:, None] * u[None, :] # (n,2) 源位样本
    th2 = np.degrees(np.arctan2(G[None, :, 1] - S2[:, None, 1],

```



```

        G[None, :, 0] - S2[:, None, 0])) % 360.0
# 注意 _fast_pair_grid 的形状约定: 坐标数组要比 th 少一个尾维 ((m,2) 对 (m,n))
S1b = np.broadcast_to(S.reshape(1, 2), (m, 2))
th1b = np.full((m,), float(theta_deg))
S2b = S2
D, _st = _fast_pair_grid(S1b, th1b, S2b, th2, err)
d2 = np.hypot(S2[:, None, 0] - G[None, :, 0], S2[:, None, 1] - G[None, :, 1])
if per_sample_gate:
    reach = d2 <= r_recv_max + 1e-9
else:
    reach = np.broadcast_to((d2 <= r_recv_max + 1e-9).any(axis=1)[: , None], D.shape)
ok = np.isfinite(D) & reach
wc = np.where(ok, w[None, :], 0.0)
sw = wc.sum(axis=1)
num = (np.where(ok, D, 0.0) * w[None, :]).sum(axis=1)
with np.errstate(invalid="ignore"):
    return np.where(sw > 0, num / np.maximum(sw, 1e-12), np.nan)

def best_refine_point(rec_pts, rec_svds, cur_xy, region, cfg: FieldConfig | None = None,
                    n_t=120, n_ang=16, radii=(100.0, 200.0, 300.0, 450.0, 600.0, 800.0),
                    travel_cap_m=900.0, err=BEARING_ERR):
    """在"已有一条/多条方位"的基础上, 选一个**局部**最优的下一检测点。

    与 :class:`ExpectField` (全局、用预计算基准图) 互补: 这里对任意  $S1/\theta1$  直接算
    精确的  $E[D]$ , 用于机器狗已经逼近目标、或刚发现一个新频道需要"再测一条方位把区域
    定下来"的场合。候选点取以当前位置为中心、半径 ``radii`` 的极角网格,
    代价 = 走到候选点 + 检测 + 走到目标区域 + 预期覆盖式清扫时间。

    ``region`` 为 ``{'center': (x, y)}``: 为 ``None`` (只有一条方位) 时用源位先验的
    中位点作为"目标区域", 从而把"测完还要往回走多远"算进代价。

    返回 ``(xy, info)``: 无可用候选时返回 ``(None, None)``。
    """
    pts = [np.asarray(p, float) for p in rec_pts]
    if not pts:
        return None, None
    S1 = pts[-1]
    th1 = float(rec_svds[-1])
    ts, w, t_hi = ray_source_samples(S1, th1, n_t=n_t)
    if ts.size == 0:
        return None, None
    cur = np.asarray(cur_xy, float)
    ang0 = math.degrees(math.atan2(cur[1] - S1[1], cur[0] - S1[0]))
    cand = [cur]
    for r in radii:
        for k in range(n_ang):
            a = math.radians(ang0 + 360.0 * k / n_ang)
            p = cur + r * np.array([math.cos(a), math.sin(a)])
            if np.hypot(*p) <= R_ARENA - 1.0:
                cand.append(p)
    C = np.array(cand, float)
    keep = np.hypot(C[:, 0] - cur[0], C[:, 1] - cur[1]) <= travel_cap_m
    C = C[keep]
    if C.size == 0:
        return None, None
    E = exact_expected_diameter(S1, th1, C, ts, w, err=err)
    dcur = np.hypot(C[:, 0] - cur[0], C[:, 1] - cur[1])
    if region is not None:
        c = np.asarray(region["center"], float)
        back_target = "region"
    else:

```

```

        u = np.array([math.cos(math.radians(th1)), math.sin(math.radians(th1))])
        c = S1 + float(ts[ts.size // 2]) * u          # 先验中位源位
        back_target = "prior_median"
        dreg = np.hypot(C[:, 0] - c[0], C[:, 1] - c[1])
        # 代价 (秒) = 走到候选点 + 检测 + 走到目标 + 按 E[D] 估计的覆盖式清扫
        # 清扫路程 ≈ 圆域面积 / 覆盖行距 =  $\pi \cdot (r)^2 / (2 \cdot 20)$ , r 取 E[D]/2 + 20 m
        v = 5.0
        sweep_m = np.pi * np.maximum(E / 2.0 + 20.0, 20.0) ** 2 / (2.0 * R_CLEAR)
        cost = dcur / v + 6.0 + dreg / v + np.where(np.isfinite(E), sweep_m / v, 1e6)
        i = int(np.argmin(cost))
        info = {"S1": tuple(float(v) for v in S1), "th1": th1,
                "xy": (float(C[i, 0]), float(C[i, 1])),
                "E_diam_m": float(E[i]), "cost_s": float(cost[i]),
                "move_s": float(dcur[i] / v), "back_s": float(dreg[i] / v),
                "n_cand": int(C.shape[0]), "t_hi": float(t_hi), "back_target": back_target}
        return (float(C[i, 0]), float(C[i, 1])), info

def _ring_candidates(S, theta_deg, radii=(400.0, 800.0, 1200.0), n_ang=8,
                    dphi0=30.0, arena_r=R_ARENA):
    """绕检测点 S 的环形候选点 (相对  $\theta$  偏开 dphi0, 避免正落在退化射线上)。"""
    S = np.asarray(S, float)
    C = []
    for r in radii:
        for k in range(n_ang):
            a = math.radians(theta_deg + dphi0 + (360.0 - 2 * dphi0) * k / max(n_ang - 1, 1))
            p = S + r * np.array([math.cos(a), math.sin(a)])
            if np.hypot(*p) <= arena_r - 1.0:
                C.append(p)
    return np.array(C) if C else np.zeros((0, 2))

# -----
# 自检
# -----

def selfcheck(base_csv=DEFAULT_BASE_CSV, verbose=True, seed=20260913):
    out = []
    ok_all = True

    def rec(name, ok, detail=""):
        nonlocal ok_all
        ok_all = ok_all and bool(ok)
        out.append({"check": name, "ok": bool(ok), "detail": detail})
        if verbose:
            print(f"[{'OK' if ok else 'FAIL'}] {name} {detail}")

    rng = np.random.default_rng(seed)
    base = BaseMap.synthetic(step=50.0)
    E, cov = base.lookup(0.0, 0.0)
    rec("T1 基准图基本可查 (原点有值)", bool(cov) and np.isfinite(float(E)),
        f"E(0,0)={float(E):.3f}, covered={bool(cov)}")

    # T2 旋转平移不变性: 把基准网格点做刚体变换后查回来, 应完全一致
    S = np.array([137.0, -421.0])
    th = 37.0
    c, s = math.cos(math.radians(th)), math.sin(math.radians(th))
    worst = 0.0
    for (x, y) in ((-500.0, 300.0), (0.0, 0.0), (1000.0, -800.0), (-1200.0, -1200.0)):
        e0, c0 = base.lookup(x, y)
        if not bool(c0):
            continue
        g = S + np.array([x * c - y * s, x * s + y * c])          # 正向变换到全局

```

```

    e1, c1 = base.transformed_lookup(S, th, g[0], g[1])
    worst = max(worst, abs(float(e1) - float(e0)))
rec("T2 旋转+平移不变性 (刚体变换下 E 不变)", worst < 1e-9,
    f"最大绝对差 {worst:.3e}")

# T3 双线性插值精度: 中心值应落在四邻点区间内, 且与真值相差小
bad = 0
worst_rel = 0.0
for _ in range(400):
    i = int(rng.integers(1, base.xs.size - 2))
    j = int(rng.integers(1, base.ys.size - 2))
    if not np.all(np.isfinite(base.E[j - 1:j + 2, i - 1:i + 2])):
        continue
    x = float(base.xs[i]) + float(rng.uniform(-0.49, 0.49)) * base.step
    y = float(base.ys[j]) + float(rng.uniform(-0.49, 0.49)) * base.step
    v, cv = base.lookup(x, y)
    if not bool(cv):
        continue
    nb = base.E[j - 1:j + 2, i - 1:i + 2]
    if not (nb.min() - 1e-9 <= float(v) <= nb.max() + 1e-9):
        bad += 1
    # 与最近邻的差不应超过 1 个格距内的局部变化
    worst_rel = max(worst_rel, abs(float(v) - base.E[j, i]) / max(abs(base.E[j, i]), 1e-9))
rec("T3 双线性插值落在邻域值域内", bad == 0,
    f"越界 {bad} 次; 与中心格最大相对差 {worst_rel:.3f}")

# T4 未覆盖 / 超上限 -> e_max
v_out, c_out = base.lookup(5000.0, 0.0)
rec("T4 靶区外/无数据 -> 默认最大值 e_max",
    (not bool(c_out)) and abs(float(v_out) - E_MAX) < 1e-9,
    f"lookup(5000,0)={float(v_out)}, covered={bool(c_out)}")

# T5 叠加 + 归一化 + 距离惩罚的结构性质
f = ExpectField(base, FieldConfig(grid_step=100.0, dist_weight=0.35))
d0 = f.argmin((0.0, 0.0))
fb = f.fallback_point((0.0, 0.0), visited=[(0.0, 0.0)])
rec("T5a 无任何方位时退化为探索点 (离已访问点最远)",
    d0 is None and fb["kind"] == "explore"
    and math.hypot(fb["xy"][0], fb["xy"][1]) > 1200.0,
    f"argmin=None, 探索点 {np.round(fb['xy'], 1).tolist()} (半径 "
    f"{math.hypot(*fb['xy']):.0f} m)")
for (x, y, t) in ((0.0, 0.0, 0.0), (0.0, 0.0, 90.0)):
    f.add_bearing((x, y), t)
sel = f.argmin((0.0, 0.0), visited=[(0.0, 0.0)])
sc, info = f.score((0.0, 0.0), visited=[(0.0, 0.0)])
i = sel["index"]
rec("T5b argmin 与全网格暴力最小一致",
    abs(float(sc[i]) - float(np.min(sc))) < 1e-9,
    f"argmin score={sel['score']:.6f}")
rec("T5c 归一化性质: E_norm ∈ [0,1] 且未覆盖格子=1",
    bool(np.all(info["e_norm"] <= 1.0 + 1e-9)) and
    float(np.max(info["e_norm"][info["coverage"] == 0])) == 1.0,
    f"max E_norm={float(np.max(info['e_norm'])):.4f}")
# 距离项的定量性质: score ≡ E_norm + λ * d / (2R_arena), 且 λ=0 时与距离无关
c0 = FieldConfig(grid_step=100.0, dist_weight=0.35, min_move=0.0)
f1 = ExpectField(base, c0)
for (x, y, t) in ((0.0, 0.0, 0.0), (0.0, 0.0, 90.0)):
    f1.add_bearing((x, y), t)
sc1, inf1 = f1.score((120.0, -340.0))
manual = inf1["e_norm"] + 0.35 * inf1["dist_m"] / DIST_REF
f0 = ExpectField(base, FieldConfig(grid_step=100.0, dist_weight=0.0, min_move=0.0))
for (x, y, t) in ((0.0, 0.0, 0.0), (0.0, 0.0, 90.0)):

```

```

f0.add_bearing((x, y), t)
sc0, _ = f0.score((120.0, -340.0))
rec("T5d 距离层: score = E_norm +  $\lambda \cdot d / (2R_{\text{arena}})$  ( $\lambda=0$  时与距离无关)",
    bool(np.all(np.isfinite(sc1)))
    and float(np.max(np.abs(sc1 - manual))) < 1e-12
    and float(np.max(np.abs(sc0 - inf1["e_norm"]))) < 1e-12,
    f"最大偏差 {float(np.max(np.abs(sc1 - manual))):.3e}")

# T6 与精确实现交叉校验 (用基准图族; 含"射线被靶区截断"的病态情形)
fam = BaseMapFamily.load_dir(DEFAULT_FAMILY_DIR) if os.path.isdir(DEFAULT_FAMILY_DIR) else None
rec("T6a 基准图族可用 (按先验外径 t_hi 分族)", fam is not None,
    "族目录 " + DEFAULT_FAMILY_DIR if fam is not None
    else f"未找到 {DEFAULT_FAMILY_DIR} (可运行 --build-family 生成); "
    f"此时只能对 t_hi=1500 的几何严格成立")

def _cmp(rb_or_fam, cases, label):
    rels, n_local = [], 0
    worst = None
    for (Sd, thd, C) in cases:
        ts, w, _ = ray_source_samples(Sd, thd, n_t=200)
        if ts.size == 0:
            continue
        E_ex = exact_expected_diameter(Sd, thd, C, ts, w)
        if rb_or_fam is None:
            continue
        if isinstance(rb_or_fam, BaseMapFamily):
            rb, _t = rb_or_fam.rebase(Sd, thd)
        else:
            rb = rb_or_fam
        E_bm, _cov = rb.transformed_lookup(Sd, thd, C[:, 0], C[:, 1])
        u = np.array([math.cos(math.radians(thd)), math.sin(math.radians(thd))])
        G = np.asarray(Sd)[None, :] + ts[:, None] * u[None, :]
        for k in range(C.shape[0]):
            if not np.isfinite(E_ex[k]) or not np.isfinite(E_bm[k]):
                continue
            u1 = G - np.asarray(Sd)[None, :]
            u2 = G - C[k][None, :]
            cosang = (u1 * u2).sum(1) / np.maximum(
                np.linalg.norm(u1, axis=1) * np.linalg.norm(u2, axis=1), 1e-9)
            ang = np.degrees(np.arccos(np.clip(cosang, -1.0, 1.0)))
            gmin = min(float(np.min(ang)), float(np.min(180.0 - ang)))
            if gmin < 15.0:  # 近简并: E 是尖峰, 插值无意义 (决策场里会截断成 e_max)
                continue
            rel = abs(float(E_bm[k]) - float(E_ex[k])) / max(float(E_ex[k]), 1e-9)
            rels.append(rel)
            n_local += 1
            if worst is None or rel > worst[0]:
                worst = (rel, float(E_bm[k]), float(E_ex[k]),
                        float(np.hypot(*(C[k] - Sd))))
        if not rels:
            return None
    return {"n": n_local, "med": float(np.median(rels)),
            "p90": float(np.percentile(rels, 90)), "max": max(rels),
            "worst": worst, "label": label}

# 生成两组几何: S 在靶区中部 (先验基本不被截断) 与 S 贴近靶区边界且朝外测 (被截断)
cases_mid, cases_edge = [], []
for _ in range(6):
    Sd = np.round(rng.uniform(-700, 700, 2), 1)
    thd = float(rng.uniform(0, 360))
    cases_mid.append((Sd, thd, _ring_candidates(Sd, thd)))
for _ in range(6):

```

```

a = float(rng.uniform(0, 360))
Sd = 1560.0 * np.array([math.cos(math.radians(a)), math.sin(math.radians(a))])
thd = float((a + rng.uniform(-25, 25)) % 360.0) # 朝靶区外测
cases_edge.append((Sd, thd, _ring_candidates(Sd, thd)))
cases = [(S, t, C) for (S, t, C) in cases_mid + cases_edge if len(C)]

r_fam = _cmp(fam, cases, "family") if fam is not None else None
r_plain = _cmp(BaseMap.load_csv(base_csv) if os.path.exists(base_csv) else None,
               cases, "plain1500") if os.path.exists(base_csv) else None
if r_fam is not None:
    ok6 = (r_fam["med"] < 0.01) and (r_fam["p90"] < 0.05)
    detail6 = (f"{r_fam['n']}" 个良态候选点 (含边界截断几何): 相对差 中位数 "
               f"{r_fam['med']:.4f}、P90 {r_fam['p90']:.4f}、最大 {r_fam['max']:.4f}")
    if r_plain is not None:
        detail6 += (f": 不用族 (只按 t_hi=1500 的图) 时中位数 {r_plain['med']:.4f}、"
                    f"最大 {r_plain['max']:.4f}")
elif r_plain is not None:
    ok6 = (r_plain["med"] < 0.01) and (r_plain["p90"] < 0.05)
    detail6 = (f"{r_plain['n']}" 个良态候选点: 中位数 {r_plain['med']:.4f}、"
               f"P90 {r_plain['p90']:.4f}")
else:
    ok6, detail6 = True, "跳过 (既无基准图族也无单张基准图)"
rec("T6b 变换+插值 ≈ 精确期望 (良态几何)", ok6, detail6)

# T7 反例: 忽略 t_hi (只用 t_hi=1500 的图) 在边界朝外几何上会明显失真
if fam is not None and r_plain is not None and r_fam is not None:
    edge_rel = [x for x in
                 [_cmp(fam, [(S, t, C)], "f") for (S, t, C) in cases_edge
                  if len(C)] if x]
    edge_plain = [x for x in
                  [_cmp(BaseMap.load_csv(base_csv), [(S, t, C)], "p")
                   for (S, t, C) in cases_edge if len(C)] if x]
    if edge_rel and edge_plain:
        med_f = float(np.median([e["med"] for e in edge_rel]))
        med_p = float(np.median([e["med"] for e in edge_plain]))
        rec("T7 边界朝外几何: 按 t_hi 分族显著优于固定 t_hi=1500",
            med_f < 0.05 and med_f < med_p / 5.0,
            f"族: 中位相对差 {med_f:.4f}; 固定 1500: {med_p:.4f} ("
            f"差 {med_p / max(med_f, 1e-6):.0f} 倍)")

if verbose:
    print(f"\n自检总结: {'全部通过' if ok_all else '存在失败项'}")
return {"ok": ok_all, "checks": out}

# -----
# 演示 / 命令行
# -----
def field_figure(f, cur_xy, sel, path, title=None):
    """把决策场画出来: 叠加后的 E 场、距离惩罚、合成 score、选点与探测器位置。

    图注用英文 (本机 matplotlib 无中文字体, 避免缺字告警)。
    """
    import matplotlib
    matplotlib.use("Agg")
    import matplotlib.pyplot as plt

    E_mean, cov, n_terms = f.aggregate()
    score, info = f.score(cur_xy)
    fig, axes = plt.subplots(1, 3, figsize=(16.5, 5.6))
    panels = [(E_mean, "E[intersection diameter] (m)\n(uncovered cells = 1500)",
               "viridis_r"),

```

```

        (np.where(np.isfinite(info["dist_m"]),
                    info["dist_m"] / DIST_REF, np.nan),
         f"distance term  $\lambda \cdot d/(2R)$ ,  $\lambda=\{f.cfg.dist\_weight\}$ ", "magma"),
        (np.where(np.isfinite(score), score, np.nan),
         "score = normalised E + distance term\n( $\Delta$  = argmin)", "cividis_r")]
for ax, (V, ttl, cmap) in zip(axes, panels):
    sc = ax.scatter(f.gx, f.gy, c=V, s=26, cmap=cmap, marker="s")
    fig.colorbar(sc, ax=ax, shrink=0.8)
    ax.set_aspect("equal")
    ax.set_title(ttl, fontsize=10)
    ax.set_xlim(-1950, 1950)
    ax.set_ylim(-1950, 1950)
    ax.set_xlabel("x (m)")
    ax.set_ylabel("y (m)")
    th = np.linspace(0, 2 * math.pi, 400)
    ax.plot(f.cfg.arena_r * np.cos(th), f.cfg.arena_r * np.sin(th), "k--", lw=0.8)
    ax.plot([cur_xy[0]], [cur_xy[1]], "r*", ms=14)
    for (S, thd, _tag, _thi) in [t[2] for t in f.terms]:
        a = math.radians(thd)
        ax.plot([S[0], S[0] + 300 * math.cos(a)],
                [S[1], S[1] + 300 * math.sin(a)], "w-", lw=1.2, alpha=0.8)
    if sel is not None:
        ax.plot([sel["xy"][0]], [sel["xy"][1]], "w^", ms=11, mec="k")
    ax.grid(alpha=0.2)
fig.suptitle(title or (f"Q3 decision field: {n_terms} bearing/discovery terms "
                      f"superimposed on the global grid"), fontsize=11)
fig.tight_layout()
os.makedirs(os.path.dirname(os.path.abspath(path)), exist_ok=True)
fig.savefig(path, dpi=150)
plt.close(fig)
return path

def demo(base_csv=DEFAULT_BASE_CSV, outdir=None, dist_weight=1.5, grid_step=100.0,
         bearings=((0.0, 0.0, 0.0),), verbose=True, family_dir=DEFAULT_FAMILY_DIR):
    fam = BaseMapFamily.load_dir(family_dir) if os.path.isdir(family_dir) else None
    base = fam if fam is not None else BaseMap.load_csv(base_csv)
    cfg = FieldConfig(grid_step=grid_step, dist_weight=dist_weight)
    f = ExpectField(base, cfg)
    for (x, y, t) in bearings:
        f.add_bearing((x, y), t)
    sel = f.argmin((0.0, 0.0), visited=[(0.0, 0.0)])
    if verbose:
        kind = "基准图族" if fam is not None else f"单张基准图 {base.source}"
        print("基准: ", kind)
        print(f"叠加 {len(f.terms)} 条方位; 选点 {np.round(sel['xy'], 1).tolist()}, "
              f"E_mean={sel['E_mean']:.1f} m, E_norm={sel['E_norm']:.3f}, "
              f"距离 {sel['dist_m']:.0f} m, score={sel['score']:.4f}")
    if outdir:
        os.makedirs(outdir, exist_ok=True)
        p = f.save_csv(os.path.join(outdir, "p3_field.csv"))
        with open(os.path.join(outdir, "p3_field_report.json"), "w", encoding="utf-8") as fh:
            json.dump({"cfg": cfg.__dict__, "terms": len(f.terms), "pick": sel},
                      fh, ensure_ascii=False, indent=2)
        try:
            fig = field_figure(f, (0.0, 0.0), sel,
                              os.path.join(outdir, "p3_field.png"))
        except Exception as e:
            # 画图失败不影响主流程
            fig = f"figure_error: {e!r}"
    if verbose:
        print("写出", p, "与", fig)
    return f, sel

```

```

return f, sel

def main():
    ap = argparse.ArgumentParser(description="问题3 期望决策场")
    ap.add_argument("--selfcheck", action="store_true")
    ap.add_argument("--demo", action="store_true")
    ap.add_argument("--build-family", action="store_true",
                    help="按第二问口径重算一族基准图（不同先验外径 t_hi）")
    ap.add_argument("--base-map", default=DEFAULT_BASE_CSV)
    ap.add_argument("--family-dir", default=DEFAULT_FAMILY_DIR)
    ap.add_argument("--family-step", type=float, default=40.0)
    ap.add_argument("--family-nt", type=int, default=200)
    ap.add_argument("--out", default=None)
    ap.add_argument("--dist-weight", type=float, default=1.5,
                    help="距离项系数  $\lambda$ （问题 3 策略默认 1.5）")
    ap.add_argument("--grid-step", type=float, default=100.0)
    args = ap.parse_args()
    if args.selfcheck:
        res = selfcheck(args.base_map)
        raise SystemExit(0 if res["ok"] else 1)
    if args.build_family:
        print(f"计算基准图族 -> {args.family_dir}")
        BaseMapFamily.build(args.family_dir, step=args.family_step, n_t=args.family_nt)
    if args.demo or not args.build_family:
        demo(args.base_map, args.out, args.dist_weight, args.grid_step,
             family_dir=args.family_dir)

if __name__ == "__main__":
    main()

```

5.5 src/p3_robot.py

"""B 题问题 3：机器狗"基于期望的行动"在线搜索一定位—清除策略。

策略流程（与设计说明一一对应）

1. **第一轮扫描**：机器狗从原点出发，测向机初始频道 1，按频道 1..20 依次 ``/measure``，得到若干条"检测点 + 示向度"（部分频道无信号属正常）。
2. **期望场选点**：对每个"只测到一次方位"的频道，用第二问的网格模拟结果（``out/p3\_family`` 或 ``out/p2\_grid``）做旋转 + 平移 + 双线性插值，把 $\mathbb{E}[D]$ 叠加到全局网格上；没有数据/超阈值的格子取最大值 1500 m；归一化后再叠加一层**与探测器距离线性增长**的代价（系数 λ 可调），取 ``argmin`` 作为下一个检测点，随后开始新一轮 20 频道扫描。
3. **清理**：某些频道已经测到 ≥ 2 条方位 —— 用问题 1 的交会算法算出定位区域（最小包围圆 = 中心 + 半径 r^* ）。对所有可清理的频道按最近邻顺序规划航路，依次前往；到达区域后继续检测把区域压小，直到进入清除范围，再用 **覆盖式试探**（六边形格点，间距 $\leq 20\sqrt{3}$ m，保证圆域内任一点到某个试探点 ≤ 20 m）逐个 ``/clear``，必然有一点成功。
4. **清除后继续扫描**：已经清除的频道、已经定位够的频道、以及"在同一个有效接收半径内重复测过的频道"都会被跳过，从而节省时间。
5. 反复执行 2-4，直到时间预算耗尽或没有新的可用信息。

关键设计

- * **为什么"未覆盖 = 1500 m"是对的**： $\mathbb{E}[D]$ 越小定位越好；把没有信息的格子记成最差，只有在附近确实没有更好格子时才会选它 —— 这天然形成了"顺路探索"的行为。
- * **为什么不能沿示向度方向逼近**：那会退化成问题 1 的"无界区域"几何。所以清理时的侧移精定位点取在**与上一条视线近似正交**的方向上。

```

* **清理阶段的覆盖式试探是"保证成功"的**: 真源必落在  $\pm 1^\circ$  锥的交集（定位区域）内，
    区域内任意一点到某个试探点的距离  $\leq 20\text{ m}$  = 清除半径。
"""
from __future__ import annotations

import argparse
import json
import math
import os
import sys
import time
from dataclasses import dataclass, field as dc_field

import numpy as np

_HERE = os.path.dirname(os.path.abspath(__file__))
if _HERE not in sys.path:
    sys.path.insert(0, _HERE)

from p1_intersection import analyse # noqa: E402
from p3_arena import ( # noqa: E402
    ArenaError, BEARING_ERR, CHANNELS, MockArena, NEAR_R, R_ARENA, R_CLEAR,
    T_CLEAR_FAIL, T_CLEAR_OK, T_MEASURE, T_SWITCH, V_ROBOT,
)
from p3_expect_field import ( # noqa: E402
    DEFAULT_BASE_CSV, DEFAULT_FAMILY_DIR, BaseMap, BaseMapFamily, ExpectField,
    FieldConfig, best_refine_point,
)

# -----
# 覆盖式试探的三种结局
# -----
# 必须区分"预算不够、一个试探点都没来得及发"与"试探了但没打中":
# 前者不是策略失败（不该累加 clear_fail、不该写进论文的失败归因），
# 后者才是真正的"区域内没找到目标"。
SWEEP_OK = "success" # 某一点 /clear 成功
SWEEP_MISS = "miss" # 试探了全部候选点（或中途超时）都没打中
SWEEP_BUDGET = "no_budget" # 剩余预算不足，**一个 /clear 都没发出**，主动放弃

# -----
# 配置
# -----
@dataclass
class P3Config:
    # ---- 期望场（全局网格 + 距离惩罚） ----
    grid_step: float = 100.0 # 全局网格步长（m）
    e_max: float = 1500.0 # 未覆盖网格的默认最大值（m）
    dist_weight: float = 1.5 # 距离项系数  $\lambda$ （离线演练调参得到，见 review/P3-design.md）
    visited_radius: float = 250.0 # 已扫描点邻域排除半径（m）
    min_move: float = 60.0 # 距当前位置过近的格子排除（m）
    field_terms: str = "pending" # pending: 只叠加"还缺方位"的频道
    field_discovery: bool = False # 始终叠加"探索项"（把扫描点推向能发现新源的地方）
    auto_discovery_when_empty: bool = True # 场上一条可用方位都没有时自动启用探索项

    # ---- 扫描 ----
    channels: tuple = CHANNELS
    scan_bearing_first: bool = True
    repeat_gap_m: float = 60.0 # 同一频道在附近重复测量的最小间隔（m）
    nosignal_gap_m: float = 1000.0 # "无信号"频道要换足够远的地方重测
    # (= 有效接收半径下界: 在 1000 m 外重测才有

```



```

# 真正的新信息；实测 450→1000 使检测次数
# 289→186、平均定位清除时间 398→315 s，
# 清除率 0.992→0.995，见 review/P3-design.md §5.1) (m)
enroute_scan: bool = True # 清理途中顺路补测
enroute_scan_max: int = 2
second_bearing_refine: bool = False # 新发现频道立刻补第二条方位（实测收益不稳，默认关）
second_bearing_max: int = 2 # 每个停点最多补几条
second_bearing_budget_s: float = 320.0 # 剩余时间低于它就不再补测
max_rounds: int = 30
tour: str = "nn2opt" # 清理航路: nn | nn2opt
dynamic_replan: bool = True # 清理阶段每步重规划（滚动时域）；False=阶段内固定顺序
clear_policy: str = "tour" # tour (NN+2-opt 全局结构) | rollout (一步前瞻+SPT)
clear_budget_truncation: bool = False # 是否按剩余预算裁剪航路（实测关掉更好）
optimistic_probe_steps: int = 3 # 打开裁剪时的试探点数封顶
min_scan_rounds_before_clear: int = 1 # 先扫描几轮再开始清理

# ---- 定位与清除 ----
clear_min_bearings: int = 2
clear_max_radius_m: float = 600.0 # 定位区域半径超过它就先不清理
# (400 → 600: 扫圈在 r≈1100 m 处交会、区域半径
# 常在 400 - 650 m, 400 会让这些源被"跳过";
# 600 实测把扫圈的清除率从 0.958 提到 0.989 - 1.000)

arrive_radius_m: float = 30.0
direct_probe_radius_m: float = 60.0 # r* 不超过它就无需侧移精定位，直接试探
probe_spacing_m: float = 34.0 # ≤ 20√3 ≈ 34.6 m 才能保证覆盖
probe_max_points: int = 60 # 候选点数上限（仅"可覆盖"档位，见下）
probe_unlimited: bool = True # True=不按点数截断（保证覆盖；截断会破坏 20 m 保证）
probe_max_radius_m: float = 200.0
refine_leg_m: float = 320.0
refine_travel_cap_m: float = 700.0
refine_mode: str = "perp" # perp（正交侧移）| exact（精确期望最优）
max_approach_steps: int = 3

# ---- 时间预算 ----
time_reserve_s: float = 3.0
min_action_budget_s: float = 10.0
min_scan_channels: int = 2 # 单轮扫描至少要能测这么多频道才值得跑一趟

# ---- 预算感知的扫描闸门（"剩余时间不够就不要再扫"） ----
# 诊断依据: 30 局里扫描阶段吃掉约 1016 s / 1200 s，而末尾有 23% 的未清除源
# **已经定位好却没时间走过去**。所以关键不是"怎么选扫描点"，而是"还要不要扫"。
scan_budget_gate: bool = False # 开: 本轮扫描后必须还够清最近的一个目标，否则不扫
scan_gate_margin: float = 0.0 # 额外预留秒数（越大越倾向"别扫了，去清除"）
scan_gate_factor: float = 1.0 # 对"清最近目标"耗时估计的放大系数
scan_only_toward_targets: bool = False # 扫描点若背着已知目标走，则该轮不扫
# 扫描时间的硬性上限（占总预算比例）。诊断显示扫描吃掉约 1016 s / 1200 s，
# 而 29/49 个"已定位未清除"的目标在变成可清目标时**剩余时间完全够用**却没去清。
scan_budget_frac: float = 1.0 # 1.0 = 不限制；0.5 = 扫描累计不超过一半预算

# ---- 清理航路: 可达近点子集（"优先清路程近的点"） ----
clear_shortlist: bool = False # 开: 只保留"预算内可达"的近点子集，再排序
clear_shortlist_reserve_s: float = 0.0 # 为后续扫描预留的秒数

# -----
# 频道台账
# -----

@dataclass
class ChannelRec:
    channel: int
    pts: list = dc_field(default_factory=list) # 测到方位的检测点
    svds: list = dc_field(default_factory=list) # 对应的示向度

```

```

meas_pts: list = dc_field(default_factory=list) # 所有检测过的位置 (含无信号)
meas_log: list = dc_field(default_factory=list) # (x, y, virtual_time, result)
meas_stations: list = dc_field(default_factory=list) # 测过该频道的**站位编号**
                                                    # (问题 4 的覆盖记账用: 只有"这一站
                                                    # 本来能听到它"才抵扣覆盖)

cleared: bool = False
cleared_at: float | None = None
near_hits: int = 0
no_signal: int = 0
clear_fail: int = 0 # 自最近一次"新增方位"以来清理失败的次数
last_bounded: tuple | None = None # (center, radius, n_bearings) 最近一次有界区域
tried_probes: list = dc_field(default_factory=list)
history: list = dc_field(default_factory=list)

@property
def n_bearings(self) -> int:
    return len(self.pts)

# -----
# 机器狗
# -----
def load_base(family_dir=DEFAULT_FAMILY_DIR, base_csv=DEFAULT_BASE_CSV):
    """优先用"基准图族" (按先验外径 t_hi 分族, 变换才严格成立), 否则退回单张基准图。"""
    if family_dir and os.path.isdir(family_dir):
        fam = BaseMapFamily.load_dir(family_dir)
        if fam is not None:
            return fam
    if os.path.exists(base_csv):
        return BaseMap.load_csv(base_csv)
    return None

def _dist_point_segment(p, a, b):
    ab = b - a
    L2 = float(ab @ ab)
    if L2 < 1e-12:
        return float(np.linalg.norm(p - a))
    t = float(np.clip((p - a) @ ab / L2, 0.0, 1.0))
    return float(np.linalg.norm(p - (a + t * ab)))

def _point_in_polygon(p, P):
    """射线法 (P 为凸多边形顶点, 逆时针或顺时针均可)。"""
    n = len(P)
    inside = False
    for i in range(n):
        a, b = P[i], P[(i + 1) % n]
        if (a[1] > p[1]) != (b[1] > p[1]):
            x = a[0] + (p[1] - a[1]) * (b[0] - a[0]) / (b[1] - a[1])
            if x > p[0]:
                inside = not inside
    return inside

def _dist_point_polygon(p, P):
    p = np.asarray(p, float)
    P = np.asarray(P, float)
    if _point_in_polygon(p, P):
        return 0.0
    n = len(P)
    return min(_dist_point_segment(p, P[i], P[(i + 1) % n]) for i in range(n))

```

```

class P3Robot:
    def __init__(self, arena, cfg: P3Config | None = None, base=None, log=None,
                  log_prefix=""):
        self.arena = arena
        self.cfg = cfg or P3Config()
        self.base = base if base is not None else load_base()
        self._log_fn = log
        self.log_prefix = log_prefix
        self.recs = {int(ch): ChannelRec(int(ch)) for ch in self.cfg.channels}
        self.scan_points: list[tuple] = []
        self.meas_pts: list[tuple] = []
        # **真实做过的检测**位置（覆盖度评估口径：
        # "到过的点"≠"测过的点"，覆盖度只能按后者算）

        self.events: list[dict] = []
        self.aborted = False
        # 覆盖式试探因"预算不足"放弃时置位：让 clear_phase 立刻收敛，
        # 不要对同一个目标反复重规划（实测会把重规划次数抬高一到两个数量级）。
        self.abort_clear_phase = False
        self.stats = {"n_measure": 0, "n_clear": 0, "n_clear_fail": 0,
                      "n_scan_rounds": 0, "n_bearings": 0, "n_rejected": 0,
                      "n_refine": 0, "n_second_bearing": 0, "n_replans": 0,
                      "n_midphase_targets": 0, "n_sweep_aborted": 0,
                      "n_sweep_miss": 0, "n_probe_issued": 0,
                      "n_scan_gated": 0, "n_shortlist_kept": 0, "n_shortlist_in": 0,
                      "t_scan_s": 0.0, "t_clear_s": 0.0, "t_clear_ends_s": 0.0,
                      "vtime_at_last_clear": 0.0}

    # ----- 基础设施 -----
    def log(self, msg):
        self.events.append({"t": float(self.arena.virtual_time_s), "msg": msg})
        if self._log_fn:
            # 日志里避免使用 GBK 控制台无法编码的符号（⇒ ★ √ × 等），
            # 万一还有，退化为替换字符也不要让整局崩掉。
            try:
                self._log_fn(f"{self.log_prefix}{msg}")
            except UnicodeEncodeError:
                self._log_fn(f"{self.log_prefix}{msg}".encode(
                    "gbk", "replace").decode("gbk"))

    def remaining(self):
        return float(self.arena.remaining_budget_s())

    def can_afford(self, seconds):
        return self.remaining() > float(seconds) + self.cfg.time_reserve_s

    @property
    def pos(self):
        return tuple(self.arena.pos)

    # ----- 检测 / 清除 -----
    def measure(self, x, y, ch):
        resp = self.arena.measure(x, y, ch)
        self.stats["n_measure"] += 1
        if resp.get("accepted") is not True:
            self.stats["n_rejected"] += 1
            self.aborted = True
            self.log(f" ! /measure 频道{ch} 未执行 ({resp.get('reason', '?')}) ")
            return resp
        res = resp.get("measure_result")
        rec = self.recs[int(ch)]
        rec.meas_pts.append((float(x), float(y)))

```

```

rec.meas_log.append((float(x), float(y), float(self.arena.virtual_time_s), res))
if res == "direction":
    self.add_bearing(int(ch), x, y, float(resp["svd_deg"]))
elif res == "near":
    rec.near_hits += 1
else:
    rec.no_signal += 1
return resp

def add_bearing(self, ch, x, y, svd):
    rec = self.recs[int(ch)]
    rec.pts.append((float(x), float(y)))
    rec.svds.append(float(svd))
    rec.clear_fail = 0
    rec.tried_probes = []
    self.stats["n_bearings"] += 1
    reg = self.region(rec)
    if reg.get("status") == "bounded":
        rec.last_bounded = (tuple(reg["min_enclosing_center"]),
                           float(reg["min_enclosing_radius_m"]), len(rec.pts))
    rec.history.append({"n": len(rec.pts), "x": float(x), "y": float(y),
                      "svd": float(svd), "status": reg.get("status"),
                      "r_star_m": (rec.last_bounded[1] if rec.last_bounded else None)})
    return reg

def do_clear(self, x, y, ch):
    resp = self.arena.clear(x, y, ch)
    self.stats["n_probe_issued"] = self.stats.get("n_probe_issued", 0) + 1
    if resp.get("accepted") is not True:
        self.stats["n_rejected"] += 1
        self.aborted = True
        self.log(f" ! /clear 频道{ch} 未执行 ({resp.get('reason', '?')}) ")
        return False
    if resp.get("clear_result") == "success":
        rec = self.recs[int(ch)]
        rec.cleared = True
        rec.cleared_at = float(self.arena.virtual_time_s)
        self.stats["n_clear"] += 1
        self.stats["vtime_at_last_clear"] = float(self.arena.virtual_time_s)
        self.log(f" [OK] 频道{ch} 已清除 (t={self.arena.virtual_time_s:.1f}s) ")
        return True
    self.stats["n_clear_fail"] += 1
    return False

# ----- 定位区域 -----
def region(self, rec: ChannelRec):
    if not rec.pts:
        return {"status": "none", "n_bearings": 0}
    return analyse(rec.pts, rec.svds, BEARING_ERR)

def bounded_region(self, rec: ChannelRec):
    """当前（或最近一次）有界区域；返回 ``{'center', 'radius', 'poly', 'status'}``。"""
    if len(rec.pts) >= 2:
        reg = self.region(rec)
        if reg.get("status") == "bounded":
            return {"center": tuple(reg["min_enclosing_center"]),
                    "radius": float(reg["min_enclosing_radius_m"]),
                    "poly": [tuple(v) for v in reg.get("vertices", [])],
                    "status": "bounded", "n_bearings": len(rec.pts)}
    if rec.last_bounded is not None:
        c, r, n = rec.last_bounded
        return {"center": tuple(c), "radius": float(r), "poly": None,

```

```

        "status": "fallback", "n_bearings": n}

    return None

# ----- 扫描 -----
def needed_channels(self):
    """还需要（且值得）拿到新方位的频道：已清除、已够定位两次的跳过。"""
    need, done = [], []
    for ch in self.cfg.channels:
        rec = self.recs[int(ch)]
        if rec.cleared or rec.n_bearings >= self.cfg.clear_min_bearings:
            continue
        (done if rec.pts else need).append(int(ch))
    order = (done + need) if self.cfg.scan_1bearing_first else sorted(done + need)
    return order

def channels_to_measure(self, limit=None, pos=None, subset=None, apply_gap=True):
    """在 ``pos`` 处值得检测的频道。

    **跳过规则**（"节省时间"的关键）：已清除 / 已定位两次的跳过；
    对"无信号"频道，新检测点必须距它所有历史检测点  $\geq$  ``nosignal_gap_m``
    （有效接收半径至少 1000 m，在同一个圈里重复扫几乎拿不到新信息）；
    对已有一条方位的频道，新检测点距历史点  $\geq$  ``repeat_gap_m``（太小交会角没意义）。
    """
    p = np.asarray(pos if pos is not None else self.arena.pos, float)
    out = []
    for ch in (subset if subset is not None else self.needed_channels()):
        rec = self.recs[int(ch)]
        if apply_gap and rec.meas_pts:
            gap = self.cfg.repeat_gap_m if rec.pts else self.cfg.nosignal_gap_m
            d = min(math.hypot(p[0] - q[0], p[1] - q[1]) for q in rec.meas_pts)
            if d < gap:
                continue
        out.append(int(ch))
    return out[:limit] if limit else out

def build_field(self, grid_step=None, dist_weight=None):
    cfg = FieldConfig(grid_step=grid_step or self.cfg.grid_step,
                      e_max=self.cfg.e_max,
                      dist_weight=(self.cfg.dist_weight if dist_weight is None
                                   else dist_weight),
                      visited_radius=self.cfg.visited_radius,
                      min_move=self.cfg.min_move)
    f = ExpectField(self.base, cfg)
    if self.base is None:
        return f, 0
    n = 0
    for ch, rec in self.recs.items():
        if rec.cleared:
            continue
        if self.cfg.field_terms == "pending" and rec.n_bearings >= self.cfg.clear_min_bearings:
            continue
        if rec.n_bearings == 0:
            continue
        for ((x, y), th) in zip(rec.pts, rec.svds):
            f.add_bearing((x, y), th, tag=ch)
            n += 1
    if n == 0:
        # 一条"缺第二方位"的频道都没有（例如后半程只剩没听到过的频道）：
        # 自动退化为**探索项**，否则场上没有任何信息、选点无从谈起。
        for ch, rec in self.recs.items():
            if rec.cleared or rec.n_bearings > 0 or not rec.meas_pts:
                continue

```

```

        if not self.cfg.field_discovery and not self.cfg.auto_discovery_when_empty:
            continue
        f.add_raw(f.discovery_term(rec.meas_pts, self.cfg.nosignal_gap_m),
                 tag=("disc", ch))
        n += 1
    return f, n

def choose_scan_point(self, dist_weight=None, max_dist=None):
    f, n = self.build_field(dist_weight=dist_weight)
    visited = list(self.scan_points)
    sel = f.argmin(self.pos, visited=visited, max_dist=max_dist)
    if sel is None:
        sel = f.fallback_point(self.pos, visited=visited, max_dist=max_dist)
    sel["n_bearing_terms"] = n
    return sel, f

def plan_scan(self, dist_weight=None):
    """选下一个检测点，并确定该点上值得测哪些频道。

    顺序很重要：**先用期望场选点，**再**判断该点上哪些频道还没被"测透"，
    否则会出现"因为要换地方才有信息、但没地方可去"的死锁。
    """
    base = self.needed_channels()
    if not base:
        return None
    # 时间不够时把可选点限制在"还能走到"的范围内（给检测与安全余量留出空间）
    slack = 2.0 * (T_MEASURE + T_SWITCH) + 4.0
    dmax = max(0.0, (self.remaining() - self.cfg.time_reserve_s - slack) * V_ROBOT)
    sel, f = self.choose_scan_point(dist_weight=dist_weight, max_dist=dmax)
    chans = self.channels_to_measure(pos=sel["xy"], subset=base)
    if not chans:
        sel2 = f.fallback_point(self.pos, visited=self.scan_points)
        chans2 = self.channels_to_measure(pos=sel2["xy"], subset=base)
        if chans2:
            sel, chans = sel2, chans2
    if not chans:
        # 兜底：宁可多测几次也不原地空转
        chans = base[:3]
    # 时间不够时自动缩减本轮的频道数（至少测 min_scan_channels 个），把边角时间也用上
    travel = math.dist(self.pos, sel["xy"]) / V_ROBOT
    avail = self.remaining() - self.cfg.time_reserve_s - travel - 4.0
    kmax = max(1, int(avail // (T_MEASURE + T_SWITCH)))
    if len(chans) > kmax:
        if kmax < self.cfg.min_scan_channels:
            return None
            # 只剩时间测一两个频道：不值得为它专门跑一趟
        self.log(f" 时间只剩 {self.remaining():.0f} s -> 本轮只测 {kmax}/{len(chans)} 个频道")
        chans = chans[:kmax]
    cost = travel + len(chans) * (T_MEASURE + T_SWITCH) + 2.0
    plan = {"chans": chans, "sel": sel, "field": f, "cost_s": cost}
    if self.cfg.scan_budget_gate and not self._scan_worth_it(plan):
        return None
    return plan

# ----- 预算感知的扫描闸门 -----
def nearest_clear_cost_after(self, at_xy):
    """估计"到达 ``at_xy`` 之后，再清掉最近一个已知目标"所需秒数；没有已知目标则 None。"""
    cand = self.clear_targets()
    if not cand:
        return None
    nxt = min(cand, key=lambda t: math.dist(at_xy, t[2]["center"]))
    return self.estimate_clear_cost(nxt[2], from_pos=at_xy,
                                    probe_cap=self.cfg.optimistic_probe_steps)

```

```

def _scan_budget_left(self):
    """允许继续扫描的剩余秒数 (= frac × 总预算 - 已花在扫描上的时间) 。”””
    if self.cfg.scan_budget_frac >= 1.0:
        return float("inf")
    return self.cfg.scan_budget_frac * float(self.arena.budget) \
        - self.stats.get("t_scan_s", 0.0)

def _scan_worth_it(self, plan):
    """本轮扫描值不值得做：**扫完还必须够清最近的一个已知目标**，否则不扫。

    动机（诊断数据）：30 局里扫描阶段平均吃掉约 1016 s / 1200 s，末尾有 23% 的
    未清除源**已经定位好却走不到**。也就是说“再多发现一个源”的边际价值，在
    剩余时间已经不足以走过去清除它时等于 0 —— 这种扫描纯属消耗预算。

    注意这里只做“取舍”不做“折中”：要么整轮扫，要么整轮不扫；
    因为扫描点本身的移动无法中途变成对目标的逼近（两者方向不同）。
    """
    rem = self.remaining()
    left = self._scan_budget_left()
    if plan["cost_s"] > left:
        self.stats["n_scan_gated"] = self.stats.get("n_scan_gated", 0) + 1
        self.log(f" ■ 放弃本轮扫描：扫描时间额度已用尽（本轮需 {plan['cost_s']:.0f} s, "
            f"额度剩 {left:.0f} s = {self.cfg.scan_budget_frac:.2f}×总预算）"
            f"-> 预算留给已定位目标")
        return False
    cand = self.clear_targets()
    need = None
    if cand:
        nxt = min(cand, key=lambda t: math.dist(plan["sel"]["xy"], t[2]["center"]))
        need = self.estimate_clear_cost(nxt[2], from_pos=plan["sel"]["xy"],
            probe_cap=self.cfg.optimistic_probe_steps)
        # 可选：扫描点不能“背着”已知目标走（否则这轮的移动无法回收为进度）
        if self.cfg.scan_only_toward_targets:
            u = np.asarray(plan["sel"]["xy"], float) - np.asarray(self.pos, float)
            w = np.asarray(nxt[2]["center"], float) - np.asarray(self.pos, float)
            nu, nw = float(np.hypot(*u)), float(np.hypot(*w))
            if nu > 1e-9 and nw > 1e-9 and float(u @ w) / (nu * nw) < 0.0:
                self.stats["n_scan_gated"] = self.stats.get("n_scan_gated", 0) + 1
                self.log(f" ■ 放弃本轮扫描：扫描点（{plan['sel']['xy'][0]:.0f},"
                    f"{plan['sel']['xy'][1]:.0f}）与已知目标方向相反 -> 预算留给清除")
                return False
    if need is None:
        return True # 手上没有可清目标：扫描是唯一的进展方式
    required = (plan["cost_s"] + self.cfg.scan_gate_margin
        + self.cfg.scan_gate_factor * need)
    if rem >= required:
        return True
    self.stats["n_scan_gated"] = self.stats.get("n_scan_gated", 0) + 1
    self.log(f" ■ 放弃本轮扫描：剩余 {rem:.0f} s < 本轮扫描 {plan['cost_s']:.0f} s "
        f"+ 清最近目标 {need:.0f} s（阈值 {required:.0f} s）"
        f"-> 预算留给已定位目标")
    return False

def execute_scan(self, plan):
    sel, chans = plan["sel"], plan["chans"]
    x, y = sel["xy"]
    if sel["kind"] == "explore":
        self.log(f"扫描轮：期望场无可用方位（{sel.get('n_bearing_terms', 0)} 项）"
            f"-> 探索点（{x:.0f},{y:.0f}），距 {sel['dist_m']:.0f} m")
    else:
        self.log(f"扫描轮：选点（{x:.0f},{y:.0f}）| E={sel['E_mean_m']:.1f} m "
            f"E_norm={sel['E_norm']:.3f} 距离惩罚={sel['dist_penalty']:.3f} ")

```

```

        f"score={sel['score']:.4f} 移动 {sel['dist_m']:.0f} m | "
        f"待测 {len(chans)} 个频道 {chans}")
    self.scan_points.append((float(x), float(y)))
    n = self.scan_at(x, y, chans)
    self.stats["n_scan_rounds"] += 1
    return n

def scan_round(self, dist_weight=None):
    plan = self.plan_scan(dist_weight=dist_weight)
    if plan is None:
        return 0
    if not self.can_afford(plan["cost_s"]):
        return 0
    return self.execute_scan(plan)

def scan_at(self, x, y, chans):
    """在 (x, y) 依次检测这些频道（第一次调用会带来移动耗时）。"""
    n = 0
    for i, ch in enumerate(chans):
        if self.aborted or not self.can_afford(T_MEASURE + T_SWITCH + 3.0):
            break
        resp = self.measure(x, y, ch)
        n += 1
        if resp.get("measure_result") == "near":
            # 已在 5 m 内: 直接清除
            self.do_clear(x, y, ch)
    return n

# ----- 清理 -----
def clear_targets(self):
    out = []
    for ch, rec in self.recs.items():
        if rec.cleared or rec.n_bearings < self.cfg.clear_min_bearings:
            continue
        if rec.clear_fail > 0:
            continue # 上次清理失败, 等下一次新方位再试
        br = self.bounded_region(rec)
        if br is None or br["radius"] > self.cfg.clear_max_radius_m:
            continue
        out.append((ch, rec, br))
    out.sort(key=lambda t: math.dist(self.pos, t[2]["center"]))
    return out

# ----- 清理: 滚动时域动态规划 -----
def _probe_count(self, br):
    """该定位区域需要多少个覆盖式试探点（带缓存: 区域不变就不重算）。

    这里**故意**对点数封顶: 它只用于"值不值得跑一趟"的代价预估,
    真实覆盖由 :meth:`probe_points` 保证。不封顶会让大  $r^*$  上的枚举变得很贵
    ( $r^*=200$  m 时 151 个点  $\times$  每步重规划), 实测把程序运行时间抬高两个数量级。
    """
    key = (round(float(br["center"][0]), 1), round(float(br["center"][1]), 1),
           round(float(br["radius"]), 1))
    cache = getattr(self, "_probe_cache", None)
    if cache is None:
        cache = self._probe_cache = {}
    if key not in cache:
        r = min(float(br["radius"]), self.cfg.probe_max_radius_m)
        n = len(self.probe_points(np.asarray(br["center"], float), r,
                                             poly=br.get("poly"))))
        cache[key] = min(int(n), int(self.cfg.probe_max_points))
    return cache[key]

```



```
def estimate_clear_cost(self, br, from_pos=None, n_probe=None, probe_cap=None):
    """估计"从 ``from_pos`` 出发清除某个目标"的耗时（秒）。
```

组成：走到区域中心 + 一次逼近检测 + 覆盖式试探（试探点数 × 平均步长/速度 + 3 s/次）
+ 成功清除 5 s。

``probe\_cap`` 用于**乐观估计**：实际执行时试探是逐点进行的、随时可以中止（多数目标前几个试探点就成功），所以判断"值不值得跑一趟"时把试探数封顶，否则会过于保守、把还有 400 s 的尾巴白白浪费掉。

```
"""
p = np.asarray(from_pos if from_pos is not None else self.pos, float)
c = np.asarray(br["center"], float)
if n_probe is None:
    n_probe = self._probe_count(br)
if probe_cap is not None:
    n_probe = min(int(n_probe), int(probe_cap))
t_probe = n_probe * (self.cfg.probe_spacing_m / V_ROBOT + T_CLEAR_FAIL)
return (math.dist(p, c) / V_ROBOT + T_MEASURE + T_SWITCH
        + t_probe + T_CLEAR_OK)
```

```
def budget_feasible_shortlist(self, targets, start=None, budget=None):
    """从 ``targets`` 里挑出"剩余预算内走得完"的**近点子集**（贪心插入）。
```

对"近点优先"的回答：滚动时域下每步只执行第一个目标，所以**只保留一个近点**在语义上已经等价于"优先清最近的点"；本函数真正的作用是判断"这一轮清理是否还排得下任何一个目标"，以及给排序一个**可达性**约束。

排序键是"距当前位置"（近点优先），逐个累计
``走到区域中心 + 一次逼近检测 + 覆盖式试探 + 清除`` 的乐观代价，
超预算就停止插入（后面更远的点更进不来）。

```
"""
if not targets:
    return []
p = np.asarray(start if start is not None else self.pos, float)
rem = self.remaining() if budget is None else float(budget)
room = rem - self.cfg.time_reserve_s - self.cfg.min_action_budget_s \
        - self.cfg.clear_shortlist_reserve_s
ordered = sorted(targets, key=lambda t: math.dist(p, t[2]["center"]))
out, used = [], 0.0
for t in ordered:
    src = p if not out else np.asarray(out[-1][2]["center"], float)
    # estimate_clear_cost 已含"从 from_pos 走到区域中心"的行程
    cost = self.estimate_clear_cost(
        t[2], from_pos=src, probe_cap=self.cfg.optimistic_probe_steps)
    if used + cost > room:
        continue # 这个点排不进，试下一个更远的（更近的已进来）
    out.append(t)
    used += cost
return out
```

```
def plan_clear_tour(self, targets, remaining_s=None):
    """**动态规划（滚动时域）**清理航路：每执行一步就重新规划一次。
```

与旧版（清理阶段开始时规划一次、阶段内顺序冻结）的区别：

- * 每次调用都重读台账 ⇒ **途中新定位出来的频道立刻进入航路**；
- * 各频道用**最新**的定位区域中心（逼近过程中区域会变小、中心会移动）；
- * 以**当前位置**为起点重新做最近邻 + 2-opt（可用 ``tour`` 配置切换）；
- * 本步只执行规划结果的第一个，执行完立刻重规划（rolling horizon）。

为什么默认不按剩余预算裁剪目标表：目标是"边走边判"执行的 ——

逼近、一次检测、逐点试探各自都检查剩余时间、随时可以中止，多数目标在前几个试探点就成功。用统一的预估代价去裁剪，会把还剩 300 - 400 s 的尾巴整段浪费掉（实测：裁剪 39.8% vs 不裁剪 41.2%）。故默认只排序、不裁剪，由 :meth:`clear\_target` / :meth:`sweep\_clear` 在动作粒度上兜底。
`clear_budget_truncation=True` 可打开裁剪做对照。

`clear_policy` 两种策略：

```
* ``tour`` (默认)：NN + 2-opt 的**全局航路结构**，按顺序执行；
* ``rollout``：一步前瞻 + SPT 推演，直接最大化"预算内可清除个数"
  （理论更贴合目标函数，但实测更差 36.3% vs 41.2%：贪心总挑最近的，
  最后把最远的目标拖成"永远清不掉"）。
"""
if not targets:
    return []
self.stats["n_replans"] = self.stats.get("n_replans", 0) + 1
remaining = self.remaining() if remaining_s is None else float(remaining_s)
budget = remaining - self.cfg.time_reserve_s - self.cfg.min_action_budget_s
if self.cfg.clear_policy == "rollout":
    return self._plan_rollout(targets, budget)
if self.cfg.clear_policy == "tour_greedy_cost":
    # 贪心"每次挑预计剩余预算花得完、且最近的"目标，滚动推进。
    # 与 'tour' 的差别：这里显式用**剩余预算**逐目标判定可达性，
    # 而不是先排一条全局航路、再边走边判。
    return self._plan_greedy_cost(targets, remaining)
# ---- 可选：先用"可达近点子集"过滤，再做航路排序 ----
pool = list(targets)
if self.cfg.clear_shortlist:
    pool = self.budget_feasible_shortlist(pool, start=self.pos, budget=remaining)
    self.stats["n_shortlist_kept"] = self.stats.get("n_shortlist_kept", 0) \
        + len(pool)
    self.stats["n_shortlist_in"] = self.stats.get("n_shortlist_in", 0) \
        + len(targets)
    if not pool:
        return []
# ---- tour 策略：全局航路结构（可选预算截断） ----
order = self.order_targets(list(pool), from_pos=self.pos)
by_ch = {t[0]: t for t in pool}
ordered = [by_ch[ch] for ch in order if ch in by_ch]
if not self.cfg.clear_budget_truncation:
    return ordered
out, used, pos = [], 0.0, tuple(self.pos)
for t in ordered:
    c = self.estimate_clear_cost(t[2], from_pos=pos,
                                probe_cap=self.cfg.optimistic_probe_steps)
    if used + c > budget:
        continue
    out.append(t)
    used += c
    pos = tuple(t[2]["center"])
return out
```

```
def _plan_greedy_cost(self, targets, remaining):
    """贪心 + 剩余预算可达性（``clear_policy='tour_greedy_cost'``）。
```

诊断动机：末尾 26/49 个"已定位未清除"的目标在变成可清目标时剩余时间完全够用，却因为全局航路把它们排在后面而没轮到（清完前面的目标后预算就没了）。
 本策略每一步只在"当前剩余预算真的能走完"的目标里挑最近的，
 从而避免"先承诺一个远点、结果后面全部落空"。

与 `clear_budget_truncation` 的区别：那个是对已排好的全局航路做事后裁剪，
 本策略是**在挑选时就**把可达性作为约束。
 """

```

pos = np.asarray(self.pos, float)
left = list(targets)
out, used = [], 0.0
room = remaining - self.cfg.time_reserve_s - self.cfg.min_action_budget_s
while left:
    best, best_cost = None, None
    for t in left:
        c = self.estimate_clear_cost(t[2], from_pos=pos,
                                     probe_cap=self.cfg.optimistic_probe_steps)
        if best_cost is None or c < best_cost:
            best, best_cost = t, c
    if best is None or used + best_cost > room:
        break
    out.append(best)
    used += best_cost
    pos = np.asarray(best[2]["center"], float)
    left.remove(best)
return out

def _plan_rollout(self, targets, budget):
    """一步前瞻 + SPT 推演 (``clear_policy='rollout'``, 见 :meth:`plan_clear_tour`)。"""
    k = len(targets)
    pos0 = tuple(self.pos)
    centers = [tuple(t[2]["center"]) for t in targets]
    nprobe = []
    for t in targets:
        nprobe.append(self._probe_count(t[2]))
    t_probe = [n * (self.cfg.probe_spacing_m / V_ROBOT + T_CLEAR_FAIL) for n in nprobe]

    def leg(a, b):
        return (math.dist(a, centers[b]) / V_ROBOT + T_MEASURE + T_SWITCH
                + t_probe[b] + T_CLEAR_OK)

    cost0 = [leg(pos0, j) for j in range(k)]

    def rollout(cur, left, tb):
        left = set(left)
        n = 0
        while True:
            best_c, best_j = None, None
            for j in left:
                c = leg(cur, j)
                if c <= tb and (best_c is None or c < best_c):
                    best_c, best_j = c, j
            if best_j is None:
                return n
            tb -= best_c
            n += 1
            cur = centers[best_j]
            left.discard(best_j)

    best_j, best_key = None, None
    for j in range(k):
        if cost0[j] > budget:
            continue
        n = 1 + rollout(centers[j], [i for i in range(k) if i != j], budget - cost0[j])
        key = (n, -cost0[j])
        if best_key is None or key > best_key:
            best_key, best_j = key, j
    if best_j is None:
        return []
    out = [targets[best_j]]

```

```

used, pos = cost0[best_j], centers[best_j]
for j in sorted((i for i in range(k) if i != best_j), key=lambda i: leg(pos, i)):
    c = leg(pos, j)
    if used + c > budget:
        continue
    out.append(targets[j])
    used += c
    pos = centers[j]
return out

def order_targets(self, targets, from_pos=None):
    """航路排序: 最近邻 + 2-opt (以 ``from_pos`` 为固定起点)。

    目标点用各自的 (最新) 定位区域中心;  $k \leq 20$ , 暴力 2-opt 代价可忽略。

    ``tour`` 三种模式 (对照用):
    * ``nn`` : 纯最近邻 —— 第一步必然最近, 但总路程偏长;
    * ``nn2opt`` : 最近邻 + 2-opt (**默认**) —— 总路程最短, 但 2-opt 的
      区间反转会改写**第一步** (60 局实测有 14.1% 的排序其第一步不是最近的);
    * ``nn2opt_keepfirst``: 先钉住最近的点作第一步, 只对**其后**的子序列做 2-opt。
      动机: 滚动时域下**只执行第一步**, 后面重排一次; 所以"总路程最短"并不是
      正确的目标函数, "第一步最近的 + 剩余结构合理"才是。
    """
    start = tuple(self.pos if from_pos is None else from_pos)
    if len(targets) <= 1 or self.cfg.tour == "nn":
        targets.sort(key=lambda t: math.dist(start, t[2]["center"]))
        return [t[0] for t in targets]
    pts = [tuple(t[2]["center"]) for t in targets]
    k = len(pts)
    cur = start
    # 最近邻构造
    left = list(range(k))
    seq = []
    p = cur
    while left:
        j = min(left, key=lambda i: math.dist(p, pts[i]))
        seq.append(j)
        left.remove(j)
        p = pts[j]

    def total(order):
        d = math.dist(cur, pts[order[0]])
        for a, b in zip(order, order[1:]):
            d += math.dist(pts[a], pts[b])
        return d

    # 钉住第一步时, 只允许反转 index >= 1 的区间
    lo = 1 if (self.cfg.tour == "nn2opt_keepfirst" and k >= 2) else 0
    best = total(seq)
    improved = True
    while improved:
        improved = False
        for i in range(lo, k - 1):
            for j in range(i + 1, k):
                cand = seq[:i] + seq[i:j + 1][::-1] + seq[j + 1:]
                d = total(cand)
                if d < best - 1e-9:
                    seq, best, improved = cand, d, True
    return [targets[i][0] for i in seq]

def clear_phase(self):
    """清理阶段: **每清一个目标就重新规划一次** (滚动时域)。

```

循环体：读最新台账 → 动态规划 → 只执行第一步 → 顺路补测（可能新增可清理频道）
→ 回到循环顶部重新规划。这样“清除之后扫描过程中新定位的位置”会立刻进入航路，
而且航路始终以当前位置为起点、用最新的区域中心优化。

```

"""
first = self.clear_targets()
if not first:
    return 0
self.abort_clear_phase = False
tour0 = self.plan_clear_tour(first) if self.cfg.dynamic_replan else None
if self.cfg.dynamic_replan and not tour0:
    return 0    # 预算内一个都排不下：静默返回，让主循环把时间用于扫描
if not self.can_afford(self.cfg.min_action_budget_s):
    return 0
self.log(f"清理阶段（动态重规划）：{len(first)} 个频道已定位两次 -> "
        f"[{(t[0], round(t[2]['radius'])) for t in first}]")
n = 0
guard = 0
static_order = None
tour = tour0
initial_chs = {t[0] for t in (tour0 or first)}
midphase = 0
while not self.aborted and guard < 4 * len(self.recs) + 8:
    guard += 1
    if self.cfg.dynamic_replan:
        tour = self.plan_clear_tour(self.clear_targets())    # ① 每步重新读台账并规划
        if not tour:
            self.log("剩余时间不足以完成任何一条清理航段 -> 结束清理阶段")
            break
        ch, rec, _br = tour[0]    # ② 只执行第一步
        if ch not in initial_chs:
            midphase += 1    # 本阶段中途才定位出来的目标
            self.log(f" 动态航路：频道{ch} 是本阶段中途新定位的，立即插队执行")
        elif len(tour) > 1:
            self.log(f" 动态航路：本步 频道{ch}，随后 "
                    f"[{(t[0] for t in tour[1:4])}]{'...' if len(tour) > 4 else ''}")
    else:
        # 静态对照（消融用）
        targets = self.clear_targets()
        if not targets:
            break
        if not self.can_afford(self.cfg.min_action_budget_s):
            break
        if static_order is None:
            static_order = self.order_targets(list(targets))
        if not static_order:
            break
        ch = static_order.pop(0)
        t = next((x for x in targets if x[0] == ch), None)
        if t is None:
            continue
        rec = t[1]
    if self.clear_target(ch, rec):
        n += 1
    if self.abort_clear_phase:
        self.log("预算不足 -> 结束清理阶段（剩余时间留给扫描）")
        break
    self.enroute_scan()    # ③ 顺路补测（可能新增可清理频道）
self.stats["n_midphase_targets"] = self.stats.get("n_midphase_targets", 0) + midphase
return n

```

```

def enroute_scan(self):
    """清理途中在当前位置顺手补测：既补充方位，也给新发现的频道补第二条方位。

```

这一步很关键：发现一个新频道只要 6 s，但只有拿到**第二条**方位才能进入清理队列。所以一旦在本停点发现新频道，就立刻用第二问的"局部最优第二检测点"再测一条，让它本轮就能被清除，而不是等到下一次扫描（往往等不到）。

```

"""
if not self.cfg.enroute_scan or self.aborted:
    return 0
chans = self.channels_to_measure(limit=self.cfg.enroute_scan_max)
if not chans:
    return 0
cost = len(chans) * (T_MEASURE + T_SWITCH) + 2.0
if not self.can_afford(cost + self.cfg.min_action_budget_s * 0.5):
    return 0
self.log(f" 顺路补测 {len(chans)} 个频道 {chans}")
n = self.scan_at(self.pos[0], self.pos[1], chans)
if self.cfg.second_bearing_refine:
    fresh = [ch for ch in chans
              if not self.recs[ch].cleared and self.recs[ch].n_bearings == 1]
    fresh.sort(key=lambda ch: -self.recs[ch].near_hits)
    for ch in fresh[:self.cfg.second_bearing_max]:
        if self.remaining() < self.cfg.second_bearing_budget_s + self.cfg.time_reserve_s:
            break
        self.acquire_second_bearing(ch)
return n

def acquire_second_bearing(self, ch):
    """给"只有一条方位"的频道补第二检测点（局部最优，含行驶代价）。"""
    rec = self.recs[int(ch)]
    if rec.n_bearings != 1 or rec.cleared:
        return False
    p, info = best_refine_point(rec.pts, rec.svds, self.pos, None, n_t=100,
                               travel_cap_m=self.cfg.refine_travel_cap_m * 1.4)
    if p is None:
        return False
    trav = math.dist(self.pos, (float(p[0]), float(p[1])))
    if not self.can_afford(trav / V_ROBOT + T_MEASURE + T_SWITCH + 20.0):
        return False
    self.log(f" 频道{ch} 新方位 -> 第二检测点 ({p[0]:.0f},{p[1]:.0f}), "
            f"移动 {trav:.0f} m (E[D]≈{info['E_diam_m']:.0f} m, "
            f"预计代价 {info['cost_s']:.0f} s)")
    resp = self.measure(float(p[0]), float(p[1]), ch)
    if resp.get("measure_result") == "near":
        return self.do_clear(float(p[0]), float(p[1]), ch)
    self.stats["n_second_bearing"] = self.stats.get("n_second_bearing", 0) + 1
    return rec.n_bearings >= 2

def choose_refine_point(self, ch, rec, br):
    """侧移精定位点：与上一条视线近似正交，把交会角拉到 60-90°。"""
    c = np.asarray(br["center"], float)
    if self.cfg.refine_mode == "exact":
        p, info = best_refine_point(rec.pts, rec.svds, self.pos,
                                    {"center": c}, n_t=120,
                                    travel_cap_m=self.cfg.refine_travel_cap_m)
        if p is not None:
            return np.asarray(p, float), info
    if not rec.pts:
        return None, None
    S_last = np.asarray(rec.pts[-1], float)
    u = c - S_last
    nu = float(np.linalg.norm(u))
    if nu < 1e-6:
        return None, None

```

```

u = u / nu
perp = np.array([-u[1], u[0]])
rho = min(max(1.5 * br["radius"], 80.0), self.cfg.refine_leg_m)
best = None
for sgn in (1.0, -1.0):
    p = c + sgn * rho * perp
    if np.hypot(*p) > R_ARENA - 1.0:
        continue
    trav = float(np.hypot(*(p - np.asarray(self.pos, float))))
    if trav > self.cfg.refine_travel_cap_m:
        continue
    if best is None or trav < best[0]:
        best = (trav, p)
if best is None:
    return None, None
return best[1], {"mode": "perp", "rho": rho, "travel_m": best[0]}

def probe_points(self, c, r, poly=None, tried=()):
    """覆盖式试探点。

    * 有定位区域多边形 ``poly`` 时: 只在"到多边形距离  $\leq 20$  m"的格点上试
      —— 交会区域常常是细长四边形, 用外接圆 (面积  $\pi r^2$ ) 试探会浪费一倍以上的点;
    * 没有多边形时退化为覆盖半径  $r$  的圆域。
    格点间距  $\leq 20\sqrt{3}$  m, 保证区域内任一点到某个试探点  $\leq 20$  m = 清除半径, **必然成功**。


注意"必然成功"只在**候选点不被截断**时成立: ``probe_max_points`` 会按"离中心
    由近及远"截断, 等于丢掉覆盖区域的**外圈**, 保证随之失效 (实测  $r^* \geq 120$  m 即开始
    截断, 60 个点对应  $r^* \approx 120$  m, 而截断前需要 61 个)。因此默认
    ``probe_unlimited=True`` 取全部候选点; ``probe_max_points`` 只在显式关掉
    ``probe_unlimited`` (消融对照) 时才起作用。


    """
    c = np.asarray(c, float)
    s = float(self.cfg.probe_spacing_m)
    rr = min(float(r), self.cfg.probe_max_radius_m) + R_CLEAR
    u = np.array([s, 0.0])
    v = np.array([s / 2.0, s * math.sqrt(3.0) / 2.0])
    N = int(math.ceil(rr / s)) + 1
    cand = []
    for i in range(-N, N + 1):
        for j in range(-N, N + 1):
            p = c + i * u + j * v
            if float(np.hypot(*(p - c))) <= rr + 1e-9:
                cand.append(p)
    if poly is not None and len(poly) >= 3:
        P = np.asarray(poly, float)
        keep = []
        for p in cand:
            if _dist_point_polygon(p, P) <= R_CLEAR + 1e-9:
                keep.append(p)
        cand = keep if keep else cand
    cand.sort(key=lambda p: float(np.hypot(*(p - c))))
    out = []
    for p in cand:
        if any(float(np.hypot(*(p - np.asarray(q, float)))) < 2.0 for q in tried):
            continue
        out.append(p)
    if self.cfg.probe_unlimited:
        return out
    return out[:self.cfg.probe_max_points]

def sweep_clear(self, ch, rec, c, r, poly=None):
    """在区域 ``(c, r, poly)`` 上做覆盖式试探。

```

```

返回 :data:`SWEEP_OK` / :data:`SWEEP_MISS` / :data:`SWEEP_BUDGET` 三者之一。
**预算不足时立刻返回 ``SWEEP_BUDGET``, 一个 ``/clear`` 都不发** ——
否则会污染 ``clear_fail``、并在论文里被误读成"区域覆盖了却没找到目标"。
"""

probes = self.probe_points(c, r, poly=poly, tried=rec.tried_probes)
full = self.probe_points(c, r, poly=poly)
n_try = 0
for p in probes:
    if self.aborted:
        self.log(f" 覆盖式试探中断: 程序已中止 (已试 {n_try} 点)")
        return SWEEP_MISS if n_try else SWEEP_BUDGET
    if not self.can_afford(T_CLEAR_FAIL + 5.0):
        self.log(f" 覆盖式试探放弃: 剩余 {self.remaining():.0f} s 不足以支付一次 "
                  f"/clear (候选 {len(full)} 个, 实际尝试 {n_try} 个)")
        self.stats["n_sweep_aborted"] += 1
        return SWEEP_BUDGET if n_try == 0 else SWEEP_MISS
    trav = math.dist(self.pos, (float(p[0]), float(p[1])))
    if not self.can_afford(trav / V_ROBOT + T_CLEAR_FAIL + 3.0):
        self.log(f" 覆盖式试探放弃: 剩余 {self.remaining():.0f} s 不足以走到下一个 "
                  f"试探点 ({trav:.0f} m; 候选 {len(full)} 个, 实际尝试 {n_try} 个)")
        self.stats["n_sweep_aborted"] += 1
        return SWEEP_BUDGET if n_try == 0 else SWEEP_MISS
    n_try += 1
    rec.tried_probes.append((float(p[0]), float(p[1])))
    if self.do_clear(float(p[0]), float(p[1]), ch):
        self.log(f" 覆盖式试探命中: 第 {n_try}/{len(full)} 个候选点 "
                  f" (间距 {self.cfg.probe_spacing_m:.0f} m)")
        return SWEEP_OK
# 候选点全部试完仍未命中: 这才是真正的"区域内没找到目标"
self.stats["n_sweep_miss"] += 1
self.log(f" 覆盖式试探未命中: {n_try}/{len(full)} 个候选点全部试过 "
          f" ({'多边形区域' if poly else '外接圆'}, r*={r:.0f} m)")
return SWEEP_MISS

def clear_target(self, ch, rec):
    """定位→逼近→(必要时)侧移精定位→覆盖式试探清除。

    失败归因**必须三分** (见 :data:`SWEEP_BUDGET`) :

    * ``SWEEP_OK`` —— 清除成功;
    * ``SWEEP_MISS`` —— 候选点全试过仍没打中 ⇒ 累加 ``clear_fail`` (真的失败了);
    * ``SWEEP_BUDGET`` —— 预算不足、**一个 /clear 都没发** ⇒ **不**累加 ``clear_fail``,
    也不报"未清除成功", 而是收敛回清理阶段由 ``plan_clear_tour`` 决定收工。
    这正是"预算耗尽后还调用 sweep_clear 并记一笔失败"那个统计/日志缺陷的修复点。
    """
    self.log(f"清理频道{ch}: 当前 {rec.n_bearings} 条方位, "
              f"起点 ({self.pos[0]:.0f},{self.pos[1]:.0f})")
    budget_stop = False
    # (1) 逼近定位区域中心 (每次逼近顺带测一条方位, 移动本来就要花)
    for _ in range(self.cfg.max_approach_steps):
        br = self.bounded_region(rec)
        if br is None:
            rec.clear_fail += 1
            return False
        c = np.asarray(br["center"], float)
        d = math.dist(self.pos, tuple(c))
        if d <= max(br["radius"], self.cfg.arrive_radius_m):
            break
        cost = d / V_ROBOT + T_MEASURE + T_SWITCH + 4.0
        if not self.can_afford(cost):
            # 走不过去了: 进入第 (3) 步, 由覆盖式试探统一判定"预算不足"

```



```

        budget_stop = True
        break
    resp = self.measure(float(c[0]), float(c[1]), ch)
    if resp.get("measure_result") == "near":
        return self.do_clear(float(c[0]), float(c[1]), ch)
    if self.aborted:
        return False
# (2) 区域还大: 侧移一条正交视线, 把 r* 压小
if not budget_stop:
    br = self.bounded_region(rec)
    if br is not None and br["radius"] > self.cfg.direct_probe_radius_m:
        p, info = self.choose_refine_point(ch, rec, br)
        if p is not None:
            trav = math.dist(self.pos, (float(p[0]), float(p[1])))
            cost = trav / V_ROBOT + T_MEASURE + T_SWITCH + 4.0
            if self.can_afford(cost):
                self.stats["n_refine"] += 1
                self.log(f" 侧移精定位 -> ({p[0]:.0f},{p[1]:.0f}), "
                        f"移动 {trav:.0f} m ({info}) ")
                resp = self.measure(float(p[0]), float(p[1]), ch)
                if resp.get("measure_result") == "near":
                    return self.do_clear(float(p[0]), float(p[1]), ch)
# (3) 覆盖式试探清除
br = self.bounded_region(rec)
if br is None:
    rec.clear_fail += 1
    return False
c = np.asarray(br["center"], float)
r = min(br["radius"], self.cfg.probe_max_radius_m)
poly = br.get("poly")
st = self.sweep_clear(ch, rec, c, r, poly=poly)
if st == SWEEP_OK:
    return True
if st == SWEEP_BUDGET:
    # 预算不足、"一个 /clear 都没发出": 不是清除失败, 不计 clear_fail、
    # 也不上升为空/随机搜索; 交由调用方收敛本阶段。
    self.abort_clear_phase = True
    self.log(f"  ↷ 频道{ch} 未尝试清除 (预算不足), 本阶段收敛")
    return False
# (4) 区域内没找到: 说明区域被算小了 (或误差超界), 扩大半径再试一轮
if br["radius"] <= self.cfg.probe_max_radius_m:
    r2 = min(br["radius"] * 1.6 + 20.0, self.cfg.probe_max_radius_m)
    self.log(f"  区域内未发现目标 -> 扩大到 r={r2:.0f} m 再试")
    st2 = self.sweep_clear(ch, rec, c, r2, poly=None)
    if st2 == SWEEP_OK:
        return True
    if st2 == SWEEP_BUDGET:
        self.abort_clear_phase = True
        self.log(f"  ↷ 频道{ch} 扩大半径后未尝试 (预算不足), 本阶段收敛")
        return False
    rec.clear_fail += 1
    self.log(f"  [x] 频道{ch} 本轮未清除成功 (候选点已试遍仍无目标, 留待后续扫描后再清)")
    return False

# ----- 主循环 -----
def run(self):
    t0 = time.time()
    resp = self.arena.enter()
    if resp.get("accepted") is not True:
        raise ArenaError(f"/enter 未被接受: {resp}")
    self.log(f"进入靶区: 可用预算 {self.remaining():.0f} s"
            f" ({self.arena.budget_kind()}) ")

```

```

try:
    # 第 0 轮: 机器狗本来就在当前位置 (原点), 直接扫描全部频道
    chans0 = self.channels_to_measure()
    self.log(f"第 0 轮扫描 (原地): 待测 {len(chans0)} 个频道 {chans0}")
    self.scan_points.append(self.pos)
    self.scan_at(self.pos[0], self.pos[1], chans0)
    rounds = 1
    stall = 0
    while (not self.aborted and rounds < self.cfg.max_rounds
           and self.remaining() > self.cfg.time_reserve_s):
        did = 0
        t_before = float(self.arena.virtual_time_s)
        if self.clear_targets() and (rounds >= self.cfg.min_scan_rounds_before_clear
                                     or not self.plan_scan()):
            did += self.clear_phase()
            self.stats["t_clear_s"] += (float(self.arena.virtual_time_s)
                                       - t_before)
            self.stats["t_clear_ends_s"] = float(self.arena.virtual_time_s)
        if self.aborted or self.remaining() <= self.cfg.time_reserve_s:
            break
        plan = self.plan_scan()
        if plan is None:
            # 扫描被预算闸门否掉时, plan_scan() 也可能只是"暂无值得测的频道";
            # 此时若还有可清目标, 就再走一次清理阶段, 而不是直接判为 stall。
            if self.clear_targets():
                did += self.clear_phase()
                self.stats["t_clear_s"] += (float(self.arena.virtual_time_s)
                                           - t_before)
                self.stats["t_clear_ends_s"] = float(self.arena.virtual_time_s)
            if self.aborted:
                break
        elif self.can_afford(plan["cost_s"]) \
             and (did == 0 or not self.clear_targets()):
            did += self.execute_scan(plan)
            self.stats["t_scan_s"] += (float(self.arena.virtual_time_s)
                                       - t_before)

            rounds += 1
        if did == 0:
            stall += 1
            if stall >= 2:
                self.log("没有可清理的频道、也没有值得检测的频道 -> 提前结束")
                break
        else:
            stall = 0
    finally:
        try:
            self.arena.exit()
        except ArenaError as e: # 接口已关闭等
            self.log(f"/exit 异常: {e}")
    self.stats["program_runtime_s"] = time.time() - t0
    return self.report()

# ----- 汇报 -----
def report(self):
    truth = self.arena.truth()
    n_cleared = sum(1 for r in self.recs.values() if r.cleared)
    total = (truth or {}).get("n_sources")
    vt = float(self.arena.virtual_time_s)
    last = float(self.stats.get("vtime_at_last_clear", vt))
    st = getattr(self.arena, "stats", {})
    rep = {
        "cleared": int(n_cleared),

```

```

        "n_sources": total,
        "clear_ratio": (n_cleared / total) if total else None,
        "virtual_time_s": vt,
        "avg_clear_time_s": (last / n_cleared) if n_cleared else None,
        "program_runtime_s": float(self.stats.get("program_runtime_s", 0.0)),
        "budget_kind": self.arena.budget_kind(),
        "remaining_budget_s": self.remaining(),
        "n_measure": self.stats["n_measure"],
        "n_bearings": self.stats["n_bearings"],
        "n_clear": self.stats["n_clear"],
        "n_clear_fail": self.stats["n_clear_fail"],
        "n_scan_rounds": self.stats["n_scan_rounds"],
        "n_refine": self.stats["n_refine"],
        "n_second_bearing": self.stats.get("n_second_bearing", 0),
        "n_replans": self.stats.get("n_replans", 0),
        "n_midphase_targets": self.stats.get("n_midphase_targets", 0),
        "n_rejected": self.stats["n_rejected"],
        "travel_m": float(st.get("travel_m", 0.0)),
        "mock_stats": dict(st),
        "n_sweep_aborted": self.stats.get("n_sweep_aborted", 0),
        "n_sweep_miss": self.stats.get("n_sweep_miss", 0),
        "n_probe_issued": self.stats.get("n_probe_issued", 0),
        "n_scan_gated": self.stats.get("n_scan_gated", 0),
        "t_scan_s": self.stats.get("t_scan_s", 0.0),
        "t_clear_s": self.stats.get("t_clear_s", 0.0),
        "t_clear_ends_s": self.stats.get("t_clear_ends_s", 0.0),
        "n_leftover_located": sum(
            1 for r in self.recs.values()
            if not r.cleared and r.n_bearings >= self.cfg.clear_min_bearings),
        "scan_points": [[round(float(x), 2), round(float(y), 2)]
                        for (x, y) in self.scan_points],
        "channels": {str(ch): {"n_bearings": rec.n_bearings,
                               "cleared": bool(rec.cleared),
                               "no_signal": rec.no_signal,
                               "near": rec.near_hits,
                               "status": self.region(rec).get("status"),
                               "r_star_m": (self.bounded_region(rec) or {}).get("radius")
                              }
                     for ch, rec in self.recs.items() if rec.n_bearings or rec.cleared},
        "truth": truth,
    }
    return rep

# -----
# 自检：策略层的结构性断言（离线 mock 上跑）
# -----

def selfcheck_mock(verbose=True, seed=20260913, cases=3):
    out = []
    ok_all = True

    def rec(name, ok, detail=""):
        nonlocal ok_all
        ok_all = ok_all and bool(ok)
        out.append({"check": name, "ok": bool(ok), "detail": detail})
        if verbose:
            print(f"[{'OK' if ok else 'FAIL'}] {name} {detail}")

    # S1 覆盖式试探点确实能覆盖圆域（任意点 20 m 内有试探点）
    rb = P3Robot(MockArena(seed=1), P3Config())
    rng = np.random.default_rng(7)
    worst = 0.0
    for (cx, cy, r) in ((0.0, 0.0, 40.0), (500.0, -300.0, 80.0), (-900.0, 900.0, 25.0)):

```

```

pts = np.array(rb.probe_points((cx, cy), r))
for _ in range(400):
    a = rng.uniform(0, 2 * math.pi)
    rad = r * math.sqrt(rng.random())
    p = np.array([cx + rad * math.cos(a), cy + rad * math.sin(a)])
    d = float(np.min(np.hypot(pts[:, 0] - p[0], pts[:, 1] - p[1])))
    worst = max(worst, d)
rec("S1 覆盖式试探: 区域内任意点到最近试探点 <= 20 m (清除半径)", worst <= R_CLEAR + 1e-6,
    f"最大覆盖距离 {worst:.2f} m (试探点间距 {rb.cfg.probe_spacing_m} m)")

# S2 端到端: mock 上跑通、清除率、时间预算
ratios, times, cleared = [], [], []
for k in range(cases):
    arena = MockArena(seed=seed + 100 * k)
    rb = P3Robot(arena, P3Config(), log=None)
    rep = rb.run()
    ratios.append(rep["clear_ratio"])
    times.append(rep["avg_clear_time_s"])
    cleared.append(rep["cleared"])
    if verbose:
        print(f"  案例{seed + 100 * k}: 清除 {rep['cleared']}/{rep['n_sources']}, "
              f"虚拟时间 {rep['virtual_time_s']:.0f}s, "
              f"平均 {rep['avg_clear_time_s']:.1f}s, "
              f"扫描 {rep['n_scan_rounds']} 轮, 检测 {rep['n_measure']} 次")
rec("S2 mock 演练: 清除率>0 且每个被清除源平均时间有限",
    all(r is not None and r > 0 for r in ratios) and all(t and t > 0 for t in times),
    f"{cases} 局: 清除率 {[round(r, 3) for r in ratios]}, 平均时间 "
    f"{[round(t, 1) for t in times]} s")

# S3 策略不会超预算 (mock 会计时; 超时会 accepted=false)
arena = MockArena(seed=seed + 7, budget_s=400.0)
rb = P3Robot(arena, P3Config(), log=None)
rep = rb.run()
rec("S3 策略在时间预算内收工 (不触发超时拒动)",
    rep["n_rejected"] == 0 and rep["virtual_time_s"] <= 400.0 + 1e-9,
    f"虚拟时间 {rep['virtual_time_s']:.1f}s / 400s, 拒动 {rep['n_rejected']} 次")

# S4 已清除的频道不再被检测 ("跳过已清理过的点, 节省时间")
arena = MockArena(seed=seed + 11)
rb = P3Robot(arena, P3Config(), log=None)
rep = rb.run()
after = {}
for ch, r in rb.recs.items():
    if r.cleared and r.cleared_at is not None:
        k = sum(1 for (_x, _y, t, _res) in r.meas_log if t > r.cleared_at + 1e-9)
        if k:
            after[ch] = k
rec("S4 已清除频道不再被检测", not after,
    f"已清除 {sum(1 for r in rb.recs.values() if r.cleared)} 个频道; "
    f"清除后仍被检测的频道 {after or '无'}")

# S5 定位区域一定包含真源 (问题 1 的交会算法 + 模拟器误差界的一致性)
bad5 = []
n5 = 0
src_map = {s["channel"]: (s["x_m"], s["y_m"]) for s in (rep["truth"] or {}).get("sources", [])}
for ch, r in rb.recs.items():
    if r.n_bearings < 2:
        continue
    br = rb.bounded_region(r)
    if not br or not br.get("poly"):
        continue
    n5 += 1

```

```

xy = src_map.get(ch)
if xy is None:
    continue
P = np.asarray(br["poly"], float)
if not _point_in_polygon(np.asarray(xy, float), P):
    bad5.append(ch)
rec("S5 定位区域（多边形）包含真源", not bad5,
    f"检查 {n5} 个已定位频道, 越界 {bad5 or '无'}")

# S6 探索兜底: 没有任何方位时选"离已访问点最远"的点
rb6 = P3Robot(MockArena(seed=seed + 3), P3Config(), log=None)
sel6, _f = rb6.choose_scan_point()
rec("S6 无方位时退化为探索点（离已访问点最远）",
    sel6["kind"] == "explore" and math.hypot(*sel6["xy"]) > 1000.0,
    f"探索点 {tuple(round(v) for v in sel6['xy'])}, 距原点 "
    f"{math.hypot(*sel6['xy']):.0f} m")

# S7 动态重规划: 每清一个目标都重规划一次; 静态对照为 0
arena7 = MockArena(seed=seed + 13)
rb7 = P3Robot(arena7, P3Config(), log=None)
rep7 = rb7.run()
arena7s = MockArena(seed=seed + 13)
rb7s = P3Robot(arena7s, P3Config(dynamic_replan=False), log=None)
rep7s = rb7s.run()
rec("S7 动态重规划: 清除途中每步重规划, 静态对照 0 次",
    rep7["n_replans"] >= rep7["cleared"] >= 1 and rep7s["n_replans"] == 0,
    f"动态: 清除 {rep7['cleared']} 个、重规划 {rep7['n_replans']} 次、"
    f"中途新定位插队 {rep7['n_midphase_targets']} 个; "
    f"静态: 重规划 {rep7s['n_replans']} 次、中途插队 {rep7s['n_midphase_targets']} 个")

# S8 动态航路: 默认不裁剪（目标齐全）、顺序与航路排序一致
rb8 = P3Robot(MockArena(seed=seed + 21), P3Config(), log=None)
rb8.arena.enter()
for ch, (gx, gy) in ((1, (300.0, 200.0)), (2, (-500.0, 700.0)), (3, (900.0, -400.0))):
    r8 = ChannelRec(ch)
    # 在真源周围取两条正交视线（交会角 90°）⇒ 定位区域小且有界
    for ang, dist in ((0.0, 500.0), (90.0, 500.0)):
        sx = gx + dist * math.cos(math.radians(ang))
        sy = gy + dist * math.sin(math.radians(ang))
        a = math.degrees(math.atan2(gy - sy, gx - sx)) % 360.0
        r8.pts.append((sx, sy))
        r8.svds.append(a)
    rb8.recs[ch] = r8
tg8 = rb8.clear_targets()
tour8 = rb8.plan_clear_tour(tg8)
rec("S8 动态航路: 默认不裁剪、顺序与航路排序一致",
    len(tg8) == 3 and len(tour8) == 3
    and [t[0] for t in tour8] == rb8.order_targets(list(tg8)),
    f"{len(tg8)} 个已定位频道 -> 航路 {[t[0] for t in tour8]}, "
    f"各区域半径 {[round(t[2]['radius']) for t in tour8]} m")

# S9 覆盖式试探**不得被点数上限截断**（截断会丢掉外圈、破坏 20 m 覆盖保证）
rb9 = P3Robot(MockArena(seed=seed + 31), P3Config(), log=None)
n9_uncapped = len(rb9.probe_points(np.zeros(2), 195.0))
rb9c = P3Robot(MockArena(seed=seed + 31),
    P3Config(probe_unlimited=False, probe_max_points=60), log=None)
n9_capped = len(rb9c.probe_points(np.zeros(2), 195.0))
# 覆盖保证: r*=195 m 的外接圆内任一点到最近候选点必须 <= 20 m
pts9 = np.asarray(rb9.probe_points(np.zeros(2), 195.0), float)
rng9 = np.random.default_rng(11)
worst9 = 0.0
for _ in range(400):

```

```

a = rng9.uniform(0, 2 * math.pi)
rad = 195.0 * math.sqrt(rng9.random())
p = np.array([rad * math.cos(a), rad * math.sin(a)])
worst9 = max(worst9, float(np.min(np.hypot(pts9[:, 0] - p[0],
                                     pts9[:, 1] - p[1]))))
rec("S9 覆盖式试探不被截断 (大 r* 时仍满足 20 m 覆盖保证)",
    worst9 <= R_CLEAR + 1e-6 and n9_uncapped > n9_capped,
    f"r*=195 m: {n9_uncapped} 个候选点 (封顶对照 {n9_capped} 个), "
    f"最大覆盖距离 {worst9:.2f} m")

# S10 预算不足时**不得**把"未尝试"记成"清除失败" (失败归因三分)
arena10 = MockArena(seed=seed + 41)
rb10 = P3Robot(arena10, P3Config(), log=None)
rb10.arena.enter()
rec10 = ChannelRec(4)
for (sx, sy) in ((0.0, 0.0), (0.0, 300.0)): # 两条正交视线 -> 小而紧的区域
    a = math.degrees(math.atan2(150.0 - sy, 100.0 - sx)) % 360.0
    rec10.pts.append((sx, sy))
    rec10.svds.append(a)
rb10.recs[4] = rec10
br10 = rb10.bounded_region(rec10)
arena10.vtime = arena10.budget - 0.5 # 只剩 0.5 s: 连一次 /clear 都付不起
f10 = rb10.remaining()
st10 = rb10.sweep_clear(4, rec10, np.asarray(br10["center"], float),
                        float(br10["radius"]), poly=br10.get("poly"))
rec("S10 预算不足时不误记 clear_fail / 不发 /clear",
    st10 == SWEEP_BUDGET and rec10.clear_fail == 0
    and rb10.stats["n_probe_issued"] == 0
    and rb10.stats["n_sweep_aborted"] == 1,
    f"剩余预算 {f10:.1f} s (付不起一次 /clear) => 返回 {st10!r}, "
    f"clear_fail={rec10.clear_fail}, 发出 /clear {rb10.stats['n_probe_issued']} 次, "
    f"n_sweep_aborted={rb10.stats['n_sweep_aborted']}")

# S11 区域中心在"排序决策 -> 执行"之间**不发生漂移** (否则排序依据会过期)
arena11 = MockArena(seed=seed + 51)
rb11 = P3Robot(arena11, P3Config(), log=None)
orig_plan11 = rb11.plan_clear_tour
orig_clear11 = rb11.clear_target
snap11 = {}
drift11 = []

def plan11(targets, remaining_s=None):
    for t in (targets or []):
        snap11[t[0]] = tuple(t[2]["center"])
    return orig_plan11(targets, remaining_s=remaining_s)

def clear11(ch, rec):
    br = rb11.bounded_region(rec)
    if br is not None and ch in snap11:
        drift11.append(math.dist(snap11[ch], tuple(br["center"])))
    return orig_clear11(ch, rec)

rb11.plan_clear_tour = plan11
rb11.clear_target = clear11
rb11.run()
rec("S11 区域中心在'排序决策-执行'之间不漂移",
    bool(drift11) and max(drift11) == 0.0,
    f"检查 {len(drift11)} 次执行, 最大漂移 {max(drift11) if drift11 else float('nan'):.1f} m"
    f"(新方位只在 /measure 后产生, 逼近途中不产生)")

# S12 航路排序的三种模式: 第一步必为最近邻 (nn / nn2opt_keepfirst),
# 或允许 2-opt 改写第一步但总路程不更差 (nn2opt)

```

```

rb12 = P3Robot(MockArena(seed=seed + 61), P3Config(), log=None)
rb12.arena.enter()
pts12 = [(300.0, 200.0), (-500.0, 700.0), (900.0, -400.0),
          (-1200.0, -300.0), (200.0, -1500.0)]
for i, (gx, gy) in enumerate(pts12, start=1):
    r12 = ChannelRec(i)
    for ang in (0.0, 90.0):
        sx = gx + 500.0 * math.cos(math.radians(ang))
        sy = gy + 500.0 * math.sin(math.radians(ang))
        r12.pts.append((sx, sy))
        r12.svds.append(math.degrees(math.atan2(gy - sy, gx - sx)) % 360.0)
    rb12.recs[i] = r12
tg12 = rb12.clear_targets()
start12 = tuple(rb12.pos)
nearest12 = min(tg12, key=lambda t: math.dist(start12, t[2]["center"]))[0]

def plen12(order):
    seq = [next(t for t in tg12 if t[0] == ch) for ch in order]
    d = math.dist(start12, seq[0][2]["center"])
    for a, b in zip(seq, seq[1:]):
        d += math.dist(a[2]["center"], b[2]["center"])
    return d

rb12.cfg.tour = "nn"
o_nn = rb12.order_targets(list(tg12))
rb12.cfg.tour = "nn2opt_keepfirst"
o_kf = rb12.order_targets(list(tg12))
rb12.cfg.tour = "nn2opt"
o_2o = rb12.order_targets(list(tg12))
rec("S12 航路排序: nn 与 keepfirst 第一步为最近邻; 2-opt 不劣于 nn",
    o_nn[0] == nearest12 and o_kf[0] == nearest12
    and plen12(o_2o) <= plen12(o_nn) + 1e-9,
    f"最近目标=频道{nearest12}; nn 第一步={o_nn[0]}, keepfirst={o_kf[0]}, "
    f"2-opt={o_2o[0]}; 路程 nn={plen12(o_nn):.0f} m, 2-opt={plen12(o_2o):.0f} m")

if verbose:
    print(f"\n自检总结: {'全部通过' if ok_all else '存在失败项'}")
return {"ok": ok_all, "checks": out}

def main():
    ap = argparse.ArgumentParser(description="问题3 机器狗策略（离线自检 / 单局演练）")
    ap.add_argument("--selfcheck", action="store_true")
    ap.add_argument("--mock", action="store_true", help="跑一局离线演练并打印明细")
    ap.add_argument("--seed", type=int, default=20260913)
    ap.add_argument("--dist-weight", type=float, default=1.5)
    ap.add_argument("--grid-step", type=float, default=100.0)
    ap.add_argument("--quiet", action="store_true")
    args = ap.parse_args()
    if args.selfcheck:
        res = selfcheck_mock(verbose=True)
        raise SystemExit(0 if res["ok"] else 1)
    cfg = P3Config(dist_weight=args.dist_weight, grid_step=args.grid_step)
    arena = MockArena(seed=args.seed)
    rb = P3Robot(arena, cfg, log=None if args.quiet else print)
    rep = rb.run()
    print(json.dumps({k: v for k, v in rep.items()
                     if k not in ("truth", "channels", "mock_stats")},
                     ensure_ascii=False, indent=2))

if __name__ == "__main__":

```

```
main()
```

5.6 src/p3_sweep.py

"""问题 3 的"覆盖扫圈"策略（与 `p3\_robot.P3Robot` 的"期望场贪心"并列可选）。

设计动机（来自 2026-09-11 两局在线演练的实测）

1. ****不必"发现即清除"****：实测第 2 局的 14 个源在 3353 s 就全部清完，程序又空转 1932 s（37%）——因为它不知道"已经清完"。反过来，第 1 局有 3114 s（45%）花在"场上一条方位都没有"的阶段，决策场 $E_{\text{norm}} \equiv 0$ 、score 只剩距离项，机器人就在圆心附近 500 m 步长乱蹭。
2. ****"1000 m 已扫过"是可以判定的****：干扰源的有效接收半径 $\geq R_{\text{RECV_MIN}} = 1000 \text{ m}$ 。因此"某点距所有已扫检测点都 $> 1000 \text{ m}$ " $\Rightarrow$ "该点不可能存在未清除的源"是****严格****的。把靶区按 1000 m 的覆盖半径铺开，就能得到一个****可证明完备****的扫描航线：走完即"靶区内不存在未清除源"，可以立刻 ``/exit``，不需要盲探、不需要走回头路。
3. ****原点那一次检测白送了内圈****：原点已扫 $\Rightarrow$ 半径 1000 m 以内不必再去。需要覆盖的只剩 $r \in [1000, 1800]$ 的环带，因此停靠点不必贴边，沿 $r \approx 1100 / 1500 / 1800$ 三圈铺开即可（见 `:func:`lattice_candidates``）。

策略本体

```
-----
score(g) = w_cover * (g 的 1000 m 圆内"未覆盖"面积占比)
          - w_dir   * cos(方位 - 期望前进方位)          # 保持顺时针推进，不回头
          - w_near  * (1 - d/最大可能距离)              # 就近取点，别横穿靶区

pick = argmax score      （只从格点候选里选；已停靠点 300 m 邻域排除）

每到一个停靠点：先 `clear_targets()`（顺路清除已经定位好的源），再测该点值得测的频道。
覆盖率达到 100%（或时间不够）就收工——**因为此时可以确定没有残留源**。
"""
from __future__ import annotations

import math
import time
from dataclasses import dataclass

import numpy as np

from p3_arena import ( # noqa: E402
    ArenaError, R_ARENA, R_CLEAR, R_RECV_MIN, R_RECV_MAX, T_CLEAR_FAIL, T_CLEAR_OK,
    T_MEASURE, T_SWITCH, V_ROBOT,
)
from p3_robot import P3Config, P3Robot # noqa: E402
from p3_expect_field import ( # noqa: E402
    best_refine_point, exact_expected_diameter, ray_source_samples,
)

# -----
# 候选停靠点：同心环格点（"扫圈"航线就是在这些点之间走）
# -----
DEFAULT_RINGS = ((1100.0, 8),)
"""`(半径 m, 该圈上的点数)`。默认只用一圈；如有需要可在后面追加"补漏圈"。

为什么一圈就够 —— 这是把题目的物理条件用足之后的结论：

* 干扰源有效接收半径  $\geq R_{\text{RECV\_MIN}} = 1000 \text{ m}$   $\Rightarrow$  某点距**所有**检测点都  $> 1000 \text{ m}$  时，
  该处不可能存在未清除源（**严格**，不是启发式）；
* 原点已经测过一次（20 个频道全扫），所以 **r  $\leq 1000 \text{ m}$  的整个内圈不必再去**；
```


* 于是只需要覆盖环带 $r \in [1000, 1800]$ 。数值枚举（`python src/p3\_sweep.py --coverage`）：

```
| 单圈 | 环带内最大未覆盖距离 | 30 局虚拟时间 | 30 局清除率 | 30 局全清 |
|---|---|---|---|---|
| **r=1100, n=8 (默认) ** | **853 m** | **4243 s** | 0.978 | 23/30 |
| r=1150, n=8 | — | 4237 s | 0.970 | 21/30 |
| r=1250, n=8 | 788 m | 4516 s | 0.973 | 22/30 |
| r=1400, n=8 | 721 m | 4621 s | 0.981 | **25/30** |
```

都完备（ < 1000 m）。**r=1100 最快、全清数居中**；r=1400 最稳（全清 25/30）但慢 9%。

若把“全清”看得比时间更重，可一行改成 `DEFAULT\_RINGS = ((1400.0, 8),)`。

"""

```
def lattice_candidates(rings=DEFAULT_RINGS, stagger=True):
    """同心环格点：`[(x, y), ...]`。相邻环之间做半格错开，避免放射状缝隙。"""
    out = []
    for k, (r, n) in enumerate(rings):
        off = (math.pi / n) if (stagger and k % 2 == 1) else 0.0
        for i in range(int(n)):
            a = off + 2.0 * math.pi * i / n
            out.append((float(r) * math.cos(a), float(r) * math.sin(a)))
    return out
```

```
def rotate_lattice(cand, level, frac, ring_n):
    """把某个圈的候选点整体旋转 ``frac`` 个站位（第 2 圈错开半格用）。"""
    out = []
    step = 2.0 * math.pi / max(int(ring_n), 1)
    for k, (x, y) in enumerate(cand):
        a = math.atan2(float(y), float(x)) + frac * step
        r = math.hypot(float(x), float(y))
        out.append((r * math.cos(a), r * math.sin(a)))
    return out
```

```
def coverage_report(pts, cover_r=R_RECV_MIN, step=25.0, r_arena=R_ARENA,
                    r_inner=0.0):
    """给定停靠点集合，返回 ``(最大未覆盖距离, 未覆盖面积占比, 覆盖率)``。

    "未覆盖距离" = 评估区内某点到最近停靠点的距离；它  $> \text{cover\_r}$  即说明该处
    若有源则必然已被听到（源的接收半径  $\geq \text{cover\_r}$ ）—— 所以这是**完备性度量**。


``r_inner``：只评估  $r_{\text{inner}} \leq |p| \leq r_{\text{arena}}$  的环带。原点已扫过时，
    ``r  $\leq R_{\text{recv\_min}}$ `` 的内圈无需再覆盖，评估环带才是正确的口径。


    """
    R = float(r_arena)
    xs = np.arange(-R, R + 1e-9, step)
    GX, GY = np.meshgrid(xs, xs)
    rr = np.hypot(GX, GY)
    m = (rr <= R + 1e-9) & (rr >= float(r_inner) - 1e-9)
    gx, gy = GX[m], GY[m]
    if not pts:
        return float("inf"), 1.0, 0.0
    P = np.asarray(pts, float)
    d2 = ((gx[:, None] - P[None, :, 0]) ** 2 + (gy[:, None] - P[None, :, 1]) ** 2)
    dmin = np.sqrt(np.min(d2, axis=1))
    unc = dmin > float(cover_r)
    return float(np.max(dmin)), float(unc.mean()), float(1.0 - unc.mean())
```

```

# 配置
# -----
@dataclass
class SweepConfig(P3Config):
    """在 `P3Config` 上追加"扫圈"参数（继承 `P3Config` 的全部字段 + 下面这些）。"""
    sweep_rings: tuple = DEFAULT_RINGS
    cover_radius_m: float = R_RECV_MIN # 覆盖判定半径（有效接收半径下界）
    cover_w: float = 1.0 # 未覆盖收益的权重
    near_w: float = 0.25 # 就近权重（避免横穿靶区）
    jump_w: float = 0.00035 # 跳跃惩罚（每米），抑制"绕回已走过的扇区"
    min_gain: float = 0.25 # 低于此边际收益就不值得专门跑一趟
    stop_visited_radius_m: float = 300.0 # 已停靠点邻域排除半径
    # ---- 清除时机 ----
    clear_mode: str = "detour" # detour（只清顺路的）| station（每站都清）| none
    clear_detour_max_m: float = 350.0 # detour 模式：绝对距离上限（m）
    onlap_clear: bool = False # **边绕边清**：一圈走完就清完，不再有"圈后统一清除"
    onlap_cos: float = 0.10 # 只在"前方"这个夹角内（cos）的候选才顺路处理
    onlap_max_m: float = 1500.0 # 候选(或单方位源的推算位置)离本站的最远距离
    onlap_min_m: float = 120.0 # 比这更近的目标不单独绕（到站测量时顺手清）
    onlap_single: bool = True # 单方位频道也用"推算位置"参与顺路判定
    onlap_after_ring: bool = False # 走完圈后再清一次剩余的已定位目标
    onlap_step_m: float = 320.0 # ★"取第二方位"的侧移步长上限（m）：小步就够，
    # 不要跑到推算位置去（实测原来跑 1400-1600 m）
    onlap_step_frac: float = 0.25 # 侧移步长 = min(onlap_step_m, 该比例 × 到目标的距离)
    onlap_min_step_m: float = 140.0 # 步长下限（太短交会角没改善）
    onlap_angle_deg: float = 50.0 # 侧移方向与旧视线的夹角（交会角 ≈ 这个值）
    onlap_max_steps: int = 3 # 最多走几步去把区域压到可清除
    skip_station_near_target_m: float = 260.0 # 站位离"顺路目标"这么近时可考虑省掉该站
    skip_station_max_uncovered: float = 1150.0 # 省站后"最大未覆盖距离"仍须低于它
    start_toward_sources: bool = False # 第一站朝"信息方向"出发（实测有 bug 且无收益，见 review §6.2）
    debug_onlap: bool = False # 打印"边绕边清"的候选判定过程
    start_min_sources: int = 1 # 至少这么多条单方位才据此定起始方向
    # 下面两个"顺路"判据默认**关闭**（实测在 30 局上都没有正向收益，见 review/p3-sweep-design.md §6）：
    enroute_min_cos: float = 1.1 # >1 表示关闭"按方向判定顺路"（仅用 350 m 兜底）
    enroute_max_m: float = 1400.0 # 按方向判定时的绕行距离上限
    initial_locate: bool = True # **开场先按期望场行动一次**
    initial_locate_max_m: float = 1400.0 # 只对"首条方位距离 ≤ 此值"的频道做开场定点
    batch_locate: bool = False # 多个"只有一条方位"的频道**合并规划**第二测点
    # （实测每局触发约 2 次、每次确实更省，但会把
    # 机器狗拉离环线，整局行程反而 +12%，默认关）
    batch_max_m: float = 700.0 # 聚类阈值：彼此最优测点相距 ≤ 此值才算"可以合并"
    batch_min_channels: int = 2 # 至少这么多条单方位才值得合并规划
    batch_max_rounds: int = 6 # 一局里最多做几次"合并定位"
    batch_every: int = 2 # 每走这么多站就考虑一次合并定位
    batch_min_gain_s: float = 60.0 # 联合方案至少要省这么多秒才值得做
    bear_cone_deg: float = 0.0 # >0 才启用"单方位频道按行进方向就地补测"
    sweep_all_rounds: int = 1 # 完整走圈的遍数（第 2 遍只补缺口）
    lap_offsets: tuple = (0.0, 0.5) # 每一圈的角向错开量（单位：站位格）
    measured_revisit_gap_m: float = 400.0 # 已测过该频道但没听到 → 换这么远再测

# -----
# 机器狗：覆盖扫圈
# -----
class SweepRobot(P3Robot):
    """覆盖扫圈。注意：`SweepConfig` 是 dataclass，其额外参数直接传给 `P3Config` 的
    ``__init__``（会把未知键忽略），所以覆盖参数通过 ``cfg`` 对象设置，见 :meth:`_apply_cfg`。
    """

    def _apply_cfg(self, cfg):
        self.cfg = cfg

```

```

def __init__(self, arena, cfg: SweepConfig | None = None, base=None, log=None):
    super().__init__(arena, cfg or SweepConfig(), base=base, log=log)
    self.cand = np.asarray(lattice_candidates(self.cfg.sweep_rings), float)
    # 每个候选点属于第几圈 (0 = 主圈; >0 = 补漏圈, 只在主圈走完还有缺口时启用)
    self.cand_level = []
    for k, (_, n) in enumerate(self.cfg.sweep_rings):
        self.cand_level += [k] * int(n)
    self.cand_level = np.asarray(self.cand_level, int)
    self.ring_n = int(self.cfg.sweep_rings[0][1]) if self.cfg.sweep_rings else 0
    self.ring_r = float(self.cfg.sweep_rings[0][0]) if self.cfg.sweep_rings else 0.0
    self.ring_dir = 0.0          # 0 = 还没出发; ±1 = 已定的绕圈方向
    self.ring_last_a = None      # 上一站的极角
    self.ring_laps = 0
    self.lap = 0                 # 第几圈 (0 起): 每圈把站位错开 lap_offsets[lap] 格
    self._last_dir = None        # 上一段的行进方向 (用于"单方位频道是否顺路补测")
    self.visited = []            # **真实**到过的位置 (含清除绕行点), 用于排除已停靠点
    self.meas_pts = []           # **真实做过的检测**位置, 用于可证明的覆盖度评估
    self._cg = None              # 覆盖度评估用细网格 (缓存)
    self.coverage = 0.0
    self.max_uncov_dist = float("inf")
    self.phase = "scan"

# ----- 覆盖度 -----
def _grid(self):
    if self._cg is None:
        step = 25.0
        R = R_ARENA
        xs = np.arange(-R, R + 1e-9, step)
        GX, GY = np.meshgrid(xs, xs)
        m = np.hypot(GX, GY) <= R + 1e-9
        self._cg = (GX[m], GY[m])
    return self._cg

def note_pos(self, ch=None):
    """记录一次**真实做过的检测**位置, 用于覆盖度评估。

    为什么要记"检测位置"而不是"到过的位置": 清除阶段会把机器人带到源附近的
    试探格点, 那些点**确实测过** (我们在那儿测到方位、也听过无信号), 所以算有效;
    但"到过"并不等于"测过每个频道", 所以覆盖度必须按**检测记录**算才是可证明的:
    "每个靶区点到某个检测点的距离 ≤ 1000 m" ⇒ 该点若有源且当时在测它, 必已被听到。
    """
    p = (float(self.arena.pos[0]), float(self.arena.pos[1]))
    if not self.meas_pts or math.dist(self.meas_pts[-1], p) > 1.0:
        self.meas_pts.append(p)

def coverage_now(self):
    """覆盖率 + 最大未覆盖点距最近**检测点**的距离 (可证明完备的判据)。"""
    gx, gy = self._grid()
    pts = list(self.meas_pts) or list(self.scan_points)
    if not pts:
        return 0.0, float("inf")
    P = np.asarray(pts, float)
    d2 = ((gx[:, None] - P[None, :, 0]) ** 2 + (gy[:, None] - P[None, :, 1]) ** 2)
    dmin = np.sqrt(np.min(d2, axis=1))
    unc = dmin > float(self.cfg.cover_radius_m)
    self.max_uncov_dist = float(np.max(dmin))
    self.coverage = float(1.0 - unc.mean())
    return self.coverage, self.max_uncov_dist

# ----- 选点 (沿圈单调推进, 不回头) -----
def _cand_xy(self, idx):
    """第 ``lap`` 圈、第 ``idx`` 个候选点的坐标 (按 ``lap_offsets`` 旋转)。"""

```

```

offs = self.cfg.lap_offsets or (0.0,)
frac = float(offs[min(self.lap, len(offs) - 1)])
c = self.cand[idx]
if abs(frac) < 1e-12:
    return float(c[0]), float(c[1])
step = 2.0 * math.pi / max(self.ring_n, 1)
a = math.atan2(float(c[1]), float(c[0])) + frac * step
r = math.hypot(float(c[0]), float(c[1]))
return float(r * math.cos(a)), float(r * math.sin(a))

def pick_stop(self, level=0):
    """选下一站: **沿当前绕圈方向单调推进**, 只在"还没覆盖"的站里挑。

    打分 ``score = w_cover * 边际覆盖 - w_near * (d/2R) - w_jump * 前向角增量(米)``,
    并对"反向站"直接剔除 (``adv < min_step``)。这样航线就自然是
    "走一段 → 沿一个方向绕圈 → 走满一圈收工", 不会出现来回横跳或走回头路。

    ``level``: 只用第 ``level`` 圈 (以及更低层) 的候选点。主圈走完仍有缺口时,
    调用方会升到补漏圈。
    """
    gx, gy = self._grid()
    P = np.asarray(self.visited, float) if self.visited else None
    if P is None:
        covered = np.zeros(gx.size, bool)
    else:
        d2 = ((gx[:, None] - P[None, :, 0]) ** 2 + (gy[:, None] - P[None, :, 1]) ** 2)
        covered = np.min(d2, axis=1) <= float(self.cfg.cover_radius_m) ** 2
    p = np.asarray(self.arena.pos, float)
    step = 2.0 * math.pi / max(self.ring_n, 1)
    min_step = 0.35 * step          # 至少要往前走这么多角度
    best, best_score, best_info = None, -1e18, {}
    for idx, c0 in enumerate(self.cand):
        if int(self.cand_level[idx]) > int(level):
            continue
        c = self._cand_xy(idx)
        dc = float(np.hypot(c[0] - p[0], c[1] - p[1]))
        if dc < self.cfg.min_move:
            continue
        if any(float(np.hypot(c[0] - q[0], c[1] - q[1]))
                < self.cfg.stop_visited_radius_m for q in self.scan_points):
            continue
        in_disk = ((gx - c[0]) ** 2 + (gy - c[1]) ** 2) <= float(self.cfg.cover_radius_m) ** 2
        cnt = int(np.count_nonzero(in_disk))
        if cnt == 0:
            continue
        gain = float(np.count_nonzero(in_disk & ~covered)) / cnt      # [0,1]
        if gain < self.cfg.min_gain:
            continue
        a = math.atan2(float(c[1]), float(c[0]))
        if self.ring_last_a is None:
            adv = 0.0          # 第一站: 方向未定, 就近去
        else:
            raw = a - self.ring_last_a
            if self.ring_dir >= 0:          # 未定方向时按"逆时针为正"试探
                adv = (raw + 2.0 * math.pi) % (2.0 * math.pi)
            else:
                adv = -(((raw) + 2.0 * math.pi) % (2.0 * math.pi))
            if adv < min_step:
                continue          # 反向或原地: 不回头
        near_term = 1.0 - dc / (2.0 * R_ARENA)
        jump = max(0.0, adv) * self.ring_r
        score = (self.cfg.cover_w * gain + self.cfg.near_w * near_term

```

```

        - self.cfg.jump_w * jump)
    if score > best_score:
        best, best_score = c, score
        best_info = {"gain": gain, "dist": dc, "adv_deg": math.degrees(adv),
                     "jump_m": jump, "level": int(self.cand_level[idx]),
                     "lap": self.lap,
                     "n_new": int(np.count_nonzero(in_disk & ~covered)),
                     "cnt": cnt}
    return best, best_info

def commit_stop(self, c):
    """记录这一站，并据此确定/保持绕圈方向。"""
    a = math.atan2(float(c[1]), float(c[0]))
    step = 2.0 * math.pi / max(self.ring_n, 1)
    if self.ring_last_a is None:
        self.ring_last_a = a
        return
    raw = (a - self.ring_last_a + math.pi) % (2.0 * math.pi) - math.pi
    if self.ring_dir == 0.0:
        # 第一段的方向即定为绕圈方向；角度增量接近 0（小于半格）时沿用正向
        self.ring_dir = 1.0 if raw >= 0 else -1.0
    if (self.ring_dir > 0 and raw < -step / 2.0) or \
        (self.ring_dir < 0 and raw > step / 2.0):
        self.ring_laps += 1          # 说明已经绕回这一圈的起点附近
    self.ring_last_a = a

# ----- 到站动作 -----
# ----- 何时可以确定"已经清完了" -----
def all_heard(self):
    """是否每个频道都"给出过明确结论"：已清除，或已测到过方位。

    这是比"覆盖 100%"更早、也更容易达到的**完工判据**：一圈走下来，
    每个频道要么被测到过方位（说明有源、随后会被清掉），要么一次都没测到。
    配合"整圈已走完"这一事实（整圈  $\Rightarrow$  每个靶区点都距某个检测点  $\leq 1000$  m
     $\Rightarrow$  若有源必被听到过），就能断定**没有再多的源了**。
    """
    for rec in self.recs.values():
        if rec.cleared:
            continue
        if rec.n_bearings == 0 and not rec.meas_pts:
            return False
    return True

def channels_here(self, pos=None, incoming_dir=None):
    """本停点值得测的频道。

    * 还缺方位的频道（未清除且方位数 < 2）一律要测；
    * **只有一条方位、且那条第方位大致指向"我们的前进方向"** 的频道也测
      —— 从当前位置看它、和第一次看它的方向差了接近 90°，交会角好，
      拿到第二条方位就能当场定位并顺路清掉（对应"不要事后走回头路"）；
    * "以前测过但没听到"的频道，若本点到它所有历史检测点都  $\geq$ 
      ``measured_revisit_gap_m`` / ``repeat_gap_m``，也再测一次。

    注意用的是**真实位置** ``pos``（清除绕行之后的位置），否则会把"计划坐标"
    当成已测过，从而在新站点漏测大量频道。
    """
    p = np.asarray(self.arena.pos if pos is None else pos, float)
    cos_need = math.cos(math.radians(self.cfg.bear_cone_deg))
    out = []
    for ch in self.needed_channels():
        rec = self.recs[int(ch)]
        if not rec.meas_pts:

```

```

        out.append(int(ch))
        continue
    gap = (self.cfg.repeat_gap_m if rec.pts
           else self.cfg.measured_revisit_gap_m)
    d = min(math.hypot(p[0] - q[0], p[1] - q[1]) for q in rec.meas_pts)
    if d >= gap:
        out.append(int(ch))
        continue
    # 已有 1 条方位但"离历史检测点太近"被 gap 挡住的频道:
    # 若那条方位与行进方向夹角 < bear_cone_deg, 就地补第二方位。
    if rec.n_bearings == 1 and incoming_dir is not None:
        (sx, sy), a = rec.pts[-1], rec.svds[-1]
        u = np.array([math.cos(math.radians(a)), math.sin(math.radians(a))])
        if float(np.asarray(incoming_dir, float) @ u) >= cos_need:
            out.append(int(ch))

    return out

# ----- 边绕边清: 候选目标 (含"只有一条方位"的推算位置) -----
def _predicted_source(self, rec):
    """给"只有一条方位"的频道一个**位置估计**, 好让它在顺路判定里参与竞争。

    做法: 把那条视线与"以原点为心、半径 ``R_RECV_MAX`` 的圆"求交, 取较远的交点。
    这个估计误差最大约  $\pm R_RECV_MAX \cdot \tan 1^\circ \approx 26$  m 的横向、以及沿视线的未知距离,
    但用来判断"这个源是不是在我前方、值不值得顺路去清"已经足够。
    """
    (sx, sy), a = rec.pts[-1], rec.svds[-1]
    u = np.array([math.cos(math.radians(a)), math.sin(math.radians(a))])
    S = np.array([float(sx), float(sy)])
    b = float(S @ u)
    c = float(S @ S) - (R_ARENA * R_ARENA)
    disc = b * b - c
    t = (-b + math.sqrt(disc)) if disc > 0 else R_RECV_MAX
    t = min(t, R_RECV_MAX * 1.0)
    return S + t * u

def _onlap_candidates(self, x, y, travel_dir):
    """本段"顺路就该处理"的候选, 按"越正前方、越近"排序。

    返回 ``[(kind, ch, point, cos, dist), ...]``, ``kind``  $\in$  {'clear', 'locate'}:

    * ``'clear'``: 已经定位 ( $\geq 2$  条方位且有界) ——直接去清;
    * ``'locate'``: **只有一条方位**——先按推算位置过去, 在那儿补一条方位把它定住再清。
      这正是用户指出的缺口: 旧实现只挑"已定位"的目标, 于是前方那些"只测到一次"的
      点全被跳过, 最后只能绕完圈再回头找。
    """
    out = []
    p = np.asarray((x, y), float)
    for (ch, rec, br) in self.clear_targets():
        c = np.asarray(br["center"], float)
        v, d = c - p, float(np.linalg.norm(c - p))
        if d > self.cfg.onlap_max_m or d < self.cfg.onlap_min_m:
            continue
        cs = 1.0 if d < 1e-9 else float((v / d) @ travel_dir)
        if cs >= self.cfg.onlap_cos:
            out.append(("clear", ch, c, cs, d))
    if self.cfg.onlap_single:
        for ch, rec in self.recs.items():
            if rec.cleared or rec.n_bearings != 1:
                continue
            if rec.clear_fail > 0:      # 刚失败过, 先换别的
                continue
            c = self._predicted_source(rec)

```

```

        v, d = c - p, float(np.linalg.norm(c - p))
        if d > self.cfg.onlap_max_m or d < self.cfg.onlap_min_m:
            continue
        cs = 1.0 if d < 1e-9 else float((v / d) @ travel_dir)
        if cs >= self.cfg.onlap_cos:
            out.append(("locate", ch, c, cs, d))
    out.sort(key=lambda t: (-t[3], t[4]))
    return out

def _bearing_step(self, rec, p):
    """**小步定位点**：从当前位置沿"与旧视线成  $\approx$  onlap_angle_deg"的方向走一小步。

    * 交会角就是这个夹角（几何上最优是  $90^\circ$ ，但  $50^\circ - 60^\circ$  已经把横向误差压到  $1/\sin$  倍，
      再大就要多走路）；
    * 步长取 ``min(onlap_step_m, frac * 到目标推算位置的距离)`` 并有下限与上限。
      旧实现直接跑到"推算位置"（距离上限 R_RECV_MAX=1500 m！），一次侧移就 1400 - 1600 m，
      这正是路径图里那些长支线的来源。
    """
    (sx, sy), a = rec.pts[-1], rec.svds[-1]
    u = np.array([math.cos(math.radians(a)), math.sin(math.radians(a))])
    p = np.asarray(p, float)
    # 目标大致在视线上、距离未知：用"当前位置沿视线方向的投影"当作尺度
    t_hat = float((p - np.array([sx, sy])) @ u)
    t_hat = min(max(t_hat, 200.0), R_RECV_MAX)
    ang = math.radians(self.cfg.onlap_angle_deg)
    best = None
    for sgn in (+1.0, -1.0):
        perm = np.array([u[0] * math.cos(sgn * ang) - u[1] * math.sin(sgn * ang),
                        u[0] * math.sin(sgn * ang) + u[1] * math.cos(sgn * ang)])
        step = min(self.cfg.onlap_step_m,
                  max(self.cfg.onlap_min_step_m, self.cfg.onlap_step_frac * t_hat))
        q = p + perm * step
        if np.hypot(*q) > R_ARENA - 1.0:
            continue
        d = float(np.linalg.norm(q - p))
        if best is None or d < best[0]:
            best = (d, q)
    return (best[1] if best else None), (best[0] if best else None)

def _locate_and_clear(self, ch, predicted):
    """顺路把一个"只有一条方位"的频道定位并清掉：**小步侧移  $\rightarrow$  量  $\rightarrow$  再清**。

    与旧实现的区别：旧版走到"推算位置"（可能 1500 m 外）再量，量完还常常不在
    清除半径内，于是又要侧移/试探，来回好几趟；新版每次只走一小步、
    量一次就把区域压小一半以上，通常 1 步就能进清除流程。
    """
    rec = self.recs[int(ch)]
    n = 0
    for _step in range(max(1, self.cfg.onlap_max_steps)):
        if rec.cleared or self.aborted:
            break
        br = self.bounded_region(rec)
        if br is not None and br["radius"] <= self.cfg.clear_max_radius_m:
            self.phase = "clear"
            n += 1 if self.clear_target(int(ch), rec) else 0
            self.phase = "scan"
            if rec.cleared:
                return n
        q, trav = self._bearing_step(rec, self.arena.pos)
        if q is None or trav is None:
            break
        if not self.can_afford(trav / V_ROBOT + T_MEASURE + T_SWITCH + 10.0):

```

```

        break
    self.log(f"  顺路定位频道{ch}: 侧移 {trav:.0f} m 到 "
            f"({q[0]:.0f},{q[1]:.0f}) 取方位"
            f" (步长上限 {self.cfg.onlap_step_m:.0f} m, 交会角 "
            f"~{self.cfg.onlap_angle_deg:.0f}°)")
    resp = self.measure(float(q[0]), float(q[1]), int(ch))
    self.note_pos()
    if resp.get("measure_result") == "near":
        return n + (1 if self.do_clear(float(q[0]), float(q[1]), int(ch)) else 0)
    if resp.get("measure_result") != "direction":
        rec.no_signal += 1
        break
    br = self.bounded_region(rec)
    if br is not None and not rec.cleared:
        self.phase = "clear"
        n += 1 if self.clear_target(int(ch), rec) else 0
        self.phase = "scan"
    return n

def _onlap_handle(self, kind, ch, pt):
    """顺路处理一个候选: ``locate`` 走小步补方位再清, ``clear`` 直接清。"""
    rec = self.recs[int(ch)]
    if kind == "locate":
        return self._locate_and_clear(int(ch), pt)
    br = self.bounded_region(rec)
    if br is None:
        return 0
    self.phase = "clear"
    n = 1 if self.clear_target(int(ch), rec) else 0
    self.phase = "scan"
    return n

# ----- 起始站: 朝"信息最多的方向"出发 -----
def pick_start_stop(self):
    """把第一站定在"单方位源聚集的那个方向", 并据此确定绕圈方向。

    为什么重要: 旧实现的第一站恒为格点里角度最小的那个(正东), 于是当所有方位
    都指向西/南时, 机器人**背着源**出发 —— 一整圈里都要绕回来(用户看到的
    "绕了两圈"里就有这一段)。原点的 20 次检测已经把每个源的方向给出来了,
    用它来定起始方向是"免费的"。
    """
    if not self.cfg.start_toward_sources:
        return None, None
    vecs = []
    for rec in self.recs.values():
        if rec.cleared or rec.n_bearings != 1:
            continue
        p = self._predicted_source(rec)
        d = float(np.hypot(*p))
        if d < 1e-6:
            continue
        vecs.append(p / d)
    if len(vecs) < self.cfg.start_min_sources:
        return None, None
    mean = np.sum(np.asarray(vecs, float), axis=0)
    if float(np.linalg.norm(mean)) < 1e-6:
        return None, None
    a_mean = math.atan2(float(mean[1]), float(mean[0]))
    # 在格点里挑与"信息方向"最接近的一站
    offs = self.cfg.lap_offsets or (0.0,)
    frac = float(offs[0])
    step = 2.0 * math.pi / max(self.ring_n, 1)

```



```

best, best_d = None, None
for idx in range(len(self.cand)):
    ang = math.atan2(float(self.cand[idx][1]), float(self.cand[idx][0])) + frac * step
    diff = abs((ang - a_mean + math.pi) % (2 * math.pi) - math.pi)
    if best_d is None or diff < best_d:
        best, best_d = idx, diff
c = self._cand_xy(best)
# 绕圈方向: 选"下一站"更靠近信息方向的那个方向
nxt = []
for sgn in (+1, -1):
    j = (best + sgn) % len(self.cand)
    cj = self._cand_xy(j)
    nxt.append((abs((math.atan2(cj[1], cj[0]) - a_mean + math.pi)
                    % (2 * math.pi) - math.pi), sgn))
self.ring_dir = 1.0 if nxt[0][0] <= nxt[1][0] else -1.0
# 注意: **不要**在这里设置 ring_last_a —— 第一站的角度要由 commit_stop 记录,
# 否则"单调前进"的约束会从"信息方向"起算, 导致真正的第一站被当成"往回走"而全部淘汰。
self.log(f"起始方向: {len(vecs)} 个频道只有单条方位, 其平均方向 "
        f"{math.degrees(a_mean):.0f}° -> 第一站定在 "
        f"({c[0]:.0f},{c[1]:.0f}), 绕圈方向 "
        f"{'逆时针' if self.ring_dir > 0 else '顺时针'}")
return c, best

def _target_near_station(self, c):
    """如果有"已定位/可定位目标"的位置就在计划站位 ``c`` 附近, 返回 ``(ch, center)``。

    用途: **用目标位置顶替环上站位**。清一个离站位 200 m 的源时, 机器狗已经站在
    那一带并做过检测 —— 让这次检测顶替"专门跑到站位再测一次", 只要覆盖度仍然够,
    就省下一个路径点(用户指出的"两个点走一遍")。
    """
    best = None
    cands = []
    for (ch, rec, br) in self.clear_targets():
        cands.append((ch, np.asarray(br["center"], float)))
    for ch, rec in self.recs.items():
        if rec.cleared or rec.n_bearings != 1 or rec.clear_fail > 0:
            continue
        cands.append((ch, self._predicted_source(rec)))
    for ch, ctr in cands:
        d = float(np.linalg.norm(ctr - np.asarray(c, float)))
        if d <= self.cfg.skip_station_near_target_m:
            if best is None or d < best[2]:
                best = (ch, ctr, d)
    return best

def station_skippable(self, c, alt_pts):
    """若用 ``alt_pts`` (顺路已经会去的点) 顶替站位 ``c`` 后, 覆盖度仍然够, 则返回 True。

    判据: 去掉 ``c``、加上 ``alt_pts`` 后, "靶区内最大未覆盖距离"仍 <
    ``skip_station_max_uncovered`` (比严格的 1000 m 留一点余地, 因为这个判据在
    过程中会反复评估, 留余量可以避免把后面的补测机会也一起砍掉)。
    """
    if not self.cfg.onlap_clear:
        return False
    try:
        cov_rep, _ = self.coverage_now()
    except Exception:
        return False
    keep = list(self.scan_points) + [(float(p[0]), float(p[1])) for p in alt_pts]
    keep = [q for q in keep if float(np.hypot(q[0] - c[0], q[1] - c[1]))
            > self.cfg.stop_visited_radius_m]
    if not keep:

```

```

        return False
    gx, gy = self._grid()
    P = np.asarray(keep, float)
    d2 = ((gx[:, None] - P[None, :, 0]) ** 2 + (gy[:, None] - P[None, :, 1]) ** 2)
    dmin = np.sqrt(np.min(d2, axis=1))
    if float(np.max(dmin)) > self.cfg.skip_station_max_uncovered:
        return False
    return True

def _enroute_target(self, x, y, travel_dir):
    """挑一个"顺路"的目标：位于本段行进方向的前方（夹角  $\cos \geq \text{enroute\_min\_cos}$ ）。

    为什么不能只看绝对距离：站与站之间相距 842 - 2000 m，目标常常"就在前方 600 - 900 m"
    却离本站超过 350 m —— 旧判据直接跳过，最后只能绕完一圈再回头（就是"几乎转了两圈"
    的来源）。改成按**方向**判定后，这类目标会当场清掉，回程自然消失。
    """
    best, best_key = None, None
    for (ch, rec, br) in self.clear_targets():
        c = np.asarray(br["center"], float)
        v = c - np.asarray((x, y), float)
        d = float(np.linalg.norm(v))
        if d > self.cfg.enroute_max_m:
            continue
        cos = 1.0 if d < 1e-9 else float((v / d) @ travel_dir)
        if cos < self.cfg.enroute_min_cos:
            continue
        key = (-cos, d) # 越"正前方"越好；同方向取近的
        if best_key is None or key < best_key:
            best, best_key = (ch, rec, br), key
    return best

def visit_stop(self, x, y, travel_dir=None, incoming_dir=None, next_dir=None):
    """移动到 (x,y) 并测量。

    ``travel_dir``: **本段**（上一站→本站）的行进方向；
    ``next_dir``: **下一段**（本站→下一站）的计划方向 —— 顺路判定要用它，
    因为到达本站后"朝本站的方向"已经是零向量（机器狗就站在站上）。

    **清除时机很关键**：第一版每站都调 `clear_phase()`，结果为了清一个源跑到
    靶区另一头，回来时"绕圈"已被打断、剩下的站点全被跳过（实测只清 6/10）。
    现在的 `detour` 模式按**方向**挑顺路目标（见 :meth:`_enroute_target`）：
    在前方就先清掉，背对行进方向的才留到一圈之后统一处理。
    """
    n = 0
    do_detour = (self.cfg.clear_mode == "detour" and not self.aborted)
    if self.cfg.clear_mode == "station" and not self.aborted:
        targets = self.clear_targets()
        if targets and self.can_afford(self.cfg.min_action_budget_s):
            self.phase = "clear"
            n += self.clear_phase()
            self.phase = "scan"
            self.note_pos() # 清除绕行点也算一次有效检测点
    # ---- 先测本点（保证"这一站真的被扫过"），再顺路清除 ----
    if not self.aborted:
        p = np.asarray(self.arena.pos, float)
        if not self.meas_pts or math.dist(self.meas_pts[-1], tuple(p)) > 1.0:
            self.meas_pts.append((float(p[0]), float(p[1])))
        chans = self.channels_here(pos=p, incoming_dir=incoming_dir)
        if not chans:
            chans = [int(c) for c in self.needed_channels()]
        if chans:
            travel_cost = math.hypot(x - self.arena.pos[0],

```

```

        y = self.arena.pos[1] / V_ROBOT
        avail = self.remaining() - self.cfg.time_reserve_s - travel_cost - 4.0
        kmax = max(1, int(avail // (T_MEASURE + T_SWITCH)))
        if len(chans) > kmax:
            self.log(f"  剩余时间只够测 {kmax}/{len(chans)} 个频道")
            chans = chans[:kmax]
        n += self.scan_at(float(x), float(y), chans)
        self.note_pos()

# ---- 边绕边清: 本点测完后, 把"前方该清的"清掉 (含只有一条方位的) ----
if self.cfg.debug_onlap:
    self.log(f"    [onlap] 本站 ({x:.0f},{y:.0f}) 当前 pos="
            f"({self.arena.pos[0]:.0f},{self.arena.pos[1]:.0f}) "
            f"onlap_clear={self.cfg.onlap_clear} aborted={self.aborted}")
if self.cfg.onlap_clear and not self.aborted:
    guard = 0
    while guard < 4 and not self.aborted:
        guard += 1
        d_now = None
        if next_dir is not None:
            d_now = np.asarray(next_dir, float)      # 用"计划前往的下一站"方向
        else:
            c0 = np.asarray(self.arena.pos, float)
            v = np.asarray((x, y), float) - c0
            dv = float(np.linalg.norm(v))
            if dv > 1e-6:
                d_now = v / dv
        if d_now is None:
            break
        cands = self._onlap_candidates(float(x), float(y), d_now)
        if self.cfg.debug_onlap:
            self.log(f"    [onlap] 本站 ({x:.0f},{y:.0f}) 行进方向 "
                    f"{math.degrees(math.atan2(d_now[1], d_now[0])):.0f}°, "
                    f"候选 {[k, ch, round(cs, 2), round(d)] for k, ch, _p, cs, d in cands[:3]}")
        if not cands:
            break
        kind, ch, pt, cs, d = cands[0]
        if not self.can_afford(self.cfg.min_action_budget_s + 20.0):
            break
        n += self._onlap_handle(kind, ch, pt)
        self.note_pos()

# ---- 兼容旧行为: 按方向/距离的"顺路清除" ----
if do_detour and not self.aborted:
    tgt = self._enroute_target(x, y, travel_dir) if travel_dir is not None else None
    if tgt is None:
        # 方向判据没命中时, 保留"很近就清"的兜底
        for t in self.clear_targets():
            if math.dist((x, y), t[2]["center"]) <= self.cfg.clear_detour_max_m:
                tgt = t
                break
    if tgt is not None and self.can_afford(self.cfg.min_action_budget_s):
        self.phase = "clear"
        self.log(f"    顺路清除频道{tgt[0]} (中心距本站 "
                f"{math.dist((x, y), tgt[2]['center']):.0f} m) ")
        n += self.clear_target(int(tgt[0]), tgt[1])
        self.phase = "scan"
        self.note_pos()

return n

# ----- 开场 / 批量: 按期望场行动 -----
def _singletons(self, max_r=None):
    """只有一条方位"且未清除的频道 (可选限制首条方位距离)。"""

```

```

out = []
for ch, rec in self.recs.items():
    if rec.cleared or rec.n_bearings != 1:
        continue
    if max_r is not None and math.hypot(rec.pts[0][0], rec.pts[0][1]) > max_r:
        continue
    out.append(int(ch))
return out

def _field_batch_locate(self, reason=""):
    """**按期望场行动**：把当前所有"只有一条方位"的频道**合并**规划第二测点。

    关键认识：**一个测点可以同时给多个源提供好的交会几何**。问题 2 的结论是
    "最优第二检测点在斜前方约 1.2 - 1.3 km、与源方向成  $\pm 30^\circ$  的两翼"，而这些"两翼"
    对不同方向的源常常落在同一片区域。所以：

    1. 对每个待定频道求**局部最优第二测点**（`best_refine_point`，代价里含
        "测完还要往回走多远"）；
    2. **聚类**：彼此最优测点相距  $\leq$  ``batch_max_m`` 的频道算一批；
    3. 每批用一个**共同测点**（各成员最优点的中点/本身，取联合代价最小者）；
    4. 去那儿按批测一遍，然后立刻清除已经定下来的目标。

    这样一次移动能定住好几个源，而不是"为一个源跑一趟、再为一个源跑一趟"。
    """
    if not self.cfg.batch_locate or self.base is None or self.aborted:
        return 0
    if self.stats.get("n_batch", 0) >= self.cfg.batch_max_rounds:
        return 0
    pend = (self._singletons(max_r=self.cfg.initial_locate_max_m)
            if reason == "开场" else self._singletons())
    if len(pend) < self.cfg.batch_min_channels:
        return 0

    per = {}
    for ch in pend:
        rec = self.recs[ch]
        p, info = best_refine_point(rec.pts, rec.svds, self.pos, None, n_t=60,
                                   radii=(150.0, 300.0, 500.0, 800.0),
                                   travel_cap_m=self.cfg.refine_travel_cap_m)

        if p is None:
            continue
        per[ch] = {"p": np.asarray(p, float), "info": info, "E": None}
    if len(per) < self.cfg.batch_min_channels:
        return 0

    # ② 聚类：**完全连接**（与组内每个成员都要够近），避免"链式"把不相干的点串成一批
    groups = []
    for ch, d in per.items():
        for g in groups:
            if all(float(np.linalg.norm(d["p"] - per[c]["p"])) <= self.cfg.batch_max_m
                   for c in g):
                g.append(ch)
                break
        else:
            groups.append([ch])

    best = None
    for g in groups:
        if len(g) < self.cfg.batch_min_channels:
            continue
        pts = [per[c]["p"] for c in g]
        cand = [np.mean(np.asarray(pts, float), axis=0)] + pts

```

```

        bt, bc = None, None
    for c0 in cand:
        tot = float(np.linalg.norm(np.asarray(c0) - np.asarray(self.pos))) / V_ROBOT
        tot += len(g) * (T_MEASURE + T_SWITCH)
        bad = False
        for ch in g:
            rec = self.recs[ch]
            S, th = rec.pts[-1], rec.svds[-1]
            ts, w, _hi = ray_source_samples(S, th, n_t=60)
            if ts.size == 0:
                bad = True
                break
            E = float(exact_expected_diameter(S, th, np.asarray(c0, float),
                                                ts, w)[0])

            if not math.isfinite(E):
                bad = True
                break
            tot += math.pi * max(E / 2.0 + 20.0, 20.0) ** 2 / (2.0 * R_CLEAR) / V_ROBOT
        if bad:
            continue
        if bt is None or tot < bt:
            bt, bc = tot, np.asarray(c0, float)
    if bc is None:
        continue
    gain = sum(per[c]["info"]["cost_s"] for c in g) - bt
    if best is None or gain > best[0]:
        best = (gain, bc, g, bt)
    if best is None:
        return 0
    gain, c, group, cost = best
    if gain <= self.cfg.batch_min_gain_s:
        return 0
    trav = float(np.linalg.norm(c - np.asarray(self.pos)))
    if not self.can_afford(trav / V_ROBOT + len(group) * (T_MEASURE + T_SWITCH) + 10.0):
        return 0
    self.stats["n_batch"] = self.stats.get("n_batch", 0) + 1
    self.log(f"定位批次{' (' + reason + ' )' if reason else ''}: {len(group)} 个频道只有一条方位 "
            f"{group} -> 合并测点 ({c[0]:.0f},{c[1]:.0f}), 移动 {trav:.0f} m, "
            f"预计比各测各的省 {gain:.0f} s")
    self.scan_points.append((float(c[0]), float(c[1])))
    n = 0
    for ch in group:
        if self.aborted or not self.can_afford(T_MEASURE + T_SWITCH + 3.0):
            break
        resp = self.measure(float(c[0]), float(c[1]), int(ch))
        n += 1
        if resp.get("measure_result") == "near":
            self.do_clear(float(c[0]), float(c[1]), int(ch))
    self.note_pos()
    if self.clear_targets() and self.can_afford(self.cfg.min_action_budget_s):
        self.phase = "clear"
        self.clear_phase()
        self.phase = "scan"
        self.note_pos()
    return n

def _initial_locate(self):
    """开场定点: 委托给 `meth:_field_batch_locate` (它自带合并规划)。"""
    if not self.cfg.initial_locate or self.base is None or self.aborted:
        return 0
    return self._field_batch_locate(reason="开场")

```

```

# ----- 主循环 -----
def run(self):
    t0 = time.time()
    resp = self.arena.enter()
    if resp.get("accepted") is not True:
        raise ArenaError(f"/enter 未被接受: {resp}")
    self.log(f"进入靶区: 可用预算 {self.remaining():.0f} s"
            f" ({self.arena.budget_kind()}) : 策略=覆盖扫圈")

    try:
        # 第 0 轮: 原点 (已经覆盖了 r<=1000 m 的整个内圈)
        chans0 = self.channels_to_measure()
        self.log(f"第 0 轮扫描 (原地): 待测 {len(chans0)} 个频道 {chans0}")
        self.scan_points.append(self.pos)
        self.scan_at(self.pos[0], self.pos[1], chans0)
        self.note_pos()
        cov, mdist = self.coverage_now()
        self.log(f" 初始覆盖率 {cov:.3f} (原点一次检测即覆盖 r<=1000 m 的内圈)")

        # ---- 开场定点 (方案 1): 先用决策场把近处那一簇源定位并清除 ----
        self._initial_locate()
        cov, mdist = self.coverage_now()
        self.log(f" 开场定点结束: 覆盖率 {cov:.3f}, "
                f"已清除 {sum(1 for r in self.recs.values() if r.cleared)} 个源")

        rounds = 1
        level = 0
        laps_at_level = 0
        done = False
        first, _first_idx = self.pick_start_stop() # 朝"信息最多的方向"出发
        while (not self.aborted and rounds < self.cfg.max_rounds
              and self.remaining() > self.cfg.time_reserve_s):
            is_first = first is None
            target_hit = None
            if is_first:
                c = first
                first = None
                info = {"dist": float(np.hypot(float(c[0]) - self.arena.pos[0],
                                                float(c[1]) - self.arena.pos[1])),
                       "adv_deg": 0.0, "gain": 1.0, "n_new": 0, "cnt": 0}
            else:
                cov, mdist = self.coverage_now()
                if mdist <= self.cfg.cover_radius_m:
                    self.log(f"* 覆盖率 100% (最大未覆盖点距最近检测点 {mdist:.0f} m "
                            f"<= {self.cfg.cover_radius_m:.0f} m) "
                            f"-> 靶区内不可能存在未清除源, 提前收工")
                    break
                c, info = self.pick_stop(level=level)
            if c is None:
                # 这一圈走完了 (该圈所有站位要么已到过、要么没有新覆盖)
                if level + 1 < len(self.cfg.sweep_rings):
                    level += 1
                    laps_at_level = 0
                    self.ring_last_a = None # 换圈后重新确定绕行方向
                    self.ring_dir = -self.ring_dir if self.ring_dir else 1.0
                    self.log(f"主圈走完仍有缺口 (覆盖率 {cov:.3f}) "
                            f"-> 启用补漏圈 level={level}"
                            f" (r={self.cfg.sweep_rings[level][0]:.0f} m) ")
                    continue
                # 所有圈都走过了: 换下一圈 (角向错开 lap_offsets), 再走一遍
                if (self.lap + 1) < max(2, len(self.cfg.lap_offsets)):
                    self.lap += 1
                    level = 0

```

```

        laps_at_level = 0
        self.ring_last_a = None
        self.log(f"所有环的站位都已到过，但仍有频道从未测到信号 "
                f"-> 第 {self.lap + 1} 圈（角向错开 "
                f"{self.cfg.lap_offsets[min(self.lap, len(self.cfg.lap_offsets) - 1)]} 格）")
        continue
    targets = self.clear_targets()
    if targets and self.can_afford(self.cfg.min_action_budget_s):
        self.log(f"扫圈完成但仍剩 {len(targets)} 个已定位未清除的源 -> 清除")
        self.clear_phase()
        continue
    self.log(f"没有可选的停靠点、也没有待清除的源 -> 结束"
            f"（覆盖率 {cov:.3f}，最大未覆盖 {mdist:.0f} m）")
    break
cost = (info["dist"] / V_ROBOT + T_MEASURE + T_SWITCH
        + self.cfg.min_action_budget_s)
if not self.can_afford(cost):
    self.log(f"剩余时间不足以再走一站（还需约 {cost:.0f} s）-> 收工")
    break
cov, mdist = self.coverage_now()
self.log(f"扫圈第 {rounds} 站(lv{level}): 选点 ({c[0]:.0f},{c[1]:.0f}) | "
        f"移动 {info['dist']:.0f} m | 前进 {info['adv_deg']:.0f}° | "
        f"本点新覆盖 {info['n_new']}/{info['cnt']} 格（收益 {info['gain']:.2f}） | "
        f"当前覆盖率 {cov:.3f}（最大未覆盖 {mdist:.0f} m）")
# ---- 用"目标位置"顶替环上站位（省一个路径点）----
if self.cfg.onlap_clear and info.get("gain", 1.0) < 0.5:
    hit = self._target_near_station(c)
    if hit is not None and self.station_skippable(c, [hit[1]]):
        self.scan_points.append((float(c[0]), float(c[1])))
        self.log(f" * 站位 ({c[0]:.0f},{c[1]:.0f}) 与频道{hit[0]} 的目标位置只差 "
                f"{hit[2]:.0f} m，且顶替后覆盖仍够（最大未覆盖 < "
                f"{self.cfg.skip_station_max_uncovered:.0f} m）-> 省掉这一站， "
                f"直接去清频道{hit[0]}")
        self._onlap_handle("locate" if self.recs[int(hit[0])].n_bearings < 2
                           else "clear", int(hit[0]), hit[1])
        self.note_pos()
        rounds += 1
        continue
p0 = np.asarray(self.arena.pos, float)
v = np.asarray(c, float) - p0
dv = float(np.linalg.norm(v))
travel_dir = (v / dv) if dv > 1e-9 else None # 本段计划行进方向
incoming_dir = self._last_dir # 上一段的行进方向
# 下一段的方向：先"试推"一次 pick_stop（只读），用它来判定"前方该清的点"
next_dir = None
try:
    _saved_last, _saved_dir = self.ring_last_a, self.ring_dir
    _saved_pts = list(self.scan_points)
    if not is_first:
        self.scan_points.append((float(c[0]), float(c[1])))
    _c2, _ = self.pick_stop(level=0)
    if _c2 is not None:
        v2 = np.asarray(_c2, float) - np.asarray(c, float)
        d2 = float(np.linalg.norm(v2))
        if d2 > 1e-9:
            next_dir = v2 / d2
except Exception:
    next_dir = None
finally:
    self.ring_last_a, self.ring_dir = _saved_last, _saved_dir
    self.scan_points = _saved_pts
if not is_first:

```

```

        self.commit_stop(c)
        self.scan_points.append((float(c[0]), float(c[1])))
    self.visit_stop(float(c[0]), float(c[1]), travel_dir=travel_dir,
                    incoming_dir=incoming_dir, next_dir=next_dir)
    self._last_dir = travel_dir
    # 起始站在访问之后才登记: `pick_stop` 用 `scan_points` 做"已到过"的排除,
    # 若提前登记, 第一站的邻域会立刻生效, 下一站就选不出来了 (零收益)
    if is_first:
        self.commit_stop(c)
        self.scan_points.append((float(c[0]), float(c[1])))
    # ---- 按期望场行动: 每走几站就把"只有一条方位"的频道合并定一次 ----
    if (self.cfg.batch_locate and not self.aborted
        and rounds % max(1, self.cfg.batch_every) == 0):
        self._field_batch_locate(reason=f"第 {rounds} 站后")
    rounds += 1
    laps_at_level += 1
    if laps_at_level > 4 * max(self.ring_n, 8) + 8:
        break

# ---- 走完圈之后: 把所有已定位的源统一清掉 (边绕边清模式下通常已清完) ----
if (not self.cfg.onlap_clear or self.cfg.onlap_after_ring) \
    and not self.aborted and self.clear_targets() \
    and self.can_afford(self.cfg.min_action_budget_s):
    self.log(f"扫圈结束 (覆盖率 {self.coverage:.3f}): 统一清除 "
            f"{len(self.clear_targets())} 个已定位的源")
    self.phase = "clear"
    self.clear_phase()
    self.phase = "scan"
    self.note_pos()

# ---- 完工判据: 整圈已走完 + 每个频道都有明确结论 -> 直接收工 ----
if not self.aborted and self.all_heard():
    self.log(f"整圈已走完, 且每个频道都有明确结论"
            f" (已清除 {sum(1 for r in self.recs.values() if r.cleared)} 个 / "
            f"其余从未测到信号) -> 靶区内无残留源")
    self.stats["n_scan_rounds"] = rounds
    done = True

# ---- 额外圈: 整圈走完但还有频道从未测到 (角向缝隙) 时再走 -----
while (not done and not self.aborted and rounds < self.cfg.max_rounds
        and self.remaining() > self.cfg.time_reserve_s):
    cov, mdist = self.coverage_now()
    if self.lap + 1 >= max(2, len(self.cfg.lap_offsets)):
        break
    c, info = self.pick_stop(level=0)
    if c is None:
        self.lap += 1
        self.level = 0
        self.ring_last_a = None
        self.log(f"第 {self.lap} 圈走完仍有频道从未测到 -> "
                f"第 {self.lap + 1} 圈 (角向错开 "
                f"{self.cfg.lap_offsets[min(self.lap, len(self.cfg.lap_offsets) - 1)]} 格)")
        continue
    cost = (info["dist"] / V_ROBOT + T_MEASURE + T_SWITCH
            + self.cfg.min_action_budget_s)
    if not self.can_afford(cost):
        break
    self.log(f"补测圈第 {rounds} 站: ({c[0]:.0f},{c[1]:.0f}) | "
            f"覆盖率 {cov:.3f} (最大未覆盖 {mdist:.0f} m)")
    self.commit_stop(c)
    self.visit_stop(float(c[0]), float(c[1]))
    rounds += 1

```



```

        if self.all_heard() and not self.clear_targets():
            self.log("补测后每个频道都有明确结论、且没有待清除目标 -> 结束")
            break
self.stats["n_scan_rounds"] = rounds

# ---- 收尾: 把"听到了但还没清掉"的少量频道用自适应方式解决 ----
guards = 0
while (not self.aborted and rounds < self.cfg.max_rounds
        and self.remaining() > self.cfg.time_reserve_s and guards < 8):
    guards += 1
    leftover = [ch for ch, rec in self.recs.items()
                if not rec.cleared and rec.n_bearings >= 1]
    if not leftover:
        break
    # 只关心"有界区域但还没清"的频道; 其余先补一条方位
    todo = [t for t in self.clear_targets()]
    if todo:
        self.log(f"收尾: 清除剩余 {len(todo)} 个已定位目标 "
                f"{[t[0] for t in todo]}")
        self.phase = "clear"
        n = self.clear_phase()
        self.phase = "scan"
        self.note_pos()
        if n == 0:
            break
        continue
    plan = self.plan_scan()
    if plan is None or not self.can_afford(plan["cost_s"]):
        self.log(f"收尾: 剩余频道 {leftover} 无法在预算内继续定位 -> 结束")
        break
    self.log(f"收尾: 为剩余频道 {leftover} 补测方位 -> "
            f"选点 {tuple(round(v) for v in plan['sel']['xy'])}")
    self.execute_scan(plan)
    rounds += 1
self.stats["n_scan_rounds"] = rounds
finally:
    try:
        self.arena.exit()
    except ArenaError as e:
        self.log(f"/exit 异常: {e}")
self.stats["program_runtime_s"] = time.time() - t0
rep = self.report()
rep["coverage"] = self.coverage
rep["max_uncovered_dist_m"] = self.max_uncov_dist
return rep

```

```
class HybridRobot(SweepRobot):
```

"""混合策略: **先用扫圈把该听的都听到, 再交给"期望场贪心"去清除**。

动机 (离线实测):

- * 纯扫圈的**发现**效率明显更好: 同样两局在线真源, 虚拟时间 3.7-4.0k vs 4.0-4.4k, 检测次数 117-145 vs 143-198, 行程少 3-5 km, 而且不会乱蹭、有完备性判据;
- * 但纯扫圈的**清除**偏弱 (外圈环带场景 0.972), 因为它把清除也绑在"绕圈"结构上。
- * 所以让两者各干自己最擅长的: 扫圈负责"把 20 个频道都听到一遍" (发现), 之后切换到自适应循环负责"把听到的源清干净" (决策场选点 + 覆盖式试探 + 顺路补测)。

"""

```

def run(self):
    t0 = time.time()
    resp = self.arena.enter()
    if resp.get("accepted") is not True:

```

```

        raise ArenaError(f"/enter 未被接受: {resp}")
self.log(f"进入靶区: 可用预算 {self.remaining():.0f} s"
        f" ({self.arena.budget_kind()}); 策略=扫圈发现 + 自适应清除")
try:
    chans0 = self.channels_to_measure()
    self.log(f"第 0 轮扫描 (原地): 待测 {len(chans0)} 个频道 {chans0}")
    self.scan_points.append(self.pos)
    self.scan_at(self.pos[0], self.pos[1], chans0)
    self.note_pos()

# ---- 阶段 1: 绕一圈, 把该听的频道都听一遍 (不清除) ----
self.phase = "scan"
rounds = 1
while (not self.aborted and rounds < self.cfg.max_rounds
        and self.remaining() > self.cfg.time_reserve_s):
    cov, mdist = self.coverage_now()
    c, info = self.pick_stop(level=0)
    if c is None:
        break
    cost = (info["dist"] / V_ROBOT + T_MEASURE + T_SWITCH
            + self.cfg.min_action_budget_s)
    if not self.can_afford(cost):
        break
    self.log(f"扫圈第 {rounds} 站: ({c[0]:.0f},{c[1]:.0f}) | "
            f"移动 {info['dist']:.0f} m | 收益 {info['gain']:.2f} | "
            f"覆盖率 {cov:.3f} (最大未覆盖 {mdist:.0f} m)")
    self.commit_stop(c)
    self.scan_points.append((float(c[0]), float(c[1])))
    self.visit_stop(float(c[0]), float(c[1]))
    rounds += 1
    if self.all_heard():
        self.log("整圈走完, 每个频道都有明确结论 -> 结束扫描阶段, "
                "转交自适应清除")
        break
self.stats["n_scan_rounds"] = rounds

# ---- 阶段 2: 切回自适应主循环 (决策场选点 + 清理 + 顺路补测) ----
self.phase = "adaptive"
stall = 0
while (not self.aborted and rounds < self.cfg.max_rounds
        and self.remaining() > self.cfg.time_reserve_s):
    did = 0
    if self.clear_targets() and (rounds >= self.cfg.min_scan_rounds_before_clear
                                or not self.plan_scan()):
        did += self.clear_phase()
    if self.aborted or self.remaining() <= self.cfg.time_reserve_s:
        break
    plan = self.plan_scan()
    if plan is not None and self.can_afford(plan["cost_s"]) \
        and (did == 0 or not self.clear_targets()):
        did += self.execute_scan(plan)
        rounds += 1
    if did == 0:
        stall += 1
        if stall >= 2:
            self.log("自适应阶段: 没有可清理的频道、也没有值得检测的频道 "
                    "-> 提前结束")
            break
    else:
        stall = 0
self.stats["n_scan_rounds"] = rounds
finally:

```

```

        try:
            self.arena.exit()
        except ArenaError as e:
            self.log(f"/exit 异常: {e}")
        self.stats["program_runtime_s"] = time.time() - t0
        rep = self.report()
        rep["coverage"] = self.coverage
        rep["max_uncovered_dist_m"] = self.max_uncov_dist
        return rep

def selfcheck(verbose=True):
    out, ok_all = [], True

    def rec(name, ok, detail=""):
        nonlocal ok_all
        ok_all = ok_all and bool(ok)
        out.append({"check": name, "ok": bool(ok), "detail": detail})
        if verbose:
            print(f"[{'OK ' if ok else 'FAIL'}] {name} {detail}")

    cand = lattice_candidates()
    mdist, unc, cov = coverage_report(cand, r_inner=R_RECV_MIN)
    rec("C1 一圈格点即可覆盖环带 r∈[1000,1800] (最大未覆盖距离 <= 1000 m)",
        mdist <= R_RECV_MIN,
        f"{len(cand)} 个候选点; 环带内最大未覆盖距离 {mdist:.0f} m; "
        f"未覆盖面积 {unc*100:.2f}%")

    mdist1, unc1, cov1 = coverage_report([(0.0, 0.0)] + cand, r_inner=0.0)
    rec("C2 原点 + 一圈覆盖整个靶区 (这才是实际航线的完备性口径)",
        mdist1 <= R_RECV_MIN,
        f"最大未覆盖 {mdist1:.0f} m; 覆盖率 {cov1*100:.2f}%")

    from p3_arena import MockArena
    a = MockArena(seed=20260913)
    rb = SweepRobot(a, SweepConfig(), base=None, log=None)
    sel, info = rb.pick_stop()
    rec("C3 第一站只从还没覆盖的格点里选, 且不选已停靠点邻域",
        sel is not None and info["gain"] >= rb.cfg.min_gain,
        f"第一个停靠点 ({sel[0]:.0f},{sel[1]:.0f}), 距原点 {info['dist']:.0f} m, "
        f"边际覆盖 {info['gain']:.2f} (新覆盖 {info['n_new']}/{info['cnt']} 格)")

    return {"ok": ok_all, "checks": out}

if __name__ == "__main__":
    import sys
    if "--selfcheck" in sys.argv:
        res = selfcheck()
        raise SystemExit(0 if res["ok"] else 1)
    if "--coverage" in sys.argv:
        for rings in (DEFAULT_RINGS,):
            cand = lattice_candidates(rings)
            mdist, unc, cov = coverage_report(cand)
            print(f"候选点 {len(cand)} 个: 最大未覆盖 {mdist:.0f} m, 未覆盖 {unc*100:.2f}%")
        print("\n只用原点一个点: ", coverage_report([(0.0, 0.0)])[:2])
        print("原点 + 一圈 r=1500 (8 点): ",
            coverage_report([(0.0, 0.0)] + [(1500 * math.cos(2 * math.pi * i / 8),
                1500 * math.sin(2 * math.pi * i / 8))
                for i in range(8)])[:2])
        print("原点 + 一圈 r=1800 (12 点): ",
            coverage_report([(0.0, 0.0)] + [(1800 * math.cos(2 * math.pi * i / 12),

```

```
1800 * math.sin(2 * math.pi * i / 12))
for i in range(12))[:2])
```

5.7 src/p3_tour.py

"""B 题问题 3: **统一滚动航路**策略 (Tour) —— "绕圈"与"清除"合成一条动态航路。

为什么需要它 (旧两套策略的实测缺陷, 详见 `review/p3-tour-design.md`)

```
-----
* `P3Robot` (期望场贪心): 虚拟时间 5270 s, 其中 **29% 是清完源之后的空转**,
  检测 186 次、行程 20.5 km。
* `SweepRobot` (覆盖扫圈): 虚拟时间降到 4243 s, 但结构是**"先绕完一圈、再统一清除"**:
  绕圈的 6.4 km 和清除的 5.2 km 是两段互不重叠的路, 实测行程 16.8 km,
  是"已知全部源位后的最优航路" (8.8 km) 的 **1.92 倍**, 其中折返 (转角 >120°)
  1.5 km/局。按题目的"平均定位清除时间 = 定位清除总时间 ÷ 清除个数"口径,
  这一半行程全是白走的。
```

本模块把扫描与清除合成**一条滚动时域的统一航路**:

```
---
每个决策点重算一次航路, 只执行第一个停点, 然后重规划
航路 = ① 环带骨架停点 (保证"没有残留源"的可证明完备性, 按角向单调推进)
      ↑ 目标 (清除 / 定位) 用"最便宜插入"插进骨架的空隙里
目标 = 已定位频道的最新定位区域中心 (clear)
      只有一条方位 / 区域无界 / 区域过大的频道 (locate, 用第二问的 E[D] 代价模型)
---
```

于是航线天然就是用户描述的**锯齿状一圈**: 从原点出发 → (期望场) 选第二个检测点、
顺手清掉靠内圈的目标 → 就近切入环带 → 沿一个方向绕一圈、边走边清 →
走完一圈即收工。不会出现"绕完再回头"的长距离折返。

相对旧实现的六个关键修正

- ```

```
1. \*\*不再分"绕圈/清除"两个阶段\*\*: 骨架停点与目标一起排序, 清完一个就地重规划。
  2. \*\*环带骨架按角向单调推进\*\* (锁定方向 + 角向游标): 这是"不折返"的结构性保证;  
旧实现每一步都重新贪心选点, 导致航路在角向上来回横跳 (实测折返 2.5 km/局)。
  3. \*\*覆盖判据按频道算\*\*: 某点是否值得测, 取决于"这个频道在那里是否还可能  
没被听到的源", 用该频道\*\*真实测过的位置\*\* (`rec.meas\_pts`) 判定 ——  
清除绕行时顺带做的检测会自动抵扣掉后面的骨架停点, 不重复走。
  4. \*\*修掉 `visit\_stop` 的 gap 判定 bug\*\*: 旧实现用"上一个动作的位置"而不是  
"本停点坐标"判断"这个频道在这儿测过没有", 导致整站漏测 (实测有源因为  
连续漏测而\*\*永远没被听到\*\*)。
  5. \*\*"只有一条方位"的频道不再无脑重测\*\*: 只有当"本停点相对上一条视线的张角"  
落在 25° - 155° 时才测 (否则交会角接近 0, 纯属浪费 6 s)。
  6. \*\*清除子过程可迭代\*\*: 区域无界 / 区域半径过大不再被永久跳过 (旧实现在  
`clear\_targets()` 里把  $r > 600$  m 的目标过滤掉, 是清除率只有 0.978 的主因),  
而是"逼近中心补方位 → 区域压小 → 覆盖式试探"循环, 直到成功或用尽预算。

```
"""
from __future__ import annotations

import math
import time
from dataclasses import dataclass

import numpy as np

from p3_arena import (# noqa: E402
 ArenaError, R_ARENA, R_CLEAR, R_RECV_MIN, T_CLEAR_FAIL, T_MEASURE, T_SWITCH,
 V_ROBOT,
)
```

```

from p3_expect_field import best_refine_point, ray_source_samples # noqa: E402
from p3_sweep import SweepConfig, SweepRobot # noqa: E402

配置

@dataclass
class TourConfig(SweepConfig):
 """在 `SweepConfig` (含 `P3Config` 全部字段) 之上追加"统一航路"参数。"""

 # ---- 环带骨架 ----
 cover_rings: tuple = ((1000.0, 8),)
 """骨架候选停点的同心环 `` (半径 m, 点数)``。

 为什么是"小半径 + 稀疏停点": 原点那一次全频道检测已经把 $r \leq 1000$ 的内圈覆盖了,
 只剩环带 $r \in [1000, 1800]$ 要覆盖; 半径 ρ 的停点最多覆盖到
 $1800^2 + \rho^2 - 2 \cdot 1800 \cdot \rho \cdot \cos \Delta \leq 1000^2$ 的角向半宽 Δ (Δ 越大, 环上可以越稀),
 于是

 | 配置 | 环带最大未覆盖 | 覆盖余量 | 单圈弦长 | 含进入段 | 每圈要测的频道数 |
 |---|---|---|---|---|---|
 | 7 点 r=1050 | 968 m | 32 m | 6378 m | 7428 m | 7×N |
 | **8 点 r=1000 (默认) ** | **943 m** | **57 m** | **6123 m** | **7123 m** | 8×N |
 | 9 点 r=950 | 964 m | 36 m | 5849 m | 6799 m | 9×N |
 | 13 点 r=880 | 969 m | 31 m | 5476 m | 6356 m | 13×N |

 半径越小弦长越短, 但停点越多、每站都要把"从没听到过"的频道整批测一遍
 (6 s/频道)。四批共 150 局实测 (python src/p3_tour_sweep.py --rings ...):

 | 配置 | 全清局数 | T/N 均值 |
 |---|---|---|
 | 7 点 r=1050 | 146/150 | 333.2 s |
 | **8 点 r=1000 (默认) ** | **149/150** | **332.5 s** |
 | 9 点 r=950 | 149/150 | 333.3 s |

 r=1000/8 点同时拿到最好的清除率、最低的 T/N, 以及最大的覆盖余量 (57 m), 故为默认。
 """

 cover_grid_step_m: float = 25.0 # 覆盖度评估网格步长
 cover_slack_m: float = 0.0 # 覆盖判据余量 (>0 更保守)
 ring_min_advance_rad: float = 0.06 # 角向单调推进的最小步长 (约 50 m)
 entry_tie_m: float = 0.0
 """入口选择的"并列"阈值: 航路长度相差不超过它就算并列, 此时才按信息方向挑。

 取 0 的含义是"优先就近进入"; 只有长度**完全相等** (机器狗正停在原点、
 8 个入口一样远) 时, 才用信息方向打破僵局" —— 这正好对应用户指出现象:
 进场位置之所以"固定", 就是因为原点扫描之后机器狗还在原点,
 $\text{atan2}(0,0)$ 恒为 0。阈值放大到 250/400 m 会把"信息方向"的权重放大成
 主导项, 实测反而变差 (常规随机 327.8 → 327.9 → 332.2)。"""

 entry_select: bool = True # 消融对照: False = 旧行为 (用当前位置极角当游标)
 cover_duty_frac: float = 0.95 # 目标停点半径 >= 骨架半径×它时, 代劳覆盖检测

 # ---- 路点合并与排序 ----
 merge_radius_m: float = 250.0 # 目标这么近的骨架停点直接省掉
 route_mode: str = "polish" # insert (只做最便宜插入) | polish (+2-opt, 骨架不倒序)
 # | tsp (完全自由 NN+2-opt, 消融对照)
 locate_max_detour_m: float = 260.0 # 定位路点的绕行超过它就本轮不排 (等航路自己给方位)
 locate_force_after: int = 2 # 连续跳过这么多轮之后强制排入 (避免永远排不进)
 clear_max_detour_m: float = 1e18 # **清除路点一律排进航路** (这个上限只是记录用)
 """清除目标绝不允许"因为绕行太远就不排"。

 实测 (`p3_paths` 场景 C): ch6 在环带 (0,-1000) 处就拿到了第二条方位、

```

```

$r^*=42$ m, 完全可以就地清除; 但当时"最便宜插入"的额外代价是 2382 m,
超过了旧的 1500 m 硬上限, 于是它被踢出航路, 一直留到收尾阶段的
`clear_leftovers` —— 那时机器人已经跑到场地另一头(1051,1295),
再横穿 3.4 km 回去清它。**"绕行大"只应该影响排序, 绝不能变成"不清"**。
"""

---- 期望场(第二问 E[D] 模型)作为可选项 ----
field_min_pending: int = 1
"""只有"待定(只有一条方位的)频道数 >= 它"时才动用期望场/批量合并定位。

期望场的好处是"一次移动定住好几个源", 也**顺带把"走多远拉基线"与
"定位精度差要多扫多少面积"放在同一个代价里权衡**(`best_refine_point`
的 cost 里含 $\pi(E[D]/2+20)^2/(2R_{\{clear\}}/v)$)。所以它不是"为了精度可以
无限跑远"—— 只是因为"基线短 $\Rightarrow r^*$ 大 $\Rightarrow$ 后面搜索面积按平方增长",
权衡下来它的最优点常常在几百米到 1 km 之间。

待定频道很少时改用 `_cheap_locate_point` (候选半径压到
`_short_locate_cap_m`) 这一档**实测是亏的** (四类情景 T/N 全部更差:
常规随机 330.0→344.4、外圈环带 359.6→371.7、在线真源 394.6→426.3),
所以默认取 1 (= 始终允许期望场)。真正解决"为单个目标跑很远"的是
`locate_patience_stops` (环形航路本来会给的方位就先等着) 与
`locate_max_detour_m` (绕行太大的定位点本轮不排)。
"""

---- 主动"防过期"补测(默认关闭; 实测在 4 类情景上都略亏, 见 $"清除时机") ----
risk_locate: bool = False
risk_lookahead: int = 3 # 往后看几个骨架停点
risk_min_lost: float = 0.5 # 先验里有这么大比例的源点"往后几个站都听不到"就该补测
risk_step_m: float = 300.0 # 补测只走这么远(小步)
risk_max_per_stop: int = 1
sector_clear_m: float = 0.0
"""刚走过的扇区里已经能清的目标"的就地清除半径(0 = 关闭)。

这两个机制(`risk_locate` / `sector_clear`)是照"每个角度只经过一次、
所以覆盖到的目标要尽快清"的思路写的, 但四类情景实测都是**略亏**
(常规随机 T/N 334.6 → 327.5、外圈环带 4575 s → 4085 s、行程 16.1 km → 13.4 km
都是关掉更好)。真正解决"该清没清"的是 `offroute_clear` (见下)。"""

---- "不在行进方向上就地补测并清除" ----
offroute_clear: bool = True
wait_angle_min_deg: float = 30.0
"""夹角粗筛: 第一条方位与"本站→下一站"方向的夹角小于它就直接等下一站。

用户的直觉("夹角小就等、夹角大就立刻走一小步定位清除")方向完全正确,
但**只按夹角判太粗**: 夹角大其实是常态, 而"拐回来清"到底多花多少,
取决于源离哪一站更近。所以夹角在这里只当粗筛, 真正的判据是
`offroute_clear` 里用射线先验直接算的"两种做法的额外绕行"
(实测: 只用夹角判 → 常规随机 330 → 350、外圈环带 4085 → 5238 s;
换成绕行比较之后才真正有收益)。
"""
offroute_step_m: float = 320.0 # 就地补测只走这么远
offroute_margin_m: float = 60.0 # "现在清"要比"等下一站"至少省这么多米才做
offroute_max_thi_m: float = 900.0
"""射线先验外径超过它就放弃判断。

判断依赖"源大致在哪", 而 `_t_hi` (射线在靶区内的长度) 越长, 先验越摊得开、
估计越不可靠 —— 实测不加这条闸门时, 常规随机机会从 330 掉到 350 s
(对着一堆"其实在 1.4 km 外"的源白跑 320 m 的补测步)。
`t_hi` 小(源被靶区边界卡住、不可能远)时判断才可信。
"""
offroute_max_per_stop: int = 1
offroute_max_dist_m: float = 450.0

```

"""源位先验的加权平均距离超过它就放弃"现在清"。

单条方位对源距几乎没有分辨力（只有射线先验），源越远估计越不可靠；  
而且"现在清"的价值来自"顺手"，源太远时顺手也顺不了。

"""

# ---- 清除 ----

```
probe_enter_r_m: float = 110.0 # r* 小到它就进入覆盖式试探
probe_max_radius_m: float = 300.0 # 试探圆域半径上限（区域更大就先补方位）
probe_max_points: int = 150
clear_max_iters: int = 6
approach_eps_m: float = 25.0 # 与既有测量点的最小间隔
fail_cooldown: int = 2 # 清除失败后冷却几个停点
big_r_m: float = 1200.0 # r* 超过它就不"去区域中心"，改用横向定位点
 # （两条近似平行的方位会把最小包围圆撑到几公里，
 # 中心甚至落在靶区外 —— 旧实现会真的走过去，
 # 实测有一局为此多走 13.5 km、多花 2700 s）
```

# ---- 定位路点（"只有一条方位"的频道）----

```
locate_mode: str = "stale" # always | stale | never
locate_stale_stops: int = 3
locate_patience_stops: int = 6
"""只有一条方位的频道，最多"等"环形航路几个停点。
```

环形航路每个角度只经过一次，所以一条只有单方位的频道\*\*通常会在它所在扇区的  
下一个骨架停点上拿到第二条方位\*\* —— 白等就行，专门跑一趟是纯浪费

（实测 `p3\_paths` 场景 D 里那些"偏离预定路线的尖锐拐角"就是这么来的：  
为了给 ch9 补方位，从 (-1716,58) 往靶区里折了 1020 m，而环形航路本来  
就会在 (1000,0) 给它第二条方位）。

因此：先按 `\_lost\_fraction` 判断"环形航路还会不会再听到它"，  
只有确实听不到了、或者已经等了 `locate\_patience\_stops` 站，才专门去定位。

"""

```
locate_travel_cap_m: float = 1100.0
locate_bearing_cap_m: float = 700.0 # "回到上一条方位的检测点附近横向补测"的半径上限
locate_radai: tuple = (150.0, 300.0, 500.0, 750.0, 1000.0)
short_locate_cap_m: float = 420.0
"""`_cheap_locate_point` 的候选半径上限（"单点定位只走一小步"）。"""
cheap_locate_angles: tuple = (55.0, 80.0, 105.0) # 小步横向补测的候选夹角（度）
min_cross_deg: float = 25.0 # 单方位频道：张角小于它就不值得重测
```

# ---- 开场定点（期望场第二检测点）----

```
initial_rounds: int = 3 # 原点附近最多做几轮"合并定位 + 顺手清除"
initial_move_growth: float = 2.0 # 每轮把移动上限放宽这么多倍
initial_locate_max_move_m: float = 350.0 # 单轮定位最多走多远
"""开场定点单轮的最远移动距离。
```

第二问的 E[D] 代价模型用的是"源位沿射线、按面积均匀"的先验，其\*\*中位源距约 1 km\*\*，  
于是对"其实就在原点附近"的那些源，它会给出一个 1 km 开外的第二检测点。  
实测（30 局 × 3 种情景）把它从 1300 m 收到 350 m：

```
情景	1300 m	**350 m**
常规随机（新种子批）	T/N 324.4	**312.5**
中心密集 12 源	T/N 282.1（还漏 1 局）	**224.9（30/30 全清）**
外圈环带 12 源	T/N 364.2	365.5（打平）
```

原因：近处的源只要在 300 - 400 m 外测一条方位就能定得很准；跑 1 km 去测既费时，  
又把它后面的清除路线拉乱（"先清较靠近原点的目标"靠的是\*\*顺路\*\*，不是专门跑一趟）。

"""

```
initial_clear_radius_m: float = 900.0 # 开场顺手清除的半径（"先清近处的源"）
```

# ---- 结束 ----

```

max_stops: int = 80
min_stop_budget_s: float = 15.0

机器狗

class TourRobot(SweepRobot):
 """统一滚动航路（环带骨架 + 目标最便宜插入 + 逐站重规划）。"""

 def __init__(self, arena, cfg: TourConfig | None = None, base=None, log=None):
 super().__init__(arena, cfg or TourConfig(), base=base, log=log)
 self._ag = None # 环带评估网格缓存
 self._unsafe_cache = {} # (ch, len(meas_pts)) -> 未排除掩码
 self._ring = None # 骨架（(角度, 半径) 按角度排序）
 self.ring_dir = 0.0 # 锁定的绕行方向 ±1
 self.ring_cursor_a = None # 角向游标（上一个骨架停点的极角）
 self.ring_served = set() # 已经被"顶替并完成覆盖职责"的骨架停点（角度键）
 self.stop_seq = 0
 self.last_stop_xy = None
 self.n_cover_stops = 0
 self.n_locate_stops = 0
 self.n_clear_iter = 0
 self._hold_until = {}
 self._locate_skips = {}
 self._last_bearing_stop = {}
 self._prev_stop_a = None
 self.tour_log = []

 # -----
 # 台账
 # -----
 def pending(self):
 return [int(ch) for ch, rec in self.recs.items() if not rec.cleared]

 def unheard(self):
 return [int(ch) for ch, rec in self.recs.items()
 if not rec.cleared and rec.n_bearings == 0]

 def heard_pending(self):
 return [int(ch) for ch, rec in self.recs.items()
 if not rec.cleared and rec.n_bearings > 0]

 def _n_cleared(self):
 return sum(1 for r in self.recs.values() if r.cleared)

 def on_hold(self, ch):
 return self.stop_seq < self._hold_until.get(int(ch), -1)

 def hold(self, ch, stops=None):
 self._hold_until[int(ch)] = self.stop_seq + int(
 self.cfg.fail_cooldown if stops is None else stops)

 def add_bearing(self, ch, x, y, svd):
 reg = super().add_bearing(ch, x, y, svd)
 self._last_bearing_stop[int(ch)] = self.stop_seq
 self._hold_until.pop(int(ch), None)
 return reg

 def region_info(self, rec):
 """当前（或最近一次）有界区域 ``{'center', 'radius', 'poly', 'status'}``。"""
 if rec.n_bearings >= 2:

```



```

 reg = self.region(rec)
 if reg.get("status") == "bounded":
 return {"center": tuple(reg["min_enclosing_center"]),
 "radius": float(reg["min_enclosing_radius_m"]),
 "poly": [tuple(v) for v in reg.get("vertices", [])],
 "status": "bounded", "n_bearings": rec.n_bearings}
 if rec.last_bounded is not None:
 c, r, n = rec.last_bounded
 return {"center": tuple(c), "radius": float(r), "poly": None,
 "status": "fallback", "n_bearings": n}
 return None

覆盖: 哪些环带格点"还可能藏着从没听到过的源"

def annulus_grid(self):
 if self._ag is None:
 step = float(self.cfg.cover_grid_step_m)
 R = R_ARENA
 xs = np.arange(-R, R + 1e-9, step)
 GX, GY = np.meshgrid(xs, xs)
 rr = np.hypot(GX, GY)
 m = (rr <= R + 1e-9) & (rr >= R_RECV_MIN - 1e-9)
 self._ag = (GX[m].copy(), GY[m].copy())
 return self._ag

def unsafe_of(self, ch):
 """频道 ``ch`` 仍然"没被排除"的环带格点掩码。

 判据: 该格点到频道 ``ch`` **真实测过的任一位置**的距离 <= 有效接收半径下界
 (1000 m) ⇒ 若那里真有源, 当时就会被听到 ⇒ 可以排除。
 """
 rec = self.recs[int(ch)]
 key = (int(ch), len(rec.meas_pts))
 got = self._unsafe_cache.get(key)
 if got is not None:
 return got
 gx, gy = self.annulus_grid()
 radius = float(self.cfg.cover_radius_m) + float(self.cfg.cover_slack_m)
 if not rec.meas_pts:
 m = np.ones(gx.size, bool)
 else:
 P = np.asarray(rec.meas_pts, float)
 d2 = ((gx[:, None] - P[None, :, 0]) ** 2
 + (gy[:, None] - P[None, :, 1]) ** 2)
 m = np.sqrt(np.min(d2, axis=1)) > radius
 if len(self._unsafe_cache) > 600:
 self._unsafe_cache.clear()
 self._unsafe_cache[key] = m
 return m

def uncovered_mask(self):
 """所有"从没听到过"的频道的联合未排除掩码。"""
 gx, gy = self.annulus_grid()
 unsafe = np.zeros(gx.size, bool)
 for ch in self.unheard():
 unsafe |= self.unsafe_of(ch)
 return gx, gy, unsafe

def adds_coverage(self, ch, x, y):
 """在 ``(x, y)`` 测频道 ``ch`` 是否还能排除掉新的环带格点。"""
 m = self.unsafe_of(ch)

```

```

 if not m.any():
 return False
 gx, gy = self.annulus_grid()
 r2 = float(self.cfg.cover_radius_m) ** 2
 return bool(np.any(m & (((gx - x) ** 2 + (gy - y) ** 2) <= r2)))

def is_complete(self):
 """能否断定"靶区内已无未清除源"。"""
 if self.heard_pending():
 return False
 return not bool(self.uncovered_mask()[2].any())

环带骨架

def ring_skeleton(self):
 """骨架停点 ``[(角度, 半径), ...]``，按角度排序（只构造一次）。"""
 if self._ring is None:
 pts = []
 for k, (r, n) in enumerate(self.cfg.cover_rings):
 off = (math.pi / float(n)) if (k % 2 == 1) else 0.0
 for i in range(int(n)):
 a = (off + 2.0 * math.pi * i / float(n)) % (2.0 * math.pi)
 pts.append((float(a), float(r)))
 pts.sort()
 self._ring = pts
 return self._ring

def ring_xy(self, a, r):
 return (float(r) * math.cos(a), float(r) * math.sin(a))

def forward_ring(self, dir_sign, cursor=None):
 """角向单调推进的骨架停点（已过去的不再选，方向锁死 $\Rightarrow$ 不折返）。"""
 sk = self.ring_skeleton()
 a0 = self.ring_cursor_a if cursor is None else cursor
 if a0 is None:
 a0 = math.atan2(self.pos[1], self.pos[0]) % (2.0 * math.pi)
 out = []
 for (a, r) in sk:
 d = ((a - a0) * dir_sign) % (2.0 * math.pi)
 if d < float(self.cfg.ring_min_advance_rad):
 continue
 out.append((d, a, r))
 out.sort()
 res = []
 for _d, a, r in out:
 if self._akey(a) in self.ring_served:
 continue # 这个角向站点的覆盖职责已经由某个目标停点代劳
 x, y = self.ring_xy(a, r)
 if not self.adds_any_coverage(x, y):
 continue
 res.append({"kind": "cover", "ch": None, "xy": (x, y), "r": None})
 return res

@staticmethod
def _akey(a):
 return round(float(a) % (2.0 * math.pi), 6)

def mark_served(self, x, y):
 """某个停点（半径不小于骨架半径）已经代劳覆盖 $\Rightarrow$ 附近的骨架点不必再走。

 不做这一步会留下一个隐蔽的浪费：目标停点测完之后目标被清除、不再出现在

```

路点里，于是它旁边的骨架点又冒出来 —— 实测会多走一站、多测十几个频道（同一片区域在 70 m 内被整批测了两遍）。

```

"""
if math.hypot(x, y) < min((r for (_a, r) in self.ring_skeleton()),
 default=0.0) * float(self.cfg.cover_duty_frac):

 return

for (a, r) in self.ring_skeleton():
 px, py = self.ring_xy(a, r)
 if math.hypot(px - x, py - y) <= self.cfg.merge_radius_m:
 self.ring_served.add(self._akey(a))

def adds_any_coverage(self, x, y):
 """该点是否还能"某些从没听到过的频道"排除新的环带格点。"""
 need = self.unheard()
 if not need:
 return False
 gx, gy = self.annulus_grid()
 r2 = float(self.cfg.cover_radius_m) ** 2
 near = ((gx - x) ** 2 + (gy - y) ** 2) <= r2
 if not near.any():
 return False
 for ch in need:
 if bool(np.any(self.unsafe_of(ch) & near)):
 return True
 return False

目标路点

def is_long_sliver(self, br):
 """区域是不是"长条形"（最小包围圆半径巨大，或圆心落到靶区外）。

 两条近似平行的方位会把最小包围圆撑到几公里，圆心甚至落在靶区外十几公里。
 旧实现会**真的朝那个圆心走过去** —— 实测有一局为此多走 13.5 km、多花 2700 s。
 注意：判据只看 r^* ，不能看"圆心离原点近不近"—— 贴着靶区边界、 r^* 只有 1 m
 的正常区域圆心也可能在 1770 m 处，那是完全正常的可清除目标。
 """

 return bool(br is not None and float(br["radius"]) > float(self.cfg.big_r_m))

def pending_singletons(self):
 """还只有一条方位的频道数（决定要不要动用期望场/批量定位）。"""
 return sum(1 for ch in self.heard_pending()
 if self.recs[ch].n_bearings == 1)

def _cheap_locate_point(self, rec, step=None):
 """**单点定位的最省做法**：只在当前位置周围一小圈里找最优第二检测点。

 和"期望场"用的是同一个代价模型（`best_refine_point` 的 $E[D]$ ），
 区别只在**候选半径**：这里把候选点限制在 ``short_locate_cap_m`` 以内
 （默认 420 m），不再允许"跑到 1 km 外去把基线拉长"。
 交会角小一点、 r^* 大一点没关系 —— 到了目标附近还会再定向
 （`clear_target` 的逼近—测量迭代会把误差迅速压下去），
 而**路上多走的时间是实打实丢掉的**。

 实测（`p3_paths` 场景 D）：旧做法为了给一个只知方向的源补方位，
 从 (-1716, 58) 往靶区里折了 1020 m；用小步之后行程 16.1 km → 13.4 km。

 返回 ``(xy, info)``；无可候选时 ``(None, None)``。
 """
 if not rec.pts:
 return None, None
 cap = float(self.cfg.short_locate_cap_m if step is None else step)

```

```

p, info = best_refine_point(rec.pts, rec.svds, self.pos, None, n_t=60,
 radii=tuple(r for r in self.cfg.locate_radii
 if r <= cap) or (cap,)),
 travel_cap_m=cap)

if p is not None:
 if "mode" not in info:
 info = dict(info, mode="short")
 return p, info
代价模型给不出候选（例如射线先验为空）时退化到"几何小步"
return self._basic_transverse_point(rec, cap)

def _basic_transverse_point(self, rec, cap):
 """纯几何兜底：从 S_1 出发、与上一条视线成 55 - 105° 走一小步。"""
 S1 = np.asarray(rec.pts[-1], float)
 th1 = float(rec.svds[-1])
 ts, _w, _t_hi = ray_source_samples(S1, th1, n_t=40)
 if ts.size == 0:
 return None, None
 rho = min(max(0.5 * float(self.cfg.locate_radii[0]), 0.4 * float(np.median(ts))),
 cap)
 cur = np.asarray(self.pos, float)
 best = None
 for ang in self.cfg.cheap_locate_angles:
 for sgn in (1.0, -1.0):
 a = math.radians(th1 + sgn * ang)
 p = S1 + rho * np.array([math.cos(a), math.sin(a)])
 if float(np.hypot(*p)) > R_ARENA - 1.0:
 continue
 dd = float(np.linalg.norm(p - cur))
 if best is None or dd < best[0]:
 best = (dd, p, ang)
 if best is None:
 return None, None
 _d, p, ang = best
 return ((float(p[0]), float(p[1])),
 {"mode": "basic_transverse", "rho": rho, "angle_deg": ang,
 "move_m": _d})

def _locate_point(self, ch, rec, br):
 """给"还需要更多方位"的频道选下一个检测点。

 两个候选来源，取总代价小者：

 * **以当前位置为中心**（`best_refine_point` 的默认做法）—— 走到候选点的
 代价已经算在 `cost_s` 里，适合"顺手就能测"的场合；
 * **以"上一条方位的检测点" S_1 为中心**（半径 150 - 700 m）——
 这对"贴靶区边界、有效接收半径又小"的源是关键：环带骨架在 $r=1000$ ，
 而 $r \approx 1800$ 处与骨架站位同方向的源，相邻两个站位离它有 1.3 km，
 早就超出它的有效接收半径了 —— 骨架只能给它**一条**方位，
 第二条必须专门回到 S_1 附近横向补测。

 待定频道很少时（< field_min_pending）不走期望场，直接用
 `_cheap_locate_point` 的小步横向补测。
 """
 if self.pending_singletons() < int(self.cfg.field_min_pending):
 p, info = self._cheap_locate_point(rec)
 if p is not None:
 return p, info
 region = {"center": br["center"]} if br is not None else None
 p, info = best_refine_point(rec.pts, rec.svds, self.pos, region, n_t=90,
 radii=tuple(self.cfg.locate_radii),
 travel_cap_m=self.cfg.locate_travel_cap_m)

```

```

if p is None:
 return self._cheap_locate_point(rec)
best = (float(info["cost_s"]) if "cost_s" in info else float("inf"), p, info)
S1 = tuple(rec.pts[-1])
if math.dist(S1, self.pos) > 1.0:
 p2, info2 = best_refine_point(
 rec.pts, rec.svds, S1, region, n_t=90,
 radii=tuple(self.cfg.locate_radii),
 travel_cap_m=self.cfg.locate_bearing_cap_m)
 if p2 is not None:
 total = (float(info2.get("cost_s", 1e9))
 + math.dist(self.pos, p2) / V_ROBOT)
 if total < best[0]:
 best = (total, p2, info2)
p, info = best[1], best[2]
if p is None and br is not None:
 p = tuple(self._safe_point(br["center"]))
 info = {"mode": "center_fallback"}
return p, info

@staticmethod
def _safe_point(p):
 """把点夹到靶区圆盘内（两条近平行方位会把区域中心撑到靶区外十几公里）。"""
 x, y = float(p[0]), float(p[1])
 d = math.hypot(x, y)
 lim = R_ARENA - 1.0
 if d > lim:
 return (x * lim / d, y * lim / d)
 return (x, y)

def target_waypoints(self):
 """当前所有"听到了但没清掉"的频道 → 下一批目标路点。

 每个频道给的是 ``clear``（有界且不是长条形 ⇒ 直接去它的定位区域中心）
 或 ``locate``（只有一条方位 / 区域无界 / 长条形 ⇒ 先去横向补一条方位）。
 长条形绝不能去最小包围圆的圆心 —— 圆心可能在靶区外十几公里。

 ``locate`` 的门槛（"该不该专门跑一趟去补方位"）分两种：

 * **只有一条方位**：先问"环形航路还会不会再听到它"（`_lost_fraction`），
 还会听到就**耐心等**（环形航路每个角度只经过一次，下一个骨架停点
 通常就在它那个方向）；只有确实听不到了、或已经等了
 ``locate_patience_stops`` 站，才专门去补测；
 * **有 >=2 条方位但区域无界/长条形**：环形航路再给方位也压不下去
 （问题在于交会角太差），必须专门换一个横向检测点。
 """
 cfg = self.cfg
 fut_xy = None
 wps = []
 for ch in self.heard_pending():
 if self.on_hold(ch):
 continue
 rec = self.recs[ch]
 br = self.region_info(rec)
 if br is not None and not self.is_long_sliver(br):
 wps.append({"kind": "clear", "ch": ch, "xy": self._safe_point(br["center"]),
 "r": float(br["radius"]), "n_bearings": rec.n_bearings})
 continue
 # 区域无界/退化/长条形：必须换一个横向检测点
 if cfg.locate_mode == "never" or self.stop_seq - \
 self._last_bearing_stop.get(int(ch), -99) < int(cfg.locate_stale_stops):
 continue

```

```

if rec.n_bearings < 2:
 # 还会被环形航路听到 => 白等就行，别专门跑一趟
 waited = self.stop_seq - self._last_bearing_stop.get(int(ch), -99)
 if waited < int(cfg.locate_patience_stops):
 if fut_xy is None:
 fut_xy = [w["xy"] for w in
 self.forward_ring(self.ring_dir if self.ring_dir else 1.0)]
 if self._lost_fraction(rec, fut_xy) < float(cfg.risk_min_lost):
 continue
 p, info = self._locate_point(ch, rec, br)
 if p is None:
 continue
 p = self._safe_point(p)
 if math.hypot(p[0] - self.pos[0], p[1] - self.pos[1]) \
 > cfg.locate_travel_cap_m:
 continue
 wps.append({"kind": "locate", "ch": ch, "xy": p,
 "r": (float(br["radius"]) if br is not None else None),
 "info": info, "n_bearings": rec.n_bearings})

return wps

航路组装: 骨架 + 目标"最便宜插入"

def assemble_route(self, dir_sign, targets=None, cursor=None):
 """返回 ``(route, length)``: 骨架按角向单调排列，目标插在最便宜的空隙里。

 ``cover_duty``: 被目标"顶替"掉的骨架停点，其覆盖职责转交给该目标 ——
 到那一站时必须把"从没听到过"的频道一并测掉，否则会留下覆盖空洞。
 """
 cfg = self.cfg
 saved = self.ring_cursor_a
 self.ring_cursor_a = cursor
 ring_all = self.forward_ring(dir_sign, cursor=cursor)
 self.ring_cursor_a = saved
 tgs = [dict(t) for t in (self.target_waypoints() if targets is None else targets)]
 for t in tgs:
 t["cover_duty"] = any(math.dist(w["xy"], t["xy"]) <= cfg.merge_radius_m
 for w in ring_all)
 ring = [w for w in ring_all
 if all(math.dist(w["xy"], t["xy"]) > cfg.merge_radius_m for t in tgs)]
 seq = list(ring)
 start = np.asarray(self.pos, float)

 def L(a, b):
 return math.dist(tuple(a), tuple(b))

 for t in sorted(tgs, key=lambda w: math.dist(self.pos, w["xy"])):
 best_i, best_extra = 0, None
 for i in range(len(seq) + 1):
 a = tuple(start) if i == 0 else seq[i - 1]["xy"]
 b = seq[i]["xy"] if i < len(seq) else None
 extra = L(a, t["xy"]) + (L(t["xy"], b) - L(a, b) if b else 0.0)
 if best_extra is None or extra < best_extra:
 best_i, best_extra = i, extra
 cap = (cfg.locate_max_detour_m if t["kind"] == "locate"
 else cfg.clear_max_detour_m)
 forced = (t["kind"] == "locate"
 and self._locate_skips.get(t["ch"], 0) >= int(cfg.locate_force_after))
 if best_extra is not None and best_extra > float(cap) and not forced:
 # **定位**路点绕行太狠: 本轮先不排它(航路绕下去往往自然会给它第二条方位)。
 # 但连续跳过 locate_force_after 轮之后就强制排进去。

```

```

 # **清除**路点不在这个规则里 —— 听到了的源必须清，绕行大只影响排序，
 # 不能变成"不清"（`clear_max_detour_m` 默认取一个极大的值）。
 self._locate_skips[t["ch"]] = self._locate_skips.get(t["ch"], 0) + 1
 continue

 seq.insert(best_i, t)
 mode = str(self.cfg.route_mode)
 if mode == "tsp":
 seq = self.nn_2opt(seq)
 elif mode == "polish":
 seq = self.polish_route(seq)
 if not seq:
 return [], 0.0
 total = L(start, seq[0]["xy"])
 for a, b in zip(seq, seq[1:]):
 total += L(a["xy"], b["xy"])
 return seq, total

---- 航路后处理：允许目标在骨架空隙间自由换位，但不许骨架倒序 ----
def _order_total(self, order, pts, s):
 d = float(np.linalg.norm(pts[order[0]] - s))
 for a, b in zip(order, order[1:]):
 d += float(np.linalg.norm(pts[b] - pts[a]))
 return d

def polish_route(self, seq):
 """2-opt 后处理：**骨架停点的相对顺序不变**，目标可以在空隙间换位。

 实测（12 局）"骨架顺序 + 最便宜插入"比"同样这些路点的最优航路"多走
 3.0 km/局（13841 vs 10806 m），其中大部分是"某个目标本该排在后面一站
 之后"这种局部次序问题；允许目标换位、但禁止骨架倒序，既拿回这部分行程，
 又保留"绕圈不折返"的结构性保证。
 """
 if len(seq) < 4:
 return seq
 s = np.asarray(self.pos, float)
 pts = [np.asarray(w["xy"], float) for w in seq]
 is_cover = [w["kind"] == "cover" for w in seq]
 cid, c = {}, 0
 for i, f in enumerate(is_cover):
 if f:
 cid[i] = c
 c += 1

 def ring_ok(order):
 last = -1
 for i in order:
 if is_cover[i]:
 if cid[i] < last:
 return False
 last = cid[i]
 return True

 order = list(range(len(seq)))
 best = self._order_total(order, pts, s)
 improved = True
 while improved:
 improved = False
 for i in range(len(order) - 1):
 for j in range(i + 1, len(order)):
 cand = order[:i] + order[i:j + 1][::-1] + order[j + 1:]
 if not ring_ok(cand):
 continue

```

```

 d = self._order_total(cand, pts, s)
 if d < best - 1e-9:
 order, best, improved = cand, d, True
 return [seq[i] for i in order]

def nn_2opt(self, seq):
 """完全自由的最近邻 + 2-opt（消融对照用；结构约束全部放开）。"""
 if len(seq) < 4:
 return seq
 s = np.asarray(self.pos, float)
 pts = [np.asarray(w["xy"], float) for w in seq]
 k = len(seq)
 left, cur, order = list(range(k)), s, []
 while left:
 j = min(left, key=lambda i: float(np.linalg.norm(pts[i] - cur)))
 order.append(j)
 left.remove(j)
 cur = pts[j]
 best = self._order_total(order, pts, s)
 improved = True
 while improved:
 improved = False
 for i in range(k - 1):
 for j in range(i + 1, k):
 cand = order[:i] + order[i:j + 1][::-1] + order[j + 1:]
 d = self._order_total(cand, pts, s)
 if d < best - 1e-9:
 order, best, improved = cand, d, True
 return [seq[i] for i in order]

def pick_direction(self):
 """选定**绕行方向**与**进入环带的位置**。

旧实现直接用"当前位置的极角"当游标：原点扫描之后机器人还在原点附近，

atan2(0,0) 恒为 0，于是每次都是"从正东开始绕"——这就是路径图上

 "进入环的位置是固定的"的原因。

现在枚举 **（每个骨架停点 × 两个方向）** 共 $2n$ 个入口，各组装一次航路，

 取总长最短的；长度相差在 entry_tie_m 以内的，再按"进入方向是否

 指向信息方向"（所有单方位频道候选源位的平均方向）来选——

 从源多的那一侧切入环带，源会更早被找到、更早被清掉，

 这也正是"平均定位清除时间"这个指标最敏感的地方。

 """
 cfg = self.cfg
 step = 2.0 * math.pi / max(len(self.ring_skeleton()), 1)
 if not cfg.entry_select:
 # 消融对照：只用"当前位置极角"当游标（旧行为）
 best = None
 for s in (1.0, -1.0):
 _seq, L = self.assemble_route(s)
 if best is None or L < best[0]:
 best = (L, s)
 self.ring_dir = best[1]
 return self.ring_dir
 cands = []
 for s in (1.0, -1.0):
 for i, (a, _r) in enumerate(self.ring_skeleton()):
 cursor = (a - s * 0.5 * step) % (2.0 * math.pi)
 seq, L = self.assemble_route(s, cursor=cursor)
 cands.append((L, s, a))
 if not cands:
 self.ring_dir = 1.0

```



```

 return self.ring_dir
 best_len = min(c[0] for c in cands)
 a_info = self.info_direction_deg()
 pool = [c for c in cands if c[0] <= best_len + float(cfg.entry_tie_m)]
 if a_info is None:
 chosen = min(pool, key=lambda c: c[0])
 else:
 def key(c):
 diff = abs((math.degrees(c[2]) - a_info + 180.0) % 360.0 - 180.0)
 return (round(diff / 15.0), c[0]) # 先看是否朝信息方向, 再看长度
 chosen = min(pool, key=key)
 _L, s, a = chosen
 self.ring_dir = s
 self.ring_cursor_a = (a - s * 0.5 * step) % (2.0 * math.pi)
 self.log(f"进入环带: 选 ({self.cfg.cover_rings[0][0] * math.cos(a):.0f}, "
 f"{self.cfg.cover_rings[0][0] * math.sin(a):.0f}) "
 f" (角 {math.degrees(a):.0f}°, 绕行方向 "
 f"{'逆时针' if s > 0 else '顺时针'}; "
 f"{len(cands)} 个入口里航路 {best_len:.0f} m 最短"
 + (f", 信息方向 {a_info:.0f}°" if a_info is not None else ""))
 return self.ring_dir

def info_direction_deg(self):
 """信息方向: 所有"只有一条方位"的频道按先验中位源位加权求平均方向。"""
 acc = np.zeros(2)
 used = 0
 for ch in self.heard_pending():
 rec = self.recs[ch]
 if rec.n_bearings != 1:
 continue
 S1 = np.asarray(rec.pts[-1], float)
 ts, _w, _t_hi = ray_source_samples(S1, float(rec.svds[-1]), n_t=30)
 if ts.size == 0:
 continue
 t = float(np.median(ts))
 u = np.array([math.cos(math.radians(float(rec.svds[-1]))),
 math.sin(math.radians(float(rec.svds[-1])))])
 g = S1 + t * u
 n = float(np.linalg.norm(g))
 if n > 1e-6:
 acc += g / n
 used += 1
 if used == 0 or float(np.linalg.norm(acc)) < 1e-6:
 return None
 return math.degrees(math.atan2(float(acc[1]), float(acc[0]))) % 360.0

停点测量

def _cross_ok(self, rec, x, y):
 """从 `(x, y)` 再测这条频道, 与上一条视线的张角是否够大 (否则白测)。"""
 if not rec.pts:
 return True
 (sx, sy), a = rec.pts[-1], rec.svds[-1]
 dx, dy = x - sx, y - sy
 n = math.hypot(dx, dy)
 if n < 1e-6:
 return False
 u = (math.cos(math.radians(a)), math.sin(math.radians(a)))
 cosang = abs((dx * u[0] + dy * u[1]) / n)
 cosang = max(-1.0, min(1.0, cosang))
 return math.degrees(math.acos(cosang)) >= float(self.cfg.min_cross_deg)

```

```

def chans_at(self, x, y, ch=None, cover_duty=False):
 """本停点该测哪些频道。

 * 目标频道一定测；
 * **0 方位**频道（这些频道是"没有源"的频道，测它们纯粹为了证明没有残留源）：
 只有在本停点**负有覆盖职责**、且本点还能排除新的环带格点时才测。
 "负有覆盖职责"= 骨架停点 / 顶替了骨架停点的目标 / **半径不小于骨架半径的
 外圈目标停点** —— 最后一条很关键：半径越大的检测点，对 r=1800 外圈 rim 的
 角向覆盖半宽越大（r=880 时 $\pm 18^\circ$ ，r=1500 时 $\pm 34^\circ$ ），所以**顺路清外圈源时
 顺手一测，就能顶掉两个骨架停点**，比"先跑完骨架再清"省一倍检测；
 * **1 方位**频道：张角够大（>= ``min_cross_deg``）才测。
 """
 out = []
 duty = bool(cover_duty) or ch is None
 if not duty:
 rr = min((r for (_a, r) in self.ring_skeleton()), default=0.0)
 duty = math.hypot(x, y) >= rr * float(self.cfg.cover_duty_frac)
 for c in self.pending():
 rec = self.recs[c]
 if rec.n_bearings == 0:
 if duty and self.adds_coverage(c, x, y):
 out.append(c)
 elif rec.n_bearings == 1:
 if self._cross_ok(rec, x, y):
 out.append(c)
 if ch is not None and c == int(ch) and c not in out:
 out.append(c)
 if ch is not None and int(ch) in out:
 out.remove(int(ch))
 out.insert(0, int(ch))
 return out

def visit(self, wp):
 x, y = wp["xy"]
 chans = self.chans_at(x, y, wp.get("ch"),
 cover_duty=bool(wp.get("cover_duty")))
 if not chans:
 return 0
 n = self.scan_at(float(x), float(y), chans)
 self.note_pos()
 return n

清除：逼近—测量—试探 的迭代

def probe_path(self, c, r, poly=None, tried=()):
 """覆盖式试探点，按"里圈起、环内按极角、贪心最近邻"排成一条短折线。"""
 from p3_robot import _dist_point_polygon
 cfg = self.cfg
 c = np.asarray(c, float)
 s = float(cfg.probe_spacing_m)
 rr = min(float(r), float(cfg.probe_max_radius_m)) + R_CLEAR
 u = np.array([s, 0.0])
 v = np.array([s / 2.0, s * math.sqrt(3.0) / 2.0])
 N = int(math.ceil(rr / s)) + 1
 cand = []
 for i in range(-N, N + 1):
 for j in range(-N, N + 1):
 p = c + i * u + j * v
 if float(np.hypot(*(p - c))) <= rr + 1e-9:
 cand.append(p)

```

```

if poly is not None and len(poly) >= 3:
 P = np.asarray(poly, float)
 keep = [p for p in cand
 if _dist_point_polygon(p, P) <= R_CLEAR + 1e-9]
 cand = keep if keep else cand
if tried:
 T = np.asarray(tried, float)
 cand = [p for p in cand
 if float(np.min(np.hypot(T[:, 0] - p[0], T[:, 1] - p[1]))) >= 2.0]
if not cand:
 return []
cand.sort(key=lambda p: (round(float(np.hypot(*(p - c))) / (2 * s)),
 math.atan2(float(p[1] - c[1]), float(p[0] - c[0]))))
cand = cand[:int(cfg.probe_max_points)]
cur = np.asarray(self.pos, float)
rest, out = list(cand), []
while rest:
 j = min(range(len(rest)),
 key=lambda i: float(np.hypot(*(rest[i] - cur))))
 p = rest.pop(j)
 out.append(p)
 cur = p
return out

兼容父类 `_probe_count` / `_estimate_clear_cost` 的调用名
def probe_points(self, c, r, poly=None, tried=()):
 return self.probe_path(c, r, poly=poly, tried=tried)

def sweep_clear(self, ch, rec, c, r, poly=None):
 probes = self.probe_path(c, r, poly=poly, tried=rec.tried_probes)
 self.log(f" 覆盖式试探: 中心 ({c[0]:.0f},{c[1]:.0f}), r={r:.0f} m, "
 f"'{'多边形区域' if poly else '外接圆'}', 试探点 {len(probes)}"
 f" (间距 {self.cfg.probe_spacing_m:.0f} m) ")
 for p in probes:
 if self.aborted or not self.can_afford(T_CLEAR_FAIL + 5.0):
 return False
 trav = math.dist(self.pos, (float(p[0]), float(p[1])))
 if not self.can_afford(trav / V_ROBOT + T_CLEAR_FAIL + 3.0):
 return False
 rec.tried_probes.append((float(p[0]), float(p[1])))
 self.stats["n_probe"] = self.stats.get("n_probe", 0) + 1
 self.stats["probe_m"] = self.stats.get("probe_m", 0.0) + trav
 if self.do_clear(float(p[0]), float(p[1]), ch):
 return True
 return False

def _measure_far(self, ch, x, y, min_gap=None):
 """在 `(x, y)` 给频道补一条方位 (与既有测量点太近就跳过)。"""
 rec = self.recs[int(ch)]
 gap = float(self.cfg.approach_eps_m if min_gap is None else min_gap)
 for q in rec.meas_pts:
 if math.hypot(x - q[0], y - q[1]) < gap:
 return None
 if not self.can_afford(
 math.dist(self.pos, (x, y)) / V_ROBOT + T_MEASURE + T_SWITCH + 4.0):
 return None
 return self.measure(float(x), float(y), int(ch))

def clear_target(self, ch, rec):
 """定位 → 逼近 → 覆盖式试探 (可迭代, 直到成功或用尽预算/迭代数)。

 旧实现在 `clear_targets()` 里把 `r* > 600 m` 的目标永久过滤掉,

```

碰到"区域无界"就直接 `clear\_fail += 1` 走人 —— 这正是清除率上不去的主因。

这里改成迭代：区域大就往中心逼近并补方位（每补一条方位区域都会明显缩小），

区域小到 ``probe\_enter\_r\_m`` 再走覆盖式试探。

```
"""
ch = int(ch)
cfg = self.cfg
self.log(f"清除频道{ch}: {rec.n_bearings} 条方位, "
 f"起点 ({self.pos[0]:.0f},{self.pos[1]:.0f})")
for it in range(int(cfg.clear_max_iters)):
 if rec.cleared or self.aborted:
 break
 self.n_clear_iter += 1
 br = self.region_info(rec)
 # (A) 没有可用区域: 换横向点补方位
 if br is None:
 p, info = self._locate_point(ch, rec, None)
 if p is None:
 break
 resp = self._measure_far(ch, p[0], p[1])
 if resp is None:
 break
 self.stats["n_refine"] += 1
 self.log(f" [{it+1}] 区域无界/退化 -> 横向补测 "
 f"({p[0]:.0f},{p[1]:.0f})")
 if resp.get("measure_result") == "near":
 return self.do_clear(float(p[0]), float(p[1]), ch)
 continue
 c = np.asarray(br["center"], float)
 r = float(br["radius"])
 d = math.dist(self.pos, tuple(c))
 long_sliver = self.is_long_sliver(br)
 # (A0) 区域还大时先靠近区域中心（顺便测一条方位把区域压小）。
 # **区域已经够小时不要走这一步**: 覆盖式试探的第一发就是"移动到
 # 区域内的试探点", 本身就是逼近; 而"逼近测量"要求新测点离既有
 # 测点至少 approach_eps_m, 对"源就在原点附近"这类目标会永远返回
 # None, 于是 clear_target 空转、clear_leftovers 反复重试
 # (实测 seed 20262713: 源在 (-4,15)、r* 只有 2.2 m, 却跑了 37.6 km)。
 if (not long_sliver) and r > cfg.probe_enter_r_m \
 and d > max(r, cfg.arrive_radius_m) + cfg.approach_eps_m:
 if not self.can_afford(d / V_ROBOT + T_MEASURE + T_SWITCH + 6.0):
 break
 resp = self._measure_far(ch, float(c[0]), float(c[1]))
 if resp is None:
 p, _info = self._locate_point(ch, rec, br)
 if p is not None:
 resp = self._measure_far(ch, p[0], p[1])
 if resp is not None:
 self.log(f" [{it+1}] 逼近区域中心 ({c[0]:.0f},{c[1]:.0f}): "
 f"移动 {d:.0f} m (r*={r:.0f} m)")
 if resp.get("measure_result") == "near":
 return self.do_clear(float(self.pos[0]), float(self.pos[1]), ch)
 continue
 # 逼近测量做不了（附近已测过）⇒ 直接进入覆盖式试探
 # (B) 区域够小: 覆盖式试探
 if r <= cfg.probe_enter_r_m and not long_sliver:
 if self.sweep_clear(ch, rec, c, r, poly=br.get("poly")):
 return True
 r2 = min(max(r * 1.8, r + 40.0), cfg.probe_max_radius_m)
 if r2 > r + 1.0:
 self.log(f" 区域内未发现目标 -> 扩大到 r={r2:.0f} m 再试")
 if self.sweep_clear(ch, rec, c, r2, poly=None):
 return True
```

```

 p, _info = self._locate_point(ch, rec, br)
 if p is None:
 break
 resp = self._measure_far(ch, p[0], p[1])
 if resp is None:
 break
 if resp.get("measure_result") == "near":
 return self.do_clear(float(p[0]), float(p[1]), ch)
 continue
(C) 区域还大 / 长条形: 补一条横向方位 (对长条形**不去圆心**, 那可能在靶区外)
if long_sliver:
 p, _info = self._locate_point(ch, rec, None)
 if p is None:
 break
 target = self._safe_point(p)
else:
 target = tuple(c)
 if d < cfg.approach_eps_m:
 p, _info = self._locate_point(ch, rec, br)
 if p is None:
 break
 target = self._safe_point(p)
resp = self._measure_far(ch, target[0], target[1])
if resp is None:
 p, _info = self._locate_point(ch, rec, None if long_sliver else br)
 if p is None:
 break
 target = self._safe_point(p)
 resp = self._measure_far(ch, target[0], target[1])
 if resp is None:
 break
self.stats["n_refine"] += 1
self.log(f" [{it+1}] 逼近测量 ({target[0]:.0f},{target[1]:.0f}): "
 f"移动 {math.dist(self.pos, target):.0f} m, r* {r:.0f} -> ?")
if resp.get("measure_result") == "near":
 return self.do_clear(float(target[0]), float(target[1]), ch)
if not rec.cleared:
 rec.clear_fail += 1
 self.hold(ch)
 self.log(f" [x] 频道{ch} 本轮未清除成功 (冷却 {cfg.fail_cooldown} 个停点)")
return rec.cleared

开场: 期望场定点 + 先清近处目标

def initial_stage(self):
 """原点扫描之后: **用期望场选第二个检测点**, 顺手清掉靠内圈的目标。

 为什么必须做: 原点那一测只听得见一定范围内的源; 这些"内圈源"若等到沿环带
 走的时候再处理, 就得从环带上专门往回跑一趟。实测 (seed 20261113) 一次
 "专门跑一趟定位"的绕行高达 2.1 km, 等于整局行程的 11%。趁还在原点附近,
 用第二问的 E[D] 代价模型在"两翼"上选第二检测点 (对多个单方位频道的候选点
 做完全连接聚类, 一次移动定住好几个源), 定位完立刻清掉, 再往环带走。

 做成**多轮**而不是一轮: 一轮只能处理一个聚类, 剩下的单方位频道会被留到
 环带上, 正是上面那种绕行的来源。
 """
 if self.aborted or self.cfg.locate_mode == "never":
 return 0
 n_total = 0
 for rnd in range(int(self.cfg.initial_rounds)):
 # 每轮的移动上限递增: 第一轮只走 350 m (近处的源这样最省),

```

```

还有没解决的频道才逐轮放宽（远处的源需要更长的基线）
cap = min(float(self.cfg.locate_travel_cap_m),
 float(self.cfg.initial_locate_max_move_m)
 * (float(self.cfg.initial_move_growth) ** rnd))

pend = []
for ch in self.heard_pending():
 rec = self.recs[ch]
 if rec.n_bearings != 1 or self.on_hold(ch):
 continue
 if math.hypot(rec.pts[0][0], rec.pts[0][1]) \
 > self.cfg.initial_locate_max_m:
 continue
 pend.append(ch)
if not pend:
 break
per = {}
for ch in pend:
 p, info = self._locate_point(ch, self.recs[ch], None)
 if p is not None:
 per[ch] = np.asarray(p, float)
if not per:
 break
groups = []
for ch, p in per.items():
 for g in groups:
 if all(float(np.linalg.norm(p - per[c2])) <= self.cfg.batch_max_m
 for c2 in g):
 g.append(ch)
 break
 else:
 groups.append([ch])
best = None
for g in groups:
 if len(g) < int(self.cfg.batch_min_channels) \
 and len(groups) > 1:
 continue
 P = np.asarray([per[c2] for c2 in g], float)
 for c0 in [P.mean(axis=0)] + [P[i] for i in range(P.shape[0])]:
 trav = float(np.linalg.norm(c0 - np.asarray(self.pos, float)))
 if trav > cap:
 continue
 cost = trav / V_ROBOT + len(g) * (T_MEASURE + T_SWITCH)
 key = (-len(g), cost) # 先看能一次定住几个频道
 if best is None or key < best[0]:
 best = (key, c0, g, trav)
if best is None:
 break
_key, c0, group, trav = best
if not self.can_afford(
 trav / V_ROBOT + len(group) * (T_MEASURE + T_SWITCH) + 20.0):
 break
self.log(f"开场定点（期望场，第 {rnd+1} 轮）：{len(group)} 个单方位频道 "
 f"{group} -> 合并第二检测点（{c0[0]:.0f},{c0[1]:.0f}）， "
 f"移动 {trav:.0f} m")
self.n_locate_stops += 1
self.scan_points.append((float(c0[0]), float(c0[1])))
before = self._state_key()
for ch in group:
 if self.aborted or not self.can_afford(T_MEASURE + T_SWITCH + 3.0):
 break
 resp = self.measure(float(c0[0]), float(c0[1]), int(ch))
 n_total += 1

```

```

 if resp.get("measure_result") == "near":
 self.do_clear(float(c0[0]), float(c0[1]), int(ch))
 self.note_pos()
 self.stop_seq += 1
 self.last_stop_xy = (float(c0[0]), float(c0[1]))
 # 顺手清掉已经定下来、又在附近的目标 ("先清较靠近原点的目标")
 self.opportunistic_clear(radius=self.cfg.initial_clear_radius_m, max_n=4)
 if self._state_key() == before:
 break
 self.opportunistic_clear(radius=self.cfg.initial_clear_radius_m, max_n=4)
 return n_total

def opportunistic_clear(self, radius=250.0, max_n=3):
 """脚下已经定位好、区域也够小的目标立刻清掉 ("边走边清"的落实)。"""
 done = 0
 for _ in range(int(max_n)):
 if self.aborted or not self.can_afford(self.cfg.min_action_budget_s):
 break
 cand_s = []
 for ch in self.heard_pending():
 if self.on_hold(ch):
 continue
 br = self.region_info(self.recs[ch])
 if br is None or br["radius"] > self.cfg.probe_enter_r_m * 2.0:
 continue
 d = math.dist(self.pos, br["center"])
 if d <= max(radius, br["radius"]):
 cand_s.append((d, ch))
 if not cand_s:
 break
 cand_s.sort()
 ch = cand_s[0][1]
 if not self.clear_target(ch, self.recs[ch]):
 break
 done += 1
 return done

角向扇区: **走过的扇区里"该清没清"的目标不许留到下一圈**

def offroute_clear(self, here, nxt, nxt2=None, max_n=None):
 """**"现在清"还是"等下一站再清"** —— 用"哪一个的额外绕行更小"来判。

 用户给的直觉是"看第一条方位与行进方向的夹角": 夹角小 (源基本在行进方向上)
 就等下一站, 夹角大就**立刻走一小步、定位并清除**。方向完全正确 ——
 环形航路每个角度只经过一次, 等下一站 = 让下一站替我们定位、
 但之后要**拐回来清**, 那个回头距离由"源离环带多远"决定。

 但**只按夹角判太粗** (实测: 常规随机 330 → 350、外圈环带 4085 → 5238 s),
 因为夹角大是常态, 而"拐回来"到底多花多少, 取决于源离哪一站更近。
 所以这里把夹角当**粗筛**, 真正的判据是拿射线先验直接算两种做法的
 额外绕行:

 * 现在清: $|A \rightarrow G| + |G \rightarrow B| - |A \rightarrow B|$
 * 等下一站: $|B \rightarrow G| + |G \rightarrow C| - |B \rightarrow C|$ (没有 C 时按"从 B 出再回 B"算)

 按面积均匀的射线先验对 $|G|$ 加权平均, 前者明显更小才就地清。
 这样"源就在正前方" (B 更近) 自然选等待, "源在侧后方" (A 更近) 自然选就地清,
 夹角判据只是它的一个粗糙近似。
 """
 cfg = self.cfg
 if not cfg.offroute_clear or self.aborted or nxt is None:

```

```

 return 0
 ax, ay = here["xy"]
 bx, by = nxt["xy"]
 A = np.array([ax, ay], float)
 B = np.array([bx, by], float)
 C = (np.array(nxt2["xy"], float) if nxt2 is not None
 else B + (B - A))
 phi = math.atan2(by - ay, bx - ax)
 cands = []
 for ch in self.heard_pending():
 rec = self.recs[ch]
 if rec.n_bearings != 1 or self.on_hold(ch):
 continue
 # 只处理"就是在这站听到的" (上一条方位的检测点离本站很近)
 if math.dist(rec.pts[-1], (ax, ay)) > 80.0:
 continue
 th = float(rec.svds[-1])
 sep = abs(math.degrees((th - math.degrees(phi) + 180.0) % 360.0 - 180.0))
 if sep < float(cfg.wait_angle_min_deg):
 continue # 源几乎在行进方向上 => 等下一站肯定更省
 ts, w, _t_hi = ray_source_samples(np.asarray(rec.pts[-1], float), th,
 n_t=50)
 if ts.size == 0:
 continue
 if float(_t_hi) > float(cfg.offroute_max_thi_m):
 continue # 射线太长 => 源位估计不可靠, 别拿它做判断
 u = np.array([math.cos(math.radians(th)), math.sin(math.radians(th))])
 S1 = np.asarray(rec.pts[-1], float)
 G = S1[None, :] + ts[:, None] * u[None, :]

 def d(X):
 return np.hypot(G[:, 0] - X[0], G[:, 1] - X[1])
 # 源位先验的加权平均距离太远时, 估计不可靠, "现在清"多半是白跑
 if float(np.dot(w, d(A))) > float(cfg.offroute_max_dist_m):
 continue
 extra_now = d(A) + d(B) - float(np.linalg.norm(B - A))
 extra_wait = d(B) + d(C) - float(np.linalg.norm(C - B))
 e_now = float(np.dot(w, extra_now))
 e_wait = float(np.dot(w, extra_wait))
 if e_now + float(cfg.offroute_margin_m) >= e_wait:
 continue # 等下一站更省 => 耐心等
 cands.append((e_wait - e_now, ch, sep))
 cands.sort(reverse=True)
 n_max = int(max_n or cfg.offroute_max_per_stop)
 done = 0
 for gain, ch, _sep in cands[:max(0, n_max)]:
 rec = self.recs[ch]
 p, _info = self._cheap_locate_point(rec, step=cfg.offroute_step_m)
 if p is None:
 continue
 dd = math.dist(self.pos, p)
 if dd > float(cfg.offroute_step_m) * 1.3:
 continue
 if not self.can_afford(dd / V_ROBOT + T_MEASURE + T_SWITCH + 8.0):
 continue
 resp = self.measure(float(p[0]), float(p[1]), int(ch))
 self.note_pos()
 self.log(f" 就地定位 频道{ch}: 预计省 {gain:.0f} m (现在清 vs 等下一站) "
 f"-> 走 {dd:.0f} m 到 ({p[0]:.0f},{p[1]:.0f}) 补第二条方位")
 if resp.get("measure_result") == "near":
 self.do_clear(float(p[0]), float(p[1]), int(ch))
 done += 1

```



```

 continue
 br = self.region_info(rec)
 if br is None:
 continue
 if self.clear_target(int(ch), rec):
 done += 1
return done

def risk_locate(self, max_n=None):
 """**防过期补测**: 对"马上就要听不到"的单方位频道, 就地走一小步补第二条方位。

 为什么必须做: 这条航路每个角度**大致只经过一次**。某个频道如果在它所在
 扇区只拿到一条方位、之后环形航路越走越远, 就只能等下一圈(或收尾阶段
 横穿全场)——实测 `p3_paths` 场景 C 里 ch6 就是这样横穿了 3.4 km。

 判据(按射线先验加权): 把源位按面积均匀撒在射线上, 统计"往后
 ``risk_lookahead`` 个骨架停点**都听不到** (距离 > 1000 m, 而任何源的
 有效接收半径都 >= 1000 m)"的比例; 比例超过 ``risk_min_lost`` 就说明
 这个源有相当大概率要"过期"。此刻我们还在它的扇区里, 走一小步补一条方位
 最便宜; 步长只有 ``risk_step_m`` (默认 300 m), 基线短一点无所谓——
 等清到目标附近还会再定向(`clear_target` 的逼近—测量迭代)。
 """
 cfg = self.cfg
 if not cfg.risk_locate or self.aborted:
 return 0
 fut = self.forward_ring(self.ring_dir if self.ring_dir else 1.0)
 fut_xy = [w["xy"] for w in fut[:int(cfg.risk_lookahead)]]
 if not fut_xy:
 return 0
 todo = []
 for ch in self.heard_pending():
 rec = self.recs[ch]
 if rec.n_bearings != 1 or self.on_hold(ch):
 continue
 if self.stop_seq - self._last_bearing_stop.get(int(ch), -99) <= 0:
 continue # 刚在本站拿到方位, 先给下一站一次机会
 if self._lost_fraction(rec, fut_xy) < float(cfg.risk_min_lost):
 continue
 p, info = self._cheap_locate_point(rec)
 if p is None:
 continue
 d = math.dist(self.pos, p)
 if d > float(cfg.risk_step_m) * 1.5:
 continue
 todo.append((d, ch, p, info))
 todo.sort()
 n = 0
 for _d, ch, p, _info in todo[:int(max_n or cfg.risk_max_per_stop)]:
 if not self.can_afford(math.dist(self.pos, p) / V_ROBOT
 + T_MEASURE + T_SWITCH + 8.0):
 break
 resp = self.measure(float(p[0]), float(p[1]), int(ch))
 n += 1
 self.log(f" 防过期补测 频道{ch}: 走到 ({p[0]:.0f},{p[1]:.0f})"
 f" ({_d:.0f} m) 取第二条方位")
 if resp.get("measure_result") == "near":
 self.do_clear(float(p[0]), float(p[1]), int(ch))
 break
 if n:
 self.note_pos()
 self.opportunistic_clear(radius=self.cfg.sector_clear_m, max_n=2)
 return n

```

```

def _lost_fraction(self, rec, fut_xy):
 """先验中"往后这些骨架停点都听不到该源"的概率（面积均匀的射线先验）。"""
 S1 = np.asarray(rec.pts[-1], float)
 th1 = float(rec.svds[-1])
 ts, w, _t_hi = ray_source_samples(S1, th1, n_t=60)
 if ts.size == 0:
 return 0.0
 u = np.array([math.cos(math.radians(th1)), math.sin(math.radians(th1))])
 G = S1[None, :] + ts[:, None] * u[None, :]
 heard = np.zeros(ts.size, bool)
 for (x, y) in fut_xy:
 heard |= (np.hypot(G[:, 0] - x, G[:, 1] - y) <= R_RECV_MIN)
 return float(w[~heard].sum())

def sector_clear(self, a_from, a_to):
 """把"刚刚走过的角向扇区"里已经定位好的目标就地清掉。

 扇区定义：从上一站到本站的方位角区间（沿绕行方向）。这就是用户说的
 "每个角度大致只经过一次，覆盖的点要尽快清理"——目标已经能清了、
 又正好在我们刚扫过的扇区里，就地清掉比留给下一圈便宜得多。
 """
 if a_from is None or a_to is None or self.aborted:
 return 0
 cap = float(self.cfg.sector_clear_m)
 todo = []
 for ch in self.heard_pending():
 if self.on_hold(ch):
 continue
 br = self.region_info(self.recs[ch])
 if br is None:
 continue
 xy = br["center"]
 d = math.dist(self.pos, xy)
 if d > max(cap, br["radius"]):
 continue
 a = math.atan2(xy[1], xy[0]) % (2.0 * math.pi)
 if self._in_sector(a_from, a_to, a):
 todo.append((d, ch))
 todo.sort()
 done = 0
 for _d, ch in todo:
 if self.aborted or not self.can_afford(self.cfg.min_action_budget_s):
 break
 if self.clear_target(ch, self.recs[ch]):
 done += 1
 return done

def _in_sector(self, a0, a1, a):
 """``a`` 是否落在"从 ``a0`` 走到 ``a1``"的短弧区间里。"""
 span = (a1 - a0) % (2.0 * math.pi)
 if span > math.pi:
 span = (a0 - a1) % (2.0 * math.pi)
 rel = (a0 - a) % (2.0 * math.pi)
 else:
 rel = (a - a0) % (2.0 * math.pi)
 return rel <= span + 1e-9

主循环

def run(self):

```

```

t0 = time.time()
resp = self.arena.enter()
if resp.get("accepted") is not True:
 raise ArenaError(f"/enter 未被接受: {resp}")
self.log(f"进入靶区: 可用预算 {self.remaining():.0f} s"
 f" ({self.arena.budget_kind()}); 策略=统一滚动航路")
try:
 # ---- 第 0 轮: 原点全频道扫描 ----
 chans0 = self.chans_at(0.0, 0.0)
 self.log(f"第 0 轮扫描 (原地): {len(chans0)} 个频道 {chans0}")
 self.scan_points.append(self.pos)
 if chans0:
 self.scan_at(self.pos[0], self.pos[1], chans0)
 self.note_pos()
 self.stop_seq += 1

 # ---- 开场: 期望场定点, 先清近处目标 ----
 self.initial_stage()

 # ---- 统一滚动航路 ----
 stall = 0
 while (not self.aborted and self.stop_seq < int(self.cfg.max_stops)
 and self.remaining() > self.cfg.min_stop_budget_s):
 if self.is_complete():
 self.log("* 所有频道要么已清除、要么已被 1000 m 覆盖排除 -> 收工")
 break
 targets = self.target_waypoints()
 if self.ring_dir == 0.0:
 self.pick_direction()
 route, _L = self.assemble_route(self.ring_dir, targets=targets,
 cursor=self.ring_cursor_a)
 if not route:
 if self.opportunistic_clear(radius=1e9, max_n=4):
 continue
 if self.clear_leftovers():
 continue
 self.log("没有可用路点、也没有可处理的目标 -> 结束")
 break
 wp = route[0]
 # 防死循环: 与上一个停点几乎重合时退到下一站
 if (self.last_stop_xy is not None
 and math.dist(wp["xy"], self.last_stop_xy) < self.cfg.min_move):
 alt = next((w for w in route[1:]
 if math.dist(w["xy"], self.last_stop_xy)
 >= self.cfg.min_move), None)
 if alt is None:
 self.log("航路剩余点都与当前停点重合 -> 结束")
 break
 wp = alt
 cost = (math.dist(self.pos, wp["xy"]) / V_ROBOT
 + T_MEASURE + T_SWITCH + 4.0)
 if not self.can_afford(cost):
 self.log(f"剩余时间不足以再走一站 (约 {cost:.0f} s) -> 收工")
 break
 before = self._state_key()
 nxt = route[1] if len(route) > 1 else None
 nxt2 = route[2] if len(route) > 2 else None
 n = self.visit(wp)
 self.stop_seq += 1
 self.last_stop_xy = tuple(wp["xy"])
 self.mark_served(float(wp["xy"][0]), float(wp["xy"][1]))
 if wp["kind"] == "cover":

```

```

 self.ring_cursor_a = math.atan2(wp["xy"][1], wp["xy"][0]) \
 % (2.0 * math.pi)
 self.tour_log.append({"stop": self.stop_seq, "kind": wp["kind"],
 "ch": wp.get("ch"),
 "xy": [round(v, 1) for v in wp["xy"]],
 "n_chans": n,
 "t": round(float(self.arena.virtual_time_s), 1)})
 self.log(f"航路第 {self.stop_seq} 站 | {self._wp_desc(wp)} | "
 f"测 {n} 个频道 | 已排除 "
 f"{1.0 - float(self.uncovered_mask()[2].mean()):.3f} | "
 f"已清除 {self._n_cleared()}")
 if n == 0:
 break
 # 到站之后：本站目标立刻处理，附近可清目标顺手清
 a_prev = self._prev_stop_a
 a_now = math.atan2(wp["xy"][1], wp["xy"][0]) % (2.0 * math.pi)
 self._prev_stop_a = a_now
 ch = wp.get("ch")
 if ch is not None and not self.recs[int(ch)].cleared \
 and self.region_info(self.recs[int(ch)]) is not None:
 self.clear_target(int(ch), self.recs[int(ch)])
 self.opportunistic_clear(radius=self.cfg.merge_radius_m, max_n=2)
 # ① 不在行进方向上的新目标：就地补测并清除（见 wait_angle_min_deg）
 self.offroute_clear(wp, nxt, nxt2)
 # ② 刚走过的角向扇区里"已经能清"的目标就地清掉（默认关闭）
 self.sector_clear(a_prev, a_now)
 # ③ 马上就要"过期"的方位频道（默认关闭）
 self.risk_locate()
 if self._state_key() == before:
 stall += 1
 if stall >= 3:
 self.log("连续 3 站没有产生任何新信息 -> 结束")
 break
 else:
 stall = 0

 # ---- 收尾 ----
 self.clear_leftovers()
finally:
 try:
 self.arena.exit()
 except ArenaError as e:
 self.log(f"/exit 异常: {e}")
self.stats["program_runtime_s"] = time.time() - t0
self.stats["n_scan_rounds"] = self.stop_seq
rep = self.report()
cov, mdist = self.coverage_now()
rep["coverage"] = cov
rep["max_uncovered_dist_m"] = mdist
rep["complete_proof"] = bool(self.is_complete())
rep["n_cover_stops"] = self.n_cover_stops
rep["n_locate_stops"] = self.n_locate_stops
rep["n_clear_iter"] = self.n_clear_iter
return rep

def clear_leftovers(self, max_steps=14):
 """收尾：剩下的"听到过但没清掉"的频道，逐个自适应定位清除。"""
 done = 0
 for _ in range(int(max_steps)):
 if self.aborted or self.remaining() <= self.cfg.min_stop_budget_s:
 break
 left = [ch for ch in self.heard_pending() if not self.on_hold(ch)]

```

```

 if not left:
 for ch in self.heard_pending():
 self._hold_until.pop(int(ch), None)
 left = self.heard_pending()
 if not left:
 break
 left.sort(key=lambda c: math.dist(
 self.pos, (self.region_info(self.recs[c]) or {"center": (0.0, 0.0)})["center"]))
 ch = left[0]
 self.log(f"收尾: 处理频道{ch}")
 if self.clear_target(ch, self.recs[ch]):
 done += 1
 else:
 self.hold(ch, 1)
 return done

def _state_key(self):
 return (self._n_cleared(), sum(len(r.pts) for r in self.recs.values()),
 sum(len(r.meas_pts) for r in self.recs.values()))

def _wp_desc(self, wp):
 x, y = wp["xy"]
 if wp["kind"] == "cover":
 self.n_cover_stops += 1
 return f"骨架覆盖 ({x:.0f},{y:.0f})"
 if wp["kind"] == "clear":
 return (f"清除 ch{wp['ch']} ({wp['n_bearings']} 方位, "
 f"r*={wp['r']:.0f} m) ({x:.0f},{y:.0f})")
 self.n_locate_stops += 1
 return f"定位 ch{wp['ch']} ({wp['n_bearings']} 方位) ({x:.0f},{y:.0f})"

自检

def _nn_2opt_len(pts, start=(0.0, 0.0)):
 """从 ``start`` 出发、遍历 ``pts`` 的最近邻 + 2-opt 航路长度（下界参考）。"""
 P = [np.asarray(p, float) for p in pts]
 S = np.asarray(start, float)
 k = len(P)
 if k == 0:
 return 0.0
 left, cur, seq = list(range(k)), S, []
 while left:
 j = min(left, key=lambda i: float(np.linalg.norm(P[i] - cur)))
 seq.append(j)
 left.remove(j)
 cur = P[j]

 def tot(order):
 d = float(np.linalg.norm(P[order[0]] - S))
 for a, b in zip(order, order[1:]):
 d += float(np.linalg.norm(P[b] - P[a]))
 return d
 best = tot(seq)
 improved = True
 while improved:
 improved = False
 for i in range(k - 1):
 for j in range(i + 1, k):
 cand = seq[:i] + seq[i:j + 1][::-1] + seq[j + 1:]

```

```

 d = tot(cand)
 if d < best - 1e-9:
 seq, best, improved = cand, d, True
 return best

def selfcheck(verbose=True, seed=20260913, cases=3):
 """T1 - T6: 统一航路策略的结构性断言。"""
 from p3_arena import MockArena

 out, ok_all = [], True

 def rec(name, ok, detail=""):
 nonlocal ok_all
 ok_all = ok_all and bool(ok)
 out.append({"check": name, "ok": bool(ok), "detail": detail})
 if verbose:
 print(f"{'OK ' if ok else 'FAIL'} {name} {detail}")

 rb = TourRobot(MockArena(seed=1), TourConfig(), base=None, log=None)
 gx, gy, unsafe = rb.uncovered_mask()
 rec("T1 只有原点一次检测时, 环带全部未经排除", bool(unsafe.all()),
 f"{gx.size} 个环带格点, 未排除 {int(unsafe.sum())} 个")

 sk = rb.ring_skeleton()
 P = np.asarray([rb.ring_xy(a, r) for (a, r) in sk] + [(0.0, 0.0)], float)
 d = np.sqrt(np.min((gx[:, None] - P[None, :, 0]) ** 2
 + (gy[:, None] - P[None, :, 1]) ** 2, axis=1))
 rec("T2 原点 + 环带骨架即可覆盖环带 (最大未覆盖 <= 1000 m)",
 float(d.max()) <= 1000.0,
 f"骨架 {len(sk)} 点, 最大未覆盖 {float(d.max()):.1f} m")

 # T3 角向单调: 锁定方向后, forward_ring 的极角严格递增
 rb3 = TourRobot(MockArena(seed=2), TourConfig(), base=None, log=None)
 rb3.arena.enter()
 rb3.ring_dir = 1.0
 rb3.ring_cursor_a = 0.3
 seq = rb3.forward_ring(1.0)
 angs = [math.atan2(w["xy"][1], w["xy"][0]) % (2 * math.pi) for w in seq]
 ok3 = len(seq) > 0 and all((b - a) % (2 * math.pi) > 0 for a, b in zip(angs, angs[1:]))
 rec("T3 骨架停点按角向单调推进 (不折返的结构性保证)", ok3,
 f"{len(seq)} 个停点, 角向增量 "
 f"{[round(math.degrees((b - a) % (2 * math.pi)), 1) for a, b in zip(angs, angs[1:])][:6]}...")

 # T4 端到端
 res = []
 for k in range(cases):
 a = MockArena(seed=seed + 100 * k)
 r4 = TourRobot(a, TourConfig(), base=None, log=None)
 res.append(r4.run())
 rec("T4 mock 端到端跑通且清除率 > 0.8",
 all(r["clear_ratio"] is not None and r["clear_ratio"] > 0.8 for r in res),
 "; ".join(f"seed{seed+100*k}: {r['cleared']}/{r['n_sources']}"
 f"T={r['virtual_time_s']:.0f}s" for k, r in enumerate(res)))

 # T5 完工判据与真值一致
 bad = [seed + 100 * k for k, r in enumerate(res)
 if r["complete_proof"] and r["cleared"] != r["n_sources"]]
 rec("T5 判定『靶区无残留源』时确实全部清除", not bad, f"反例 {bad or '无'}")

 # T6 行程相对"结构下界"的倍数
 # 下界 = TSP(原点 + 骨架停点 + 全部真源位置) (预知全部源位、且骨架点免费给的理想航路)

```

```

实测 30 局该比值中位数约 1.2-1.5; 最坏出现在"源数取到上限 16 个"的个别案例上
(路点最多、组合最乱, 实测 seed 20261113 约 1.9)。这是"滚动时域在线决策"
相对"离线最优"的真实差距, 也是尾部风险, 故留到 2.1 作为报警线。
worst, ratios = 1.0, []
for k, r in enumerate(res):
 a = MockArena(seed=seed + 100 * k)
 pts = [(rr * math.cos(2 * math.pi * i / n), rr * math.sin(2 * math.pi * i / n))
 for (rr, n) in TourConfig().cover_rings for i in range(int(n))]
 pts += [(x.x, x.y) for x in a.sources]
 lb = _nn_2opt_len(pts)
 ratios.append(r["travel_m"] / max(lb, 1.0))
 worst = max(worst, ratios[-1])
rec("T6 行程不超过「骨架 + 源位最优航路」的 2.1 倍 (30 局中位数 1.2-1.5)",
 worst <= 2.1,
 f"最坏 {worst:.2f}, 逐局 {[round(x, 2) for x in ratios]}")

if verbose:
 print(f"\n自检总结: {'全部通过' if ok_all else '存在失败项'}")
return {"ok": ok_all, "checks": out}

if __name__ == "__main__":
 import argparse
 ap = argparse.ArgumentParser(description="问题3 统一滚动航路策略自检")
 ap.add_argument("--selfcheck", action="store_true")
 ap.add_argument("--cases", type=int, default=3)
 args = ap.parse_args()
 r = selfcheck(cases=args.cases)
 raise SystemExit(0 if r["ok"] else 1)

```

## 5.8 src/p3\_homing.py

```

"""Experimental Q3 homing: plan through source estimates, localize on the way.

All decisions use measurement replies only. Ground truth is read by inherited
reporting after /exit, as in TourRobot.
"""

from dataclasses import dataclass
import math
import numpy as np

from p1_intersection import clip_halfplane, wedge_halfplanes, min_enclosing_circle
from p3_arena import R_ARENA, R_RECV_MAX, R_CLEAR
from p3_tour import TourConfig, TourRobot

@dataclass
class HomingConfig(TourConfig):
 initial_rounds: int = 0
 offroute_clear: bool = False
 route_mode: str = 'tsp'
 homing_lateral_m: float = 100.0
 homing_fraction: float = 0.55
 homing_all_single: bool = True
 homing_negatives: bool = True
 homing_max_steps: int = 12

class HomingRobot(TourRobot):
 def __init__(self, arena, cfg=None, base=None, log=None):

```

```

super().__init__(arena, cfg or HomingConfig(), base=base, log=log)
self._home_regions = {}

def region(self, rec):
 br = self.region_info(rec)
 if br is None:
 return {'status':'none', 'n_bearings':rec.n_bearings}
 return dict(status='bounded', min_enclosing_center=br['center'],
 min_enclosing_radius_m=br['radius'], vertices=br['poly'] or [])

def region_info(self, rec):
 if rec.near_hits:
 x,y,t,res = next(row for row in reversed(rec.meas_log) if row[3]=='near')
 return dict(center=(x,y),radius=5.0,poly=None,status='bounded',n_bearings=rec.n_bearings)
 if not rec.pts:
 return None
 key = (rec.channel, len(rec.pts), len(rec.meas_pts))
 if key in self._home_regions:
 return self._home_regions[key]
 P = [np.array(p, float) for p in [(-1800,-1800),(1800,-1800),(1800,1800),(-1800,1800)]]
 # Tangent half planes circumscribe disks, so rounding never excludes truth.
 for S, radius in [(np.zeros(2), R_ARENA)] + [(np.array(s), R_RECV_MAX) for s in rec.pts]:
 for a in np.arange(0, 2*math.pi, math.pi/16):
 u = np.array([math.cos(a), math.sin(a)])
 P = clip_halfplane(P, u, -float(u @ S) - radius)
 for S, theta in zip(rec.pts, rec.svds):
 for a, b in wedge_halfplanes(S, theta, 1.005 + 1e-9):
 P = clip_halfplane(P, a, b)
 if not P:
 return None
 arr = np.array(P)
 origin = arr.mean(axis=0)
 circle = min_enclosing_circle(arr-origin)
 center, radius = circle if circle is not None else (np.zeros(2), np.inf)
 center = center + origin
 c = np.asarray(center, float)
 # Negative replies exclude the 1000m disk. Use them only to improve an
 # estimate, never to certify completion or shrink the outer polygon.
 if rec.n_bearings == 1 and self.cfg.homing_negatives:
 S = np.array(rec.pts[0]); a = math.radians(rec.svds[0])
 u = np.array([math.cos(a), math.sin(a)])
 t = np.linspace(5, 1500, 301)
 G = S + t[:,None]*u
 keep = np.linalg.norm(G,axis=1) <= 1800
 for x,y,time,res in rec.meas_log:
 if res == 'no_signal':
 keep &= np.linalg.norm(G - (x,y),axis=1) >= 1000
 if keep.any():
 c = np.average(G[keep], axis=0, weights=t[keep])
 radius = float(np.max(np.linalg.norm(arr-c,axis=1)))
 out = dict(center=tuple(c), radius=radius, poly=arr.tolist(),
 status='bounded', n_bearings=rec.n_bearings)
 self._home_regions[key] = out
 return out

def target_waypoints(self):
 if not self.cfg.homing_all_single:
 return super().target_waypoints()
 return [dict(kind='clear', ch=ch, xy=self._safe_point(br['center']),
 r=br['radius'], n_bearings=self.recs[ch].n_bearings)
 for ch in self.heard_pending() if not self.on_hold(ch)
 if (br := self.region_info(self.recs[ch])) is not None]

```



```

def _home_step(self, ch):
 rec = self.recs[ch]
 br = self.region_info(rec)
 if br is None:
 return self.pos
 c = np.array(br['center']); cur = np.array(self.pos)
 delta = c-cur; d = float(np.linalg.norm(delta))
 if rec.n_bearings == 1:
 a = math.radians(rec.svds[-1]); u = np.array([math.cos(a),math.sin(a)])
 normal = np.array([-u[1],u[0]])
 ahead = cur + self.cfg.homing_fraction*delta
 lat = min(self.cfg.homing_lateral_m, max(35., 0.25*d))
 opts = [self._safe_point(ahead + sign*lat*normal) for sign in [-1,1]]
 return min(opts,key=lambda p:math.dist(p,c)+math.dist(p,cur))
 if d > 25:
 return self._safe_point(c)
 a = math.radians(rec.svds[-1]); normal = np.array([-math.sin(a),math.cos(a)])
 lat = min(80., max(25., br['radius']*0.3))
 return self._safe_point(cur + lat*normal)

def visit(self, wp):
 ch = wp.get('ch')
 if ch is not None and self.recs[ch].n_bearings == 1:
 p = self._home_step(ch)
 wp['xy'] = p
 chans = [ch] + [k for k in self.heard_pending() if k != ch
 and self.recs[k].n_bearings == 1 and self._cross_ok(self.recs[k],*p)]
 n = self.scan_at(*p, chans)
 self.note_pos()
 return n
 return super().visit(wp)

def mark_served(self, x, y):
 # The actual channel observations, not proximity, certify coverage.
 pass

def clear_target(self, ch, rec):
 for _ in range(self.cfg.homing_max_steps):
 if rec.cleared or self.aborted or not self.can_afford(15):
 break
 br = self.region_info(rec)
 if br is None:
 break
 c, radius = br['center'], br['radius']
 if radius <= R_CLEAR or (rec.n_bearings >= 2 and radius < 65 and math.dist(self.pos,c)<100):
 if self.do_clear(*c, ch):
 return True
 p = self._home_step(ch)
 if not self.can_afford(math.dist(self.pos,p)/5+12):
 break
 resp = self.measure(*p,ch)
 if resp.get('measure_result') == 'near':
 return self.do_clear(*p,ch)
 if resp.get('measure_result') == 'no_signal':
 # Retry toward the last positive position; never infer absence.
 S=np.array(rec.pts[-1]); p=(np.array(p)+S)/2
 resp = self.measure(*p,ch)
 if resp.get('measure_result') == 'near':
 return self.do_clear(*p,ch)
 if not rec.cleared:
 return super().clear_target(ch,rec)

```

```
return True
```

## 5.9 src/p3\_frontier.py

```
"""Experimental receding-horizon covering-tour planner for B question 3.

Unlike TourRobot, this planner has no fixed ring stations or angular cursor.
At each decision it builds a route through known targets, then solves a greedy
covering-tour problem over the *remaining per-channel coverage gap*. Target
stops compete with temporary measurement stations, including their scan cost.
Only the first stop is executed; actual observations determine the next plan.

This module deliberately inherits the existing localization and clearance
procedure so routing changes can be compared separately. No truth is consulted.
The inherited completion predicate evaluates a 25 m grid; its report field
``complete_proof`` is not a continuous geometric coverage certificate.
"""

from __future__ import annotations

import argparse
import json
import math
from dataclasses import dataclass, fields, replace
from pathlib import Path

import numpy as np

from p3_arena import MockArena, T_MEASURE, T_SWITCH, V_ROBOT
from p3_bench import (LIVE1, LIVE2, HDR, arena_from, center_cluster, fmt_row,
 outer_annulus, run_one, summarize)
from p3_robot import load_base
from p3_tour import TourConfig, TourRobot

@dataclass
class FrontierConfig(TourConfig):
 candidate_radii: tuple = (900.0, 1100.0, 1300.0, 1500.0)
 candidate_angles: int = 72
 scan_cost_weight: float = 0.3
 travel_cost_floor_m: float = 100.0
 gain_power: float = 2.0
 target_first: bool = False
 target_cover_duty: bool = False
 route_mode: str = "tsp"
 locate_mode: str = "never"
 offroute_clear: bool = False
 enroute_spacing_m: float = 0.0

class FrontierRobot(TourRobot):
 """Dynamic set cover with cheapest insertion and unconstrained 2-opt."""

 def __init__(self, arena, cfg=None, base=None, log=None):
 super().__init__(arena, cfg or FrontierConfig(), base=base, log=log)
 self._frontier_candidates = None
 self._frontier_cover = None

 def pick_direction(self):
 # Retained only to satisfy the inherited loop; no angular restriction.
 self.ring_dir = 1.0
```

```

def mark_served(self, x, y):
 # Coverage is recomputed from real observations, never by proximity.
 return None

def forward_ring(self, dir_sign, cursor=None):
 # Stale-localization mode is intentionally disabled in this experiment.
 return []

def chans_at(self, x, y, ch=None, cover_duty=False):
 out = []
 for c in self.pending():
 rec = self.recs[c]
 if rec.n_bearings == 0:
 if (cover_duty or ch is None) and self.adds_coverage(c, x, y):
 out.append(c)
 elif rec.n_bearings == 1 and self._cross_ok(rec, x, y):
 out.append(c)
 if ch is not None and c == int(ch) and c not in out:
 out.append(c)
 if ch is not None and int(ch) in out:
 out.remove(int(ch))
 out.insert(0, int(ch))
 return out

def _candidate_cover(self):
 if self._frontier_candidates is None:
 angles = np.arange(self.cfg.candidate_angles) * (2 * math.pi /
 self.cfg.candidate_angles)
 self._frontier_candidates = np.asarray([
 (r * math.cos(a), r * math.sin(a))
 for r in self.cfg.candidate_radii for a in angles])
 gx, gy = self.annulus_grid()
 self._frontier_cover = ((self._frontier_candidates[:, 0, None] - gx) ** 2
 + (self._frontier_candidates[:, 1, None] - gy) ** 2
 <= self.cfg.cover_radius_m ** 2)
 return self._frontier_candidates, self._frontier_cover

def _insert_cost(self, pts, seq):
 """Cost and index for inserting every candidate into the open route."""
 prev = np.asarray([self.pos] + [w["xy"] for w in seq], float)
 d = np.linalg.norm(pts[:, None, :] - prev[None, :, :], axis=2)
 cost = d.copy()
 if seq:
 cost[:, :-1] += d[:, 1:] - np.linalg.norm(np.diff(prev, axis=0), axis=1)
 ind = np.argmin(cost, axis=1)
 return cost[np.arange(len(pts)), ind], ind

def assemble_route(self, dir_sign, targets=None, cursor=None):
 seq = [dict(w, cover_duty=False) for w in
 (self.target_waypoints() if targets is None else targets)]
 seq = self.nn_2opt(seq)
 lattice, lattice_cover = self._candidate_cover()
 gx, gy, unsafe = self.uncovered_mask()
 if not unsafe.any():
 return seq, self._route_length(seq)

 if self.cfg.enroute_spacing_m > 0.0 and seq:
 path = [self.pos] + [w["xy"] for w in seq]
 extras = []
 for a, b in zip(path, path[1:]):
 parts = int(math.ceil(math.dist(a, b) / self.cfg.enroute_spacing_m))

```

```

 extras.extend(((1-t)*a[0] + t*b[0], (1-t)*a[1] + t*b[1])
 for t in np.arange(1, parts) / parts)
 if extras:
 extra_pts = np.asarray(extras, float)
 extra_cover = ((extra_pts[:, 0, None] - gx) ** 2
 + (extra_pts[:, 1, None] - gy) ** 2
 <= self.cfg.cover_radius_m ** 2)
 lattice = np.concatenate((lattice, extra_pts), axis=0)
 lattice_cover = np.concatenate((lattice_cover, extra_cover), axis=0)

Known targets are mandatory route stops but their coverage scan is
optional. The set-cover objective decides which should perform it.
targets_copy = list(seq)
if targets_copy:
 target_pts = np.asarray([w["xy"] for w in targets_copy], float)
 target_cover = ((target_pts[:, 0, None] - gx) ** 2
 + (target_pts[:, 1, None] - gy) ** 2
 <= self.cfg.cover_radius_m ** 2)
 pts = np.concatenate((lattice, target_pts), axis=0)
 covers = np.concatenate((lattice_cover, target_cover), axis=0)
else:
 pts, covers = lattice, lattice_cover
available = np.ones(len(pts), bool)
n_lattice = len(lattice)
scan_cost = (T_MEASURE + T_SWITCH) * max(1, len(self.unheard())) * V_ROBOT
scan_cost *= self.cfg.scan_cost_weight

Retain the inherited grid coverage requirement, without early stopping.
The score decides route shape only.
for _ in range(30):
 if not unsafe.any():
 break
 gains = np.count_nonzero(covers[:, unsafe], axis=1).astype(float)
 costs, inserts = self._insert_cost(pts, seq)
 costs[n_lattice:] = 0.0
 score = gains ** self.cfg.gain_power / (scan_cost +
 self.cfg.travel_cost_floor_m + np.maximum(0.0, costs))
 score[~available] = -1.0
 best = int(np.argmax(score))
 if gains[best] == 0:
 break
 available[best] = False
 if best >= n_lattice:
 targets_copy[best - n_lattice]["cover_duty"] = True
 else:
 seq.insert(int(inserts[best]), {"kind": "cover", "ch": None,
 "xy": tuple(pts[best]), "r": None, "cover_duty": True})
 unsafe = unsafe & ~covers[best]

Reorder the joint route without the original angular constraints.
seq = self.nn_2opt(seq)
if self.cfg.target_first and targets_copy:
 nearest = min(targets_copy, key=lambda w: math.dist(self.pos, w["xy"]))
 seq.remove(nearest)
 seq.insert(0, nearest)
return seq, self._route_length(seq)

def _route_length(self, seq):
 pts = [self.pos] + [w["xy"] for w in seq]
 return sum(math.dist(a, b) for a, b in zip(pts, pts[1:]))

```

```

class OriginalTourRobot(TourRobot):
 """Freeze the user's original singleton measurement behavior for controls."""

 def _singleton_reachable(self, rec, x, y):
 return True

def original_config():
 vals = dict(cover_rings=((1000.0, 8),), route_mode="polish",
 locate_mode="stale", offroute_clear=True, target_cover_duty=True)
 return TourConfig(**{k: v for k, v in vals.items() if k in
 {f.name for f in fields(TourConfig)}})

def main():
 ap = argparse.ArgumentParser(description=__doc__)
 ap.add_argument("--cases", type=int, default=5)
 ap.add_argument("--seed", type=int, default=20260913)
 ap.add_argument("--step", type=int, default=100)
 ap.add_argument("---scenarios", default="random,annulus,center,live1,live2")
 ap.add_argument("---variants", default="original,tour,frontier")
 ap.add_argument("--cfg", default="{}")
 ap.add_argument("--json-out", default="")
 args = ap.parse_args()
 makers = {"random": lambda s: MockArena(seed=s), "annulus": outer_annulus,
 "center": center_cluster, "live1": lambda s: arena_from(LIVE1, seed=s),
 "live2": lambda s: arena_from(LIVE2, seed=s)}
 cfg = replace(FrontierConfig(), **json.loads(args.cfg))
 variants = {"original": (OriginalTourRobot, original_config()),
 "tour": (TourRobot, TourConfig(cover_rings=((1050.0, 7),),
 route_mode="tsp", locate_mode="never", offroute_clear=False)),
 "frontier": (FrontierRobot, cfg)}
 base = load_base()
 output = {"arguments": vars(args), "results": {}}
 for scenario in args.scenarios.split(","):
 print(scenario, flush=True)
 print(HDR, flush=True)
 for name in args.variants.split(","):
 cls, config = variants[name]
 reps = [run_one(cls, config, makers[scenario](args.seed + i * args.step),
 base, want_path=True) for i in range(args.cases)]
 summary = summarize(reps)
 summary["complete_proof_cases"] = sum(bool(r["complete_proof"]) for r in reps)
 print(fmt_row(name, summary) + " proof=" + str(summary["complete_proof_cases"]),
 flush=True)
 output["results"][scenario + "/" + name] = {**summary,
 "per_case": [{k: v for k, v in r.items() if not k.startswith("_")
 and k not in ("truth", "channels", "mock_stats")} for r in reps]}
 if args.json_out:
 p = Path(args.json_out)
 p.parent.mkdir(parents=True, exist_ok=True)
 p.write_text(json.dumps(output, ensure_ascii=False, indent=2, default=str),
 encoding="utf-8")

if __name__ == "__main__":
 main()

```

## 5.10 src/p3\_coverage.py

```

"""Conservative continuous-disk coverage from actual no-signal replies.

Closed square cells tile $[-1800,1800]^2$, including every cell intersecting
the arena disk. A center covered at radius $1000 - \text{step}/\sqrt{2} - \text{epsilon}$
certifies the entire cell by the triangle inequality. No positive/near
reply is ever used to certify absence. This may over-cover boundary cells.
"""

import math
import numpy as np
from p3_arena import R_ARENA, R_RECV_MIN

class CertifiedCoverageMixin:
 def annulus_grid(self):
 if self._ag is None:
 requested = float(self.cfg.cover_grid_step_m)
 if requested <= 0:
 raise ValueError('cover_grid_step_m must be positive')
 n = int(math.ceil(2 * R_ARENA / requested))
 step = 2 * R_ARENA / n
 xs = -R_ARENA + (np.arange(n)+0.5)*step
 gx, gy = np.meshgrid(xs,xs)
 intersects = (np.maximum(np.abs(gx)-step/2,0)**2 +
 np.maximum(np.abs(gy)-step/2,0)**2 <= R_ARENA**2+1e-6)
 self._ag = (gx[intersects],gy[intersects])
 self._cert_radius = R_RECV_MIN-step/math.sqrt(2)-1e-6
 if self._cert_radius <= 0:
 raise ValueError('coverage cells are too large')
 return self._ag

 def certified_radius(self):
 self.annulus_grid()
 return self._cert_radius

 def unsafe_of(self,ch):
 rec = self.recs[int(ch)]
 key = (int(ch),len(rec.meas_log))
 if key not in self._unsafe_cache:
 gx,gy = self.annulus_grid()
 out = np.ones(len(gx),dtype=bool)
 r2 = self.certified_radius()**2
 for x,y,t,result in rec.meas_log:
 if result == 'no_signal':
 out &= (gx-x)**2+(gy-y)**2 > r2
 self._unsafe_cache[key] = out
 return self._unsafe_cache[key]

 def heard_pending(self):
 return [int(ch) for ch,r in self.recs.items()
 if not r.cleared and (r.n_bearings or r.near_hits)]

 def unheard(self):
 return [int(ch) for ch,r in self.recs.items()
 if not r.cleared and not r.n_bearings and not r.near_hits]

 def adds_coverage(self,ch,x,y):
 gx,gy = self.annulus_grid()
 return bool(np.any(self.unsafe_of(ch) &
 ((gx-x)**2+(gy-y)**2 <= self.certified_radius()**2)))

 def adds_any_coverage(self,x,y):
 return any(self.adds_coverage(ch,x,y) for ch in self.unheard())

```

```

def is_complete(self):
 return (not self.heard_pending() and
 all(not self.unsafe_of(ch).any() for ch in self.unheard()))

```

## 5.11 src/p3\_adaptive.py

```

"""Experimental combination of homing and dynamic covering-tour planning."""
from dataclasses import dataclass
import math
import numpy as np
from p3_homing import HomingConfig, HomingRobot
from p3_frontier import FrontierConfig, FrontierRobot
from p3_coverage import CertifiedCoverageMixin

@dataclass
class AdaptiveConfig(HomingConfig, FrontierConfig):
 locate_mode: str = 'never'
 quick_baseline_m: float = 400.0
 cover_grid_step_m: float = 50.0

class AdaptiveRobot(CertifiedCoverageMixin, HomingRobot, FrontierRobot):
 def __init__(self, arena, cfg=None, base=None, log=None):
 super().__init__(arena, cfg or AdaptiveConfig(), base=base, log=log)
 # The route's candidate masks must use the same conservative radius.
 from dataclasses import replace
 self.cfg = replace(self.cfg, cover_radius_m=self.certified_radius(), cover_slack_m=0.0)

 def initial_stage(self):
 if self.cfg.quick_baseline_m <= 0:
 return
 pending = self.heard_pending()
 if not pending:
 return
 angles = np.arange(72)*2*math.pi/72
 bearings = np.radians([self.recs[c].svds[0] for c in pending])
 # One short shared transverse baseline, then approach while measuring.
 score = np.sum(np.sin(angles[:,None]-bearings)**2,axis=1)
 a = angles[int(np.argmax(score))]
 p = self.cfg.quick_baseline_m*np.array([math.cos(a),math.sin(a)])
 self.scan_at(*p,pending)
 self.note_pos()
 self.stop_seq += 1

```

## 5.12 src/p3\_joint.py

```

"""Q3 joint routing and scan scheduling with continuous coverage certificates.

Only actual replies enter decisions. Probability gates affect optional bearing
measurements, never mandatory clearance or the deterministic completion proof.
"""
from dataclasses import dataclass
import math
import numpy as np
from p3_adaptive import AdaptiveRobot, AdaptiveConfig

```

```

@dataclass
class JointConfig(AdaptiveConfig):
 after_service_scan: str = 'duty'
 reachable_filter: bool = True
 polish_small_routes: bool = True
 candidate_radii: tuple = (1050.,1200.,1350.,1500.)
 speculative_radius_m: float = 400.0
 cover_polish: bool = True
 planner_multistart: bool = True
 opportunistic_receive_probability: float = 0.5
 continuous_cover: bool = True
 probe_fail_measure: bool = True
 min_move: float = 0.0

class JointRobot(AdaptiveRobot):
 def __init__(self, arena, cfg=None, base=None, log=None):
 super().__init__(arena, cfg or JointConfig(), base=base, log=log)

 def clear_target(self, ch, rec):
 if self.cfg.speculative_radius_m and rec.n_bearings>=2 and not rec.cleared:
 br=self.region_info(rec)
 if br and br['radius'] <= self.cfg.speculative_radius_m:
 cost=math.dist(self.pos,br['center'])/5+15
 if self.can_afford(cost):
 if self.do_clear(*br['center'],ch):
 return True
 if self.cfg.probe_fail_measure and self.can_afford(8):
 resp=self.measure(*self.pos,ch)
 if resp.get('measure_result')=='near':
 return self.do_clear(*self.pos,ch)
 return super().clear_target(ch,rec)

 def nn_2opt(self, seq):
 if self.cfg.polish_small_routes and len(seq)<4:
 from itertools import permutations
 return list(min(permutations(seq),key=self._route_length)) if seq else []
 return super().nn_2opt(seq)

 def assemble_route(self, dir_sign, targets=None, cursor=None):
 if self.cfg.planner_multistart:
 from dataclasses import replace
 cfg=self.cfg
 proposals=[]
 try:
 for power,weight in ((1.,.3),(1.,1.), (2.,.3),(2.,1.), (3.,.3)):
 self.cfg=replace(cfg,planner_multistart=False,gain_power=power,scan_cost_weight=weight)
 seq,length=self.assemble_route(dir_sign,targets=targets,cursor=cursor)
 proposals.append((self._plan_cost(seq),seq,length))
 finally:
 self.cfg=cfg
 _cost,seq,length=min(proposals,key=lambda q:q[0])
 return seq,length
 seq,length=super().assemble_route(dir_sign,targets=targets,cursor=cursor)
 if not self.cfg.cover_polish or not seq:
 return seq,length
 pts,covers=self._candidate_cover()
 gx,gy,needed=self.uncovered_mask()
 if not needed.any():

```



```

 return seq,length
r2=self.cfg.cover_radius_m**2
for _ in range(3):
 changed=False
 for w in list(seq):
 if not w.get('cover_duty'):
 continue
 others=np.zeros(len(gx),bool)
 for v in seq:
 if v is not w and v.get('cover_duty'):
 x,y=v['xy'];others |= (gx-x)**2+(gy-y)**2<=r2
 must=needed&~others
 if not must.any():
 if w['kind']=='cover':
 seq.remove(w)
 else:
 w['cover_duty']=False
 changed=True
 continue
 if w['kind']!='cover':
 continue
 allowed=np.all(covers[:,must],axis=1)
 if not allowed.any():
 continue
 reduced=[v for v in seq if v is not w]
 costs,places=self._insert_cost(pts,reduced)
 costs[-allowed]=np.inf
 j=int(np.argmin(costs))
 old_cost=self._route_length(seq)-self._route_length(reduced)
 if costs[j] < old_cost-1e-5:
 w['xy']=tuple(pts[j]);reduced.insert(int(places[j]),w)
 seq=reduced;changed=True
 if self.cfg.continuous_cover:
 reduced=[v for v in seq if v is not w]
 start=np.asarray(w['xy'],float)
 Q=np.column_stack([gx[must],gy[must]])
 delta=start-Q
 c=np.einsum('ij,ij->i',delta,delta)-r2
 if c.max(initial=-np.inf)>1e-5:
 continue
 chain=np.array([self.pos]+[v['xy'] for v in reduced])
 destinations=[chain[-1]]
 for a,b in zip(chain,chain[1:]):
 v=b-a;den=v@v
 t=float(np.clip((start-a)@v/max(den,1e-12),0,1))
 destinations.append(a+t*v)
 candidates=[]
 for z in destinations:
 v=z-start;aa=float(v@v)
 if aa<1e-10:continue
 bb=2*delta@v
 roots=(-bb+np.sqrt(np.maximum(0,bb*bb-4*aa*c)))/(2*aa)
 t=max(0.,min(1.,float(roots.min())))-1e-8)
 candidates.append(start+t*v)
 if candidates:
 P=np.asarray(candidates)
 costs,places=self._insert_cost(P,reduced)
 j=int(np.argmin(costs))
 old_cost=self._route_length(seq)-self._route_length(reduced)
 if costs[j]<old_cost-1e-5:
 w['xy']=tuple(P[j]);reduced.insert(int(places[j]),w)
 seq=reduced;changed=True

```

```

 seq=self.nn_2opt(seq)
 if not changed:
 break
 return seq,self._route_length(seq)

def _plan_cost(self,seq):
 gx,gy=self.annulus_grid();r2=self.cfg.cover_radius_m**2
 pending={ch:self.unsafe_of(ch).copy() for ch in self.unheard()}
 scans=0
 for w in seq:
 if not w.get('cover_duty'):
 continue
 x,y=w['xy'];mask=(gx-x)**2+(gy-y)**2<=r2
 for ch,m in pending.items():
 if np.any(m&mask):
 scans+=1;pending[ch]=m&~mask
 return self._route_length(seq)/5+6*scans

def _cross_ok(self,rec,x,y):
 if self.cfg.opportunistic_receive_probability>0 and rec.n_bearings==1:
 s=np.array(rec.pts[0]);a=math.radians(rec.svds[0])
 t=np.linspace(5.,1500.,151);u=np.array([math.cos(a),math.sin(a)])
 p=s+t[:,None]*u
 lo=np.maximum(1000.,t);hi=np.full(len(t),1500.)
 for qx,qy,_t,res in rec.meas_log:
 if res=='no_signal':hi=np.minimum(hi,np.linalg.norm(p-(qx,qy),axis=1))
 density=t*(np.linalg.norm(p,axis=1)<=1800.)
 total=np.sum(density*np.maximum(0.,hi-lo))
 heard=np.sum(density*np.maximum(0.,hi-np.maximum(lo,np.linalg.norm(p-(x,y),axis=1))))
 if total>0 and heard/total < self.cfg.opportunistic_receive_probability:
 return False
 if self.cfg.reachable_filter and rec.n_bearings==1:
 br=self.region_info(rec)
 if br:
 poly=br['poly']
 from p3_robot import _dist_point_polygon
 if poly and _dist_point_polygon(np.array([x,y]),np.array(poly))>1500.:
 return False
 return super()._cross_ok(rec,x,y)

def visit(self,wp):
 ch=wp.get('ch')
 if ch is None or self.cfg.after_service_scan=='off':
 return super().visit(wp)
 rec=self.recs[ch]
 before=self.stats['n_measure']
 self.clear_target(ch,rec)
 # The route's target was an estimate; perform its scan at the actual
 # endpoint after localization, then replan using that measured position.
 wp['xy']=tuple(self.pos)
 chans=[]
 duty=wp.get('cover_duty') or self.cfg.after_service_scan=='all'
 if self.cfg.after_service_scan=='replan':
 here=dict(kind='scan_here',ch=None,xy=tuple(self.pos),r=None)
 route,_=self.assemble_route(1.,targets=[here]+self.target_waypoints())
 duty=any(w['kind']=='scan_here' and w.get('cover_duty') for w in route)
 for k in self.pending():
 r=self.recs[k]
 if not r.n_bearings and duty and self.adds_coverage(k,*self.pos):
 chans.append(k)
 elif r.n_bearings==1 and self._cross_ok(r,*self.pos):
 chans.append(k)

```

```

 if chans:
 self.scan_at(*self.pos,chans)
 self.note_pos()
 # The inherited loop also accepts a successful clear as progress.
 return max(1,self.stats['n_measure']-before)

```

## 5.13 src/p3\_bench.py

"""问题 3 改进用的批量对照台（不进入正式交付，只服务于调参与回归）。

用法示例::

```

python src/p3_bench.py --cases 30 # 默认对照集
python src/p3_bench.py --cases 30 --set onlap # 指定对照集
python src/p3_bench.py --cases 10 --only A,onlap # 只跑其中几项

```

输出：一行一配置的汇总表（清除率 / 平均定位清除时间 / 虚拟时间 / 检测 / 行程 / 折返指标），并可选写出 JSON 供后续比对。

```

"""
from __future__ import annotations

import argparse
import json
import math
import os
import statistics as st
import sys
import time
from dataclasses import replace

import numpy as np

_HERE = os.path.dirname(os.path.abspath(__file__))
sys.path.insert(0, _HERE)
_ROOT = os.path.normpath(os.path.join(_HERE, ".."))

Windows 控制台默认 GBK：中文与表格符号混排时容易抛 UnicodeEncodeError
for _s in (sys.stdout, sys.stderr):
 try:
 _s.reconfigure(encoding="utf-8", errors="replace")
 except Exception:
 pass

from p3_arena import MockArena, MockSource, R_ARENA, V_ROBOT # noqa: E402
from p3_robot import P3Config, P3Robot, load_base # noqa: E402
from p3_sweep import SweepConfig, SweepRobot # noqa: E402

LIVE1 = [(2, 937, -782), (5, 363, 418), (6, -939, -1491), (9, 1652, -112),
 (10, 1606, -590), (11, -591, -82), (13, 18, -197), (14, 470, -1292),
 (18, 1024, 1242), (20, -316, 241)]
LIVE2 = [(1, 693, -1157), (2, 722, 1401), (4, 288, 1723), (6, 622, 86),
 (8, 1066, 1008), (10, 1060, -550), (11, -386, 628), (14, 1378, 265),
 (15, 56, 734), (16, -722, -719), (17, 279, 945), (18, 1746, -247),
 (19, -509, 1384), (20, 498, 769)]

def arena_from(sources, r_recv=1400.0, seed=0, budget=360000.0):
 return MockArena(seed=seed, budget_s=budget,
 sources=[MockSource(c, x, y, r_recv) for (c, x, y) in sources])

```

```

def outer_annulus(seed=0, n=12, r_lo=1300.0, r_hi=1800.0, budget=360000.0):
 rng = np.random.default_rng(seed)
 chans = rng.choice(np.arange(1, 21), size=n, replace=False)
 out = []
 for ch in chans:
 r = math.sqrt(rng.uniform(r_lo ** 2, r_hi ** 2))
 a = rng.uniform(0, 2 * math.pi)
 out.append(MockSource(int(ch), r * math.cos(a), r * math.sin(a),
 float(rng.uniform(1000.0, 1500.0))))
 return MockArena(seed=seed, sources=out, budget_s=budget)

def center_cluster(seed=0, n=12, budget=360000.0):
 out = []
 rng = np.random.default_rng(seed)
 for c in range(1, n + 1):
 r = rng.uniform(0, 260.0)
 a = rng.uniform(0, 2 * math.pi)
 out.append(MockSource(c, r * math.cos(a), r * math.sin(a),
 float(rng.uniform(1000.0, 1500.0))))
 return MockArena(seed=seed, sources=out, budget_s=budget)

def path_metrics(acts, robot=None):
 """路径形态指标: 折返 (相邻两段夹角 > 120°) 次数与折返累计距离。

 用户关心的"大量折返 / 绕远路"需要一个可量化的代理指标: 把动作序列看成折线,
 统计"转弯角 > 120° 且折返段长度 > 150 m"的段数 (记为 ``u_turns``) 以及这些段的
 总长度 (``u_m``)。纯绕圈 + 顺路清除的理想路径此值应接近 0。
 """
 pts = [(0.0, 0.0)] + [(a["x"], a["y"]) for a in acts
 if a.get("kind") in ("measure", "clear")
 and "x" in a and "y" in a]

 # 去掉原地重复点
 P = []
 for p in pts:
 if not P or math.dist(P[-1], p) > 1.0:
 P.append(p)
 u_turns, u_m, max_u = 0, 0.0, 0.0
 for i in range(1, len(P) - 1):
 a = np.asarray(P[i - 1], float)
 b = np.asarray(P[i], float)
 c = np.asarray(P[i + 1], float)
 v1, v2 = b - a, c - b
 n1, n2 = float(np.linalg.norm(v1)), float(np.linalg.norm(v2))
 if n1 < 30.0 or n2 < 30.0:
 continue
 cosang = float((v1 / n1) @ (v2 / n2))
 ang = math.degrees(math.acos(max(-1.0, min(1.0, cosang))))
 if ang > 120.0:
 u_turns += 1
 u_m += n2
 max_u = max(max_u, n2)
 return {"u_turns": u_turns, "u_m": u_m, "u_max_m": max_u, "n_seg": len(P) - 1}

def run_one(cls, cfg, arena, base, want_path=False):
 rb = cls(arena, cfg, base=base, log=None)
 t0 = time.time()
 rep = rb.run()

```

```

rep["_wall"] = time.time() - t0
st_ = getattr(arena, "stats", {}) or {}
rep["travel_m"] = float(st_.get("travel_m", 0.0))
if want_path:
 rep.update(path_metrics(getattr(arena, "actions", [])))
 rep["_actions"] = [(a["x"], a["y"], a.get("kind"), a.get("virtual_time_s"))
 for a in getattr(arena, "actions", [])
 if "x" in a and "y" in a]

return rep

def summarize(reps):
 cl = sum(r["cleared"] for r in reps)
 tot = sum(r["n_sources"] for r in reps)
 tm = [r["avg_clear_time_s"] for r in reps if r["avg_clear_time_s"]]
 idle = []
 for r in reps:
 last = (r["avg_clear_time_s"] or 0) * r["cleared"]
 idle.append((r["virtual_time_s"] - last) / max(r["virtual_time_s"], 1e-9))
 return {
 "clear_ratio": cl / tot if tot else 0.0,
 "cleared_mean": st.mean(r["cleared"] for r in reps),
 "full_clear_cases": sum(1 for r in reps if r["cleared"] == r["n_sources"]),
 "cases": len(reps),
 "avg_clear_time_s": st.mean(tm) if tm else 0.0,
 "vtime_s": st.mean(r["virtual_time_s"] for r in reps),
 "n_measure": st.mean(r["n_measure"] for r in reps),
 "travel_m": st.mean(r["travel_m"] for r in reps),
 "idle": st.mean(idle),
 "u_turns": st.mean(r.get("u_turns", 0) for r in reps),
 "u_m": st.mean(r.get("u_m", 0.0) for r in reps),
 "wall_s": st.mean(r["_wall"] for r in reps),
 "n_rejected": sum(r["n_rejected"] for r in reps),
 }

def fmt_row(name, s):
 return (f"{name:<26}{s['clear_ratio']:>8.3f}{s['cleared_mean']:>8.2f}"
 f"{s['full_clear_cases']:>6}/{s['cases']:<3}"
 f"{s['avg_clear_time_s']:>9.1f}{s['vtime_s']:>9.0f}"
 f"{s['n_measure']:>7.0f}{s['travel_m']:>9.0f}"
 f"{s['idle']:>8.1%}{s['u_turns']:>7.1f}{s['u_m']:>8.0f}"
 f"{s['wall_s']:>7.2f}")

HDR = (f"{'配置':<26}{ '清除率':>8}{ '平均清除':>8}{ '全清':>9}"
 f"{'定位清除/s':>9}{ '虚拟/s':>9}{ '检测':>7}{ '行程/m':>9}"
 f"{'空转':>8}{ '折返':>7}{ '折返/m':>8}{ '实跑':>7}")

def variant_sets(names=None, cfg_over=None):
 """候选配置。键为名字，值为 `` (robot_cls, cfg, extra_kwargs)``。"""
 from dataclasses import replace as _rep
 from p3_tour import TourConfig, TourRobot
 over = cfg_over or {}
 tc = _rep(TourConfig(), **over) if over else TourConfig()
 tag = "E_统一航路" + (" if not over else "(" + ", ".join(
 f"{k}={v}" for k, v in over.items() if k in ("cover_rings",)) + ")")
 A = ("A_基线(扫圈)", SweepRobot, SweepConfig(), {})
 B = ("B_边绕边清", SweepRobot,
 SweepConfig(onlap_clear=True), {})

```

```

C = ("C_边缘边清+开场定点", SweepRobot,
 SweepConfig(onlap_clear=True, initial_locate=True), {})
D = ("D_期望场贪心", P3Robot, P3Config(), {})
E = (tag, TourRobot, tc, {})
sets = {"base": [A], "onlap": [A, B, C, D], "all": [A, D, E],
 "tour": [E], "cmp": [A, D, E]}

if names:
 keys = [n.strip() for n in names.split(",") if n.strip()]
 out = []
 for k in keys:
 for item in sets.get(k, []):
 out.append(item)
 return out
return sets["all"]

def main():
 ap = argparse.ArgumentParser()
 ap.add_argument("--cases", type=int, default=30)
 ap.add_argument("--seed", type=int, default=20260913)
 ap.add_argument("--step", type=int, default=100)
 ap.add_argument("--scenario", default="random",
 choices=["random", "annulus", "live1", "live2", "center"])
 ap.add_argument("--set", default="all")
 ap.add_argument("--only", default="")
 ap.add_argument("--json-out", default="")
 ap.add_argument("--cfg", default="", help='TourConfig 覆盖 (JSON), 如 {"cover_rings": [[950,9]]}')
 args = ap.parse_args()

 base = load_base()
 seeds = [args.seed + args.step * i for i in range(args.cases)]

 def mk(seed):
 if args.scenario == "random":
 return MockArena(seed=seed)
 if args.scenario == "annulus":
 return outer_annulus(seed=seed)
 if args.scenario == "center":
 return center_cluster(seed=seed)
 if args.scenario == "live1":
 return arena_from(LIVE1, seed=seed)
 return arena_from(LIVE2, seed=seed)

 rows = variant_sets(args.set, cfg_over=(json.loads(args.cfg) if args.cfg else None))
 if args.only:
 keep = {k.strip() for k in args.only.split(",") if k.strip()}
 rows = [r for r in rows if r[0].split("_")[0] in keep or r[0] in keep]

 print(f"场景 {args.scenario}, {args.cases} 局 (seed {seeds[0]}..{seeds[-1]}) ")
 print(HDR)
 print("-" * len(HDR))
 store = {}
 for name, cls, cfg, kw in rows:
 reps = [run_one(cls, cfg, mk(s), base, want_path=True) for s in seeds]
 s = summarize(reps)
 store[name] = {**s, "per_case": [
 {k: v for k, v in r.items() if not k.startswith("_") and k != "truth"
 and k != "channels" and k != "mock_stats"} for r in reps]}
 print(fmt_row(name, s), flush=True)
 if args.json_out:
 os.makedirs(os.path.dirname(os.path.abspath(args.json_out)), exist_ok=True)
 with open(args.json_out, "w", encoding="utf-8") as f:

```

```

 json.dump(store, f, ensure_ascii=False, indent=2, default=str)
 print(f"\n-> {args.json_out}")

if __name__ == "__main__":
 main()

```

## 5.14 src/p3\_adaptive\_bench.py

```

"""Frozen paired benchmark; all policies receive fresh copies of each scene."""
from __future__ import annotations
import argparse
import csv
from dataclasses import asdict
import hashlib
import json
from pathlib import Path
import numpy as np
from p3_arena import MockArena
from p3_adaptive import AdaptiveConfig, AdaptiveRobot
from p3_tour import TourConfig, TourRobot
from p3_bench import LIVE1, LIVE2, arena_from, outer_annulus, center_cluster, run_one, load_base

class HashErrorArena(MockArena):
 """Different bounded error model: deterministic per location, not smooth."""
 def env_error(self, x, y):
 key=f'{self._local_seed}:{float(x).hex()}:{float(y).hex()}'.encode()
 value=int.from_bytes(hashlib.sha256(key).digest()[:8], 'little')/(2**64-1)
 return (2*value-1)*0.995

def make_scene(kind, seed):
 if kind=='annulus':
 return outer_annulus(seed)
 if kind=='center':
 return center_cluster(seed)
 if kind in ('live1', 'live2'):
 return arena_from(LIVE1 if kind=='live1' else LIVE2, seed=seed)
 if kind=='hash':
 return HashErrorArena(seed=seed)
 arena=MockArena(seed=seed)
 if kind=='worstrecv':
 for source in arena.sources:
 source.r_rcv=1000.0
 elif kind!='random':
 raise ValueError(kind)
 return arena

def statistics(rows):
 good=[r for r in rows if 'error' not in r]
 out={'cases':len(rows), 'paired_completed':len(good)}
 if not good:
 return out
 for name in ('tour', 'adaptive'):
 vals=[r[name] for r in good]
 times=[r['virtual_time_s'] for r in vals]
 out[name]={
 'mean_s':float(np.mean(times)),

```

```

 'p90_s':float(np.percentile(times,90)),
 'max_s':float(np.max(times)),
 'last_clear_mean_s':float(np.mean([r['last_clear_s'] for r in vals])),
 'last_clear_per_source_mean_s':float(np.mean([r['avg_clear_time_s'] or 0 for r in vals])),
 'exit_per_source_mean_s':float(np.mean([r['virtual_time_s']/max(r['cleared'],1) for r in vals])),
 'travel_mean_m':float(np.mean([r['travel_m'] for r in vals])),
 'measure_mean':float(np.mean([r['n_measure'] for r in vals])),
 'wall_mean_s':float(np.mean([r['_wall'] for r in vals])),
 'full_clear_cases':sum(r['cleared']==r['n_sources'] for r in vals),
 'cleared':sum(r['cleared'] for r in vals),
 'sources':sum(r['n_sources'] for r in vals),
 'complete_predicate_cases':sum(bool(r['complete_proof']) for r in vals),
 'rejections':sum(r['n_rejected'] for r in vals),
 }

 pairs=np.asarray([[r['tour']['virtual_time_s'],r['adaptive']['virtual_time_s']] for r in good])
 out['improvement_fraction']=float(1-pairs[:,1].mean()/pairs[:,0].mean())
 out['adaptive_faster_cases']=int(np.count_nonzero(pairs[:,1]<pairs[:,0]))
 sampled=np.random.default_rng(121212).integers(0,len(pairs),size=(4000,len(pairs)))
 means=pairs[sampled].mean(axis=1)
 out['paired_bootstrap_95pct']=np.percentile(1-means[:,1]/means[:,0],[2.5,97.5]).tolist()
 return out

def main():
 ap=argparse.ArgumentParser(description=__doc__)
 ap.add_argument('--counts',default='random:50,annulus:20,center:20,live1:15,live2:15,hash:20,worstrecv:20')
 ap.add_argument('--seed',type=int,default=777000)
 ap.add_argument('--step',type=int,default=37)
 ap.add_argument('--out',default=str(Path(__file__).resolve().parents[1] / 'out/p3_adaptive_validation'))
 args=ap.parse_args()
 outdir=Path(args.out);outdir.mkdir(parents=True,exist_ok=True)
 configs={'tour':TourConfig(),'adaptive':AdaptiveConfig()}
 classes={'tour':TourRobot,'adaptive':AdaptiveRobot}
 source_dir=Path(__file__).resolve().parent
 files=['p3_tour.py','p3_homing.py','p3_frontier.py','p3_coverage.py','p3_adaptive.py','p3_arena.py']
 result={'arguments':vars(args),'configs':{k:asdict(v) for k,v in configs.items()},
 'source_sha256':{f:hashlib.sha256((source_dir/f).read_bytes()).hexdigest() for f in files},
 'note':'live1/live2 are reconstructed coordinates with assumed reception radius 1400 m, not live runs
 ; tour complete_proof is only its original grid predicate.',
 'scenarios':{}}
 base=load_base()
 for part in args.counts.split(','):
 kind,count=part.split(':');rows=[]
 for i in range(int(count)):
 seed=args.seed+args.step*i
 row={'seed':seed}
 for name in ('tour','adaptive'):
 arena=make_scene(kind,seed)
 try:
 r=run_one(classes[name],configs[name],arena,base,want_path=True)
 r['last_clear_s']=(r['avg_clear_time_s'] or 0)*r['cleared']
 row[name]={k:v for k,v in r.items() if k not in ('truth','channels','mock_stats','_actions')}
 if i<3:
 trace={'scenario':kind,'seed':seed,'policy':name,'actions':arena.actions,'truth':arena.
 truth()}
 (outdir/f'{kind}_{seed}_{name}_trace.json').write_text(json.dumps(trace,ensure_ascii=
 False,indent=2,default=str),encoding='utf-8')
 except Exception as exc:
 row['error']=f'{name}: {type(exc).__name__}: {exc}'
 print(kind,seed,row['error'],flush=True)
 break
 rows.append(row)

```



```

 if (i+1)%10==0:
 print(f'{kind}: {i+1}/{count}',flush=True)
 summary=statistics(rows)
 result['scenarios'][kind]={'summary':summary,'rows':rows}
 (outdir/'paired_results.json').write_text(json.dumps(result,ensure_ascii=False,indent=2,default=str),
 encoding='utf-8')
 print(kind,json.dumps(summary,ensure_ascii=False),flush=True)
 with (outdir/'paired_results.csv').open('w',newline='',encoding='utf-8-sig') as f:
 w=csv.writer(f)
 w.writerow(['scenario','seed','tour_s','adaptive_s','saved_s','tour_cleared','adaptive_cleared','
 n_sources','adaptive_certified','error'])
 for kind,data in result['scenarios'].items():
 for row in data['rows']:
 if 'error' in row:
 w.writerow([kind,row['seed'],','','','','',row['error']]);continue
 a,b=row['tour'],row['adaptive']
 w.writerow([kind,row['seed'],a['virtual_time_s'],b['virtual_time_s'],a['virtual_time_s']-b[
 virtual_time_s'],a['cleared'],b['cleared'],a['n_sources'],b['complete_proof'],''])

if __name__=='__main__':
 main()

```

## 5.15 scripts/run\_p3\_paper.py

```

#!/usr/bin/env python3
"""Frozen Q3 paired experiment and independent, replay-only audits.

Default execution resumes missing runs; --audit-only never constructs a robot.
Changing this reporting script does not change the experiment definition or
invalidate completed runs. --smoke always writes to a separate smoke directory.
Only NumPy and the standard library are required. See out/paper_q3/README.md.
"""
from __future__ import annotations
import argparse
import ast
import csv
import hashlib
import json
import math
import platform
import shutil
import signal
import sys
import time
from collections import Counter, defaultdict
from dataclasses import asdict, replace
from pathlib import Path

sys.dont_write_bytecode = True
import numpy as np

HERE = Path(__file__).resolve()
Q3 = HERE.parents[1]
SRC = Q3 / 'src'
OUT_DEFAULT = Q3 / 'out/paper_q3'
sys.path.insert(0, str(SRC))
from p3_robot import P3Config, P3Robot, load_base
from p3_sweep import SweepConfig, SweepRobot
from p3_tour import TourConfig, TourRobot

```

```

from p3_adaptive import AdaptiveConfig, AdaptiveRobot
from p3_joint import JointConfig, JointRobot
from p3_adaptive_bench import make_scene

SCENARIOS = (('random', 50), ('annulus', 20), ('center', 10), ('hash', 20), ('worstrecv', 20))
POLICIES = (('field', P3Robot, P3Config), ('sweep', SweepRobot, SweepConfig),
 ('tour', TourRobot, TourConfig), ('adaptive', AdaptiveRobot, AdaptiveConfig),
 ('joint', JointRobot, JointConfig))
ABLATIONS = (('joint_no_after_service', {'after_service_scan': 'off'}),
 ('joint_single_plan', {'planner_multistart': False}),
 ('joint_no_cover_polish', {'cover_polish': False}),
 ('joint_no_probability_gate', {'opportunistic_receive_probability': 0.0}))
SCHEMA_VERSION = 'q3-paper-v1'
AUDIT_VERSION = 'q3-independent-audit-v2'
SEED_BASE = 2026091200
BOOTSTRAP_N = 4000
BOOTSTRAP_SEED = 20260912
TOL = 1e-7
R_ARENA, R_RECV_MIN = 1800.0, 1000.0
Literal physical rules for independent verification; never import simulator
timing constants, reported switch_s, or reported clocks as calculation inputs.
SPEED, MEASURE_S, SWITCH_S, CLEAR_OK_S, CLEAR_FAIL_S = 5.0, 5.0, 1.0, 5.0, 3.0
POLICY_CLASSES = {'P3Robot', 'SweepRobot', 'TourRobot', 'AdaptiveRobot', 'JointRobot',
 'HomingRobot', 'FrontierRobot', 'CertifiedCoverageMixin'}
POLICY_FILES = ('p3_robot.py', 'p3_sweep.py', 'p3_tour.py', 'p3_homing.py',
 'p3_frontier.py', 'p3_coverage.py', 'p3_adaptive.py', 'p3_joint.py')

def clean(v):
 if isinstance(v, np.ndarray): return clean(v.tolist())
 if isinstance(v, np.generic): return clean(v.item())
 if isinstance(v, dict): return {str(k): clean(x) for k, x in v.items()}
 if isinstance(v, (list, tuple)): return [clean(x) for x in v]
 if isinstance(v, float) and not math.isfinite(v): return None
 return v

def digest(obj):
 return hashlib.sha256(json.dumps(clean(obj), ensure_ascii=False, sort_keys=True,
 separators=(',', ':'), allow_nan=False).encode()).hexdigest()

def sha256(path): return hashlib.sha256(Path(path).read_bytes()).hexdigest()

def atomic_json(path, obj):
 path = Path(path); path.parent.mkdir(parents=True, exist_ok=True)
 tmp = path.with_suffix(path.suffix + '.tmp')
 tmp.write_text(json.dumps(clean(obj), ensure_ascii=False, indent=2, allow_nan=False), encoding='utf-8')
 tmp.replace(path)

def read_json(path): return json.loads(Path(path).read_text(encoding='utf-8'))

def frozen_inputs():
 source = {str(p.relative_to(Q3)): sha256(p) for p in sorted(SRC.glob('*.py'))}
 # Obsolete scratch entry point was never imported by the paper runner.
 source.pop('src/run_p3_paper.py', None)
 files = sorted((Q3/'out/p3_family').glob('*.csv'))
 files += [Q3/'out/p3_family/p2_grid_family.json', Q3/'out/p2_grid/p2_grid_map.csv']
 fields = {str(p.relative_to(Q3)): sha256(p) for p in files if p.exists()}

```

```

configs = {name: asdict(cfg()) for name, _, cfg in POLICIES}
configs.update({name: asdict(replace(JointConfig(), **override)) for name, override in ABLATIONS})
return source, fields, clean(configs)

def make_manifest(prior=None):
 source, fields, configs = frozen_inputs()
 definition = {'source_sha256': source, 'field_sha256': fields, 'configs': configs,
 'scenarios': SCENARIOS, 'seed_base': SEED_BASE, 'seed_step': 97,
 'scenario_seed_stride': 100000, 'virtual_limit_s': 360000.0,
 'ablation_random_cases': 20}
 fingerprint = digest(definition)
 if prior:
 # Compare all previously frozen algorithm/field hashes before touching
 # outputs. Runner/reporting changes are deliberately independent.
 combined = {**source, **fields}
 for path, expected in prior.get('source_sha256', {}).items():
 if path in ('scripts/run_p3_paper.py', 'src/run_p3_paper.py'): continue
 if combined.get(path) != expected:
 raise RuntimeError(f'Frozen algorithm/field changed: {path}; use a new --out directory')
 for path, expected in prior.get('field_sha256', {}).items():
 if fields.get(path) != expected: raise RuntimeError(f'Frozen field changed: {path}')
 if prior.get('configs') != configs: raise RuntimeError('Frozen requested configurations changed')
 if prior.get('definition_fingerprint') not in (None, fingerprint):
 raise RuntimeError('Frozen experiment definition changed; use a new --out directory')
 return {'schema_version': SCHEMA_VERSION, 'experiment_id': (prior or {}).get('experiment_id', fingerprint
[:20]),
 'definition_fingerprint': fingerprint, 'definition': definition,
 'source_sha256': source, 'field_sha256': fields, 'configs': configs,
 'seed_formula': '2026091200 + 100000*scenario_index + 97*i',
 'scenarios': clean(SCENARIOS), 'policies': [p[0] for p in POLICIES],
 'ablations': dict(ABLATIONS), 'python_at_audit': platform.python_version(),
 'numpy_at_audit': np.__version__, 'platform_at_audit': platform.platform(),
 'execution_runtime_recorded': (prior or {}).get('execution_runtime_recorded', {k: (prior or {}).get(k
) for k in ('python', 'numpy')}),
 'audit_version': AUDIT_VERSION, 'audit_runner_sha256': sha256(HERE),
 'bootstrap': {'resamples': BOOTSTRAP_N, 'rng_seed': BOOTSTRAP_SEED,
 'statistic': '1 - mean(comparison) / mean(reference)',
 'sampling': 'paired scene indices, percentile interval; fresh seeded generator per
statistic'},
 'physical_rules': {'start_xy': [0, 0], 'initial_measuring_channel': 1,
 'speed_m_s': SPEED, 'measure_s': MEASURE_S, 'switch_s': SWITCH_S,
 'clear_success_s': CLEAR_OK_S, 'clear_failure_s': CLEAR_FAIL_S,
 'virtual_limit_s': 360000, 'clear_does_not_change_measuring_channel': True},
 'offline_real_time_policy': {'allowance_s': 1200,
 'historical_runs': 'Recorded policy-construction + run wall time; no real-time watchdog was
installed. Virtual limit was 360000 s.',
 'future_runs': '1200 s POSIX timer around construction + run; timeouts retained as error rows.',
 'limitations': 'Wall time is local diagnostic timing, excludes source generation/base-map loading
and independent audit; host load is uncontrolled. No formal online evaluation.'},
 'provenance_note': 'Algorithm/field hashes are checked against frozen records. Reporting-script
revisions do not rerun algorithms. Earlier runner-hash metadata had been refreshed after reporting edits, so
it is not treated as a byte-exact execution-script snapshot.',
 'primary_metric': 'complete exit_s, reported as mean and P90; all status values retained',
 'per_source_metric': 'mean across cases of exit_s / cleared; undefined when cleared=0; defined-value
count disclosed',
 'coverage_note': '5 m sampled-disk audit is numerical evidence, not a continuous proof. The 50 m
square-cell certificate is separately reconstructed from no_signal actions.'}

def scene_manifest(arena):
 sources = [{k: getattr(s, k) for k in ('channel', 'x', 'y', 'r_recv', 'cone_half', 'dir_deg')} for s in arena

```

```

.sources]
error = {'class': type(arena).__name__, 'waves': arena._waves, 'scale': arena._scale,
 'local_w': arena._local_w, 'local_seed': arena._local_seed,
 'probes': {f'{x},{y}': float(arena.env_error(x, y)) for x, y in
 ((0., 0.), (500., -500.), (-500., 500.), (1200., 0.), (0., -1200.))}}
scene = {'class': type(arena).__name__, 'seed': int(arena.seed), 'r_arena': float(arena.r_arena), 'sources':
 sources}
return {'class': type(arena).__name__, 'seed': int(arena.seed), 'sources': sources,
 'scene_fingerprint': digest(scene), 'error_field': error, 'error_fingerprint': digest(error)}

class TruthGuard:
 def __init__(self, arena):
 self.arena = arena; self.exit_called = False; self.truth_calls = []
 self._truth, self._exit = arena.truth, arena.exit
 arena.truth, arena.exit = self.truth, self.exit
 def truth(self):
 self.truth_calls.append({'virtual_time_s': float(self.arena.virtual_time_s), 'exit_called': self.
 exit_called})
 if not self.exit_called: raise RuntimeError('truth() called before accepted exit()')
 return self._truth()
 def exit(self):
 result = self._exit()
 if result.get('accepted') is True: self.exit_called = True
 return result

def action_audit(actions, report=None, truth=None, scene=None):
 position, measuring_channel, elapsed, travel = (0., 0.), 1, 0., 0.
 counts = Counter(); records = []; errors = []; maxima = Counter(); last_clear = None
 cleared_channels = set(); entered = False; exited = False
 def compare(label, observed, expected, index=None):
 if observed is None:
 errors.append({'check': label, 'index': index, 'reason': 'missing value'}); return None
 delta = float(observed) - expected
 maxima[label] = max(maxima[label], abs(delta))
 if not math.isfinite(delta) or abs(delta) > TOL:
 errors.append({'check': label, 'index': index, 'observed': observed, 'expected': expected, 'error':
 delta})
 return delta
 for i, a in enumerate(actions):
 kind = a.get('kind'); before_channel = measuring_channel
 rec = {'index': i, 'kind': kind, 'independent_before_s': elapsed,
 'measuring_channel_before': before_channel}
 if kind == 'enter':
 if entered or exited: errors.append({'check': 'enter order', 'index': i})
 entered = True; counts['enter'] += 1
 elif kind == 'exit':
 if not entered or exited: errors.append({'check': 'exit order', 'index': i})
 exited = True; counts['exit'] += 1
 elif kind in ('measure', 'clear'):
 if not entered or exited: errors.append({'check': 'action outside session', 'index': i})
 xy = (float(a['x']), float(a['y'])); distance = math.dist(position, xy)
 channel = int(a['channel']); switch = 0.0
 if not 1 <= channel <= 20: errors.append({'check': 'channel range', 'index': i})
 if kind == 'measure':
 switch = SWITCH_S if channel != measuring_channel else 0.0
 measuring_channel = channel; service = MEASURE_S
 counts['measure'] += 1; counts['switch'] += int(switch > 0)
 counts[a.get('measure_result', 'invalid_measure_result')] += 1
 if a.get('measure_result') not in ('direction', 'near', 'no_signal'):
 errors.append({'check': 'measurement result', 'index': i})

```

```

 rec['switch_error_s'] = compare('switch', a.get('switch_s'), switch, i)
 else:
 ok = a.get('clear_result') == 'success'
 service = CLEAR_OK_S if ok else CLEAR_FAIL_S
 counts['clear_success' if ok else 'clear_failure'] += 1
 if a.get('clear_result') not in ('success', 'no_target_in_range'):
 errors.append({'check': 'clear result', 'index': i})
 if ok:
 if channel in cleared_channels: errors.append({'check': 'duplicate clear success', 'index': i
 })

 cleared_channels.add(channel)
 expected_dt = distance / SPEED + switch + service
 elapsed += expected_dt; travel += distance; position = xy
 if kind == 'clear' and a.get('clear_result') == 'success': last_clear = elapsed
 rec.update({'distance_m': distance, 'distance_error_m': compare('distance', a.get('dist_m'), distance
, i),
 'expected_switch_s': switch, 'expected_dt': expected_dt,
 'dt_error': compare('dt', a.get('dt'), expected_dt, i)})
 else: errors.append({'check': 'unknown action kind', 'index': i, 'kind': kind})
 rec.update({'expected_cumulative': elapsed, 'reported_time': a.get('virtual_time_s'),
 'cumulative_error': compare('cumulative', a.get('virtual_time_s'), elapsed, i),
 'measuring_channel_after': measuring_channel)})
 records.append(rec)
if not entered or not exited: errors.append({'check': 'missing enter or exit', 'entered': entered, 'exited':
exited})
pieces = {'travel_s': travel / SPEED, 'measure_s': MEASURE_S * counts['measure'],
 'switch_s': SWITCH_S * counts['switch'],
 'clear_s': CLEAR_OK_S * counts['clear_success'] + CLEAR_FAIL_S * counts['clear_failure']}
compare('cost_piece_sum', sum(pieces.values()), elapsed)
checks = {}
if report is not None:
 expected = {'virtual_time_s': elapsed, 'travel_m': travel, 'cleared': counts['clear_success'],
 'n_clear': counts['clear_success'], 'n_clear_fail': counts['clear_failure']}
 # Policy measurement count can include rejected calls; accepted action
 # count is authoritative and report n_rejected is disclosed separately.
 if report.get('n_rejected', 0) == 0: expected['n_measure'] = counts['measure']
 if scene: expected['n_sources'] = len(scene['sources'])
 if counts['clear_success']: expected['avg_clear_time_s'] = last_clear / counts['clear_success']
 for name, value in expected.items():
 checks[name] = compare('report.' + name, report.get(name), value)
 expected_stats = {'travel_m': travel, 'n_measure': counts['measure'],
 'n_clear': counts['clear_success'] + counts['clear_failure'],
 'n_clear_fail': counts['clear_failure'], 'n_direction': counts['direction'],
 'n_near': counts['near'], 'n_no_signal': counts['no_signal']}
 for name, value in expected_stats.items():
 if name in report.get('mock_stats', {}):
 checks['mock_stats.' + name] = compare('stats.' + name, report['mock_stats'][name], value)
if truth is not None:
 compare('truth.n_cleared', truth.get('n_cleared'), len(cleared_channels))
 if scene: compare('truth.n_sources', truth.get('n_sources'), len(scene['sources']))
 actual = {int(s['channel']) for s in truth.get('sources', []) if s.get('cleared')}
 if actual != cleared_channels: errors.append({'check': 'truth cleared channels', 'truth': sorted(actual),
 'actions': sorted(cleared_channels)})
return {'version': AUDIT_VERSION, 'records': records, 'max_abs_dt_error_s': maxima['dt'],
 'max_abs_cumulative_error_s': maxima['cumulative'], 'max_abs_distance_error_m': maxima['distance'],
 'max_abs_switch_error_s': maxima['switch'], 'maxima': dict(maxima),
 'cost_pieces': pieces, 'independent_exit_s': elapsed, 'independent_last_clear_s': last_clear,
 'independent_travel_m': travel, 'counts': dict(counts), 'cleared_channels': sorted(cleared_channels),
 'accepted_action_count': counts['measure'] + counts['clear_success'] + counts['clear_failure'],
 'report_checks': checks, 'errors': errors, 'audit_ok': not errors}

```

```

def flat_metrics(report, audit, scene, wall_s, status='ok', error=None, completion_kind='unknown'):
 n = len(scene['sources']); counts = audit['counts']; cleared = counts.get('clear_success', 0)
 elapsed, last = audit['independent_exit_s'], audit['independent_last_clear_s']; p = audit['cost_pieces']
 return {'exit_s': elapsed, 'last_clear_s': last, 'tail_s': elapsed-last if last is not None else None,
 'exit_per_source_s': elapsed/cleared if cleared else None,
 'last_clear_per_source_s': last/cleared if cleared else None,
 'travel_m': audit['independent_travel_m'], 'n_measure': counts.get('measure', 0),
 'n_switch': counts.get('switch', 0), 'n_clear_success': cleared,
 'n_clear_failure': counts.get('clear_failure', 0),
 't_travel_s': p['travel_s'], 't_measure_s': p['measure_s'],
 't_switch_s': p['switch_s'], 't_clear_s': p['clear_s'],
 'cleared': cleared, 'n_sources': n, 'clear_ratio': cleared/n if n else None,
 'full_clear': bool(n > 0 and cleared == n), 'completion_kind': completion_kind,
 'complete_proof': report.get('complete_proof'), 'wall_s': wall_s,
 'wall_within_1200s': wall_s <= 1200 if wall_s is not None else None,
 'n_rejected': report.get('n_rejected', 0), 'status': status, 'error': error,
 'audit_ok': audit['audit_ok']}

def row_from_trace(t, path):
 return {k: t[k] for k in ('split', 'scenario', 'index', 'seed', 'policy')} | {'trace_file': str(path)} | t['metrics']

def run_case(kind, index, seed, policy, cls, cfg, base, outdir, manifest, split='main'):
 arena = make_scene(kind, seed); scene = scene_manifest(arena); guard = TruthGuard(arena)
 forced_exit = False; status = 'ok'; error = None; robot = None; report = {}
 def timeout_handler(_signal, _frame): raise TimeoutError('1200 s real-time offline allowance exceeded')
 old_handler = signal.signal(signal.SIGALRM, timeout_handler)
 start = time.perf_counter(); signal.setitimer(signal.ITIMER_REAL, 1200.0)
 try:
 robot = cls(arena, cfg, base=base, log=None); report = robot.run()
 except Exception as exc:
 status = 'error'; error = f'{type(exc).__name__}: {exc}'
 finally:
 signal.setitimer(signal.ITIMER_REAL, 0); signal.signal(signal.SIGALRM, old_handler)
 wall = time.perf_counter() - start
 if not guard.exit_called:
 forced_exit = True
 try: arena.exit()
 except Exception as exc: error = f'{error or ""}; exit: {exc}'
 truth = arena.truth() if guard.exit_called else None
 completion = policy if policy in ('field', 'sweep', 'tour') else 'continuous_cell'
 audit = action_audit(arena.actions, report or None, truth, scene)
 metrics = flat_metrics(report, audit, scene, wall, status, error, completion)
 name = f'{split}_{kind}_{index:03d}_{seed}_{policy}'
 path = Path('traces') / f'{name}.json'
 trace = {'schema_version': SCHEMA_VERSION, 'run_id': name, 'split': split, 'scenario': kind,
 'index': index, 'seed': seed, 'policy': policy, 'scene_manifest': scene,
 'requested_config': asdict(cfg), 'effective_config': asdict(robot.cfg if robot else cfg),
 'actions': arena.actions, 'post_exit_truth': truth, 'report': report,
 'action_audit': audit, 'guard': {'exit_called': guard.exit_called, 'truth_calls': guard.truth_calls,
 'forced_harness_exit': forced_exit}, 'metrics': metrics,
 'provenance': {'experiment_id': manifest['experiment_id'],
 'definition_fingerprint': manifest['definition_fingerprint'],
 'source_sha256': manifest['source_sha256'], 'field_sha256': manifest['field_sha256'],
 'execution_runner_sha256': sha256(HERE)}}
 atomic_json(outdir/path, trace)
 return row_from_trace(trace, path)

class CoverageAuditor:

```

```

"""Grid verification using only per-channel raw no_signal observations."""
def __init__(self, step=5.0):
 self.step = step
 xs = np.arange(-1800., 1800. + step/2, step)
 gx, gy = np.meshgrid(xs, xs); mask = gx*gx + gy*gy <= 1800.**2
 self.px, self.py = gx[mask], gy[mask]
 self.cache = {}; self.hits = 0; self.misses = 0; self.point_sets = {}
 # Independent reconstruction of the 50 m cell certificate also keeps
 # boundary-intersecting cells whose centers lie outside the disk.
 h = 50.; cx = -1800. + (np.arange(72) + .5)*h; qx, qy = np.meshgrid(cx, cx)
 m = np.maximum(np.abs(qx)-h/2, 0)**2 + np.maximum(np.abs(qy)-h/2, 0)**2 <= 1800.**2 + 1e-6
 self.qx, self.qy = qx[m], qy[m]; self.cell_radius = 1000.-h/math.sqrt(2)-1e-6
def set_audit(self, points):
 # Sorting/deduplicating lets absent channels and unchanged variants
 # share a single distance computation without dropping any observation.
 points = tuple(sorted(set(points))); key = digest(points)
 if key in self.cache: self.hits += 1; return key, self.cache[key]
 self.misses += 1; self.point_sets[key] = [list(x) for x in points]
 dmin2 = np.full(len(self.px), np.inf); cell_min2 = np.full(len(self.qx), np.inf)
 for x, y in points:
 # Chunking bounds scratch memory; no SciPy nearest-neighbor code.
 for start in range(0, len(self.px), 32768):
 stop = start + 32768
 d2 = (self.px[start:stop]-x)**2 + (self.py[start:stop]-y)**2
 np.minimum(dmin2[start:stop], d2, out=dmin2[start:stop])
 np.minimum(cell_min2, (self.qx-x)**2 + (self.qy-y)**2, out=cell_min2)
 worst = int(np.argmax(dmin2)); unc = int(np.count_nonzero(dmin2 > 1000.**2))
 cunc = int(np.count_nonzero(cell_min2 > self.cell_radius**2))
 result = {'max_nearest_m': math.sqrt(dmin2[worst]) if points else None,
 'worst_xy': [float(self.px[worst]), float(self.py[worst])] if points else None,
 'uncovered_points': unc, 'no_signal_stations': len(points),
 'uncovered_certificate_cells': cunc, 'cell_certificate': cunc == 0}
 self.cache[key] = result
 return key, result
def audit(self, trace):
 acts = trace['actions']; cleared = {int(a['channel']) for a in acts if a.get('kind') == 'clear' and a.get('clear_result') == 'success'}
 channels = {}
 for ch in range(1, 21):
 if ch in cleared: channels[str(ch)] = {'status': 'cleared'}; continue
 obs = [a for a in acts if a.get('kind') == 'measure' and int(a['channel']) == ch]
 if any(a['measure_result'] in ('direction', 'near') for a in obs):
 channels[str(ch)] = {'status': 'positive_pending'}; continue
 points = [(float(a['x']), float(a['y'])) for a in obs if a['measure_result'] == 'no_signal']
 key, result = self.set_audit(points)
 channels[str(ch)] = {'status': 'covered' if result['uncovered_points'] == 0 else 'uncovered',
 'observation_set_sha256': key, **result}
 return {'grid_step_m': self.step, 'grid_points_inside': len(self.px), 'channels': channels,
 'claim_5m_grid': all(c['status'] in ('cleared', 'covered') for c in channels.values()),
 'independent_cell_certificate': all(c['status'] == 'cleared' or c.get('cell_certificate', False)
 for c in channels.values()),
 'certificate_grid_cells': len(self.qx), 'certificate_radius_m': self.cell_radius,
 'note': '5 m point-grid check is numerical evidence; the separate 50 m cell reconstruction implements a conservative geometric certificate.'}

def finite(v): return isinstance(v, (int, float)) and not isinstance(v, bool) and math.isfinite(v)

def bootstrap(a, b):
 if not a: return {'n': 0, 'saving': None, 'ci95': [None, None]}
 a, b = np.asarray(a, float), np.asarray(b, float)

```

```

 if a.mean() <= 0: return {'n': len(a), 'saving': None, 'ci95': [None, None]}
 rng = np.random.default_rng(BOOTSTRAP_SEED)
 idx = rng.integers(0, len(a), size=(BOOTSTRAP_N, len(a)))
 amean, bmean = a[idx].mean(axis=1), b[idx].mean(axis=1)
 if np.any(amean <= 0): raise ValueError('Undefined bootstrap relative saving due to zero reference mean')
 vals = 1-bmean/amean
 return {'n': len(a), 'saving': float(1-b.mean()/a.mean()), 'ci95': np.percentile(vals, [2.5, 97.5]).tolist()}

def aggregate(rows):
 def values(key): return [r[key] for r in rows if finite(r.get(key))]
 def mean(key):
 v = values(key); return float(np.mean(v)) if v else None
 times = values('exit_s'); sources = sum(r.get('n_sources', 0) for r in rows)
 out = {'cases': len(rows), 'completed': sum(r['status'] == 'ok' for r in rows),
 'errors': sum(r['status'] != 'ok' for r in rows), 'all_statuses_included': True,
 'full_clear_cases': sum(bool(r['full_clear']) for r in rows),
 'cleared': sum(r.get('cleared', 0) for r in rows), 'sources': sources,
 'clear_ratio_source_weighted': sum(r.get('cleared', 0) for r in rows)/sources if sources else None,
 'zero_cleared_cases': sum(r.get('cleared', 0) == 0 for r in rows),
 'primary_metric_defined_cases': len(times),
 'undefined_primary_cases': len(rows)-len(times),
 'per_source_metric_defined_cases': len(values('exit_per_source_s')),
 'undefined_per_source_cases': len(rows)-len(values('exit_per_source_s')),
 'p90_exit_s': float(np.percentile(times, 90)) if times else None,
 'max_exit_s': max(times) if times else None,
 'wall_max_s': max(values('wall_s'), default=None),
 'rejections': sum(r.get('n_rejected', 0) for r in rows),
 'completion_kinds': dict(Counter(r['completion_kind'] for r in rows)),
 'claimed_complete_cases': sum(r.get('complete_proof') is True for r in rows),
 'audit_failure_cases': sum(not r.get('audit_ok', False) for r in rows)}
 mapping = {'mean_exit_s': 'exit_s', 'mean_exit_per_source_s': 'exit_per_source_s',
 'mean_last_clear_s': 'last_clear_s', 'mean_last_clear_per_source_s': 'last_clear_per_source_s',
 'mean_tail_s': 'tail_s', 'mean_travel_m': 'travel_m', 'mean_measure': 'n_measure',
 'mean_switch': 'n_switch', 'mean_clear_failure': 'n_clear_failure', 'wall_mean_s': 'wall_s',
 'mean_t_travel_s': 't_travel_s', 'mean_t_measure_s': 't_measure_s',
 'mean_t_switch_s': 't_switch_s', 'mean_t_clear_s': 't_clear_s'}
 out.update({dst: mean(src) for dst, src in mapping.items()})
 out['metric_defined_counts'] = {src: len(values(src)) for src in mapping.values()}
 return out

def paired(reference, comparison):
 ref = {(r['scenario'], r['seed']): r for r in reference}
 cmp = {(r['scenario'], r['seed']): r for r in comparison}
 keys = sorted(ref.keys() & cmp.keys()); pairs = [(ref[k], cmp[k]) for k in keys]
 def one(metric):
 keep = [(a, b) for a, b in pairs if finite(a.get(metric)) and finite(b.get(metric))]
 result = bootstrap([a[metric] for a, b in keep], [b[metric] for a, b in keep])
 result.update({'candidate_pairs': len(pairs), 'undefined_metric_pairs': len(pairs)-len(keep),
 'non_ok_pairs_included': sum(a['status'] != 'ok' or b['status'] != 'ok' for a, b in keep),
 'incomplete_clear_pairs_included': sum(not a['full_clear'] or not b['full_clear'] for a, b
 in keep),
 'comparison_faster_cases': sum(b[metric] < a[metric] for a, b in keep)})
 return result
 out = one('exit_s'); out['per_source'] = one('exit_per_source_s')
 out['reference_only_cases'] = len(ref.keys()-cmp.keys()); out['comparison_only_cases'] = len(cmp.keys()-ref.
 keys())
 out['convention'] = 'positive saving means comparison faster; include failures/incomplete-clear pairs when
 numeric metrics exist'
 return out

```



```

def summarize(rows):
 main = [r for r in rows if r['split'] == 'main']; variants = [r for r in rows if r['split'] == 'ablation']
 def group(rs):
 policies = sorted(set(r['policy'] for r in rs)); out = {p: aggregate([r for r in rs if r['policy'] == p])
 for p in policies}
 out['paired_vs_field'] = {p: paired([r for r in rs if r['policy'] == 'field'], [r for r in rs if r['
 policy'] == p]) for p in policies if p != 'field'}
 out['joint_vs'] = {p: paired([r for r in rs if r['policy'] == p], [r for r in rs if r['policy'] == 'joint
 ']) for p in policies if p != 'joint'}
 return out
 out = {'schema_version': SCHEMA_VERSION, 'counts': {'rows': len(rows), 'main': len(main), 'ablation': len(
 variants),
 'status': dict(Counter(r['status'] for r in rows))), 'overall': group(main),
 'scenarios': {k: group([r for r in main if r['scenario'] == k]) for k, _ in SCENARIOS if any(r['
 scenario'] == k for r in main)},
 'ablations': {}, 'bootstrap': {'resamples': BOOTSTRAP_N, 'seed': BOOTSTRAP_SEED},
 'failure_policy': 'All runs contribute to aggregates, including error-status and incomplete-clear runs
 . Undefined values are null and counts are disclosed; never replace zero-cleared denominator with one.'}
 if variants:
 out['ablations']['random'] = {p: aggregate([r for r in variants if r['policy'] == p]) for p, _ in
 ABLATIONS}
 out['ablations']['random']['joint_baseline'] = aggregate([r for r in main if r['scenario'] == 'random'
 and r['index'] < 20 and r['policy'] == 'joint'])
 out['ablations']['random']['paired_vs_joint'] = {p: paired([r for r in main if r['scenario'] == 'random'
 and r['index'] < 20 and r['policy'] == 'joint'], [r for r in variants if r['policy'] == p]) for p, _ in
 ABLATIONS}
 return out

def static_policy_audit():
 accesses = []; suspicious = []; truth = []; dynamic = []
 blocked = {'sources', 'seed', 'rng', 'env_error', '_src_of', '_waves', '_scale', '_local_w', '_local_seed', '
 __dict__', '__getattr__'}
 methods = []
 for filename in POLICY_FILES:
 root = ast.parse((SRC/filename).read_text(encoding='utf-8'))
 for cls in root.body:
 if not isinstance(cls, ast.ClassDef) or cls.name not in POLICY_CLASSES: continue
 for method in cls.body:
 if not isinstance(method, (ast.FunctionDef, ast.AsyncFunctionDef)): continue
 methods.append({'file': 'src/'+filename, 'class': cls.name, 'method': method.name, 'line': method
 .lineno})
 for node in ast.walk(method):
 entry = {'file': 'src/'+filename, 'class': cls.name, 'method': method.name,
 'line': getattr(node, 'lineno', None)}
 if isinstance(node, ast.Attribute):
 expr = ast.unparse(node)
 if '.arena.' in expr:
 accesses.append(**entry, 'expression': expr)
 if node.attr in blocked: suspicious.append(**entry, 'expression': expr)
 if node.attr == 'truth': truth.append(**entry, 'expression': expr)
 if isinstance(node, ast.Call) and isinstance(node.func, ast.Name) and node.func.id in ('
 getattr', 'setattr', 'hasattr'):
 if node.args and 'arena' in ast.unparse(node.args[0]):
 expr = ast.unparse(node); dynamic.append(**entry, 'expression': expr)
 if len(node.args) < 2 or not isinstance(node.args[1], ast.Constant) or node.args[1].
 value in blocked:
 suspicious.append(**entry, 'expression': expr)
 allowed_truth = all(x['class'] == 'P3Robot' and x['method'] == 'report' for x in truth)
 return {'audit_ok': not suspicious and allowed_truth, 'files': list(POLICY_FILES),
 'classes': sorted(POLICY_CLASSES), 'method_count': len(methods),

```

```

 'decision_method_count': sum(x['method'] != 'report' for x in methods),
 'methods': methods, 'arena_attribute_accesses': accesses,
 'dynamic_attribute_accesses': dynamic, 'sensitive_hits': suspicious,
 'truth_accesses': truth, 'truth_only_in_report': allowed_truth,
 'manual_crosscheck': 'Independent reviewer checked 147 decision methods plus report, module-level
benchmark imports, dynamic getattr/eval/exec/vars, and p3_expect_field/p1_intersection/p2_grid_expectation
helpers. No protected arena state found in active decisions. LIVE1/LIVE2 exist as module imports in
p3_frontier but are not referenced in policy methods.',
 'normal_exit_before_report_lines': {'p3_robot.py': [1189,1193], 'p3_sweep.py': [1086,1090], 'p3_tour.
py': [1522,1527]},
 'scope': 'AST inspection of decision classes and all their methods; excludes module-level mock tests/
scene generators. Runtime truth guard independently checks actual exit order.',
 'limitation': 'Static inspection and endpoint guard are not a language-level sandbox; reflective/
dynamic evasion is not claimed impossible.'}

def audit_selfcheck():
 # A clear on channel 8 must leave the detector on its initial channel 1.
 actions = [{'kind': 'enter', 'virtual_time_s': 0.},
 {'kind': 'clear', 'x': 0., 'y': 0., 'channel': 8, 'clear_result': 'no_target_in_range', 'dt': 3., 'dist_m'
: 0., 'virtual_time_s': 3.},
 {'kind': 'measure', 'x': 15., 'y': 20., 'channel': 1, 'measure_result': 'no_signal', 'switch_s': 0., 'dt'
: 10., 'dist_m': 25., 'virtual_time_s': 13.},
 {'kind': 'measure', 'x': 15., 'y': 20., 'channel': 2, 'measure_result': 'no_signal', 'switch_s': 1., 'dt'
: 6., 'dist_m': 0., 'virtual_time_s': 19.},
 {'kind': 'exit', 'virtual_time_s': 19.}]
 checks = {'valid_clear_preserves_channel': action_audit(actions)['audit_ok']}
 for key in ('switch_s', 'dist_m', 'dt', 'virtual_time_s'):
 corrupt = [dict(a) for a in actions]; corrupt[2][key] += 1
 checks['detect_corrupted_'+key] = not action_audit(corrupt)['audit_ok']
 corrupt = [dict(a) for a in actions]
 for a in corrupt[2:]: a['virtual_time_s'] += 1
 checks['detect_cumulative_offset'] = not action_audit(corrupt)['audit_ok']
 m = flat_metrics({}, action_audit(actions), {'sources': [{}]}, .1, 'error', 'synthetic failure', 'field')
 checks['zero_clear_is_undefined'] = m['exit_per_source_s'] is None and m['last_clear_s'] is None
 agg = aggregate([m]); checks['failed_run_retained'] = agg['cases'] == 1 and agg['errors'] == 1 and agg['
mean_exit_s'] == 19.
 if not all(checks.values()): raise AssertionError(checks)
 return checks

def raw_payload(t):
 keys = ('run_id', 'split', 'scenario', 'index', 'seed', 'policy', 'scene_manifest',
 'requested_config', 'effective_config', 'actions', 'post_exit_truth', 'truth_error', 'report', 'guard
')
 return {k: t.get(k) for k in keys} | {'execution_wall_s': t['metrics'].get('wall_s')}

def archive_scratch(outdir, rows):
 scratch = Q3.parents[1] / 'tmp/q3-paper-scratch'
 scratch.mkdir(parents=True, exist_ok=True)
 refs = {(outdir/r['trace_file']).resolve() for r in rows}
 moved = []
 for path in sorted((outdir/'traces').glob('*.*json')) + [outdir/'metadata.json']:
 if not path.exists() or path.resolve() in refs: continue
 relative = path.relative_to(outdir); dest = scratch/'previous-products'/relative
 dest.parent.mkdir(parents=True, exist_ok=True)
 if dest.exists(): dest = dest.with_name(dest.stem + '-' + sha256(path)[:12] + dest.suffix)
 if dest.exists(): raise RuntimeError(f'Archive destination already exists: {dest}')
 value = sha256(path); path.rename(dest)
 moved.append({'from': str(relative), 'to': str(dest), 'sha256': value})
 return moved

```

```

def snapshot_sources(outdir, manifest):
 hashes = {**manifest['source_sha256'], **manifest['field_sha256'], 'scripts/run_p3_paper.py': sha256(HERE)}
 for relative, expected in hashes.items():
 source = Q3/relative; target = outdir/'source_snapshot'/relative
 target.parent.mkdir(parents=True, exist_ok=True)
 if sha256(source) != expected: raise RuntimeError(f'Input changed during audit: {source}')
 if not target.exists() or sha256(target) != expected: shutil.copyfile(source, target)
 atomic_json(outdir/'source_snapshot/SHA256.json', hashes)
 return len(hashes)

def write_results(outdir, rows, manifest):
 atomic_json(outdir/'paired_results.json', {'schema_version': SCHEMA_VERSION, 'experiment_id': manifest['
 experiment_id'],
 'definition_fingerprint': manifest['definition_fingerprint'], 'manifest_file': 'manifest.json', 'rows':
 rows})
 columns = sorted({k for r in rows for k in r})
 with (outdir/'paired_results.csv').open('w', encoding='utf-8', newline='') as stream:
 writer = csv.DictWriter(stream, fieldnames=columns); writer.writeheader(); writer.writerows(rows)

def audit_all(outdir, rows, manifest):
 start = time.perf_counter(); auditor = CoverageAuditor(); audits = []; all_checks = []; new_rows = []
 fingerprints = defaultdict(set); scene_checks = []; configs_bad = []; max_errors = Counter(); raw_ok = True
 static = static_policy_audit(); selfcheck = audit_selfcheck(); prior = outdir/'audit_report.json'
 prior_report = read_json(prior) if prior.exists() else {}
 moved = archive_scratch(outdir, rows)
 for n, row in enumerate(rows):
 path = outdir/row['trace_file']; trace = read_json(path); raw_before = digest(raw_payload(trace))
 # Recompute from the raw action stream, never from the previous audit.
 audit = action_audit(trace['actions'], trace.get('report'), trace.get('post_exit_truth'), trace['
 scene_manifest'])
 old = trace['metrics']; trace['action_audit'] = audit
 trace['metrics'] = flat_metrics(trace.get('report') or {}, audit, trace['scene_manifest'], old.get('
 wall_s'),
 old.get('status', 'error'), old.get('error'), old['completion_kind'])
 scene = trace['scene_manifest']; fingerprints[(row['scenario'], row['seed'])].add((scene['
 scene_fingerprint'], scene['error_fingerprint']))
 scene_obj = {'class': scene['class'], 'seed': scene['seed'], 'r_arena': 1800.0, 'sources': scene['sources
 ']}
 # Historical fingerprint serialization used ASCII default; values here
 # contain ASCII-only strings so this is byte-equivalent.
 scene_ok = digest(scene_obj) == scene['scene_fingerprint'] and digest(scene['error_field']) == scene['
 error_fingerprint']
 scene_checks.append(scene_ok)
 requested = trace['requested_config']; cfg_ok = requested == manifest['configs'][row['policy']]
 effective = dict(requested)
 if old['completion_kind'] == 'continuous_cell': effective.update({'cover_radius_m': 1000.-50./math.sqrt
 (2)-1e-6, 'cover_slack_m': 0.0})
 effective_ok = trace['effective_config'] == effective
 if not cfg_ok or not effective_ok: configs_bad.append({'trace_file': row['trace_file'], 'requested_ok':
 cfg_ok, 'effective_ok': effective_ok})
 exits = [a for a in trace['actions'] if a.get('kind') == 'exit']
 calls = trace.get('guard', {}).get('truth_calls', [])
 guard_ok = bool(exits and calls and trace.get('guard', {}).get('exit_called') and
 all(c.get('exit_called') and c['virtual_time_s'] >= exits[0]['virtual_time_s']-TOL for c
 in calls))
 trace['audit_provenance'] = {'audit_version': AUDIT_VERSION, 'runner_sha256': sha256(HERE),
 'raw_payload_sha256': raw_before, 'definition_fingerprint': manifest['definition_fingerprint'],
 'execution_algorithms_unchanged': True,

```

```

 'verified_source_sha256': manifest['source_sha256'],
 'verified_field_sha256': manifest['field_sha256']}]
 raw_after = digest(raw_payload(trace)); raw_ok &= raw_before == raw_after
 trace['audit_provenance']['raw_payload_unchanged'] = raw_before == raw_after
 check = {'trace_file': row['trace_file'], 'policy': row['policy'], 'scenario': row['scenario'], 'seed':
row['seed'],
 'raw_payload_sha256': raw_before, 'raw_unchanged': raw_before == raw_after,
 'action_audit_ok': audit['audit_ok'], 'truth_guard_ok': guard_ok, 'scene_fingerprint_ok':
scene_ok,
 'requested_config_ok': cfg_ok, 'effective_config_ok': effective_ok,
 'errors': audit['errors'], 'maxima': audit['maxima']}]
 all_checks.append(check)
 for k, value in audit['maxima'].items(): max_errors[k] = max(max_errors[k], value)
 if old['completion_kind'] == 'continuous_cell' and trace['report'].get('complete_proof') is True:
 coverage = auditor.audit(trace)
 audits.append({'scenario': row['scenario'], 'index': row['index'], 'seed': row['seed'], 'policy': row
['policy'],
 'split': row['split'], 'trace_file': row['trace_file'], 'audit': coverage})
 trace['independent_coverage_audit'] = {'claim_5m_grid': coverage['claim_5m_grid'],
 'cell_certificate': coverage['independent_cell_certificate'], 'details_file': 'coverage_audit.
json'}
 atomic_json(path, trace); new_rows.append(row_from_trace(trace, row['trace_file']))
 if (n+1) % 50 == 0: print(f'Audited {n+1}/{len(rows)} traces; {len(audits)} certificates, {auditor.misses
} unique observation sets', flush=True)
 unpaired = [{'scenario': k[0], 'seed': k[1], 'different_fingerprints': len(v)} for k, v in fingerprints.items
() if len(v) != 1]
 coverage_result = {'schema_version': SCHEMA_VERSION, 'experiment_id': manifest['experiment_id'], '
audit_version': AUDIT_VERSION,
 'scope': 'All adaptive/joint/ablation runs claiming continuous-cell completion', 'audits':
audits,
 'counts': {'runs': len(audits), 'numerical_pass': sum(x['audit']['claim_5m_grid'] for x in
audits),
 'cell_certificate_pass': sum(x['audit']['independent_cell_certificate'] for x
in audits),
 'unique_observation_sets': auditor.misses, 'observation_cache_hits': auditor.
hits},
 'observation_sets': auditor.point_sets, 'observation_set_results': auditor.cache}
 atomic_json(outdir/'coverage_audit.json', coverage_result)
 summary = summarize(new_rows); summary['experiment_id'] = manifest['experiment_id']; summary['audit'] =
coverage_result['counts']
 atomic_json(outdir/'summary.json', summary); write_results(outdir, new_rows, manifest)
 atomic_json(outdir/'manifest.json', manifest)
 snapshot_count = snapshot_sources(outdir, manifest)
 report = {'version': AUDIT_VERSION, 'experiment_id': manifest['experiment_id'], 'runs': len(new_rows),
 'all_raw_payloads_unchanged': raw_ok, 'policy_runs_executed_by_this_audit': 0,
 'action_audit_pass': sum(x['action_audit_ok'] for x in all_checks),
 'truth_guard_pass': sum(x['truth_guard_ok'] for x in all_checks), 'scene_fingerprint_pass': sum(
scene_checks),
 'pairing_mismatches': unpaired, 'configuration_mismatches': configs_bad, 'max_errors': dict(max_errors),
 'coverage': coverage_result['counts'], 'static_policy_audit': static, 'audit_contract_checks': selfcheck,
 'source_snapshot_files': snapshot_count, 'source_snapshot_index': 'source_snapshot/SHA256.json',
 'archive_movements': prior_report.get('archive_movements', []) + moved,
 'per_run_checks': all_checks, 'audit_wall_s': time.perf_counter()-start}
 atomic_json(outdir/'audit_report.json', report)
 write_readme_and_evidence(outdir, summary, report, manifest)
 print(json.dumps({'rows': len(new_rows), 'action_pass': report['action_audit_pass'], 'truth_pass': report['
truth_guard_pass'],
 'coverage': report['coverage'], 'audit_wall_s': report['audit_wall_s']}), flush=True)
 return new_rows

```

```

def write_readme_and_evidence(outdir, summary, audit, manifest):

```

```
evidence = {'experiment_id': manifest['experiment_id'], 'claims': [
 {'claim': 'Five policies compared on the same 120 scenes', 'evidence': ['manifest.json#/scenarios', '
paired_results.json#/rows', 'audit_report.json#/pairing_mismatches']},
 {'claim': 'Every reported action obeys independently recomputed physical timing', 'evidence': ['
audit_report.json#/max_errors', 'traces/*/action_audit']},
 {'claim': 'No truth call before accepted exit, and no sensitive arena attributes in policy classes', '
evidence': ['audit_report.json#/truth_guard_pass', 'audit_report.json#/static_policy_audit']},
 {'claim': 'Every continuous-cell completion claim passes 5 m numerical and independent 50 m cell
verification', 'evidence': ['coverage_audit.json#/counts', 'coverage_audit.json#/audits'], 'limit': '
Numerical point verification alone is not a proof; finite-sample success is not a universal algorithm
guarantee.'},
 {'claim': 'Joint efficiency improvements and uncertainty', 'evidence': ['summary.json#/scenarios', '
summary.json#/overall/joint_vs']},
 {'claim': 'Ablation modules have different effect sizes; after-service scan CI may include zero', '
evidence': ['summary.json#/ablations/random/paired_vs_joint'], 'limit': '20 paired random cases, one module
removed at a time; effects need not add linearly.'}],
 'key_results': {'overall': summary.get('overall'), 'ablations': summary.get('ablations')},
 'remaining_limitations': ['Offline mock evidence only; formal online runs remain pending.',
 'Historical execution runner bytes were not preserved before metadata-only edits. Algorithm and field
hashes are verified; current audit runner is snapshotted.',
 'Historical real-time allowance was measured but not enforced by a watchdog.',
 'The error field is a bounded synthetic model, with hash-noise stress scenes; these do not cover
every physically possible error field.']}
atomic_json(outdir/'evidence_map.json', evidence)
text = f'''# Q3 frozen paper experiment
```

Experiment ID: `{manifest['experiment\_id']}`. Definition fingerprint: `{manifest['definition\_fingerprint']}`.

The final matrix contains 120 paired scenes × 5 default policies (600 runs), plus  
20 random seeds × 4 single-module joint ablations (80 runs). Scenario order is  
random 50, annulus 20, center 10, hash 20, worstrecv 20. Seed =  
2026091200 + 100000 × scenario index + 97 × within-scenario index.  
Every policy uses a fresh scene with the same exact source and error-field fingerprints.

## Reproduce or audit

From the repository root, with Python 3.12 and NumPy 2.3.5 (no SciPy needed):

```
```sh
python3 -B B_locator/Q3/scripts/run_p3_paper.py --audit-only
python3 -B B_locator/Q3/scripts/run_p3_paper.py --smoke
python3 -B B_locator/Q3/scripts/run_p3_paper.py --out /path/to/new-experiment
```

`--audit-only` only replays existing actions and never constructs/runs a robot.
`--smoke` writes to a separate smoke subdirectory. A new experiment runs serially;
existing finished traces are validated and skipped. Incompatible algorithm/config
hashes raise an error instead of overwriting data. Audit/report script changes do
not invalidate or rerun algorithms. `--max-new-runs N` can bound a partial new run.
The stored source snapshot contains all Q3 Python modules, the current runner,
nine loaded expectation maps, their manifest and the fallback map; SHA256.json
verifies their bytes. The immutable algorithm/field definition excludes reporting
script revisions. The discarded early src/run_p3_paper.py was not an imported
algorithm dependency and is excluded from the final manifest.
```

## Metric definitions

$T = \text{travel}_m/5 + 5*N_{\text{measure}} + N_{\text{switch}} + 5*N_{\text{clear\_success}} + 3*N_{\text{clear\_failure}}$ .  
The initial location is (0,0) and measuring channel is 1. Only a measurement  
changes the measuring channel; clear and enter/exit do not. Every distance,  
switch, dt and cumulative timestamp is independently recomputed from raw actions.  
The cumulative sum never uses a reported timestamp as its next starting value.

Final metrics are derived from this reconstruction and checked against report, mock statistics and post-exit truth. The paper primary metric is the complete T, reported as mean and P90. The per-source statistic is mean(T/cleared), with null for cleared=0 and a disclosed defined-value count. All status values, including failed and incomplete-clear runs, remain in aggregates. Last-clear time and tail after last clear are diagnostics and are undefined when no clear succeeds. The bootstrap uses 4000 paired scene resamples, RNG seed 20260912, fresh for each comparison. A positive saving means the comparison uses less time than the reference. Both total time and per-source time have percentile 95% confidence intervals.

The simulator virtual limit is 360000 s. The 1200 s allowance is real computation time. Historical policy construction/run wall times were recorded without a timer; all are summarized, but local wall measurements are not official online validation. Future new runs install a 1200 s timer and retain timeouts as failed cases.

## ## Coverage and information isolation

The independent 5 m grid audit uses only actual no\_signal responses for each uncleared channel and successful-clear actions to retire channels. Any positive observation left uncleared blocks completion. Identical observation station sets are deduplicated and cached. All {audit['coverage']['runs']} claimed continuous-cell completions are audited ({audit['coverage']['numerical\_pass']} numerical passes). This numerical audit is not a continuous proof. A separate independent 50 m closed-square-cell reconstruction checks all cells intersecting the disk at radius 1000 - 50/sqrt(2) - 1e-6; its triangle-inequality argument certifies entire cells. {audit['coverage']['cell\_certificate\_pass']} such reconstructions pass.

The runtime truth wrapper logs calls and permits them only after accepted exit. AST inspection covers every method in the eight decision classes, with sensitive arena properties and dynamic getattr calls separately listed. Module-level mock tests and scene-generation helpers are outside decision scope. This is transparent inspection plus a runtime guard, not a security sandbox.

## ## Files and provenance

- paired\_results.json / .csv: the authoritative 680 rows, each pointing to one trace.
- traces/: only final referenced traces; raw actions/configuration/report/truth are preserved.
- summary.json: all cases, each scenario, all baselines and four paired ablations.
- coverage\_audit.json: per-run/per-channel verification and deduplicated observation sets.
- audit\_report.json: timing, truth, configuration, static-source, pairing and payload-integrity checks.
- evidence\_map.json: paper claims linked to precise evidence and limitations.
- source\_snapshot/: verified algorithm, field and current audit-runner bytes.

600 redundant early traces and obsolete metadata are moved (never deleted) to `tmp/q3-paper-scratch/previous-products/`. Movements and hashes are listed in the audit report. Historical provenance fields are retained in traces; the separate audit\_provenance section records this audit and canonical raw-payload hashes. Earlier runner metadata was refreshed after reporting edits, so its hash is not claimed to prove the exact historical runner bytes. Algorithm/field hashes remain verified. This audit did not rerun any policy.

```
'''
 (outdir/'README.md').write_text(text, encoding='utf-8')

def main(argv=None):
 parser = argparse.ArgumentParser(description=__doc__)
 parser.add_argument('--out', default=str(OUT_DEFAULT)); parser.add_argument('--audit-only', action='store_true')
 parser.add_argument('--smoke', action='store_true'); parser.add_argument('--max-new-runs', type=int, default=0)
 args = parser.parse_args(argv); outdir = Path(args.out)
 if args.smoke: outdir = outdir/'smoke'
```

```

(outdir/'traces').mkdir(parents=True, exist_ok=True)
old_manifest = read_json(outdir/'manifest.json') if (outdir/'manifest.json').exists() else None
manifest = make_manifest(old_manifest)
result = read_json(outdir/'paired_results.json') if (outdir/'paired_results.json').exists() else {'rows': []}
rows = result['rows']; by_key = {(r['split'], r['scenario'], r['index'], r['policy']): r for r in rows}
if len(by_key) != len(rows): raise RuntimeError('Duplicate result key')
if args.audit_only:
 if not rows: raise RuntimeError('--audit-only requires existing paired_results.json')
 audit_all(outdir, rows, manifest); return
atomic_json(outdir/'manifest.json', manifest)
registry = {p: (cls, cfg) for p, cls, cfg in POLICIES}; jobs = []
for s, (kind, count) in enumerate(SCENARIOS):
 if args.smoke and s: continue
 for i in range(1 if args.smoke else count):
 for p, cls, cfg in POLICIES: jobs.append(('main', kind, i, SEED_BASE+100000*s+97*i, p, cls, cfg()))
for i in range(1 if args.smoke else 20):
 for p, override in ABLATIONS: jobs.append(('ablation', 'random', i, SEED_BASE+97*i, p, JointRobot,
 replace(JointConfig(), **override)))
new = 0; base = None
for split, kind, i, seed, p, cls, cfg in jobs:
 key = split, kind, i, p
 if key in by_key:
 existing = read_json(outdir/by_key[key]['trace_file'])
 if existing['seed'] != seed or existing['requested_config'] != clean(asdict(cfg)):
 raise RuntimeError(f'Invalid resume metadata: {key}')
 continue
 if args.max_new_runs and new >= args.max_new_runs: break
 if base is None: base = load_base()
 row = run_case(kind, i, seed, p, cls, cfg, base, outdir, manifest, split)
 by_key[key] = row; new += 1; rows = list(by_key.values())
 write_results(outdir, rows, manifest)
 print(f'New run {new}: {kind} {i} {p} {row["status"]}', flush=True)
audit_all(outdir, list(by_key.values()), manifest)
print(f'Executed {new} new policy runs', flush=True)

if __name__ == '__main__': main()

```

## 5.16 scripts/make\_paper\_tables.py

```

"""Read frozen Q3 rows and emit paper tables, macros and supporting statistics."""
import json
import hashlib
from pathlib import Path
import numpy as np

Q3 = Path(__file__).resolve().parents[1]
DATA = Q3 / 'out/paper_q3'
SECTIONS = Q3.parent / 'paper/sections'
POLICIES = ('field', 'sweep', 'tour', 'adaptive', 'joint')
SCENARIOS = ('random', 'annulus', 'center', 'hash', 'worstrecv')

def main():
 obj = json.loads((DATA/'paired_results.json').read_text())
 rows = obj['rows']
 summary = json.loads((DATA/'summary.json').read_text())
 random = summary['scenarios']['random']
 body=[]
 for p in POLICIES:
 s=random[p]

```



```

 body.append(f"{p} & {s['clear_ratio_source_weighted']:.3f} & {s['full_clear_cases']}/50 & {s['mean_exit_s']:.0f} & {s['p90_exit_s']:.0f} & {s['mean_exit_per_source_s']:.0f}" + r' \\')
(SESSIONS/'q3-core-rows.tex').write_text('\n'.join(body).rstrip().removesuffix(r'\\')+'\n')
full={p:sum(r['full_clear'] for r in rows if r['split']=='main' and r['policy']==p) for p in POLICIES}
pair=random['paired_vs_field']['joint']
base20=[r for r in rows if r['split']=='main' and r['scenario']=='random' and r['policy']=='joint' and r['index']<20]
values={'QThreeFieldFull':str(full['field']), 'QThreeSweepFull':str(full['sweep']),
 'QThreeSaving':f"{100*pair['saving']:.1f}", 'QThreeSavingLow':f"{100*pair['ci95'][0]:.1f}",
 'QThreeSavingHigh':f"{100*pair['ci95'][1]:.1f}", 'QThreeAblationBase':f"{np.mean([r['exit_s'] for r
in base20]):.1f}"}
for policy in POLICIES:
 title=policy.capitalize()
 first=next(r for r in rows if r['split']=='main' and r['scenario']=='random' and r['index']==0 and r['policy']==policy)
 values[f'QThreeFirst{title}Time']=f"{first['exit_s']:.0f}"
 values[f'QThreeFirst{title}Travel']=f"{first['travel_m']/1000:.2f}"
 values[f'QThreeFirst{title}Measure']=str(first['n_measure'])
 values[f'QThreeRandom{title}Travel']=f"{random[policy]['mean_travel_m']/1000:.2f}"
 values[f'QThreeRandom{title}Measure']=f"{random[policy]['mean_measure']:.1f}"
 values[f'QThreeCenter{title}Time']=f"{summary['scenarios'][policy]['center']['mean_exit_s']:.1f}"
for suffix, policy in [('After','joint_no_after_service'),('Multi','joint_single_plan'),('Polish','joint_no_cover_polish'),('Prob','joint_no_probability_gate')]:
 values[f'QThreeAblation{suffix}Increase']=f"{-100*summary['ablations'][policy]['paired_vs_joint'][policy]['saving']:.2f}"
(SESSIONS/'q3-numbers.tex').write_text('% Generated from the frozen Q3 data; do not hand edit.\n'+'.join('\\newcommand{\\'+k+'}{'+v+'}\\n' for k,v in values.items()))
abstract=(r'{\heiti 针对问题三}\songti, 建立{\heiti 联合覆盖、定位与清除模型}\songti, '
 '以保守定位控制清除风险, 以连续覆盖判据验证任务完成, 并逐步引入共用航路、沿途定位和逐频道检测调度。'
 '在 120 组场景开展五策略配对实验, 另作 80 次单因素消融。随机 50 组中, '
 r'$\textit{joint}$ 全部清除, 完整退出时间均值 '
 +f"{random['joint']['mean_exit_s']:.0f}" +r' s, {\heiti 较初代减少 '
 +values['QThreeSaving']+r'\%}\songti. 结果表明, 联合规划能同时减少行程与检测成本。'+'\n')
(SESSIONS/'q3-abstract.tex').write_text('% Generated Q3 abstract from frozen data.\n'+abstract)
lines=[]
for kind in SCENARIOS:
 for p in POLICIES:
 rs=[r for r in rows if r['split']=='main' and r['scenario']==kind and r['policy']==p]
 s=summary['scenarios'][kind][p]
 lines.append(f"{kind}/{p} & {s['clear_ratio_source_weighted']:.3f} & {s['full_clear_cases']}/{len(rs)} & {s['mean_exit_s']:.1f} & {s['p90_exit_s']:.1f} & {s['mean_exit_per_source_s']:.1f} & {s['mean_travel_m']/1000:.2f} & {s['mean_measure']:.1f} & {s['wall_mean_s']:.3f}" +r' \\')
(SESSIONS/'q3-supplement-rows.tex').write_text('\n'.join(lines).rstrip().removesuffix(r'\\')+'\n')
lines=[]
for p in ('joint_no_after_service','joint_single_plan','joint_no_cover_polish','joint_no_probability_gate'):
 s=summary['ablations']['random'][p]
 pair=summary['ablations']['random']['paired_vs_joint'][p]
 label={'joint_no_after_service':'取消清后扫描','joint_single_plan':'单方案规划','joint_no_cover_polish':'取消测站精修','joint_no_probability_gate':'取消概率门槛'}[p]
 lines.append(f"{label} & {s['mean_exit_s']:.1f} & {-100*pair['saving']:.2f} & [{-100*pair['ci95'][1]:.2f}, {-100*pair['ci95'][0]:.2f}] & {s['mean_travel_m']/1000:.2f} & {s['mean_measure']:.1f} & {s['wall_mean_s']:.3f}" +r' \\')
(SESSIONS/'q3-ablation-rows.tex').write_text('\n'.join(lines).rstrip().removesuffix(r'\\')+'\n')
lines=[]
for p,label in [('joint_baseline','完整 joint'),('joint_no_after_service','取消清后扫描'),('joint_single_plan','单方案规划'),('joint_no_cover_polish','取消测站精修'),('joint_no_probability_gate','取消概率门槛')]:
 s=summary['ablations']['random'][p]
 lines.append(f"{label} & {s['clear_ratio_source_weighted']:.3f} & {s['full_clear_cases']}/20 & {s['mean_exit_s']:.1f} & {s['p90_exit_s']:.1f} & {s['mean_exit_per_source_s']:.1f}" +r' \\')
(SESSIONS/'q3-ablation-complete-rows.tex').write_text('\n'.join(lines).rstrip().removesuffix(r'\\')+'\n')
(DATA/'table_provenance.json').write_text(json.dumps({'experiment_id':obj['experiment_id'], 'data_sha256':hashlib.sha256((DATA/'paired_results.json').read_bytes()).hexdigest(), 'summary_sha256':hashlib.sha256((DATA/

```



```

 /'summary.json').read_bytes()).hexdigest(), 'generated':values, 'all_main_full_clear':full},indent=2)+'\n')

if __name__=='__main__':main()

```

## 5.17 scripts/make\_paper\_figures.py

```

"""Make the compact Q3 paper figures from frozen experiment records.

The script is intentionally a renderer only: it never runs a policy and never
rewrites the frozen JSON. The runner writes ``out/paper_q3`` using the schema
documented in the module docstring of the evidence runner; this script reads
that directory and writes ``paper/figures/q3-*.pdf`` and ``.png``.

Usage (from the repository root)::

 python B_locator/Q3/scripts/make_paper_figures.py

The five figures are sized for the 160 mm text width of the paper. Full trajectories remain in the frozen
evidence directory.
"""
from __future__ import annotations

import argparse
import json
import math
from pathlib import Path
from typing import Any, Iterable

import numpy as np

import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt
from matplotlib.lines import Line2D
from matplotlib.patches import Circle, Rectangle

MM = 1.0 / 25.4
FIG_W = 160 * MM
DPI = 330
ROOT = Path(__file__).resolve().parents[2]
DATA = ROOT / "Q3" / "out" / "paper_q3"
OUT = ROOT / "paper" / "figures"

POLICIES = ["field", "sweep", "tour", "adaptive", "joint"]
POLICY_LABEL = {
 "field": "field",
 "sweep": "sweep",
 "tour": "tour",
 "adaptive": "adaptive",
 "joint": "joint",
}
POLICY_COLOR = {
 "field": "#667085", "sweep": "#b7791f", "tour": "#3b82a0",
 "adaptive": "#7656a6", "joint": "#118a8c",
}
SCENE_LABEL = {
 "random": "随机", "annulus": "外圈", "center": "中心",
 "hash": "非平滑误差", "worstrecv": "最小接收半径",
 "live1": "重建布局1", "live2": "重建布局2",

```

```

}

def setup_style() -> None:
 """Use paper-safe sizes; all visible text stays >= 8.5 pt."""
 from matplotlib import font_manager
 font = ROOT / "paper" / "fonts" / "simsun.ttc"
 font_manager.fontManager.addfont(str(font))
 matplotlib.rcParams["font.family"] = [font_manager.FontProperties(fname=font).get_name(), "Times New Roman"]
 matplotlib.rcParams["mathtext.fontset"] = "stix"
 matplotlib.rcParams.update({
 "font.size": 9.0,
 "axes.titlesize": 9.2,
 "axes.labelsize": 9.0,
 "xtick.labelsize": 8.5,
 "ytick.labelsize": 8.5,
 "legend.fontsize": 8.5,
 "axes.spines.top": False,
 "axes.spines.right": False,
 "axes.unicode_minus": False,
 "pdf.fonttype": 42,
 "ps.fonttype": 42,
 })

def load_data(data_dir: Path) -> tuple[dict[str, Any], list[dict[str, Any]]]:
 path = data_dir / "paired_results.json"
 if not path.exists():
 raise FileNotFoundError(f"冻结数据不存在: {path}")
 obj = json.loads(path.read_text(encoding="utf-8"))
 rows = obj.get("rows")
 if rows is None:
 # Backward-compatible reader for the older nested benchmark files.
 rows = []
 for scenario, group in obj.get("scenarios", {}).items():
 for old in group.get("rows", []):
 seed = old.get("seed")
 for policy in POLICIES:
 if policy in old:
 r = dict(old[policy])
 r.update({"scenario": scenario, "seed": seed,
 "policy": policy, "split": "main"})
 rows.append(r)
 if not isinstance(rows, list) or not rows:
 raise ValueError("paired_results.json 中没有可绘图的 rows")
 return obj, rows

def val(row: dict[str, Any], *names: str, default: Any = None) -> Any:
 """Look up a scalar in metrics/report/audit and then at row level."""
 buckets: list[Any] = [row.get("metrics"), row.get("report"),
 row.get("audit"), row]
 for name in names:
 for bucket in buckets:
 if isinstance(bucket, dict) and name in bucket:
 return bucket[name]
 return default

def num(row: dict[str, Any], *names: str, default: float = float("nan")) -> float:
 x = val(row, *names, default=default)
 try:

```

```

 return float(x)
 except (TypeError, ValueError):
 return default

def policy_of(row: dict[str, Any]) -> str:
 return str(row.get("policy", row.get("strategy", ""))).lower()

def scene_of(row: dict[str, Any]) -> str:
 return str(row.get("scenario", row.get("scene", "random"))).lower()

def is_ok(row: dict[str, Any]) -> bool:
 return str(row.get("status", "ok")).lower() in {"ok", "complete", "success", ""} and not row.get("error")

def time_value(row: dict[str, Any]) -> float:
 return num(row, "virtual_time_s", "exit_s", "total_time_s", "exit_time_s", "T_s")

def trace_path(row: dict[str, Any], data_dir: Path) -> Path | None:
 p = row.get("trace_file")
 if not p:
 return None
 q = Path(str(p))
 return q if q.is_absolute() else data_dir / q

def first_trace(rows: list[dict[str, Any]], policies: Iterable[str], data_dir: Path,
 scenario: str = "random") -> dict[str, dict[str, Any]]:
 out: dict[str, dict[str, Any]] = {}
 policies = list(policies)
 candidates = [r for r in rows if is_ok(r) and scene_of(r) == scenario and
 str(r.get("split", "main")) == "main" and int(r.get("index", -1)) == 0]
 # Select one key for which every requested policy has a saved trace. This
 # prevents silently comparing different seeds when one policy failed.
 groups: dict[tuple[Any, Any], dict[str, dict[str, Any]]] = {}
 for row in candidates:
 p = trace_path(row, data_dir)
 if not p or not p.exists():
 continue
 key = (row.get("index", 0), row.get("seed", 0))
 groups.setdefault(key, {})[policy_of(row)] = row
 for key in sorted(groups, key=lambda z: (int(z[0]), int(z[1]))):
 if all(pol in groups[key] for pol in policies):
 for pol in policies:
 row = groups[key][pol]; p = trace_path(row, data_dir)
 out[pol] = json.loads(p.read_text(encoding="utf-8"))
 break
 return out

def actions(trace: dict[str, Any]) -> list[dict[str, Any]]:
 return [a for a in trace.get("actions", [])
 if isinstance(a, dict) and "x" in a and "y" in a]

def action_points(trace: dict[str, Any]) -> np.ndarray:
 aa = actions(trace)
 return np.asarray([[0.0, 0.0]] + [[float(a["x"]), float(a["y"])] for a in aa], float)

```

```

def source_points(trace: dict[str, Any]) -> np.ndarray:
 truth = trace.get("post_exit_truth", trace.get("truth", {})) or {}
 src = truth.get("sources", []) if isinstance(truth, dict) else []
 return np.asarray([[float(s["x_m"]), float(s["y_m"])] for s in src
 if "x_m" in s and "y_m" in s], float)

def route_legend():
 return [Line2D([], [], color="#d7a06b", marker="o", ls="", ms=3, label="检测点"),
 Line2D([], [], color="#b33e36", marker="x", ls="", ms=4.5, label="成功清除"),
 Line2D([], [], color="#b7791f", marker="^", mfc="none", ls="", ms=4, label="清除失败"),
 Line2D([], [], color="#424b54", marker="*", ls="", ms=5, label="源位(事后)"),
 Line2D([], [], color="#252a35", marker="s", ls="", ms=3.5, label="起点")]

def draw_route(ax, trace: dict[str, Any], color: str, title: str,
 annotate_sources: bool = True, show_ylabel=True) -> None:
 p = action_points(trace)
 if len(p) > 1:
 ax.plot(p[:, 0], p[:, 1], color=color, lw=1.05, zorder=3)
 aa = actions(trace)
 m = np.asarray([[a['x'], a['y']] for a in aa if a.get('kind') == 'measure'])
 if len(m):
 m = np.unique(np.round(m, 2), axis=0)
 ax.scatter(m[:, 0], m[:, 1], s=6, color='#d7a06b', alpha=.6, zorder=4)
 src = source_points(trace)
 if annotate_sources and len(src):
 ax.scatter(src[:, 0], src[:, 1], marker='*', s=26, color='#424b54', zorder=6)
 for success, marker, size in ((True, 'x', 20), (False, '^', 16)):
 q = np.asarray([[a['x'], a['y']] for a in aa if a.get('kind') == 'clear' and
 (a.get('clear_result') == 'success') == success])
 if len(q):
 if success:
 ax.scatter(q[:, 0], q[:, 1], marker=marker, s=size,
 color='#b33e36', linewidth=.8, zorder=7)
 else:
 ax.scatter(q[:, 0], q[:, 1], marker=marker, s=size,
 facecolors='none', edgecolors='#b7791f', linewidth=.8, zorder=8)
 th = np.linspace(0, 2 * math.pi, 400)
 ax.plot(1800*np.cos(th), 1800*np.sin(th), '--', color='#aeb5bd', lw=.65)
 ax.scatter([0], [0], marker='s', s=13, color='#252a35', zorder=9)
 met = trace.get('metrics', {})
 t = met.get('exit_s', trace.get('report', {})).get('virtual_time_s', 0)
 length = met.get('travel_m', trace.get('report', {})).get('travel_m', 0)
 n = met.get('cleared', 0); total = met.get('n_sources', 0)
 ax.set_title(f'{title}\nT={t:.0f} s; L={length/1000:.2f} km\n清除 {n}/{total}',
 fontsize=9, linespacing=1.25, pad=5)
 ax.set(xlim=(-1950, 1950), ylim=(-1950, 1950), xlabel='x / m')
 if show_ylabel: ax.set_ylabel('y / m')
 else: ax.tick_params(labelleft=False)
 ax.set_xticks([-1000, 0, 1000]); ax.set_yticks([-1000, 0, 1000])
 ax.set_aspect('equal'); ax.grid(alpha=.17, lw=.4)

def save_figure(fig: plt.Figure, stem: str) -> None:
 OUT.mkdir(parents=True, exist_ok=True)
 # Exact print canvas; individual builders reserve their own text margins.
 fig.savefig(OUT / f'{stem}.pdf')
 fig.savefig(OUT / f'{stem}.png', dpi=DPI)
 plt.close(fig)

```

```

def fig1_geometry(rows, data_dir) -> None:
 """Two deterministic geometric diagrams, explicitly independent of data."""
 fig, (ax, bx) = plt.subplots(1, 2, figsize=(FIG_W, 57*MM))
 fig.subplots_adjust(left=.02, right=.98, bottom=.04, top=.84, wspace=.16)
 ax.set_title('(a) 单元整体排除: 三角不等式', fontsize=9.5)
 ax.set_xlim(-3.9, 1.7); ax.set_ylim(-1.4, 2.05); ax.set_aspect('equal'); ax.axis('off')
 s = np.array([-3., 0.]); q = np.array([0., 0.]); x = np.array([.65, .65])
 ax.add_patch(Circle(s, 3.98, fill=False, lw=1.2, ec='#3b82a0'))
 ax.add_patch(Rectangle((-0.65, -0.65), 1.3, 1.3, fc='#e3f0f1', ec='#118a8c'))
 ax.plot([s[0], q[0], x[0]], [s[1], q[1], x[1]], color='#118a8c', lw=1.2)
 ax.plot([s[0], x[0]], [s[1], x[1]], ls='--', color='#b7791f', lw=1)
 ax.plot(s[0], s[1], 's', color='#252a35', ms=4)
 ax.plot(q[0], q[1], 'o', color='#118a8c', ms=3)
 ax.plot(x[0], x[1], 'o', color='#b7791f', ms=3)
 ax.text(-2.9, -.60, 's: 无信号测点', ha='center')
 ax.text(.03, -.34, 'q', ha='center'); ax.text(.75, .74, 'x')
 ax.text(-1.55, -.22, r'$\|q-s\|\leq r_{\rm cert}$', ha='center')
 ax.text(.20, 1.13, r'$\|x-q\|\leq h/\sqrt{2}$', ha='center')
 ax.text(-2.95, 1.5, r'$R_{\min}=1000$ m', color='#3b82a0')
 ax.text(-.95, -1.13, r'$r_{\rm cert}=1000-h/\sqrt{2}-\varepsilon$', ha='center')
 bx.set_title('(b) 测站滑移: 保留独占覆盖职责', fontsize=9.5)
 bx.set_xlim(-.4, 5.2); bx.set_ylim(-.95, 3.8); bx.set_aspect('equal'); bx.axis('off')
 a=np.array([0., 0.]); b=np.array([4.8, 0.]); old=np.array([2.2, 2.0])
 centers=[np.array([1.4,2.4]),np.array([2.95,2.0])]; radius=1.7
 v=np.array([0.,-2.]); delta=np.array(centers)-old
 bb=-2*delta*v; cc=np.sum(delta**2,axis=1)-radius**2
 tt=min(1., float(np.min((-bb+np.sqrt(bb**2-4*(v@v)*cc))/(2*(v@v))))))
 new=old+tt*v
 for c in centers:
 bx.add_patch(Circle(c,radius,fill=False,ec='#cbd6dc',lw=.7,ls=':'))
 bx.add_patch(Rectangle(c-.12,.24,.24,fc='#bededb',ec='#118a8c',lw=.7))
 bx.plot([a[0], b[0]], [0,0], color='#aeb5bd', lw=.8)
 bx.plot([a[0],old[0],b[0]], [0,old[1],0], '--', color='#b7791f', lw=1)
 bx.plot([a[0],new[0],b[0]], [0,new[1],0], color='#118a8c', lw=1.6)
 bx.annotate(' ',xy=new,xytext=old,arrowprops=dict(arrowstyle='->',color='#252a35',lw=1))
 bx.plot(old[0],old[1], 'o', mfc='white', mec='#b7791f', ms=5)
 bx.plot(new[0],new[1], 'o', color='#118a8c', ms=4)
 bx.text(old[0]+.50,old[1]+.50,'原站 p_0')
 bx.text(new[0]+.18,new[1]+.14,'新站 $p(t)$')
 bx.text(.25,3.1,'方格: 必须保留的单元中心 Q')
 bx.text(2.4,-.6,r'$\forall q\in Q:\|p(t)-q\|\leq r_{\rm cert}$',ha='center')
 bx.text(0,-.30,'a'); bx.text(4.8,-.30,'b')
 save_figure(fig,'q3-geometry-certificate')

def route_comparison(rows, data_dir, selected, stem):
 trs=first_trace(rows,POLICIES,data_dir)
 if len(trs)!=5: raise ValueError('固定首局的五策略轨迹必须齐全')
 fig,axes=plt.subplots(1,3,figsize=(FIG_W,65*MM))
 fig.subplots_adjust(left=.09,right=.985,bottom=.25,top=.70,wspace=.12)
 for i,pol in enumerate(selected):
 draw_route(axes[i],trs[pol],POLICY_COLOR[pol],f'({chr(97+i)}) {pol}',show_ylabel=i==0)
 fig.legend(handles=route_legend(),loc='lower center',ncol=5,frameon=False,
 bbox_to_anchor=(.5,.005),columnspacing=.7,handletextpad=.4,handlelength=1.0)
 save_figure(fig,stem)

def fig2_field_sweep_tour(rows,data_dir):
 route_comparison(rows,data_dir,['field','sweep','tour'],'q3-route-field-sweep-tour')

```

```

def fig3_tour_adaptive_joint(rows,data_dir):
 route_comparison(rows,data_dir,['tour','adaptive','joint'],'q3-route-tour-adaptive-joint')

def audit_from_actions(trace: dict[str, Any]) -> dict[str, float]:
 aa = actions(trace)
 if not aa:
 return {"travel": 0., "measure": 0., "switch": 0., "clear": 0., "time": 0., "cleared": 0.}
 prev = np.array([0., 0.]); channel = 1; parts = {"travel": 0., "measure": 0., "switch": 0., "clear": 0.}
 cleared = 0
 for a in aa:
 q = np.array([float(a["x"]), float(a["y"])])
 d = float(np.linalg.norm(q - prev)); parts["travel"] += d / 5.; prev = q
 if a.get("kind") == "measure":
 parts["measure"] += 5.; parts["switch"] += float(int(a.get("channel", channel)) != channel)
 channel = int(a.get("channel", channel))
 elif a.get("kind") == "clear":
 ok = a.get("clear_result") == "success"; parts["clear"] += 5. if ok else 3.; cleared += int(ok)
 parts["time"] = sum(parts[k] for k in ("travel", "measure", "switch", "clear"))
 parts["cleared"] = float(cleared)
 return parts

def fig4_cost_and_clear(rows: list[dict[str, Any]], data_dir: Path) -> None:
 main=[r for r in rows if str(r.get('split','main'))=='main' and scene_of(r)=='random' and policy_of(r) in POLICIES]
 fig,(ax,bx)=plt.subplots(1,2,figsize=(FIG_W,70*MM))
 fig.subplots_adjust(left=.13,right=.985,bottom=.25,top=.88,wspace=.45)
 parts=['t_travel_s','t_measure_s','t_switch_s','t_clear_s']
 cols=['#3b82a0','#d97736','#7656a6','#55a868']; labels=['移动','检测','切频','清除']
 means=np.array([[np.mean([num(r,key) for r in main if policy_of(r)==pol]) for key in parts] for pol in POLICIES])
 y=np.arange(5); left=np.zeros(5)
 for k in range(4):
 ax.barh(y,means[:,k],left=left,color=cols[k],label=labels[k],height=.60); left+=means[:,k]
 for i,t in enumerate(left):ax.text(t+60,i,f'{t:.0f}',va='center',fontsize=8.5)
 ax.set_yticks(y,POLICIES);ax.invert_yaxis();ax.set_xlim(0,left.max()*1.18)
 ax.set_xlabel('平均完整时间 / s');ax.set_title('(a) 随机50局: 动作时间分解',fontsize=9.5)
 ax.grid(axis='x',alpha=.16,lw=.4)
 fig.legend(*ax.get_legend_handles_labels(),frameon=False,ncol=4,loc='lower center',
 bbox_to_anchor=(.31,.01),columnspacing=.65,handlelength=.8,handletextpad=.35)
 trs=first_trace(rows,POLICIES,data_dir)
 for i,pol in enumerate(POLICIES):
 tr=trs[pol];aa=actions(tr)
 ts=[float(a['virtual_time_s']) for a in aa if a.get('kind')=='clear' and a.get('clear_result')=='success'
]
 tend=float(tr.get('metrics',{}).get('exit_s',aa[-1]['virtual_time_s']))
 bx.step([0.]+ts+[tend],[0]+list(range(1,len(ts)+1))+[len(ts)],where='post',
 color=POLICY_COLOR[pol],lw=1.2,ls=['-','-','-.',':','-'][i],label=pol)
 bx.plot(tend,len(ts),'s',ms=3.5,color=POLICY_COLOR[pol],zorder=6)
 bx.set_xlabel('虚拟时间 / s',ylabel='累计清除数',ylim=(-.3,13.8))
 bx.set_title('(b) 固定首局: 清除进度',fontsize=9.5)
 bx.text(.97,.96,'■ 为退出时刻',transform=bx.transAxes,ha='right',va='top',fontsize=8.5)
 bx.set_xticks([0,2000,4000]);bx.set_yticks([0,4,8,12]);bx.grid(alpha=.16,lw=.4)
 bx.legend(frameon=False,ncol=2,loc='lower right',handlelength=1.2,columnspacing=.6,handletextpad=.35)
 save_figure(fig,'q3-time-cost-clearance')

def fig5_scenarios_ablation(rows: list[dict[str,Any]]) -> None:
 main=[r for r in rows if str(r.get('split','main'))=='main' and policy_of(r) in POLICIES]
 scenes=['random','annulus','center','hash','worstrecv']
 mat=np.array([[np.mean([time_value(r) for r in main if scene_of(r)==sc and policy_of(r)==p]) for p in

```

```

POLICIES] for sc in scenes])
bad=np.array([[any(not val(r,'full_clear',default=False) for r in main if scene_of(r)==sc and policy_of(r)==p
) for p in POLICIES] for sc in scenes])
ratio=mat/mat[:,2,None]
fig,(ax,bx)=plt.subplots(2,1,figsize=(FIG_W,92*MM),gridspec_kw={'height_ratios':[1.1,1]})
fig.subplots_adjust(left=.19,right=.89,bottom=.14,top=.92,hspace=.85)
from matplotlib.colors import TwoSlopeNorm
norm=TwoSlopeNorm(vmin=min(.7,float(ratio.min())),vcenter=1.,vmax=max(1.5,float(ratio.max())))
cmap=plt.get_cmap('RdYlBu_r');im=ax.imshow(ratio,cmap=cmap,norm=norm,aspect='auto')
ax.set_xticks(range(5),POLICIES);ax.set_yticks(range(5),[SCENE_LABEL[s] for s in scenes])
ax.set_title('(a) 五类场景: 平均完整时间 (s)',fontsize=9.5,pad=6)
for i in range(5):
 for j in range(5):
 rgba=cmap(norm(ratio[i,j]));lum=.299*rgba[0]+.587*rgba[1]+.114*rgba[2]
 ax.text(j,i,f'{mat[i,j]:.0f}'+('*' if bad[i,j] else ''),ha='center',va='center',fontsize=9,color='
white' if lum<.48 else '#1f2933')
cax=fig.add_axes([.91,.632,.015,.28]); cb=fig.colorbar(im,cax=cax,ticks=[.75,1,1.5]);cb.ax.tick_params(
 labelsz=8.5)
ax.text(0,-.39,'颜色: 相对 tour 耗时; *: 该组存在未全清局',transform=ax.transAxes,fontsize=8.5)
base={(scene_of(r),r['seed']):r for r in main if policy_of(r)=='joint'}
variants=['joint_no_after_service','joint_single_plan','joint_no_cover_polish','joint_no_probability_gate']
labels=['取消先清后测','单一规划权重','取消测站精修','取消概率筛选']
values=[];ci=[]
for p in variants:
 ab=[r for r in rows if r.get('split')=='ablation' and policy_of(r)==p and scene_of(r)=='random']
 pair=np.array([[time_value(base[(scene_of(r),r['seed'])]),time_value(r)] for r in ab])
 if len(pair)!=20:raise ValueError(f'{p}: requires 20 paired ablations, got {len(pair)}')
 values.append(100*(pair[:,1].mean()/pair[:,0].mean()-1))
 rng=np.random.default_rng(918120);idx=rng.integers(0,len(pair),(4000,len(pair)))
 means=pair[idx].mean(axis=1);ci.append(np.percentile(100*(means[:,1]/means[:,0]-1),[2.5,97.5]))
values=np.asarray(values);ci=np.asarray(ci)
bx.bar(range(4),values,color='#5b7f99',width=.50)
bx.errorbar(range(4),values,yerr=[values-ci[:,0],ci[:,1]-values],fmt='none',ecolor='#263846',capsize=3,lw=1)
bx.axhline(0,color='#767e87',lw=.65)
for i,v in enumerate(values):bx.annotate(f'{v:+.2f}%',(i,ci[i,1]),xytext=(0,3),textcoords='offset points',ha=
'center',fontsize=8.5)
bx.set_xticks(range(4),labels);bx.set_ylabel('耗时增加 / %')
bx.set_title('(b) 配对20局消融: 均值及95%区间',fontsize=9.5,pad=6)
bx.set_ylim(min(-.8,float(ci.min())-.6),float(ci.max())+1.0)
bx.grid(axis='y',alpha=.16,lw=.4)
save_figure(fig,'q3-scenarios-ablation')

def main() -> None:
 global OUT
 ap = argparse.ArgumentParser()
 ap.add_argument("--data", type=Path, default=DATA)
 ap.add_argument("--out", type=Path, default=OUT)
 args = ap.parse_args()
 OUT = args.out
 setup_style()
 obj, rows = load_data(args.data)
 fig1_geometry(rows, args.data)
 fig2_field_sweep_tour(rows, args.data)
 fig3_tour_adaptive_joint(rows, args.data)
 fig4_cost_and_clear(rows, args.data)
 fig5_scenarios_ablation(rows)
 # Full raw trajectories remain available in the frozen evidence directory.
 print(f"wrote five Q3 paper figures to {OUT} (width={160} mm, dpi={DPI})")

if __name__ == "__main__":

```

```
main()
```

## 5.18 scripts/make\_paper\_trajectory\_atlas.py

```
"""Render every frozen main-experiment scene as a five-policy atlas page.
```

This command reads paired\_results.json and its existing trace files. It never loads a policy class, reruns a simulation, or rewrites experiment records.

```
python B_locator/Q3/scripts/make_paper_trajectory_atlas.py
"""
from __future__ import annotations

import argparse
import json
from collections import defaultdict
from pathlib import Path

import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt
from matplotlib.backends.backend_pdf import PdfPages

from make_paper_figures import (DATA, MM, POLICIES, POLICY_COLOR, SCENE_LABEL,
 draw_route, route_legend, setup_style)

SCENES = ["random", "annulus", "center", "hash", "worstrecv"]

def load_groups(data: Path) -> list[tuple[tuple[str, int, int], dict]]:
 records = json.loads((data / "paired_results.json").read_text(encoding="utf-8"))
 groups = defaultdict(dict)
 for row in records["rows"]:
 if row["split"] != "main":
 continue
 key = (row["scenario"], int(row["index"]), int(row["seed"]))
 policy = row["policy"]
 if policy in groups[key]:
 raise ValueError(f"Duplicate record for {key}/{policy}")
 groups[key][policy] = row
 for key, rows in groups.items():
 if set(rows) != set(POLICIES):
 raise ValueError(f"Incomplete five-policy group: {key}")
 return sorted(groups.items(), key=lambda x: (SCENES.index(x[0][0]), x[0][1], x[0][2]))

def load_traces(data: Path, rows: dict) -> dict:
 traces = {}
 for policy in POLICIES:
 source = Path(rows[policy]["trace_file"])
 if not source.is_absolute():
 source = data / source
 trace = json.loads(source.read_text(encoding="utf-8"))
 if trace["policy"] != policy:
 raise ValueError(f"Trace policy mismatch: {source}")
 traces[policy] = trace
 for field in ("scene_fingerprint", "error_fingerprint"):
 if len({t["scene_manifest"][field] for t in traces.values()}) != 1:
 raise ValueError(f"The five policies do not share {field}")
 return traces
```



```

def atlas_page(key: tuple[str, int, int], rows: dict, traces: dict, page: int, total: int):
 scenario, index, seed = key
 fig, axes = plt.subplots(2, 3, figsize=(297 * MM, 210 * MM))
 fig.subplots_adjust(left=.07, right=.97, bottom=.12, top=.82, wspace=.10, hspace=.72)
 for i, policy in enumerate(POLICIES):
 ax = axes.flat[i]
 draw_route(ax, traces[policy], POLICY_COLOR[policy], f"({chr(97+i)}) {policy}",
 show_ylabel=(i % 3 == 0))
 ax = axes[1, 2]
 ax.axis("off")
 ax.text(.04, 1.12, "图例与读取说明", transform=ax.transAxes, fontsize=11, va="top")
 ax.legend(handles=route_legend(), loc="upper left", bbox_to_anchor=(.015, 1.02),
 frameon=False, fontsize=10, handlelength=1.4, labelspace=.8)
 ax.text(.04, .33, "同页共用源位、接收半径及误差场。\\n"
 "T 为从进入到退出的完整虚拟时间；\\n"
 "L 为动作账本记录的总行程。\\n"
 "源位仅在退出之后用于绘图。\\n"
 "未全清的运行照常保留。", transform=ax.transAxes,
 fontsize=9.5, va="top", linespace=1.6)
 fig.text(.5, .955, "问题三补充材料：五策略同场景轨迹对照", ha="center", fontsize=16)
 fig.text(.5, .908, f"{SCENE_LABEL.get(scenario, scenario)} * 第 {index+1} 组 * seed {seed}",
 ha="center", fontsize=12)
 fig.text(.07, .025, f"冻结记录: main_{scenario}_{index:03d}_{seed}_{policy}.json",
 fontsize=8.5, color="#626d78")
 fig.text(.97, .025, f"{page} / {total}", ha="right", fontsize=9.5)
 return fig

def main() -> None:
 parser = argparse.ArgumentParser(description=__doc__)
 parser.add_argument("--data", type=Path, default=DATA)
 parser.add_argument("--out", type=Path, default=DATA / "supplementary_trajectories.pdf")
 args = parser.parse_args()
 setup_style()
 groups = load_groups(args.data)
 if len(groups) != 120:
 raise ValueError(f"Expected 120 main-experiment scenes, found {len(groups)}")
 args.out.parent.mkdir(parents=True, exist_ok=True)
 with PdfPages(args.out, metadata={"Title": "Q3 Supplementary Trajectories",
 "Subject": "120 frozen paired scenes; five policies per page",
 "Author": "", "CreationDate": None, "ModDate": None}) as pdf:
 for page, (key, rows) in enumerate(groups, 1):
 traces = load_traces(args.data, rows)
 fig = atlas_page(key, rows, traces, page, len(groups))
 pdf.savefig(fig)
 plt.close(fig)
 if page % 20 == 0:
 print(f"Rendered {page}/{len(groups)} frozen scenes", flush=True)
 print(args.out)

if __name__ == "__main__":
 main()

```

## 附录 F 问题四参数、复现数据与程序

### 6.1 参数与连续证书细分

表 15 问题四最终严格档的主要参数

| 参数      | 取值与作用                                      |
|---------|--------------------------------------------|
| 严格布局    | 原点、998 m 八点环、1867 m 十二点环（相位 $15^\circ$ ）   |
| 规划用覆盖网格 | 40 m 方格、48 个带角余量的方向区间                      |
| 最终无源核验  | 连续圆弧精确合并；初始 40 m，仅细分未证明格，最细允许 0.625 m      |
| 角楔与圆外包  | 半宽 $(1.005 + 10^{-8})^\circ$ ，每圆 32 条外切半平面 |
| 定位横向基线  | 前进比例 0.6、侧移上限 100 m、下限 22 m；缩放 1, 1/2, 1/4 |
| 逼近及清除覆盖 | 最多 8 轮；小区域阈值 75 m；兜底方格间距 28 m              |
| 单方位沿站补测 | 与上次正测站相距至少 80 m、距估计中心不超过 1450 m            |
| 后验中心    | 距离 150 点、接收半径 9 点、相容朝向 36 点，全向先验 0.5       |
| 航路局部搜索  | 上轮热启动加至多 5 个最近邻起点，至多 4 轮翻转与移位              |
| 信息参考状态  | 1024 个；独立种子 170120；代价下限系数 0.35             |
| 虚拟时钟上限  | 360000 s；预算不足不宣称严格完成                       |

定位圆外包和双侧负观测截断保持真源包含性，后验中心与信息代价只影响动作选择。实际记录的发射方向字段采用“测点指向源”的比较约定；(39) 使用物理发射方向，作图时将该字段转过  $180^\circ$ 。覆盖证明遍历全部连续朝向，因而不依赖字段正反约定。

表 16 21 站布局的连续证书逐层核验

| 方格边长/m | 本层待证明方格 | 核验后未证明方格 |
|--------|---------|----------|
| 40     | 6536    | 348      |
| 20     | 1072    | 380      |
| 10     | 1112    | 252      |
| 5      | 624     | 172      |
| 2.5    | 336     | 0        |

后续每层只处理未证明格的子格，并删除与靶区不相交的子格。接受条件一直是整格与全部连续朝向的覆盖。某层未通过只表示未证明，不能仅凭粗网格将其判为真实盲区；源码中的点检仅用于提前拒绝明显不可行布局，不用于接受布局。

6.2 固定实验定义与统计口径

四类场景按随机混合、最小半径、贴边朝外、全向编号  $s = 0, 1, 2, 3$ ，第  $i$  组种子为  $2026091201 + 1000000s + 79i$ ， $i$  从零开始。主集 30 组，其余各 10 组。源数在 10 ~ 16 离散均匀抽取；混合集每源以概率 0.5 为定向，方向均匀。最小半径组全部定向且接收半径固定 1000 m；贴边组半径均匀取 1750 ~ 1800 m，接收半径 1000 m，发射半平面朝径向外侧；全向组作为对照。一般源位按圆域面积均匀、接收半径均匀取 1000 ~ 1500 m。同一源、同一测点误差固定，三策略的场景参数完全一致。

后验中心与信息代价各作一项开关消融，速度档取 *fast99*, *fast95*, *fast90*，均预先固定主集前 10 组，复用对应完整严格档作为配对基线。所有 230 次运行均纳入统计；图中代表轨迹固定为主集第一组，不按表现选例。场景和算法源码校验值保存在 Q4/out/paper\_q4/manifest.json。

bootstrap 重抽样 10000 次，以整组场景为抽样单位，保留策略间配对关系。版本节省率为  $1 - \bar{T}_{\text{new}}/\bar{T}_{\text{old}}$ ；消融增长率为  $\bar{T}_{\text{ablated}}/\bar{T}_{\text{complete}} - 1$ ，区间均为百分位区间，未作多重比较校正。每源时间先算各局比值再平均，不等同于总时间除以总清除数。退化的零失败样本区间不构成未来全清概率为 100% 的证据。

表 17 单因素消融：相同十组场景中的完整时间变化

| 设置     | 时间/s   | 增长/%  | 95% 区间/%     | 全清    |
|--------|--------|-------|--------------|-------|
| 关闭后验中心 | 6131.7 | +7.97 | [2.51,13.41] | 10/10 |
| 关闭信息代价 | 5826.0 | +2.59 | [-1.47,7.15] | 10/10 |

表 18 稀疏速度档：相同十组场景中的清除结果与两种时间口径

| 设置      | 已清/源数   | 全清    | 完整时间/s | 末清每源/s | 退出每源/s |
|---------|---------|-------|--------|--------|--------|
| compact | 133/133 | 10/10 | 5679.0 | 392.2  | 436.2  |
| fast99  | 133/133 | 10/10 | 5407.1 | 395.2  | 413.2  |
| fast95  | 130/133 | 8/10  | 4789.7 | 353.1  | 372.0  |
| fast90  | 129/133 | 6/10  | 4210.5 | 323.4  | 330.0  |

6.3 完整场景统计与计算开销

下表“末清每源”截止末次成功清除，“退出每源”进一步计入残留验证；计算时间为本地进程中策略执行与终止认证的墙钟耗时，不含构造初始化。

表 19 问题四三版策略在全部场景上的补充统计

| 场景/策略          | 已清/源数   | 全清    | 时间/s   | 末清每源  | 退出每源  | 检测    | 计算/s |
|----------------|---------|-------|--------|-------|-------|-------|------|
| 随机混合/grid      | 388/388 | 30/30 | 8426.1 | 552.5 | 666.7 | 237.2 | 0.09 |
| 随机混合/joint     | 388/388 | 30/30 | 6556.9 | 479.9 | 520.1 | 275.5 | 0.63 |
| 随机混合/compact   | 388/388 | 30/30 | 5819.1 | 423.7 | 461.3 | 223.0 | 4.19 |
| 最小接收半径/grid    | 126/126 | 10/10 | 8604.9 | 639.8 | 703.9 | 284.1 | 0.09 |
| 最小接收半径/joint   | 126/126 | 10/10 | 7535.2 | 602.0 | 614.8 | 329.7 | 0.52 |
| 最小接收半径/compact | 126/126 | 10/10 | 6612.2 | 509.4 | 540.1 | 264.7 | 2.43 |

| 场景/策略        | 已清/源数   | 全清    | 时间/s    | 未清每源  | 退出每源   | 检测    | 计算/s |
|--------------|---------|-------|---------|-------|--------|-------|------|
| 贴边朝外/grid    | 128/135 | 4/10  | 13185.4 | 945.6 | 1032.6 | 449.9 | 0.09 |
| 贴边朝外/joint   | 135/135 | 10/10 | 8225.0  | 599.2 | 624.9  | 391.6 | 0.44 |
| 贴边朝外/compact | 135/135 | 10/10 | 7603.1  | 571.2 | 574.7  | 381.4 | 1.07 |
| 全向/grid      | 130/130 | 10/10 | 7437.9  | 373.1 | 580.7  | 197.7 | 0.06 |
| 全向/joint     | 130/130 | 10/10 | 6013.8  | 438.6 | 473.7  | 257.9 | 0.50 |
| 全向/compact   | 130/130 | 10/10 | 5328.0  | 383.0 | 419.9  | 205.5 | 3.24 |

本次采用 Python 3.12.14、NumPy 2.3.5，六个独立进程运行，单局策略执行墙钟最大 7.55 s。并行资源争用和本机环境会改变墙钟时间，不能将其替代正式模拟器的在线程序运行时间。所有 230 份完整轨迹、65229 条动作通过独立落盘复核，移动、检测、切频、成功及失败清除的账本均吻合，拒绝动作数为零。六组已有专项检查共 37 项通过，包括连续证书、极端定位、航路及整策略回归；日志见 Q4/out/paper\_q4/checks/。

#### 6.4 文件清单及复现命令

1. Q4/src/: 各版算法与离线场景；Q4/scripts/run\_p4.py 为运行转发入口。
  2. Q4/run\_paper\_q4.py: 固定场景的配对实验、完整动作记录、统计汇总。
  3. Q4/out/paper\_q4/runs.jsonl、summary.json: 逐局指标及汇总；traces/: 230 份压缩动作与退出后真值记录。
  4. Q4/out/paper\_q4/manifest.json、audit.json: 配置、源位和源码校验信息、独立审计结果。
  5. Q4/make\_paper\_tables.py、Q4/scripts/make\_paper\_figures.py: 由同一批冻结结果生成数值与五幅正文图。
  6. Q4/PAPER\_REPRODUCE.md: 完整实验定义、运行环境、复现与解释边界。
- 从仓库根目录执行下列命令；新实验写入单独目录，避免覆盖本文冻结数据。

Listing 7: 问题四实验与论文图表的复现命令

```
python B_locator/Q4/run_paper_q4.py --workers 6 --out B_locator/Q4/out/paper_q4_reproduce
python B_locator/Q4/make_paper_tables.py
python B_locator/Q4/scripts/make_paper_figures.py
python B_locator/Q4/scripts/make_paper_source.py
bash B_locator/paper/build.sh
```

## 附录 G 问题四完整算法源程序

以下为三版策略及其公共依赖的完整源码展示副本；少量 Unicode 数学记号只在展示副本中替换为可排版形式，实际运行源码保持原样。文件清单与源码校验信息随支撑材料提供。以下逐文件列出问题四默认策略、上一版联合策略及原格网策略的本地依赖源码、运行入口与论文实验、制表和绘图程序。为避免字体缺字，仅在本排版副本中将数学符号转换为等义的 LaTeX 命令，告警符号显示为 [!], 省略无可见字形的表情呈现选择符；可执行源码未作修改。逐文件原始字节 SHA256、依赖关系及转换数量保存在支撑材料的源码清单中。

### 7.1 Q4/make\_paper\_tables.py

```
"""Generate Q4 paper numbers and tables from the frozen paired experiments."""
from pathlib import Path
import json
import numpy as np

ROOT = Path(__file__).resolve().parent
OUT = ROOT / 'out/paper_q4'
TEX = ROOT.parent / 'paper/sections'
summary = json.loads((OUT / 'summary.json').read_text(encoding='utf-8'))
runs = [json.loads(line) for line in (OUT / 'runs.jsonl').read_text(encoding='utf-8').splitlines()]
names = {'legacy': 'grid', 'joint_strict': 'joint', 'compact_strict': 'compact',
 'compact_no_posterior': '关闭后验中心', 'compact_no_information': '关闭信息代价',
 'compact_fast99': 'fast99', 'compact_fast95': 'fast95', 'compact_fast90': 'fast90'}
scenes = {'mixed_uniform': '随机混合', 'minrange': '最小接收半径', 'edge': '贴边朝外', 'omni': '全向'}

def group(policy, scenario='mixed_uniform', experiment='main'):
 return next(g for g in summary['groups'] if (g['policy'], g['scenario'], g['experiment']) == (policy,
 scenario, experiment))

def rows_file(name, rows):
 (TEX / name).write_text(' \\\n'.join(rows) + '\n', encoding='utf-8')

def p90(policy, scenario):
 return np.percentile([r['exit_s'] for r in runs if r['policy'] == policy and r['scenario'] == scenario], 90)

policies = ('legacy', 'joint_strict', 'compact_strict')
rows_file('q4-main-rows.tex', [
 f"{names[p]} & {100*g['clearance_rate']:.2f}\\% & {g['full_clear_cases']}/{g['cases']} & {g['exit_s']:.1f} & {p90(p, 'mixed_uniform'):.1f} & {g['exit_per_cleared']:.1f} & {g['travel_m']/1000:.2f}"
 for p in policies for g in [group(p)]]
rows_file('q4-stress-rows.tex', [scenes[s] + ' & ' + ' & '.join(
 f"{g['full_clear_cases']}/{g['cases']} & {g['exit_s']:.1f}" for p in policies for g in [group(p, s, 'stress')])
 for s in ('minrange', 'edge', 'omni')])
rows_file('q4-supplement-rows.tex', [
 f"{scenes[g['scenario']]}{names[g['policy']] & {g['cleared']}/{g['total']} & {g['full_clear_cases']}/{g['cases']} & {g['exit_s']:.1f} & {g['last_per_cleared']:.1f} & {g['exit_per_cleared']:.1f} & {g['measures']:.1f} & {g['runtime_s']:.2f}"
 for g in summary['groups'] if g['experiment'] in ('main', 'stress')])
extra = [g for g in summary['groups'] if g['experiment'] == 'speed']
rows_file('q4-speed-rows.tex', [
 f"{names[g['policy']] & {g['cleared']}/{g['total']} & {g['full_clear_cases']}/{g['cases']} & {g['exit_s']:.1f} & {g['last_per_cleared']:.1f} & {g['exit_per_cleared']:.1f}"
 for g in extra])
```

```

 for g in extra])
comparison = next(c for c in summary['comparisons'] if c['experiment']=='main' and c['before']=='joint_strict'
 and c['after']=='compact_strict')
complete = group('compact_strict')
all_compact = [r for r in runs if r['policy']=='compact_strict']
numbers = {
 'Saving': f"{100*comparison['saved_fraction']:.2f}",
 'SavingLow': f"{100*comparison['saved_fraction_ci'][0]:.2f}",
 'SavingHigh': f"{100*comparison['saved_fraction_ci'][1]:.2f}",
 'LastPerSource': f"{complete['last_per_cleared']:.1f}",
 'ExitTime': f"{complete['exit_s']:.1f}",
 'JointExitTime': f"{group('joint_strict')['exit_s']:.1f}",
 'ExitPerSource': f"{complete['exit_per_cleared']:.1f}",
 'ClearedTotal': str(sum(r['cleared'] for r in all_compact)),
 'SourceTotal': str(sum(r['total'] for r in all_compact)),
 'CertifiedTotal': str(sum(r['certified'] and r['certification_mode']=='continuous' for r in all_compact)),
}
ablation_rows=[]
for policy, macro in [('compact_no_posterior', 'PosteriorChange'), ('compact_no_information', 'InformationChange')
]:
 c=next(c for c in summary['comparisons'] if c['experiment']=='ablation' and c['before']==policy)
 # Transform the same paired bootstrap samples from savings/ablated to
 # increase/complete. This monotonic transform preserves percentile bounds.
 fraction=c['saved_fraction']
 change=fraction/(1-fraction)
 lo,hi=[v/(1-v) for v in c['saved_fraction_ci']]
 numbers[macro]=f"{100*change:+.2f}"
 g=group(policy,experiment='ablation')
 ablation_rows.append(f"{names[policy]} & {g['exit_s']:.1f} & {100*change:+.2f} & [{100*lo:.2f},{100*hi:.2f}]
 & {g['full_clear_cases']}/{g['cases']}"")
rows_file('q4-ablation-rows.tex',ablation_rows)
(TEX/'q4-numbers.tex').write_text('% Generated from Q4/out/paper_q4/summary.json.\n'+''.join(
 f'\\providecommand{{{QFour{k}}}}{{{v}}}\n' for k,v in numbers.items()),encoding='utf-8')
(OUT/'table_provenance.json').write_text(json.dumps({'source':'summary.json','p90_source':'runs.jsonl','numbers':
 numbers,'bootstrap':summary.get('manifest','manifest.json')},ensure_ascii=False,indent=2),encoding='utf-8')
print(json.dumps(numbers,ensure_ascii=False))

```

## 7.2 Q4/src/p4\_certificate.py

"""Conservative continuous position/orientation coverage for problem 4.

A source at  $p$  is heard at  $q$  if  $|q-p| \leq 1000$  and  $(q-p).d \geq 0$  for some unknown unit direction  $d$ . Reversing the protocol's direction convention just replaces  $d$  by  $-d$ ; this certificate includes all directions.

The disk is covered by square cells and the unit circle by angle intervals. For a cell centre  $c$ , radius  $\rho$  and orientation half width  $h$ , a measurement  $q$  certifies an entire position/orientation cell when

$$|q-c| + \rho \leq 1000,$$

$$\text{angle}(q-c, d_{\text{center}}) + h \leq \arccos(\rho / |q-c|).$$

The first inequality is the triangle inequality. The second gives  $(q-c).d \geq \rho$  for every direction in the interval, hence  $(q-p).d \geq 0$  for every  $p$  in the square. Thus there is no interpolation assumption. Only actual no\_signal replies may exclude states of an uncleared channel.

Partial coverage is a lower bound on area-times-angle coverage under a uniform position and orientation reference measure, NOT a clearance-rate guarantee or an inferred source probability. Boundary squares are assigned

```

zero area in this lower bound, unless every state has been certified.
"""
from __future__ import annotations

from dataclasses import dataclass
import math

import numpy as np

@dataclass(frozen=True)
class CertificateResult:
 complete: bool
 excluded_fraction_lower_bound: float
 n_uncovered_states: int
 n_cells: int
 n_orientations: int

class DirectionalCertificate:
 """Reusable state discretisation with strict spatial and angular margins.

 ``station_mask(q)`` returns a boolean (n_cells, n_dir) array. Union masks
 of no_signal measurements to certify an absent channel; union masks of
 intended stations to certify a static exploration layout. Use
 ``from_log(rec.meas_log)`` for an actual ChannelRec measurement log.
 Coordinates are never read from the simulator's hidden source state.
 """

 def __init__(self, step_m=60.0, n_dir=32, arena_radius_m=1800.0,
 reception_radius_m=1000.0):
 if step_m <= 0 or n_dir < 4 or arena_radius_m <= 0:
 raise ValueError("positive geometry and at least 4 orientations required")
 self.step_m = float(step_m)
 self.n_dir = int(n_dir)
 self.arena_radius_m = float(arena_radius_m)
 self.reception_radius_m = float(reception_radius_m)
 # An integer number of exact squares covers $[-R, R]^2$. Keep every
 # square intersecting the disk, including cells whose centres lie out.
 n_side = int(math.ceil(2 * self.arena_radius_m / self.step_m))
 side = 2 * self.arena_radius_m / n_side
 self.cell_side_m = side
 ax = -self.arena_radius_m + (np.arange(n_side) + 0.5) * side
 xx, yy = np.meshgrid(ax, ax)
 pts = np.column_stack((xx.ravel(), yy.ravel()))
 closest = np.maximum(np.abs(pts) - side / 2, 0)
 keep = np.sum(closest * closest, axis=1) <= self.arena_radius_m ** 2
 self.centers = pts[keep]
 self.rho = side / math.sqrt(2.0)
 self.angles = (np.arange(self.n_dir) + 0.5) * (2 * math.pi / self.n_dir)
 self.angle_half_width = math.pi / self.n_dir
 farthest = np.abs(self.centers) + side / 2
 self.interior = np.sum(farthest * farthest, axis=1) <= self.arena_radius_m ** 2
 self._weight = side ** 2 / (math.pi * self.arena_radius_m ** 2 * self.n_dir)
 self._cache = {}

 @property
 def shape(self):
 return (len(self.centers), self.n_dir)

 def empty(self):
 return np.zeros(self.shape, dtype=bool)

```

```

def station_mask(self, xy, *, cache=True):
 xy = (float(xy[0]), float(xy[1]))
 if not all(math.isfinite(v) for v in xy):
 raise ValueError("station coordinates must be finite")
 if cache and xy in self._cache:
 return self._cache[xy]
 delta = np.asarray(xy) - self.centers
 dist = np.hypot(delta[:, 0], delta[:, 1])
 valid = (dist + self.rho <= self.reception_radius_m - 1e-8) & (dist > self.rho)
 out = self.empty()
 if np.any(valid):
 a = np.arctan2(delta[valid, 1], delta[valid, 0])
 diff = np.abs((a[:, None] - self.angles + math.pi) % (2 * math.pi) - math.pi)
 half = np.arccos(self.rho / dist[valid]) - self.angle_half_width
 out[valid] = diff <= half[:, None] - 1e-12
 if cache:
 if len(self._cache) >= 256:
 self._cache.clear()
 out.flags.writeable = False
 self._cache[xy] = out
 return out

def mask_for_points(self, points):
 out = self.empty()
 for xy in points:
 out |= self.station_mask(xy)
 return out

def all_orientation_cells(self, points):
 """Certify spatial cells by exact circular-interval union.

 This stronger final check has the same strict spatial margin, but no
 angular discretisation loss. A cell is True only when the station
 arcs cover all of $[0, 2\pi]$. A False value means unproved, not a known
 blind spot. Useful for final certificates with step_m=20 or 30.
 """
 points = np.asarray(list(points), dtype=float).reshape(-1, 2)
 if not len(points):
 return np.zeros(len(self.centers), dtype=bool)
 if not np.isfinite(points).all():
 raise ValueError("station coordinates must be finite")
 delta = points[None, :, :] - self.centers[:, None, :]
 dist = np.linalg.norm(delta, axis=2)
 valid = (dist + self.rho <= self.reception_radius_m - 1e-8) & (dist > self.rho)
 half = np.arccos(np.minimum(self.rho / np.maximum(dist, self.rho), 1.0))
 angle = np.arctan2(delta[:, :, 1], delta[:, :, 0])
 start = (angle - half) % (2 * math.pi)
 end = start + 2 * half
 starts = np.concatenate((start, start - 2 * math.pi), axis=1)
 ends = np.concatenate((end, end - 2 * math.pi), axis=1)
 valid2 = np.concatenate((valid, valid), axis=1)
 # Clip wrapping arcs to the interval we certify; invalid/empty arcs
 # become a zero-width interval at 2π .
 live = valid2 & (ends > 0) & (starts < 2 * math.pi)
 starts = np.where(live, np.maximum(starts, 0), 2 * math.pi)
 ends = np.where(live, np.minimum(ends, 2 * math.pi), 0.0)
 order = np.argsort(starts, axis=1)
 starts = np.take_along_axis(starts, order, axis=1)
 ends = np.take_along_axis(ends, order, axis=1)
 reach = np.maximum.accumulate(ends, axis=1)
 # Use a small negative tolerance: tiny numerical gaps never establish

```



```

coverage. Initial/final equality is clipping at exact 0 and 2*pi.
Placeholders at 2*pi should not count as a required join; their end
is zero so they cannot extend the actual covered interval.
joined = np.all((starts[:, 1:] == 2 * math.pi)
 | (starts[:, 1:] <= reach[:, :-1] - 1e-12), axis=1)
return (starts[:, 0] == 0) & joined & (reach[:, -1] == 2 * math.pi)

def continuous_complete(self, points):
 return bool(np.all(self.all_orientation_cells(points)))

def from_log(self, meas_log):
 """Accept ChannelRec entries (x, y, time, result) or action dicts."""
 out = self.empty()
 for rec in meas_log:
 if isinstance(rec, dict):
 result = rec.get("measure_result", rec.get("result"))
 xy = (rec.get("x", rec.get("x_m")), rec.get("y", rec.get("y_m")))
 else:
 xy, result = rec[:2], rec[3]
 if result == "no_signal":
 out |= self.station_mask(xy)
 return out

def summarize(self, mask):
 mask = np.asarray(mask, dtype=bool)
 if mask.shape != self.shape:
 raise ValueError(f"expected coverage shape {self.shape}, got {mask.shape}")
 left = int(mask.size - np.count_nonzero(mask))
 frac = 1.0 if left == 0 else float(np.count_nonzero(mask[self.interior]) * self._weight)
 return CertificateResult(left == 0, frac, left, len(self.centers), self.n_dir)

def gap_witnesses(self, mask, limit=20):
 """Representative uncertain (x,y,d_deg), useful for gap planning.

 These are centres of unproved cells, not asserted physical blind spots.
 In particular a centre can lie just outside the arena on boundary cells.
 """
 ii, jj = np.nonzero(~np.asarray(mask, dtype=bool))
 if len(ii) > int(limit):
 take = np.linspace(0, len(ii) - 1, int(limit), dtype=int)
 ii, jj = ii[take], jj[take]
 return [(float(self.centers[i, 0]), float(self.centers[i, 1]),
 math.degrees(float(self.angles[j]))) for i, j in zip(ii, jj)]

def gap_candidates(self, mask, limit=32, offset_m=650.0):
 """Stations which individually certify a representative missing state."""
 offset = float(offset_m)
 if not (self.rho / math.cos(self.angle_half_width) < offset
 < self.reception_radius_m - self.rho):
 raise ValueError("offset cannot certify a whole position/orientation cell")
 return [(x + offset * math.cos(math.radians(a)),
 y + offset * math.sin(math.radians(a)))
 for x, y, a in self.gap_witnesses(mask, limit)]

def ring_layout(rings, include_origin=True):
 """Generate (radius,count,phase_deg) rings for static comparisons."""
 out = [(0.0, 0.0)] if include_origin else []
 for ring in rings:
 radius, count = float(ring[0]), int(ring[1])
 phase = math.radians(float(ring[2])) if len(ring) > 2 else 0.0
 out.extend((radius * math.cos(phase + 2 * math.pi * k / count),

```

```

 radius * math.sin(phase + 2 * math.pi * k / count))
 for k in range(count))
return out

def strict_layout():
 """25 stations incl. origin; certified at step=20, n_dir=96.

 Origin + 950 m x 6 + 950*sqrt(3) m x 6 (30 degree phase) + 1900 m x 12.
 An open nearest-neighbour/2-opt tour from the origin is about 19.49 km.
 This is a usable certified candidate layout, not a station-count optimum.
 """
 return ring_layout(((950.0, 6, 0.0), (950.0 * math.sqrt(3.0), 6, 30.0),
 (1900.0, 12, 0.0)))

def certify_channels(records, *, certificate=None):
 """Report each uncleared channel; a heard/near pending source prevents exit."""
 cert = certificate or DirectionalCertificate()
 out = {}
 for ch, rec in records.items():
 if rec.cleared:
 out[int(ch)] = {"complete": True, "reason": "cleared"}
 continue
 positive = any(entry[3] in ("direction", "near") for entry in rec.meas_log)
 result = cert.summarize(cert.from_log(rec.meas_log))
 out[int(ch)] = {"complete": bool(result.complete and not positive),
 "reason": "heard_pending" if positive else "no_signal_certificate",
 "excluded_fraction_lower_bound": result.excluded_fraction_lower_bound,
 "n_uncovered_states": result.n_uncovered_states}
 return out

```

### 7.3 Q4/src/p3\_arena.py

"""B 题问题 3：机器狗与模拟器之间的统一接口层。

两个后端对上层暴露\*\*完全相同的接口与返回字段\*\*（字段名与模拟器 JSON 一致），  
因此 :mod:`p3\_robot` 里的策略代码在"离线演练"和"真实模拟器"上完全一致：

```

* :class:`MockArena` -- 完全离线复刻附件 1/附件 2 的物理与计时规则。
 用于自检、调参、批量演练（可以跑几百局，正式测试只有 3 次机会）。
* :class:`HttpArena` -- 真实模拟器的 HTTP+JSON 客户端，负责协议里最容易踩的坑：
 串行发送、``request_id`` 幂等与重试、``accepted`` 与 HTTP 状态双重校验、
 ``remaining_real_duration_s`` 动态预算、自身行为日志（论文支撑材料需要）。

```

计时规则（附件 1 表 1 / 附件 2 第 4 节）

```

* ``/measure``: 移动距离/5 + （频道变化 ? 1 : 0） + 5 秒；只有 ``/measure`` 会切换频道；
* ``/clear``: 移动距离/5 + （成功 5 / 未发现 3）秒；**不切换频道**；
* ``/enter``、``/exit`` 不推进虚拟时钟；
* 示向度误差在 $[-1^\circ, 1^\circ]$ 内，**同一地点固定**（本模块用"平滑随机场 + 局地量化项"建模）。
"""

from __future__ import annotations

import json
import math
import os
import socket
import time
import urllib.error

```

```

import urllib.request
from dataclasses import dataclass, field as dc_field

import numpy as np

物理常数（题目附录 1/2、模拟器接口说明）

R_ARENA = 1800.0 # 靶区半径（m）
V_ROBOT = 5.0 # 移动速度（m/s）
T_MEASURE = 5.0 # 一次检测耗时（s）
T_SWITCH = 1.0 # 频道切换耗时（s）
T_CLEAR_OK = 5.0 # 成功清除耗时（s）：光学 3 + 激光 2
T_CLEAR_FAIL = 3.0 # 未发现目标时的光学精确定位耗时（s）
T_CLEAR = T_CLEAR_OK
NEAR_R = 5.0 # 近场阈值（m）：更近时返回 near，无示向度
R_CLEAR = 20.0 # 清除半径（m）
R_RECV_MIN, R_RECV_MAX = 1000.0, 1500.0
BEARING_ERR = 1.0 # 示向度误差界（度）
CHANNELS = tuple(range(1, 21))
程序运行时间（现实时间）上限（s）：题目附录 3，/enter 响应里由 remaining_real_duration_s 给出
DEF_BUDGET_S = 1200.0
虚拟世界限时（s）：接口说明 1.4/4.5，默认 360000（100 小时）。
注意：这是虚拟时钟的上限，与上面 1200 s 的现实时间预算是两回事。
离线 mock 必须用同一个值，否则等于给离线演练加了一条真实测试里不存在的约束。
DEF_VIRTUAL_LIMIT_S = 360000.0
COORD_LIMIT = 2_000_000.0 # 接口允许的坐标绝对值上限（m）

class ArenaError(RuntimeError):
 """接口层不可恢复错误（网络重试用尽、响应无法解析等）。"""

离线模拟器

@dataclass
class MockSource:
 channel: int
 x: float
 y: float
 r_recv: float
 cone_half: float = 180.0 # 覆盖半角：180 = 全向（问题 4 会用到 <180）
 dir_deg: float | None = None
 cleared: bool = False

 @property
 def xy(self):
 return (self.x, self.y)

class MockArena:
 """离线复刻的模拟器（问题 3：全向源）。"""

 def __init__(self, seed=0, n_sources=None, budget_s=DEF_VIRTUAL_LIMIT_S,
 r_arena=R_ARENA, sources=None, error_corr_m=250.0,
 error_local_w=0.25, verbose=False):
 """`budget_s` = **虚拟时钟上限**（默认 360000 s，与真实模拟器一致）。"""
 self.seed = int(seed)
 self.rng = np.random.default_rng(self.seed)
 self.r_arena = float(r_arena)
 self.budget = float(budget_s)

```

```

self.sources = list(sources) if sources is not None else self._gen_sources(n_sources)
self._init_error_field(error_corr_m, error_local_w)
self.pos = (0.0, 0.0)
self.channel = 1
self.vtime = 0.0
self.entered = False
self.finished = False
self.verbose = verbose
self.actions: list[dict] = []
self.stats = {"n_measure": 0, "n_clear": 0, "n_clear_fail": 0,
 "travel_m": 0.0, "n_direction": 0, "n_near": 0, "n_no_signal": 0}

----- 场景生成 -----
def _gen_sources(self, n_sources=None):
 rng = self.rng
 n = int(n_sources) if n_sources else int(rng.integers(10, 17))
 n = max(1, min(n, len(CHANNELS)))
 chans = rng.choice(np.asarray(CHANNELS), size=n, replace=False)
 out = []
 for ch in chans:
 r = self.r_arena * math.sqrt(float(rng.random()))
 a = float(rng.uniform(0, 2 * math.pi))
 out.append(MockSource(channel=int(ch), x=r * math.cos(a), y=r * math.sin(a),
 r_recv=float(rng.uniform(R_RECV_MIN, R_RECV_MAX))))
 return out

def _init_error_field(self, corr_m, local_w):
 """示向度误差场：平滑随机场（模拟电磁环境）+ 局地量化项；量程严格 $\pm 1^\circ$ 。"""
 rng = self.rng
 self._waves = []
 for _ in range(12):
 lam = float(rng.uniform(max(corr_m * 1.5, 120.0), 3.0 * self.r_arena))
 ang = float(rng.uniform(0, 2 * math.pi))
 self._waves.append((2 * math.pi * math.cos(ang) / lam,
 2 * math.pi * math.sin(ang) / lam,
 float(rng.uniform(0, 2 * math.pi)),
 1.0 / lam))
 # 归一化：用靶区内抽样点的最大绝对值做标定，再乘 1.05 并截断到 ± 1
 pts = self._sample_disc(4000, rng)
 raw = self._raw_field(pts[:, 0], pts[:, 1])
 self._scale = max(float(np.max(np.abs(raw))), 1e-9) * 1.05, 1e-9
 self._local_w = float(local_w)
 self._local_seed = int(rng.integers(1, 10 ** 6))

def _sample_disc(self, n, rng):
 r = self.r_arena * np.sqrt(rng.random(n))
 a = rng.uniform(0, 2 * math.pi, n)
 return np.stack([r * np.cos(a), r * np.sin(a)], axis=1)

def _raw_field(self, x, y):
 x = np.asarray(x, float)
 y = np.asarray(y, float)
 v = np.zeros_like(x)
 for (kx, ky, ph, a) in self._waves:
 v = v + a * np.sin(kx * x + ky * y + ph)
 return v

def _local_field(self, x, y):
 i = int(math.floor(x / 25.0))
 j = int(math.floor(y / 25.0))
 h = (i * 73856093) ^ (j * 19349663) ^ (self._local_seed * 83492791)
 h &= 0x7FFFFFFF

```

```

 return (h % 200001) / 100000.0 - 1.0

def env_error(self, x, y):
 """该地点的示向度误差（度，[-1,1]，同一地点固定）。"""
 v = (float(self._raw_field(x, y)) / self._scale
 + self._local_w * self._local_field(x, y))
 return float(max(-1.0, min(1.0, v)))

----- 状态查询 -----
@property
def virtual_time_s(self):
 return self.vtime

def remaining_budget_s(self):
 return max(0.0, self.budget - self.vtime)

def budget_kind(self):
 return "virtual"

def truth(self):
 return {"n_sources": len(self.sources),
 "n_cleared": sum(1 for s in self.sources if s.cleared),
 "sources": [{"channel": s.channel, "x_m": round(s.x, 3),
 "y_m": round(s.y, 3), "r_recv_m": round(s.r_recv, 3),
 "cleared": bool(s.cleared)} for s in self.sources]}

----- 内部工具 -----
def _src_of(self, channel):
 for s in self.sources:
 if s.channel == int(channel):
 return s
 return None

def _reject(self, reason, **kw):
 r = {"accepted": False, "real_timestamp_ms": int(time.time() * 1000),
 "virtual_time_s": 0.0, "reason": reason}
 r.update(kw)
 return r

def _advance(self, x, y, dt, kind, payload):
 d = math.hypot(x - self.pos[0], y - self.pos[1])
 self.stats["travel_m"] += d
 self.vtime += dt
 self.pos = (float(x), float(y))
 rec = {"kind": kind, "x": float(x), "y": float(y), "dt": float(dt),
 "dist_m": float(d), "virtual_time_s": float(self.vtime)}
 rec.update(payload)
 self.actions.append(rec)
 if self.verbose:
 print(f" [mock] {kind} ({x:.0f},{y:.0f}) dt={dt:.1f}s "
 f"t={self.vtime:.1f}s {payload}")
 return rec

----- 四条指令 -----
def enter(self):
 if self.entered:
 return self._reject("already_entered")
 self.entered = True
 self.actions.append({"kind": "enter", "virtual_time_s": 0.0})
 return {"accepted": True, "real_timestamp_ms": int(time.time() * 1000),
 "virtual_time_s": 0.0, "max_virtual_duration_s": self.budget,
 "max_real_duration_s": self.budget,

```

```

 "remaining_real_duration_s": int(self.budget)}

def measure(self, x, y, channel):
 if not self.entered:
 return self._reject("not_entered")
 if self.finished:
 return self._reject("finished")
 x, y, ch = float(x), float(y), int(channel)
 if not (math.isfinite(x) and math.isfinite(y)) \
 or abs(x) > COORD_LIMIT or abs(y) > COORD_LIMIT:
 return self._reject("bad_position")
 if ch not in CHANNELS:
 return self._reject("bad_channel")
 switch = T_SWITCH if ch != self.channel else 0.0
 d = math.hypot(x - self.pos[0], y - self.pos[1])
 dt = d / V_ROBOT + switch + T_MEASURE
 if self.vtime + dt > self.budget + 1e-9:
 self.finished = True
 return self._reject("timeout")
 src = self._src_of(ch)
 svd = None
 if src is None or src.cleared:
 result = "no_signal"
 else:
 dist = math.hypot(x - src.x, y - src.y)
 in_cone = True
 if src.cone_half < 180.0:
 a = math.degrees(math.atan2(src.y - y, src.x - x))
 ref = src.dir_deg if src.dir_deg is not None else 0.0
 in_cone = abs((a - ref + 180.0) % 360.0 - 180.0) <= src.cone_half + 1e-12
 if dist > src.r_recv or not in_cone:
 result = "no_signal"
 elif dist <= NEAR_R:
 result = "near"
 else:
 result = "direction"
 a = math.degrees(math.atan2(src.y - y, src.x - x))
 svd = round((a + self.env_error(x, y)) % 360.0, 2)
 self.channel = ch
 self.stats["n_measure"] += 1
 self.stats["n_" + result] += 1
 self._advance(x, y, dt, "measure",
 {"channel": ch, "measure_result": result, "svd_deg": svd,
 "switch_s": switch})
 resp = {"accepted": True, "real_timestamp_ms": int(time.time() * 1000),
 "virtual_time_s": self.vtime, "measure_result": result}
 if svd is not None:
 resp["svd_deg"] = svd
 return resp

def clear(self, x, y, channel):
 if not self.entered:
 return self._reject("not_entered")
 if self.finished:
 return self._reject("finished")
 x, y, ch = float(x), float(y), int(channel)
 if not (math.isfinite(x) and math.isfinite(y)) \
 or abs(x) > COORD_LIMIT or abs(y) > COORD_LIMIT:
 return self._reject("bad_position")
 if ch not in CHANNELS:
 return self._reject("bad_channel")
 d = math.hypot(x - self.pos[0], y - self.pos[1])

```

```

src = self._src_of(ch)
ok = bool(src is not None and not src.cleared
 and math.hypot(x - src.x, y - src.y) <= R_CLEAR + 1e-9)
dt = d / V_ROBOT + (T_CLEAR_OK if ok else T_CLEAR_FAIL)
if self.vtime + dt > self.budget + 1e-9:
 self.finished = True
 return self._reject("timeout")
if ok:
 src.cleared = True
self.stats["n_clear"] += 1
if not ok:
 self.stats["n_clear_fail"] += 1
self._advance(x, y, dt, "clear",
 {"channel": ch, "clear_result": "success" if ok else "no_target_in_range"})
return {"accepted": True, "real_timestamp_ms": int(time.time() * 1000),
 "virtual_time_s": self.vtime,
 "clear_result": "success" if ok else "no_target_in_range"}

def exit(self):
 self.finished = True
 self.actions.append({"kind": "exit", "virtual_time_s": self.vtime})
 return {"accepted": True, "real_timestamp_ms": int(time.time() * 1000),
 "virtual_time_s": self.vtime, "exit_reason": "user_exit"}

真实模拟器客户端

class HttpArena:
 """模拟器 HTTP+JSON 客户端（默认 http://127.0.0.1:2026）。"""

 def __init__(self, base_url="http://127.0.0.1:2026", robot_id="",
 arena_id="default", timeout=5.0, max_retries=4, retry_delay=0.4,
 log_path=None, budget_s=DEF_BUDGET_S, safety_margin_s=3.0,
 enter_wait_s=40.0):
 self.base_url = base_url.rstrip("/")
 self.robot_id = robot_id
 self.arena_id = arena_id
 self.timeout = float(timeout)
 self.max_retries = int(max_retries)
 self.retry_delay = float(retry_delay)
 self.log_path = log_path
 self.budget = float(budget_s)
 self.safety_margin_s = float(safety_margin_s)
 self.enter_wait_s = float(enter_wait_s)
 self._n = 0
 self.entered = False
 self.pos = (0.0, 0.0)
 self.channel = 1
 self.vtime = 0.0
 self.remaining_real0 = float(budget_s)
 self._t0 = None
 self.actions: list[dict] = []
 # 与 MockArena 同名的统计（接口不返回行程，客户端自己按动作位置累加，
 # 只统计 accepted=true 的动作，与虚拟时间的推进口径一致）
 self.stats = {"n_measure": 0, "n_clear": 0, "n_clear_fail": 0,
 "travel_m": 0.0, "n_direction": 0, "n_near": 0, "n_no_signal": 0}
 if self.log_path:
 os.makedirs(os.path.dirname(os.path.abspath(self.log_path)), exist_ok=True)
 with open(self.log_path, "w", encoding="utf-8") as f:
 f.write(json.dumps({"kind": "session", "base_url": self.base_url,
 "robot_id": self.robot_id, ensure_ascii=False} + "\n")

```

```

----- HTTP -----
def _rid(self, tag):
 self._n += 1
 return f"{tag}-{self._n}"

def _base(self, rid):
 return {"arena_id": self.arena_id, "robot_id": self.robot_id, "request_id": rid}

def _post(self, path, payload, tag):
 data = json.dumps(payload, ensure_ascii=False).encode("utf-8")
 last_err = None
 for attempt in range(self.max_retries):
 req = urllib.request.Request(
 self.base_url + path, data=data,
 headers={"Content-Type": "application/json"}, method="POST")
 try:
 with urllib.request.urlopen(req, timeout=self.timeout) as r:
 body = r.read().decode("utf-8")
 resp = json.loads(body)
 self._log({"kind": "resp", "path": path, "attempt": attempt, "resp": resp})
 return resp
 except urllib.error.HTTPError as e:
 body = ""
 try:
 body = e.read().decode("utf-8", "replace")
 resp = json.loads(body)
 self._log({"kind": "http_error", "path": path, "code": e.code,
 "resp": resp})
 return resp
 except Exception:
 self._log({"kind": "http_error", "path": path, "code": e.code,
 "body": body[:500]})
 raise ArenaError(f"{path} HTTP {e.code}: {body[:200]}") from e
 except (urllib.error.URLError, socket.timeout, TimeoutError, ConnectionError) as e:
 # 网络超时/中断: 按协议重试完全相同的动作 (复用 request_id 与内容)
 last_err = e
 self._log({"kind": "net_error", "path": path, "attempt": attempt,
 "error": repr(e)})
 if attempt + 1 < self.max_retries:
 time.sleep(self.retry_delay * (attempt + 1))
 raise ArenaError(f"{path} 请求失败 (已重试 {self.max_retries} 次): {last_err!r}")

def _log(self, rec):
 rec["wall_s"] = round(time.time() - (self._t0 or time.time()), 3)
 self.actions.append(rec)
 if self.log_path:
 with open(self.log_path, "a", encoding="utf-8") as f:
 f.write(json.dumps(rec, ensure_ascii=False) + "\n")

def _new(self, path, extra, tag):
 rid = self._rid(tag)
 payload = self._base(rid)
 payload.update(extra)
 self._log({"kind": "req", "path": path, "payload": payload})
 return self._post(path, payload, tag)

----- 状态 -----
@property
def virtual_time_s(self):
 return self.vtime

```



```

def remaining_budget_s(self):
 if self._t0 is None:
 return self.remaining_real0
 used = time.monotonic() - self._t0
 return max(0.0, self.remaining_real0 - used - self.safety_margin_s)

def budget_kind(self):
 return "real"

def truth(self):
 return None

----- 四条指令 -----
def enter(self):
 """进入靶区。接口未开放时（倒计时未结束 / 测试未开始）会重试同一 request_id。

 协议规定"同一 request_id + 同一内容返回第一次的完整响应"，所以用固定的
 request_id 重试 /enter 是安全的（不会重复进入、也不会推进时钟）。
 """
 rid = self._rid("enter")
 payload = self._base(rid)
 self._log({"kind": "req", "path": "/enter", "payload": payload})
 t0 = time.monotonic()
 last = None
 keep_retries = self.max_retries
 self.max_retries = min(keep_retries, 2) # 接口未开放时缩短单次重试，尽快进入下一轮
 try:
 while True:
 try:
 resp = self._post("/enter", payload, "enter")
 except ArenaError as e:
 resp, last = None, e
 if resp is not None and resp.get("accepted") is True:
 self.entered = True
 self._t0 = time.monotonic()
 self.remaining_real0 = float(resp.get("remaining_real_duration_s",
 self.budget))
 self.vtime = float(resp.get("virtual_time_s", 0.0))
 return resp
 if resp is not None and resp.get("accepted") is False \
 and resp.get("reason") == "already_entered":
 return resp
 if time.monotonic() - t0 > self.enter_wait_s:
 if resp is not None:
 return resp
 raise ArenaError(
 f"/enter 在 {self.enter_wait_s:.0f} s 内一直失败: {last!r}")
 time.sleep(2.0)
 finally:
 self.max_retries = keep_retries

def measure(self, x, y, channel):
 resp = self._new("/measure", {"position": {"x": float(x), "y": float(y)},
 "channel": int(channel)}, "measure")
 if resp.get("accepted") is True:
 self.stats["travel_m"] += math.hypot(float(x) - self.pos[0],
 float(y) - self.pos[1])
 self.pos = (float(x), float(y))
 self.channel = int(channel)
 self.vtime = float(resp.get("virtual_time_s", self.vtime))
 res = resp.get("measure_result")
 self.stats["n_measure"] += 1

```

```

 if res in ("direction", "near", "no_signal"):
 self.stats["n_" + res] += 1
 return resp

def clear(self, x, y, channel):
 resp = self._new("/clear", {"position": {"x": float(x), "y": float(y)},
 "channel": int(channel)}, "clear")

 if resp.get("accepted") is True:
 self.stats["travel_m"] += math.hypot(float(x) - self.pos[0],
 float(y) - self.pos[1])

 self.pos = (float(x), float(y))
 self.vtime = float(resp.get("virtual_time_s", self.vtime))
 self.stats["n_clear"] += 1
 if resp.get("clear_result") != "success":
 self.stats["n_clear_fail"] += 1
 return resp

def exit(self):
 return self._new("/exit", {}, "exit")

自检

def selfcheck(verbose=True, seed=20260913):
 """核对 MockArena 是否忠实复刻附件 1/2 的规则。"""
 out = []
 ok_all = True

 def rec(name, ok, detail=""):
 nonlocal ok_all
 ok_all = ok_all and bool(ok)
 out.append({"check": name, "ok": bool(ok), "detail": detail})
 if verbose:
 print(f"[{'OK' if ok else 'FAIL'}] {name} {detail}")

 # A1 附件 1 表 2 的计时算例 ((300,400) 频道1 -> 频道2 -> /clear 未发现 -> 频道2)
 a = MockArena(seed=1, sources=[MockSource(3, 100000.0, 0.0, 1000.0)])
 a.enter()
 r1 = a.measure(300.0, 400.0, 1)
 r2 = a.measure(300.0, 400.0, 2)
 r3 = a.clear(300.0, 0.0, 3)
 r4 = a.measure(300.0, 0.0, 2)
 ok1 = (abs(r1["virtual_time_s"] - 105.0) < 1e-9
 and abs(r2["virtual_time_s"] - 111.0) < 1e-9
 and abs(r3["virtual_time_s"] - 194.0) < 1e-9
 and abs(r4["virtual_time_s"] - 199.0) < 1e-9)
 rec("A1 计时算例与附件1表2逐条一致 (105/111/194/199 s)", ok1,
 f"{r1['virtual_time_s']}, {r2['virtual_time_s']}, "
 f"{r3['virtual_time_s']}, {r4['virtual_time_s']}")

 # A2 /clear 不切换频道, 也不产生切换耗时
 b = MockArena(seed=2, sources=[MockSource(1, 0.0, 0.0, 1000.0)])
 b.enter()
 b.measure(0.0, 100.0, 1)
 t0 = b.virtual_time_s
 b.clear(0.0, 50.0, 7)
 rec("A2 /clear 不切换频道、不产生切换耗时",
 abs(b.virtual_time_s - t0 - (50.0 / V_ROBOT + T_CLEAR_FAIL)) < 1e-9
 and b.channel == 1,
 f"Δt={b.virtual_time_s - t0:.1f}s, 当前频道={b.channel}")

```

```

A3 三种检测结果与清除半径
src = MockSource(5, 0.0, 0.0, 1000.0)
c = MockArena(seed=3, sources=[src])
c.enter()
m_far = c.measure(0.0, 0.0, 5) # 距离 0 <= 5 m -> near
m_dir = c.measure(0.0, 300.0, 5) # direction (真方位 270°)
m_no = c.measure(0.0, 1200.0, 5) # 超出 1000 m -> no_signal
ok3 = (m_far["measure_result"] == "near" and m_dir["measure_result"] == "direction"
 and abs(m_dir["svd_deg"] - 270.0) <= BEARING_ERR + 1e-9
 and m_no["measure_result"] == "no_signal")
rec("A3 near / direction / no_signal 三种结果与真方位一致", ok3,
 f"near={m_far['measure_result']}, dir={m_dir.get('svd_deg')}°, "
 f"no={m_no['measure_result']}")
cl_far = c.clear(0.0, 1000.0, 5)
cl_ok = c.clear(0.0, 20.0, 5)
rec("A4 清除半径 20 m: 远处未发现、20 m 内成功",
 cl_far["clear_result"] == "no_target_in_range"
 and cl_ok["clear_result"] == "success",
 f"{cl_far['clear_result']} / {cl_ok['clear_result']}")
rec("A5 同一干扰源只能清除一次",
 c.clear(0.0, 0.0, 5)["clear_result"] == "no_target_in_range",
 "二次清除返回 no_target_in_range")

A6 误差界与"同一地点固定"
d = MockArena(seed=4)
e1 = d.env_error(123.0, -456.0)
e2 = d.env_error(123.0, -456.0)
vals = [d.env_error(float(x), float(y)) for x, y in d.sample_disc(500, d.rng)]
rec("A6 示向度误差: |e|<=1° 且同一地点固定",
 abs(e1 - e2) < 1e-12 and max(abs(v) for v in vals) <= 1.0 + 1e-12,
 f"e={e1:.4f}, 重复一致; 500 点最大 |e|={max(abs(v) for v in vals):.4f}")

A7 真源一定落在"测得方位 ±1°"的楔内 (区间算法成立的前提)
bad = 0
nchk = 0
d2 = MockArena(seed=4, budget_s=1e9)
d2.enter()
for s in d2.sources:
 for _ in range(3):
 aa = float(d2.rng.uniform(0, 2 * math.pi))
 dd = float(d2.rng.uniform(50.0, 0.9 * s.r_recv))
 x, y = s.x + dd * math.cos(aa), s.y + dd * math.sin(aa)
 r = d2.measure(x, y, s.channel)
 if r.get("accepted") is not True or r.get("measure_result") != "direction":
 continue
 nchk += 1
 true_a = math.degrees(math.atan2(s.y - y, s.x - x)) % 360.0
 if abs((true_a - r["svd_deg"] + 180.0) % 360.0 - 180.0) > BEARING_ERR + 1e-9:
 bad += 1
rec("A7 真方位与示向度之差恒 <= 1°", bad == 0 and nchk > 0,
 f"{nchk} 次检测, 越界 {bad} 次")

A8 超时拒动
e = MockArena(seed=5, budget_s=50.0, sources=[MockSource(1, 0.0, 0.0, 1000.0)])
e.enter()
e.measure(0.0, 1000.0, 1) # 200 + 5 = 205 s > 50 s
rec("A8 超出时间预算的请求 accepted=false 且不推进时钟",
 e.virtual_time_s == 0.0 and e.finished,
 f"vtime={e.virtual_time_s}")

if verbose:
 print(f"\n自检总结: {'全部通过' if ok_all else '存在失败项'}")

```

```

 return {"ok": ok_all, "checks": out}

if __name__ == "__main__":
 import argparse
 ap = argparse.ArgumentParser(description="问题3 接口层自检")
 ap.add_argument("--selfcheck", action="store_true")
 args = ap.parse_args()
 res = selfcheck()
 raise SystemExit(0 if res["ok"] else 1)

```

## 7.4 Q4/src/p1\_intersection.py

"""B 题问题 1: 示向度交会定位区域、区域直径与直径圆覆盖判定。

模型的严格表述

检测点  $S_i = (x_i, y_i)$ , 该点测得示向度  $\theta_i$  (度,  $x$  轴正向逆时针,  $[0, 360)$ ), 示向度误差有界  $|e_i| \leq \epsilon$  (题目附录 2 图 2: 实线与相邻虚线夹角均为  $1^\circ$ )。真源必在

$$R = \bigcap_i W_i, \quad W_i = \{ X : |\text{ang}(X - S_i) - \theta_i| \leq \epsilon \}$$

$W_i$  是以  $S_i$  为顶点、张角  $2\epsilon$  的\*\*角楔\*\*, 等价于两个半平面之交:

$$W_i = \{ X : a_i^- \cdot X + b_i^- \leq 0 \} \cap \{ X : a_i^+ \cdot X + b_i^+ \leq 0 \}$$

因此  $R$  是  $2k$  个半平面的交 -- \*\*凸多边形\*\*, 顶点数  $\leq 2k$ 。

算法要点 (含两处易错点的正确处理)

- 可行性 / 有界性** (先判空、再判无界, 顺序不能反)
  - 空集: 两两示向度可以相差  $> 2\epsilon$ , 但两个"错开"的角楔也可能没有公共点;
  - 无界: 存在方向  $d$  使  $|\text{ang}(d) - \theta_i| \leq \epsilon$  对所有  $i$  成立  
 $\iff$  所有示向度两两夹角  $\leq 2\epsilon$ ; 此时直径  $= +\infty$ , 不能报有限值。
- 区域顶点**: 凸集的顶点必是两条起作用约束边界直线的交点, 故枚举  $2k$  条边界直线的  $O(k^2)$  个交点、用\*\*半平面判据\*\*  $a \cdot X + b \leq 0$  筛选即可 (精确)。  
 注意: \*\*检测点本身也可能是顶点\*\* (当它落在其它所有楔内时)。旧实现在这里用"候选点到检测点的距离  $< 1e-9$  则丢弃" 处理角度奇异, 会把这种顶点误删 -- 见 ``region_by_vertices`` 的半平面判据。
- 提供两套\*\*相互独立\*\*的区域算法, 供自检交叉验证:
  - ``region_by_vertices``: 顶点枚举 + 凸包 (主算法, 精确);
  - ``region_by_clipping``: Sutherland-Hodgman 逐个半平面裁剪大框 (参考实现, 可判无界)。
- 直径**: 顶点数  $\leq \text{DIAM\_BRUTE\_LIMIT}$  时暴力枚举; 否则旋转卡壳, 两者在自检中互相校验。
- 覆盖判定**: 以直径  $AB$  为直径的圆  $C$ ,  
 $V \in C \iff \angle AVB \geq 90^\circ$  (Thales)  $\iff |V - c| \leq D/2$  ( $c$  为  $AB$  中点)。  
 一般\*\*不能覆盖\*\*: 正确做法是用\*\*最小包围圆\*\* (半径  $r \in [D/2, D/\sqrt{3}]$ , Jung 定理)。

```

"""
from __future__ import annotations
import csv
import json
import math
import os
import sys

import numpy as np

BEARING_ERR = 1.0 # 示向度误差界 (度)
ANG_TOL = 1e-9 # 角度判据容差 (度)
DIST_TOL = 1e-6 # 半平面判据容差 (米)

```

```

CLIP = 1e7 # 裁剪参考实现的外框半边长（米）
DIAM_BRUTE_LIMIT = 128 # 顶点数不超过该值时直径用暴力枚举

基本几何

def bearing(p, q):
 """p 处指向 q 的方位角（度，[0,360)）"""
 return math.degrees(math.atan2(q[1] - p[1], q[0] - p[0])) % 360.0

def ang_diff(a, b):
 """a-b 归一化到 (-180,180]"""
 return (a - b + 180.0) % 360.0 - 180.0

def wedge_halfplanes(p, theta, err=BEARING_ERR):
 """角楔 W = {X : |ang(X-p) - theta| <= err} 的两个半平面，形式 a·X + b <= 0。

 err < 90°，故 W 是两条边界直线所夹的尖楔。
 """
 p = np.asarray(p, float)
 tm = math.radians((theta - err) % 360.0)
 tp = math.radians((theta + err) % 360.0)
 dm = np.array([math.cos(tm), math.sin(tm)])
 dp = np.array([math.cos(tp), math.sin(tp)])
 # cross(dm, X-p) >= 0 <=> a1·X + b1 <= 0
 a1 = np.array([dm[1], -dm[0]])
 b1 = float(-dm[1] * p[0] + dm[0] * p[1])
 # cross(dp, X-p) <= 0 <=> a2·X + b2 <= 0
 a2 = np.array([-dp[1], dp[0]])
 b2 = float(dp[1] * p[0] - dp[0] * p[1])
 return [(a1, b1), (a2, b2)]

def region_is_unbounded(svds, err=BEARING_ERR):
 """区域无界判据：存在方向 d 与所有示向度夹角 <= err。

 返回一个这样的方向（度），有界时返回 None。
 仅在区域非空时有意义（先判空、再判无界）。
 """
 for th in svds:
 for cand in ((th - err) % 360.0, (th + err) % 360.0):
 if all(abs(ang_diff(cand, t)) <= err + ANG_TOL for t in svds):
 return cand
 return None

def _line_intersection(P, u, Q, v):
 """直线 P + t·u 与 Q + s·v 的交点，平时返回 None"""
 det = u[0] * (-v[1]) - (-v[0]) * u[1]
 if abs(det) < 1e-14:
 return None
 r = Q - P
 t = (r[0] * (-v[1]) - (-v[0]) * r[1]) / det
 return P + t * u

def convex_hull(P):
 """Andrew monotone chain, 逆时针，去掉共线点"""
 def half(pts):

```

```

 st = []
 for p in pts:
 while len(st) >= 2 and _cross2(st[-1] - st[-2], p - st[-2]) <= 0:
 st.pop()
 st.append(p)
 return st
 P = P[np.lexsort((P[:, 1], P[:, 0]))]
 lo = half(P)
 hi = half(P[::-1])
 return np.array(lo[:-1] + hi[:-1])

def _cross2(u, v):
 return float(u[0] * v[1] - u[1] * v[0])

def _dedup_hull(cand):
 """候选顶点去重 + 凸包, 返回 (P, "empty"|"degenerate"|"polygon")"""
 if len(cand) == 0:
 return np.zeros((0, 2)), "empty"
 P = np.array(cand, float)
 P = P[np.lexsort((P[:, 1], P[:, 0]))]
 ded = [P[0]]
 for v in P[1:]:
 if np.linalg.norm(v - ded[-1]) > 1e-7:
 ded.append(v)
 P = np.array(ded)
 if len(P) < 3:
 return P, "degenerate"
 H = convex_hull(P)
 if len(H) < 3:
 return H, "degenerate"
 return H, "polygon"

区域算法 1: 顶点枚举 + 凸包 (主算法, 精确)

def region_by_vertices(pts, svds, err=BEARING_ERR, tol=DIST_TOL):
 """ $R = \cap W_i$ 的顶点集合。

 R 是凸集, 其任一极点必是两条起作用约束边界直线的交点; 反之, 落在所有半平面内的
 边界直线交点必为极点 (若 $X = (Y+Z)/2$ 且两个非平行约束在 X 处取等, 则 Y, Z 也
 必须取等, 故 $Y = Z = X$)。因此枚举 + 筛选是精确的。
 """
 if len(pts) == 0:
 return np.zeros((0, 2)), "empty"
 H, lines = [], []
 for p, th in zip(pts, svds):
 H += wedge_halfplanes(p, th, err)
 for s in (-1.0, 1.0):
 a = math.radians((th + s * err) % 360.0)
 lines.append((np.asarray(p, float), np.array([math.cos(a), math.sin(a)])))
 cand = []
 n = len(lines)
 for i in range(n):
 for j in range(i + 1, n):
 X = _line_intersection(lines[i][0], lines[i][1], lines[j][0], lines[j][1])
 if X is None or not np.all(np.isfinite(X)):
 continue
 # 半平面判据: 在楔顶点处也良定义 (无角度奇异), 故检测点可作顶点保留
 if all(float(a @ X + b) <= tol for (a, b) in H):

```

```

 cand.append(X)
 return _dedup_hull(cand)

区域算法 2: 半平面裁剪 (独立参考实现, 可直接判无界)

def clip_halfplane(poly, a, b, tol=DIST_TOL):
 """Sutherland-Hodgman: 保留 {X : a·X + b ≤ 0} 的部分"""
 if not poly:
 return []
 out = []
 n = len(poly)
 for i in range(n):
 P = poly[i]
 Q = poly[(i + 1) % n]
 fp = float(a @ P + b)
 fq = float(a @ Q + b)
 if fp <= tol:
 out.append(P)
 if (fp < -tol and fq > tol) or (fp > tol and fq < -tol):
 t = fp / (fp - fq)
 out.append(P + t * (Q - P))
 ded = []
 for v in out:
 if not ded or np.linalg.norm(v - ded[-1]) > 1e-9:
 ded.append(v)
 if len(ded) > 1 and np.linalg.norm(ded[0] - ded[-1]) < 1e-9:
 ded.pop()
 return ded

def region_by_clipping(pts, svds, err=BEARING_ERR, box=CLIP, tol=DIST_TOL):
 """从大框出发逐个半平面裁剪。

 返回 (P, status): status ∈ {"empty", "degenerate", "polygon", "unbounded"};
 多边形触到外框 (|坐标| > box/2) 即判定区域无界, P 是被截断的多边形。
 """
 poly = [np.array([-box, -box], float), np.array([box, -box], float),
 np.array([box, box], float), np.array([-box, box], float)]
 for p, th in zip(pts, svds):
 for (a, b) in wedge_halfplanes(p, th, err):
 poly = clip_halfplane(poly, a, b, tol)
 if not poly:
 return np.zeros((0, 2)), "empty"
 P = np.array(poly, float)
 if len(P) < 3:
 return P, "degenerate"
 if float(np.max(np.abs(P))) > box * 0.5:
 return P, "unbounded"
 H = convex_hull(P)
 return (H if len(H) >= 3 else P), "polygon"

def region_from_bearings(pts, svds, err=BEARING_ERR):
 """主入口 (兼容旧接口): 返回 (顶点数组, status)。

 status ∈ {"empty", "unbounded", "degenerate", "bounded"}.
 """
 P, st = region_by_vertices(pts, svds, err)
 if st == "empty":
 return P, "empty"

```

```

 rec = region_is_unbounded(list(svds), err) if len(svds) else None
 if rec is not None:
 return P, "unbounded"
 if st == "degenerate":
 return P, "degenerate"
 return P, "bounded"

直径与最小包围圆

def _diameter_brute(P):
 n = len(P)
 best, pair = -1.0, (P[0], P[1])
 for i in range(n - 1):
 d = np.linalg.norm(P[i + 1:] - P[i], axis=1)
 k = int(np.argmax(d))
 if float(d[k]) > best:
 best, pair = float(d[k]), (P[i], P[i + 1 + k])
 return best, pair

def rotating_calipers(P):
 """凸多边形直径（旋转卡壳，O(n)）。P 需为逆时针凸序（允许共线点）。"""
 n = len(P)
 if n < 3:
 return diameter(P, brute_limit=2)
 k = 1
 best, pair = -1.0, (P[0], P[1])
 for i in range(n):
 i2 = (i + 1) % n
 e = P[i2] - P[i]
 while True:
 k2 = (k + 1) % n
 if abs(_cross2(e, P[k2] - P[i])) > abs(_cross2(e, P[k] - P[i])):
 k = k2
 else:
 break
 for j in (k, (k + 1) % n):
 d = float(np.linalg.norm(P[i] - P[j]))
 if d > best:
 best, pair = d, (P[i], P[j])
 d = float(np.linalg.norm(P[i2] - P[j]))
 if d > best:
 best, pair = d, (P[i2], P[j])
 return best, pair

def diameter(P, brute_limit=DIAM_BRUTE_LIMIT):
 """凸多边形直径，返回 (D, (A, B))"""
 n = len(P)
 if n == 0:
 return 0.0, None
 if n == 1:
 return 0.0, (P[0], P[0])
 if n == 2:
 return float(np.linalg.norm(P[0] - P[1])), (P[0], P[1])
 if n <= brute_limit:
 return _diameter_brute(P)
 return rotating_calipers(P)

```



```

def min_enclosing_circle(P):
 """最小包围圆 (center, radius)。顶点数很少，直接枚举 1/2/3 点候选圆。

 理论: $r \in [D/2, D/\sqrt{3}]$ (Jung 定理, 平面), 等号右端仅当区域为等边三角形时取到。
 """
 P = np.asarray(P, float)
 n = len(P)
 if n == 0:
 return None, 0.0
 if n == 1:
 return P[0].copy(), 0.0
 cands = [(P[i].copy(), 0.0) for i in range(n)]
 for i in range(n):
 for j in range(i + 1, n):
 c = 0.5 * (P[i] + P[j])
 cands.append((c, float(np.linalg.norm(P[i] - c))))
 for i in range(n):
 for j in range(i + 1, n):
 for k in range(j + 1, n):
 A, B, C = P[i], P[j], P[k]
 d = 2.0 * (A[0] * (B[1] - C[1]) + B[0] * (C[1] - A[1])
 + C[0] * (A[1] - B[1]))
 if abs(d) < 1e-12:
 continue
 aa, bb, cc = float(A @ A), float(B @ B), float(C @ C)
 ux = (aa * (B[1] - C[1]) + bb * (C[1] - A[1]) + cc * (A[1] - B[1])) / d
 uy = (aa * (C[0] - B[0]) + bb * (A[0] - C[0]) + cc * (B[0] - A[0])) / d
 c = np.array([ux, uy])
 cands.append((c, float(np.linalg.norm(A - c))))
 best = None
 for c, r in cands:
 if all(float(np.linalg.norm(P[i] - c)) <= r + 1e-9 for i in range(n)):
 if best is None or r < best[1]:
 best = (c, r)
 return best

def region_area(P):
 n = len(P)
 if n < 3:
 return 0.0
 return abs(0.5 * float(np.sum(P[:, 0] * np.roll(P[:, 1], -1)
 - np.roll(P[:, 0], -1) * P[:, 1])))

汇总

def analyse(pts, svds, err=BEARING_ERR):
 """给定检测点与示向度, 输出定位区域的全套几何量。"""
 P, status = region_from_bearings(pts, svds, err)
 base = {"n_bearings": len(pts), "status": status}
 if status == "empty":
 base["reason"] = ("所有示向度锥无公共交集 (数据不一致: 两两夹角可以 $> 2^\circ$, "
 "但错开的两个楔也可能不相交)")
 return base
 if status == "unbounded":
 rec = region_is_unbounded(list(svds), err)
 base["reason"] = ("示向度锥近似平行 (两两夹角 $\leq 2^\circ$), 定位区域无界, "
 "直径 = +inf; 必须换检测点重测")
 base["recession_direction_deg"] = None if rec is None else round(float(rec), 4)
 base["n_vertices"] = int(len(P))

```

```

 base["vertices"] = [[round(float(v[0]), 4), round(float(v[1]), 4)] for v in P]
 return base
 if status == "degenerate" or len(P) < 3:
 base["status"] = "degenerate"
 base["reason"] = "公共交集退化为线段或点"
 base["vertices"] = [[round(float(v[0]), 4), round(float(v[1]), 4)] for v in P]
 return base

D, (A, B) = diameter(P)
c = 0.5 * (A + B)
maxr = float(max(np.linalg.norm(v - c) for v in P))
circle = min_enclosing_circle(P)
if circle is None:
 # Very thin polygons far from the origin can lose circumcircle
 # precision. Translation preserves the geometric problem.
 origin = np.mean(P, axis=0)
 local = min_enclosing_circle(P - origin)
 mc = origin + local[0] if local is not None else origin
 mr = float(np.max(np.linalg.norm(P - mc, axis=1)))
else:
 mc, mr = circle
base.update({
 "n_vertices": int(len(P)),
 "vertices": [[round(float(v[0]), 4), round(float(v[1]), 4)] for v in P],
 "diameter_m": round(D, 4),
 "diameter_endpoints": [[round(float(A[0]), 4), round(float(A[1]), 4)],
 [round(float(B[0]), 4), round(float(B[1]), 4)]],
 "circle_center": [round(float(c[0]), 4), round(float(c[1]), 4)],
 "circle_radius_m": round(D / 2.0, 4),
 "max_vertex_dist_m": round(maxr, 4),
 "diameter_circle_covers": bool(maxr <= D / 2.0 + 1e-9),
 "min_enclosing_center": [round(float(mc[0]), 4), round(float(mc[1]), 4)],
 "min_enclosing_radius_m": round(float(mr), 4),
 "min_circle_over_half_diameter": round(float(mr) / (D / 2.0), 6),
 "area_m2": round(region_area(P), 4),
})
return base

def run_case_csv(path, truth_path=None, err=BEARING_ERR):
 with open(path, encoding="utf-8") as f:
 rows = list(csv.DictReader(f))
 pts = [(float(r["x_m"]), float(r["y_m"])) for r in rows]
 svds = [float(r["svd_deg"]) for r in rows]
 res = analyse(pts, svds, err)
 res["n_bearings"] = len(rows)
 if truth_path and os.path.exists(truth_path):
 with open(truth_path, encoding="utf-8") as f:
 truth = json.load(f)
 G = truth["source_xy"]
 res["truth_xy"] = G
 if "circle_center" in res:
 res["center_error_vs_truth_m"] = round(math.dist(res["circle_center"], G), 3)
 if "min_enclosing_center" in res:
 res["mincircle_error_vs_truth_m"] = round(
 math.dist(res["min_enclosing_center"], G), 3)
 # 真值是否满足全部约束（等价于在半平面内）
 H = []
 for p, th in zip(pts, svds):
 H += wedge_halfplanes(p, th, err)
 res["truth_inside_region"] = bool(all(a @ np.asarray(G, float) + b <= 1e-6
 for (a, b) in H))

```

```

return res

def main():
 d = sys.argv[1] if len(sys.argv) > 1 else os.path.join(
 os.path.dirname(os.path.abspath(__file__)), "..", "data")
 out = []
 for fn in sorted(os.listdir(d)):
 if not (fn.startswith("p1_case") and fn.endswith(".csv")):
 continue
 res = run_case_csv(os.path.join(d, fn),
 os.path.join(d, fn.replace(".csv", ".truth.json")))
 res["case"] = fn
 out.append(res)
 jl = os.path.join(d, "p1_results.jsonl")
 with open(jl, "w", encoding="utf-8") as f:
 for r in out:
 f.write(json.dumps(r, ensure_ascii=False) + "\n")

 ok = [r for r in out if r["status"] == "bounded"]
 unlimited = [r for r in out if r["status"] == "unbounded"]
 summary = {
 "n_cases": len(out),
 "n_bounded": len(ok),
 "n_unbounded": len(unlimited),
 "n_degenerate": sum(1 for r in out if r["status"] == "degenerate"),
 "n_empty": sum(1 for r in out if r["status"] == "empty"),
 "diameter_m_range": [min((r["diameter_m"] for r in ok), default=None),
 max((r["diameter_m"] for r in ok), default=None)],
 "n_diameter_circle_misses": sum(1 for r in ok if not r["diameter_circle_covers"]),
 "min_circle_over_half_diameter_max": max(
 (r["min_circle_over_half_diameter"] for r in ok), default=None),
 "n_truth_inside_region": sum(1 for r in ok if r.get("truth_inside_region")),
 }
 with open(os.path.join(d, "p1_summary.json"), "w", encoding="utf-8") as f:
 json.dump(summary, f, ensure_ascii=False, indent=2)
 for r in out:
 print("%-16s %-10s D=%-9s cover=%-5s n=%-2s r*/D*2=%-8s center_err=%s"
 % (r["case"], r["status"], r.get("diameter_m", "-"),
 r.get("diameter_circle_covers", "-"), r.get("n_bearings", "-"),
 r.get("min_circle_over_half_diameter", "-"),
 r.get("center_error_vs_truth_m", "-")))
 print(json.dumps(summary, ensure_ascii=False))

if __name__ == "__main__":
 main()

```

## 7.5 Q4/src/p2\_grid\_expectation.py

""B 题问题 2（模拟）：固定首次探测扇形后，第二检测点网格上的"交会区域直径期望"热力图。

问题设定（按需求方给定的简化模型）

1. \*\*固定当前探测得到的扇形区域\*\*：探测器（机器狗）位于原点 ``S1 = (0, 0)``，探测方向为 x 轴正方向，示向度误差界 ``eps = 1 deg``。于是第一次探测给出的信息是角楔

$$W1 = \{ X : |\text{ang}(X - S1) - \text{theta1}| \leq \text{eps} \}$$

``theta1`` 取自当前探测数据（默认 0 deg，即"探测方向为 x 轴正方向"）。

2. **干扰源均匀分布在扇形内**，近似认为都在  $x$  轴上\*\*：源位  $G = (t, 0)$ ，权重由"扇形内面积均匀"退化得到：面积元  $dA = r dr d\theta \rightarrow w(t) \propto t$ （`--weight area`，默认）；另提供等权（`--weight length`）作对照。
3. **网格\*\***：把每个网格点当作第二个检测点  $S_2$ 。默认网格框在右侧多留  $0.2R$ （`--x_range=(-1800, 2160)`），因为最优点在右半区，这样同样点数下右半区更密。  
注意\*\*可行域仍是靶区域\*\*  $R \leq 1800 \text{ m}$ ， $x > 1800$  的部分是空框。
4. **交会区域与直径\*\***：给定源位  $G$ ，两个检测点各得一条带  $\pm\epsilon$  的示向度锥，两者之交是凸四边形  $R = W_1 \cap W_2$ ，其\*\*直径\*\*  $D$  = 区域内任意两点距离的最大值。
5. **期望\*\***： $E[D](S_2) = \sum_i w_i D(S_2, t_i)$ ，只在"可检测且有界"的样本上取平均，同时记录"可检测概率"，避免把不可用点伪装成好点。
6. **热力图\*\***： $E[D](S_2)$  作为网格点  $(x, y)$  的函数，对数色标。

两套口径（主 / 辅）

```

* ``measured``（主）：两个楔都用**测得值** $\pm 1^\circ$ ，即 $R = W_1(\theta_1) \cap W_2(\theta_2)$ ，
 期望只对源位 t 取 -- 完全对应"固定当前探测数据"的字面含义。
* ``error_avg``（辅）：同一个区域，但期望**同时对 $\pm 1^\circ$ 测量误差取**，
 即 $E_{\{t, e_1, e_2\}}[D]$ -- 更贴近"定位效果"的真实统计含义。

```

为什么网格必须是真正二维的（重要结论）

若把  $S_2$  放在  $x$  轴上（ $y = 0$ ），则两条示向度近似平行，夹角  $|d\theta| \leq 2\epsilon$ ，  
 交会区域\*\*无界\*\*、直径为  $+\infty$ （这正是 `p1_intersection.region_by_vertices` 会报 `unbounded` 的退化几何，等价于"沿示向度方向逼近"）。所以  $y = 0$  必须在网格上被剔除。更进一步， $|y|$  很小但未退化的点，交会区域会被拉成极长的细条（直径可达  $10^4\text{--}10^5 \text{ m}$ ），数学上没错但工程上无用，程序用 `usable_threshold = 10 \times` 中位数标注并掩码。

可检测性门控

第二个检测点要能收到该频道信号，必须  $|S_2 G| \leq$  有效接收半径上界  $1500 \text{ m}$ 。  
 不加这个门控会选出几何有效但物理收不到信号的假最优点，所以它是模型的一部分（默认开启，`--no-reach` 可关闭对照）。

正确性验证

直径由三条\*\*互相独立\*\*的实现给出，随机算例最大相对差  $< 1e-13$ ：  
`p1_intersection.region_by_vertices`（半平面枚举）、`independent_diameter`（世界坐标直接解边界交点）、`fast_diameters_vec`（角点枚举 + 楔定义筛选）。  
`--selfcheck` 另含蒙特卡洛与退化算例（T0-T10）。

用法

```

python src/p2_grid_expectation.py --selfcheck
python src/p2_grid_expectation.py --step 20 --n-t 500 --out out/p2_grid
python src/p2_grid_expectation.py --theta1 30 --out out/p2_grid_t30
"""

from __future__ import annotations

import argparse
import csv
import json
import math
import os
import sys
import time

import numpy as np

_HERE = os.path.dirname(os.path.abspath(__file__))
if _HERE not in sys.path:
 sys.path.insert(0, _HERE)

```

```

from p1_intersection import BEARING_ERR, ang_diff, bearing, region_by_vertices # noqa: E402

物理常数（题目附录 1/2）

EPS_DEG = BEARING_ERR # 示向度误差界 1°
R_ARENA = 1800.0 # 目标区域半径 (m)
R_RECV_MIN = 1000.0 # 有效接收半径下界 (m, 接口不返回)
R_RECV_MAX = 1500.0 # 有效接收半径上界 (m, 取信号的必要条件)
NEAR_R = 5.0 # 近场阈值 (m): 更近则没有示向度

默认计算参数
DEF_STEP = 20.0
DEF_X_RANGE = (-R_ARENA, 1.2 * R_ARENA) # 右侧多留 0.2R (框内留白, 便于看全)
DEF_Y_RANGE = (-R_ARENA, R_ARENA)
DEF_N_T = 500 # x 轴上源位采样数
DEF_N_E = 400 # 误差 MC 次数 (error_avg 口径)
DEFAULT_SEED = 20260913

基本几何

def point_in_wedge(X, p, theta, err=EPS_DEG, tol=1e-9):
 """X 是否在角楔内 (与楔顶点重合视为在楔内) """
 X = np.asarray(X, float)
 p = np.asarray(p, float)
 if float(np.linalg.norm(X - p)) < tol:
 return True
 return abs(ang_diff(bearing(p, X), theta)) <= err + 1e-12

def _unbounded_bearings(theta1, theta2, err=EPS_DEG):
 """两楔交无界 <=> 存在方向与两条示向度夹角都 <= err。

 对两个示向度, 等价于 $\min(|d_{th}|, 180 - |d_{th}|) \leq 2 * err$ 。
 """
 d = abs(ang_diff(theta2, theta1))
 return min(d, 180.0 - d) <= 2.0 * err + 1e-12

def _corner_points(Pw, thP, Qw, thQ, eps):
 """两个角楔交的 6 个候选角点, **全部在世界坐标下求解**。

 每个角楔 $|\text{ang}(X - P) - \theta| \leq \text{eps}$ 的两条边界线方向为 $\theta \mp \text{eps}$,
 交点由四线两两相交得到 (6 个候选); 顶点判定直接用楔的定义
 (候选点 X 是顶点 $\iff$ 对每个楔都有 $|\text{ang}(X - P) - \theta| \leq \text{eps}$ 或 X 恰在顶点 P)。
 这个判据与坐标系定向、2x2 解法的行列式符号都无关, 因此最稳健。

 返回 $(\text{corner_world}, \text{valid})$, 形状 $(\dots, 6, 2)$ 与 $(\dots, 6)$ 。
 """
 Pw = np.asarray(Pw, float)
 Qw = np.asarray(Qw, float)
 thP = np.asarray(thP, float)
 thQ = np.asarray(thQ, float)

 def two_lines(P, th):
 out = []
 for s in (-eps, eps):
 a = np.radians(th + s)
 n = np.stack([np.sin(a), -np.cos(a)], axis=-1)

```

```

 out.append((n, np.einsum("...i,...i->...", n, P)))
 return out

lines = two_lines(Pw, thP) + two_lines(Qw, thQ)

def in_wedge(X, P, th):
 d = X - P
 dist = np.hypot(d[..., 0], d[..., 1])
 ang = np.degrees(np.arctan2(d[..., 1], d[..., 0]))
 diff = (ang - th + 180.0) % 360.0 - 180.0
 return (np.abs(diff) <= eps + 1e-9) | (dist <= 1e-9)

corners, valid = [], []
for ii in range(4):
 for jj in range(ii + 1, 4):
 (n1, c1), (n2, c2) = lines[ii], lines[jj]
 det = n1[..., 0] * n2[..., 1] - n1[..., 1] * n2[..., 0]
 safe = np.where(np.abs(det) < 1e-15, 1e-15, det)
 X = np.stack([(c1 * n2[..., 1] - c2 * n1[..., 1]) / safe,
 (n1[..., 0] * c2 - n2[..., 0] * c1) / safe], axis=-1)
 ok = in_wedge(X, Pw, thP) & in_wedge(X, Qw, thQ)
 corners.append(X)
 valid.append(ok)
 return np.stack(corners, axis=-2), np.stack(valid, axis=-1)

def _diam_from_corners(cw, v, shp):
 """从候选角点与其可用性掩码求凸四边形直径（世界坐标下，距离即真实距离）"""
 nP = cw.shape[-2]
 best = np.zeros(shp)
 for ii in range(nP):
 for jj in range(ii + 1, nP):
 d2 = ((cw[..., ii, 0] - cw[..., jj, 0]) ** 2
 + (cw[..., ii, 1] - cw[..., jj, 1]) ** 2)
 best = np.where(v[..., ii] & v[..., jj], np.maximum(best, d2), best)
 return best, v.sum(axis=-1)

def independent_diameter(t, S2, eps=EPS_DEG):
 """交会区域直径的**独立参考实现**（标量、世界坐标、直接解边界交点）。

 与 ``_corner_points`` 走的是不同代码路径（这里用射线参数式解交点）。
 无界（``min(|dth|, 180-|dth|) <= 2*eps``）返回 ``inf``。
 """
 S2 = np.asarray(S2, float)
 t = float(t)
 th1 = 0.0 # S1 -> G
 th2 = math.degrees(math.atan2(-S2[1], t - S2[0])) % 360.0 # S2 -> G
 if _unbounded_bearings(th1, th2, eps):
 return math.inf

 lines = []
 for apex, th in (((0.0, 0.0), th1), (tuple(S2), th2)):
 for s in (-eps, eps):
 a = math.radians(th + s)
 lines.append((apex, (math.cos(a), math.sin(a))))

 def in_wedge(X, apex, th):
 dx, dy = X[0] - apex[0], X[1] - apex[1]
 if math.hypot(dx, dy) < 1e-9:
 return True
 ad = math.degrees(math.atan2(dy, dx))

```

```

 return abs((ad - th + 180.0) % 360.0 - 180.0) <= eps + 1e-9

verts = []
for i in range(4):
 for j2 in range(i + 1, 4):
 (P1, u), (P2, v) = lines[i], lines[j2]
 det = u[0] * (-v[1]) - (-v[0]) * u[1]
 if abs(det) < 1e-14:
 continue
 rv = (P2[0] - P1[0], P2[1] - P1[1])
 s = (rv[0] * (-v[1]) - (-v[0]) * rv[1]) / det
 X = (P1[0] + s * u[0], P1[1] + s * u[1])
 if in_wedge(X, (0.0, 0.0), th1) and in_wedge(X, tuple(S2), th2):
 verts.append(X)
if len(verts) < 2:
 return math.inf
return max(math.dist(a, b) for i, a in enumerate(verts) for b in verts[i + 1:])

def fast_diameters_vec(S2s, ts, eps=EPS_DEG):
 """批量快路径: S2s (m,2) × ts (n,) → D (m,n), status (m,n)。

 源必须固定在 x 轴正半轴 (S1 在原点)，这是本题设定。

 status 编码: 0 = 有界多边形, 1 = 空, 2 = 退化, 3 = 无界。
 """
 S2s = np.asarray(S2s, float)
 ts = np.asarray(ts, float).ravel()
 m, n = S2s.shape[0], ts.size
 D = np.full((m, n), np.nan)
 status = np.ones((m, n), dtype=np.int8)
 if m == 0 or n == 0:
 return D, status
 r = np.hypot(S2s[:, 0], S2s[:, 1])
 ok_row = r >= 1e-9
 status[~ok_row, :] = 2
 th2 = np.degrees(np.arctan2(-S2s[:, 1][:, None],
 ts[None, :] - S2s[:, 0][:, None])) % 360.0
 dth = np.abs((th2 - 0.0 + 180.0) % 360.0 - 180.0)
 sep = np.minimum(dth, 180.0 - dth)
 unb = (sep <= 2.0 * eps + 1e-12) | (~ok_row[:, None])
 status[unb] = np.where(np.broadcast_to(ok_row[:, None], unb.shape), 3, 2)[unb]
 ok = ~unb
 if not np.any(ok):
 return D, status

 S1w = np.zeros((m, n, 2))
 S2w = np.broadcast_to(S2s[:, None, :], (m, n, 2))
 th1g = np.zeros((m, n))
 corner, valid = _corner_points(S1w, th1g, S2w, th2, eps) # 世界坐标
 cw = corner.reshape(m, n, corner.shape[-2], 2)
 v = valid.reshape(m, n, valid.shape[-1])
 best, n_valid = _diam_from_corners(cw, v, (m, n))
 status = np.where(ok & (n_valid < 2), 1, status).astype(np.int8)
 finite = ok & (n_valid >= 2) & np.isfinite(best)
 D = np.where(finite, np.sqrt(np.maximum(best, 0.0)), np.nan)
 status = np.where(finite, 0, status).astype(np.int8)
 return D, status

def fast_diameters(S2, ts, eps=EPS_DEG):
 """`fast_diameters_vec` 的单品版"""

```

```

D, st = fast_diameters_vec(np.asarray(S2, float)[None, :],
 np.asarray(ts, float).ravel(), eps)

return D[0], st[0]

def _fast_pair_grid(S1, th1, S2, th2, eps):
 """通用批量快路径: ``(...,)`` 形状的 S1/th1/S2/th2 -> D 与 status。

 与 ``fast_diameters_vec`` 同一套做法, 但两个角楔都任意
 (不要求 S1 在原点、源在 x 轴上)。用于 error_avg 口径。
 """
 P1 = np.asarray(S1, float)
 P2 = np.asarray(S2, float)
 th1 = np.asarray(th1, float)
 th2 = np.asarray(th2, float)
 if th2.ndim > th1.ndim:
 th1 = th1.reshape(th1.shape + (1,) * (th2.ndim - th1.ndim))
 shp = np.broadcast_shapes(th1.shape, th2.shape)
 th1 = np.broadcast_to(th1, shp)
 th2 = np.broadcast_to(th2, shp)
 P1 = np.broadcast_to(np.asarray(P1, float)[..., None, :], shp + (2,))
 P2 = np.broadcast_to(np.asarray(P2, float)[..., None, :], shp + (2,))

 rho = np.linalg.norm(P1 - P2, axis=-1)
 sep = np.abs((th2 - th1 + 180.0) % 360.0 - 180.0)
 sep = np.minimum(sep, 180.0 - sep)
 D = np.full(shp, np.nan)
 status = np.ones(shp, dtype=np.int8)
 status[sep <= 2.0 * eps + 1e-12] = 3
 status[rho < 1e-9] = 2
 ok = ~(sep <= 2.0 * eps + 1e-12 | (rho < 1e-9))
 if not np.any(ok):
 return D, status
 corner, valid = _corner_points(P1, th1, P2, th2, eps)
 cw = corner.reshape(shp + (corner.shape[-2], 2))
 v = valid.reshape(shp + (valid.shape[-1],))
 best, n_valid = _diam_from_corners(cw, v, shp)
 status = np.where(ok & (n_valid < 2), 1, status).astype(np.int8)
 finite = ok & (n_valid >= 2) & np.isfinite(best)
 D = np.where(finite, np.sqrt(np.maximum(best, 0.0)), np.nan)
 status = np.where(finite, 0, status).astype(np.int8)
 return D, status

源位采样与权重

def sector_ray_exit(S1, a_deg, R=R_ARENA):
 """从 S1 沿方位角 a 到半径 R 圆域边界的距离"""
 S1 = np.asarray(S1, float)
 u = np.array([math.cos(math.radians(a_deg)), math.sin(math.radians(a_deg))])
 b = float(S1 @ u)
 c = float(S1 @ S1) - R * R
 disc = b * b - c
 return 0.0 if disc <= 0 else float(-b + math.sqrt(disc))

def source_samples(S1, theta1, err=EPS_DEG, n_t=DEF_N_T, r_arena=R_ARENA,
 r_recv_max=R_RECV_MAX, near=NEAR_R):
 """x 轴上的源位采样 (面积均匀权重)。

 来源: 干扰源在扇形内面积均匀 -> ``dA = r dr dtheta`` ,

```



```

 对角向积分后 $w(r) \propto r$, 故 $w_i \propto t_i$ (归一化后即离散先验)。
 $t_{hi} = \min(r_recv_max, \text{沿 } \theta_{t1} \text{ 到靶区边界的距离})$ 。
 """
 S1 = np.asarray(S1, float)
 t_hi = min(r_recv_max, sector_ray_exit(S1, theta1, r_arena))
 t_lo = near
 if t_hi <= t_lo:
 return np.zeros(0), np.zeros(0), {"t_lo": t_lo, "t_hi": t_hi, "n_t": 0}
 ts = np.linspace(t_lo, t_hi, int(n_t))
 w_area = ts / float(ts.sum())
 info = {"t_lo": float(t_lo), "t_hi": float(t_hi), "n_t": int(n_t),
 "weight": "area",
 "t_peak_note": "w(t) $\propto$ t (面积均匀退化的严格结果)",
 "sector_half_angle_deg": float(err),
 "sector_full_width_at_t_hi_m": float(2.0 * t_hi * math.tan(math.radians(err)))}
 return ts, w_area, info

def length_weights(ts):
 w = np.ones_like(np.asarray(ts, float))
 return w / float(w.sum())

网格与期望

def make_grid(step=DEF_STEP, x_range=DEF_X_RANGE, y_range=DEF_Y_RANGE,
 arena_r=R_ARENA, domain="arena"):
 """生成网格点 (仅保留靶区圆域内、且不贴 x 轴退化带的点)。

 默认网格框右侧多留 0.2R ($x_range=(-1800, 2160)$) : 最优点在右半区
 (约 $x \approx +1080$ m), 把点堆在那里比均匀铺满 ± 1800 更划算。
 注意可行域仍是靶区圆域 $R \leq 1800$ m, $x > 1800$ 的部分不会进入结果。
 """
 xs = np.arange(x_range[0], x_range[1] + 1e-9, step)
 ys = np.arange(y_range[0], y_range[1] + 1e-9, step)
 DX, DY = np.meshgrid(xs, ys) # shape (ny, nx)
 X, Y = DX.ravel(), DY.ravel()
 keep = (X * X + Y * Y <= arena_r * arena_r) & (np.abs(Y) > 1e-9)
 if domain == "reach":
 d = np.sqrt(np.minimum(X ** 2, (np.abs(X) - R_RECV_MAX) ** 2) + Y ** 2)
 keep &= (d <= R_RECV_MAX)
 return X[keep], Y[keep], xs, ys, DX.shape

def reach_ok(X, Y, ts, r_recv_max=R_RECV_MAX):
 """可检测性门控矩阵: $|S_2 - G_i| \leq r_recv_max$ 。

 源在 x 轴上, 对固定 (x,y) 关于 t 的最小值在 $t = \text{clip}(x, t_lo, t_hi)$ 处取到。
 """
 X = np.asarray(X, float)[: , None]
 Y = np.asarray(Y, float)[: , None]
 ts = np.asarray(ts, float)[None, :]
 t_clip = np.clip(X, float(np.min(ts)), float(np.max(ts)))
 return np.sqrt((X - t_clip) ** 2 + Y ** 2) <= r_recv_max + 1e-9

def expected_diameter_measured(X, Y, ts, w, eps=EPS_DEG, reach=True, chunk=4096):
 """主口径: 区域用测得值 $\pm \text{eps}$, 期望只对源位取。

 返回 E ($E[D \times 1\{\text{可检测}\}]$)、 E_{det} ($E[D|\text{可检测}]$, 主目标)、
 $\text{cov}/p_{\text{det}}/D_{\text{min}}/D_{\text{max}}/n_{\text{ok}}$ 。
 """

```

```

"""
S2 = np.column_stack([X, Y])
n = X.size
E = np.zeros(n)
E_det = np.full(n, np.nan)
cov = np.zeros(n)
p_det = np.zeros(n)
Dmin = np.full(n, np.nan)
Dmax = np.full(n, np.nan)
n_ok = np.zeros(n, dtype=int)
wsum = float(np.sum(w))
for s in range(0, n, chunk):
 e = min(n, s + chunk)
 D, _ = fast_diameters_vec(S2[s:e], ts, eps)
 m = np.isfinite(D)
 if reach:
 m &= reach_ok(S2[s:e, 0], S2[s:e, 1], ts)
 sw = np.where(m, w[None, :], 0.0).sum(axis=1)
 contrib = (np.where(m, D, 0.0) * w[None, :]).sum(axis=1)
 with np.errstate(invalid="ignore"):
 E[s:e] = contrib
 E_det[s:e] = np.where(sw > 0, contrib / np.maximum(sw, 1e-300), np.nan)
 Dmin[s:e] = np.where(m.any(axis=1),
 np.min(np.where(m, D, np.inf), axis=1), np.nan)
 Dmax[s:e] = np.where(m.any(axis=1),
 np.max(np.where(m, D, -np.inf), axis=1), np.nan)
 cov[s:e] = sw
 p_det[s:e] = sw / wsum
 n_ok[s:e] = m.sum(axis=1)
return {"E": E, "E_det": E_det, "cov": cov, "p_det": p_det,
 "Dmin": Dmin, "Dmax": Dmax, "n_ok": n_ok}

def expected_diameter_error_avg(X, Y, ts, w, eps=EPS_DEG, n_e=DEF_N_E,
 chunk=1024, seed=DEFAULT_SEED, reach=True):
 """辅口径：区域仍取 $W_1(\theta_1) \cap W_2(\theta_2)$ ，但期望同时对 $\pm\epsilon$ 误差取**。

定义为 $E_{err}(X) = E_{\{e_1, e_2\}}[E_{\{t|e\}}[D * 1_{\{\text{可检测}\}}]]$ 。

实现用**分块蒙特卡洛**：对每批网格点抽 n_e 组独立误差 (e_1, e_2) ，第 k 组只作用到该批第 k 个点上 -- 每个网格点得到 n_e 个独立无偏估计，取均值。代价是 $n_e \times n_t \times$ 网格点数，而不是误差笛卡尔积（后者在 1 万网格点上要十几 GB，实测会被拖死）。

"""
S2 = np.column_stack([X, Y])
n = X.size
if n == 0:
 return {"E_err": np.zeros(0), "E_err_det": np.zeros(0),
 "cov_err": np.zeros(0), "n_pairs": 0}
rng = np.random.default_rng(seed)
E_err = np.zeros(n)
E_err_det = np.full(n, np.nan)
cov_err = np.zeros(n)
for s in range(0, n, chunk):
 e = min(n, s + chunk)
 blk = e - s
 E1 = rng.uniform(-eps, eps, blk)
 E2 = rng.uniform(-eps, eps, blk)
 S2b = S2[s:e]
 S1b = np.zeros((blk, 2))
 base2 = np.degrees(np.arctan2(-S2b[:, 1][:, None],
 ts[None, :] - S2b[:, 0][:, None])) % 360.0

```

```

D, _ = _fast_pair_grid(S1b, E1[:, None], S2b, base2 + E2[:, None], eps)
m = np.isfinite(D)
if reach:
 m &= reach_ok(S2b[:, 0], S2b[:, 1], ts)
 contrib = (np.where(m, D, 0.0) * w[None, :]).sum(axis=1)
 wsum = np.where(m, w[None, :], 0.0).sum(axis=1)
 with np.errstate(invalid="ignore"):
 E_err_det[s:e] = np.where(wsum > 0, contrib / np.maximum(wsum, 1e-300), np.nan)
 cov_err[s:e] = wsum
 E_err[s:e] = contrib
return {"E_err": E_err, "E_err_det": E_err_det, "cov_err": cov_err,
 "n_pairs": int(n_e), "estimator": "逐点蒙特卡洛（无偏）"}

自检

def selfcheck(verbose=True, seed=7):
 """独立实现交叉校验 + 退化算例 + 期望一致性 (T0-T10) """
 out = []
 ok_all = True

 def rec(name, ok, detail=""):
 nonlocal ok_all
 ok_all = ok_all and ok
 out.append({"check": name, "ok": bool(ok), "detail": detail})
 if verbose:
 print(f"[{'OK ' if ok else 'FAIL'}] {name} {detail}")

 def th2_of(S2, t):
 return float(np.degrees(np.arctan2(-S2[1], t - S2[0])) % 360.0)

 def mc_points(S2, t, n_pt, rng_):
 """在两个楔各自锥内采样，保留落在交会区域内的点（向量化）"""
 th2 = th2_of(S2, t)
 L = rng_.uniform(5.0, 1900.0, n_pt)
 a1 = np.radians(rng_.uniform(-EPS_DEG, EPS_DEG, n_pt))
 P1 = np.stack([L * np.cos(a1), L * np.sin(a1)], axis=1)
 a2 = math.radians(th2) + np.radians(rng_.uniform(-EPS_DEG, EPS_DEG, n_pt))
 P2 = np.asarray(S2)[None, :] + np.stack([L * np.cos(a2), L * np.sin(a2)], axis=1)
 K = np.vstack([P1, P2])

 def inw(P, apex, th):
 d = P - np.asarray(apex)[None, :]
 dist = np.hypot(d[:, 0], d[:, 1])
 angd = np.degrees(np.arctan2(d[:, 1], d[:, 0]))
 return ((np.abs((angd - th + 180.0) % 360.0 - 180.0) <= EPS_DEG + 1e-9)
 & (dist > 1e-9))

 return K[inw(K, (0.0, 0.0), 0.0) & inw(K, S2, th2)]

 def max_pair_dist(K, blk=512):
 dmax = 0.0
 for s0 in range(0, len(K), blk):
 dmax = max(dmax, float(np.max(
 np.linalg.norm(K[s0:s0 + blk, None, :] - K[None, :, :], axis=2))))
 return dmax

 rng = np.random.default_rng(seed)

 # ---- T0: 顶点个数=4 且每个顶点真在两个楔内 ----
 n0, bad0 = 0, 0

```

```

for _ in range(2500):
 S2 = np.array([rng.uniform(-1500, 1500), rng.uniform(-1400, 1400)])
 if np.hypot(*S2) < 200.0:
 continue
 t = float(rng.uniform(200.0, 1450.0))
 if np.hypot(S2[0] - t, S2[1]) > R_RECV_MAX:
 continue
 th2 = th2_of(S2, t)
 if min(abs(ang_diff(th2, 0.0)), 180 - abs(ang_diff(th2, 0.0))) <= 10.0:
 continue
 D, _ = fast_diameters(S2, np.array([t]))
 if not np.isfinite(D[0]):
 continue
 n0 += 1
 c, v = _corner_points(np.zeros((1, 1, 2)), np.zeros((1, 1)),
 S2.reshape(1, 1, 2), np.array([[th2]]), EPS_DEG)
 c, v = c[0, 0], v[0, 0]
 if int(v.sum()) != 4:
 bad0 += 1
 continue
 for k in np.nonzero(v)[0]:
 if not (point_in_wedge(c[k], (0.0, 0.0), 0.0)
 and point_in_wedge(c[k], S2, th2)):
 bad0 += 1
 break
rec("T0 顶点个数=4 且每个顶点真在两个楔内", bad0 <= 2,
 f"检查 {n0} 组, 异常 {bad0} (允许 ≤2 组数值退化)")

---- T1: 快路径 vs p1 参考实现 (良态) ----
n_bad, n_cmp, n_skip, max_dd = 0, 0, 0, 0.0
for _ in range(6000):
 S2 = np.array([rng.uniform(-1500, 1500), rng.uniform(-1500, 1500)])
 if np.hypot(*S2) < 1.0:
 continue
 t = float(rng.uniform(30.0, 1500.0))
 th2 = th2_of(S2, t)
 if min(abs(ang_diff(th2, 0.0)), 180 - abs(ang_diff(th2, 0.0))) <= 2 * EPS_DEG + 0.3:
 n_skip += 1
 continue
 D, _ = fast_diameters(S2, np.array([t]))
 P, pst = region_by_vertices([(0.0, 0.0), tuple(S2)], [0.0, th2])
 if pst != "polygon" or len(P) < 3:
 continue
 n_cmp += 1
 Pa = np.asarray(P, float)
 d_ref = max(math.dist(a, b) for i, a in enumerate(Pa) for b in Pa[i + 1:])
 if not np.isfinite(D[0]):
 n_bad += 1
 continue
 rel = abs(float(D[0]) - d_ref) / max(d_ref, 1e-12)
 max_dd = max(max_dd, rel)
 if rel > 1e-7:
 n_bad += 1
rec("T1 快路径 vs p1 参考实现 (直径, 良态)", n_bad == 0,
 f"比较 {n_cmp} 组, 跳过近临界 {n_skip} 组, 最大相对差 {max_dd:.3e}")

---- T2: 快路径 vs 独立实现 ----
worst, worst_cfg, n2 = 0.0, None, 0
for _ in range(8000):
 S2 = np.array([rng.uniform(-1500, 1500), rng.uniform(-1400, 1400)])
 if np.hypot(*S2) < 5.0:
 continue

```

```

t = float(rng.uniform(20.0, 1500.0))
D, _ = fast_diameters(S2, np.array([t]))
if not np.isfinite(D[0]):
 continue
dc = independent_diameter(t, S2)
if not math.isfinite(dc):
 continue
n2 += 1
rel = abs(float(D[0]) - dc) / max(dc, 1e-12)
if rel > worst:
 worst, worst_cfg = rel, (S2.copy(), t, float(D[0]), dc)
rec("T2 快路径 vs 独立实现 (直接解边界交点)", worst < 1e-9,
 f"{n2} 组, 最大相对差 {worst:.3e}"
 + (" if worst_cfg is None else
 f" (最差 S2={np.round(worst_cfg[0],3)}, t={worst_cfg[1]:.1f}, "
 f"数值={worst_cfg[2]:.4f}, 独立={worst_cfg[3]:.4f}) ")

---- T3: 蒙特卡洛真值--直径 >= 区域内采样点对最大距离 ----
worst3, worst3_cfg, n3 = 0.0, None, 0
for _ in range(40):
 S2 = np.array([rng.uniform(-1200, 1200), rng.uniform(-1200, 1200)])
 if np.hypot(*S2) < 50.0:
 continue
 t = float(rng.uniform(100.0, 1400.0))
 if np.hypot(S2[0] - t, S2[1]) > R_RECV_MAX:
 continue
 th2 = th2_of(S2, t)
 if _unbounded_bearings(0.0, th2):
 continue
 D, _ = fast_diameters(S2, np.array([t]))
 if not np.isfinite(D[0]):
 continue
 K = mc_points(S2, t, 20000, rng)
 if len(K) < 2:
 continue
 dmax = max_pair_dist(K)
 n3 += 1
 over = dmax / float(D[0]) - 1.0
 if over > worst3:
 worst3, worst3_cfg = over, (S2.copy(), t, float(D[0]), dmax)
rec("T3 直径 >= 区域内 MC 采样点对最大距离", worst3 <= 1e-9,
 f"{n3} 组, 最大超出 {worst3:.3e}"
 + (" if worst3_cfg is None else
 f" (S2={np.round(worst3_cfg[0],2)}, t={worst3_cfg[1]:.1f}, "
 f"数值={worst3_cfg[2]:.4f}, MC={worst3_cfg[3]:.4f}) ")

---- T4: MC 逼近性 ----
n4, ratios = 0, []
for _ in range(40):
 S2 = np.array([rng.uniform(-800, 800), rng.uniform(-800, 800)])
 if np.hypot(*S2) < 50.0:
 continue
 t = float(rng.uniform(100.0, 1400.0))
 if np.hypot(S2[0] - t, S2[1]) > R_RECV_MAX:
 continue
 th2 = th2_of(S2, t)
 if _unbounded_bearings(0.0, th2):
 continue
 D, _ = fast_diameters(S2, np.array([t]))
 if not np.isfinite(D[0]):
 continue
 K = mc_points(S2, t, 40000, rng)

```

```

 if len(K) < 2:
 continue
 n4 += 1
 ratios.append(max_pair_dist(K) / float(D[0]))
 ratios = np.array(ratios)
 ok4 = bool(ratios.size and np.all(ratios >= 0.6) and np.all(ratios <= 1.0 + 1e-9))
 rec("T4 MC 最大距离 / 数值直径 ∈ [0.6, 1]", ok4,
 f"{n4} 组, 最小比值 {ratios.min() if ratios.size else float('nan'):.4f}, "
 f"最大比值 {ratios.max() if ratios.size else float('nan'):.4f}")

---- T5: 贴 x 轴退化必须被识别 ----
bad5, det5 = 0, []
for y in (0.0, 1e-6, 1.0):
 S2 = np.array([600.0, y])
 D, _ = fast_diameters(S2, np.array([300.0, 900.0]))
 det5.append(f"y={y}: D={np.round(D, 3).tolist()}")
 if not np.all(np.isnan(D)):
 bad5 += 1
rec("T5 贴 x 轴退化被识别 (无界/退化)", bad5 == 0, " | ".join(det5))

---- T6: y=0 且源比 S2 远 => 无界 ----
D, st = fast_diameters(np.array([600.0, 0.0]), np.array([900.0]))
rec("T6 y=0 且源比 S2 远 -> 无界 (必须剔除)",
 np.isnan(D[0]) and int(st[0]) == 3, f"D={D[0]}, status={int(st[0])}")

---- T7: 交会角 90° (S2 在源正上方) 时 $D \approx 2 \tan(\epsilon) |S1S2|$ ----
t = 1000.0
t7_bad, t7_worst = [], 0.0
for y in (30.0, 60.0, 120.0, 1000.0):
 S2 = np.array([t, y])
 Dv, _ = fast_diameters(S2, np.array([t]))
 dc = independent_diameter(t, S2)
 target = 2 * math.tan(math.radians(EPS_DEG)) * float(np.hypot(*S2))
 rel = abs(float(Dv[0]) - target) / target
 t7_worst = max(t7_worst, rel)
 if abs(float(Dv[0]) - dc) / dc > 1e-9:
 t7_bad.append(f"y={y}: 数值={float(Dv[0]):.6f} 独立={dc:.6f} 不一致")
 if rel > 2e-3:
 t7_bad.append(f"y={y}: 相对差={rel:.2e}")
if _unbounded_bearings(0.0, 270.0):
 t7_bad.append("gamma=90° 应判定为良态")
rec("T7 交会角=90° 时 $D \approx 2 \tan(\epsilon) |S1S2|$ (实测误差 < 2e-3)", not t7_bad,
 "; ".join(t7_bad) if t7_bad else f"四组, 最大相对差 {t7_worst:.2e}")

---- T8: 源位采样权重归一 + 上界 ----
ts, w, info = source_samples((0.0, 0.0), 0.0, n_t=200)
rec("T8 源位采样权重归一 + 上界正确",
 abs(w.sum() - 1.0) < 1e-12 and abs(info["t_hi"] - min(R_RECV_MAX, R_arena)) < 1e-9,
 f"t∈[{info['t_lo']:.1f},{info['t_hi']:.1f}], Σw-1={w.sum()-1:.2e}")

---- T9: E[D]可检测 与逐点加权平均一致 ----
X = np.array([300.0, 447.2, -200.0, 900.0])
Y = np.array([600.0, 800.0, 300.0, 0.0])
tt, ww, _ = source_samples((0.0, 0.0), 0.0, n_t=60)
res9 = expected_diameter_measured(X, Y, tt, ww)
manual = []
for i in range(X.size):
 num = den = 0.0
 for j in range(tt.size):
 D, _ = fast_diameters(np.array([X[i], Y[i]]), np.array([tt[j]]))
 if np.isfinite(D[0]):
 num += ww[j] * float(D[0])

```

```

 den += ww[j]

 manual.append(num / den if den > 0 else np.nan)
ok9 = np.allclose(np.array(manual), res9["E_det"], rtol=1e-10, equal_nan=True)
rec("T9 E[D]可检测] 与逐点加权平均一致", ok9,
 f"manual={np.round(manual,4).tolist()}, batch={np.round(res9['E_det'],4).tolist()}")

---- T10: 网格框扩展的语义（诚实版）----
靶区是半径 1800 m 的圆域，格点是"圆域 ∩ 网格框"：因此**平移/放大框边界
不会改变圆域内的点数**（只改变框内空白的多少）。要真正加密只能缩小 step。
Xa, Ya, xsa, _, _ = make_grid(step=50.0, x_range=(-R_ARENA, R_ARENA))
Xb, Yb, xsb, _, _ = make_grid(step=50.0)
inside = (Xb ** 2 + Yb ** 2) <= R_ARENA ** 2 + 1e-9
same_count = (Xa.size == Xb.size)
rec("T10 网格框扩展不改变圆域内点数（加密只能靠缩小 step）",
 bool(np.all(inside)) and same_count and xsb[-1] > xsa[-1],
 f"x_max {xsa[-1]:.0f}->{xsb[-1]:.0f} m: 圆域内点数均为 {Xa.size}（不变），"
 f"全部满足 R<=1800; 右半区点数与左半区相同（格点本已对称）")

if verbose:
 print(f"\n自检总结: {'全部通过' if ok_all else '存在失败项'}")
return {"ok": ok_all, "checks": out}

输出: CSV / JSON

def _json_default(o):
 """把 numpy 标量/布尔/数组转成可序列化对象（自检结果里会混进 np.bool_）"""
 if isinstance(o, (np.bool_,)):
 return bool(o)
 if isinstance(o, np.integer):
 return int(o)
 if isinstance(o, np.floating):
 return float(o)
 if isinstance(o, np.ndarray):
 return o.tolist()
 raise TypeError(f"Object of type {type(o).__name__} is not JSON serializable")

def _fmt(v, nd=6):
 if v is None:
 return ""
 if isinstance(v, float) and (math.isnan(v) or math.isinf(v)):
 return "nan" if math.isnan(v) else "inf"
 return round(float(v), nd)

def save_outputs(res, outdir, tag=""):
 os.makedirs(outdir, exist_ok=True)
 X, Y = res["X"], res["Y"]
 path = os.path.join(outdir, f"p2_grid_map{tag}.csv")
 thr = res.get("usable_threshold_m", float("inf"))
 with open(path, "w", newline="", encoding="utf-8-sig") as f:
 wr = csv.writer(f)
 wr.writerow(["x_m", "y_m", "r_m", "phi_deg",
 "E_diam_given_detectable_m", "E_diam_mixed_mm",
 "p_detectable", "cov_weight", "Dmin_m", "Dmax_m",
 "E_diam_err_given_detectable_m", "E_diam_err_mixed_m",
 "cov_err", "n_ok", "usable", "near_axis_unusable_band"])
 for i in range(X.size):
 r = math.hypot(X[i], Y[i])
 phi = math.degrees(math.atan2(Y[i], X[i])) % 360.0

```

```

 ed = res["E_det"][i]
 usable = int(np.isfinite(ed) and ed <= thr and res["p_det"][i] > 0)
 wr.writerow([_fmt(X[i], 4), _fmt(Y[i], 4), _fmt(r, 4), _fmt(phi, 4),
 _fmt(ed), _fmt(res["E"][i]),
 _fmt(res["p_det"][i]), _fmt(res["cov"][i]),
 _fmt(res["Dmin"][i]), _fmt(res["Dmax"][i]),
 _fmt(res["E_err_det"][i]), _fmt(res["E_err"][i]),
 _fmt(res["cov_err"][i]), int(res["n_ok"][i]), usable,
 int(np.isfinite(ed) and ed > thr)])]
 rep = {k: v for k, v in res.items()
 if k not in ("X", "Y", "E", "E_det", "cov", "p_det", "Dmin", "Dmax", "n_ok",
 "E_err", "E_err_det", "cov_err", "usable")}
 rep["csv"] = os.path.basename(path)
 with open(os.path.join(outdir, f"p2_grid_report{tag}.json"), "w", encoding="utf-8") as f:
 json.dump(rep, f, ensure_ascii=False, indent=2, default=_json_default)
 return path

热力图

def _savefig(fig, path, dpi=150, tries=6):
 """保存图片并重试（Windows 上杀毒/索引偶尔短暂锁住刚写出的 PNG）。"""
 import time as _t
 last = None
 for k in range(tries):
 try:
 fig.savefig(path, dpi=dpi)
 return path
 except OSError as ex:
 last = ex
 _t.sleep(0.5 * (k + 1))
 raise last

def make_figures(res, outdir, tag=""):
 import matplotlib
 matplotlib.use("Agg")
 import matplotlib.pyplot as plt
 from matplotlib.colors import LogNorm
 from matplotlib.ticker import FixedLocator, FuncFormatter

 os.makedirs(outdir, exist_ok=True)
 X, Y, xs, ys, shape = res["X"], res["Y"], res["xs"], res["ys"], res["shape"]
 ny, nx = shape
 figs = []
 t_hi = float(res["source_prior"]["t_hi"]) # 扇形径向上界 = 有效接收半径上界

 def to_grid(vals):
 g = np.full((ny, nx), np.nan)
 g[np.searchsorted(ys, Y), np.searchsorted(xs, X)] = vals
 return g

 XLIM = (-1500.0, 2200.0) # x 负方向小一点；正方向盖住右侧扩展后的网格框
 YLIM = (-2000.0, 2000.0)

 # 关注区（E ~ 50-1500 m）用对数色标，低值段的差异才看得出来
 ZMIN, ZMAX = 50.0, 2000.0
 lnorm = LogNorm(vmin=ZMIN, vmax=ZMAX)
 MAJOR = [50, 70, 100, 150, 200, 300, 500, 700, 1000, 1500, 2000]
 MINOR = [60, 80, 120, 250, 400, 600, 800, 1200]

```



```

ok_use = res.get("usable", np.ones(X.size, bool))
GX = to_grid(X)
GY = to_grid(Y)
Gm = to_grid(np.where(ok_use, res["E_det"], np.nan))
Ge = to_grid(np.where(ok_use, res["E_err_det"], np.nan))
Gc = to_grid(res["p_det"])
th = np.linspace(0, 2 * math.pi, 2000)

def best_xy(G):
 r, c = np.unravel_index(int(np.nanargmin(G)), G.shape)
 return float(GX[r, c]), float(GY[r, c]), float(G[r, c])

def show(ax, G, cmap, vmin=None, vmax=None, norm=None, ttl=None):
 kw = {"norm": norm} if norm is not None else {"vmin": vmin, "vmax": vmax}
 im = ax.imshow(G, origin="lower", extent=[xs[0], xs[-1], ys[0], ys[-1]],
 cmap=cmap, interpolation="nearest", aspect="equal", **kw)
 ax.set_xlim(XLIM[0], XLIM[1])
 ax.set_ylim(YLIM[0], YLIM[1])
 ax.set_xlabel("x (m)"); ax.set_ylabel("y (m)")
 if ttl:
 ax.set_title(ttl, fontsize=9)
 return im

def ring(ax, style="w--", lw=1.0, alpha=0.85):
 """只画落在显示范围内的靶区圆弧 (R=1800 m)"""
 cx, cy = R_ARENA * np.cos(th), R_ARENA * np.sin(th)
 m = ((cx >= XLIM[0]) & (cx <= XLIM[1]) & (cy >= YLIM[0]) & (cy <= YLIM[1]))
 ax.plot(np.where(m, cx, np.nan), np.where(m, cy, np.nan),
 style, lw=lw, alpha=alpha)

def draw_sector(ax, zorder=6, label=True):
 """画出**首次探测的扇形** (源位先验所在区域), 半透明。

 角楔半张角 ``eps = 1°``, 径向范围 ``[near, t_hi]``。``eps`` 只有 1°,
 所以这个扇形在整张图上很窄 (1500 m 处全宽仅约 52 m) -- 为让它看得见,
 填充用半透明 + 亮色描边。**图内不放任何说明文字/框** (会遮挡内容),
 相关信息一律写进图例、标题或 `report.json`。

 注意 ``ax.fill`` 的 label 会按**网格点数**产生成百上千条图例项,
 所以这里不传 label, 图例项改由下面的 ``sector_patch`` 提供。
 """
 t1 = t_hi
 ang_f = np.linspace(-EPS_DEG, EPS_DEG, 400)
 ax.plot(t1 * np.cos(np.radians(ang_f)), t1 * np.sin(np.radians(ang_f)),
 "-", color="darkorange", lw=1.6, alpha=0.95, zorder=zorder)
 n = 240
 rr = np.linspace(NEAR_R, t1, n)
 aa = np.linspace(-EPS_DEG, EPS_DEG, n)
 A, R_ = np.meshgrid(np.radians(aa), rr)
 ax.fill(R_ * np.cos(A), R_ * np.sin(A), color="darkorange", alpha=0.6,
 lw=0, zorder=zorder)
 for s in (-EPS_DEG, EPS_DEG):
 a = math.radians(s)
 ax.plot([NEAR_R * math.cos(a), t1 * math.cos(a)],
 [NEAR_R * math.sin(a), t1 * math.sin(a)],
 "-", color="darkorange", lw=1.6, alpha=0.95, zorder=zorder)

def sector_patch():
 from matplotlib.patches import Patch
 return Patch(fc="darkorange", alpha=0.6, ec="darkorange",
 label=f"1st-detection sector ($\pm$ {EPS_DEG:g}°, $r \leq$ {t_hi:.0f} m)")

```

```

def set_ticks(cb, size=9):
 """色标刻度: 主/次刻度都带数字, 且**字号统一** (不再大小不一) """
 cb.set_ticks(MAJOR)
 cb.ax.set_yticklabels([f"{t:g}" for t in MAJOR])
 cb.ax.yaxis.set_minor_locator(FixedLocator(MINOR))
 cb.ax.yaxis.set_minor_formatter(FuncFormatter(lambda v, pos: f"{v:g}"))
 cb.ax.tick_params(which="both", labelsz=size, length=3.5)

def origin_marker(ax, ms=13, label=True):
 """第一检测点 (加粗圆点)。返回 artist, 便于显式组织图例。"""
 ln, = ax.plot([0], [0], marker="o", ms=ms, mfc="red", mec="k", mew=1.5,
 ls="none", zorder=9,
 label=("S1 = origin (detector)" if label else None))
 return ln

def best_marker(ax, G, ms=10, label=True):
 """最优第二检测点及关于 x 轴对称的那一点 (图内不加文字, 信息进图例) """
 bx, by, vb = best_xy(G)
 pts = [by, -by] if abs(by) > 1e-9 else [by]
 first = None
 for n_, yy in enumerate(pts):
 ln, = ax.plot([bx], [yy], "wo", ms=ms, mfc="none", mew=2, ls="none",
 zorder=7,
 label=("best S2 (symmetric pair)" if (label and n_ == 0) else None))
 if first is None:
 first = ln
 return bx, by, vb, first

----- 图 1: 主口径热力图 -----
fig, ax = plt.subplots(figsize=(10.6, 10.6))
im = show(ax, Gm, "viridis_r", norm=lnorm)
fig.subplots_adjust(top=0.88, bottom=0.07, left=0.09, right=0.98)
ax.set_title("B-Problem 2: expected intersection-region diameter vs 2nd detection point",
 fontsize=10, pad=10)
fig.text(0.5, 0.965,
 "log colour scale; measured bearings; source prior: area-uniform inside the"
 " 1st-detection sector;\n"
 f"$\\varepsilon$={res['eps_deg']}°, $\\theta_1$={res['theta1']}°, "
 f"reach gate $|S_2|\\leq\\{int(R_RECV_MAX)\\}$ m, arena $R\\leq\\{int(R_ARENA)\\}$ m, "
 f"grid step {res['step']:g} m ({res['grid_points']} points)",
 ha="center", va="bottom", fontsize=9)
cb = fig.colorbar(im, ax=ax, shrink=0.8, extend="both")
cb.set_label(f"E[intersection diameter | detectable] (m, log scale), "
 f"colourbar range {ZMIN:g}-{ZMAX:g} m")
set_ticks(cb)
ring(ax)
draw_sector(ax)
h_origin = origin_marker(ax)
bx, by, vb, h_best = best_marker(ax, Gm)
ax.legend(handles=[h_origin, h_best, sector_patch()],
 loc="lower left", fontsize=8, framealpha=0.9)
f1 = os.path.join(outdir, f"p2_grid_E_heatmap{tag}.png")
savefig(fig, f1); plt.close(fig); figs.append(f1)

----- 图 2: 两口径并排 -----
fig, axes = plt.subplots(1, 2, figsize=(16.4, 9.4))
for ax2, G, ttl in (
 (axes[0], Gm, "(a) measured bearings fixed\n"
 "expectation over source position only"),
 (axes[1], Ge, "(b) error-averaged\n"
 "expectation over source position AND $\pm 1^\circ$ reading errors")):
 im = show(ax2, G, "viridis_r", norm=lnorm)

```

```

ring(ax2)
draw_sector(ax2, label=False)
origin_marker(ax2, ms=11, label=False)
ibx, iby, ibv, _ = best_marker(ax2, G, ms=8, label=False)
ax2.set_title(f"${t1l}\\nmin E = {ibv:.1f} m at ({ibx:.0f}, {iby:.0f}) m", fontsize=9)
cb2 = fig.colorbar(im, ax=ax2, shrink=0.75, extend="both")
cb2.set_label("E[D | detectable] (m, log scale)")
set_ticks(cb2, size=8)
f2 = os.path.join(outdir, f"p2_grid_E_heatmap_two{tag}.png")
fig.tight_layout(); _savefig(fig, f2); plt.close(fig); figs.append(f2)

----- 图 3: 可检测概率 -----
fig, ax = plt.subplots(figsize=(10.6, 10.0))
im = show(ax, Gc, "magma", 0.0, 1.0,
 ttl="Detectability: probability the 2nd detector can even hear the channel")
cbp = fig.colorbar(im, ax=ax, shrink=0.8)
cbp.set_label("P(|S2G| ≤ 1500 m) under source prior")
cbp.ax.tick_params(which="both", labels=9)
ring(ax, "c--", 1.0, 0.9)
draw_sector(ax)
h_origin = origin_marker(ax)
ax.legend(handles=[h_origin, sector_patch()],
 loc="lower left", fontsize=8, framealpha=0.9)
f3 = os.path.join(outdir, f"p2_grid_coverage{tag}.png")
_savefig(fig, f3); plt.close(fig); figs.append(f3)

----- 图 4: E(r) 与 E(phi) 剖面 -----
fig, axes = plt.subplots(1, 2, figsize=(14.0, 5.4))
r_all = np.hypot(X, Y)
phi_all = np.degrees(np.arctan2(Y, X)) % 360.0
for phi0, style in ((90.0, "-"), (80.0, "--"), (70.0, "-."), (45.0, ":")):
 m = np.abs(((phi_all - phi0 + 180) % 360) - 180) < \
 res["step"] / (2 * np.maximum(r_all, 1)) * 1.2
 if m.sum() < 3:
 continue
 o = np.argsort(r_all[m])
 axes[0].plot(r_all[m][o], res["E_det"][m][o], style, lw=1.4, label=f"φ={phi0:.0f}°")
axes[0].set_xlabel("baseline r = |S1S2| (m)"); axes[0].set_ylabel("E[D | detectable] (m)")
axes[0].set_title("radial profile (measured bearings)", fontsize=10)
axes[0].legend(fontsize=8); axes[0].grid(alpha=0.3)
for r0, style in ((200.0, "-"), (400.0, "--"), (800.0, "-."), (1200.0, ":")):
 m = np.abs(r_all - r0) < res["step"] * 1.2
 if m.sum() < 3:
 continue
 o = np.argsort(phi_all[m])
 axes[1].plot(phi_all[m][o], res["E_det"][m][o], style, lw=1.4, label=f"r={r0:.0f} m")
axes[1].set_xlabel("φ = arg(S2) (deg)"); axes[1].set_ylabel("E[D | detectable] (m)")
axes[1].set_title("angular profile: minima near the 90°-crossing geometry", fontsize=10)
axes[1].legend(fontsize=8); axes[1].grid(alpha=0.3)
f4 = os.path.join(outdir, f"p2_grid_profiles{tag}.png")
fig.tight_layout(); _savefig(fig, f4); plt.close(fig); figs.append(f4)

----- 图 5: 交会区域随 S2 变化的示意 -----
from p1_intersection import convex_hull
fig, axes = plt.subplots(1, 3, figsize=(15.0, 5.2))
t_demo = 900.0
for ax2, (rr_, pp_) in zip(axes, ((900.0, 90.0), (180.0, 80.0), (900.0, 20.0))):
 S2 = np.array([rr_ * math.cos(math.radians(pp_)), rr_ * math.sin(math.radians(pp_))])
 th2 = float(np.degrees(np.arctan2(-S2[1], t_demo - S2[0])) % 360.0)
 P, stt = region_by_vertices([(0.0, 0.0), tuple(S2)], [0.0, th2])
 D, stf = fast_diameters(S2, np.array([t_demo]))
 for base, colr, anch in ((0.0, "tab:blue", np.zeros(2)), (th2, "tab:orange", S2)):

```

```

 for s in (-1, 1):
 a = math.radians(base + s * EPS_DEG)
 L = 2200.0
 ax2.plot([anch[0], anch[0] + L * math.cos(a)],
 [anch[1], anch[1] + L * math.sin(a)],
 color=colr, lw=1.0, alpha=0.75)
 if len(P) >= 3 and not _unbounded_bearings(0.0, th2):
 H = convex_hull(np.asarray(P, float))
 H = np.vstack([H, H[1:]])
 ax2.fill(H[:, 0], H[:, 1], color="crimson", alpha=0.55, zorder=5)
 ax2.plot([0], [0], marker="o", ms=11, mfc="red", mec="k", mew=1.4, zorder=6)
 ax2.plot([S2[0]], [S2[1]], "o", color="tab:orange", ms=7, zorder=6)
 ax2.plot([t_demo], [0], "k^", ms=8, zorder=6)
 ax2.set_xlim(-1500, 2400); ax2.set_ylim(-1400, 2100)
 dv = float(D[0]) if np.isfinite(D[0]) else float("nan")
 ax2.set_title(f"source t=900 m, r={rr:.0f} m, ϕ ={pp:.0f}^\circ\n"
 f"intersection D = {dv:.1f} m", fontsize=9)
 ax2.set_xlabel("x (m)"); ax2.set_ylabel("y (m)"); ax2.grid(alpha=0.25)
f5 = os.path.join(outdir, f"p2_grid_region_demo{tag}.png")
fig.tight_layout(); _savefig(fig, f5); plt.close(fig); figs.append(f5)
return figs

主流程

def run(theta1=0.0, step=DEF_STEP, n_t=DEF_N_T, n_e=DEF_N_E, weight="area",
 domain="arena", eps=EPS_DEG, outdir=None, figures=True,
 seed=DEFAULT_SEED, reach=True, checks=True, verbose=True,
 x_range=DEF_X_RANGE, y_range=DEF_Y_RANGE):
 t0 = time.time()
 S1 = (0.0, 0.0)
 ts, w_area, sinfo = source_samples(S1, theta1, eps, n_t)
 w = w_area if weight == "area" else length_weights(ts)
 if ts.size == 0:
 raise SystemExit("源位采样为空: 检查 theta1 与靶区/接收半径设置")
 X, Y, xs, ys, shape = make_grid(step=step, x_range=x_range, y_range=y_range,
 arena_r=R_ARENA, domain=domain)

 if verbose:
 print(f"网格点 {X.size} 个 (step={step} m, domain={domain}, "
 f"x $\in$ [{xs[0]:.0f}, {xs[-1]:.0f}], y $\in$ [{ys[0]:.0f}, {ys[-1]:.0f}]); "
 f"源位 {ts.size} 个 t $\in$ [{ts[0]:.1f}, {ts[-1]:.1f}] m, 权重={weight}")
 meas = expected_diameter_measured(X, Y, ts, w, eps, reach=reach)
 err = expected_diameter_error_avg(X, Y, ts, w, eps, n_e=n_e, seed=seed, reach=reach)

 t_nom = float(np.sum(w * ts))
 med = float(np.nanmedian(meas["E_det"]))
 usable_thr = 10.0 * med
 usable = np.isfinite(meas["E_det"]) & (meas["E_det"] <= usable_thr) & (meas["p_det"] > 0)
 E_use = np.where(usable, meas["E_det"], np.nan) if np.any(usable) else meas["E_det"]

 def _pick(a):
 a = np.asarray(a, float)
 if not np.any(np.isfinite(a)):
 return None
 i = int(np.nanargmin(a))
 return {"S2": [float(X[i]), float(Y[i])],
 "r_m": float(math.hypot(X[i], Y[i])),
 "phi_deg": float(math.degrees(math.atan2(Y[i], X[i])) % 360.0),
 "value_m": float(a[i]), "index": i,
 "p_det": float(meas["p_det"][i]),
 "E_mixed_m": float(meas["E"][i]),

```

```

 "E_err_det_m": (float(err["E_err_det"][i])
 if np.isfinite(err["E_err_det"][i]) else None),
 "E_err_mixed_m": float(err["E_err"][i])}

best_det, best_mix = _pick(E_use), _pick(meas["E"])
n_band = int((np.isfinite(meas["E_det"]) & (meas["E_det"] > usable_thr)).sum())
res = {
 "model": "P2 grid simulation: E[intersection-region diameter] over 2nd detection point",
 "theta1": float(theta1), "eps_deg": float(eps), "weight": weight, "domain": domain,
 "step": float(step), "reach_gate": bool(reach),
 "grid_x_range": [float(xs[0]), float(xs[-1])],
 "grid_y_range": [float(ys[0]), float(ys[-1])],
 "source_prior": sinfo, "arena_r": R_ARENA,
 "r_recv_range": [R_RECV_MIN, R_RECV_MAX], "near_r": NEAR_R,
 "S1": list(S1), "grid_points": int(X.size), "shape": list(shape),
 "xs": [float(v) for v in xs], "ys": [float(v) for v in ys],
 "usable_threshold_m": float(usable_thr),
 "X": X, "Y": Y,
 "E": meas["E"], "E_det": meas["E_det"], "cov": meas["cov"], "p_det": meas["p_det"],
 "Dmin": meas["Dmin"], "Dmax": meas["Dmax"], "n_ok": meas["n_ok"], "usable": usable,
 "E_err": err["E_err"], "E_err_det": err["E_err_det"],
 "cov_err": err["cov_err"], "n_err_pairs": err["n_pairs"],
 "summary": {
 "objective": "E[D | detectable] (|S2G| <= 1500 m 门控, 按扇形内面积均匀先验加权)",
 "E_det_min_m": float(np.nanmin(E_use)),
 "E_det_median_m": med,
 "E_det_max_m": float(np.nanmax(meas["E_det"])),
 "E_mixed_min_m": float(np.nanmin(meas["E"])),
 "usable_points": int(usable.sum()),
 "usable_fraction": float(usable.mean()),
 "near_axis_band_points": n_band,
 "best_det": best_det, "best_mixed": best_mix,
 "best_S2": best_det["S2"] if best_det else None,
 "best_r_m": best_det["r_m"] if best_det else None,
 "best_phi_deg": best_det["phi_deg"] if best_det else None,
 "best_r_over_t": (best_det["r_m"] / t_nom) if best_det else None,
 "mean_source_range_m": t_nom,
 "theory_note": ("交会角 90° 时 D 取该基线下的最小值 $\approx 2 \tan(\text{eps}) |S1S2|$; "
 "一般情形 D 随基线与近简并程度增大"),
 "x_axis_degenerate": ("y≈0 时两示向度近平行($|\Delta\theta| \leq 2\text{eps}$) → 交会区域无界、直径=+∞, "
 "被剔除; 其邻域(|y| 小)直径可达 104-105 m, 已用 "
 f"usable_threshold={usable_thr:.1f} m 标注为不可用带"),
 "reach_gate": ("|S2G| > 有效接收半径上界 1500 m 的源位样本不可检测, "
 "不计入 E[D]可检测; 混口径 E[D·1{可检测}] 另列"),
 "grid_note": ("网格框右侧多留 0.2R (x 到 2160 m) 以便右半区更密; "
 "可行域仍为靶区圆域 R<=1800 m"),
 "runtime_s": None,
 },
},
}
if checks:
 res["checks"] = selfcheck(verbose=verbose)["checks"]
res["summary"]["runtime_s"] = round(time.time() - t0, 3)
if outdir:
 save_outputs(res, outdir)
if figures:
 try:
 figs = make_figures(res, outdir)
 res["figures"] = [os.path.basename(f) for f in figs]
 except Exception as ex:
 # 图失败不影响数值结果落盘
 res["figures_error"] = f"{type(ex).__name__}: {ex}"
 if verbose:
 print(f"[warn] 出图失败 (数值结果已保存): {ex}")

```

```

 save_outputs(res, outdir)
 if verbose:
 s = res["summary"]
 b = s["best_det"]
 print(f"\n最优第二检测点 S2* = ({b['S2'][0]:.1f}, {b['S2'][1]:.1f}) m "
 f" (r={b['r_m']:.1f} m, ϕ = $\{b['phi_deg']:.1f\}^\circ$, $r/\langle t \rangle$ = $\{s['best_r_over_t']:.3f\}$) ")
 print(f"E[D|可检测] 最小 = {s['E_det_min_m']:.3f} m; 中位数 = {s['E_det_median_m']:.3f} m; "
 f"单条基线 $2\tan(\epsilon)r = \{2*\text{math.tan}(\text{math.radians}(\epsilon))*b['r_m']:.3f\} m$ ")
 print(f"该点 P(可检测) = {b['p_det']:.4f}; 混口径 E[D·1] = {b['E_mixed_m']:.3f} m; "
 f"误差口径 E[D|可检测] = {b['E_err_det_m']}")
 print(f"可用点 {s['usable_points']}/{res['grid_points']}")
 f" (近轴不可用带 {s['near_axis_band_points']} 个, 阈值 {usable_thr:.1f} m) "
 print(f"耗时 {s['runtime_s']} s")
 return res

def main(argv=None):
 ap = argparse.ArgumentParser(
 description="B 题问题 2: 第二检测点网格上的交会区域直径期望热力图")
 ap.add_argument("--theta1", type=float, default=0.0,
 help="首次探测的示向度(度), 默认 0 (x 轴正向)")
 ap.add_argument("--step", type=float, default=DEF_STEP, help="网格步长 (m)")
 ap.add_argument("--n-t", type=int, default=DEF_N_T, help="x 轴源位采样数")
 ap.add_argument("--n-e", type=int, default=DEF_N_E, help="误差 MC 采样数")
 ap.add_argument("--weight", choices=["area", "length"], default="area",
 help="源位先验权重: area $\propto t$ (面积均匀, 默认) / length 等权")
 ap.add_argument("--domain", choices=["arena", "reach"], default="arena",
 help="S2 取值范围: 整个靶区圆域 / 还要满足 |S2G|<=1500")
 ap.add_argument("--x-range", type=float, nargs=2, default=list(DEF_X_RANGE),
 metavar=("XMIN", "XMAX"), help="网格 x 范围 (m), 默认右侧多留 0.2R")
 ap.add_argument("--y-range", type=float, nargs=2, default=list(DEF_Y_RANGE),
 metavar=("YMIN", "YMAX"), help="网格 y 范围 (m)")
 ap.add_argument("--out", default=None, help="输出目录")
 ap.add_argument("--no-reach", action="store_true", help="关闭可检测性门控")
 ap.add_argument("--no-figures", action="store_true")
 ap.add_argument("--no-checks", action="store_true", help="跳过自检(只出结果, 快很多)")
 ap.add_argument("--selfcheck", action="store_true")
 a = ap.parse_args(argv)
 if a.selfcheck:
 r = selfcheck()
 if a.out:
 os.makedirs(a.out, exist_ok=True)
 with open(os.path.join(a.out, "p2_grid_selfcheck.json"), "w",
 encoding="utf-8") as f:
 json.dump(r, f, ensure_ascii=False, indent=2, default=_json_default)
 return 0 if r["ok"] else 1
 run(theta1=a.theta1, step=a.step, n_t=a.n_t, n_e=a.n_e, weight=a.weight,
 domain=a.domain, outdir=a.out, figures=not a.no_figures,
 checks=not a.no_checks, reach=not a.no_reach,
 x_range=tuple(a.x_range), y_range=tuple(a.y_range))
 return 0

if __name__ == "__main__":
 raise SystemExit(main())

```

## 7.6 Q4/src/p3\_expect\_field.py

"""B 题问题 3: 机器狗"基于期望的行动"决策场。

本模块只负责问题 3 的\*\*决策层\*\*：把"下一步该去哪里检测"变成一个可以在全局网格上逐格比较的标量场。它严格按下面的思路实现：

1. **全局网格\*\***：在半径 1800 m 的靶区域域内铺一张规则网格（`grid_step`，默认 100 m），每个格子记录"如果机器狗去那里做下一次检测，这一步有多好"。
2. **第二问的网格模拟结果\*\***：直接复用 `mod:p2_grid_expectation` 产出的热力图 `out/p2_grid/p2_grid_map.csv`。该图是在"第一检测点  $S_1=(0,0)$ 、示向度  $\theta_1=0^\circ$ "的基准几何下算出的  $E[D]$ （ $D$  = 交会定位区域直径）。
3. **变换 + 插值\*\***： $E[D]$  在\*\*刚体变换下不变\*\*（旋转 + 平移保持距离与夹角）。因此真实一次检测  $(S, \theta)$  对应的场，就是把基准图\*\*绕原点旋转  $\theta$ 、再平移到  $S$ ：

$$\begin{aligned} \mathbf{b} &= \mathbf{R}(-\theta) \cdot (\mathbf{g} - \mathbf{S}) & (\mathbf{g} \text{ 是全局网格点, } \mathbf{b} \text{ 是基准图坐标}) \\ E_{\mathbf{g}} &= E_{\text{base}}(\mathbf{b}) & (\text{双线性插值}) \end{aligned}$$

4. **逐次叠加\*\***：把每个"待测频道"的每一次检测都按上式投到全局网格上叠加。
5. **未覆盖的网格取最大值\*\***：基准图里没有数据（靶区外 / 该点不可检测 / 近轴退化带）或数值超过上限的格子一律记  $E_{\max}=1500$  m -- 也就是"最差的位置"。这样它们只有在附近没有更好的格子时才会被选中，天然形成探索行为。
6. **归一化\*\***： $E_{\text{norm}} = E_{\text{mean}} / E_{\max} \in [0,1]$ （0 = 交会几何最好，1 = 最差）。
7. **距离惩罚层\*\***：行动要花时间，所以在归一化场之上再叠加一层\*\*与探测器距离线性增长\*\*的代价：

$$\text{score}(\mathbf{g}) = E_{\text{norm}}(\mathbf{g}) + \lambda \cdot |\mathbf{g} - \mathbf{p}_{\text{robot}}| / (2 \cdot R_{\text{arena}})$$

$\lambda$  即"距离项系数"，可调（`dist_weight`，默认 0.35； $\lambda=1$  表示"跑到靶区对面"等价于把定位质量从最好拉到最差）。已访问过的扫描点邻域、以及离当前位置过近的格子会被直接排除（重复检测不产生新信息）。

8. **选点\*\***：取 `argmin score`。若整张图都被排除（例如一条方位都还没有），退化为"最大最小距离"探索点（离所有已访问点最远的格子）。

用法

```
python src/p3_expect_field.py --selfcheck
python src/p3_expect_field.py --demo --dist-weight 0.35 --out out/p3

from __future__ import annotations

import argparse
import csv
import json
import math
import os
import sys
from dataclasses import dataclass, field as dc_field

import numpy as np

_HERE = os.path.dirname(os.path.abspath(__file__))
if _HERE not in sys.path:
 sys.path.insert(0, _HERE)

from p1_intersection import BEARING_ERR, ang_diff # noqa: E402

物理常数（题目附录 1/2 与模拟器接口说明）

R_ARENA = 1800.0 # 靶区半径 (m)
E_MAX = 1500.0 # 未覆盖网格的默认最大值 (m) --即"最差位置"
R_RECV_MIN = 1000.0 # 有效接收半径下界 (m)
```

```

R_RECV_MAX = 1500.0 # 有效接收半径上界 (m)
NEAR_R = 5.0 # 近场阈值 (m)
R_CLEAR = 20.0 # 清除半径 (m)
DIST_REF = 2.0 * R_ARENA # 距离惩罚的归一化尺度 (靶区直径, m)

DEFAULT_BASE_CSV = os.path.normpath(
 os.path.join(_HERE, "..", "out", "p2_grid", "p2_grid_map.csv"))
DEFAULT_FAMILY_DIR = os.path.normpath(os.path.join(_HERE, "..", "out", "p3_family"))
E_COL_CANDIDATES = ("E_diam_given_detectable_m", "E_diam_mixed_mm",
 "E_diam_mixed_m")

基准图 (第二问的网格模拟结果)

@dataclass
class BaseMap:
 """问题 2 的期望热力图: ``E[j, i]`` 对应 ``(xs[i], ys[j])``, nan = 无数据."""

 xs: np.ndarray
 ys: np.ndarray
 E: np.ndarray
 source: str = "p2_grid_expectation"
 e_col: str = "E_diam_given_detectable_m"

 # ----- 构造 -----
 @property
 def step(self) -> float:
 return float(self.xs[1] - self.xs[0])

 @property
 def extent(self) -> tuple:
 return (float(self.xs[0]), float(self.xs[-1]), float(self.ys[0]), float(self.ys[-1]))

 @classmethod
 def from_points(cls, X, Y, E, source="synthetic") -> "BaseMap":
 """由散点 (规则网格掩码后) 构造."""
 X = np.asarray(X, float)
 Y = np.asarray(Y, float)
 E = np.asarray(E, float)
 xs = np.unique(np.round(X, 6))
 ys = np.unique(np.round(Y, 6))
 # 步长取最小正间隔 (避免浮点噪声把网格拆散)
 dx = _grid_step(xs)
 dy = _grid_step(ys)
 xs = np.round(np.arange(xs[0], xs[-1] + 1e-9, dx), 6)
 ys = np.round(np.arange(ys[0], ys[-1] + 1e-9, dy), 6)
 G = np.full((ys.size, xs.size), np.nan)
 ix = np.searchsorted(xs, np.round(X, 6))
 iy = np.searchsorted(ys, np.round(Y, 6))
 ok = (ix < xs.size) & (iy < ys.size)
 G[iy[ok], ix[ok]] = E[ok]
 return cls(xs=xs, ys=ys, E=G, source=source)

 @classmethod
 def load_csv(cls, path=DEFAULT_BASE_CSV, e_col=None) -> "BaseMap":
 if not os.path.exists(path):
 raise FileNotFoundError(
 f"找不到第二问的网格模拟结果 {path}。请先运行\n"
 f"python src/p2_grid_expectation.py --step 40 --out out/p2_grid")
 with open(path, encoding="utf-8-sig") as f:
 rows = list(csv.DictReader(f))

```



```

if not rows:
 raise ValueError(f"{path} 为空")
col = e_col
if col is None:
 for c in E_COL_CANDIDATES:
 if c in rows[0]:
 col = c
 break
if col is None:
 raise ValueError(f"{path} 缺少期望值列, 现有列: {list(rows[0])}")
X = np.array([float(r["x_m"]) for r in rows])
Y = np.array([float(r["y_m"]) for r in rows])
E = np.array([_to_float(r.get(col)) for r in rows])
m = cls.from_points(X, Y, E, source=os.path.basename(path))
m.e_col = col
return m

@classmethod
def synthetic(cls, step=100.0, R=R_ARENA) -> "BaseMap":
 """自检用解析基准图 (形状与真实图同类: 近轴退化带 + 斜前方最优翼)。"""
 xs = np.arange(-R, R + 1e-9, step)
 X, Y = np.meshgrid(xs, xs)
 r = np.hypot(X, Y)
 phi = np.abs(np.degrees(np.arctan2(Y, X)))
 E = 60.0 + 900.0 * (1.0 - np.exp(-r / 700.0)) + 700.0 * np.exp(-phi / 12.0)
 E[r > R] = np.nan
 return cls(xs=xs, ys=xs.copy(), E=E, source="synthetic")

----- 查询 -----
def lookup(self, X, Y, e_max=E_MAX):
 """双线性插值查询。返回 ``E, covered``; 四点任一无数据即视为未覆盖。

 ``E`` 已按"未覆盖/超过上限一律取 e_max"的规则裁剪, 可直接用于叠加。
 """
 X = np.asarray(X, float)
 Y = np.asarray(Y, float)
 shp = np.broadcast(X, Y).shape
 Xf = np.broadcast_to(X, shp).astype(float).ravel()
 Yf = np.broadcast_to(Y, shp).astype(float).ravel()
 x0, y0 = float(self.xs[0]), float(self.ys[0])
 dx, dy = self.step, float(self.ys[1] - self.ys[0])
 nx, ny = self.xs.size, self.ys.size
 fx = (Xf - x0) / dx
 fy = (Yf - y0) / dy
 i = np.floor(fx).astype(int)
 j = np.floor(fy).astype(int)
 inside = (i >= 0) & (i <= nx - 2) & (j >= 0) & (j <= ny - 2)
 ic = np.clip(i, 0, nx - 2)
 jc = np.clip(j, 0, ny - 2)
 tx = np.clip(fx - ic, 0.0, 1.0)
 ty = np.clip(fy - jc, 0.0, 1.0)
 G = self.E
 e00 = G[jc, ic]
 e10 = G[jc, ic + 1]
 e01 = G[jc + 1, ic]
 e11 = G[jc + 1, ic + 1]
 ok = inside & np.isfinite(e00) & np.isfinite(e10) & np.isfinite(e01) & np.isfinite(e11)
 val = (e00 * (1 - tx) * (1 - ty) + e10 * tx * (1 - ty)
 + e01 * (1 - tx) * ty + e11 * tx * ty)
 val = np.where(ok, val, np.nan)
 E_out = np.where(ok, np.minimum(val, e_max), e_max)
 return E_out.reshape(shp), ok.reshape(shp)

```

```

----- 变换 -----
def transformed_lookup(self, S, theta_deg, X, Y, e_max=E_MAX):
 """把基准图按"检测点 S + 示向度 θ "旋转平移后查询全局网格点 (X, Y)。

 $\vec{b} = R(-\theta) \cdot (g - S)$: 把全局点转回基准图坐标系（基准图的源在 +x 轴方向）。
 E[D] 在刚体变换下不变，故 $\vec{E}(g) = E_{\text{base}}(b)$ 。
 """
 S = np.asarray(S, float)
 c = math.cos(math.radians(theta_deg))
 s = math.sin(math.radians(theta_deg))
 Xf = np.asarray(X, float) - S[0]
 Yf = np.asarray(Y, float) - S[1]
 bx = Xf * c + Yf * s
 by = -Xf * s + Yf * c
 return self.lookup(bx, by, e_max=e_max)

def _grid_step(v):
 d = np.diff(np.asarray(v, float))
 d = d[d > 1e-9]
 return float(np.min(d)) if d.size else 1.0

def _to_float(v):
 if v is None or v == "" or v == "nan":
 return float("nan")
 try:
 return float(v)
 except (TypeError, ValueError):
 return float("nan")

def save_map_csv(path, X, Y, E, e_col="E_diam_given_detectable_m"):
 os.makedirs(os.path.dirname(os.path.abspath(path)), exist_ok=True)
 with open(path, "w", newline="", encoding="utf-8-sig") as f:
 w = csv.writer(f)
 w.writerow(["x_m", "y_m", e_col])
 for i in range(len(X)):
 v = E[i]
 w.writerow([round(float(X[i]), 4), round(float(Y[i]), 4),
 "" if not np.isfinite(v) else round(float(v), 6)])]

 return path

基准图族: 按"先验外径 t_hi"参数化

class BaseMapFamily:
 """一族基准图 $\{t_{\text{hi}}: \text{BaseMap}\}$ 。

 为什么需要它（这一步不做对，整个"变换 + 插值"就不成立）

 基准图里源位先验是"从检测点沿示向度射线、 $t \in [5, t_{\text{hi}}]$ 、权重 $\propto t$ "。
 真实场景中源还必须落在靶区内，所以射线上先验的**外径** t_{hi} 随 (S, θ) 变化（射线出靶区的距离可以只有几百米）。而 $\vec{E}[D]$ 只依赖（相对位置 b, t_{hi} ）这一对量，因此把 t_{hi} 也当作族参数，旋转 + 平移 + 插值才严格成立。

 查询时按 t_{hi} 在族内做线性插值（先把未覆盖格子填成 e_{max} ，再混合）。
 """

 def __init__(self, maps, t_ref=R_RECV_MAX, e_max=E_MAX):

```

```

self.maps = {float(k): v for k, v in sorted(self.maps.items())}
self.t_list = np.array(sorted(self.maps.keys()))
self.t_ref = float(t_ref)
self.e_max = float(e_max)
self._cache = {}

----- 构造 -----
@classmethod
def build(cls, outdir, t_his=(150.0, 250.0, 350.0, 450.0, 600.0, 800.0, 1000.0,
 1250.0, 1500.0), step=40.0, n_t=200, verbose=True):
 """按 p2 的口径重算一族基准图并落盘。"""
 from p2_grid_expectation import (# 局部导入, 避免循环依赖
 expected_diameter_measured, make_grid, source_samples)

 X, Y, xs, ys, shape = make_grid(step=step, arena_r=R_ARENA, domain="arena")
 os.makedirs(outdir, exist_ok=True)
 maps = {}
 files = []
 for T in t_his:
 ts, w, info = source_samples((0.0, 0.0), 0.0, n_t=n_t, r_arena=float(T))
 res = expected_diameter_measured(X, Y, ts, w)
 E = np.asarray(res["E_det"], float)
 m = BaseMap.from_points(X, Y, E, source=f"p3_family_thi{T:g}")
 fn = os.path.join(outdir, f"p2_grid_map_thi{int(round(T)):04d}.csv")
 save_map_csv(fn, X, Y, E)
 maps[float(T)] = m
 files.append(os.path.basename(fn))
 if verbose:
 fin = E[np.isfinite(E)]
 print(f" t_hi={T:6.0f} m -> {os.path.basename(fn)} "
 f"有效格 {fin.size}/{E.size}, 中位数 "
 f"{np.median(fin) if fin.size else float('nan'):.1f} m")
 man = {"kind": "p3_base_map_family", "step": float(step), "n_t": int(n_t),
 "arena_r": R_ARENA, "t_his": [float(t) for t in t_his], "files": files,
 "note": "每个文件是 S1=(0,0)、θ1=0 下, 源位先验 t∈[5,t_hi] 的 E[D]可检测"}
 with open(os.path.join(outdir, "p2_grid_family.json"), "w", encoding="utf-8") as f:
 json.dump(man, f, ensure_ascii=False, indent=2)
 return cls(maps)

@classmethod
def load_dir(cls, outdir):
 man_path = os.path.join(outdir, "p2_grid_family.json")
 if not os.path.exists(man_path):
 return None
 with open(man_path, encoding="utf-8") as f:
 man = json.load(f)
 maps = {}
 for t, fn in zip(man["t_his"], man["files"]):
 maps[float(t)] = BaseMap.load_csv(os.path.join(outdir, fn))
 return cls(maps)

@classmethod
def load_or_build(cls, outdir, **kw):
 fam = cls.load_dir(outdir)
 if fam is not None:
 return fam
 return cls.build(outdir, **kw)

----- 查询 -----
def select(self, t_hi):
 """返回该 t_hi 对应的（可缓存的）混合基准图。"""
 t = float(min(max(t_hi, self.t_list[0]), self.t_list[-1]))

```

```

key = round(t / 25.0)
if key in self._cache:
 return self._cache[key]
tt = key * 25.0
if tt <= self.t_list[0]:
 m = self.maps[float(self.t_list[0])]
elif tt >= self.t_list[-1]:
 m = self.maps[float(self.t_list[-1])]
else:
 i = int(np.searchsorted(self.t_list, tt) - 1)
 T1, T2 = float(self.t_list[i]), float(self.t_list[i + 1])
 a = (tt - T1) / max(T2 - T1, 1e-9)
 m = blend_maps(self.maps[T1], self.maps[T2], a, self.e_max)
if len(self._cache) < 64:
 self._cache[key] = m
return m

def rebase(self, S, theta_deg):
 """给出 (S, θ) 检测对应的基准图: 先按射线出靶区距离取 t_hi, 再在族内插值。"""
 S = np.asarray(S, float)
 u = np.array([math.cos(math.radians(theta_deg)), math.sin(math.radians(theta_deg))])
 b = float(S @ u)
 c = float(S @ S) - R_ARENA * R_ARENA
 disc = b * b - c
 exit_d = float(-b + math.sqrt(disc)) if disc > 0 else 0.0
 t_hi = min(R_RECV_MAX, exit_d)
 return self.select(t_hi), t_hi

def blend_maps(m1: BaseMap, m2: BaseMap, a, e_max=E_MAX) -> BaseMap:
 """把两张同网格基准图按权重 a 线性混合 (未覆盖格子先填 e_max, 偏保守)。"""
 a = float(min(max(a, 0.0), 1.0))
 E1 = np.where(np.isfinite(m1.E), m1.E, e_max)
 E2 = np.where(np.isfinite(m2.E), m2.E, e_max)
 covered = np.isfinite(m1.E) | np.isfinite(m2.E)
 E = np.where(covered, (1.0 - a) * E1 + a * E2, np.nan)
 return BaseMap(xs=m1.xs, ys=m1.ys, E=E, source=f"blend({m1.source},{m2.source},{a:.2f})")

决策场

@dataclass
class FieldConfig:
 grid_step: float = 100.0 # 全局网格步长 (m)
 e_max: float = E_MAX # 未覆盖网格的默认最大值 (m)
 dist_weight: float = 0.35 # 距离项系数 λ (可调)
 visited_radius: float = 250.0 # 已访问扫描点的排除半径 (m)
 min_move: float = 60.0 # 离当前位置太近的格子直接排除 (m)
 arena_r: float = R_ARENA
 normalize: str = "mean" # mean (期望均值归一化) | max

class ExpectField:
 """全局网格上的"期望定位质量 + 距离"标量场。"""

 def __init__(self, base, cfg: FieldConfig | None = None):
 self.base = base
 self.family = base if isinstance(base, BaseMapFamily) else None
 self.cfg = cfg or FieldConfig()
 step = float(self.cfg.grid_step)
 R = float(self.cfg.arena_r)

```

```

xs = np.arange(-R, R + 1e-9, step)
GX, GY = np.meshgrid(xs, xs)
m = np.hypot(GX, GY) <= R + 1e-9
self.gx = GX[m]
self.gy = GY[m]
self.terms: list[tuple[np.ndarray, float, tuple]] = []

----- 叠加 -----
def add_bearing(self, S, theta_deg, weight=1.0, tag=None):
 """叠加一次检测 ``S, θ`` 给出的期望场（已裁剪：未覆盖 = e_max）。

 有基准图族时先用 (S, θ) 的"射线出靶区距离"选 t_hi, 再在族内插值,
 这样"旋转 + 平移 + 插值"才是严格的刚体不变。
 """
 S = np.asarray(S, float)
 bm = self.base
 t_hi = R_RECV_MAX
 if self.family is not None:
 bm, t_hi = self.family.rebase(S, theta_deg)
 E, cov = bm.transformed_lookup(S, theta_deg, self.gx, self.gy,
 e_max=self.cfg.e_max)
 self.terms.append((E, float(weight),
 (tuple(float(v) for v in S), float(theta_deg), tag, float(t_hi))))
 return cov

def add_channel_bearings(self, bearings):
 """bearings = [(x, y, svd_deg), ...]"""
 for (x, y, th) in bearings:
 self.add_bearing((x, y), th)

def add_raw(self, values, weight=1.0, tag=None):
 """叠加一层已经算好的、直接定义在全局网格上的值（用于"探索项"）。"""
 v = np.asarray(values, float)
 if v.shape != self.gx.shape:
 v = np.broadcast_to(v, self.gx.shape)
 self.terms.append((v, float(weight), ("raw", tag)))
 return v

def discovery_term(self, meas_pts, min_dist_m, e_max=None):
 """探索项：某频道"在哪些格点上重新检测才有新信息"。

 对"还没听到过"的频道，若新检测点距它所有历史检测点都 < ``min_dist_m``,
 那么能新覆盖到的区域很小（有效接收半径至少 1000 m），几乎不可能听到 -- 记 e_max;
 反之记 0（无惩罚）。这样全局网格就会自动把扫描点推到"能发现新源"的地方。
 """
 e_max = float(self.cfg.e_max if e_max is None else e_max)
 if not meas_pts:
 return np.zeros(self.gx.shape)
 P = np.asarray(meas_pts, float)
 dmin = np.min(np.hypot(self.gx[:, None] - P[None, :, 0],
 self.gy[:, None] - P[None, :, 1]), axis=1)
 return np.where(dmin >= float(min_dist_m), 0.0, e_max)

----- 聚合 / 打分 -----
def aggregate(self):
 """返回 ``(E_mean_m, coverage_count, n_terms)``。

 * 每一项先把未覆盖格子记成 ``e_max`` ("最差位置");
 * ``normalize='mean'``: 对各项取加权平均（即"总和归一化"）;
 * ``coverage_count``: 至少有一项覆盖到的格子数。
 """
 n = self.gx.size

```

```

if not self.terms:
 return (np.full(n, float(self.cfg.e_max)), np.zeros(n, dtype=int), 0)
W = 0.0
acc = np.zeros(n)
cov = np.zeros(n, dtype=int)
covered = np.zeros(n, dtype=bool)
for (E, w, _tag) in self.terms:
 acc += w * E
 W += w
 covered |= (E < self.cfg.e_max - 1e-9)
 cov += (E < self.cfg.e_max - 1e-9).astype(int)
E_mean = acc / max(W, 1e-12)
if self.cfg.normalize == "max" and np.any(covered):
 ref = float(np.max(E_mean))
 E_mean = E_mean * (float(self.cfg.e_max) / max(ref, 1e-12))
 E_mean = np.minimum(E_mean, float(self.cfg.e_max))
return E_mean, cov, len(self.terms)

def score(self, cur_xy, visited=(), extra_penalty=None, max_dist=None):
 """计算整张网格的 score = 归一化期望 + λ·距离/靶区直径（排除项 = +inf）。"""
 E_mean, cov, n_terms = self.aggregate()
 e_norm = E_mean / max(float(self.cfg.e_max), 1e-12)
 if extra_penalty is not None:
 e_norm = e_norm + np.asarray(extra_penalty, float)
 p = np.asarray(cur_xy, float)
 d = np.hypot(self.gx - p[0], self.gy - p[1])
 score = e_norm + float(self.cfg.dist_weight) * d / DIST_REF
 mask = d < float(self.cfg.min_move)
 if max_dist is not None:
 mask |= d > float(max_dist)
 for q in visited:
 q = np.asarray(q, float)
 mask |= np.hypot(self.gx - q[0], self.gy - q[1]) < float(self.cfg.visited_radius)
 score = np.where(mask, np.inf, score)
 return score, {"E_mean_m": E_mean, "coverage": cov, "n_terms": n_terms,
 "e_norm": e_norm, "dist_m": d, "excluded": mask}

----- 选点 -----
def argmin(self, cur_xy, visited=(), extra_penalty=None, max_dist=None):
 if not self.terms:
 return None # 一条方位都没有：交给探索兜底
 score, info = self.score(cur_xy, visited, extra_penalty, max_dist=max_dist)
 if not np.any(np.isfinite(score)):
 return None
 i = int(np.argmin(np.where(np.isfinite(score), score, np.inf)))
 return self._describe(i, score[i], info)

def fallback_point(self, cur_xy, visited=(), prefer_near=True, max_dist=None):
 """探索兜底：离所有已访问点"最远"的格子（最大最小距离准则）。

 一条方位都没有、或整张图都被排除时使用；也用于避免反复在同一片区域打转。
 ``max_dist`` 给定时只在"还走得"的格子里挑（用于时间预算收尾阶段）。
 """
 p = np.asarray(cur_xy, float)
 if visited:
 V = np.asarray(visited, float)
 dmin = np.min(np.hypot(self.gx[:, None] - V[None, :, 0],
 self.gy[:, None] - V[None, :, 1]), axis=1)
 else:
 dmin = np.hypot(self.gx - p[0], self.gy - p[1])
 dc = np.hypot(self.gx - p[0], self.gy - p[1])
 key = dmin - (1e-6 * dc if prefer_near else 0.0)

```

```

key = np.where(dc < float(self.cfg.min_move), -np.inf, key)
if max_dist is not None:
 key = np.where(dc > float(max_dist), -np.inf, key)
if not np.any(np.isfinite(key)):
 key = dmin - 1e-6 * dc # 放宽: 忽略 max_dist/min_move 再挑一次
i = int(np.argmax(key))
E_mean, cov, n_terms = self.aggregate()
info = {"E_mean_m": E_mean, "coverage": cov, "n_terms": n_terms,
 "e_norm": E_mean / max(float(self.cfg.e_max), 1e-12),
 "dist_m": dc, "excluded": np.zeros(self.gx.size, bool)}
return self._describe(i, float("nan"), info, kind="explore")

def _describe(self, i, score, info, kind="min"):
 return {
 "kind": kind,
 "index": int(i),
 "xy": (float(self.gx[i]), float(self.gy[i])),
 "score": float(score),
 "E_mean_m": float(info["E_mean_m"][i]),
 "E_norm": float(info["e_norm"][i]),
 "dist_m": float(info["dist_m"][i]),
 "dist_penalty": float(self.cfg.dist_weight * info["dist_m"][i] / DIST_REF),
 "n_terms": int(info["n_terms"]),
 "n_cover": int(info["coverage"][i]),
 "covered": bool(info["coverage"][i] > 0),
 }

----- 导出 -----
def save_csv(self, path):
 E_mean, cov, n_terms = self.aggregate()
 score, info = self.score((0.0, 0.0))
 os.makedirs(os.path.dirname(os.path.abspath(path)), exist_ok=True)
 with open(path, "w", newline="", encoding="utf-8-sig") as f:
 w = csv.writer(f)
 w.writerow(["x_m", "y_m", "E_mean_m", "E_norm", "n_cover", "score_at_origin"])
 for k in range(self.gx.size):
 w.writerow([round(float(self.gx[k]), 3), round(float(self.gy[k]), 3),
 round(float(E_mean[k]), 4), round(float(E_mean[k]) / self.cfg.e_max, 6),
 int(cov[k]),
 "" if not np.isfinite(score[k]) else round(float(score[k]), 6)])

 return path

精确期望（独立实现）：任意 S_1/θ_1 下，候选第二检测点的 $E[D]$

def ray_source_samples(S, theta_deg, err=BEARING_ERR, n_t=200, r_arena=R_ARENA,
 r_recv_max=R_RECV_MAX, near=NEAR_R):
 """源位先验：在 S 沿 θ 的射线上、靶区内、可被有效接收的范围内按面积均匀采样。

 与 :func:`p2_grid_expectation.source_samples` 同一先验，但支持任意 S（不只是原点）。
 返回 ``(ts, w, t_hi)``: ``ts`` 为源到 S 的距离采样。
 """
 S = np.asarray(S, float)
 u = np.array([math.cos(math.radians(theta_deg)), math.sin(math.radians(theta_deg))])
 b = float(S @ u)
 c = float(S @ S) - r_arena * r_arena
 disc = b * b - c
 exit_d = float(-b + math.sqrt(disc)) if disc > 0 else 0.0
 t_hi = min(r_recv_max, exit_d)
 if t_hi <= near:
 return np.zeros(0), np.zeros(0), 0.0

```

```

ts = np.linspace(near, t_hi, int(n_t))
w = ts.copy()
return ts, w / float(w.sum()), t_hi

def exact_expected_diameter(S, theta_deg, S2, ts, w, err=BEARING_ERR,
 r_recv_max=R_RECV_MAX, per_sample_gate=False):
 """独立实现（用于自检）：直接枚举源位样本，用精确的两楔交会算 E[D]。

 与基准图完全同构（源位先验 = 射线上的面积均匀），但**不做任何插值/变换**，
 因此可以用来校验 "旋转平移 + 双线性插值" 的正确性。

 ``per_sample_gate=False``（默认，与 :mod:`p2_grid_expectation` 一致）：
 "可检测"门控按**候选点**判定 -- 只要该点能收到先验中某个源位的信号就保留全部样本；
 置 True 则逐样本判定 ``|S2 - G| <= 有效接收半径上界``。
 """
 from p2_grid_expectation import _fast_pair_grid # 局部导入，避免循环依赖

 S = np.asarray(S, float)
 S2 = np.atleast_2d(np.asarray(S2, float))
 ts = np.asarray(ts, float)
 w = np.asarray(w, float)
 m, n = S2.shape[0], ts.size
 if n == 0:
 return np.full(m, np.nan)
 u = np.array([math.cos(math.radians(theta_deg)), math.sin(math.radians(theta_deg))])
 G = S[None, :] + ts[:, None] * u[None, :] # (n,2) 源位样本
 th2 = np.degrees(np.arctan2(G[None, :, 1] - S2[:, None, 1],
 G[None, :, 0] - S2[:, None, 0])) % 360.0
 # 注意 _fast_pair_grid 的形状约定：坐标数组要比 th 少一个尾维（(m,2) 对 (m,n)）
 S1b = np.broadcast_to(S.reshape(1, 2), (m, 2))
 th1b = np.full((m,), float(theta_deg))
 S2b = S2
 D, _st = _fast_pair_grid(S1b, th1b, S2b, th2, err)
 d2 = np.hypot(S2[:, None, 0] - G[None, :, 0], S2[:, None, 1] - G[None, :, 1])
 if per_sample_gate:
 reach = d2 <= r_recv_max + 1e-9
 else:
 reach = np.broadcast_to((d2 <= r_recv_max + 1e-9).any(axis=1)[:, None], D.shape)
 ok = np.isfinite(D) & reach
 wc = np.where(ok, w[None, :], 0.0)
 sw = wc.sum(axis=1)
 num = (np.where(ok, D, 0.0) * w[None, :]).sum(axis=1)
 with np.errstate(invalid="ignore"):
 return np.where(sw > 0, num / np.maximum(sw, 1e-12), np.nan)

def best_refine_point(rec_pts, rec_svds, cur_xy, region, cfg: FieldConfig | None = None,
 n_t=120, n_ang=16, radii=(100.0, 200.0, 300.0, 450.0, 600.0, 800.0),
 travel_cap_m=900.0, err=BEARING_ERR):
 """在"已有一条/多条方位"的基础上，选一个**局部**最优的下一检测点。

 与 :class:`ExpectField`（全局、用预计算基准图）互补：这里对任意 S1/θ1 直接算
 精确的 E[D]，用于机器狗已经逼近目标、或刚发现一个新频道需要"再测一条方位把区域
 定下来"的场合。候选点取以当前位置为中心、半径 ``radii`` 的极角网格，
 代价 = 走到候选点 + 检测 + 走到目标区域 + 预期覆盖式清扫时间。

 ``region`` 为 ``{'center': (x, y)}``；为 ``None``（只有一条方位）时用源位先验的
 中位点作为"目标区域"，从而把"测完还要往回走多远"算进代价。

 返回 ``(xy, info)``；无可用候选时返回 ``(None, None)``。
 """

```



```

pts = [np.asarray(p, float) for p in rec_pts]
if not pts:
 return None, None
S1 = pts[-1]
th1 = float(rec_svds[-1])
ts, w, t_hi = ray_source_samples(S1, th1, n_t=n_t)
if ts.size == 0:
 return None, None
cur = np.asarray(cur_xy, float)
ang0 = math.degrees(math.atan2(cur[1] - S1[1], cur[0] - S1[0]))
cand = [cur]
for r in radii:
 for k in range(n_ang):
 a = math.radians(ang0 + 360.0 * k / n_ang)
 p = cur + r * np.array([math.cos(a), math.sin(a)])
 if np.hypot(*p) <= R_ARENA - 1.0:
 cand.append(p)
C = np.array(cand, float)
keep = np.hypot(C[:, 0] - cur[0], C[:, 1] - cur[1]) <= travel_cap_m
C = C[keep]
if C.size == 0:
 return None, None
E = exact_expected_diameter(S1, th1, C, ts, w, err=err)
dcur = np.hypot(C[:, 0] - cur[0], C[:, 1] - cur[1])
if region is not None:
 c = np.asarray(region["center"], float)
 back_target = "region"
else:
 u = np.array([math.cos(math.radians(th1)), math.sin(math.radians(th1))])
 c = S1 + float(ts[ts.size // 2]) * u # 先验中位源位
 back_target = "prior_median"
dreg = np.hypot(C[:, 0] - c[0], C[:, 1] - c[1])
代价(秒) = 走到候选点 + 检测 + 走到目标 + 按 E[D] 估计的覆盖式清扫
清扫路程 ≈ 圆域面积 / 覆盖行距 = $\pi \cdot (r)^2 / (2 \cdot 20)$, r 取 E[D]/2 + 20 m
v = 5.0
sweep_m = np.pi * np.maximum(E / 2.0 + 20.0, 20.0) ** 2 / (2.0 * R_CLEAR)
cost = dcur / v + 6.0 + dreg / v + np.where(np.isfinite(E), sweep_m / v, 1e6)
i = int(np.argmin(cost))
info = {"S1": tuple(float(v) for v in S1), "th1": th1,
 "xy": (float(C[i, 0]), float(C[i, 1])),
 "E_diam_m": float(E[i]), "cost_s": float(cost[i]),
 "move_s": float(dcur[i] / v), "back_s": float(dreg[i] / v),
 "n_cand": int(C.shape[0]), "t_hi": float(t_hi), "back_target": back_target}
return (float(C[i, 0]), float(C[i, 1])), info

def _ring_candidates(S, theta_deg, radii=(400.0, 800.0, 1200.0), n_ang=8,
 dphi0=30.0, arena_r=R_ARENA):
 """绕检测点 S 的环形候选点(相对 θ 偏开 dphi0, 避免正落在退化射线上)。"""
 S = np.asarray(S, float)
 C = []
 for r in radii:
 for k in range(n_ang):
 a = math.radians(theta_deg + dphi0 + (360.0 - 2 * dphi0) * k / max(n_ang - 1, 1))
 p = S + r * np.array([math.cos(a), math.sin(a)])
 if np.hypot(*p) <= arena_r - 1.0:
 C.append(p)
 return np.array(C) if C else np.zeros((0, 2))

自检

```

```

def selfcheck(base_csv=DEFAULT_BASE_CSV, verbose=True, seed=20260913):
 out = []
 ok_all = True

 def rec(name, ok, detail=""):
 nonlocal ok_all
 ok_all = ok_all and bool(ok)
 out.append({"check": name, "ok": bool(ok), "detail": detail})
 if verbose:
 print(f"[{'OK' if ok else 'FAIL'}] {name} {detail}")

 rng = np.random.default_rng(seed)
 base = BaseMap.synthetic(step=50.0)
 E, cov = base.lookup(0.0, 0.0)
 rec("T1 基准图基本可查（原点有值）", bool(cov) and np.isfinite(float(E)),
 f"E(0,0)={float(E):.3f}, covered={bool(cov)}")

 # T2 旋转平移不变性：把基准网格点做刚体变换后查回来，应完全一致
 S = np.array([137.0, -421.0])
 th = 37.0
 c, s = math.cos(math.radians(th)), math.sin(math.radians(th))
 worst = 0.0
 for (x, y) in ((-500.0, 300.0), (0.0, 0.0), (1000.0, -800.0), (-1200.0, -1200.0)):
 e0, c0 = base.lookup(x, y)
 if not bool(c0):
 continue
 g = S + np.array([x * c - y * s, x * s + y * c]) # 正向变换到全局
 e1, c1 = base.transformed_lookup(S, th, g[0], g[1])
 worst = max(worst, abs(float(e1) - float(e0)))
 rec("T2 旋转+平移不变性（刚体变换下 E 不变）", worst < 1e-9,
 f"最大绝对差 {worst:.3e}")

 # T3 双线性插值精度：中心值应落在四邻点区间内，且与真值相差小
 bad = 0
 worst_rel = 0.0
 for _ in range(400):
 i = int(rng.integers(1, base.xs.size - 2))
 j = int(rng.integers(1, base.ys.size - 2))
 if not np.all(np.isfinite(base.E[j - 1:j + 2, i - 1:i + 2])):
 continue
 x = float(base.xs[i]) + float(rng.uniform(-0.49, 0.49)) * base.step
 y = float(base.ys[j]) + float(rng.uniform(-0.49, 0.49)) * base.step
 v, cv = base.lookup(x, y)
 if not bool(cv):
 continue
 nb = base.E[j - 1:j + 2, i - 1:i + 2]
 if not (nb.min() - 1e-9 <= float(v) <= nb.max() + 1e-9):
 bad += 1
 # 与最近邻的差应不超过 1 个格距内的局部变化
 worst_rel = max(worst_rel, abs(float(v) - base.E[j, i]) / max(abs(base.E[j, i]), 1e-9))
 rec("T3 双线性插值落在邻域值域内", bad == 0,
 f"越界 {bad} 次；与中心格最大相对差 {worst_rel:.3f}")

 # T4 未覆盖 / 超上限 -> e_max
 v_out, c_out = base.lookup(5000.0, 0.0)
 rec("T4 靶区外/无数据 -> 默认最大值 e_max",
 (not bool(c_out)) and abs(float(v_out) - E_MAX) < 1e-9,
 f"lookup(5000,0)={float(v_out)}, covered={bool(c_out)}")

 # T5 叠加 + 归一化 + 距离惩罚的结构性质
 f = ExpectField(base, FieldConfig(grid_step=100.0, dist_weight=0.35))

```

```

d0 = f.argmin((0.0, 0.0))
fb = f.fallback_point((0.0, 0.0), visited=[(0.0, 0.0)])
rec("T5a 无任何方位时退化为探索点（离已访问点最远）",
 d0 is None and fb["kind"] == "explore"
 and math.hypot(fb["xy"][0], fb["xy"][1]) > 1200.0,
 f"argmin=None, 探索点 {np.round(fb['xy'], 1).tolist()}（半径 "
 f"{math.hypot(*fb['xy']):.0f} m）")
for (x, y, t) in ((0.0, 0.0, 0.0), (0.0, 0.0, 90.0)):
 f.add_bearing((x, y), t)
sel = f.argmin((0.0, 0.0), visited=[(0.0, 0.0)])
sc, info = f.score((0.0, 0.0), visited=[(0.0, 0.0)])
i = sel["index"]
rec("T5b argmin 与全网格暴力最小一致",
 abs(float(sc[i]) - float(np.min(sc))) < 1e-9,
 f"argmin score={sel['score']:.6f}")
rec("T5c 归一化性质: E_norm ∈ [0,1] 且未覆盖格子=1",
 bool(np.all(info["e_norm"] <= 1.0 + 1e-9)) and
 float(np.max(info["e_norm"][info["coverage"] == 0])) == 1.0,
 f"max E_norm={float(np.max(info['e_norm'])):.4f}")
距离项的定量性质: score ≡ E_norm + λ·d/(2R_arena), 且 λ=0 时与距离无关
c0 = FieldConfig(grid_step=100.0, dist_weight=0.35, min_move=0.0)
f1 = ExpectField(base, c0)
for (x, y, t) in ((0.0, 0.0, 0.0), (0.0, 0.0, 90.0)):
 f1.add_bearing((x, y), t)
sc1, inf1 = f1.score((120.0, -340.0))
manual = inf1["e_norm"] + 0.35 * inf1["dist_m"] / DIST_REF
f0 = ExpectField(base, FieldConfig(grid_step=100.0, dist_weight=0.0, min_move=0.0))
for (x, y, t) in ((0.0, 0.0, 0.0), (0.0, 0.0, 90.0)):
 f0.add_bearing((x, y), t)
sc0, _ = f0.score((120.0, -340.0))
rec("T5d 距离层: score = E_norm + λ·d/(2R_arena) (λ=0 时与距离无关)",
 bool(np.all(np.isfinite(sc1)))
 and float(np.max(np.abs(sc1 - manual))) < 1e-12
 and float(np.max(np.abs(sc0 - inf1["e_norm"]))) < 1e-12,
 f"最大偏差 {float(np.max(np.abs(sc1 - manual)):.3e}")

T6 与精确实现交叉校验（用基准图族；含“射线被靶区截断”的病态情形）
fam = BaseMapFamily.load_dir(DEFAULT_FAMILY_DIR) if os.path.isdir(DEFAULT_FAMILY_DIR) else None
rec("T6a 基准图族可用（按先验外径 t_hi 分族）", fam is not None,
 "族目录 " + DEFAULT_FAMILY_DIR if fam is not None
 else f"未找到 {DEFAULT_FAMILY_DIR}（可运行 --build-family 生成）； "
 f"此时只能对 t_hi=1500 的几何严格成立")

def _cmp(rb_or_fam, cases, label):
 rels, n_local = [], 0
 worst = None
 for (Sd, thd, C) in cases:
 ts, w, _ = ray_source_samples(Sd, thd, n_t=200)
 if ts.size == 0:
 continue
 E_ex = exact_expected_diameter(Sd, thd, C, ts, w)
 if rb_or_fam is None:
 continue
 if isinstance(rb_or_fam, BaseMapFamily):
 rb, _t = rb_or_fam.rebase(Sd, thd)
 else:
 rb = rb_or_fam
 E_bm, _cov = rb.transformed_lookup(Sd, thd, C[:, 0], C[:, 1])
 u = np.array([math.cos(math.radians(thd)), math.sin(math.radians(thd))])
 G = np.asarray(Sd)[None, :] + ts[:, None] * u[None, :]
 for k in range(C.shape[0]):
 if not np.isfinite(E_ex[k]) or not np.isfinite(E_bm[k]):

```

```

 continue
 u1 = G - np.asarray(Sd)[None, :]
 u2 = G - C[k][None, :]
 cosang = (u1 * u2).sum(1) / np.maximum(
 np.linalg.norm(u1, axis=1) * np.linalg.norm(u2, axis=1), 1e-9)
 ang = np.degrees(np.arccos(np.clip(cosang, -1.0, 1.0)))
 gmin = min(float(np.min(ang)), float(np.min(180.0 - ang)))
 if gmin < 15.0: # 近简并: E 是尖峰, 插值无意义 (决策场里会截断成 e_max)
 continue
 rel = abs(float(E_bm[k]) - float(E_ex[k])) / max(float(E_ex[k]), 1e-9)
 rels.append(rel)
 n_local += 1
 if worst is None or rel > worst[0]:
 worst = (rel, float(E_bm[k]), float(E_ex[k]),
 float(np.hypot(*(C[k] - Sd))))

if not rels:
 return None
return {"n": n_local, "med": float(np.median(rels)),
 "p90": float(np.percentile(rels, 90)), "max": max(rels),
 "worst": worst, "label": label}

生成两组几何: S 在靶区中部 (先验基本不被截断) 与 S 贴近靶区边界且朝外测 (被截断)
cases_mid, cases_edge = [], []
for _ in range(6):
 Sd = np.round(rng.uniform(-700, 700, 2), 1)
 thd = float(rng.uniform(0, 360))
 cases_mid.append((Sd, thd, _ring_candidates(Sd, thd)))
for _ in range(6):
 a = float(rng.uniform(0, 360))
 Sd = 1560.0 * np.array([math.cos(math.radians(a)), math.sin(math.radians(a))])
 thd = float((a + rng.uniform(-25, 25)) % 360.0) # 朝靶区外测
 cases_edge.append((Sd, thd, _ring_candidates(Sd, thd)))
cases = [(S, t, C) for (S, t, C) in cases_mid + cases_edge if len(C)]

r_fam = _cmp(fam, cases, "family") if fam is not None else None
r_plain = _cmp(BaseMap.load_csv(base_csv) if os.path.exists(base_csv) else None,
 cases, "plain1500") if os.path.exists(base_csv) else None
if r_fam is not None:
 ok6 = (r_fam["med"] < 0.01) and (r_fam["p90"] < 0.05)
 detail6 = (f"{r_fam['n']} 个良态候选点 (含边界截断几何): 相对差 中位数 "
 f"{r_fam['med']:.4f}、P90 {r_fam['p90']:.4f}、最大 {r_fam['max']:.4f}")
 if r_plain is not None:
 detail6 += (f"; 不用族 (只按 t_hi=1500 的图) 时中位数 {r_plain['med']:.4f}、"
 f"最大 {r_plain['max']:.4f}")
elif r_plain is not None:
 ok6 = (r_plain["med"] < 0.01) and (r_plain["p90"] < 0.05)
 detail6 = (f"{r_plain['n']} 个良态候选点: 中位数 {r_plain['med']:.4f}、"
 f"P90 {r_plain['p90']:.4f}")
else:
 ok6, detail6 = True, "跳过 (既无基准图族也无单张基准图)"
rec("T6b 变换+插值 ≈ 精确期望 (良态几何)", ok6, detail6)

T7 反例: 忽略 t_hi (只用 t_hi=1500 的图) 在边界朝外几何上会明显失真
if fam is not None and r_plain is not None and r_fam is not None:
 edge_rel = [x for x in
 [_cmp(fam, [(S, t, C)], "f") for (S, t, C) in cases_edge
 if len(C)] if x]
 edge_plain = [x for x in
 [_cmp(BaseMap.load_csv(base_csv), [(S, t, C)], "p")
 for (S, t, C) in cases_edge if len(C)] if x]
 if edge_rel and edge_plain:
 med_f = float(np.median([e["med"] for e in edge_rel]))

```

```

 med_p = float(np.median([e["med"] for e in edge_plain]))
 rec("T7 边界朝外几何: 按 t_hi 分族显著优于固定 t_hi=1500",
 med_f < 0.05 and med_f < med_p / 5.0,
 f"族: 中位相对差 {med_f:.4f}; 固定 1500: {med_p:.4f} ("
 f"差 {med_p / max(med_f, 1e-6):.0f} 倍) ")

 if verbose:
 print(f"\n自检总结: {'全部通过' if ok_all else '存在失败项'}")
 return {"ok": ok_all, "checks": out}

演示 / 命令行

def field_figure(f, cur_xy, sel, path, title=None):
 """把决策场画出来: 叠加后的 E 场、距离惩罚、合成 score、选点与探测器位置。

 图注用英文 (本机 matplotlib 无中文字体, 避免缺字告警)。
 """
 import matplotlib
 matplotlib.use("Agg")
 import matplotlib.pyplot as plt

 E_mean, cov, n_terms = f.aggregate()
 score, info = f.score(cur_xy)
 fig, axes = plt.subplots(1, 3, figsize=(16.5, 5.6))
 panels = [(E_mean, "E[intersection diameter] (m)\n(uncovered cells = 1500)",
 "viridis_r"),
 (np.where(np.isfinite(info["dist_m"]),
 info["dist_m"] / DIST_REF, np.nan),
 f"distance term $\lambda \cdot d / (2R)$, $\lambda = \{f.cfg.dist_weight\}$ ", "magma"),
 (np.where(np.isfinite(score), score, np.nan),
 "score = normalised E + distance term\n($\Delta = \text{argmin}$)", "cividis_r")]
 for ax, (V, ttl, cmap) in zip(axes, panels):
 sc = ax.scatter(f.gx, f.gy, c=V, s=26, cmap=cmap, marker="s")
 fig.colorbar(sc, ax=ax, shrink=0.8)
 ax.set_aspect("equal")
 ax.set_title(ttl, fontsize=10)
 ax.set_xlim(-1950, 1950)
 ax.set_ylim(-1950, 1950)
 ax.set_xlabel("x (m)")
 ax.set_ylabel("y (m)")
 th = np.linspace(0, 2 * math.pi, 400)
 ax.plot(f.cfg.arena_r * np.cos(th), f.cfg.arena_r * np.sin(th), "k--", lw=0.8)
 ax.plot([cur_xy[0]], [cur_xy[1]], "r*", ms=14)
 for (S, thd, _tag, _thi) in [t[2] for t in f.terms]:
 a = math.radians(thd)
 ax.plot([S[0], S[0] + 300 * math.cos(a)],
 [S[1], S[1] + 300 * math.sin(a)], "w-", lw=1.2, alpha=0.8)
 if sel is not None:
 ax.plot([sel["xy"][0]], [sel["xy"][1]], "w^", ms=11, mec="k")
 ax.grid(alpha=0.2)
 fig.suptitle(title or (f"Q3 decision field: {n_terms} bearing/discovery terms "
 f"superimposed on the global grid"), fontsize=11)
 fig.tight_layout()
 os.makedirs(os.path.dirname(os.path.abspath(path)), exist_ok=True)
 fig.savefig(path, dpi=150)
 plt.close(fig)
 return path

def demo(base_csv=DEFAULT_BASE_CSV, outdir=None, dist_weight=1.5, grid_step=100.0,

```

```

 bearings=((0.0, 0.0, 0.0),), verbose=True, family_dir=DEFAULT_FAMILY_DIR):
 fam = BaseMapFamily.load_dir(family_dir) if os.path.isdir(family_dir) else None
 base = fam if fam is not None else BaseMap.load_csv(base_csv)
 cfg = FieldConfig(grid_step=grid_step, dist_weight=dist_weight)
 f = ExpectField(base, cfg)
 for (x, y, t) in bearings:
 f.add_bearing((x, y), t)
 sel = f.argmin((0.0, 0.0), visited=[(0.0, 0.0)])
 if verbose:
 kind = "基准图族" if fam is not None else f"单张基准图 {base.source}"
 print("基准: ", kind)
 print(f"叠加 {len(f.terms)} 条方位: 选点 {np.round(sel['xy'], 1).tolist()}, "
 f"E_mean={sel['E_mean_m']:.1f} m, E_norm={sel['E_norm']:.3f}, "
 f"距离 {sel['dist_m']:.0f} m, score={sel['score']:.4f}")
 if outdir:
 os.makedirs(outdir, exist_ok=True)
 p = f.save_csv(os.path.join(outdir, "p3_field.csv"))
 with open(os.path.join(outdir, "p3_field_report.json"), "w", encoding="utf-8") as fh:
 json.dump({"cfg": cfg.__dict__, "terms": len(f.terms), "pick": sel},
 fh, ensure_ascii=False, indent=2)
 try:
 fig = field_figure(f, (0.0, 0.0), sel,
 os.path.join(outdir, "p3_field.png"))
 except Exception as e:
 # 画图失败不影响主流程
 fig = f"figure_error: {e!r}"
 if verbose:
 print("写出", p, "与", fig)
 return f, sel
 return f, sel

def main():
 ap = argparse.ArgumentParser(description="问题3 期望决策场")
 ap.add_argument("--selfcheck", action="store_true")
 ap.add_argument("--demo", action="store_true")
 ap.add_argument("--build-family", action="store_true",
 help="按第二问口径重算一族基准图（不同先验外径 t_hi）")
 ap.add_argument("--base-map", default=DEFAULT_BASE_CSV)
 ap.add_argument("--family-dir", default=DEFAULT_FAMILY_DIR)
 ap.add_argument("--family-step", type=float, default=40.0)
 ap.add_argument("--family-nt", type=int, default=200)
 ap.add_argument("--out", default=None)
 ap.add_argument("--dist-weight", type=float, default=1.5,
 help="距离项系数 λ （问题 3 策略默认 1.5）")
 ap.add_argument("--grid-step", type=float, default=100.0)
 args = ap.parse_args()
 if args.selfcheck:
 res = selfcheck(args.base_map)
 raise SystemExit(0 if res["ok"] else 1)
 if args.build_family:
 print(f"计算基准图族 -> {args.family_dir}")
 BaseMapFamily.build(args.family_dir, step=args.family_step, n_t=args.family_nt)
 if args.demo or not args.build_family:
 demo(args.base_map, args.out, args.dist_weight, args.grid_step,
 family_dir=args.family_dir)

if __name__ == "__main__":
 main()

```

## 7.7 Q4/src/p3\_robot.py

"""B 题问题 3: 机器狗"基于期望的行动"在线搜索-定位-清除策略。

策略流程（与设计说明一一对应）

1. **\*\*第一轮扫描\*\***: 机器狗从原点出发, 测向机初始频道 1, 按频道 1..20 依次 ``/measure``, 得到若干条"检测点 + 示向度" (部分频道无信号属正常)。
2. **\*\*期望场选点\*\***: 对每个"只测到一次方位"的频道, 用第二问的网格模拟结果 (``out/p3\_family`` 或 ``out/p2\_grid``) 做旋转 + 平移 + 双线性插值, 把  $E[D]$  叠加到全局网格上; 没有数据/超阈值的格子取最大值 1500 m; 归一化后再叠加一层\*\*与探测器距离线性增长\*\*的代价 (系数  $\lambda$  可调), 取 ``argmin`` 作为下一个检测点, 随后开始新一轮 20 频道扫描。
3. **\*\*清理\*\***: 某些频道已经测到  $\geq 2$  条方位 -- 用问题 1 的交会算法算出定位区域 (最小包围圆 = 中心 + 半径  $r^*$ )。对所有可清理的频道按最近邻顺序规划航路, 依次前往; 到达区域后继续检测把区域压小, 直到进入清除范围, 再用 **\*\*覆盖式试探\*\*** (六边形格点, 间距  $\leq 20\sqrt{3}$  m, 保证圆域内任一点到某个试探点  $\leq 20$  m) 逐个 ``/clear``, 必然有一点成功。
4. **\*\*清除后继续扫描\*\***: 已经清除的频道、已经定位够的频道、以及"在同一个有效接收半径内重复测过的频道"都会被跳过, 从而节省时间。
5. 反复执行 2-4, 直到时间预算耗尽或没有新的可用信息。

关键设计

- \* **\*\*为什么"未覆盖 = 1500 m"是对的\*\***:  $E[D]$  越小定位越好; 把没有信息的格子记成最差, 只有在附近确实没有更好格子时才会选它 -- 这天然形成了"顺路探索"的行为。
- \* **\*\*为什么不能沿示向度方向逼近\*\***: 那会退化成问题 1 的"无界区域"几何。所以清理时的侧移精定位点取在\*\*与上一条视线近似正交\*\*的方向上。
- \* **\*\*清理阶段的覆盖式试探是"保证成功"的\*\***: 真源必落在  $\pm 1^\circ$  锥的交集 (定位区域) 内, 区域内任意一点到某个试探点的距离  $\leq 20$  m = 清除半径。

"""

```
from __future__ import annotations
```

```
import argparse
```

```
import json
```

```
import math
```

```
import os
```

```
import sys
```

```
import time
```

```
from dataclasses import dataclass, field as dc_field
```

```
import numpy as np
```

```
_HERE = os.path.dirname(os.path.abspath(__file__))
```

```
if _HERE not in sys.path:
```

```
 sys.path.insert(0, _HERE)
```

```
from p1_intersection import analyse # noqa: E402
```

```
from p3_arena import (# noqa: E402
```

```
 ArenaError, BEARING_ERR, CHANNELS, MockArena, NEAR_R, R_ARENA, R_CLEAR,
```

```
 T_CLEAR_FAIL, T_CLEAR_OK, T_MEASURE, T_SWITCH, V_ROBOT,
```

```
)
```

```
from p3_expect_field import (# noqa: E402
```

```
 DEFAULT_BASE_CSV, DEFAULT_FAMILY_DIR, BaseMap, BaseMapFamily, ExpectField,
```

```
 FieldConfig, best_refine_point,
```

```
)
```

```

```

```
覆盖式试探的三种结局
```

```

```

```
必须区分"预算不够、一个试探点都没来得及发"与"试探了但没打中":
```

```

前者不是策略失败（不该累加 clear_fail、不该写进论文的失败归因），
后者才是真正的"区域内没找到目标"。
SWEEP_OK = "success" # 某一点 /clear 成功
SWEEP_MISS = "miss" # 试探了全部候选点（或中途超时）都没打中
SWEEP_BUDGET = "no_budget" # 剩余预算不足，**一个 /clear 都没发出**，主动放弃

配置

@dataclass
class P3Config:
 # ---- 期望场（全局网格 + 距离惩罚） ----
 grid_step: float = 100.0 # 全局网格步长（m）
 e_max: float = 1500.0 # 未覆盖网格的默认最大值（m）
 dist_weight: float = 1.5 # 距离项系数 λ （离线演练调参得到，见 review/P3-design.md）
 visited_radius: float = 250.0 # 已扫描点邻域排除半径（m）
 min_move: float = 60.0 # 距当前位置过近的格子排除（m）
 field_terms: str = "pending" # pending: 只叠加"还缺方位"的频道
 field_discovery: bool = False # 始终叠加"探索项"（把扫描点推向能发现新源的地方）
 auto_discovery_when_empty: bool = True # 场上一条可用方位都没有时自动启用探索项

 # ---- 扫描 ----
 channels: tuple = CHANNELS
 scan_bearing_first: bool = True
 repeat_gap_m: float = 60.0 # 同一频道在附近重复测量的最小间隔（m）
 nosignal_gap_m: float = 1000.0 # "无信号"频道要换足够远的地方重测
 # (= 有效接收半径下界: 在 1000 m 外重测才有
 # 真正的新信息; 实测 450→1000 使检测次数
 # 289→186、平均定位清除时间 398→315 s,
 # 清除率 0.992→0.995, 见 review/P3-design.md §5.1) (m)
 enroute_scan: bool = True # 清理途中顺路补测
 enroute_scan_max: int = 2
 second_bearing_refine: bool = False # 新发现频道立刻补第二条方位（实测收益不稳，默认关）
 second_bearing_max: int = 2 # 每个停点最多补几条
 second_bearing_budget_s: float = 320.0 # 剩余时间低于它就不再补测
 max_rounds: int = 30
 tour: str = "nn2opt" # 清理航路: nn | nn2opt
 dynamic_replan: bool = True # 清理阶段每步重规划（滚动时域）; False=阶段内固定顺序
 clear_policy: str = "tour" # tour (NN+2-opt 全局结构) | rollout (一步前瞻+SPT)
 clear_budget_truncation: bool = False # 是否按剩余预算裁剪航路（实测关掉更好）
 optimistic_probe_steps: int = 3 # 打开裁剪时的试探点数封顶
 min_scan_rounds_before_clear: int = 1 # 先扫描几轮再开始清理

 # ---- 定位与清除 ----
 clear_min_bearings: int = 2
 clear_max_radius_m: float = 600.0 # 定位区域半径超过它就先不清理
 # (400 → 600: 扫圈在 $r \approx 1100$ m 处交会、区域半径
 # 常在 400-650 m, 400 会让这些源被"跳过";
 # 600 实测把扫圈的清除率从 0.958 提到 0.989-1.000)
 arrive_radius_m: float = 30.0
 direct_probe_radius_m: float = 60.0 # r^* 不超过它就无需侧移精定位，直接试探
 probe_spacing_m: float = 34.0 # $\leq 20\sqrt{3} \approx 34.6$ m 才能保证覆盖
 probe_max_points: int = 60 # 候选点数上限（仅"可覆盖"档位，见下）
 probe_unlimited: bool = True # True=不按点数截断（保证覆盖；截断会破坏 20 m 保证）
 probe_max_radius_m: float = 200.0
 refine_leg_m: float = 320.0
 refine_travel_cap_m: float = 700.0
 refine_mode: str = "perp" # perp（正交侧移）| exact（精确期望最优）
 max_approach_steps: int = 3

 # ---- 时间预算 ----

```



```

time_reserve_s: float = 3.0
min_action_budget_s: float = 10.0
min_scan_channels: int = 2 # 单轮扫描至少要能测这么多频道才值得跑一趟

---- 预算感知的扫描闸门 ("剩余时间不够就不要再扫") ----
诊断依据: 30 局里扫描阶段吃掉约 1016 s / 1200 s, 而末尾有 23% 的未清除源
已经定位好却没时间走过去。所以关键不是"怎么选扫描点", 而是"还要不要扫"。
scan_budget_gate: bool = False # 开: 本轮扫描后必须还够清最近的一个目标, 否则不扫
scan_gate_margin: float = 0.0 # 额外预留秒数 (越大越倾向"别扫了, 去清除")
scan_gate_factor: float = 1.0 # 对"清最近目标"耗时估计的放大系数
scan_only_toward_targets: bool = False # 扫描点若背着已知目标走, 则该轮不扫
扫描时间的硬性上限 (占总预算比例)。诊断显示扫描占用约 1016 s / 1200 s,
而 29/49 个"已定位未清除"的目标在变成可清目标时**剩余时间完全够用**却没去清。
scan_budget_frac: float = 1.0 # 1.0 = 不限制; 0.5 = 扫描累计不超过一半预算

---- 清理航路: 可达近点子集 ("优先清路程近的点") ----
clear_shortlist: bool = False # 开: 只保留"预算内可达"的近点子集, 再排序
clear_shortlist_reserve_s: float = 0.0 # 为后续扫描预留的秒数

频道台账

@dataclass
class ChannelRec:
 channel: int
 pts: list = dc_field(default_factory=list) # 测到方位的检测点
 svds: list = dc_field(default_factory=list) # 对应的示向度
 meas_pts: list = dc_field(default_factory=list) # 所有检测过的位置 (含无信号)
 meas_log: list = dc_field(default_factory=list) # (x, y, virtual_time, result)
 meas_stations: list = dc_field(default_factory=list) # 测过该频道的**站位编号**
 # (问题 4 的覆盖记账用: 只有"这一站"
 # 本来能听到它"才抵扣覆盖)

 cleared: bool = False
 cleared_at: float | None = None
 near_hits: int = 0
 no_signal: int = 0
 clear_fail: int = 0 # 自最近一次"新增方位"以来清理失败的次数
 last_bounded: tuple | None = None # (center, radius, n_bearings) 最近一次有界区域
 tried_probes: list = dc_field(default_factory=list)
 history: list = dc_field(default_factory=list)

 @property
 def n_bearings(self) -> int:
 return len(self.pts)

机器狗

def load_base(family_dir=DEFAULT_FAMILY_DIR, base_csv=DEFAULT_BASE_CSV):
 """优先用"基准图族" (按先验外径 t_hi 分族, 变换才严格成立), 否则退回单张基准图。"""
 if family_dir and os.path.isdir(family_dir):
 fam = BaseMapFamily.load_dir(family_dir)
 if fam is not None:
 return fam
 if os.path.exists(base_csv):
 return BaseMap.load_csv(base_csv)
 return None

def _dist_point_segment(p, a, b):

```

```

ab = b - a
L2 = float(ab @ ab)
if L2 < 1e-12:
 return float(np.linalg.norm(p - a))
t = float(np.clip((p - a) @ ab / L2, 0.0, 1.0))
return float(np.linalg.norm(p - (a + t * ab)))

def _point_in_polygon(p, P):
 """射线法 (P 为凸多边形顶点, 逆时针或顺时针均可)。"""
 n = len(P)
 inside = False
 for i in range(n):
 a, b = P[i], P[(i + 1) % n]
 if (a[1] > p[1]) != (b[1] > p[1]):
 x = a[0] + (p[1] - a[1]) * (b[0] - a[0]) / (b[1] - a[1])
 if x > p[0]:
 inside = not inside
 return inside

def _dist_point_polygon(p, P):
 p = np.asarray(p, float)
 P = np.asarray(P, float)
 if _point_in_polygon(p, P):
 return 0.0
 n = len(P)
 return min(_dist_point_segment(p, P[i], P[(i + 1) % n]) for i in range(n))

class P3Robot:
 def __init__(self, arena, cfg: P3Config | None = None, base=None, log=None,
 log_prefix=""):
 self.arena = arena
 self.cfg = cfg or P3Config()
 self.base = base if base is not None else load_base()
 self._log_fn = log
 self.log_prefix = log_prefix
 self.recs = {int(ch): ChannelRec(int(ch)) for ch in self.cfg.channels}
 self.scan_points: list[tuple] = []
 self.meas_pts: list[tuple] = [] # **真实做过的检测**位置 (覆盖度评估口径:
 # "到过的点"≠"测过的点", 覆盖度只能按后者算)

 self.events: list[dict] = []
 self.aborted = False
 # 覆盖式试探因"预算不足"放弃时置位: 让 clear_phase 立刻收敛,
 # 不要对同一个目标反复重规划 (实测会把重规划次数抬高一到两个数量级)。
 self.abort_clear_phase = False
 self.stats = {"n_measure": 0, "n_clear": 0, "n_clear_fail": 0,
 "n_scan_rounds": 0, "n_bearings": 0, "n_rejected": 0,
 "n_refine": 0, "n_second_bearing": 0, "n_replans": 0,
 "n_midphase_targets": 0, "n_sweep_aborted": 0,
 "n_sweep_miss": 0, "n_probe_issued": 0,
 "n_scan_gated": 0, "n_shortlist_kept": 0, "n_shortlist_in": 0,
 "t_scan_s": 0.0, "t_clear_s": 0.0, "t_clear_ends_s": 0.0,
 "vtime_at_last_clear": 0.0}

 # ----- 基础设施 -----
 def log(self, msg):
 self.events.append({"t": float(self.arena.virtual_time_s), "msg": msg})
 if self._log_fn:
 # 日志里避免使用 GBK 控制台无法编码的符号 (⇒ * √ × 等),
 # 万一还有, 退化为替换字符也不要让整局崩掉。

```

```

 try:
 self._log_fn(f"{self.log_prefix}{msg}")
 except UnicodeEncodeError:
 self._log_fn(f"{self.log_prefix}{msg}".encode(
 "gbk", "replace").decode("gbk"))

def remaining(self):
 return float(self.arena.remaining_budget_s())

def can_afford(self, seconds):
 return self.remaining() > float(seconds) + self.cfg.time_reserve_s

@property
def pos(self):
 return tuple(self.arena.pos)

----- 检测 / 清除 -----
def measure(self, x, y, ch):
 resp = self.arena.measure(x, y, ch)
 self.stats["n_measure"] += 1
 if resp.get("accepted") is not True:
 self.stats["n_rejected"] += 1
 self.aborted = True
 self.log(f" ! /measure 频道{ch} 未执行 ({resp.get('reason', '?')}) ")
 return resp
 res = resp.get("measure_result")
 rec = self.recs[int(ch)]
 rec.meas_pts.append((float(x), float(y)))
 rec.meas_log.append((float(x), float(y), float(self.arena.virtual_time_s), res))
 if res == "direction":
 self.add_bearing(int(ch), x, y, float(resp["svd_deg"]))
 elif res == "near":
 rec.near_hits += 1
 else:
 rec.no_signal += 1
 return resp

def add_bearing(self, ch, x, y, svd):
 rec = self.recs[int(ch)]
 rec.pts.append((float(x), float(y)))
 rec.svds.append(float(svd))
 rec.clear_fail = 0
 rec.tried_probes = []
 self.stats["n_bearings"] += 1
 reg = self.region(rec)
 if reg.get("status") == "bounded":
 rec.last_bounded = (tuple(reg["min_enclosing_center"]),
 float(reg["min_enclosing_radius_m"]), len(rec.pts))
 rec.history.append({"n": len(rec.pts), "x": float(x), "y": float(y),
 "svd": float(svd), "status": reg.get("status"),
 "r_star_m": (rec.last_bounded[1] if rec.last_bounded else None)})
 return reg

def do_clear(self, x, y, ch):
 resp = self.arena.clear(x, y, ch)
 self.stats["n_probe_issued"] = self.stats.get("n_probe_issued", 0) + 1
 if resp.get("accepted") is not True:
 self.stats["n_rejected"] += 1
 self.aborted = True
 self.log(f" ! /clear 频道{ch} 未执行 ({resp.get('reason', '?')}) ")
 return False
 if resp.get("clear_result") == "success":

```

```

 rec = self.recs[int(ch)]
 rec.cleared = True
 rec.cleared_at = float(self.arena.virtual_time_s)
 self.stats["n_clear"] += 1
 self.stats["vtime_at_last_clear"] = float(self.arena.virtual_time_s)
 self.log(f" [OK] 频道{ch} 已清除 (t={self.arena.virtual_time_s:.1f}s)")
 return True

 self.stats["n_clear_fail"] += 1
 return False

----- 定位区域 -----
def region(self, rec: ChannelRec):
 if not rec.pts:
 return {"status": "none", "n_bearings": 0}
 return analyse(rec.pts, rec.svds, BEARING_ERR)

def bounded_region(self, rec: ChannelRec):
 """当前（或最近一次）有界区域；返回 ``{'center', 'radius', 'poly', 'status'}``。"""
 if len(rec.pts) >= 2:
 reg = self.region(rec)
 if reg.get("status") == "bounded":
 return {"center": tuple(reg["min_enclosing_center"]),
 "radius": float(reg["min_enclosing_radius_m"]),
 "poly": [tuple(v) for v in reg.get("vertices", [])],
 "status": "bounded", "n_bearings": len(rec.pts)}
 if rec.last_bounded is not None:
 c, r, n = rec.last_bounded
 return {"center": tuple(c), "radius": float(r), "poly": None,
 "status": "fallback", "n_bearings": n}
 return None

----- 扫描 -----
def needed_channels(self):
 """还需要（且值得）拿到新方位的频道：已清除、已够定位两次的跳过。"""
 need, done = [], []
 for ch in self.cfg.channels:
 rec = self.recs[int(ch)]
 if rec.cleared or rec.n_bearings >= self.cfg.clear_min_bearings:
 continue
 (done if rec.pts else need).append(int(ch))
 order = (done + need) if self.cfg.scan_1bearing_first else sorted(done + need)
 return order

def channels_to_measure(self, limit=None, pos=None, subset=None, apply_gap=True):
 """在 ``pos`` 处值得检测的频道。

 跳过规则（"节省时间"的关键）：已清除 / 已定位两次的不测；
 对"无信号"频道，新检测点必须距它所有历史检测点 $\geq$ ``nosignal_gap_m``
 （有效接收半径至少 1000 m，在同一个圈里重复扫几乎拿不到新信息）；
 对已有一条方位的频道，新检测点距历史点 $\geq$ ``repeat_gap_m``（太小交会角没意义）。
 """
 p = np.asarray(pos if pos is not None else self.arena.pos, float)
 out = []
 for ch in (subset if subset is not None else self.needed_channels()):
 rec = self.recs[int(ch)]
 if apply_gap and rec.meas_pts:
 gap = self.cfg.repeat_gap_m if rec.pts else self.cfg.nosignal_gap_m
 d = min(math.hypot(p[0] - q[0], p[1] - q[1]) for q in rec.meas_pts)
 if d < gap:
 continue
 out.append(int(ch))
 return out[:limit] if limit else out

```

```

def build_field(self, grid_step=None, dist_weight=None):
 cfg = FieldConfig(grid_step=grid_step or self.cfg.grid_step,
 e_max=self.cfg.e_max,
 dist_weight=(self.cfg.dist_weight if dist_weight is None
 else dist_weight),
 visited_radius=self.cfg.visited_radius,
 min_move=self.cfg.min_move)
 f = ExpectField(self.base, cfg)
 if self.base is None:
 return f, 0
 n = 0
 for ch, rec in self.recs.items():
 if rec.cleared:
 continue
 if self.cfg.field_terms == "pending" and rec.n_bearings >= self.cfg.clear_min_bearings:
 continue
 if rec.n_bearings == 0:
 continue
 for ((x, y), th) in zip(rec.pts, rec.svds):
 f.add_bearing((x, y), th, tag=ch)
 n += 1
 if n == 0:
 # 一条"缺第二方位"的频道都没有（例如后半程只剩没听到过的频道）：
 # 自动退化为**探索项**，否则场上没有任何信息、选点无从谈起。
 for ch, rec in self.recs.items():
 if rec.cleared or rec.n_bearings > 0 or not rec.meas_pts:
 continue
 if not self.cfg.field_discovery and not self.cfg.auto_discovery_when_empty:
 continue
 f.add_raw(f.discovery_term(rec.meas_pts, self.cfg.nosignal_gap_m),
 tag=("disc", ch))
 n += 1
 return f, n

def choose_scan_point(self, dist_weight=None, max_dist=None):
 f, n = self.build_field(dist_weight=dist_weight)
 visited = list(self.scan_points)
 sel = f.argmin(self.pos, visited=visited, max_dist=max_dist)
 if sel is None:
 sel = f.fallback_point(self.pos, visited=visited, max_dist=max_dist)
 sel["n_bearing_terms"] = n
 return sel, f

def plan_scan(self, dist_weight=None):
 """选下一个检测点，并确定该点上值得测哪些频道。

 顺序很重要：**先**用期望场选点，**再**判断该点上哪些频道还没被"测透"，
 否则会出现"因为要换地方才有信息、但没地方可去"的死锁。
 """
 base = self.needed_channels()
 if not base:
 return None
 # 时间不够时把可选点限制在"还能走到"的范围内（给检测与安全余量留出空间）
 slack = 2.0 * (T_MEASURE + T_SWITCH) + 4.0
 dmax = max(0.0, (self.remaining() - self.cfg.time_reserve_s - slack) * V_ROBOT)
 sel, f = self.choose_scan_point(dist_weight=dist_weight, max_dist=dmax)
 chans = self.channels_to_measure(pos=sel["xy"], subset=base)
 if not chans:
 sel2 = f.fallback_point(self.pos, visited=self.scan_points)
 chans2 = self.channels_to_measure(pos=sel2["xy"], subset=base)
 if chans2:

```

```

 sel, chans = sel2, chans2
 if not chans: # 兜底：宁可多测几次也不原地空转
 chans = base[:3]
 # 时间不够时自动缩减本轮的频道数（至少测 min_scan_channels 个），把边角时间也用上
 travel = math.dist(self.pos, sel["xy"]) / V_ROBOT
 avail = self.remaining() - self.cfg.time_reserve_s - travel - 4.0
 kmax = max(1, int(avail // (T_MEASURE + T_SWITCH)))
 if len(chans) > kmax:
 if kmax < self.cfg.min_scan_channels:
 return None # 只剩时间测一两个频道：不值得为它专门跑一趟
 self.log(f" 时间只剩 {self.remaining():.0f} s -> 本轮只测 {kmax}/{len(chans)} 个频道")
 chans = chans[:kmax]
 cost = travel + len(chans) * (T_MEASURE + T_SWITCH) + 2.0
 plan = {"chans": chans, "sel": sel, "field": f, "cost_s": cost}
 if self.cfg.scan_budget_gate and not self._scan_worth_it(plan):
 return None
 return plan

----- 预算感知的扫描闸门 -----
def nearest_clear_cost_after(self, at_xy):
 """估计"到达 ``at_xy`` 之后，再清掉最近一个已知目标"所需秒数；没有已知目标则 None。"""
 cand = self.clear_targets()
 if not cand:
 return None
 nxt = min(cand, key=lambda t: math.dist(at_xy, t[2]["center"]))
 return self.estimate_clear_cost(nxt[2], from_pos=at_xy,
 probe_cap=self.cfg.optimistic_probe_steps)

def _scan_budget_left(self):
 """允许继续扫描的剩余秒数 (= frac × 总预算 - 已花在扫描上的时间)。"""
 if self.cfg.scan_budget_frac >= 1.0:
 return float("inf")
 return self.cfg.scan_budget_frac * float(self.arena.budget) \
 - self.stats.get("t_scan_s", 0.0)

def _scan_worth_it(self, plan):
 """本轮扫描值不值得做：**扫完还必须够清最近的一个已知目标**，否则不扫。

 动机（诊断数据）：30 局里扫描阶段平均吃掉约 1016 s / 1200 s，末尾有 23% 的
 未清除源**已经定位好却走不到**，也就是说"再多发现一个源"的边际价值，在
 剩余时间已经不足以走过去清掉它时等于 0 -- 这种扫描纯属消耗预算。

 注意这里只做"取舍"不做"折中"：要么整轮扫，要么整轮不扫；
 因为扫描点本身的移动无法中途变成对目标的逼近（两者方向不同）。
 """
 rem = self.remaining()
 left = self._scan_budget_left()
 if plan["cost_s"] > left:
 self.stats["n_scan_gated"] = self.stats.get("n_scan_gated", 0) + 1
 self.log(f" ■ 放弃本轮扫描：扫描时间额度已用尽（本轮需 {plan['cost_s']:.0f} s, "
 f"额度剩 {left:.0f} s = {self.cfg.scan_budget_frac:.2f}×总预算）"
 f"-> 预算留给已定位目标")
 return False
 cand = self.clear_targets()
 need = None
 if cand:
 nxt = min(cand, key=lambda t: math.dist(plan["sel"]["xy"], t[2]["center"]))
 need = self.estimate_clear_cost(nxt[2], from_pos=plan["sel"]["xy"],
 probe_cap=self.cfg.optimistic_probe_steps)
 # 可选：扫描点不能"背着"已知目标走（否则这轮的移动无法回收为进度）
 if self.cfg.scan_only_toward_targets:
 u = np.asarray(plan["sel"]["xy"], float) - np.asarray(self.pos, float)

```

```

 w = np.asarray(nxt[2]["center"], float) - np.asarray(self.pos, float)
 nu, nw = float(np.hypot(*u)), float(np.hypot(*w))
 if nu > 1e-9 and nw > 1e-9 and float(u @ w) / (nu * nw) < 0.0:
 self.stats["n_scan_gated"] = self.stats.get("n_scan_gated", 0) + 1
 self.log(f" ■ 放弃本轮扫描: 扫描点 ({plan['sel']['xy'][0]:.0f}, "
 f"{plan['sel']['xy'][1]:.0f}) 与已知目标方向相反 -> 预算留给清除")
 return False

 if need is None:
 return True # 手上没有可清目标: 扫描是唯一的进展方式
 required = (plan["cost_s"] + self.cfg.scan_gate_margin
 + self.cfg.scan_gate_factor * need)
 if rem >= required:
 return True
 self.stats["n_scan_gated"] = self.stats.get("n_scan_gated", 0) + 1
 self.log(f" ■ 放弃本轮扫描: 剩余 {rem:.0f} s < 本轮扫描 {plan['cost_s']:.0f} s "
 f"+ 清最近目标 {need:.0f} s (阈值 {required:.0f} s) "
 f"-> 预算留给已定位目标")
 return False

def execute_scan(self, plan):
 sel, chans = plan["sel"], plan["chans"]
 x, y = sel["xy"]
 if sel["kind"] == "explore":
 self.log(f"扫描轮: 期望场无可用方位 ({sel.get('n_bearing_terms', 0)} 项) "
 f"-> 探索点 ({x:.0f},{y:.0f}), 距 {sel['dist_m']:.0f} m")
 else:
 self.log(f"扫描轮: 选点 ({x:.0f},{y:.0f}) | E={sel['E_mean_m']:.1f} m "
 f"E_norm={sel['E_norm']:.3f} 距离惩罚={sel['dist_penalty']:.3f} "
 f"score={sel['score']:.4f} 移动 {sel['dist_m']:.0f} m | "
 f"待测 {len(chans)} 个频道 {chans}")
 self.scan_points.append((float(x), float(y)))
 n = self.scan_at(x, y, chans)
 self.stats["n_scan_rounds"] += 1
 return n

def scan_round(self, dist_weight=None):
 plan = self.plan_scan(dist_weight=dist_weight)
 if plan is None:
 return 0
 if not self.can_afford(plan["cost_s"]):
 return 0
 return self.execute_scan(plan)

def scan_at(self, x, y, chans):
 """在 (x, y) 依次检测这些频道 (第一次调用会带来移动耗时)。"""
 n = 0
 for i, ch in enumerate(chans):
 if self.aborted or not self.can_afford(T_MEASURE + T_SWITCH + 3.0):
 break
 resp = self.measure(x, y, ch)
 n += 1
 if resp.get("measure_result") == "near":
 # 已在 5 m 内: 直接清除
 self.do_clear(x, y, ch)
 return n

----- 清理 -----
def clear_targets(self):
 out = []
 for ch, rec in self.recs.items():
 if rec.cleared or rec.n_bearings < self.cfg.clear_min_bearings:
 continue

```

```

 if rec.clear_fail > 0:
 continue # 上次清理失败，等下一次新方位再试
 br = self.bounded_region(rec)
 if br is None or br["radius"] > self.cfg.clear_max_radius_m:
 continue
 out.append((ch, rec, br))
 out.sort(key=lambda t: math.dist(self.pos, t[2]["center"]))
 return out

----- 清理：滚动时域动态规划 -----
def _probe_count(self, br):
 """该定位区域需要多少个覆盖式试探点（带缓存：区域不变就不重算）。

 这里**故意**对点数封顶：它只用于“值不值得跑一趟”的代价预估，
 真实覆盖由 :meth:`probe_points` 保证。不封顶会让大 r^* 上的枚举变得很贵
 ($r^*=200$ m 时 151 个点 $\times$ 每步重规划)，实测把程序运行时间抬高两个数量级。
 """
 key = (round(float(br["center"][0]), 1), round(float(br["center"][1]), 1),
 round(float(br["radius"]), 1))
 cache = getattr(self, "_probe_cache", None)
 if cache is None:
 cache = self._probe_cache = {}
 if key not in cache:
 r = min(float(br["radius"]), self.cfg.probe_max_radius_m)
 n = len(self.probe_points(np.asarray(br["center"], float), r,
 poly=br.get("poly"))))
 cache[key] = min(int(n), int(self.cfg.probe_max_points))
 return cache[key]

def estimate_clear_cost(self, br, from_pos=None, n_probe=None, probe_cap=None):
 """估计从 ``from_pos`` 出发清掉某个目标的耗时（秒）。

 组成：走到区域中心 + 一次逼近检测 + 覆盖式试探（试探点数 $\times$ 平均步长/速度 + 3 s/次）
 + 成功清除 5 s。

 ``probe_cap`` 用于**乐观估计**：实际执行时试探是逐点进行的、随时可以中止
 （多数目标前几个试探点就成功），所以判断“值不值得跑一趟”时把试探数封顶，
 否则会过于保守、把还有 400 s 的尾巴白白浪费掉。
 """
 p = np.asarray(from_pos if from_pos is not None else self.pos, float)
 c = np.asarray(br["center"], float)
 if n_probe is None:
 n_probe = self._probe_count(br)
 if probe_cap is not None:
 n_probe = min(int(n_probe), int(probe_cap))
 t_probe = n_probe * (self.cfg.probe_spacing_m / V_ROBOT + T_CLEAR_FAIL)
 return (math.dist(p, c) / V_ROBOT + T_MEASURE + T_SWITCH
 + t_probe + T_CLEAR_OK)

def budget_feasible_shortlist(self, targets, start=None, budget=None):
 """从 ``targets`` 里挑出“剩余预算内走得完”的**近点子集**（贪心插入）。

 对“近点优先”的回答：滚动时域下每步只执行第一个目标，所以**只保留一个近点**
 在语义上已经等价于“优先清最近的点”；本函数真正的作用是判断
 “这一轮清理是否还排得下任何一个目标”，以及给排序一个**可达性**约束。

 排序键是“距当前位置”（近点优先），逐个累计
 ``走到区域中心 + 一次逼近检测 + 覆盖式试探 + 清除`` 的乐观代价，
 超预算就停止插入（后面更远的点更进不来）。
 """
 if not targets:
 return []

```



```

p = np.asarray(start if start is not None else self.pos, float)
rem = self.remaining() if budget is None else float(budget)
room = rem - self.cfg.time_reserve_s - self.cfg.min_action_budget_s \
 - self.cfg.clear_shortlist_reserve_s
ordered = sorted(targets, key=lambda t: math.dist(p, t[2]["center"]))
out, used = [], 0.0
for t in ordered:
 src = p if not out else np.asarray(out[-1][2]["center"], float)
 # estimate_clear_cost 已含"从 from_pos 走到区域中心"的行程
 cost = self.estimate_clear_cost(
 t[2], from_pos=src, probe_cap=self.cfg.optimistic_probe_steps)
 if used + cost > room:
 continue # 这个点排不进, 试下一个更远的 (更近的已进来)
 out.append(t)
 used += cost
return out

def plan_clear_tour(self, targets, remaining_s=None):
 """**动态规划 (滚动时域)**清理航路: 每执行一步就重新规划一次。

 与旧版 (清理阶段开始时规划一次、阶段内顺序冻结) 的区别:

 * 每次调用都重读台账 => **途中新定位出来的频道立刻进入航路**；
 * 各频道用**最新**的定位区域中心 (逼近过程中区域会变小、中心会移动)；
 * 以**当前位置**为起点重新做最近邻 + 2-opt (可用 ``tour`` 配置切换)；
 * 本步只执行规划结果的第一个, 执行完立刻重规划 (rolling horizon)。

 为什么默认不按剩余预算裁剪目标表：目标是"边走边判"执行的 --
 逼近、一次检测、逐点试探各自都检查剩余时间、随时可以中止, 多数目标在
 前几个试探点就成功。用统一的预估代价去裁剪, 会把还剩 300-400 s 的尾巴
 整段浪费掉 (实测: 裁剪 39.8% vs 不裁剪 41.2%)。故默认只排序、不裁剪,
 由 :meth:`clear_target` / :meth:`sweep_clear` 在动作粒度上兜底。
 ``clear_budget_truncation=True`` 可打开裁剪做对照。

 ``clear_policy`` 两种策略:
 * ``tour`` (默认): NN + 2-opt 的**全局航路结构**, 按顺序执行;
 * ``rollout``: 一步前瞻 + SPT 推演, 直接最大化"预算内可清除个数"
 (理论更贴合目标函数, 但实测更差 36.3% vs 41.2%: 贪心总挑最近的,
 最后把最远的目标拖成"永远清不掉")。
 """
 if not targets:
 return []
 self.stats["n_replans"] = self.stats.get("n_replans", 0) + 1
 remaining = self.remaining() if remaining_s is None else float(remaining_s)
 budget = remaining - self.cfg.time_reserve_s - self.cfg.min_action_budget_s
 if self.cfg.clear_policy == "rollout":
 return self._plan_rollout(targets, budget)
 if self.cfg.clear_policy == "tour_greedy_cost":
 # 贪心"每次挑预计剩余预算花得完、且最近的"目标, 滚动推进。
 # 与 'tour' 的差别: 这里显式用**剩余预算**逐目标判定可达性,
 # 而不是先排一条全局航路、再边走边判。
 return self._plan_greedy_cost(targets, remaining)
 # ---- 可选: 先用"可达点子集"过滤, 再做航路排序 ----
 pool = list(targets)
 if self.cfg.clear_shortlist:
 pool = self.budget_feasible_shortlist(pool, start=self.pos, budget=remaining)
 self.stats["n_shortlist_kept"] = self.stats.get("n_shortlist_kept", 0) \
 + len(pool)
 self.stats["n_shortlist_in"] = self.stats.get("n_shortlist_in", 0) \
 + len(targets)
 if not pool:
 return []

```

```

---- tour 策略: 全局航路结构 (可选预算截断) ----
order = self.order_targets(list(pool), from_pos=self.pos)
by_ch = {t[0]: t for t in pool}
ordered = [by_ch[ch] for ch in order if ch in by_ch]
if not self.cfg.clear_budget_truncation:
 return ordered
out, used, pos = [], 0.0, tuple(self.pos)
for t in ordered:
 c = self.estimate_clear_cost(t[2], from_pos=pos,
 probe_cap=self.cfg.optimistic_probe_steps)

 if used + c > budget:
 continue
 out.append(t)
 used += c
 pos = tuple(t[2]["center"])
return out

def _plan_greedy_cost(self, targets, remaining):
 """贪心 + 剩余预算可达性 (``clear_policy='tour_greedy_cost'``) 。”

 诊断动机: 末尾 26/49 个"已定位未清除"的目标在变成可清目标时剩余时间完全够用,
 却因为全局航路把它们排在后面而没轮到 (清完前面的目标后预算就没了)。
 本策略每一步只在"当前剩余预算真的能走完"的目标里挑最近的,
 从而避免"先承诺一个远点、结果后面全部落空"。

 与 ``clear_budget_truncation`` 的区别: 那个是对已排好的全局航路做事后裁剪,
 本策略是**在挑选时就**把可达性作为约束。
 """
 pos = np.asarray(self.pos, float)
 left = list(targets)
 out, used = [], 0.0
 room = remaining - self.cfg.time_reserve_s - self.cfg.min_action_budget_s
 while left:
 best, best_cost = None, None
 for t in left:
 c = self.estimate_clear_cost(t[2], from_pos=pos,
 probe_cap=self.cfg.optimistic_probe_steps)

 if best_cost is None or c < best_cost:
 best, best_cost = t, c
 if best is None or used + best_cost > room:
 break
 out.append(best)
 used += best_cost
 pos = np.asarray(best[2]["center"], float)
 left.remove(best)
 return out

def _plan_rollout(self, targets, budget):
 """一步前瞻 + SPT 推演 (``clear_policy='rollout'`` , 见 :meth:`plan_clear_tour`) 。”"""
 k = len(targets)
 pos0 = tuple(self.pos)
 centers = [tuple(t[2]["center"]) for t in targets]
 nprobe = []
 for t in targets:
 nprobe.append(self._probe_count(t[2]))
 t_probe = [n * (self.cfg.probe_spacing_m / V_ROBOT + T_CLEAR_FAIL) for n in nprobe]

 def leg(a, b):
 return (math.dist(a, centers[b]) / V_ROBOT + T_MEASURE + T_SWITCH
 + t_probe[b] + T_CLEAR_OK)

 cost0 = [leg(pos0, j) for j in range(k)]

```

```

def rollout(cur, left, tb):
 left = set(left)
 n = 0
 while True:
 best_c, best_j = None, None
 for j in left:
 c = leg(cur, j)
 if c <= tb and (best_c is None or c < best_c):
 best_c, best_j = c, j
 if best_j is None:
 return n
 tb -= best_c
 n += 1
 cur = centers[best_j]
 left.discard(best_j)

best_j, best_key = None, None
for j in range(k):
 if cost0[j] > budget:
 continue
 n = 1 + rollout(centers[j], [i for i in range(k) if i != j], budget - cost0[j])
 key = (n, -cost0[j])
 if best_key is None or key > best_key:
 best_key, best_j = key, j
if best_j is None:
 return []
out = [targets[best_j]]
used, pos = cost0[best_j], centers[best_j]
for j in sorted((i for i in range(k) if i != best_j), key=lambda i: leg(pos, i)):
 c = leg(pos, j)
 if used + c > budget:
 continue
 out.append(targets[j])
 used += c
 pos = centers[j]
return out

def order_targets(self, targets, from_pos=None):
 """航路排序：最近邻 + 2-opt（以 ``from_pos`` 为固定起点）。

 目标点用各自的（最新）定位区域中心； $k \leq 20$ ，暴力 2-opt 代价可忽略。

 ``tour`` 三种模式（对照用）：
 * ``nn`` ：纯最近邻 -- 第一步必然最近，但总路程偏长；
 * ``nn2opt`` ：最近邻 + 2-opt (**默认**) -- 总路程最短，但 2-opt 的
 区间反转会改写**第一步**（60 局实测有 14.1% 的排序其第一步不是最近的）；
 * ``nn2opt_keepfirst``：先钉住最近的点作第一步，只对**其后**的子序列做 2-opt。
 动机：滚动时域下**只执行第一步**，后面重排一次；所以"总路程最短"并不是
 正确的目标函数，"第一步最近的 + 剩余结构合理"才是。
 """
 start = tuple(self.pos if from_pos is None else from_pos)
 if len(targets) <= 1 or self.cfg.tour == "nn":
 targets.sort(key=lambda t: math.dist(start, t[2]["center"]))
 return [t[0] for t in targets]
 pts = [tuple(t[2]["center"]) for t in targets]
 k = len(pts)
 cur = start
 # 最近邻构造
 left = list(range(k))
 seq = []
 p = cur

```

```

while left:
 j = min(left, key=lambda i: math.dist(p, pts[i]))
 seq.append(j)
 left.remove(j)
 p = pts[j]

def total(order):
 d = math.dist(cur, pts[order[0]])
 for a, b in zip(order, order[1:]):
 d += math.dist(pts[a], pts[b])
 return d

钉住第一步时, 只允许反转 index >= 1 的区间
lo = 1 if (self.cfg.tour == "nn2opt_keepfirst" and k >= 2) else 0
best = total(seq)
improved = True
while improved:
 improved = False
 for i in range(lo, k - 1):
 for j in range(i + 1, k):
 cand = seq[:i] + seq[i:j + 1][::-1] + seq[j + 1:]
 d = total(cand)
 if d < best - 1e-9:
 seq, best, improved = cand, d, True
return [targets[i][0] for i in seq]

def clear_phase(self):
 """清理阶段: **每清一个目标就重新规划一次** (滚动时域)。

 循环体: 读最新台账 → 动态规划 → 只执行第一步 → 顺路补测 (可能新增可清理频道)
 → 回到循环顶部重新规划。这样"清除之后扫描过程中新定位的位置"会立刻进入航路,
 而且航路始终以当前位置为起点、用最新的区域中心优化。
 """
 first = self.clear_targets()
 if not first:
 return 0
 self.abort_clear_phase = False
 tour0 = self.plan_clear_tour(first) if self.cfg.dynamic_replan else None
 if self.cfg.dynamic_replan and not tour0:
 return 0 # 预算内一个都排不下: 静默返回, 让主循环把时间用于扫描
 if not self.can_afford(self.cfg.min_action_budget_s):
 return 0
 self.log(f"清理阶段 (动态重规划): {len(first)} 个频道已定位两次 -> "
 f"[{(t[0], round(t[2]['radius'])) for t in first}]")
 n = 0
 guard = 0
 static_order = None
 tour = tour0
 initial_chs = {t[0] for t in (tour0 or first)}
 midphase = 0
 while not self.aborted and guard < 4 * len(self.recs) + 8:
 guard += 1
 if self.cfg.dynamic_replan:
 tour = self.plan_clear_tour(self.clear_targets()) # ① 每步重新读台账并规划
 if not tour:
 self.log("剩余时间不足以完成任何一条清理航段 -> 结束清理阶段")
 break
 ch, rec, _br = tour[0] # ② 只执行第一步
 if ch not in initial_chs:
 midphase += 1 # 本阶段中途才定位出来的目标
 self.log(f"动态航路: 频道{ch} 是本阶段中途新定位的, 立即插队执行")
 elif len(tour) > 1:

```

```

 self.log(f" 动态航路: 本步 频道{ch}, 随后 "
 f"[t[0] for t in tour[1:4]]{'...' if len(tour) > 4 else ''}")
 else:
 # 静态对照 (消融合)
 targets = self.clear_targets()
 if not targets:
 break
 if not self.can_afford(self.cfg.min_action_budget_s):
 break
 if static_order is None:
 static_order = self.order_targets(list(targets))
 if not static_order:
 break
 ch = static_order.pop(0)
 t = next((x for x in targets if x[0] == ch), None)
 if t is None:
 continue
 rec = t[1]
 if self.clear_target(ch, rec):
 n += 1
 if self.abort_clear_phase:
 self.log("预算不足 -> 结束清理阶段 (剩余时间留给扫描)")
 break
 self.enroute_scan()
 # ③ 顺路补测 (可能新增可清理频道)
 self.stats["n_midphase_targets"] = self.stats.get("n_midphase_targets", 0) + midphase
 return n

def enroute_scan(self):
 """清理途中在当前位置顺手补测: 既补充方位, 也给新发现的频道补第二条方位。

 这一步很关键: 发现一个新频道只要 6 s, 但只有拿到**第二条**方位才能进入清理队列。
 所以一旦在本停点发现新频道, 就立刻用第二问的"局部最优第二检测点"再测一条,
 让它本轮就能被清除, 而不是等到下一次扫描 (往往等不到)。
 """
 if not self.cfg.enroute_scan or self.aborted:
 return 0
 chans = self.channels_to_measure(limit=self.cfg.enroute_scan_max)
 if not chans:
 return 0
 cost = len(chans) * (T_MEASURE + T_SWITCH) + 2.0
 if not self.can_afford(cost + self.cfg.min_action_budget_s * 0.5):
 return 0
 self.log(f" 顺路补测 {len(chans)} 个频道 {chans}")
 n = self.scan_at(self.pos[0], self.pos[1], chans)
 if self.cfg.second_bearing_refine:
 fresh = [ch for ch in chans
 if not self.recs[ch].cleared and self.recs[ch].n_bearings == 1]
 fresh.sort(key=lambda ch: -self.recs[ch].near_hits)
 for ch in fresh[:self.cfg.second_bearing_max]:
 if self.remaining() < self.cfg.second_bearing_budget_s + self.cfg.time_reserve_s:
 break
 self.acquire_second_bearing(ch)
 return n

def acquire_second_bearing(self, ch):
 """给"只有一条方位"的频道补第二检测点 (局部最优, 含行驶代价)。"""
 rec = self.recs[int(ch)]
 if rec.n_bearings != 1 or rec.cleared:
 return False
 p, info = best_refine_point(rec.pts, rec.svds, self.pos, None, n_t=100,
 travel_cap_m=self.cfg.refine_travel_cap_m * 1.4)
 if p is None:
 return False

```

```

trav = math.dist(self.pos, (float(p[0]), float(p[1])))
if not self.can_afford(trav / V_ROBOT + T_MEASURE + T_SWITCH + 20.0):
 return False
self.log(f" 频道{ch} 新方位 -> 第二检测点 ({p[0]:.0f},{p[1]:.0f}), "
 f"移动 {trav:.0f} m (E[D]≈{info['E_diam_m']:.0f} m, "
 f"预计代价 {info['cost_s']:.0f} s) ")
resp = self.measure(float(p[0]), float(p[1]), ch)
if resp.get("measure_result") == "near":
 return self.do_clear(float(p[0]), float(p[1]), ch)
self.stats["n_second_bearing"] = self.stats.get("n_second_bearing", 0) + 1
return rec.n_bearings >= 2

def choose_refine_point(self, ch, rec, br):
 """侧移精定位点：与上一条视线近似正交，把交会角拉到 60-90°。"""
 c = np.asarray(br["center"], float)
 if self.cfg.refine_mode == "exact":
 p, info = best_refine_point(rec.pts, rec.svds, self.pos,
 {"center": c}, n_t=120,
 travel_cap_m=self.cfg.refine_travel_cap_m)

 if p is not None:
 return np.asarray(p, float), info
 if not rec.pts:
 return None, None
 S_last = np.asarray(rec.pts[-1], float)
 u = c - S_last
 nu = float(np.linalg.norm(u))
 if nu < 1e-6:
 return None, None
 u = u / nu
 perp = np.array([-u[1], u[0]])
 rho = min(max(1.5 * br["radius"], 80.0), self.cfg.refine_leg_m)
 best = None
 for sgn in (1.0, -1.0):
 p = c + sgn * rho * perp
 if np.hypot(*p) > R_ARENA - 1.0:
 continue
 trav = float(np.hypot(*(p - np.asarray(self.pos, float))))
 if trav > self.cfg.refine_travel_cap_m:
 continue
 if best is None or trav < best[0]:
 best = (trav, p)
 if best is None:
 return None, None
 return best[1], {"mode": "perp", "rho": rho, "travel_m": best[0]}

def probe_points(self, c, r, poly=None, tried=()):
 """覆盖式试探点。

 * 有定位区域多边形 ``poly`` 时：只在"到多边形距离 ≤ 20 m"的格点上试
 -- 交会区域常常是细长四边形，用外接圆（面积 πr^2 ）试探会浪费一倍以上的点；
 * 没有多边形时退化为覆盖半径 r 的圆域。
 格点间距 ≤ $20/\sqrt{3}$ m，保证区域内任一点到某个试探点 ≤ 20 m = 清除半径，**必然成功**。

注意"必然成功"只在**候选点不被截断**时成立：``probe_max_points`` 会按"离中心由近及远"截断，等于丢掉覆盖区域的**外圈**，保证随之失效（实测 $r^* \geq 120$ m 即开始截断，60 个点对应 $r^* \approx 120$ m，而截断前需要 61 个）。因此默认

 ``probe_unlimited=True`` 取全部候选点；``probe_max_points`` 只在显式关掉

 ``probe_unlimited``（消融对照）时才起作用。

 """
 c = np.asarray(c, float)
 s = float(self.cfg.probe_spacing_m)
 rr = min(float(r), self.cfg.probe_max_radius_m) + R_CLEAR

```

```

u = np.array([s, 0.0])
v = np.array([s / 2.0, s * math.sqrt(3.0) / 2.0])
N = int(math.ceil(rr / s)) + 1
cand = []
for i in range(-N, N + 1):
 for j in range(-N, N + 1):
 p = c + i * u + j * v
 if float(np.hypot(*(p - c))) <= rr + 1e-9:
 cand.append(p)
if poly is not None and len(poly) >= 3:
 P = np.asarray(poly, float)
 keep = []
 for p in cand:
 if _dist_point_polygon(p, P) <= R_CLEAR + 1e-9:
 keep.append(p)
 cand = keep if keep else cand
cand.sort(key=lambda p: float(np.hypot(*(p - c))))
out = []
for p in cand:
 if any(float(np.hypot(*(p - np.asarray(q, float)))) < 2.0 for q in tried):
 continue
 out.append(p)
if self.cfg.probe_unlimited:
 return out
return out[:self.cfg.probe_max_points]

def sweep_clear(self, ch, rec, c, r, poly=None):
 """在区域 ``(c, r, poly)`` 上做覆盖式试探。

 返回 :data:`SWEEP_OK` / :data:`SWEEP_MISS` / :data:`SWEEP_BUDGET` 三者之一。
 预算不足时立刻返回 ``SWEEP_BUDGET``，一个 ``/clear`` 都不发 --
 否则会污染 ``clear_fail``、并在论文里被误读成"区域覆盖了却没找到目标"。
 """
 probes = self.probe_points(c, r, poly=poly, tried=rec.tried_probes)
 full = self.probe_points(c, r, poly=poly)
 n_try = 0
 for p in probes:
 if self.aborted:
 self.log(f" 覆盖式试探中断: 程序已中止 (已试 {n_try} 点)")
 return SWEEP_MISS if n_try else SWEEP_BUDGET
 if not self.can_afford(T_CLEAR_FAIL + 5.0):
 self.log(f" 覆盖式试探放弃: 剩余 {self.remaining():.0f} s 不足以支付一次 "
 f"/clear (候选 {len(full)} 个, 实际尝试 {n_try} 个)")
 self.stats["n_sweep_aborted"] += 1
 return SWEEP_BUDGET if n_try == 0 else SWEEP_MISS
 trav = math.dist(self.pos, (float(p[0]), float(p[1])))
 if not self.can_afford(trav / V_ROBOT + T_CLEAR_FAIL + 3.0):
 self.log(f" 覆盖式试探放弃: 剩余 {self.remaining():.0f} s 不足以走到下一个 "
 f"试探点 ({trav:.0f} m; 候选 {len(full)} 个, 实际尝试 {n_try} 个)")
 self.stats["n_sweep_aborted"] += 1
 return SWEEP_BUDGET if n_try == 0 else SWEEP_MISS
 n_try += 1
 rec.tried_probes.append((float(p[0]), float(p[1])))
 if self.do_clear(float(p[0]), float(p[1]), ch):
 self.log(f" 覆盖式试探命中: 第 {n_try}/{len(full)} 个候选点 "
 f" (间距 {self.cfg.probe_spacing_m:.0f} m)")
 return SWEEP_OK
 # 候选点全部试完仍未命中: 这才是真正的"区域内没找到目标"
 self.stats["n_sweep_miss"] += 1
 self.log(f" 覆盖式试探未命中: {n_try}/{len(full)} 个候选点全部试过 "
 f" ({'多边形区域' if poly else '外接圆'}, r*={r:.0f} m)")
 return SWEEP_MISS

```

```

def clear_target(self, ch, rec):
 """定位→逼近→（必要时）侧移精定位→覆盖式试探清除。

 失败归因**必须三分**（见 :data:`SWEEP_BUDGET`）:

 * ``SWEEP_OK`` -- 清除成功;
 * ``SWEEP_MISS`` -- 候选点全试过仍没打中 ⇒ 累加 ``clear_fail``（真的失败了）;
 * ``SWEEP_BUDGET`` -- 预算不足、**一个 /clear 都没发** ⇒ **不**累加 ``clear_fail``,
 也不报"未清除成功", 而是收敛回清理阶段由 ``plan_clear_tour`` 决定收工。
 这正是"预算耗尽后还调用 sweep_clear 并记一笔失败"那个统计/日志缺陷的修复点。
 """
 self.log(f"清理频道{ch}: 当前 {rec.n_bearings} 条方位, "
 f"起点 ({self.pos[0]:.0f},{self.pos[1]:.0f})")
 budget_stop = False
 # (1) 逼近定位区域中心（每次逼近顺带测一条方位，移动本来就要花）
 for _ in range(self.cfg.max_approach_steps):
 br = self.bounded_region(rec)
 if br is None:
 rec.clear_fail += 1
 return False
 c = np.asarray(br["center"], float)
 d = math.dist(self.pos, tuple(c))
 if d <= max(br["radius"], self.cfg.arrive_radius_m):
 break
 cost = d / V_ROBOT + T_MEASURE + T_SWITCH + 4.0
 if not self.can_afford(cost):
 # 走不过去了: 进入第 (3) 步, 由覆盖式试探统一判定"预算不足"
 budget_stop = True
 break
 resp = self.measure(float(c[0]), float(c[1]), ch)
 if resp.get("measure_result") == "near":
 return self.do_clear(float(c[0]), float(c[1]), ch)
 if self.aborted:
 return False
 # (2) 区域还大: 侧移一条正交视线, 把 r* 压小
 if not budget_stop:
 br = self.bounded_region(rec)
 if br is not None and br["radius"] > self.cfg.direct_probe_radius_m:
 p, info = self.choose_refine_point(ch, rec, br)
 if p is not None:
 trav = math.dist(self.pos, (float(p[0]), float(p[1])))
 cost = trav / V_ROBOT + T_MEASURE + T_SWITCH + 4.0
 if self.can_afford(cost):
 self.stats["n_refine"] += 1
 self.log(f" 侧移精定位 -> ({p[0]:.0f},{p[1]:.0f}), "
 f"移动 {trav:.0f} m ({info}) ")
 resp = self.measure(float(p[0]), float(p[1]), ch)
 if resp.get("measure_result") == "near":
 return self.do_clear(float(p[0]), float(p[1]), ch)
 # (3) 覆盖式试探清除
 br = self.bounded_region(rec)
 if br is None:
 rec.clear_fail += 1
 return False
 c = np.asarray(br["center"], float)
 r = min(br["radius"], self.cfg.probe_max_radius_m)
 poly = br.get("poly")
 st = self.sweep_clear(ch, rec, c, r, poly=poly)
 if st == SWEEP_OK:
 return True
 if st == SWEEP_BUDGET:

```



```

预算不足、"一个 /clear 都没发出": 不是清除失败, 不计 clear_fail、
也不上升为空/随机搜索; 交由调用方收敛本阶段。
self.abort_clear_phase = True
self.log(f" ↪ 频道{ch} 未尝试清除 (预算不足), 本阶段收敛")
return False

(4) 区域内没找到: 说明区域被算小了 (或误差超界), 扩大半径再试一轮
if br["radius"] <= self.cfg.probe_max_radius_m:
 r2 = min(br["radius"] * 1.6 + 20.0, self.cfg.probe_max_radius_m)
 self.log(f" 区域内未发现目标 -> 扩大到 r={r2:.0f} m 再试")
 st2 = self.sweep_clear(ch, rec, c, r2, poly=None)
 if st2 == SWEEP_OK:
 return True
 if st2 == SWEEP_BUDGET:
 self.abort_clear_phase = True
 self.log(f" ↪ 频道{ch} 扩大半径后未尝试 (预算不足), 本阶段收敛")
 return False
rec.clear_fail += 1
self.log(f" [x] 频道{ch} 本轮未清除成功 (候选点已试遍仍无目标, 留待后续扫描后再清)")
return False

----- 主循环 -----
def run(self):
 t0 = time.time()
 resp = self.arena.enter()
 if resp.get("accepted") is not True:
 raise ArenaError(f"/enter 未被接受: {resp}")
 self.log(f"进入靶区: 可用预算 {self.remaining():.0f} s"
 f" ({self.arena.budget_kind()}) ")
 try:
 # 第 0 轮: 机器狗本来就在当前位置 (原点), 直接扫描全部频道
 chans0 = self.channels_to_measure()
 self.log(f"第 0 轮扫描 (原地): 待测 {len(chans0)} 个频道 {chans0}")
 self.scan_points.append(self.pos)
 self.scan_at(self.pos[0], self.pos[1], chans0)
 rounds = 1
 stall = 0
 while (not self.aborted and rounds < self.cfg.max_rounds
 and self.remaining() > self.cfg.time_reserve_s):
 did = 0
 t_before = float(self.arena.virtual_time_s)
 if self.clear_targets() and (rounds >= self.cfg.min_scan_rounds_before_clear
 or not self.plan_scan()):
 did += self.clear_phase()
 self.stats["t_clear_s"] += (float(self.arena.virtual_time_s)
 - t_before)
 self.stats["t_clear_ends_s"] = float(self.arena.virtual_time_s)
 if self.aborted or self.remaining() <= self.cfg.time_reserve_s:
 break
 plan = self.plan_scan()
 if plan is None:
 # 扫描被预算闸门否掉时, plan_scan() 也可能只是"暂无值得测的频道";
 # 此时若还有可清目标, 就再走一次清理阶段, 而不是直接判为 stall。
 if self.clear_targets():
 did += self.clear_phase()
 self.stats["t_clear_s"] += (float(self.arena.virtual_time_s)
 - t_before)
 self.stats["t_clear_ends_s"] = float(self.arena.virtual_time_s)
 if self.aborted:
 break
 elif self.can_afford(plan["cost_s"]) \
 and (did == 0 or not self.clear_targets()):
 did += self.execute_scan(plan)

```

```

 self.stats["t_scan_s"] += (float(self.arena.virtual_time_s)
 - t_before)

 rounds += 1
 if did == 0:
 stall += 1
 if stall >= 2:
 self.log("没有可清理的频道、也没有值得检测的频道 -> 提前结束")
 break
 else:
 stall = 0

 finally:
 try:
 self.arena.exit()
 except ArenaError as e: # 接口已关闭等
 self.log(f"/exit 异常: {e}")

 self.stats["program_runtime_s"] = time.time() - t0
 return self.report()

----- 汇报 -----
def report(self):
 truth = self.arena.truth()
 n_cleared = sum(1 for r in self.recs.values() if r.cleared)
 total = (truth or {}).get("n_sources")
 vt = float(self.arena.virtual_time_s)
 last = float(self.stats.get("vtime_at_last_clear", vt))
 st = getattr(self.arena, "stats", {})
 rep = {
 "cleared": int(n_cleared),
 "n_sources": total,
 "clear_ratio": (n_cleared / total) if total else None,
 "virtual_time_s": vt,
 "avg_clear_time_s": (last / n_cleared) if n_cleared else None,
 "program_runtime_s": float(self.stats.get("program_runtime_s", 0.0)),
 "budget_kind": self.arena.budget_kind(),
 "remaining_budget_s": self.remaining(),
 "n_measure": self.stats["n_measure"],
 "n_bearings": self.stats["n_bearings"],
 "n_clear": self.stats["n_clear"],
 "n_clear_fail": self.stats["n_clear_fail"],
 "n_scan_rounds": self.stats["n_scan_rounds"],
 "n_refine": self.stats["n_refine"],
 "n_second_bearing": self.stats.get("n_second_bearing", 0),
 "n_replans": self.stats.get("n_replans", 0),
 "n_midphase_targets": self.stats.get("n_midphase_targets", 0),
 "n_rejected": self.stats["n_rejected"],
 "travel_m": float(st.get("travel_m", 0.0)),
 "mock_stats": dict(st),
 "n_sweep_aborted": self.stats.get("n_sweep_aborted", 0),
 "n_sweep_miss": self.stats.get("n_sweep_miss", 0),
 "n_probe_issued": self.stats.get("n_probe_issued", 0),
 "n_scan_gated": self.stats.get("n_scan_gated", 0),
 "t_scan_s": self.stats.get("t_scan_s", 0.0),
 "t_clear_s": self.stats.get("t_clear_s", 0.0),
 "t_clear_ends_s": self.stats.get("t_clear_ends_s", 0.0),
 "n_leftover_located": sum(
 1 for r in self.recs.values()
 if not r.cleared and r.n_bearings >= self.cfg.clear_min_bearings),
 "scan_points": [[round(float(x), 2), round(float(y), 2)]
 for (x, y) in self.scan_points],
 "channels": {str(ch): {"n_bearings": rec.n_bearings,
 "cleared": bool(rec.cleared),
 "no_signal": rec.no_signal,

```

```

 "near": rec.near_hits,
 "status": self.region(rec).get("status"),
 "r_star_m": (self.bounded_region(rec) or {}).get("radius")}
 for ch, rec in self.recs.items() if rec.n_bearings or rec.cleared,
 "truth": truth,
}
return rep

自检: 策略层的结构性断言 (离线 mock 上跑)

def selfcheck_mock(verbose=True, seed=20260913, cases=3):
 out = []
 ok_all = True

 def rec(name, ok, detail=""):
 nonlocal ok_all
 ok_all = ok_all and bool(ok)
 out.append({"check": name, "ok": bool(ok), "detail": detail})
 if verbose:
 print(f"[{'OK ' if ok else 'FAIL'}] {name} {detail}")

 # S1 覆盖式试探点确实能覆盖圆域 (任意点 20 m 内有试探点)
 rb = P3Robot(MockArena(seed=1), P3Config())
 rng = np.random.default_rng(7)
 worst = 0.0
 for (cx, cy, r) in ((0.0, 0.0, 40.0), (500.0, -300.0, 80.0), (-900.0, 900.0, 25.0)):
 pts = np.array(rb.probe_points((cx, cy), r))
 for _ in range(400):
 a = rng.uniform(0, 2 * math.pi)
 rad = r * math.sqrt(rng.random())
 p = np.array([cx + rad * math.cos(a), cy + rad * math.sin(a)])
 d = float(np.min(np.hypot(pts[:, 0] - p[0], pts[:, 1] - p[1])))
 worst = max(worst, d)
 rec("S1 覆盖式试探: 区域内任意点到最近试探点 <= 20 m (清除半径)", worst <= R_CLEAR + 1e-6,
 f"最大覆盖距离 {worst:.2f} m (试探点间距 {rb.cfg.probe_spacing_m} m)")

 # S2 端到端: mock 上跑通、清除率、时间预算
 ratios, times, cleared = [], [], []
 for k in range(cases):
 arena = MockArena(seed=seed + 100 * k)
 rb = P3Robot(arena, P3Config(), log=None)
 rep = rb.run()
 ratios.append(rep["clear_ratio"])
 times.append(rep["avg_clear_time_s"])
 cleared.append(rep["cleared"])
 if verbose:
 print(f" 案例{seed + 100 * k}: 清除 {rep['cleared']}/{rep['n_sources']}, "
 f"虚拟时间 {rep['virtual_time_s']:.0f}s, "
 f"平均 {rep['avg_clear_time_s']:.1f}s, "
 f"扫描 {rep['n_scan_rounds']} 轮, 检测 {rep['n_measure']} 次")
 rec("S2 mock 演练: 清除率>0 且每个被清除源平均时间有限",
 all(r is not None and r > 0 for r in ratios) and all(t and t > 0 for t in times),
 f"{cases} 局: 清除率 {[round(r, 3) for r in ratios]}, 平均时间 "
 f"{[round(t, 1) for t in times]} s")

 # S3 策略不会超预算 (mock 会计时; 超时会 accepted=false)
 arena = MockArena(seed=seed + 7, budget_s=400.0)
 rb = P3Robot(arena, P3Config(), log=None)
 rep = rb.run()
 rec("S3 策略在时间预算内收工 (不触发超时拒动)",

```

```

rep["n_rejected"] == 0 and rep["virtual_time_s"] <= 400.0 + 1e-9,
f"虚拟时间 {rep['virtual_time_s']:.1f}s / 400s, 拒动 {rep['n_rejected']} 次")

S4 已清除的频道不再被检测 ("跳过已清理过的点, 节省时间")
arena = MockArena(seed=seed + 11)
rb = P3Robot(arena, P3Config(), log=None)
rep = rb.run()
after = {}
for ch, r in rb.recs.items():
 if r.cleared and r.cleared_at is not None:
 k = sum(1 for (_, y, t, _res) in r.meas_log if t > r.cleared_at + 1e-9)
 if k:
 after[ch] = k
rec("S4 已清除频道不再被检测", not after,
f"已清除 {sum(1 for r in rb.recs.values() if r.cleared)} 个频道; "
f"清除后仍被检测的频道 {after or '无'}")

S5 定位区域一定包含真源 (问题 1 的交会算法 + 模拟器误差界的一致性)
bad5 = []
n5 = 0
src_map = {s["channel"]: (s["x_m"], s["y_m"]) for s in (rep["truth"] or {}).get("sources", [])}
for ch, r in rb.recs.items():
 if r.n_bearings < 2:
 continue
 br = rb.bounded_region(r)
 if not br or not br.get("poly"):
 continue
 n5 += 1
 xy = src_map.get(ch)
 if xy is None:
 continue
 P = np.asarray(br["poly"], float)
 if not _point_in_polygon(np.asarray(xy, float), P):
 bad5.append(ch)
rec("S5 定位区域 (多边形) 包含真源", not bad5,
f"检查 {n5} 个已定位频道, 越界 {bad5 or '无'}")

S6 探索兜底: 没有任何方位时选"离已访问点最远"的点
rb6 = P3Robot(MockArena(seed=seed + 3), P3Config(), log=None)
sel6, _f = rb6.choose_scan_point()
rec("S6 无方位时退化为探索点 (离已访问点最远)",
sel6["kind"] == "explore" and math.hypot(*sel6["xy"]) > 1000.0,
f"探索点 {tuple(round(v) for v in sel6['xy'])}, 距原点 "
f"{math.hypot(*sel6['xy']):.0f} m")

S7 动态重规划: 每清一个目标都重规划一次; 静态对照为 0
arena7 = MockArena(seed=seed + 13)
rb7 = P3Robot(arena7, P3Config(), log=None)
rep7 = rb7.run()
arena7s = MockArena(seed=seed + 13)
rb7s = P3Robot(arena7s, P3Config(dynamic_replan=False), log=None)
rep7s = rb7s.run()
rec("S7 动态重规划: 清除途中每步重规划, 静态对照 0 次",
rep7["n_replans"] >= rep7["cleared"] >= 1 and rep7s["n_replans"] == 0,
f"动态: 清除 {rep7['cleared']} 个、重规划 {rep7['n_replans']} 次、"
f"中途新定位插队 {rep7['n_midphase_targets']} 个; "
f"静态: 重规划 {rep7s['n_replans']} 次、中途插队 {rep7s['n_midphase_targets']} 个")

S8 动态航路: 默认不裁剪 (目标齐全)、顺序与航路排序一致
rb8 = P3Robot(MockArena(seed=seed + 21), P3Config(), log=None)
rb8.arena.enter()
for ch, (gx, gy) in ((1, (300.0, 200.0)), (2, (-500.0, 700.0)), (3, (900.0, -400.0))):

```

```

r8 = ChannelRec(ch)
在真源周围取两条正交视线（交会角 90°）⇒ 定位区域小且有界
for ang, dist in ((0.0, 500.0), (90.0, 500.0)):
 sx = gx + dist * math.cos(math.radians(ang))
 sy = gy + dist * math.sin(math.radians(ang))
 a = math.degrees(math.atan2(gy - sy, gx - sx)) % 360.0
 r8.pts.append((sx, sy))
 r8.svds.append(a)
rb8.recs[ch] = r8
tg8 = rb8.clear_targets()
tour8 = rb8.plan_clear_tour(tg8)
rec("S8 动态航路：默认不裁剪、顺序与航路排序一致",
 len(tg8) == 3 and len(tour8) == 3
 and [t[0] for t in tour8] == rb8.order_targets(list(tg8)),
 f"{len(tg8)} 个已定位频道 -> 航路 {[t[0] for t in tour8]}, "
 f"各区域半径 {[round(t[2]['radius']) for t in tour8]} m")

S9 覆盖式试探**不得被点数上限截断**（截断会丢掉外圈、破坏 20 m 覆盖保证）
rb9 = P3Robot(MockArena(seed=seed + 31), P3Config(), log=None)
n9_uncapped = len(rb9.probe_points(np.zeros(2), 195.0))
rb9c = P3Robot(MockArena(seed=seed + 31),
 P3Config(probe_unlimited=False, probe_max_points=60), log=None)
n9_capped = len(rb9c.probe_points(np.zeros(2), 195.0))
覆盖保证：r*=195 m 的外接圆内任一点到最近候选点必须 <= 20 m
pts9 = np.asarray(rb9.probe_points(np.zeros(2), 195.0), float)
rng9 = np.random.default_rng(11)
worst9 = 0.0
for _ in range(400):
 a = rng9.uniform(0, 2 * math.pi)
 rad = 195.0 * math.sqrt(rng9.random())
 p = np.array([rad * math.cos(a), rad * math.sin(a)])
 worst9 = max(worst9, float(np.min(np.hypot(pts9[:, 0] - p[0],
 pts9[:, 1] - p[1]))))
rec("S9 覆盖式试探不被截断（大 r* 时仍满足 20 m 覆盖保证）",
 worst9 <= R_CLEAR + 1e-6 and n9_uncapped > n9_capped,
 f"r*=195 m: {n9_uncapped} 个候选点（封顶对照 {n9_capped} 个），"
 f"最大覆盖距离 {worst9:.2f} m")

S10 预算不足时**不得**把"未尝试"记成"清除失败"（失败归因三分）
arena10 = MockArena(seed=seed + 41)
rb10 = P3Robot(arena10, P3Config(), log=None)
rb10.arena.enter()
rec10 = ChannelRec(4)
for (sx, sy) in ((0.0, 0.0), (0.0, 300.0)): # 两条正交视线 -> 小而紧的区域
 a = math.degrees(math.atan2(150.0 - sy, 100.0 - sx)) % 360.0
 rec10.pts.append((sx, sy))
 rec10.svds.append(a)
rb10.recs[4] = rec10
br10 = rb10.bounded_region(rec10)
arena10.vtime = arena10.budget - 0.5 # 只剩 0.5 s: 连一次 /clear 都付不起
f10 = rb10.remaining()
st10 = rb10.sweep_clear(4, rec10, np.asarray(br10["center"], float),
 float(br10["radius"]), poly=br10.get("poly"))
rec("S10 预算不足时不误记 clear_fail / 不发 /clear",
 st10 == SWEEP_BUDGET and rec10.clear_fail == 0
 and rb10.stats["n_probe_issued"] == 0
 and rb10.stats["n_sweep_aborted"] == 1,
 f"剩余预算 {f10:.1f} s（付不起一次 /clear）⇒ 返回 {st10!r}, "
 f"clear_fail={rec10.clear_fail}, 发出 /clear {rb10.stats['n_probe_issued']} 次, "
 f"n_sweep_aborted={rb10.stats['n_sweep_aborted']}")

S11 区域中心在"排序决策 → 执行"之间**不发生漂移**（否则排序依据会过期）

```

```

arena11 = MockArena(seed=seed + 51)
rb11 = P3Robot(arena11, P3Config(), log=None)
orig_plan11 = rb11.plan_clear_tour
orig_clear11 = rb11.clear_target
snap11 = {}
drift11 = []

def plan11(targets, remaining_s=None):
 for t in (targets or []):
 snap11[t[0]] = tuple(t[2]["center"])
 return orig_plan11(targets, remaining_s=remaining_s)

def clear11(ch, rec):
 br = rb11.bounded_region(rec)
 if br is not None and ch in snap11:
 drift11.append(math.dist(snap11[ch], tuple(br["center"])))
 return orig_clear11(ch, rec)

rb11.plan_clear_tour = plan11
rb11.clear_target = clear11
rb11.run()
rec("S11 区域中心在'排序决策→执行'之间不漂移",
 bool(drift11) and max(drift11) == 0.0,
 f"检查 {len(drift11)} 次执行, 最大漂移 {max(drift11) if drift11 else float('nan'):.1f} m"
 f"(新方位只在 /measure 后产生, 逼近途中不产生)")

S12 航路排序的三种模式: 第一步必为最近邻 (nn / nn2opt_keepfirst),
或允许 2-opt 改写第一步但总路程不更差 (nn2opt)
rb12 = P3Robot(MockArena(seed=seed + 61), P3Config(), log=None)
rb12.arena.enter()
pts12 = [(300.0, 200.0), (-500.0, 700.0), (900.0, -400.0),
 (-1200.0, -300.0), (200.0, -1500.0)]
for i, (gx, gy) in enumerate(pts12, start=1):
 r12 = ChannelRec(i)
 for ang in (0.0, 90.0):
 sx = gx + 500.0 * math.cos(math.radians(ang))
 sy = gy + 500.0 * math.sin(math.radians(ang))
 r12.pts.append((sx, sy))
 r12.svds.append(math.degrees(math.atan2(gy - sy, gx - sx)) % 360.0)
 rb12.recs[i] = r12
tg12 = rb12.clear_targets()
start12 = tuple(rb12.pos)
nearest12 = min(tg12, key=lambda t: math.dist(start12, t[2]["center"]))[0]

def plen12(order):
 seq = [next(t for t in tg12 if t[0] == ch) for ch in order]
 d = math.dist(start12, seq[0][2]["center"])
 for a, b in zip(seq, seq[1:]):
 d += math.dist(a[2]["center"], b[2]["center"])
 return d

rb12.cfg.tour = "nn"
o_nn = rb12.order_targets(list(tg12))
rb12.cfg.tour = "nn2opt_keepfirst"
o_kf = rb12.order_targets(list(tg12))
rb12.cfg.tour = "nn2opt"
o_2o = rb12.order_targets(list(tg12))
rec("S12 航路排序: nn 与 keepfirst 第一步为最近邻; 2-opt 不劣于 nn",
 o_nn[0] == nearest12 and o_kf[0] == nearest12
 and plen12(o_2o) <= plen12(o_nn) + 1e-9,
 f"最近目标=频道{nearest12}; nn 第一步={o_nn[0]}, keepfirst={o_kf[0]}, "
 f"2-opt={o_2o[0]}; 路程 nn={plen12(o_nn):.0f} m, 2-opt={plen12(o_2o):.0f} m")

```

```

 if verbose:
 print(f"\n自检总结: {'全部通过' if ok_all else '存在失败项'}")
 return {"ok": ok_all, "checks": out}

def main():
 ap = argparse.ArgumentParser(description="问题3 机器狗策略（离线自检 / 单局演练）")
 ap.add_argument("--selfcheck", action="store_true")
 ap.add_argument("--mock", action="store_true", help="跑一局离线演练并打印明细")
 ap.add_argument("--seed", type=int, default=20260913)
 ap.add_argument("--dist-weight", type=float, default=1.5)
 ap.add_argument("--grid-step", type=float, default=100.0)
 ap.add_argument("--quiet", action="store_true")
 args = ap.parse_args()
 if args.selfcheck:
 res = selfcheck_mock(verbose=True)
 raise SystemExit(0 if res["ok"] else 1)
 cfg = P3Config(dist_weight=args.dist_weight, grid_step=args.grid_step)
 arena = MockArena(seed=args.seed)
 rb = P3Robot(arena, cfg, log=None if args.quiet else print)
 rep = rb.run()
 print(json.dumps({k: v for k, v in rep.items()
 if k not in ("truth", "channels", "mock_stats")},
 ensure_ascii=False, indent=2))

if __name__ == "__main__":
 main()

```

## 7.8 Q4/src/p4\_grid.py

"""问题 4 的扫描几何：三角形格网（triangular lattice）覆盖半径与"必被听到"判据。

用户的设想

- > 如果取一个合适大小的三角形，它的三个顶点里\*\*至少有一个\*\*能接收到落在它内部的
- > 干扰源信号；要保证一定能发现，边长取 1000 m 左右比较合适。于是用三角形网格
- > 覆盖整个区域，会形成内外两个圈；先从原点出发到第一个圈上的点扫描，再到第二个
- > 圈上的顶点扫描，就基本能覆盖所有干扰源。

本模块把这个直觉写成\*\*可计算的判据\*\*，并给出格网参数。

三个定义（都被后面的数值报告使用）

1. \*\*覆盖半径\*\*  $\text{cov}(p)$  =  $p$  到最近停靠点的距离。源的接收半径  $\geq 1000$  m，所以  $\text{cov}(p) > 1000 \Rightarrow p$  处若有源，任何停靠点都听不到它。
2. \*\*定向覆盖半径\*\*  $\text{dir\_cov}(p)$  =  $p$  到最近\*\*可被听到\*\*停靠点的距离：

$$q \text{ 能听到 } p \text{ 处的源} \iff |q-p| \leq 1000 \text{ 且 } (q-p) \cdot d \geq 0$$

其中  $d$  是源的定向方向（ $180^\circ$  覆盖角）。后一个条件等价于

" $q$  落在以  $p$  为界、朝向  $d$  的那个闭半平面里"。

3. \*\*顶点三向覆盖（用户的直觉）\*\*：若  $p$  落在三角形  $v_1v_2v_3$  内部，则  $p = \sum \lambda_i v_i$  ( $\lambda_i \geq 0$ )，于是对任意方向  $d$  有  $\sum \lambda_i (v_i - p) \cdot d = 0$   
 $\Rightarrow$  必有某个  $i$  使  $(v_i - p) \cdot d \geq 0$ 。 \*\*换句话说：三角形的三个顶点方向正向张成整个平面，任何定向方向都至少被一个顶点"看到"。 \*\*  
 所以只要再保证  $\max_i |v_i - p| \leq 1000$ （等边三角形内的上界为边长  $a$ ），这个三角形内的任何源（不论朝向）都必被某个顶点听到。

推论：完整三角剖分且边长  $a \leq 1000$  m 是充分条件；还必须覆盖靶区边界。  
 $a/\sqrt{3}$  仅表示最近格点距离，不能代替最远顶点距离。下方离散评估只作诊断；  
 连续位置与任意朝向的完成证书见 `p4_certificate.py`。

```

"""
from __future__ import annotations

import math

import numpy as np

from p3_arena import R_ARENA, R_RECV_MIN

三角形格网的基向量（边长为 a 的等边三角形）：u = (a,0), v = (a/2, a√3/2)
SQRT3 = math.sqrt(3.0)

格网生成

def lattice_index_range(a, r_cover):
 """列出"到原点距离 ≤ r_cover"的格点所需的格坐标范围。"""
 k = int(math.ceil(r_cover / (a * SQRT3 / 2.0))) + 1
 return range(-k, k + 1)

def triangular_lattice(a, r_cover=R_ARENA, origin=(0.0, 0.0), dedup=True):
 """边长为 ``a`` 的三角形格网中，到 ``origin`` 距离 ≤ ``r_cover`` 的全部顶点。

 这是"覆盖整个靶区"所需的**一对一点集**（每个点在格网里的坐标唯一）。
 """
 ux, uy = float(a), 0.0
 vx, vy = float(a) / 2.0, float(a) * SQRT3 / 2.0
 ox, oy = float(origin[0]), float(origin[1])
 out = []
 seen = set()
 for i in lattice_index_range(a, r_cover):
 for j in lattice_index_range(a, r_cover):
 x = i * ux + j * vx
 y = i * uy + j * vy
 if math.hypot(x - ox, y - oy) <= r_cover + 1e-9:
 key = (round(x, 6), round(y, 6))
 if dedup and key in seen:
 continue
 seen.add(key)
 out.append((x, y))
 out.sort(key=lambda p: (math.hypot(p[0] - ox, p[1] - oy),
 math.atan2(p[1] - oy, p[0] - ox)))
 return out

def hex_ring_lattice(a, rings=2, origin=(0.0, 0.0)):
 """同心六边形环形式的格点：第 k 环半径 = k·a，含 6k 个点。

 与 :func:`triangular_lattice` 是同一个格网，只是**按环分组**--
 这正是用户说的"内外两个圈"：环 0 = 原点，环 1 = 一圈 6 个点，
 环 2 = 一圈 12 个点。按环序走就是"先内圈、再外圈"的扫描顺序。
 """
 ox, oy = float(origin[0]), float(origin[1])
 u = np.array([float(a), 0.0])
 v = np.array([float(a) / 2.0, float(a) * SQRT3 / 2.0])
 out = [[] for _ in range(int(rings) + 1)]
 seen = set()

```



```

for i in range(-int(rings), int(rings) + 1):
 for j in range(-int(rings), int(rings) + 1):
 p = i * u + j * v
 r = float(np.hypot(*p))
 k = int(round(r / float(a)))
 if k > int(rings):
 continue
 key = (round(float(p[0]), 6), round(float(p[1]), 6))
 if key in seen:
 continue
 seen.add(key)
 out[k].append((ox + float(p[0]), oy + float(p[1])))
for k in range(len(out)):
 out[k].sort(key=lambda q: math.atan2(q[1] - oy, q[0] - ox) % (2 * math.pi))
return out

覆盖判据（数值）

def eval_grid(step=20.0, r_arena=R_ARENA):
 """用于评估的细网格（默认 20 m，靶区内约 2.5 万个点）。"""
 xs = np.arange(-r_arena, r_arena + 1e-9, step)
 GX, GY = np.meshgrid(xs, xs)
 rr = np.hypot(GX, GY)
 m = rr <= r_arena + 1e-9
 return GX[m].copy(), GY[m].copy()

def cov_stats(pts, gx=None, gy=None, r_cover=R_RECV_MIN):
 """``(最大未覆盖距离，未覆盖点占比)``：到最近停靠点的距离分布。

 完备性判据：``最大未覆盖距离 <= 1000 m`` $\Rightarrow$ 任一源（不论朝向）都会被某个
 停靠点听到 -- 对**全向**源严格成立；对定向源还需要半平面条件，见
 :func:`dir_cov_stats`。
 """
 if gx is None:
 gx, gy = eval_grid()
 P = np.asarray(pts, float).reshape(-1, 2)
 d2 = ((gx[:, None] - P[None, :, 0]) ** 2 + (gy[:, None] - P[None, :, 1]) ** 2)
 dmin = np.sqrt(np.min(d2, axis=1))
 return float(dmin.max()), float((dmin > float(r_cover)).mean())

def dir_cov_stats(pts, gx=None, gy=None, r_cover=R_RECV_MIN, r_arena=R_ARENA,
 n_dir=16, origin=(0.0, 0.0), chunk=4000):
 """定向源的"必被听到"判据：对每个评估点、每个方向求最近**可听**停靠点距离。

 ``dir_cov(p)`` = min over d of max over admissible q ... 不对，正确口径是：
 对给定方向 d，"能被听到"要求存在 q 同时满足距离与半平面条件，所以

$$\text{dir_cov}(p, d) = \min\{ |q-p| : |q-p| \leq r_{\text{cover}}, (q-p) \cdot d \geq 0 \}$$

 我们报告 ``max over d of dir_cov(p, d)``（最坏朝向）与"无任何可听停靠点"的占比。
 返回 `` (最坏朝向距离，无覆盖占比，半平面不可覆盖占比) ``：
 第三项 = 连"忽略距离上限"的半平面里都没有停靠点的比例（这是几何硬伤，
 加多少圈都救不回来，只能把停靠点铺到靶区边界之外）。
 """
 if gx is None:
 gx, gy = eval_grid()
 P = np.asarray(pts, float).reshape(-1, 2)
 ang = np.arange(int(n_dir)) * (2.0 * math.pi / int(n_dir))

```

```

worst = np.zeros(gx.size)
unc = np.zeros(gx.size, bool)
hard = np.zeros(gx.size, bool)
r2 = float(r_cover) ** 2
for i0 in range(0, gx.size, int(chunk)):
 sl = slice(i0, min(i0 + int(chunk), gx.size))
 px = gx[sl][:, None] - P[None, :, 0]
 py = gy[sl][:, None] - P[None, :, 1]
 d2 = px ** 2 + py ** 2
 d = np.sqrt(d2)
 # 每个停靠点方向（用于半平面判定）
 for a in ang:
 cos, sin = math.cos(a), math.sin(a)
 par = px * cos + py * sin # (q-p)·d
 ok = (par >= -1e-9) & (d <= float(r_cover) + 1e-9)
 any_ok = ok.any(axis=1)
 best = np.where(any_ok, np.min(np.where(ok, d, np.inf), axis=1),
 np.inf)
 worst[sl] = np.maximum(worst[sl], np.where(np.isfinite(best), best, 0.0))
 unc[sl] |= (~any_ok)
 any_par = (par >= -1e-9).any(axis=1)
 hard[sl] |= (~any_par)
 return (float(worst.max()), float(unc.mean()), float(hard.mean()))

def hex_cover_radius(a):
 """无限三角形格网的覆盖半径 = $a/\sqrt{3}$ （外接圆半径）。"""
 return float(a) / SQRT3

def ring_set(rings):
 """`[(r, n), ...]` -> 停靠点列表（环间半格错开，避免放射状缝隙）。"""
 out = []
 for k, (r, n) in enumerate(rings):
 off = (math.pi / float(n)) if (k % 2 == 1) else 0.0
 for i in range(int(n)):
 ang = off + 2.0 * math.pi * i / float(n)
 out.append((float(r) * math.cos(ang), float(r) * math.sin(ang)))
 return out

def tour_len(pts, start=(0.0, 0.0), two_opt=True):
 """最近邻 + 2-opt 的航路长度（用于比较"走完这些停靠点要多少米"）。"""
 P = np.asarray(pts, float).reshape(-1, 2)
 if P.size == 0:
 return 0.0, []
 cur = np.asarray(start, float)
 left = list(range(P.shape[0]))
 seq = []
 while left:
 j = min(left, key=lambda i: float(np.hypot(*(P[i] - cur))))
 seq.append(j)
 left.remove(j)
 cur = P[j]

 def total(order):
 d = float(np.hypot(*(P[order[0]] - np.asarray(start, float))))
 for x, y in zip(order, order[1:]):
 d += float(np.hypot(*(P[y] - P[x])))
 return d

 best = total(seq)

```

```

 if two_opt:
 improved = True
 while improved:
 improved = False
 for i in range(len(seq) - 1):
 for j in range(i + 1, len(seq)):
 cand = seq[:i] + seq[i:j + 1][::-1] + seq[j + 1:]
 d = total(cand)
 if d < best - 1e-9:
 seq, best, improved = cand, d, True
 return best, seq

```

## 7.9 Q4/src/p4\_robot.py

```

"""Q4 legacy grid/spiral policy, retained for paired comparisons.

Use p4_search.P4SearchRobot for the optimized strategy and strict completion.
The radial half-plane masks and their flipped second pass below are legacy
heuristics. They do not certify arbitrary source orientations, and removing
unreachable cells does not prove absence. Historical comments and benchmark
claims below describe those older experiments, not a continuous guarantee.
"""

from __future__ import annotations

import math
import os
import sys
import time
from dataclasses import dataclass, field as dc_field

import numpy as np

_HERE = os.path.dirname(os.path.abspath(__file__))
if _HERE not in sys.path:
 sys.path.insert(0, _HERE)

from p3_arena import (# noqa: E402
 ArenaError, CHANNELS, R_ARENA, R_CLEAR, R_RECV_MIN, T_CLEAR_FAIL, T_MEASURE,
 T_SWITCH, V_ROBOT,
)
from p3_robot import SWEEP_OK, ChannelRec, P3Config, P3Robot # noqa: E402
from p4_grid import Sqrt3, triangular_lattice # noqa: E402

配置

@dataclass
class P4Config(P3Config):
 """在 `P3Config` 之上追加"三角形格网 + 定向源"相关参数。"""

 # ---- 格网 ----
 grid_a: float = 1000.0 # 三角形格网边长 (m) : a/√3 = 577 m 覆盖半径
 grid_r_max: float = R_ARENA # 格点铺到多大半径 (靶区半径 1800 m)
 outer_rings: tuple = ((1850.0, 12),)
 """外侧补漏环 `(半径 m, 点数)` : 救"贴边朝内发射"的定向源。

 [!] 先说清这个环到底在救什么 (实测修正了我最初的直觉):
 mock 的定向判断是"站→源"的视线落在 `dir_deg` 的 ±90° 内",
 诊断 100 局漏源 (`src/p4_miss_where.py`) 显示漏掉的全是

```

**\*\*半径 1622-1794 m、朝向与径向夹角 117°-178°（几乎正对圆心）的定向源\*\*。**  
也就是说：**\*\*"朝内发射"的源必须从它外侧的站位才听得到\*\*** -- 外环就是干这个的。

靶区边界附近 ( $r \gtrsim 1000$ ) 的源若朝内发射，覆盖半平面里没有任何"域内"站位，  
所以只能把站位铺到它外侧。数值枚举（`src/p4\_ring\_shape.py`）：

```
外环	站数	覆盖半径	定向盲区
r1850×12（默认）	25	575 m	0.00%
r1900×10	24	615 m	0.66%
r1900×6（≈"格网整体铺到 r=2000"）	19	575 m	1.91%
r1850×9	22	639 m	2.59%
无外环（只铺域内）	13	800 m	38.66%
```

允许降覆盖时可换 ``((1900.0, 6),)``: 实测 100 局清除率 1.0000（全向/定向混合  
与全定向两批都满分）、指标基本不变，站数少 6 个；亦可换成"格网直接铺到 2000"  
（``outer\_rings=(), grid\_r\_max=2000``），两者等价。  
"""

```
---- 探索 ----
spiral_radius_w: float = 0.35 # 选下一站时对"半径大"的加罚，形成由内向外
skeleton_radius_w: float = 0.25 # 骨架排序的半径偏置（由内向外；太大会横穿靶区）
radius_sort_w: float = 0.0 # 试过"显式由内向外排序"，实测无正向收益（见 scan_skeleton），默认关闭
route_mode: str = "spiral" # 航路排法: "spiral"（问题 3 式绕圈，默认）| "tsp"（几何最短）
ring_band_frac: float = 0.5 # 螺旋航路的"环"分档宽度 = grid_a × 该比例（m）。
0.5 → r=1732 与 1850 合成一层（实测最好，航程 26.3 km）；
0.35/0.25/0.18 → 分成 3 层，反而变差（28.0 km、长跳 +1）

replan_drift_m: float = 700.0 # 保留字段（当前实现不再按当前位置重排航线）
station_min_gain: float = 90.0 # 某站的边际覆盖低于它就跳过（单位: 25 m 网格点数；
90 格 ≈ 一个 140 m 见方的小片）。
100 局实测（混合场景，中位平均定位清除 / 清除率）：
gain=90 : 472 s / 1.0000（100 局全清）← 默认
gain=180: 与 90 接近，但全定向会掉源
gain=250: 569 s / 0.9977（慢 97 s 且漏 3 个源）
结论: **别为了省站抬高这个门限** -- 省下的站次
远抵不上"漏源 + 听到更晚"的代价。

detour_follow_m: float = 1400.0 # 顺路清除后离队首超过它就"就近切入"航线
（正常站间距 850-1732 m，取 1400 只在真正被带偏时触发）

cover_reduce: float = 0.0 # **降覆盖档**：>0 时主动放弃这么大比例的可听格点，
只保留"代价最小、覆盖面最大"的若干站位（见 measure_set）：
0.0（默认）= 全覆盖；0.3 / 0.5 / 0.7 = 逐档变快、逐档漏源
实测见 `src/p4_cover_reduce.py` 与 review §3.12

plan_tour: bool = False # False（默认）: **逐站贪心，把覆盖判据说"还有用"的站
全部走完** -- 这是模型下"必被发现"的完整保证
（实测 20 局 268 个源 100% 清除，代价是 27 站/局）；
True: 先算"最小覆盖集 + 半径偏置 TSP"，只走 14-16 站，
航程少 ~4 km、虚拟时间少 ~10%，代价是有 ~0.4% 的
漏源风险（边界处"朝外但偏离径向"的定向源）。

replan_every: int = 0 # >0: 每走这么多站就用"当前位置 + 最新覆盖集"重排航线
cover_slack_m: float = 0.0 # 半平面判据的保守余量（m）
halfplane_eps: float = 1.0 # 半平面判据的绝对余量（m2）：避免"正侧向"的边界抖动
cover_grid_step_m: float = 25.0 # 覆盖评估网格步长（m）：太粗会漏判边界附近的源
（实测 100 m 会真的漏源；50 m 仍漏 1/268；
25 m 只剩"格点边界"量级的漏判风险）

two_pass: bool = True # 扫两遍（两种半平面约定）⇒ 对协议约定不敏感
max_station_visits: int = 0 # 0 = 不限制（按覆盖判据自然收工）

---- 定位（定向源闭环：沿视线二分 + 侧移交会）----
loc_walk_m: float = 520.0 # 沿视线走位的单步初值（m）；走位用 /clear（3 s）
loc_step_growth: float = 1.4 # 每确认一次"有信号"就把步长乘这个系数（指数跳出界）
loc_max_moves: int = 4 # 走位次数上限（走出无信号边界就停）
loc_bisect_steps: int = 5 # 二分步数上限
```

```

loc_bisect_tol_m: float = 40.0 # 二分区间的收敛门限 (m)
loc_cross_radius_m: float = 420.0 # 侧移交会的离源距离 (m): 交会角 $\approx 45^\circ$
loc_clear_radius_m: float = 90.0 # 第一轮试探半径 (m): 由定位精度决定
loc_probe_radius_max_m: float = 160.0 # 区域半径超过它就先做定位而不是硬试
max_solve_attempts: int = 3 # 同一频道最多进入 solve 几次 (防止清除阶段空转)
clear_detour_max_m: float = 300.0 # **阶段式** (`unified=False`) 下"到站顺手清"的绕行上限。
[!] 统一航路**不用**这个值 (用 interleave_detour_max_m,
默认 ∞)。曾经误把 300 m 套到统一航路上, 结果把大多数
清理延后、积压到航程末尾: 平均定位清除 648 $\rightarrow$ 1141 s、
行程 34.9 $\rightarrow$ 64.1 km。两套策略的参数不能混用。
每个停点最多顺路清几个
enroute_max_n: int = 3
---- 统一航路 (边扫描边清理, 见 run_unified) ----
unified: bool = True # True: 一条滚动航路走完两圈, 点也清完 (默认)
interleave_detour_max_m: float = 1e9
实测最优是不设门限 (听到就清): 扫参见
`pick_unified_action` 的文档与 `p4_tune_unified.py`。
保留这个参数是为了复现"摸着清"的对照。
interleave_loc_radius_m: float = 110.0
目标定到"区域半径 $\leq$ 它"就算定住了, 可以清
interleave_probe_m: float = 80.0 # 统一航路里的小范围试探半径 (m)
interleave_clear_cost_s: float = 90.0
估算一次"就地清理"的动作耗时 (定位 + 试探)

---- 清除 (复用问题 3 的覆盖式试探, 保证成功) ----
clear_approach_max: int = 3
clear_est_probe_m: float = 40.0 # 估计点已在这么近时先直接试一次 /clear

机器狗

class P4GridRobot(P3Robot):
 """三角形格网扫描 + 定向源闭环定位清除。"""

 def __init__(self, arena, cfg: P4Config | None = None, base=None, log=None,
 log_prefix=""):
 super().__init__(arena, cfg or P4Config(), base=None, log=log,
 log_prefix=log_prefix)
 self.stations = self._build_stations()
 self.station_hits = [0] * len(self.stations)
 self.visited_stations: set[int] = set()
 self.loc_pts: dict[int, list] = {} # 定位过程中新增的检测点 (画图/复盘用)
 self._near_pt: dict[int, tuple] = {} # "距离 ≤ 5 m" 的检测点: 到那儿直接清
 self._tried_ray: set[int] = set() # 已经做过"沿视线落到二分点"的频道
 self.coverage_flip = False # 当前扫描使用的半平面约定 (见 coverable_mask)
 self.unified_second_pass = False # 统一航路: 是否处于"第二遍 (必须清)"
 self.first_pass_heard: set | None = None # 第一遍结束时就已听到的频道
 self.phase = "scan"
 self._cov_cache: dict = {}
 self._covered_cache: dict = {}

 # ----- 格网 -----
 def _build_stations(self):
 cfg = self.cfg
 pts = list(triangular_lattice(cfg.grid_a, r_cover=cfg.grid_r_max))
 for (r, n) in (cfg.outer_rings or ()):
 for i in range(int(n)):
 a = 2.0 * math.pi * i / float(n)
 pts.append((float(r) * math.cos(a), float(r) * math.sin(a)))
 # 去掉重复点 (外环可能与格点重合) 与原点 (原点单独作为第 0 站)
 out, seen = [], set()
 for (x, y) in pts:

```

```

 if math.hypot(x, y) < 1.0:
 continue
 key = (round(x, 1), round(y, 1))
 if key in seen:
 continue
 seen.add(key)
 out.append((float(x), float(y)))
 out.sort(key=lambda p: (math.hypot(*p), math.atan2(p[1], p[0]) % (2 * math.pi)))
 return out

def station_xy(self, i):
 return self.stations[int(i)]

----- 选哪些站、按什么顺序走 -----
def covering_set(self):
 """**最小覆盖站集**：贪心集合覆盖，保证"每个可听格点至少被一个站覆盖"。

 为什么需要它：覆盖判据 (:meth:`coverable_mask`) 是"站点 × 格点"的集合覆盖
 问题，30 个站里往往只有十几个真正贡献新覆盖。把"必须去的站"算出来，
 航线才不会为了几个格点多跑几公里。

 贪心在这里是**安全**的：它只决定"访问顺序"，不改变覆盖判据 --
 每站到了之后仍然按 :meth:`station_gain` 复核，收益为 0 的站会被跳过。
 """
 got = getattr(self, "_covering", None)
 if got is not None:
 return got
 gx = self._grid_pts()[0]
 left = ~self.unreachable_mask()
 masks = [self.coverable_mask(i) & ~self.unreachable_mask()
 for i in range(len(self.stations))]
 chosen = []
 while left.any():
 best, bi, bg = None, -1, 0
 for i, m in enumerate(masks):
 if i in chosen:
 continue
 g = int(np.count_nonzero(m & left))
 if g > bg:
 best, bi, bg = i, i, g
 if bi < 0 or bg <= 0:
 break
 chosen.append(bi)
 left = left & ~masks[bi]
 self._covering = chosen
 return chosen

----- 降覆盖档：把"测哪些站"显式做成一档 -----
def measure_set(self):
 """本局实际做检测的站位集合（``cover_reduce > 0`` 时会**主动缩水**）。

 为什么这是一个独立的省时杠杆：时间 $\approx \Sigma_{\text{站}} (\text{移动} + \text{该站要测的频道数} \times 6 \text{ s})$ 。
 "访问哪些站"由覆盖判据决定（全测一遍才敢说"没有残留源"），
 但如果允许**放弃一部分区域**，就可以只保留一个"代价最小、覆盖大部分"的站集：

 ``cover_reduce = r`` $\Rightarrow$ 用贪心集合覆盖，一直取到"剩余未覆盖格点
 只剩全部格点的 r 倍"为止 -- 于是**故意留 r 比例的面积不测**。

 这是"覆盖 $\rightarrow$ 用时"最直接的旋钮： $r=0$ 就是原来的全覆盖行为。
 """
 r = float(self.cfg.cover_reduce)
 if r <= 0.0:

```

```

 return None # None = 不做缩减 (全覆盖)
got = getattr(self, "_mset", None)
if got is not None:
 return got
gx = self._grid_pts()[0]
unreach = self.unreachable_mask()
left = ~unreach
total = int(left.sum())
masks = [self.coverable_mask(i, flip=False) & ~unreach
 for i in range(len(self.stations))]
chosen: list[int] = []
target = total * float(r)
while True:
 best, bi, bg = None, -1, 0
 for i, m in enumerate(masks):
 if i in chosen:
 continue
 g = int(np.count_nonzero(m & left))
 if g > bg:
 best, bi, bg = i, i, g
 if bi < 0 or bg <= 0:
 break
 nxt = int(np.count_nonzero(left & ~masks[bi]))
 if nxt <= target:
 break # 已经放弃够了: 剩下的区域不测
 chosen.append(bi)
 left = left & ~masks[bi]
self._mset = set(chosen)
self.stats["n_measure_set"] = len(chosen)
self.stats["cover_abandoned"] = float(
 int(np.count_nonzero(left)) / max(total, 1))
self.log(f"降覆盖档 cover_reduce={r:.2f}: 只测 {len(chosen)} 个站位, "
 f"主动放弃 {self.stats['cover_abandoned']:.1%} 的可听格点")
return self._mset

def plan_station_tour(self, start=None):
 """把"必去的站"排成一条航线: 最近邻 + 2-opt, **代价里带半径惩罚**。

 为什么不是纯最短航线: 平均定位清除时间 = 定位清除总时刻 / 清除个数,
 "早点听到源"比"少走路"更值钱 -- 源一旦被听到就会立刻进入清理队列
 (实测: 纯最短航线让某一局的 14 个源晚了 2000 s 才被听到, 指标掉 2 个百分点)。
 所以在距离代价上加 ``spiral_radius_w * 半径``: 航线自然由内向外展开,
 同时 2-opt 仍能消掉最明显的绕远。

 ``start`` 给定时以它为起点重排 (滚动重规划用); 默认从原点出发。
 排序只花几十毫秒, 却能把扫描段从 24 km 压到 16-18 km。
 """
 idx = list(self.covering_set())
 if not idx:
 return []
 w = float(self.cfg.spiral_radius_w)
 P = {i: np.asarray(self.stations[i], float) for i in idx}
 origin = np.asarray(self.pos if start is None else start, float)

 def cost(order):
 """总代价 = 航程 + w * Σ 半径 (由内向外偏置)。"""
 if not order:
 return 0.0
 d = float(np.hypot(*(P[order[0]] - origin)))
 for a_, b_ in zip(order, order[1:]):
 d += float(np.hypot(*(P[b_] - P[a_])))
 d += w * sum(float(np.hypot(*P[i])) for i in order)

```

```

 return d

 cur = origin
 left = set(idx)
 seq = []
 while left:
 j = min(left, key=lambda i: float(np.hypot(*(P[i] - cur)))
 + w * float(np.hypot(*P[i])))
 seq.append(j)
 left.discard(j)
 cur = P[j]
 best = cost(seq)
 improved = True
 while improved:
 improved = False
 for i in range(len(seq) - 1):
 for j in range(i + 1, len(seq)):
 cand = seq[:i] + seq[i:j + 1][::-1] + seq[j + 1:]
 c = cost(cand)
 if c < best - 1e-9:
 seq, best, improved = cand, c, True
 self._tour_len_m = cost(seq) - w * sum(float(np.hypot(*P[i])) for i in seq)
 return seq

def greedy_station_order(self):
 """逐步贪心版航线（``plan_tour=False`` 时的对照）：每步按
 "收益 / 距离 + 半径偏置" 选下一站，直到没有站还能补新覆盖。

 与 :meth:`plan_station_tour` 的差别：贪心每步都看最新覆盖状态（响应性好），
 但没有全局航路结构（实测航程多 5-8 km）；规划版反过来。
 """
 cur = np.asarray(self.pos, float)
 left = set(range(len(self.stations)))
 seq = []
 w = float(self.cfg.spiral_radius_w)
 for _ in range(len(self.stations)):
 cover = self.unheard_mask()
 if not cover.any():
 break
 best, bs = None, None
 for si in left:
 sx, sy = self.stations[si]
 d = math.hypot(sx - cur[0], sy - cur[1])
 if d < self.cfg.min_move:
 continue
 gain = self.station_gain(si, cover)
 if gain <= self.cfg.station_min_gain:
 continue
 s = float(gain) / (1.0 + d / 500.0) - w * math.hypot(sx, sy) / 100.0
 if bs is None or s > bs:
 best, bs = si, s
 if best is None:
 break
 seq.append(best)
 left.discard(best)
 cur = np.asarray(self.stations[best], float)
 self._tour_len_m = 0.0
 return seq

----- 覆盖判据 -----
def _grid_pts(self):
 """覆盖评估网格（50 m）：用来算"这一站还能补多少覆盖"。

```



分辨率不能太粗：覆盖判据是“格点到站点的距离  $\leq$  有效接收半径下界 1000 m”，而三角格网的覆盖半径已经接近这个量级（577 m），用 100 m 粗网格会让边界附近的源被漏判（实测有源因为格点间隙而永远没被听到 -- 这是\*\*真的漏源\*\*，不是启发式误差）。50 m 网格在 30 个站上的额外计算量可以忽略。

"""

```
c = getattr(self, "_cov_grid", None)
if c is None:
 step = float(self.cfg.cover_grid_step_m)
 R = R_ARENA
 xs = np.arange(-R, R + 1e-9, step)
 GX, GY = np.meshgrid(xs, xs)
 rr = np.hypot(GX, GY)
 m = rr <= R + 1e-9
 c = (GX[m].copy(), GY[m].copy())
 self._cov_grid = c
return c
```

```
def coverable_mask(self, si, flip=None):
```

""" 站点 ``si`` 能“听到”的格点掩码（定向源的\*\*保守\*\*判据）。

半平面条件（以“源在  $g$ 、记发射方向为  $d$ ”为准）：站点 ``q`` 能听到  $\iff$   
 $|q-g| \leq 1000$  且  $(q-g) \cdot d \geq 0$ （视线方向落在  $\pm 90^\circ$  覆盖角内）。

本题只给了“定向方向未知、覆盖角  $180^\circ$ ”，\*\*没有\*\*规定  $d$  的指向约定  
 （是“朝  $d$  发射”还是“ $d$  的反方向发射”）。两种约定给出的可听半平面正好相反：

\* ``flip=False``:  $d =$  径向向外  $\Rightarrow$  要求在\*\*外侧\*\* ( $(q-g) \cdot g \geq 0$ ) ;  
 \* ``flip=True``:  $d =$  径向向内  $\Rightarrow$  要求在\*\*内侧\*\* ( $(q-g) \cdot g \leq 0$ ) 。

为了让策略对协议约定\*\*不敏感\*\*，扫描阶段做\*\*两遍\*\*：第一遍按 ``flip=False`` 覆盖，若有频道仍然没被听到，第二遍按 ``flip=True`` 把另一半半平面也扫掉。  
 两遍合起来  $\Rightarrow$  无论真实约定是哪一种（甚至两向都发），源都必被听到  
 （自检 G3/G4 分别对两种约定核验盲区为 0）。

"""

```
if flip is None:
 flip = self.coverage_flip
key = (int(si), bool(flip))
a = self._cov_cache.get(key)
if a is not None:
 return a
gx, gy = self._grid_pts()
qx, qy = self.stations[int(si)]
d2 = (gx - qx) ** 2 + (gy - qy) ** 2
dot = (qx - gx) * gx + (qy - gy) * gy # (q-g)·g: q 在源的外侧
if flip:
 dot = -dot
m = (d2 <= (R_RECV_MIN + self.cfg.cover_slack_m) ** 2) \
 & (dot >= -self.cfg.halfplane_eps)
self._cov_cache[key] = m
return m
```

```
def unreachable_mask(self, flip=None):
```

"""\*\*任何\*\*站点都听不到的格点（在给定半平面约定下）。

这些点不参与“要不要再跑一站”的判据 -- 加再多站也覆盖不到  
 （唯一的办法是换一个半平面约定重扫，见 :meth:`coverable\_mask`）。

"""

```
flip = self.coverage_flip if flip is None else bool(flip)
key = ("unreach", flip)
got = self._cov_cache.get(key)
if got is not None:
```

```

 return got
 m = np.ones(self._grid_pts()[0].size, bool)
 for si in range(len(self.stations)):
 m &= ~self.coverable_mask(si, flip=flip)
 self._cov_cache[key] = m
 return m

def _covered_of(self, ch):
 """频道 ``ch`` 已经被排除的格点掩码（"在那里测过而且能听到"）。

 只在**测过的站位**上算：那一站能听到的格点范围（距离 + 半平面）里若有源，
 当时就会被听到；没听到 $\Rightarrow$ 那里没有该频道的源。
 """
 rec = self.recs[int(ch)]
 key = (int(ch), len(rec.meas_stations))
 cache = self._covered_cache
 got = cache.get(key)
 if got is not None:
 return got
 out = np.zeros(self._grid_pts()[0].size, bool)
 for si in rec.meas_stations:
 out |= self.coverable_mask(si)
 if len(cache) > 80:
 cache.clear()
 cache[key] = out
 return out

def pending_cover(self, ch):
 """频道 ``ch`` 还剩多少"可能有源但我们还没听过"的格点。"""
 return int(np.count_nonzero(~self._covered_of(ch)))

----- 站位的边际覆盖 -----
def unheard_mask(self):
 """还没听到过的频道"的联合未覆盖格点掩码（一次算好，供所有站位复用）。

 已经排除 :meth:`unreachable_mask`：那些点任何站位都听不到（定向源朝外、
 外侧又没有站位），不该继续驱动航线。
 """
 gx, _gy = self._grid_pts()
 m = np.zeros(gx.size, bool)
 for ch in self.unheard():
 m |= ~self._covered_of(ch)
 return m & ~self.unreachable_mask()

def station_gain(self, si, unheard_cover=None):
 """这一站对"还没听到过的频道"能补多少未覆盖格点（越少越不值得去）。"""
 m = self.coverable_mask(si)
 if unheard_cover is None:
 unheard_cover = self.unheard_mask()
 return int(np.count_nonzero(m & unheard_cover))

def unheard(self):
 """还没听到过、也没被清除的频道（探索阶段要覆盖的目标）。"""
 return [int(ch) for ch, rec in self.recs.items()
 if not rec.cleared and rec.n_bearings == 0]

def heard_pending(self):
 """听到了但还没清除的频道（清理阶段的目标）。"""
 return [int(ch) for ch, rec in self.recs.items()
 if not rec.cleared and rec.n_bearings > 0]

def unheard_cover_left(self):

```

```

"""所有未听频道剩余未覆盖格点的总量 (0 $\Rightarrow$ 可以断定靶区内没有别的源了)。"""
return int(np.count_nonzero(self.unheard_mask()))

----- 选下一站 (由内向外螺旋) -----
def pick_station(self):
 """选下一站: **覆盖收益 vs 行驶代价**, 并带一个"由内向外"的螺旋偏置。

 ``score = 新增覆盖格点数 / (1 + 行驶距离/500) - w*半径/100``

 * 分子是"这一站还能补多少我们没听过的地方" (用覆盖判据算, 见
 :meth:`coverable_mask`); 分子为 0 的站直接淘汰 -- 这是"该收工就收工"
 的依据, 也是"不必把 30 个站全跑一遍"的依据;
 * 分母是行驶代价: **实测过: 只用收益选点会得到 43 km 的锯齿航路**
 (站与站之间来回横穿靶区), 加上距离项后回到最近邻量级;
 * 半径偏置让航线自然形成"由内向外"的螺旋 (用户说的"先第一个圈、再第二个圈"),
 既不会在内外圈之间乱跳, 也符合"越早听到源越好"的直觉。
 """
 if not self.unheard():
 return None
 px, py = self.pos
 left = self.unheard_mask()
 if not left.any():
 return None
 best, best_score, best_info = None, None, None
 w = float(self.cfg.spiral_radius_w)
 for si, (sx, sy) in enumerate(self.stations):
 if si in self.visited_stations:
 continue
 d = math.hypot(sx - px, sy - py)
 if d < self.cfg.min_move:
 continue
 gain = self.station_gain(si, left)
 if gain <= self.cfg.station_min_gain:
 continue
 score = (float(gain) / (1.0 + d / 500.0)
 - w * math.hypot(sx, sy) / 100.0)
 if best_score is None or score > best_score:
 best, best_score, best_info = si, score, {"gain": gain, "dist": d}
 return None if best is None else (best, best_info)

def station_channels(self, si, pos=None):
 """在这一站值得测的频道: **还没听到过**的、且这一站能补到新覆盖。

 已经听到过的频道一律不再在这里测 -- 定位由清理阶段的专用测点负责
 (实测教训: 在扫描途中反复测同一个已发现频道, 会把它带离定位几何、
 也把航程和虚拟时间白白放大一倍以上)。

 ``cover_reduce > 0`` (降覆盖档) 时还要再过滤一层: 这一站必须**在
 :meth:`measure_set` 里**才测 -- 不在集合里的站, 走到也不测 (省下 6 s/频道)。
 """
 keep = self.measure_set()
 if keep is not None and int(si) not in keep:
 return []
 m = self.coverable_mask(si) & ~self.unreachable_mask()
 p = np.asarray(pos if pos is not None else self.pos, float)
 out = []
 for ch in self.unheard():
 rec = self.recs[int(ch)]
 if rec.meas_pts:
 d = min(math.hypot(p[0] - q[0], p[1] - q[1]) for q in rec.meas_pts)
 if d < self.cfg.repeat_gap_m:
 continue

```

```

 if not np.any(m & ~self._covered_of(ch)):
 continue
 out.append(int(ch))
 return out

----- 探站 -----
def visit_station(self, si):
 """到站位 ``si``: 先清除"就在跟前"的目标, 再测该测的频道。"""
 x, y = self.stations[int(si)]
 self.scan_points.append((float(x), float(y)))
 self.visited_stations.add(int(si))
 self.note_meas((x, y))
 chans = self.station_channels(si, pos=(x, y))
 travel = math.hypot(x - self.pos[0], y - self.pos[1]) / V_ROBOT
 avail = self.remaining() - self.cfg.time_reserve_s - travel - 4.0
 kmax = max(1, int(avail // (T_MEASURE + T_SWITCH)))
 if len(chans) > kmax:
 self.log(f" 剩余时间只够测 {kmax}/{len(chans)} 个频道")
 chans = chans[:kmax]
 if not chans:
 self.log(f"站位 {si} ({x:.0f},{y:.0f}): 没有值得测的频道, 跳过")
 return 0
 self.log(f"站位 {si}/{len(self.stations)} ({x:.0f},{y:.0f})"
 f" (r={math.hypot(x, y):.0f} m, 移动 {travel * V_ROBOT:.0f} m)"
 f" | 待测 {len(chans)} 个 {chans}")
 n = self.scan_at(x, y, chans, station=int(si))
 self.note_meas((float(self.arena.pos[0]), float(self.arena.pos[1])))
 return n

def scan_at(self, x, y, chans, station=None):
 """在 (x, y) 依次检测这些频道 (第一次调用会带来移动耗时)。

 与问题 3 的差别只有一个: 把**站位编号**记进台账, 覆盖记账才成立
 (``_covered_of`` 只认"在能听到的站位上测过"这个事实)。"""
 n = 0
 for ch in chans:
 if self.aborted or not self.can_afford(T_MEASURE + T_SWITCH + 3.0):
 break
 resp = self.measure(x, y, ch, station=station)
 n += 1
 if resp.get("measure_result") == "near":
 self.do_clear(x, y, ch)
 return n

def note_meas(self, p):
 """记录一次"真实做过的检测"位置 (覆盖度评估口径, 与问题 3 一致)。"""
 p = (float(p[0]), float(p[1]))
 if not self.meas_pts or math.dist(self.meas_pts[-1], p) > 1.0:
 self.meas_pts.append(p)

def measure(self, x, y, ch, station=None):
 """检测一次; 同时登记"这一站测过这个频道"以便覆盖记账。"""
 resp = super().measure(x, y, ch)
 if resp.get("accepted") is True:
 rec = self.recs[int(ch)]
 if station is not None and int(station) not in rec.meas_stations:
 rec.meas_stations.append(int(station))
 if resp.get("measure_result") == "near":
 self._near_pt[int(ch)] = (float(x), float(y))
 return resp

```

```

----- 清理阶段：定向源闭环定位 -----
def estimate(self, ch):
 """频道的当前位置估计：优先用有界交会区域的中心，否则用当前方位外推。

 返回 ``(est, radius, br)``。只有一条方位时半径记为 ``inf``：视线外推点
 在视线方向上完全不可信（误差就是整条视线的长度），不能当“区域半径 400 m”
 用 -- 那会让清除阶段去试探一个根本不对的圆。
 """
 rec = self.recs[int(ch)]
 br = self.bounded_region(rec)
 if br is not None:
 return np.asarray(br["center"], float), float(br["radius"]), br
 g = self.cross_estimate(rec)
 if g is not None:
 return g, float(self.cfg.loc_clear_radius_m), None
 est = self.ray_estimate(rec)
 return (est, float("inf"), None)

def ray_estimate(self, rec):
 """只有一条方位时的粗略位置：视线与“半径 R_RECV_MAX 圆”的交点。"""
 if not rec.pts:
 return None
 (sx, sy), a = rec.pts[-1], rec.svds[-1]
 u = np.array([math.cos(math.radians(a)), math.sin(math.radians(a))])
 S = np.array([float(sx), float(sy)])
 b = float(S @ u)
 c = float(S @ S) - R_ARENA * R_ARENA
 disc = b * b - c
 t = (-b + math.sqrt(disc)) if disc > 0 else 1000.0
 t = min(max(t, 100.0), 1500.0)
 return S + t * u

def cross_estimate(self, rec):
 """**最小二乘交会点**：把所有方位线两两求交再取中位数，稳健于个别粗差。

 只有一条方位时返回 ``None``（一条视线定不了点）。视线近似平行时返回 ``None``
 -- 那时交会区域无界，必须换检测点（问题 1 的结论）。
 """
 if rec.n_bearings < 2:
 return None
 pts = []
 n = rec.n_bearings
 for i in range(n):
 for j in range(i + 1, n):
 (x1, y1), a1 = rec.pts[i], rec.svds[i]
 (x2, y2), a2 = rec.pts[j], rec.svds[j]
 if math.hypot(x1 - x2, y1 - y2) < 50.0:
 continue # 基线太短，交会无意义
 u1 = np.array([math.cos(math.radians(a1)), math.sin(math.radians(a1))])
 u2 = np.array([math.cos(math.radians(a2)), math.sin(math.radians(a2))])
 den = float(u1[0] * u2[1] - u1[1] * u2[0])
 if abs(den) < 0.05:
 continue # 交会角 < 3°：病态
 d = np.array([x2 - x1, y2 - y1], float)
 t1 = float(d[0] * u2[1] - d[1] * u2[0]) / den
 if t1 <= 0.0:
 continue
 pts.append(np.array([x1, y1], float) + t1 * u1)
 if not pts:
 return None
 return np.median(np.asarray(pts, float), axis=0)

```

```

def locate_on_ray(self, ch):
 """沿已测方位做**两侧二分查找**，直接量出"源在这条视线上有多远"。

 物理依据（关键，且与朝向无关）：

 * 方位是从检测点 ``S`` 指向源 ``G`` 的**射线**，源一定在这条射线上；
 * ``S`` 自己**一定在源的有效覆盖范围内**（那条方位就是在这里测到的）
 ⇒ 视线上 $t=0$ 处必有信号；
 * 沿射线往外走，信号迟早消失：要么是距离超过接收半径，要么是走出了
 $\pm 90^\circ$ 覆盖角 -- 两种情况都意味着"源在这条视线上的位置就在消失点附近"，
 而且**消失点是单调的**（定向源的覆盖范围是一个凸集，射线与凸集的交
 是一段区间）；
 * 于是"有信号/无信号"的分界点就能把源夹住，二分几步即可收到 40 m 以内。

 走位一律用 ``/clear``：未命中只要 3 s（``/measure`` 要 6 s）、不改频道、
 不污染方位台账，而且万一进到 20 m 内会直接把源清掉。

 返回 ``(est, radius, br)``：``radius = half_width`` 是二分区间的一半。
 """
 rec = self.recs[int(ch)]
 if not rec.pts:
 return None, float("inf"), None
 # 已经有界且够小：根本不必再走，直接交给覆盖式试探
 br0 = self.bounded_region(rec)
 if br0 is not None and float(br0["radius"]) * 2.0 <= R_CLEAR:
 return np.asarray(br0["center"], float), float(br0["radius"]), br0
 (sx, sy), a = rec.pts[-1], rec.svds[-1]
 u = np.array([math.cos(math.radians(a)), math.sin(math.radians(a))])
 S = np.array([float(sx), float(sy)])

 def at(t):
 return S + u * float(t)

 def far_t():
 """射线出靶区的距离（再远就没有意义了）。"""
 b = float(S @ u)
 c = float(S @ S) - (R_ARENA - 5.0) ** 2
 disc = b * b - c
 return (math.sqrt(disc) - b) if disc > 0 else 600.0

 def go_and_clear(p):
 """纯走位：用 /clear 移动过去，命中就返回 True。"""
 if not self.can_afford(math.hypot(*(p - np.asarray(self.pos, float)))
 / V_ROBOT + 5.0):
 return False, None
 if self.do_clear(float(p[0]), float(p[1]), int(ch)):
 return True, "success"
 if self.aborted:
 return False, None
 self.note_meas((float(p[0]), float(p[1])))
 return False, "moved"

 # ---- (1) 向外**指数跳步**，尽快建立上界 ----
 # t=0 就是当初测到方位的地方，那里必有信号（省掉一次 6 s 的测量）；
 # 向外用 1.4 倍递增的步长跳（520 → 730 → 1020 ...），边界通常在 2-3 步内被夹住。
 # 为什么要跳大步：成本的大头是**走路**（5 m/s），每多一次"走 500 m 再测一次"
 # 就是 100 s + 6 s；指数跳步把 5 次试探压到 2 次。
 t_lo, t_hi = 0.0, None
 t_cur = max(self.cfg.loc_walk_m,
 min(float(np.asarray(self.pos, float) - S) @ u) + self.cfg.loc_walk_m,
 far_t()))
 step = self.cfg.loc_walk_m

```

```

for _ in range(self.cfg.loc_max_moves):
 hit, _st = go_and_clear(at(t_cur))
 if hit:
 return at(t_cur), 0.0, None
 if self.aborted:
 break
 r = self.measure(float(at(t_cur)[0]), float(at(t_cur)[1]), int(ch))
 self.note_meas((float(at(t_cur)[0]), float(at(t_cur)[1])))
 res = r.get("measure_result")
 self.stats["n_locate"] = self.stats.get("n_locate", 0) + 1
 self.log(f" 定位频道{ch}: 沿视线 t={t_cur:.0f} m -> {res}")
 if res == "near":
 self._near_pt[int(ch)] = (float(at(t_cur)[0]), float(at(t_cur)[1]))
 return at(t_cur), 0.0, None
 if res == "direction":
 t_lo = t_cur
 step *= self.cfg.loc_step_growth
 nxt = t_cur + step
 if nxt > far_t():
 nxt = far_t()
 if nxt <= t_cur + 1.0:
 break
 t_cur = nxt
 else:
 t_hi = t_cur
 break

---- (2) 二分夹逼 ----
n_bi = 0
while (t_hi is not None and n_bi < self.cfg.loc_bisect_steps
 and (t_hi - t_lo) > self.cfg.loc_bisect_tol_m):
 tm = 0.5 * (t_lo + t_hi)
 p = at(tm)
 if not self.can_afford(math.hypot(*(p - np.asarray(self.pos, float)))
 / V_ROBOT + T_MEASURE + T_SWITCH + 4.0):
 break
 r = self.measure(float(p[0]), float(p[1]), int(ch))
 self.note_meas((float(p[0]), float(p[1])))
 n_bi += 1
 self.stats["n_locate"] = self.stats.get("n_locate", 0) + 1
 res = r.get("measure_result")
 self.log(f" 定位频道{ch}: 二分 t={tm:.0f} m -> {res}")
 if res == "near":
 self._near_pt[int(ch)] = (float(p[0]), float(p[1]))
 return p, 0.0, None
 if res == "direction":
 t_lo = tm
 else:
 t_hi = tm
if t_hi is None:
 t_est, half = t_lo, float(self.cfg.loc_walk_m)
else:
 t_est, half = 0.5 * (t_lo + t_hi), 0.5 * (t_hi - t_lo)
return at(t_est), half, None

定位二: 拿第二条方位做交会 (把区域压到 20 m 量级)

def admissible_near(self, q, est, r_est=None, margin=40.0, flip=None):
 """估计站位 ``q`` 能否听到"位于 ``est``、朝向径向"的定向源。

 判据与 :meth:`coverable_mask` 一致 (距离 + 半平面), 只是作用在单点上,

```

```

用于排候选 ("这个侧移点还在源的覆盖角里吗")。 ``flip`` 见 :meth:`coverable_mask`。
"""
if flip is None:
 flip = self.coverage_flip
q = np.asarray(q, float)
e = np.asarray(est, float)
v = q - e # 源 → 站位
d = float(np.hypot(*v))
lim = (R_RECV_MIN if r_est is None else float(r_est)) - float(margin)
if d > lim:
 return False
dot = float(v[0] * e[0] + v[1] * e[1])
return dot <= 0.0 if flip else dot >= 0.0

def locate_by_cross(self, ch):
 """在"旁侧"再取一条方位: 与已有视线交会, 把位置压到几十米量级。

 侧移点选在**与最后一条视线垂直**的方向上、离估计位置
 ``loc_cross_radius_m`` 处 -- 交会角 $\approx 45^\circ$ 、基线 ≈ 400 m
 ⇒ 区域半径 $\approx R \cdot \tan 1^\circ / \sin 45^\circ \approx 10$ m, 覆盖式试探一次就中。

 侧移点还要满足"估计位置在自己朝原点一侧" (:meth:`admissible_near`):
 这是定向源"朝外发射"时唯一还听得到的半平面。返回是否确实拿到了方位。
 """
 rec = self.recs[int(ch)]
 if not rec.pts:
 return False
 (sx, sy), a = rec.pts[-1], rec.svds[-1]
 u = np.array([math.cos(math.radians(a)), math.sin(math.radians(a))])
 S = np.array([float(sx), float(sy)])
 t_p = float(np.asarray(self.pos, float) - S) @ u
 t_hat = min(max(t_p, 220.0), 1900.0)
 G = S + u * t_hat
 perp = np.array([-u[1], u[0]])
 best = None
 for sgn in (+1.0, -1.0):
 for frac in (1.0, 0.6, 0.35): # 先试全侧移, 再退而求其次
 q = G + perp * sgn * self.cfg.loc_cross_radius_m * frac
 if float(np.hypot(*q)) > R_ARENA - 1.0:
 continue
 if not self.admissible_near(q, G):
 continue
 key = float(np.hypot(*(q - np.asarray(self.pos, float))))
 if best is None or key < best[0]:
 best = (key, q, frac)
 break
 if best is None:
 # 覆盖角筛不出候选: 直接取"离原点更近"的那一侧 (一定在朝外的覆盖半平面里)
 cand = [G + perp * sgn * self.cfg.loc_cross_radius_m * 0.5
 for sgn in (+1.0, -1.0)]
 cand = [c for c in cand if float(np.hypot(*c)) <= R_ARENA - 1.0]
 if not cand:
 return False
 q = min(cand, key=lambda c: float(np.hypot(*c)))
 best = (float(np.hypot(*(q - np.asarray(self.pos, float)))), q, 0.5)
 _d, q, frac = best
 trav = float(np.hypot(*(q - np.asarray(self.pos, float))))
 if not self.can_afford(trav / V_ROBOT + T_MEASURE + T_SWITCH + 6.0):
 return False
 self.log(f" 定位频道{ch}: 侧移到 ({q[0]:.0f},{q[1]:.0f})"
 f" (移动 {trav:.0f} m, 离估计位置 {self.cfg.loc_cross_radius_m * frac:.0f} m, "
 f"交会角 $\approx 45^\circ$ ")

```



```

r = self.measure(float(q[0]), float(q[1]), int(ch))
self.note_meas((float(q[0]), float(q[1])))
self.stats["n_locate"] = self.stats.get("n_locate", 0) + 1
if r.get("measure_result") == "near":
 self._near_pt[int(ch)] = (float(q[0]), float(q[1]))
 return True
return r.get("measure_result") == "direction"

def localize(self, ch):
 """闭环定位: **先侧移交会(准、贵) → 交会不成再沿视线二分(便宜、糙)**。

 为什么这个顺序: 误差的**横向分量**才是清除的瓶颈(沿视线的距离误差只会让
 试探点沿视线前后偏一点, 而横向偏 200 m 就完全试不到)。侧移交会同时压掉
 横向与纵向误差(基线 400 m、交会角 45° ⇒ 区域半径 ≈ 10 m),
 所以它是"值得付 180 s"的一步; 只有连交会都拿不到方位时,
 才退回"沿视线二分"(对贴边朝外的源, 它的横向误差 ≈ $d \cdot \tan 1^\circ$)。

 返回 ``(est, radius, br)``。
 """
 rec = self.recs[int(ch)]
 br = self.bounded_region(rec)
 if br is not None and float(br["radius"]) <= self.cfg.loc_probe_radius_max_m:
 return np.asarray(br["center"], float), float(br["radius"]), br
 # 1) 侧移交会(最多两轮: 第二轮是在"位置估计更准"之后再做一次)
 for _ in range(2):
 if not self.locate_by_cross(ch):
 break
 br = self.bounded_region(rec)
 if br is not None and float(br["radius"]) <= self.cfg.loc_probe_radius_max_m:
 return np.asarray(br["center"], float), float(br["radius"]), br
 if not self.can_afford(self.cfg.min_action_budget_s + 40.0):
 break
 g = self.cross_estimate(rec)
 if g is not None:
 return g, float(self.cfg.loc_clear_radius_m), None
 # 2) 交会不成: 沿视线二分距离
 est, rad, _br = self.locate_on_ray(ch)
 if est is None:
 return None, float("inf"), None
 return est, float(rad), None

def sweep_ok(self, ch, rec, c, r, poly=None):
 """覆盖式试探的**布尔**结论。

 ``P3Robot.sweep_clear`` 现在返回三态字符串
 (``SWEEP_OK`` / ``SWEEP_MISS`` / ``SWEEP_BUDGET``, 见 ``p3_robot`` 的
 "失败归因三分"); 三者在布尔上下文里**都非空**, 直接拿来做 ``if`` 判定会把
 "预算不足、一个 /clear 都没发"误判成成功。本模块只需要"清掉了没有",
 因此统一折算回布尔。
 """
 return self.sweep_clear(ch, rec, c, r, poly=poly) == SWEEP_OK

def solve_channel(self, ch):
 """把一个已发现频道定位并清除(定向源版本)。"""
 rec = self.recs[int(ch)]
 if rec.cleared:
 return True
 # (0a) 定位过程中出现过 near(距离 ≤ 5 m): 回那个点直接清, 别再做任何试探
 near_p = self._near_pt.get(int(ch))
 if near_p is not None:
 d = float(np.hypot(*(np.asarray(self.pos, float) - np.asarray(near_p, float))))
 if self.can_afford(d / V_ROBOT + 6.0):

```

```

 self.log(f" 频道{ch} 曾在 ({near_p[0]:.0f},{near_p[1]:.0f}) 测到 near"
 f" (距离 ≤ 5 m) -> 回到该点直接清除")
 if self.do_clear(float(near_p[0]), float(near_p[1]), int(ch)):
 return True
 self._near_pt.pop(int(ch), None)
 est, rad, _br = self.estimate(ch)
 # (0b) 估计点就在跟前: 直接试一次 /clear (3 s 的事)
 if est is not None and float(np.hypot(*(np.asarray(self.pos, float) - est))) \
 <= self.cfg.clear_est_probe_m:
 if self.do_clear(float(self.pos[0]), float(self.pos[1]), int(ch)):
 return True
 # (1) 定位: 区域够小就直接用; 否则沿视线二分把"距离"量准
 br = self.bounded_region(rec)
 if not (br is not None
 and float(br["radius"]) <= self.cfg.loc_probe_radius_max_m):
 est, rad, br = self.localize(ch)
 if est is None:
 rec.clear_fail += 1
 self.log(f" [x] 频道{ch} 无法定位 (方位数据不足)")
 return False
 # (2) 逼近定位区域中心 (顺带补一条方位, 常常直接 near)
 for _ in range(self.cfg.clear_approach_max):
 br = self.bounded_region(rec)
 if br is None:
 break
 c = np.asarray(br["center"], float)
 rr = float(br["radius"])
 if rr > self.cfg.loc_probe_radius_max_m:
 break
 d = math.hypot(*(np.asarray(self.pos, float) - c))
 if d <= max(rr, self.cfg.arrive_radius_m):
 break
 if not self.can_afford(d / V_ROBOT + T_MEASURE + T_SWITCH + 6.0):
 break
 resp = self.measure(float(c[0]), float(c[1]), int(ch))
 self.note_meas((float(c[0]), float(c[1])))
 if resp.get("measure_result") == "near":
 return self.do_clear(float(c[0]), float(c[1]), int(ch))
 if resp.get("accepted") is not True:
 return False
 # (3) 第一轮试探: 有界区域 (且不大) 就按**交会多边形**试 -- 交会区域常是细长
 # 四边形, 按多边形只在"到区域 ≤ 20 m"的格点上试, 比外接圆省一半以上的点,
 # 而且"区域必含真源"是问题 1 的结论, 所以这一轮命中率最高。
 self.phase = "clear"
 br = self.bounded_region(rec)
 if br is not None and float(br["radius"]) <= self.cfg.loc_probe_radius_max_m:
 if self.sweep_ok(int(ch), rec, np.asarray(br["center"], float),
 float(br["radius"]), poly=br.get("poly")):
 return True
 # (4) 第二轮: 以**估计点**为中心的小圆 (半径由定位精度决定)
 # 定向源的清除与朝向无关 ⇒ 只要"估计点到真源 ≤ 20 m"就够了;
 # 沿视线二分的距离精度约 ±20 m, 横向误差约 0.0175·d ⇒ 半径 90 m 足够。
 if est is not None:
 if self.sweep_ok(int(ch), rec, np.asarray(est, float),
 self.cfg.loc_clear_radius_m, poly=None):
 return True
 # (5) 第三轮: 估计点也不可信时, 用"最后一条视线"上的二分区间中点再试一次
 if rec.n_bearings >= 1:
 br = self.bounded_region(rec)
 if br is None:
 est2, _r2, _b2 = self.locate_on_ray(ch)
 if est2 is not None and self.sweep_ok(

```

```

 int(ch), rec, np.asarray(est2, float),
 self.cfg.loc_clear_radius_m, poly=None):
 return True
(5) 第四轮: **沿视线二分给出的是"与协议约定无关"的距离信息** --
只要二分区间收得够窄 (≤ 230 m, 说明测点确实落在源的覆盖范围内),
就以该位置为中心做一次小圆试探。这一轮专门对付"几条视线近似平行
(交会角小)、有界区域又细又长"的目标 -- 那类目标的交会区域虽然把源
夹住了, 但细长区域用外接圆试探会大量空跑。
if (rec.n_bearings >= 1 and self.can_afford(self.cfg.min_action_budget_s + 30.0)
 and int(ch) not in self._tried_ray):
 self._tried_ray.add(int(ch))
 est3, rad3, _b3 = self.locate_on_ray(ch)
 if est3 is not None and float(rad3) <= 230.0:
 d3 = float(np.hypot(*(np.asarray(self.pos, float)
 - np.asarray(est3, float))))
 if self.can_afford(d3 / V_ROBOT + T_MEASURE + T_SWITCH + 8.0):
 self.log(f" 沿视线落到 ({est3[0]:.0f},{est3[1]:.0f})"
 f" (区间 $\pm$ {float(rad3):.0f} m) 再取一条方位")
 r3 = self.measure(float(est3[0]), float(est3[1]), int(ch))
 self.note_meas((float(est3[0]), float(est3[1])))
 if r3.get("measure_result") == "near":
 self._near_pt[int(ch)] = (float(est3[0]), float(est3[1]))
 if self.do_clear(float(est3[0]), float(est3[1]), int(ch)):
 return True
 br = self.bounded_region(rec)
 if br is not None \
 and float(br["radius"]) <= self.cfg.loc_probe_radius_max_m:
 if self.sweep_ok(int(ch), rec,
 np.asarray(br["center"], float),
 float(br["radius"]), poly=br.get("poly")):
 return True
 if self.sweep_ok(int(ch), rec, np.asarray(est3, float),
 self.cfg.loc_clear_radius_m, poly=None):
 return True
 self.phase = "locate"
 rec.clear_fail += 1
 self.log(f" [x] 频道{ch} 本轮未清除成功")
 return False

----- 主循环 -----
def sweep_pass(self, flip=False):
 """走一遍扫描航线: 按"必去站位"顺序移动, 每站只测还没听到过的频道。

 返回本遍实际访问的站数。三个提前收工判据:

 * 该半平面约定下"还没有被排除的格点"清零 $\Rightarrow$ 这一遍已经证明没有漏源;
 * 航线走完;
 * 时间不够。

 每站之后还会做两件"顺路"的事: 把估计位置就在附近的目标清除
 (:meth:`_opportunistic_clear`), 以及按需重排航线 (`replan_every`)。
 """
 self.coverage_flip = bool(flip)
 tour = (self.plan_station_tour() if self.cfg.plan_tour
 else self.greedy_station_order())
 self.stats["n_covering_stations"] = len(tour)
 self.stats["tour_len_m"] = float(getattr(self, "_tour_len_m", 0.0))
 self.log(f"扫描航线: {len(tour)} 个必去站位"
 f" ({'覆盖集 + 半径偏置 TSP' if self.cfg.plan_tour else '逐步贪心'}) , "
 f"预计长度 {getattr(self, '_tour_len_m', 0.0):.0f} m")
 visited = 0
 step_i = 0

```

```

while step_i < len(tour):
 if self.aborted or self.remaining() <= self.cfg.time_reserve_s:
 break
 if self.unheard_cover_left() <= 0: # 已经没有"可能有源"的地方了
 self.log(" 所有未听频道的可能位置都已被排除 -> 提前结束扫描")
 break
 if self.cfg.max_station_visits and visited >= self.cfg.max_station_visits:
 break
 si = tour[step_i]
 step_i += 1
 # 复核: 这一站对"还没听到过的频道"还有没有新覆盖 (有的站会被别的站代劳)
 if self.station_gain(si, self.unheard_mask()) <= self.cfg.station_min_gain:
 self.log(f" 站位 {si} 的覆盖职责已被其他站代劳 -> 跳过")
 continue
 x, y = self.stations[int(si)]
 dist = math.hypot(x - self.pos[0], y - self.pos[1])
 if not self.can_afford(dist / V_ROBOT + T_MEASURE + T_SWITCH + 4.0):
 self.log("剩余时间不足以再跑一站 -> 结束扫描")
 break
 self.visit_station(si)
 visited += 1
 # 顺路: 刚听到的目标若绕不了多远就当场清掉 ("早清一个源"直接改善指标)
 self._opportunistic_clear(max_n=self.cfg.enroute_max_n)
 # 滚动重规划 (可选)
 if (self.cfg.replan_every and visited % self.cfg.replan_every == 0
 and self.remaining() > self.cfg.time_reserve_s
 and not self.aborted):
 tour = self.plan_station_tour()
 step_i = 0
 self.stats["n_replans"] = self.stats.get("n_replans", 0) + 1
 self.log(f" 滚动重规划: 剩余必去站位 {len(tour)} 个")
return visited

def run(self):
 t0 = time.time()
 resp = self.arena.enter()
 if resp.get("accepted") is not True:
 raise ArenaError(f"/enter 未被接受: {resp}")
 base = self.cfg
 self.log(f"进入靶区: 预算 {self.remaining():.0f} s ({self.arena.budget_kind()}); "
 f"策略=三角形格网扫描 (a={base.grid_a:.0f} m, "
 f"{len(self.stations)} 个站位 + 原点)")
 try:
 # ----- 阶段 0: 原点 (频道 1 起步, 依次测全部频道) -----
 self.phase = "scan"
 self.scan_points.append(self.pos)
 chans0 = self.channels_to_measure()
 self.log(f"第 0 站 (原点): 待测 {len(chans0)} 个频道 {chans0}")
 self.note_meas(self.pos)
 self.scan_at(self.pos[0], self.pos[1], chans0)
 self.note_meas((float(self.arena.pos[0]), float(self.arena.pos[1])))
 self.log(f" 原点检测完成: 听到 {len(self.heard_pending())} 个频道, "
 f"剩余未覆盖格点 {self.unheard_cover_left()} 个"
 f" (定向覆盖模型: 这些格点还可能藏着没听到过的源)")

 # ----- 阶段 1: 扫描 + 清理 -----
 if self.cfg.unified:
 # **边扫描边清理**: 一条滚动重规划的航路走完两圈, 点也清完 (默认)
 self.run_unified()
 else:
 # 阶段式对照: 先扫两遍, 再统一清理
 self._staged_scan_then_clear()

```

```

finally:
 try:
 self.arena.exit()
 except ArenaError as e:
 self.log(f"/exit 异常: {e}")
self.stats["program_runtime_s"] = time.time() - t0
return self.report()

----- 统一航路: 边扫描边清理 -----
def ring_index(self):
 """把站位按半径分"环": 返回 ``{站号: 环号}`` (环号 = 半径分档序号)。

 分档宽度取 ``grid_a/2`` (默认 500 m): 于是 r=0 / r=1000 / r=1732 / r=1850
 落在 0 / 2 / 3 / 3 档 -- 内环、中环、外环分得开,
 而"1732 与 1850"这种只差 118 m 的两层会被合到同一环(它们本来就该连着走)。
 """
 got = getattr(self, "_ringidx", None)
 if got is not None:
 return got
 band = max(150.0, float(self.cfg.grid_a) * 0.5)
 self._ringidx = {i: int(round(math.hypot(x, y) / band))
 for i, (x, y) in enumerate(self.stations)}
 return self._ringidx

def scan_skeleton(self, start=None):
 """扫描站位的骨架顺序: **全局最近邻 + 2-opt, 代价里带"由内向外"偏置**。

 这里修正了旧实现的一个真实缺陷(`src/p4_route_audit.py` 量化):
 旧版按"半径分档 + 档内极角"排成**固定**顺序 -- 进环永远从固定角度切进去,
 于是"切换环"时经常要横穿靶区。实测骨架直线航程 18.7 km,
 但跳站之后实际扫描段走了 **26.2 km** (多出来的就是跨环长跳)。

 规则(对应用户的要求: **切换环时就近进入**):

 1. **全局最近邻** (不分环): 下一个站取"离我最近的那个" --
 "进环点"自然就是最近的站, 而不是某个固定角度;
 2. 代价里加 ``w * 半径`` 的小偏置: 让航线**整体**由内向外推进
 (内环先扫 $\Rightarrow$ 中心附近的源先被听到, 对指标有利),
 但偏置足够小, 不会为了"先内后外"去横穿靶区;
 3. **2-opt** 消掉自交与绕远。

 [!] ``start`` 只用于"排一条**基准航线**", 不要拿"顺路清除之后的当前位置"来排
 -- 从偏离航线很远的点做最近邻会得到一条"以该点为中心的星形路线",
 把原来的环形结构彻底打散(实测这是"跨过中间节点"长跳的真正来源)。
 真正的处理是"绕行清掉目标后沿基准航线继续", 见 :meth:`build_queue`。
 """
 key = None if start is None else (round(float(start[0]), 1),
 round(float(start[1]), 1))
 cache = getattr(self, "_skeleton_cache", None)
 if cache is None:
 cache = self._skeleton_cache = {}
 if key is not None and key in cache:
 return cache[key]
 if key is None and getattr(self, "_skeleton", None) is not None:
 return self._skeleton
 w = float(self.cfg.skeleton_radius_w)
 si_all = list(range(len(self.stations)))
 P = {si: np.asarray(self.stations[si], float) for si in si_all}
 s0 = np.asarray((0.0, 0.0) if start is None else start, float)

 def cost(order):
 cur, d = s0, 0.0

```

```

 for si in order:
 d += float(np.hypot(*(P[si] - cur))) + w * float(np.hypot(*P[si]))
 cur = P[si]
 return d

def geo_len(order):
 cur, d = s0, 0.0
 for si in order:
 d += float(np.hypot(*(P[si] - cur)))
 cur = P[si]
 return d

def two_opt(order, rounds=40):
 best = cost(order)
 improved, guard = True, 0
 while improved and guard < rounds:
 improved = False
 guard += 1
 for i in range(len(order) - 1):
 for j in range(i + 1, len(order)):
 cand = order[:i] + order[i:j + 1][::-1] + order[j + 1:]
 c = cost(cand)
 if c < best - 1e-9:
 order, best, improved = cand, c, True
 return order

起点构造: 最近邻 / 角向扫描 (内外分环、正反两向)
starts = []
left, cur, seq = set(si_all), s0, []
while left:
 j = min(left, key=lambda si: float(np.hypot(*(P[si] - cur)))
 + w * float(np.hypot(*P[si])))
 seq.append(j)
 left.discard(j)
 cur = P[j]
starts.append(seq)
c0 = P[seq[0]]
a0 = math.atan2(c0[1] - s0[1], c0[0] - s0[0])
for by_ring in (True, False):
 starts.append(sorted(
 si_all, key=lambda si: ((round(math.hypot(*P[si]) / 900.0)
 if by_ring else 0),
 (math.atan2(P[si][1], P[si][0]) - a0)
 % (2 * math.pi))))
for sgn in (+1, -1):
 starts.append(sorted(si_all,
 key=lambda si: (math.atan2(P[si][1], P[si][0]) - a0)
 * sgn % (2 * math.pi)))
cheapest insertion (圆盘布局上比"最近邻 + 2-opt"更能避免跨环长跳:
它总把新站插到"绕行代价最小"的位置, 而不是从当前位置一路贪心往外走)
order = [si_all[0]]
rest = set(si_all[1:])
while rest:
 bo, bd = None, None
 for si in rest:
 p = P[si]
 for k in range(len(order)):
 a_ = s0 if k == 0 else P[order[k - 1]]
 b_ = P[order[k]]
 delta = (math.hypot(*(p - a_)) + math.hypot(*(b_ - p))
 - math.hypot(*(b_ - a_)))
 if bd is None or delta < bd:

```

```

 bo, bd = (si, k), delta
 order.insert(bo[1], bo[0])
 rest.discard(bo[0])
 starts.append(order)
 best_seq, best_geo = None, float("inf")
 for cand in starts:
 cand = two_opt(list(cand))
 g = geo_len(cand)
 if g < best_geo - 1e-9:
 best_seq, best_geo = cand, g
 seq = best_seq

试过但**否掉**的一个改动：显式"由内向外"排序（按半径 sort 再 2-opt）。
动机是"先扫内环 => 中心附近的源早被听到 => 指标更好"，
但实测（20 局）：gain=180 时从 620 s 变 625 s（更差）；
gain=250 时从 602 s 变 578 s（更快）**但清除率 1.0000 -> 0.9925**（掉 2 个源）。
既然"清除率是硬指标、优化不达标就回滚"，这个改动没有保留
（相关字段 `radius_sort_w` 仍留在配置里，默认 0 = 关闭，便于复现）。
if key is not None:
 cache[key] = seq
else:
 self._skeleton = seq
 self._skeleton_len_m = best_geo
return seq

def spiral_tour(self, start=None):
 """**问题 3 式的"绕圈"航线**：按环由内向外，每环**角向单调推进、不回头**。

这是把问题 3 的覆盖扫圈（`src/p3_sweep.py`：`pick_stop` 只允许"沿当前绕圈方向单调前进"，`adv` ≥ 0.35 格）搬到问题 4 的格网上：

1. 环 = 半径分档（`grid_a/2`，于是 $r=1000 / 1732 / 1850$ 各成一层）；
2. **由内向外**逐环扫（内环先扫 => 中心附近的源先被听到，指标对"听到时刻"敏感）；
3. 每环内**角向单调**：方向在第一环的第一段确定（取"往哪边转更近"），之后只沿这个方向走 -- 绝对不在同一环上回头；
4. 环间**就近衔接**：下一环的起点取"离当前所在站最近的那个站"，方向**反过来**（下一环沿反方向扫），这样绕完一圈正好回到起点附近，衔接距离最短；
5. 最后用 2-opt 在"保持环序"的前提下微调（只允许环内重排）。

与"几何最短 TSP"的差别：TSP 排法会在环之间来回穿插（半径序列 1000,1850,1732,1000,...），几何上更短，但**角向来回横跳**；绕圈排法一圈只走一个方向，路径形态与问题 3 的八边形/十二边形一致。

"""

 band = max(150.0, float(self.cfg.grid_a) * float(self.cfg.ring_band_frac))
 rings: dict[int, list[int]] = {}
 for si, (x, y) in enumerate(self.stations):
 rings.setdefault(int(round(math.hypot(x, y) / band)), []).append(si)
 cur = np.asarray((0.0, 0.0) if start is None else start, float)
 seq: list[int] = []
 direction = 0.0 # 0 = 未定; +1 = 逆时针; -1 = 顺时针
 for k in sorted(rings):
 grp = rings[k]
 if not grp:
 continue
 # 进环点：离当前位置最近的站（"切换环时就近进入"）
 entry = min(grp, key=lambda si: float(np.hypot(
 *(np.asarray(self.stations[si], float) - cur)))
 a0 = math.atan2(self.stations[entry][1], self.stations[entry][0])
 if direction == 0.0:
 # 定方向：比较"往两个方向扫完这一环"的角向总路程，取短的
 direction = 1.0 if len(grp) else 1.0

```

```

 # 用"下一站"的角向来判断（避免固定方向导致第一段就横穿）
 others = [si for si in grp if si != entry]
 if others:
 nxt = min(others, key=lambda si: abs(
 ((math.atan2(self.stations[si][1], self.stations[si][0]) - a0
 + math.pi) % (2 * math.pi)) - math.pi))
 da = (math.atan2(self.stations[nxt][1], self.stations[nxt][0])
 - a0 + math.pi) % (2 * math.pi) - math.pi
 direction = 1.0 if da >= 0 else -1.0
 order = [entry]
 left = [si for si in grp if si != entry]
 cursor = a0
 while left:
 def adv(si, _c=cursor):
 a = math.atan2(self.stations[si][1], self.stations[si][0])
 d = (a - _c) * direction % (2 * math.pi)
 return d if d > 1e-9 else d + 2 * math.pi
 nxt = min(left, key=adv)
 order.append(nxt)
 left.remove(nxt)
 cursor = math.atan2(self.stations[nxt][1], self.stations[nxt][0])
 seq += order
 cur = np.asarray(self.stations[order[-1]], float)
 direction = -direction # 下一环反向：绕回起点附近，衔接最短
 return seq

def build_queue(self, start=None):
 """建立本遍的扫描航线：**一次排好，全程不再按当前位置重排**。”

 为什么不做"动态最近邻/滚动重排"（这是被实测否掉的两版）：

 * 旧版 A：每一步在"还没去过且此刻有用的站"里取最近 --
 会跳过"此刻收益为 0"的几何中间节点，出现 2 500 m 的长跳，之后还要回头补；
 * 旧版 B：顺路清除后按当前位置重排剩余站位 --
 从偏离航线 1 200 m 的位置做最近邻，会把环形航线打散成"以该点为中心的星形"，
 实测航程反而从 17.9 km 涨到 26.9 km。

 排法由 ``route_mode`` 决定（对照用）：

 * ``"tsp"``（默认）：:meth:`scan_skeleton`（全局 NN + 2-opt + cheapest-insertion
 多起点取优）-- 几何最短，但会在环之间交替穿插；
 * ``"spiral"``：:meth:`spiral_tour` -- 问题 3 式"由内向外绕圈 + 角向单调不回头"。

 顺路清除只是"绕出去一趟再回来"，跳站只发生在"队首此刻确实没有新覆盖"时
 (:meth:`next_scan_station`)。
 """
 if self.cfg.route_mode == "spiral":
 base = self.spiral_tour(start=start)
 self._queue = [si for si in base if si not in self.visited_stations]
 return self._queue # 绕圈排法**不能**再做 2-opt（会把角向单调性打乱）
 base = self.scan_skeleton(start=start)
 self._queue = [si for si in base if si not in self.visited_stations]
 self._queue, _ = self._two_opt(self._queue,
 start=np.asarray(self.pos, float))

 return self._queue

def _two_opt(self, order, start):
 """对 ``order``（站号列表）做 2-opt，返回 ``(新顺序, 长度)``。"""
 P = {si: np.asarray(self.stations[si], float) for si in order}
 s0 = np.asarray(start, float)

 def cost(o):

```



```

 cur, d = s0, 0.0
 for si in o:
 d += float(np.hypot(*(P[si] - cur)))
 cur = P[si]
 return d

if len(order) < 4:
 return list(order), cost(order)
best = cost(order)
improved = True
while improved:
 improved = False
 for i in range(len(order) - 1):
 for j in range(i + 1, len(order)):
 cand = order[:i] + order[i:j + 1][::-1] + order[j + 1:]
 c = cost(cand)
 if c < best - 1e-9:
 order, best, improved = cand, c, True
 return list(order), best

def next_scan_station(self, flip=None):
 """下一个要去的扫描站位。

 两步走（这是把"就近选站"和"不跨环乱跳"同时做到的办法）：

 1. **正常情况**：走基准航线的队首。基准航线是全局 NN + 2-opt 排出来的，
 几何上连贯（实测单跳最大 1 104 m、无 >1 200 m 的长跳）；
 2. **被顺路清除带偏之后**（当前位置到队首的距离 > ``detour_follow_m``）：
 不再硬走回队首，改为**就近切入**--在基准航线里挑"离我最近的那一站"
 作为新的起点，然后**顺着基准航线的顺序**往下走（不是重新做最近邻！）。

 为什么"顺着基准航线"而不是"重新最近邻"：重新最近邻会从偏离点长出一颗
 星形路线，把环形结构彻底打散（实测航程 17.9 km → 26.9 km），
 而且会跳过几何上的中间站、之后还要回头补。
 """
 if not self.unheard():
 return None
 keep = self.measure_set() # 降覆盖档：只在这些站里选（None = 全覆盖）
 # 当前队首是否还值得去
 def usable(si):
 return (si not in self.visited_stations
 and (keep is None or int(si) in keep)
 and self.station_gain(si, self.unheard_mask())
 > self.cfg.station_min_gain)

 while self._queue and not usable(self._queue[0]):
 self._queue.pop(0)
 if not self._queue:
 return None
 pos = np.asarray(self.pos, float)
 d0 = float(np.hypot(*(np.asarray(self.stations[self._queue[0]], float) - pos)))
 if d0 <= self.cfg.detour_follow_m:
 return self._queue[0]
 # 被带偏了：就近切入基准航线（取"离我最近且还有用"的那一站作为新起点）
 best, bd = None, None
 for k, si in enumerate(self._queue):
 if not usable(si):
 continue
 d = float(np.hypot(*(np.asarray(self.stations[si], float) - pos)))
 if bd is None or d < bd:
 best, bd = k, d
 if best is None:

```

```

 return None
 if best > 0:
 self._queue = self._queue[best:] + self._queue[:best]
 self.stats["n_reroutes"] = self.stats.get("n_reroutes", 0) + 1
 self.log(f" 就近切入航线: 跳过 {best} 站, 从站 {self._queue[0]} 继续"
 f" (距当前 {bd:.0f} m, 原队首 {d0:.0f} m)")
 return self._queue[0]

def replan_queue(self, force=False):
 """覆盖职责变化后用**基准航线**重排剩余站位（不再按当前位置重排）。

 旧实现按"当前位置"重排，实测会被顺路清除带偏、把环形航线打散成星形
 （见 :meth:`build_queue` 的说明）。现在只在"有站因为收益为 0 被弹出"或
 显式 ``force`` 时，回到**基准航线**的顺序重排剩余站位。
 """
 self._queue = [si for si in self._queue if si not in self.visited_stations]
 if not self._queue or not force:
 return
 base = self.scan_skeleton()
 rank = {si: k for k, si in enumerate(base)}
 self._queue.sort(key=lambda si: rank.get(si, 10 ** 6))
 self.stats["n_replans"] = self.stats.get("n_replans", 0) + 1

def localize_fast(self, ch):
 """**小幅度移动**把目标定到"能清"的精度，返回 ``(est, radius)``。

 与全量 :meth:`localize` 的差别：这里要的是"够用"而不是"最优"，
 因为每多走 100 m 就是 20 s 的虚拟时间：

 1. 交会区域已经 $\leq$ ``interleave_loc_radius_m`` $\Rightarrow$ 直接用（0 次额外测量）；
 2. 侧移交会一次（420 m、交会角 $\approx 45^\circ$ ）-- 一步就把区域压到 10-40 m，
 这就是用户说的"较小幅度的移动来确定大致方位"；
 3. 只做一次；不成再退回"沿视线二分"（与协议约定无关的那条路）。
 """
 rec = self.recs[int(ch)]
 br = self.bounded_region(rec)
 lim = float(self.cfg.interleave_loc_radius_m)
 if br is not None and float(br["radius"]) <= lim:
 return np.asarray(br["center"], float), float(br["radius"]), br
 if rec.n_bearings == 1 and self.can_afford(60.0):
 self.locate_by_cross(ch)
 br = self.bounded_region(rec)
 if br is not None and float(br["radius"]) <= lim:
 return np.asarray(br["center"], float), float(br["radius"]), br
 g = self.cross_estimate(rec)
 if g is not None:
 r = float(br["radius"]) if br is not None else lim
 return g, r, br
 est, rad, _br = self.locate_on_ray(ch)
 if est is None:
 return None, float("inf"), None
 return est, float(rad), None

def clear_here(self, ch):
 """在当前附近把一个已听到的目标清掉：小幅度定位 $\rightarrow$ 小范围覆盖式试探。

 与全量 :meth:`solve_channel` 的差别：不做"逼近中心 + 多边形试探"那一串
 （那是阶段式清理才划算的），只保留"定位一次 + 以估计点为中心的小圆试探"。
 """
 rec = self.recs[int(ch)]
 if rec.cleared:
 return True

```

```

near_p = self._near_pt.get(int(ch))
if near_p is not None:
 if self.do_clear(float(near_p[0]), float(near_p[1]), int(ch)):
 return True
est, rad, br = self.localize_fast(ch)
if est is None:
 return False
顺带: 人已经在估计点上/附近, 直接试一次 /clear (3-5 s 的事)
d = float(np.hypot(*(np.asarray(self.pos, float) - np.asarray(est, float))))
if d <= self.cfg.clear_est_probe_m and \
 self.do_clear(float(self.pos[0]), float(self.pos[1]), int(ch)):
 return True
self.phase = "probe"
ok = self.sweep_ok(int(ch), rec, np.asarray(est, float),
 self.cfg.interleave_probe_m, poly=None)
if not ok and br is not None and float(br["radius"]) <= self.cfg.probe_max_radius_m:
 ok = self.sweep_ok(int(ch), rec, np.asarray(br["center"], float),
 float(br["radius"]), poly=br.get("poly"))
self.phase = "scan"
if not ok:
 rec.clear_fail += 1
return ok

def pick_unified_action(self, attempts):
 """统一航路的下一步: 在"继续扫描"和"就地清掉已发现目标"之间选。

 结论 (**实测出来的**, 不是先验): **听到就清**永远优于"攒着以后清"。
 `src/p4_tune_unified.py` 的扫参 (20 局) 把"绕去清理的距离门限"从 0 扫到 ∞ :

 | 门限 | 清除率 | 平均定位清除 | 虚拟时间 | 行程 |
 |---|---|---|---|---|
 | 0 (只在第二遍清) | 0.9963 | 1 019 s | 13 607 s | 56.6 km |
 | 700 m | 0.9963 | 976 s | 13 027 s | 54.1 km |
 | 1 100 m | 0.9963 | 884 s | 11 804 s | 48.7 km |
 | ** ∞ (听到就清) ** | **1.0000** | **833 s** | **11 162 s** | **45.9 km** |

 门限越松越好、一直放到"不设门限"最优 -- 原因是**流水线**:

 "绕去清" 的动作与"扫描"的动作在时间上是重叠的 (清完就地从当前站继续扫,

 不用回到原航点), 而"攒着"会把所有清除压到航程末尾, 既拖长总时间、

 又让后面的目标等更久。所以这里不设距离门限, 只按"最近的目标优先"排。

 保留"选哪个目标"的排序 (最近优先) 与"第二遍必清"的兜底:

 第二遍开始时若还有没清的目标 (例如上一遍预算不够), 它们的优先级翻倍。
 """
 pos = np.asarray(self.pos, float)
 best = None
 for ch in self.heard_pending():
 if attempts.get(ch, 0) >= self.cfg.max_solve_attempts:
 continue
 est = self.estimate(ch)[0]
 if est is None:
 continue
 d = float(np.hypot(*(est - pos)))
 # 绕行上限用 ``interleave_detour_max_m`` (默认 ∞ = 不设限)。
 # [!] 别用 ``clear_detour_max_m``: 那是阶段式时代留给"到站顺手清"的 300 m,
 # 套到统一航路上会把大多数清理都延后 $\Rightarrow$ 积压到航程末尾、来回折返
 # (实测 300 m 上限: 648 s $\rightarrow$ 1141 s、行程 34.9 $\rightarrow$ 64.1 km)。
 if d > self.cfg.interleave_detour_max_m and not self.unified_second_pass:
 continue
 cost = d / V_ROBOT + self.cfg.interleave_clear_cost_s
 if self.unified_second_pass:
 cost *= 0.5 # 第二遍残留的目标优先

```

```

 if best is None or cost < best[0]:
 best = (cost, ch, d)
 if best is not None:
 return ("clear", best[1])
 si = self.next_scan_station()
 return None if si is None else ("station", si)

def run_unified(self):
 """**边扫描边清理**：一条滚动重规划的航路走完两圈，点也清完。

 结构（对应用户的要求）：

 ...
 队列 = 扫描站位（由内向外螺旋）
 while 队列非空 或 还有没清的目标：
 每走一站/清一个目标：重读台账、用最新位置重排剩余队列
 下一步 = 已发现目标里最近的一个（就地清），没有就取队首站
 第二遍（换半平面约定）时残留目标优先级翻倍
 ...

 * **听到就地清**（实测最优，见 :meth:`pick_unified_action`）：扫描与清除在同一条
 航路上交错，清完直接从当前站位继续扫，不需要回到原航点；
 * **第二遍**：换另一个半平面约定重扫（兜住"定向方向"的协议约定不确定性），
 这一遍里残留的目标优先清掉，所以两圈走完时点也清完；
 * 收工判据不变：两遍都走完且所有目标已清 $\Rightarrow$ 靶区内没有未发现的源。
 """
 n_clear = 0
 attempts: dict[int, int] = {}
 total_visited = 0
 passes = (False, True) if self.cfg.two_pass else (False,)
 for pi, flip in enumerate(passes):
 self.unified_second_pass = bool(pi > 0)
 self.coverage_flip = bool(flip)
 if pi > 0:
 self.first_pass_heard = {int(ch) for ch in self.heard_pending()}
 if pi > 0 and not self.unheard() and not self.heard_pending():
 break
 self.log(f"-- 第 {pi + 1} 遍（半平面约定 "
 f"{'朝内' if flip else '朝外'}）： "
 f"未听频道 {len(self.unheard())}，待清目标 "
 f"{len(self.heard_pending())} --")
 # 本遍的航线：几何一次排好（全局最近邻 + 2-opt），全程不再按当前位置重排
 self.build_queue()
 guard = 0
 while not self.aborted:
 guard += 1
 if guard > 3 * (len(self.stations) + len(self.recs)) + 20:
 self.log(" 航路步数超限 -> 结束本遍")
 break
 if self.remaining() <= self.cfg.time_reserve_s:
 self.log(" 时间不够 -> 结束本遍")
 break
 # 先看"能不能顺手清掉"，没有可清的就取队首扫描站
 act = self.pick_unified_action(attempts)
 if act is None:
 break
 kind, target = act
 if kind == "clear":
 ch = int(target)
 attempts[ch] = attempts.get(ch, 0) + 1
 est = self.estimate(ch)[0]
 d = (float("inf") if est is None

```

```

 else float(np.hypot(*(est - np.asarray(self.pos, float))))))
 tag = "（第二遍优先）" if self.unified_second_pass else ""
 self.log(f" 统一航路{tag}: 转去清频道{ch}"
 f"（估计距离 {d:.0f} m, 第 {attempts[ch]} 次尝试）")
 self.phase = "enroute"
 if self.clear_here(ch):
 n_clear += 1
 self.phase = "scan"
 # 顺路清除只是"绕出去一趟再回来": **不重排航线**
 # （按当前位置重排会把环形航线打散成星形, 实测航程 +50%）
 continue
 si = int(target)
 x, y = self.stations[si]
 dist = math.hypot(x - self.pos[0], y - self.pos[1])
 if not self.can_afford(dist / V_ROBOT + T_MEASURE + T_SWITCH + 4.0):
 self.log(" 剩余时间不足以再跑一站 -> 结束本遍")
 break
 self.visit_station(si)
 total_visited += 1
 # 到站后先顺手清掉"就在跟前"的目标, 再继续
 if self.cfg.clear_detour_max_m > 0 and not self.aborted:
 self._opportunistic_clear(max_n=self.cfg.enroute_max_n)
 if self.aborted:
 break
 self.stats["n_visited_stations"] = len(self.visited_stations)
 self.stats["vtime_after_scan"] = float(self.arena.virtual_time_s)
 left = self.heard_pending()
 self.log(f"统一航路结束: 访问 {len(self.visited_stations)} 站"
 f"（共 {total_visited} 站次）, 清除 {n_clear} 个, "
 f"仍未清 {left} | 虚拟 {self.arena.virtual_time_s:.0f} s")
 # 收尾: 还有目标没清就用全量定位-清除流程兜底
 if left and not self.aborted and self.remaining() > self.cfg.time_reserve_s:
 self.log(f"收尾阶段: 用完整流程清剩余 {left}")
 self.phase = "clear"
 n_clear += self.clear_all_heard()
 self.stats["n_cleared"] = n_clear
 return n_clear

def _staged_scan_then_clear(self):
 """阶段式（对照）：先按覆盖判据扫两遍，再统一清理。

 ``P4Config(unified=False)`` 时使用 -- 保留它是为了做"边扫边清 vs
 先扫后清"的 A/B（消融表要用实测数字说话）。
 """
 total_visited = 0
 for flip in ((False, True) if self.cfg.two_pass else (False,)):
 self.coverage_flip = bool(flip)
 if flip and not self.unheard():
 break
 left0 = self.unheard()
 if not left0:
 break
 self.log(f"扫描第 {1 + int(flip)} 遍（半平面约定 "
 f"{'朝内' if flip else '朝外'}）: "
 f"还有 {len(left0)} 个频道没听到过 {left0}")
 total_visited += self.sweep_pass(flip=bool(flip))
 if self.aborted:
 break
 self.stats["n_visited_stations"] = len(self.visited_stations)
 self.stats["vtime_after_scan"] = float(self.arena.virtual_time_s)
 left = self.unheard_cover_left()
 self.log(f"扫描阶段结束: 访问 {len(self.visited_stations)} 站"

```

```

 f" (共 {total_visited} 站次), "
 f"听到 {len(self.heard_pending())} 个频道, "
 f"未覆盖格点 {left} 个"
 f"{' (=0, 可断定靶区内已无未发现的源)' if left == 0 else ''}"
 f"| 虚拟 {self.arena.virtual_time_s:.0f} s")
self.phase = "clear"
self.stats["n_cleared"] = self.clear_all_heard()

def _opportunistic_clear(self, radius_m=None, max_n=2):
 """扫到哪儿就清哪儿: 刚听到的目标, 只要**绕不了多远**就当场清掉。

 为什么重要 (路径图直接暴露的问题): 平均定位清除时间 = 定位清除总时刻 ÷ 清除个数,
 什么时候听到"决定了指标的下限。第一遍扫描要跑 ~30 站、花掉一半以上的虚拟
 时间: 如果一个源在 origin 就被听到了却要等整圈扫完才清, 那它的"定位清除时间"
 就是整圈的时间 (实测"中心密集"场景平均 1286 s)。听到就清, 等于把这个数
 压到"走到它跟前"的量级。

 代价与约束:

 * 绕行 = 从当前位置到目标再折回航线的距离; 只有 ``dist ≤ clear_detour_max_m``
 才值得绕 (否则等航线自然路过更省);
 * 估计位置来自"沿视线二分" (与协议约定无关), 精度 ~20-40 m,
 配 90 m 小圆覆盖式试探足够保证清除成功。
 """
 lim = float(self.cfg.clear_detour_max_m if radius_m is None else radius_m)
 n = 0
 for ch in self.heard_pending():
 if n >= max_n or self.aborted:
 break
 if not self.can_afford(self.cfg.min_action_budget_s + 40.0):
 break
 est, rad, _br = self.estimate(ch)
 if est is None:
 continue
 d = float(np.hypot(*(np.asarray(self.pos, float) - np.asarray(est, float))))
 if d > lim:
 continue
 self.log(f" 顺路清除频道{ch} (估计位置距当前 {d:.0f} m, 绕行上限 {lim:.0f} m)")
 self.phase = "enroute"
 if self.solve_channel(ch):
 n += 1
 self.phase = "scan"
 return n

def clear_all_heard(self):
 """清理阶段: 滚动时域--每清一个就按最新位置重排剩余目标。

 三条防呆 (都是实测出来的教训):

 * 同一频道最多进 :meth:`solve_channel` ``max_solve_attempts`` 次
 (否则"定位→试探→失败→再定位"会在同一个点上无限循环, 实测烧掉 30 000 s);
 * 剩余时间不够就退出, 不做无谓动作;
 * 优先清"估计位置离当前位置近"的目标 (更可能顺手清掉)。
 """
 n = 0
 attempts: dict[int, int] = {}
 while not self.aborted:
 todo = [ch for ch in self.heard_pending()
 if attempts.get(ch, 0) < self.cfg.max_solve_attempts]
 if not todo:
 break
 if not self.can_afford(self.cfg.min_action_budget_s + 20.0):

```

```

 self.log("剩余时间不足 -> 结束清理阶段")
 break
 pos = np.asarray(self.pos, float)
 order = []
 for ch in todo:
 est = self.estimate(ch)[0]
 d = float("inf") if est is None else float(np.hypot(*(est - pos)))
 order.append((d, ch))
 order.sort()
 ch = order[0][1]
 attempts[ch] = attempts.get(ch, 0) + 1
 self.log(f"清理阶段: {len(todo)} 个已发现频道待清 {sorted(todo)}"
 f" -> 先清频道{ch} (第 {attempts[ch]} 次尝试)")
 if self.solve_channel(ch):
 n += 1
 left = self.heard_pending()
 if left:
 self.log(f"清理阶段结束: 仍未清除 {left} (方位不足或时间不够)")
 return n

```

## 7.10 Q4/src/p4\_homing.py

```

"""Reply-only homing for Q4, including sources with arbitrary 180 degree cones.

`P4HomingMixin` can precede any P4GridRobot subclass in its MRO. It replaces
estimate/region and both local clearance entry points. Negative measurements
never exclude a distance disk: loss of signal can be caused by the cone.
"""
from __future__ import annotations

from dataclasses import dataclass
import math

import numpy as np

from p1_intersection import clip_halfplane, wedge_halfplanes, min_enclosing_circle
from p3_arena import R_ARENA, R_RECV_MAX, R_CLEAR, V_ROBOT
from p3_robot import _dist_point_polygon
from p4_robot import P4Config, P4GridRobot

@dataclass
class P4HomingConfig(P4Config):
 home_fraction: float = 0.60
 home_lateral_m: float = 100.0
 home_max_steps: int = 8
 home_sweep_radius_m: float = 75.0
 home_clear_spacing_m: float = 28.0

class P4HomingMixin:
 """Finite positive-bearing polygon, short baseline, then adaptive approach."""

 def __init__(self, *args, **kwargs):
 self._p4_home_regions = {}
 self._p4_home_clear_points = {}
 self._p4_home_caps = {}
 super().__init__(*args, **kwargs)

 def home_region(self, rec):

```

```

if rec.near_hits:
 row = next(row for row in reversed(rec.meas_log) if row[3] == 'near')
 return dict(center=tuple(row[:2]), radius=5.0, poly=None,
 status='bounded', n_bearings=rec.n_bearings)

if not rec.pts:
 return None

caps = self._p4_home_caps.get(rec.channel, [])
key = (rec.channel, len(rec.pts), len(caps))
if key in self._p4_home_regions:
 return self._p4_home_regions[key]
poly = [np.array(p, float) for p in
 [(-R_ARENA, -R_ARENA), (R_ARENA, -R_ARENA),
 (R_ARENA, R_ARENA), (-R_ARENA, R_ARENA)]]
Circumscribed disk approximations retain every physically possible
source. The extra 0.005 degrees covers bearing serialization rounding.
for station, radius in [(np.zeros(2), R_ARENA)] + [
 (np.array(p), R_RECV_MAX) for p in rec.pts]:
 for angle in np.arange(0, 2 * math.pi, math.pi / 16):
 u = np.array([math.cos(angle), math.sin(angle)])
 poly = clip_halfplane(poly, u, -float(u @ station) - radius)
for station, bearing in zip(rec.pts, rec.svds):
 for normal, constant in wedge_halfplanes(station, bearing, 1.005 + 1e-8):
 poly = clip_halfplane(poly, normal, constant)
for normal, constant in caps:
 poly = clip_halfplane(poly, normal, constant)
if not poly:
 return None
arr = np.asarray(poly)
origin = arr.mean(axis=0)
circle = min_enclosing_circle(arr - origin)
center = np.asarray(circle[0]) + origin if circle else origin
radius = float(np.max(np.linalg.norm(arr - center, axis=1)))
result = dict(center=tuple(center), radius=radius, poly=arr.tolist(),
 status='bounded', n_bearings=rec.n_bearings)
self._p4_home_regions[key] = result
return result

def region(self, rec):
 br = self.home_region(rec)
 if br is None:
 return dict(status='none', n_bearings=rec.n_bearings)
 return dict(status='bounded', n_bearings=rec.n_bearings,
 min_enclosing_center=br['center'],
 min_enclosing_radius_m=br['radius'], vertices=br['poly'] or [])

def bounded_region(self, rec):
 return self.home_region(rec)

def estimate(self, ch):
 br = self.home_region(self.recs[int(ch)])
 if br is None:
 return None, float('inf'), None
 return np.asarray(br['center']), br['radius'], br

def _home_clear(self, ch, point):
 point = tuple(map(float, point))
 tried = self._p4_home_clear_points.setdefault(int(ch), [])
 if any(math.dist(point, prev) < 0.2 for prev in tried):
 return False
 if not self.can_afford(math.dist(self.pos, point) / V_ROBOT + 6):
 return False
 if self.do_clear(*point, int(ch)):

```



```

 return True
 if not self.aborted:
 tried.append(point)
 return False

def _home_measure(self, ch, point):
 point = tuple(map(float, point))
 if not self.can_afford(math.dist(self.pos, point) / V_ROBOT + 12):
 return None
 reply = self.measure(*point, int(ch))
 self.note_meas(point)
 self.stats['n_locate'] = self.stats.get('n_locate', 0) + 1
 if reply.get('measure_result') == 'near':
 self._home_clear(ch, point)
 return reply.get('measure_result')

def _home_cross(self, ch, br, scale=1.0):
 """Try both sides of a short baseline, retreating after two negatives.

 All candidate coordinates come from an actual positive bearing. A
 lone negative does not change the feasible source polygon. Trying opposite
 sides avoids depending on a guessed radial emission orientation.
 """
 rec = self.recs[int(ch)]
 source_station = np.asarray(rec.pts[-1], float)
 angle = math.radians(rec.svds[-1])
 u = np.array([math.cos(angle), math.sin(angle)])
 normal = np.array([-u[1], u[0]])
 center = np.asarray(br['center'])
 along = max(0.0, float((center - source_station) @ u))
 forward = getattr(self.cfg, 'home_fraction', 0.60) * along * scale
 lateral = min(getattr(self.cfg, 'home_lateral_m', 100.0),
 max(22.0, 0.3 * max(along, 1.0))) * scale
 midpoint = source_station + forward * u
 candidates = [midpoint + sign * lateral * normal for sign in (-1, 1)]
 candidates.sort(key=lambda p: math.dist(p, self.pos))
 old_n = rec.n_bearings
 results = []
 for point in candidates:
 result = self._home_measure(ch, point)
 results.append(result)
 if rec.cleared or result is None:
 return rec.cleared
 if rec.n_bearings > old_n:
 return True
 # Paired negatives have more information than one negative. Let S be
 # the positive station and $Q_{\pm} = S + t \cdot u_{\pm}$. If a feasible source
 # G lies beyond that cross-section, its bearing ray crosses segment
 # $Q-Q_{+}$. At least one endpoint is in the same closed emission halfplane
 # as S. Moreover $t \geq 1/(\sec(e) + \tan(e))$ makes BOTH endpoints closer to
 # every such G than S, hence inside its actual reception radius. Thus
 # two negatives rule out the entire far part of this bearing wedge.
 # This uses neither a radial orientation assumption nor a negative
 # 1000 m exclusion disk; a lone negative never adds a constraint.
 err = math.radians(1.005 + 1e-8)
 if (results == ['no_signal', 'no_signal']
 and forward * math.tan(err) <= lateral - 1e-5
 and forward >= lateral * (1.0 / math.cos(err) + math.tan(err)) + 1e-5):
 constant = -float(u @ source_station) - forward - 1e-6
 self.p4_home_caps.setdefault(int(ch), []).append((u, constant))
 self.stats['n_home_pair_caps'] = self.stats.get('n_home_pair_caps', 0) + 1
 return False

```

```

def _home_cover_polygon(self, ch, br):
 """Finite geometric fallback, independent of signal reception.

 A square lattice with spacing <= 20 sqrt(2) covers its entire plane
 with clear disks. Keep every lattice point whose disk meets the outer
 polygon. This also handles a source exactly on its cone boundary.
 """
 if not br['poly']:
 return self._home_clear(ch, br['center'])
 poly = np.asarray(br['poly'], float)
 step = min(getattr(self.cfg, 'home_clear_spacing_m', 28.0), 28.0)
 # Rotate to the polygon principal direction: a one-bearing wedge is
 # narrow, so this avoids a large rectangular grid of irrelevant cells.
 c = np.asarray(br['center'])
 cov = (poly - c).T @ (poly - c)
 _values, basis = np.linalg.eigh(cov)
 local = (poly - c) @ basis
 low = np.floor(local.min(axis=0) / step).astype(int) - 1
 high = np.ceil(local.max(axis=0) / step).astype(int) + 1
 candidates = []
 for i in range(low[0], high[0] + 1):
 for j in range(low[1], high[1] + 1):
 point = c + (np.array([i, j]) * step) @ basis.T
 if _dist_point_polygon(point, poly) <= R_CLEAR + 1e-7:
 candidates.append(tuple(point))
 self.stats['n_home_fallback'] = self.stats.get('n_home_fallback', 0) + 1
 while candidates and not self.aborted:
 index = min(range(len(candidates)),
 key=lambda k: math.dist(candidates[k], self.pos))
 point = candidates.pop(index)
 if self._home_clear(ch, point):
 return True
 if not self.can_afford(12):
 break
 return False

def clear_here(self, ch):
 rec = self.recs[int(ch)]
 if rec.cleared:
 return True
 previous_phase = self.phase
 self.phase = 'home'
 for iteration in range(getattr(self.cfg, 'home_max_steps', 8)):
 if self.aborted or not self.can_afford(15):
 break
 br = self.home_region(rec)
 if br is None:
 break
 if rec.n_bearings >= 2 or br['radius'] <= R_CLEAR:
 if self._home_clear(ch, br['center']):
 self.phase = previous_phase
 return True
 if br['radius'] <= getattr(self.cfg, 'home_sweep_radius_m', 75.0):
 result = self._home_measure(ch, br['center'])
 if rec.cleared:
 self.phase = previous_phase
 return True
 updated = self.home_region(rec)
 if updated and updated['radius'] < br['radius'] * 0.8:
 continue
 ok = self._home_cover_polygon(ch, updated or br)

```

```

 self.phase = previous_phase
 return ok
 result = self._home_measure(ch, br['center'])
 if rec.cleared:
 self.phase = previous_phase
 return True
 if result in ('direction', 'near'):
 continue

 # Retrying with a smaller forward step covers both an overshoot and
 # a cone-boundary rejection, without inferring which caused it.
 br = self.home_region(rec)
 if br is None:
 break
 before = rec.n_bearings
 for scale in (1.0, 0.5, 0.25):
 if self._home_cross(ch, br, scale) or self.aborted:
 break
 if rec.cleared:
 self.phase = previous_phase
 return True
 if rec.n_bearings == before:
 break
 br = self.home_region(rec)
 ok = bool(br and self._home_cover_polygon(ch, br))
 self.phase = previous_phase
 if not ok:
 rec.clear_fail += 1
 return ok

def solve_channel(self, ch):
 return self.clear_here(ch)

class P4HomingRobot(P4HomingMixin, P4GridRobot):
 def __init__(self, arena, cfg=None, **kwargs):
 super().__init__(arena, cfg or P4HomingConfig(), **kwargs)

```

## 7.11 Q4/src/p4\_search.py

```

"""Q4 joint routing, directional coverage, and reply-only homing.

The legacy radial half-plane coverage model is deliberately not used here.
All scheduled stations are actually measured once on every unheard channel.
"""

from __future__ import annotations
from dataclasses import dataclass
import math
import time
import numpy as np
from p3_arena import ArenaError
from p4_homing import P4HomingMixin, P4HomingConfig
from p4_robot import P4GridRobot
from p4_certificate import DirectionalCertificate, strict_layout

@dataclass
class P4SearchConfig(P4HomingConfig):
 tier: str = 'strict'
 search_rings: tuple = ((900.,6),(1900.,12))
 search_layout: str = 'certified'

```

```

joint_route: bool = True
after_clear_scan: bool = False
scan_heard: bool = True
route_polish: bool = True
immediate_service: bool = False
certify: bool = True
certificate_step_m: float = 40.
certificate_n_dir: int = 48
prune_stations: bool = False
endpoint_scan_min_saved: float = 0.0
min_move: float = 0.
opening_baseline_m: float = 0.
jointly_scan_targets: bool = False

def tier_config(name='strict'):
 configs={
 'strict':dict(search_layout='certified',certify=True,prune_stations=False),
 'compact':dict(search_rings=((900.,6),(1900.,12))),
 'dense':dict(search_layout='legacy'),
 'fast99':dict(search_rings=((1000.,6),(1850.,9))),
 'fast98':dict(search_rings=((1000.,4),(1750.,8))),
 'fast95':dict(search_rings=((1100.,4),(1650.,8))),
 'fast90':dict(search_rings=((1450.,8))),
 'inner':dict(search_rings=((1100.,6))),
 }
 if name not in configs:raise ValueError(name)
 options=dict(search_layout='rings',certify=False,prune_stations=True)
 options.update(configs[name])
 return P4SearchConfig(tier=name,**options)

class P4SearchRobot(P4HomingMixin, P4GridRobot):
 def __init__(self,arena,cfg=None,base=None,log=None):
 super().__init__(arena,cfg or P4SearchConfig(),base=base,log=log)
 self._search_visits=0
 self.certificate=DirectionalCertificate(self.cfg.certificate_step_m,self.cfg.certificate_n_dir)
 self._station_bits=[np.packbits(self.certificate.station_mask(p)) for p in self.stations]
 self._covered_bits=np.packbits(self.certificate.empty())
 self._desired_bits=self._covered_bits.copy()
 self._desired_bits |= np.packbits(self.certificate.station_mask((0.,0.)))
 for bits in self._station_bits:self._desired_bits |= bits

 def _build_stations(self):
 if self.cfg.search_layout=='certified':return strict_layout()[1:]
 if self.cfg.search_layout=='legacy':return super()._build_stations()
 points=[]
 for k,(r,n) in enumerate(self.cfg.search_rings):
 offset=0. if k%2==0 else math.pi/n
 points.extend((r*math.cos(offset+2*math.pi*i/n),
 r*math.sin(offset+2*math.pi*i/n)) for i in range(n))
 return points

 @staticmethod
 def _length(seq,start):
 return sum(math.dist(a,b['xy']) for a,b in zip([start]+[w['xy'] for w in seq],seq))

 def _ordered(self,items):
 if not items:return []
 pts=np.array([self.pos]+[w['xy'] for w in items]);d=np.linalg.norm(pts[:,None]-pts[None,:],axis=2)
 left=list(range(1,len(pts)));seq=[0]
 while left:

```

```

 k=min(left,key=lambda j:d[seq[-1],j]);seq.append(k);left.remove(k)
 if self.cfg.route_polish:
 for _ in range(8):
 changed=False
 for i in range(1,len(seq)-1):
 for j in range(i+1,len(seq)):
 delta=d[seq[i-1],seq[j]]-d[seq[i-1],seq[i]]
 if j+1<len(seq):delta+=d[seq[i],seq[j+1]]-d[seq[j],seq[j+1]]
 if delta < -1e-7:seq[i:j+1]=seq[i:j+1][::-1];changed=True
 if not changed:break
 return [items[i-1] for i in seq[1:]]

def _route(self):
 scans=[dict(kind='station',si=i,xy=p) for i,p in enumerate(self.stations)
 if i not in self.visited_stations] if self.unheard() else []
 targets=[]
 for ch in self.heard_pending():
 c,_,_=self.estimate(ch)
 if c is not None:targets.append(dict(kind='clear',ch=ch,xy=tuple(c)))
 if self.cfg.jointly_scan_targets and scans:
 scans,targets=self._joint_duties(scans,targets)
 elif self.cfg.prune_stations and scans:
 scans=self._prune(scans,targets)
 if self.cfg.immediate_service and targets:
 return self._ordered(targets)
 if self.cfg.joint_route:return self._ordered(scans+targets)
 return self._ordered(scans) or self._ordered(targets)

def _joint_duties(self,scans,targets):
 items=[dict(w) for w in scans+targets]
 for w in items:
 w['duty']=True
 w['bits']=self._station_bits[w['si']] if w['kind']=='station' else np.packbits(self.certificate.
station_mask(w['xy']))
 route=self._ordered(items)
 costs={id(w):0. for w in items}
 for i,w in enumerate(route):
 if w['kind']!='station':continue
 a=self.pos if i==0 else route[i-1]['xy']
 savings=math.dist(a,w['xy'])
 if i+1<len(route):savings+=math.dist(w['xy'],route[i+1]['xy'])-math.dist(a,route[i+1]['xy'])
 costs[id(w)]=savings
 for w in sorted(items,key=lambda w:costs[id(w)],reverse=True):
 covered=self._covered_bits.copy()
 for v in items:
 if v is not w and v['duty']:covered |= v['bits']
 if not np.any(self._desired_bits & ~covered):w['duty']=False
 for w in items:del w['bits']
 return ([w for w in items if w['kind']=='station' and w['duty']],
 [w for w in items if w['kind']=='clear'])

def _prune(self,scans,targets,covered=None):
 covered=self._covered_bits if covered is None else covered
 scans=list(scans)
 # Prefer deleting the stations with the largest travel/scan cost.
 route=self._ordered(scans+targets)
 score={}
 for i,w in enumerate(route):
 if w['kind']!='station':continue
 prev=self.pos if i==0 else route[i-1]['xy']
 saving=math.dist(prev,w['xy'])
 if i+1<len(route):saving+=math.dist(w['xy'],route[i+1]['xy'])-math.dist(prev,route[i+1]['xy'])

```

```

 score[w['si']] = saving
 for w in sorted(scans, key=lambda w: score.get(w['si'], 0), reverse=True):
 all_bits = covered.copy()
 for v in scans:
 if v is not w: all_bits |= self._station_bits[v['si']]
 if not np.any(self._desired_bits & ~all_bits): scans.remove(w)
 return scans

def _scan_actual(self, p, extra=True):
 channels = list(self.unheard())
 if self.cfg.scan_heard and extra:
 for ch in self.heard_pending():
 rec = self.recs[ch]
 if rec.n_bearings > 1: continue
 if math.dist(rec.pts[-1], p) < 80.: continue
 c, _, _ = self.estimate(ch)
 if c is not None and math.dist(c, p) <= 1450: channels.append(ch)
 self._last_scan_complete = True
 if channels:
 self.scan_points.append(tuple(p)); n = self.scan_at(*p, channels)
 self.note_meas(self.pos)
 self._last_scan_complete = n == len(channels) and not self.aborted
 if self._last_scan_complete:
 self._covered_bits |= np.packbits(self.certificate.station_mask(p))
 return len(channels)

def scan_at(self, x, y, chans, station=None):
 n = 0
 for ch in chans:
 if self.aborted or not self.can_afford(math.dist(self.pos, (x, y)) / 5 + 12): break
 reply = self.measure(x, y, ch, station=station)
 if reply.get('accepted') is not True: break
 n += 1
 if reply.get('measure_result') == 'near': self.do_clear(x, y, ch)
 return n

def _certify_remaining(self):
 cert = DirectionalCertificate(20, 48)
 cache = {}
 for rec in self.recs.values():
 if rec.cleared: continue
 if rec.n_bearings or rec.near_hits: return False
 points = tuple(sorted(set(tuple(row[:2]) for row in rec.meas_log if row[3] == 'no_signal')))
 if points not in cache: cache[points] = cert.continuous_complete(points)
 if not cache[points]: return False
 return True

def _endpoint_worth_scan(self):
 if not self.cfg.endpoint_scan_min_saved: return True
 if not self.unheard(): return False
 scans = [dict(kind='station', si=i, xy=p) for i, p in enumerate(self.stations)
 if i not in self.visited_stations]
 before = self._prune(scans, [])
 covered = self._covered_bits | np.packbits(self.certificate.station_mask(self.pos))
 after = self._prune(scans, [], covered)
 n_saved = len(before) - len(after)
 time_saved = (self._length(self._ordered(before), self.pos) - self._length(self._ordered(after), self.pos)) / 5
 time_saved += (n_saved - 1) * 6 * len(self.unheard())
 return time_saved >= self.cfg.endpoint_scan_min_saved

def _opening_baseline(self):
 channels = self.heard_pending()

```

```

if len(channels)<2 or self.cfg.opening_baseline_m<=0:return
route=self._route()
dest=np.array(route[0]['xy']) if route else np.zeros(2)
best=None
for a in np.arange(24)*2*math.pi/24:
 p=self.cfg.opening_baseline_m*np.array([math.cos(a),math.sin(a)])
 error=0.
 for ch in channels:
 c,_,_=self.estimate(ch);u=c/np.linalg.norm(c)
 v=c-p;cross=abs(float(u[0]*v[1]-u[1]*v[0]))
 error+=min(500.,math.radians(1.005)*np.linalg.norm(v)*np.linalg.norm(c)/max(cross,1.))
 cost=error+0.25*np.linalg.norm(dest-p)
 if best is None or cost<best[0]:best=(cost,p)
self.scan_at(*best[1],channels)

def run(self):
 t0=time.perf_counter()
 reply=self.arena.enter()
 if reply.get('accepted') is not True:raise ArenaError(str(reply))
 complete=False
 try:
 self._scan_actual(self.pos,extra=False)
 self._opening_baseline()
 for _ in range(400):
 if self.aborted or not self.can_afford(15):break
 route=self._route()
 if not route:break
 w=route[0]
 if w['kind']=='station':
 self._scan_actual(w['xy'])
 if self._last_scan_complete:
 self.visited_stations.add(w['si']);self._search_visits+=1
 else:break
 else:
 ch=w['ch']
 before=(self.stats['n_measure'],self.stats['n_clear'])
 self.clear_here(ch)
 if self.recs[ch].cleared and (w.get('duty') or (self.cfg.after_clear_scan and self._
_endpoint_worth_scan())):
 self._scan_actual(self.pos)
 if before==(self.stats['n_measure'],self.stats['n_clear']):break
 self.stats['search_stations']=self._search_visits
 if self.cfg.certify:complete=self._certify_remaining()
 finally:
 self.arena.exit()
 self.stats['program_runtime_s']=time.perf_counter()-t0
 rep=self.report();rep['search_stations']=self._search_visits
 mask=np.unpackbits(self._covered_bits,count=int(np.prod(self.certificate.shape))).reshape(self.
certificate.shape).astype(bool)
 rep['completion_certified']=bool(self.certificate.summarize(mask).complete and not self.heard_pending())
 if self.cfg.certify:
 rep['completion_certified']=complete
 rep['policy']=self.cfg.tier
 rep['last_clear_time_s']=self.stats['vtime_at_last_clear']
 rep['termination']='certified_complete' if complete else ('budget_or_incomplete' if self.cfg.certify else
'configured_search_complete')
 return rep

```

## 7.12 Q4/src/p4\_adaptive\_cover.py

```

"""Joint coverage/clearance planning with witness refinement and full proof.

A sparse witness subset ranks candidates cheaply. Every proposed route is
then repaired against the complete 20 m x 48 orientation certificate. Only
actual negative replies establish the final absence certificate.
"""

from dataclasses import dataclass
import math
import numpy as np
from p4_search import P4SearchConfig, P4SearchRobot
from p4_certificate import DirectionalCertificate

@dataclass
class P4AdaptiveConfig(P4SearchConfig):
 tier: str = 'adaptive'
 certificate_step_m: float = 20.
 certificate_n_dir: int = 48
 free_radai: tuple = (800., 1000., 1200., 1400., 1650., 1875., 1950.)
 free_angles: int = 24
 gain_power: float = 1.0
 scan_cost_weight: float = 1.0
 use_target_scans: bool = True
 witness_stride: int = 6
 refine_passes: int = 2

class P4AdaptiveRobot(P4SearchRobot):
 def __init__(self, arena, cfg=None, **kwargs):
 super().__init__(arena, cfg or P4AdaptiveConfig(), **kwargs)
 anchors=list(self.stations)
 for r in self.cfg.free_radai:
 for a in np.arange(self.cfg.free_angles)*2*math.pi/self.cfg.free_angles:
 p=(float(r*math.cos(a)),float(r*math.sin(a)))
 if all(math.dist(p,q)>1 for q in self.stations):self.stations.append(p)
 self._fine_cache={tuple(p):bits for p,bits in zip(anchors,self._station_bits)}
 self.witness=DirectionalCertificate(20,self.cfg.certificate_n_dir)
 ij=np rint((self.witness.centers+1790)/20).astype(int)
 keep=np.all(ij%self.cfg.witness_stride==0,axis=1)
 self.witness.centers=self.witness.centers[keep]
 self._witness_bits=np.array([np.packbits(self.witness.station_mask(p)) for p in self.stations])
 self._witness_actual=np.zeros(self._witness_bits.shape[1],np.uint8)
 self._witness_desired=np.packbits(np.ones(self.witness.shape,bool))
 self._actual_points=[]

 def _fine(self,p):
 key=tuple(p)
 if key not in self._fine_cache:
 self._fine_cache[key]=np.packbits(self.certificate.station_mask(p,cache=False))
 return self._fine_cache[key]

 def _scan_actual(self,p,extra=True):
 n=super()._scan_actual(p,extra)
 if self._last_scan_complete and n:
 self._witness_actual |= np.packbits(self.witness.station_mask(p))
 self._actual_points.append(tuple(p))
 return n

 def _insertion_costs(self,pts,seq):
 prev=np.array([self.pos]+[w['xy'] for w in seq])
 distances=np.linalg.norm(pts[:,None]-prev[None,:],axis=2)
 cost=distances.copy()

```



```

 if seq:cost[:, :-1] += distances[:, 1:] - np.linalg.norm(np.diff(prev, axis=0), axis=1)
 inds = np.argmin(cost, axis=1)
 return cost[np.arange(len(pts)), inds], inds

def _select(self, items, masks, missing, targets, available, initial=None):
 seq = self._ordered([dict(w) for w in targets]) if initial is None else initial
 pts = np.array([w['xy'] for w in items])
 scan_cost = 5*6*len(self.unheard())*self.cfg.scan_cost_weight
 selected = []
 for _ in range(60):
 if not np.any(missing): break
 gain = np.bitwise_count(masks & missing).sum(axis=1).astype(float)
 cost, place = self._insertion_costs(pts, seq)
 for i, w in enumerate(items):
 if w['kind'] == 'clear': cost[i] = 0.
 score = gain**self.cfg.gain_power / (np.maximum(cost, 0) + scan_cost + 100.)
 score[~available] = -1.
 j = int(np.argmax(score))
 if not available[j] or gain[j] <= 0: break
 available[j] = False; selected.append(j); missing &= ~masks[j]
 w = dict(items[j], duty=True)
 if w['kind'] == 'station': seq.insert(int(place[j]), w)
 else:
 next(v for v in seq if v.get('ch') == w['ch'])['duty'] = True
 return seq, selected, missing

def _route(self):
 targets = []
 for ch in self.heard_pending():
 c, _, _ = self.estimate(ch)
 if c is not None: targets.append(dict(kind='clear', ch=ch, xy=tuple(c), duty=False))
 if not self.unheard(): return self._ordered(targets)
 items = [dict(kind='station', si=i, xy=p) for i, p in enumerate(self.stations)]
 masks = self._witness_bits
 if self.cfg.use_target_scans and targets:
 items += targets
 masks = np.concatenate([masks, np.array([np.packbits(self.witness.station_mask(w['xy'])) for w in
targets])])
 available = np.ones(len(items), bool)
 for i in self.visited_stations: available[i] = False
 missing = self._witness_desired & ~self._witness_actual
 seq, chosen, missing = self._select(items, masks, missing.copy(), targets, available)
 # Refine the sparse witness plan using all states. This is the actual
 # safety constraint, including boundary cells and interval margins.
 covered = self._covered_bits.copy()
 for j in chosen: covered |= self._fine(items[j]['xy'])
 full = np.packbits(np.ones(self.certificate.shape, bool))
 missing = full & ~covered
 if np.any(missing):
 active_bytes = np.flatnonzero(missing)
 fine_masks = np.array([self._fine(w['xy'])[active_bytes] for w in items])
 seq, extra, left = self._select(items, fine_masks, missing[active_bytes].copy(), targets, available, seq)
 chosen += extra
 if np.any(left): raise RuntimeError('Candidate union cannot certify the proposed route')
 # Remove redundant duties, favouring stations with costly detours.
 seq = self._ordered(seq)
 selected = [w for w in seq if w.get('duty')]
 costs = {id(w): 0. for w in selected}
 for i, w in enumerate(seq):
 if w['kind'] != 'station': continue
 a = self.pos if i == 0 else seq[i-1]['xy']
 costs[id(w)] = math.dist(a, w['xy'])

```

```

 if i+1<len(seq):costs[id(w)]+=math.dist(w['xy'],seq[i+1]['xy'])-math.dist(a,seq[i+1]['xy'])
 for w in sorted(selected,key=lambda w:costs[id(w)],reverse=True):
 other=self._covered_bits.copy()
 for v in selected:
 if v is not w and v.get('duty'):other |= self._fine(v['xy'])
 if not np.any(full & ~other):w['duty']=False
 return self._ordered([w for w in seq if w['kind']=='clear' or w.get('duty')])

```

## 7.13 Q4/src/p4\_layout\_v2.py

```

"""Adaptive continuous directional layout certificate and candidate layouts.

Every refined square is certified by the same exact angular-interval union
and strict spatial margin as DirectionalCertificate. Refinement changes the
proof's resolution; it does not replace the continuous square by samples.
"""

from __future__ import annotations
import math
import numpy as np
from p4_certificate import DirectionalCertificate, ring_layout

def adaptive_complete(points, initial_step_m=40., min_step_m=.625, *, detail=False):
 cert=DirectionalCertificate(initial_step_m,48)
 counts=[]
 while True:
 covered=cert.all_orientation_cells(points)
 left=cert.centers[~covered]
 counts.append((cert.cell_side_m,len(cert.centers),len(left)))
 if not len(left):
 return (True,counts) if detail else True
 # Find an actual unsupported interior point before refining large
 # blind regions. This is only an early rejection, never acceptance.
 centers,rho=cert.centers,cert.rho
 cert.centers=left[np.sum(left*left,axis=1)<cert.arena_radius_m**2]
 cert.rho=0.
 point_gap=not np.all(cert.all_orientation_cells(points))
 cert.centers,cert.rho=centers,rho
 if point_gap:
 return (False,counts) if detail else False
 if cert.cell_side_m/2 < min_step_m-1e-12:
 return (False,counts) if detail else False
 side=cert.cell_side_m/2
 offsets=np.array([[-1.,-1.],[-1.,1.],[1.,-1.],[1.,1.]])*side/2
 centers=(left[:,None,:]+offsets[None,:,:]).reshape(-1,2)
 closest=np.maximum(np.abs(centers)-side/2,0)
 cert.centers=centers[np.sum(closest*closest,axis=1)<=cert.arena_radius_m**2]
 cert.cell_side_m=side
 cert.rho=side/math.sqrt(2)

def compact_layout():
 """22 points; 17.70 km static open tour; certified by 5 m refinement."""
 return ring_layout(((980.,7,0.), (1856.,14,0.)))

def compact21_layout():
 """21 points; 17.91 km tour; certificate refines to 2.5 m."""
 return ring_layout(((998.,8,0.), (1867.,12,15.)))

```

```

if __name__=='__main__':
 from p4_grid import tour_len
 p=compact_layout()
 print(adaptive_complete(p,detail=True))
 print(tour_len(p)[0])

```

## 7.14 Q4/src/p4\_posterior.py

"""Use negative replies for a source estimate, retaining an outer proof region.

The quadrature is a planning prior only. It never excludes geometric source possibilities, stops search, or replaces the final continuous certificate.  
 """

```

import math
import numpy as np

```

```

class P4PosteriorMixin:
 def home_region(self,rec):
 br=super().home_region(rec)
 if br is None or rec.n_bearings!=1 or not br.get('poly') or rec.near_hits:return br
 if not getattr(self.cfg,'posterior_estimate',False):return br
 key=(rec.channel,len(rec.meas_log),len(self._p4_home_caps.get(rec.channel,[])))
 cache=getattr(self,'_posterior_cache',None)
 if cache is None:self._posterior_cache={};cache=self._posterior_cache
 if key in cache:return cache[key]
 station=np.array(rec.pts[0]);angle=math.radians(rec.svds[0])
 u=np.array([math.cos(angle),math.sin(angle)])
 polygon=np.array(br['poly']);along=(polygon-station)@u
 ts=np.linspace(max(5.,along.min()),min(1500.,along.max()),150)
 candidates=station+ts[:,None]*u
 radii=np.linspace(1000,1500,9)
 ds=angle+np.linspace(-math.pi/2+math.pi/72,math.pi/2-math.pi/72,36)
 directions=np.stack([np.cos(ds),np.sin(ds)],axis=1)
 valid=(ts[:,None,None]<=radii[None,:,None])*np.ones((1,1,len(ds)),bool)
 omni=(ts[:,None]<=radii[None,:])
 for x,y,_,res in rec.meas_log:
 if res!='no_signal':continue
 v=candidates-(x,y);d=np.linalg.norm(v,axis=1)
 within=d[:,None]<=radii[None,:]
 bearing=(v@directions.T)>=0
 valid &= ~(within[:,None]&bearing[:,None,:])
 omni &= ~within
 prior=getattr(self.cfg,'posterior_omni_prior',.5)
 weights=ts*(np.linalg.norm(candidates,axis=1)<=1800)
 weights*=prior*omni.mean(axis=1)+(1-prior)*valid.mean(axis=(1,2))
 if weights.sum()>0:
 center=np.average(candidates,axis=0,weights=weights)
 radius=float(np.max(np.linalg.norm(polygon-center,axis=1)))
 br=dict(br,center=tuple(center),radius=radius)
 cache[key]=br
 return br

```

## 7.15 Q4/src/p4\_route\_v2.py

"""Optional stronger open-route planning for Q4; no hidden-state access.

This mixin changes only the proposed ordering. P4SearchRobot still owns all

```

measurements, localization, termination and continuous completion checks.
"""
from __future__ import annotations

import math
from dataclasses import dataclass
import numpy as np
from p4_search import P4SearchConfig, P4SearchRobot

@dataclass
class P4RouteV2Config(P4SearchConfig):
 route_v2_rounds: int = 4
 route_v2_starts: int = 5
 route_service_entry: bool = False
 route_fixed_stations: bool = False
 route_information: bool = False

class P4RouteV2Mixin:
 def _route_information(self, items):
 """Reference-distribution scan latency, used only to rank full tours."""
 if not hasattr(self, '_route_info_points'):
 # Fixed quadrature nodes are independent of mock scenarios/seeds.
 rng=np.random.default_rng(170120)
 n=1024
 angle=rng.uniform(0,2*math.pi,n);radius=1800*np.sqrt(rng.uniform(0,1,n))
 self._route_info_points=np.column_stack([radius*np.cos(angle),radius*np.sin(angle)])
 direct=rng.uniform(0,2*math.pi,n)
 self._route_info_direct=np.column_stack([np.cos(direct),np.sin(direct)])
 self._route_info_range=rng.uniform(1000,1500,n)
 self._route_info_cache={}
 def bits(p):
 p=tuple(p)
 if p not in self._route_info_cache:
 delta=np.asarray(p)-self._route_info_points
 mask=(np.linalg.norm(delta,axis=1)<=self._route_info_range)&(np.sum(delta*self._route_info_direct,axis=1)>=0)
 self._route_info_cache[p]=int.from_bytes(np.packbits(mask).tobytes(),'big')
 return self._route_info_cache[p]
 covered=0
 for p in self.scan_points:covered |= bits(p)
 masks=[None]+[bits(w['xy']) if w['kind']=='station' else None for w in items]
 n=len(self._route_info_points);base=0.35+0.65*(1-covered.bit_count()/n)
 factor=30*len(self.unheard())/base
 return masks,covered,n,factor

 def _fixed_ordered(self, items):
 if not hasattr(self, '_fixed_station_order'):
 scans=[w for w in items if w['kind']=='station']
 self._fixed_station_order=[w['si'] for w in self._strong_ordered(scans)]
 by_station={w['si']:w for w in items if w['kind']=='station'}
 seq=[by_station[i] for i in self._fixed_station_order if i in by_station]
 for w in (w for w in items if w['kind']=='clear'):
 def cost_at(k):
 a=self.pos if k==0 else seq[k-1]['xy']
 b=seq[k]['xy'] if k<len(seq) else None
 return math.dist(a,w['xy'])+(math.dist(w['xy'],b)-math.dist(a,b) if b is not None else 0.)
 k=min(range(len(seq)+1),key=cost_at);seq.insert(k,w)
 # Move targets between gaps while preserving the scan sequence.
 for _ in range(3):
 changed=False

```

```

 for w in list(seq):
 if w['kind']!='clear':continue
 old=seq.index(w);seq.remove(w)
 k=min(range(len(seq)+1),key=lambda k:self._length(seq[:k]+[w]+seq[k:],self.pos))
 seq.insert(k,w);changed |= old!=k
 if not changed:break
 return seq

def _route_key(self, item):
 return (item['kind'], item.get('si', item.get('ch'))))

def _entry_points(self, item):
 c = np.asarray(item['xy'], float)
 if not getattr(self.cfg, 'route_service_entry', False) or item['kind'] != 'clear':
 return c.reshape(1, 2)
 rec = self.recs[item['ch']]
 if rec.n_bearings != 1:
 return c.reshape(1, 2)
 s = np.asarray(rec.pts[-1], float)
 a = math.radians(rec.svds[-1]);u = np.array([math.cos(a), math.sin(a)])
 normal = np.array([-u[1], u[0]])
 along = max(0., float((c-s) @ u))
 forward = self.cfg.home_fraction * along
 lateral = min(self.cfg.home_lateral_m, max(22., .3 * max(along, 1.)))
 midpoint = s + forward * u
 return np.asarray([midpoint-lateral*normal, midpoint+lateral*normal])

def _ordered(self, items):
 if getattr(self.cfg, 'route_fixed_stations', False):
 return self._fixed_ordered(items)
 return self._strong_ordered(items)

def _strong_ordered(self, items):
 if len(items) <= 1:
 return list(items)
 n = len(items)
 pts = np.asarray([self.pos] + [w['xy'] for w in items], float)
 entries = [pts[0].reshape(1, 2)] + [self._entry_points(w) for w in items]
 d = np.zeros((n+1,n+1))
 for j,entry in enumerate(entries):
 d[:,j] = np.min(np.linalg.norm(pts[:,None,:]-entry[None,:,:],axis=2)
 + np.linalg.norm(entry-pts[j],axis=1)[None,:],axis=1)
 np.fill_diagonal(d, 0.)
 info=self._route_information(items) if getattr(self.cfg, 'route_information', False) else None
 symmetric=not getattr(self.cfg, 'route_service_entry', False)

 def cost(seq):
 total=float(np.sum(d[np.asarray([0]+seq[:-1]),seq]))
 if info:
 masks,covered,ns,factor=info
 for j in seq:
 if masks[j] is not None:
 total+=factor*(0.35+0.65*(1-covered.bit_count()/ns))
 covered |= masks[j]
 return total

 def improve(seq):
 seq = list(seq); value = cost(seq)
 for iteration in range(getattr(self.cfg, 'route_v2_rounds', 4)):
 best, best_value = None, value
 # Full objective also works for directed service-entry costs.
 for i in range(n-1):

```

```

 for j in range(i+1,n):
 trial = seq[:i] + seq[i:j+1][::-1] + seq[j+1:]
 if symmetric and info is None:
 a=0 if i==0 else seq[i-1]
 delta=d[a,seq[j]]-d[a,seq[i]]
 if j+1<n:delta+=d[seq[i],seq[j+1]]-d[seq[j],seq[j+1]]
 val=value+delta
 else:val = cost(trial)
 if val < best_value-1e-7:
 best,best_value = trial,val
 for i in range(n):
 node = seq[i];rest=seq[:i]+seq[i+1:]
 a=0 if i==0 else seq[i-1]
 removal=-d[a,node]
 if i+1<n:removal+=d[a,seq[i+1]]-d[node,seq[i+1]]
 for j in range(n):
 if i==j:continue
 trial=rest[:j]+[node]+rest[j:]
 if info is None:
 prev=0 if j==0 else rest[j-1]
 delta=removal+d[prev,node]
 if j<len(rest):delta+=d[node,rest[j]]-d[prev,rest[j]]
 val=value+delta
 else:val=cost(trial)
 if val<best_value-1e-7:
 best,best_value=trial,val
 if best is None:break
 seq,value=best,cost(best)
 return seq,value

A warm start avoids discarding the global path after a small move.
id_to_i = {self._route_key(w):i+1 for i,w in enumerate(items)}
warm = [id_to_i[k] for k in getattr(self,'_route_v2_previous',[]) if k in id_to_i]
for j in range(1,n+1):
 if j in warm:continue
 best = min(range(len(warm)+1), key=lambda k:cost(warm[:k]+[j]+warm[k:]))
 warm.insert(best,j)
starts=[warm]
first_candidates=np.argsort(d[0,1:])[getattr(self.cfg,'route_v2_starts',5)]+1
for first in first_candidates:
 seq=[int(first)];left=set(range(1,n+1))-set(seq)
 while left:
 j=min(left,key=lambda j:d[seq[-1],j]);seq.append(j);left.remove(j)
 starts.append(seq)
seq,value=min((improve(seq) for seq in starts),key=lambda x:x[1])
self._route_v2_previous=[self._route_key(items[i-1]) for i in seq]
return [items[i-1] for i in seq]

class P4RouteV2Robot(P4RouteV2Mixin, P4SearchRobot):
 pass

```

## 7.16 Q4/src/p4\_compact.py

```

"""Compact directional survey with adaptively refined continuous proof."""
from dataclasses import dataclass, asdict
import math
import numpy as np
from p4_search import P4SearchRobot,P4SearchConfig,tier_config
from p4_certificate import ring_layout

```

```

from p4_layout_v2 import adaptive_complete
from p4_route_v2 import P4RouteV2Mixin
from p4_posterior import P4PosteriorMixin

@dataclass
class P4CompactConfig(P4SearchConfig):
 tier: str = 'strict'
 compact_rings: tuple = ((998.,8,0.), (1867.,12,15.))
 stronger_route: bool = True
 route_service_entry: bool = False
 route_v2_rounds: int = 4
 route_v2_starts: int = 5
 route_information: bool = True
 extra_bearings: bool = False
 opportunistic_baseline_m: float = 300.
 defer_single: bool = False
 defer_distance_m: float = 1100.
 defer_angle_deg: float = 70.
 posterior_estimate: bool = True
 posterior_omni_prior: float = .5
 rotate_layout: bool = False
 rotation_candidates: int = 14

class P4CompactRobot(P4PosteriorMixin, P4RouteV2Mixin, P4SearchRobot):
 def __init__(self, arena, cfg=None, **kwargs):
 cfg = cfg or P4CompactConfig()
 if cfg.certify and not adaptive_complete(ring_layout(cfg.compact_rings)):
 raise ValueError('Compact layout lacks a continuous certificate')
 super().__init__(arena, cfg, **kwargs)

 def _build_stations(self):
 return ring_layout(self.cfg.compact_rings, False)

 def _opening_baseline(self):
 if self.cfg.rotate_layout and self.heard_pending():
 base = np.array(self.stations)
 targets = [dict(kind='clear', ch=ch, xy=tuple(self.estimate(ch)[0])) for ch in self.heard_pending()]
 best = None
 # The certified arena is a disk, so a rigid rotation preserves
 # its geometric validity. Pick phase using observed targets only.
 symmetry = math.gcd(*(int(n) for _, n, _ in self.cfg.compact_rings))
 for phi in np.arange(self.cfg.rotation_candidates)*2*math.pi/(symmetry*self.cfg.rotation_candidates):
 mat = np.array([[math.cos(phi), -math.sin(phi)], [math.sin(phi), math.cos(phi)]])
 pts = base @ mat.T
 items = [dict(kind='station', si=i, xy=tuple(p)) for i, p in enumerate(pts)] + targets
 self._route_v2_previous = []
 seq = self._ordered(items)
 cost = self._length(seq, self.pos)
 if best is None or cost < best[0]: best = (cost, pts)
 self.stations = [tuple(p) for p in best[1]]
 self._station_bits = [np.packbits(self.certificate.station_mask(p)) for p in self.stations]
 self._route_v2_previous = []
 return super()._opening_baseline()

 def _ordered(self, items):
 if self.cfg.stronger_route: return P4RouteV2Mixin._ordered(self, items)
 return P4SearchRobot._ordered(self, items)

 def _route(self):
 seq = super()._route()

```

```

 if not self.cfg.defer_single: return seq
 scans=[w for w in seq if w['kind']=='station']
 if not scans: return seq
 keep=[]
 for w in seq:
 if w['kind']!='clear' or self.recs[w['ch']].n_bearings!=1:
 keep.append(w); continue
 rec=self.recs[w['ch']]; c=np.array(w['xy']); old=np.array(rec.pts[-1])-c
 can_defer=False
 for scan in scans:
 q=np.array(scan['xy']); v=q-c; dist=np.linalg.norm(v)
 if dist>self.cfg.defer_distance_m or math.dist(q,rec.pts[-1])<300: continue
 angle=math.degrees(math.acos(float(np.clip((old@v)/max(1e-9,np.linalg.norm(old)*dist),-1,1))))
 if angle<=self.cfg.defer_angle_deg: can_defer=True; break
 if not can_defer: keep.append(w)
 return self._ordered(keep)

 def _certify_remaining(self):
 cache={}
 for rec in self.recs.values():
 if rec.cleared: continue
 if rec.n_bearings or rec.near_hits: return False
 points=tuple(sorted(set(tuple(row[:2]) for row in rec.meas_log if row[3]=='no_signal'))))
 if points not in cache: cache[points]=adaptive_complete(points)
 if not cache[points]: return False
 return True

 def _scan_actual(self,p,extra=True):
 n=super()._scan_actual(p,extra)
 if self.cfg.extra_bearings and extra:
 for ch in self.heard_pending():
 rec=self.recs[ch]
 if rec.n_bearings<2: continue
 c,rad,_=self.estimate(ch)
 if rad<20 or c is None or math.dist(c,p)>1000: continue
 if math.dist(rec.pts[-1],p)<self.cfg.opportunistic_baseline_m: continue
 self.scan_at(*p,[ch])
 return n

def compact_config(tier='strict'):
 """Second-round presets; fast tier labels are empirical targets only."""
 base=tier_config(tier)
 options=asdict(base)
 options.update(stronger_route=True,posterior_estimate=True,route_information=True)
 if tier=='strict':
 options['compact_rings']=((998.,8,0.),(1867.,12,15.))
 else:
 options['compact_rings']=tuple((r,n,0. if k%2==0 else 180./n)
 for k,(r,n) in enumerate(base.search_rings))
 return P4CompactConfig(**options)

```

## 7.17 Q4/src/p4\_polish.py

```

"""Move scan stops inside their exact assigned coverage constraints."""
from dataclasses import dataclass
import math
import numpy as np
from p4_search import P4SearchRobot,P4SearchConfig

```



```

@dataclass
class P4PolishConfig(P4SearchConfig):
 tier: str = 'polish'
 certificate_step_m: float = 20.
 certificate_n_dir: int = 48
 polish_target_duties: bool = True
 polish_rounds: int = 2
 polish_move: bool = True

class P4PolishRobot(P4SearchRobot):
 def __init__(self, arena, cfg=None, **kwargs):
 super().__init__(arena, cfg or P4PolishConfig(), **kwargs)
 self._full=np.packbits(np.ones(self.certificate.shape,bool))

 def _bits(self,w):
 if w['kind']=='station':return self._station_bits[w['si']]
 return np.packbits(self.certificate.station_mask(w['xy']))

 def _slide(self,start,z,missing):
 cert=self.certificate
 mask=np.unpackbits(missing,count=int(np.prod(cert.shape))).reshape(cert.shape)
 ci,ai=np.nonzero(mask)
 if not len(ci):return z
 centers=cert.centers[ci];theta=cert.angles[ai]
 delta=np.array(start)-centers;v=z-start;aa=float(v@v)
 if aa<1e-9:return start
 bb=2*delta@v;cc=np.einsum('ij,ij->i',delta,delta)-(1000-cert.rho)**2
 roots=(-bb+np.sqrt(np.maximum(0,bb*bb-4*aa*cc)))/(2*aa)
 t=float(min(1.,roots.min()))
 for side in [-1,1]:
 angles=theta+side*cert.angle_half_width
 normals=np.column_stack([np.cos(angles),np.sin(angles)])
 slack=np.einsum('ij,ij->i',delta,normals)-cert.rho
 change=normals@v;bad=change<0
 if np.any(bad):t=min(t,float(np.min(slack[bad]/-change[bad])))
 return start+max(0.,t-1e-6)*v

 def _route(self):
 targets=[]
 for ch in self.heard_pending():
 c,_,_=self.estimate(ch)
 if c is not None:targets.append(dict(kind='clear',ch=ch,xy=tuple(c),duty=self.cfg.
polish_target_duties))
 if not self.unheard():return self._ordered(targets)
 scans=[dict(kind='station',si=i,xy=p,duty=True) for i,p in enumerate(self.stations) if i not in self.
visited_stations]
 seq=self._ordered(scans+targets)
 for _ in range(self.cfg.polish_rounds):
 for w in list(seq):
 if not w.get('duty'):continue
 covered=self._covered_bits.copy()
 for v in seq:
 if v is not w and v.get('duty'):covered |= self._bits(v)
 must=self._full & ~covered
 if not np.any(must):
 if w['kind']=='station':seq.remove(w)
 else:w['duty']=False
 continue
 if not self.cfg.polish_move or w['kind']!='station':continue
 # A target may shift after service, so only commit movement

```

```

now; every following plan checks the actual measured mask.
if np.any(must & ~self._bits(w)):continue
i=seq.index(w);a=np.array(self.pos if i==0 else seq[i-1]['xy']);start=np.array(w['xy'])
ends=[a]
if i+1<len(seq):
 b=np.array(seq[i+1]['xy']);v=b-a
 t=np.clip((start-a)@v/max(float(v@v),1e-8),0,1)
 ends=[a+t*v,a,b,(a+b)/2]
 def cost(p):return np.linalg.norm(p-a)+np.linalg.norm(p-b)
else:
 def cost(p):return np.linalg.norm(p-a)
opts=[self._slide(start,z,must) for z in ends]
p=min(opts,key=cost)
if cost(p)<cost(start)-.01:
 bits=np.packbits(self.certificate.station_mask(p))
 if not np.any(must & ~bits):
 w['xy']=tuple(p)
 self.stations[w['si']]=tuple(p);self._station_bits[w['si']]=bits
seq=self._ordered(seq)
covered=self._covered_bits.copy()
for w in seq:
 if w.get('duty'):covered |= self._bits(w)
if np.any(self._full & ~covered):
 # Target coordinates change after localization: restore the
 # original certified anchor set, preserving all actual evidence.
 from p4.certificate import strict_layout
 for p in strict_layout()[1:]:
 if p not in self.stations:
 i=len(self.stations);self.stations.append(p)
 bits=np.packbits(self.certificate.station_mask(p));self._station_bits.append(bits)
 seq.append(dict(kind='station',si=i,xy=p,duty=True));covered |= bits
 if np.any(self._full & ~covered):raise RuntimeError('Coverage repair failed')
 seq=self._ordered(seq)
return seq

```

## 7.18 Q4/src/p4\_arena\_ext.py

"""问题 4 的场景生成（定向 / 全向混合案例）。

单独成一个模块，是为了让 `p4\_run.py`（演练/正式测试入口）、`p4\_selftest.py`（自检）与 `p4\_diag.py`（诊断）用\*\*同一套\*\*案例定义 -- 否则"自检通过但演练漏源"这类问题会被掩盖。

题目口径（附件 2 第 2.2 节 + 附录 1）：

\* 定向源的\*\*有效覆盖角度范围是  $180^\circ$ \*\*  $\Rightarrow$  覆盖半角 ``cone\_half =  $90^\circ$ ``;

\* 定向方向未知，可以是任意角度；

\* 有效接收半径在 1000-1500 m 之间；

\* 干扰源总数 10-16 个（本题按 10-16 均匀取）。

"""

```
from __future__ import annotations
```

```
import math
```

```
import numpy as np
```

```
from p3_arena import MockArena, MockSource, R_ARENA, R_RECV_MAX, R_RECV_MIN
```

```
SCENARIOS = ("directed", "directed_only", "directed_edge", "omni", "uniform", "mixed_uniform", "minrange")
```

```

def directed_case(seed=0, n=None, spec=None, directed_frac=0.5,
 r_recv=(R_RECV_MIN, R_RECV_MAX), budget=360000.0,
 outward_frac=0.5):
 """造一个案例; 返回 ``(arena, sources)``。

 * ``spec`` 给定 ``[(channel, x, y, r_recv, dir_deg), ...]`` 时直接照做（自检用）；
 * ``n`` 不给就随机取 10-16 个源；
 * ``directed_frac``: 定向源比例； ``outward_frac``: 其中"朝向背离原点"的比例
 （这是最难的一类：靶区里的点几乎都落在它背后）。
 """
 if spec is not None:
 srcls = [MockSource(int(c), float(x), float(y), float(rr),
 cone_half=90.0, dir_deg=float(dd))
 for (c, x, y, rr, dd) in spec]
 return MockArena(seed=int(seed), budget_s=float(budget), sources=srcls), srcls

 rng = np.random.default_rng(int(seed))
 k = int(n) if n else int(rng.integers(10, 17))
 k = max(1, min(k, 20))
 chans = rng.choice(np.arange(1, 21), size=k, replace=False)
 srcls = []
 for ch in chans:
 r = R_ARENA * math.sqrt(float(rng.random()))
 a = float(rng.uniform(0.0, 2.0 * math.pi))
 x, y = r * math.cos(a), r * math.sin(a)
 if float(rng.random()) < float(directed_frac):
 d = (math.degrees(a) % 360.0) if float(rng.random()) < float(outward_frac) \
 else float(rng.uniform(0.0, 360.0))
 srcls.append(MockSource(int(ch), x, y, float(rng.uniform(*r_recv)),
 cone_half=90.0, dir_deg=d))
 else:
 srcls.append(MockSource(int(ch), x, y, float(rng.uniform(*r_recv))))
 return MockArena(seed=int(seed), budget_s=float(budget), sources=srcls), srcls

def make_arena(scenario, seed, n_sources=None, budget=360000.0,
 directed_frac=0.5, r_recv=(R_RECV_MIN, R_RECV_MAX)):
 """按场景名造案例（`p4_run.py` 的入口用）。

 ``directed``: 全向/定向混合； ``directed_only``: 全部定向；
 ``directed_edge``: 源偏外圈 $r \in [1500, 1800]$ （专打"贴边朝外"的弱点）；
 ``omni``: 全向（作为对照，检验策略没有为定向源牺牲全向性能）。
 """
 if scenario in ('uniform', 'mixed_uniform', 'minrange'):
 return directed_case(seed, n=n_sources, directed_frac=.5 if scenario=='mixed_uniform' else 1.,
 outward_frac=0., budget=budget,
 r_recv=(1000., 1000.) if scenario=='minrange' else r_recv)[0]
 if scenario == "omni":
 arena, _ = directed_case(seed, n=n_sources, directed_frac=0.0,
 budget=budget, r_recv=r_recv)
 return arena
 if scenario == "directed_only":
 arena, _ = directed_case(seed, n=n_sources, directed_frac=1.0,
 budget=budget, r_recv=r_recv)
 return arena
 if scenario == "directed_edge":
 rng = np.random.default_rng(int(seed) + 991)
 k = int(n_sources) if n_sources else int(rng.integers(10, 17))
 k = max(1, min(k, 20))
 chans = rng.choice(np.arange(1, 21), size=k, replace=False)
 srcls = []
 for ch in chans:

```

```

 r = math.sqrt(rng.uniform(1500.0 ** 2, R_ARENA ** 2))
 a = float(rng.uniform(0.0, 2.0 * math.pi))
 x, y = r * math.cos(a), r * math.sin(a)
 if float(rng.random()) < float(directed_frac):
 d = (math.degrees(a) % 360.0) if float(rng.random()) < 0.5 \
 else float(rng.uniform(0.0, 360.0))
 srcs.append(MockSource(int(ch), x, y, float(rng.uniform(*r_recv)),
 cone_half=90.0, dir_deg=d))
 else:
 srcs.append(MockSource(int(ch), x, y, float(rng.uniform(*r_recv))))
 return MockArena(seed=int(seed), budget_s=float(budget), sources=srcs)
arena, _ = directed_case(seed, n=n_sources, directed_frac=directed_frac,
 budget=budget, r_recv=r_recv)

return arena

```

## 7.19 Q4/src/p4\_search\_bench.py

```

"""Paired offline evaluation for Q4 policies; truth is used only after exit."""
from __future__ import annotations
import argparse
from dataclasses import asdict
import hashlib
import json
import math
from pathlib import Path
import time
import numpy as np
from p3_arena import MockArena, MockSource
from p4_arena_ext import directed_case, make_arena
from p4_robot import P4Config, P4GridRobot

def scenario_arena(scenario, seed):
 if scenario in ('mixed', 'directed', 'omni'):
 return directed_case(seed, directed_frac={'mixed':.5, 'directed':1., 'omni':0.}[scenario])[0]
 if scenario in ('uniform', 'mixed_uniform', 'minrange'):
 return directed_case(seed, directed_frac=.5 if scenario=='mixed_uniform' else 1.,
 outward_frac=0., r_recv=(1000., 1000.) if scenario=='minrange' else (1000., 1500.))[0]
 if scenario == 'edge':
 rng=np.random.default_rng(seed)
 sources=[]
 for ch in rng.choice(np.arange(1,21), size=int(rng.integers(10,17)), replace=False):
 a=rng.uniform(0,2*math.pi); r=rng.uniform(1750,1800)
 # In the protocol's station->source convention, this orientation
 # can be heard only on the outside half-plane.
 sources.append(MockSource(int(ch), r*math.cos(a), r*math.sin(a), 1000.,
 cone_half=90., dir_deg=math.degrees(a)+180.))
 return MockArena(seed=seed, sources=sources, budget_s=360000.)
 raise ValueError(scenario)

def policy_robot(name, arena, overrides=None):
 if name.startswith('legacy'):
 cfg=P4Config()
 if name=='legacy_l2':cfg.outer_rings=((1850.,9),)
 elif name=='legacy_l3':cfg.grid_a=1100.;cfg.outer_rings=((1900.,4),)
 elif name=='legacy_l4':cfg.outer_rings=()
 elif name=='legacy_l6':cfg.grid_a=1100.;cfg.outer_rings=()
 cls=P4GridRobot
 elif name=='homing':

```

```

 from p4_homing import P4HomingRobot,P4HomingConfig
 cls,cfg=P4HomingRobot,P4HomingConfig()
 else:
 from p4_search import P4SearchRobot, tier_config
 cls,cfg=P4SearchRobot,tier_config(name)
 for k,v in (overrides or {}).items():
 if not hasattr(cfg,k):raise ValueError(k)
 setattr(cfg,k,v)
 return cls(arena,cfg)

def run_one(name,scenario,seed,overrides=None,trace=False):
 a=scenario_arena(scenario,seed)
 rb=policy_robot(name,a,overrides)
 t=time.perf_counter();rep=rb.run()
 row=dict(policy=name,scenario=scenario,seed=seed,cleared=rep['cleared'],
 total=rep['n_sources'],exit_s=rep['virtual_time_s'],
 last_clear_s=rb.stats['vtime_at_last_clear'],travel_m=rep['travel_m'],
 measures=rep['n_measure'],clear_fail=rep['n_clear_fail'],
 rejected=rep['n_rejected'],runtime_s=time.perf_counter()-t,
 completion_certified=rep.get('completion_certified'),
 stations=rep.get('search_stations',len(rb.visited_stations)),
 config=asdict(rb.cfg))
 if trace:row['actions']=a.actions;row['truth']=a.truth();row['events']=rb.events
 return row

def summary(rows):
 out=[]
 for policy,scenario in sorted(set((r['policy'],r['scenario']) for r in rows)):
 rr=[r for r in rows if (r['policy'],r['scenario'])==(policy,scenario)]
 out.append(dict(policy=policy,scenario=scenario,cases=len(rr),
 sources=sum(r['total'] for r in rr),
 cleared=sum(r['cleared'] for r in rr),
 rate=sum(r['cleared'] for r in rr)/sum(r['total'] for r in rr),
 full=sum(r['cleared']==r['total'] for r in rr),
 exit_s=float(np.mean([r['exit_s'] for r in rr])),
 last_s=float(np.mean([r['last_clear_s'] for r in rr])),
 travel_m=float(np.mean([r['travel_m'] for r in rr])),
 measures=float(np.mean([r['measures'] for r in rr])),
 fails=sum(r['clear_fail'] for r in rr),
 runtime_s=float(np.mean([r['runtime_s'] for r in rr]))))
 return out

def main():
 ap=argparse.ArgumentParser()
 ap.add_argument('--policies',default='legacy')
 ap.add_argument('--scenarios',default='mixed,uniform')
 ap.add_argument('--cases',type=int,default=12)
 ap.add_argument('--seed',type=int,default=20260914)
 ap.add_argument('--stride',type=int,default=100)
 ap.add_argument('--out',default='B_locator/Q4/out/p4_search_dev/baseline.jsonl')
 ap.add_argument('--overrides',default='{}')
 ap.add_argument('--trace',action='store_true')
 args=ap.parse_args();path=Path(args.out);path.parent.mkdir(parents=True,exist_ok=True)
 files=['p4_search.py','p4_homing.py','p4_certificate.py','p4_robot.py',
 'p3_robot.py','p3_arena.py','p1_intersection.py','p4_arena_ext.py','p4_search_bench.py']
 root=Path(__file__).parent
 hashes={name:hashlib.sha256((root/name).read_bytes()).hexdigest() for name in files}
 path.with_suffix('.meta.json').write_text(json.dumps(dict(arguments=vars(args),files=hashes,
 started_unix_s=time.time()),indent=2),encoding='utf8')

```

```

rows=[]
with path.open('w',encoding='utf8') as f:
 for name in args.policies.split(','):
 for scenario in args.scenarios.split(','):
 for i in range(args.cases):
 row=run_one(name,scenario,args.seed+args.stride*i,json.loads(args.overrides),args.trace)
 rows.append(row);f.write(json.dumps(row,ensure_ascii=False)+'\n');f.flush()
 print(json.dumps(summary([r for r in rows if r['policy']==name and r['scenario']==scenario])[0],
 ensure_ascii=False),flush=True)
path.with_suffix('.summary.json').write_text(json.dumps(summary(rows),indent=2),encoding='utf8')

if __name__=='__main__':main()

```

## 7.20 Q4/src/p4\_service\_v2.py

```

"""Explore route-aware local service without changing completion evidence."""
from dataclasses import dataclass
import math
import numpy as np
from p4_search import P4SearchRobot,P4SearchConfig

@dataclass
class P4ServiceConfig(P4SearchConfig):
 tier: str = 'service'
 direct_single: bool = True
 direct_fraction: float = 1.
 direct_lateral: float = 0.
 single_estimate_fraction: float = 0.5
 route_center_cross: bool = False

class P4ServiceMixin:
 def estimate(self,ch):
 c,r,br=super().estimate(ch)
 rec=self.recs[ch]
 fraction=getattr(self.cfg,'single_estimate_fraction',.5)
 if c is not None and rec.n_bearings==1 and fraction!=.5:
 s=np.array(rec.pts[-1]);angle=math.radians(rec.svds[-1])
 u=np.array([math.cos(angle),math.sin(angle)])
 t=(np.array(br['poly'])-s)@u
 c=s+u*(t.min()+fraction*(t.max()-t.min()))
 return c,r,br

 def clear_here(self,ch):
 rec=self.recs[ch]
 if rec.n_bearings==1 and not rec.cleared and getattr(self.cfg,'direct_single',True):
 c,_,br=self.estimate(ch)
 if c is not None:
 s=np.array(rec.pts[-1]);p=s+self.cfg.direct_fraction*(c-s)
 u=c-s;u=math.linalg.norm(u,1e-9);normal=np.array([-u[1],u[0]])
 lat=getattr(self.cfg,'direct_lateral',0.)
 opts=[p+lat*normal,p-lat*normal]
 p=min(opts,key=lambda q:math.dist(q,self.pos))
 if lat==0 and self._home_clear(ch,p):return True
 result=self._home_measure(ch,p)
 if rec.cleared:return True
 return super().clear_here(ch)

```

```

class P4ServiceRobot(P4ServiceMixin,P4SearchRobot):
 def __init__(self,arena,cfg=None,**kwargs):
 super().__init__(arena,cfg or P4ServiceConfig(),**kwargs)

```

## 7.21 Q4/src/p4\_improve\_bench.py

```

"""Development/evaluation harness for the second Q4 optimization round."""
from dataclasses import asdict
import argparse
import hashlib
import json
from pathlib import Path
import time
import numpy as np
from p4_search_bench import scenario_arena,policy_robot

def robot_for(policy,arena,overrides=None):
 if policy.startswith('compact_') and policy!='compact_v2':
 from p4_compact import P4CompactRobot,compact_config
 cfg=compact_config(policy.removeprefix('compact_'))
 for key,value in (overrides or {}).items():
 if not hasattr(cfg,key):raise ValueError(key)
 setattr(cfg,key,value)
 rb=P4CompactRobot(arena,cfg)
 elif policy=='adaptive':
 from p4_adaptive_cover import P4AdaptiveRobot,P4AdaptiveConfig
 rb=P4AdaptiveRobot(arena,P4AdaptiveConfig(**(overrides or {})))
 elif policy=='compact_v2':
 from p4_compact import P4CompactRobot,P4CompactConfig
 rb=P4CompactRobot(arena,P4CompactConfig(**(overrides or {})))
 elif policy=='polish':
 from p4_polish import P4PolishRobot,P4PolishConfig
 rb=P4PolishRobot(arena,P4PolishConfig(**(overrides or {})))
 elif policy=='service':
 from p4_service_v2 import P4ServiceRobot,P4ServiceConfig
 rb=P4ServiceRobot(arena,P4ServiceConfig(**(overrides or {})))
 else:rb=policy_robot(policy,arena,overrides)
 return rb

def run_case(policy,scenario,seed,overrides=None,trace=False):
 arena=scenario_arena(scenario,seed);rb=robot_for(policy,arena,overrides)
 t=time.perf_counter();rep=rb.run()
 row=dict(policy=policy,scenario=scenario,seed=seed,cleared=rep['cleared'],total=rep['n_sources'],
 exit_s=rep['virtual_time_s'],last_s=rb.stats['vtime_at_last_clear'],
 travel_m=rep['travel_m'],measures=rep['n_measure'],failures=rep['n_clear_fail'],
 rejected=rep['n_rejected'],certified=rep.get('completion_certified'),
 runtime_s=time.perf_counter()-t,config=asdict(rb.cfg))
 if trace:row.update(actions=arena.actions,truth=arena.truth())
 return row

def summarize(rows):
 per_source=[r['exit_s']/r['cleared'] for r in rows if r['cleared']]
 last_per_source=[r['last_s']/r['cleared'] for r in rows if r['cleared']]
 return dict(cases=len(rows),sources=sum(r['total'] for r in rows),cleared=sum(r['cleared'] for r in rows),
 full=sum(r['cleared']==r['total'] for r in rows),certified=sum(r['certified'] is True for r in
 rows),

```

```

 exit_s=float(np.mean([r['exit_s'] for r in rows])),
 exit_per_source=float(np.mean(per_source)) if per_source else None,
 last_per_source=float(np.mean(last_per_source)) if last_per_source else None,
 travel_m=float(np.mean([r['travel_m'] for r in rows])),
 measures=float(np.mean([r['measures'] for r in rows])),runtime_s=float(np.mean([r['runtime_s']
for r in rows])))

def main():
 ap=argparse.ArgumentParser()
 ap.add_argument('--policies',default='strict,adaptive')
 ap.add_argument('--scenarios',default='mixed_uniform,uniform')
 ap.add_argument('--cases',type=int,default=12)
 ap.add_argument('--seed',type=int,default=20260914)
 ap.add_argument('--stride',type=int,default=100)
 ap.add_argument('--overrides',default='{}')
 ap.add_argument('--out',default='B_locator/Q4/out/p4_improve_dev/initial.jsonl')
 ap.add_argument('--trace',action='store_true')
 args=ap.parse_args();p=Path(args.out);p.parent.mkdir(parents=True,exist_ok=True)
 hashes={f.name:hashlib.sha256(f.read_bytes()).hexdigest() for f in Path(__file__).parent.glob('p4_*.py')}
 p.with_suffix('.meta.json').write_text(json.dumps(dict(args=vars(args),files=hashes),indent=2),encoding='utf8')
 results=[]
 with p.open('w',encoding='utf8') as f:
 for policy in args.policies.split(','):
 for scenario in args.scenarios.split(','):
 rows=[]
 for i in range(args.cases):
 r=run_case(policy,scenario,args.seed+i*args.stride,json.loads(args.overrides),args.trace)
 f.write(json.dumps(r)+'\n');f.flush();rows.append(r)
 result=dict(policy=policy,scenario=scenario,**summarize(rows));results.append(result)
 print(json.dumps(result),flush=True)
 p.with_suffix('.summary.json').write_text(json.dumps(results,indent=2),encoding='utf8')

if __name__=='__main__':main()

```

## 7.22 Q4/run\_paper\_q4.py

```

"""Fresh, paired offline experiments for the Q4 paper (no online requests).

Run from the repository root. Each accepted action and exact source definition
is retained in a gzip trace. The policy receives only the existing arena API;
ground truth is read after run() has exited and is used solely for evaluation.
"""
from __future__ import annotations

import argparse
from collections import defaultdict
from concurrent.futures import ThreadPoolExecutor, as_completed
from dataclasses import asdict
from datetime import datetime, timezone
import gzip
import hashlib
import json
import math
from pathlib import Path
import platform
import sys
import time

```



```

import numpy as np

ROOT = Path(__file__).resolve().parent
SRC = ROOT / "src"
sys.path.insert(0, str(SRC))

from p4_search_bench import scenario_arena
from p4_improve_bench import robot_for

BASELINES = ("legacy", "joint_strict", "compact_strict")
POLICIES = {
 "legacy": ("legacy", {}),
 "joint_strict": ("strict", {}),
 "compact_strict": ("compact_strict", {}),
 "compact_no_posterior": ("compact_strict", {"posterior_estimate": False}),
 "compact_no_information": ("compact_strict", {"route_information": False}),
 "compact_fast99": ("compact_fast99", {}),
 "compact_fast95": ("compact_fast95", {}),
 "compact_fast90": ("compact_fast90", {}),
}

SCENARIO_DESCRIPTIONS = {
 "mixed_uniform": "10-16 sources; position uniform in area in the 1800 m disk; each source directional with probability 0.5; directional bearing uniform on [0, 360); reception radius uniform on [1000, 1500] m.",
 "minrange": "10-16 sources; position uniform in disk area; all directional, uniform bearings; reception radius exactly 1000 m.",
 "edge": "10-16 sources; radial coordinate uniform on [1750, 1800] m and polar angle uniform; all directional with station-to-source bearing set to polar angle + 180 degrees; reception radius 1000 m, so sources can be heard only from their outside half-plane.",
 "omni": "10-16 sources; position uniform in disk area; all omnidirectional; reception radius uniform on [1000, 1500] m.",
}

def sha256(path):
 return hashlib.sha256(Path(path).read_bytes()).hexdigest()

def json_write(path, payload):
 Path(path).write_text(json.dumps(payload, ensure_ascii=False, indent=2, allow_nan=False) + "\n", encoding="utf-8")

def build_tasks(args):
 tasks = []
 main_seeds = [args.seed + args.stride * i for i in range(args.main_cases)]
 for i, seed in enumerate(main_seeds):
 for policy in BASELINES:
 tasks.append(dict(policy=policy, scenario="mixed_uniform", seed=seed, case_index=i, cohort="main"))
 for offset, scenario in enumerate(("minrange", "edge", "omni"), start=1):
 for i in range(args.stress_cases):
 seed = args.seed + offset * 1_000_000 + args.stride * i
 for policy in BASELINES:
 tasks.append(dict(policy=policy, scenario=scenario, seed=seed, case_index=i, cohort="stress"))
 for i, seed in enumerate(main_seeds[args.extra_cases:]):
 for policy in ("compact_no_posterior", "compact_no_information", "compact_fast99", "compact_fast95", "compact_fast90"):
 tasks.append(dict(policy=policy, scenario="mixed_uniform", seed=seed, case_index=i, cohort="ablation" if "no_" in policy else "speed"))
 return tasks

```

```

def run_task(task, out_dir):
 task_start = time.perf_counter()
 arena = scenario_arena(task["scenario"], task["seed"])
 actual_policy, overrides = POLICIES[task["policy"]]
 robot = robot_for(actual_policy, arena, overrides)
 run_start = time.perf_counter()
 report = robot.run()
 runtime = time.perf_counter() - run_start
 setup_runtime = run_start - task_start
 # Evaluation data are acquired after policy exit, never passed into policy.
 sources = [asdict(source) for source in arena.sources]
 initial_sources = [{key: value for key, value in source.items() if key != "cleared"} for source in sources]
 truth_digest = hashlib.sha256(json.dumps(initial_sources, sort_keys=True).encode()).hexdigest()
 actions = arena.actions
 measures = [a for a in actions if a["kind"] == "measure"]
 clears = [a for a in actions if a["kind"] == "clear"]
 successes = [a for a in clears if a["clear_result"] == "success"]
 failures = len(clears) - len(successes)
 travel = sum(a.get("dist_m", 0.0) for a in actions)
 decomposition = dict(move_s=travel / 5.0, measure_s=5.0 * len(measures),
 switch_s=sum(a["switch_s"] for a in measures),
 clear_success_s=5.0 * len(successes), clear_failure_s=3.0 * failures)
 exit_s = float(report["virtual_time_s"])
 ledger_s = sum(a.get("dt", 0.0) for a in actions)
 last_s = float(robot.stats["vtime_at_last_clear"])
 assert actions[0]["kind"] == "enter" and actions[-1]["kind"] == "exit"
 assert math.isclose(sum(decomposition.values()), exit_s, abs_tol=1e-7)
 assert math.isclose(ledger_s, exit_s, abs_tol=1e-7)
 assert math.isclose(travel, report["travel_m"], abs_tol=1e-7)
 assert len(measures) == report["n_measure"]
 assert failures == report["n_clear_fail"]
 assert len(successes) == report["cleared"]
 assert math.isclose(last_s, successes[-1]["virtual_time_s"] if successes else 0.0, abs_tol=1e-7)
 clear_count, total = report["cleared"], report["n_sources"]
 row = dict(**task, total=total, cleared=clear_count, full_clear=clear_count == total,
 exit_s=exit_s, last_s=last_s, exit_per_cleared=exit_s / clear_count if clear_count else None,
 last_per_cleared=last_s / clear_count if clear_count else None,
 tail_s=exit_s-last_s, travel_m=travel, measures=len(measures), clear_attempts=len(clears),
 failures=failures, rejected=report["n_rejected"],
 certified=report.get("completion_certified"),
 certification_mode="continuous" if getattr(robot.cfg, "certify", False) else "not_requested",
 termination=report.get("termination", "legacy_exit"),
 stations=report.get("search_stations", len(robot.visited_stations)),
 runtime_s=runtime, setup_runtime_s=setup_runtime, source_sha256=truth_digest,
 time_components=decomposition, config=asdict(robot.cfg),
 actual_policy=actual_policy, overrides=overrides)
 trace_name = f"{task['scenario']}_{task['seed']}_{task['policy']}.json.gz"
 trace_path = Path(out_dir) / "traces" / trace_name
 trace = dict(row=row, sources=sources, actions=actions, events=robot.events, report=report,
 robot_stats=robot.stats, arena_stats=arena.stats,
 scheduled_stations=[list(p) for p in robot.stations])
 with gzip.open(trace_path, "wt", encoding="utf-8") as stream:
 json.dump(trace, stream, ensure_ascii=False, allow_nan=False)
 row["trace"] = f"traces/{trace_name}"
 row["trace_sha256"] = sha256(trace_path)
 return row

def interval(values):
 return [float(x) for x in np.quantile(values, [.025, .975])]

```

```

def describe(rows, bootstrap_reps):
 size = len(rows)
 rng = np.random.default_rng(2026091209)
 idx = rng.integers(0, size, size=(bootstrap_reps, size))
 cleared = np.array([r["cleared"] for r in rows], dtype=float)
 total = np.array([r["total"] for r in rows], dtype=float)
 exit_times = np.array([r["exit_s"] for r in rows])
 full = cleared == total
 numeric = ["exit_s", "last_s", "exit_per_cleared", "last_per_cleared", "tail_s", "travel_m", "measures", "
 clear_attempts", "failures", "stations", "runtime_s", "setup_runtime_s"]
 means = {key: float(np.mean([r[key] for r in rows if r[key] is not None])) for key in numeric}
 time_components = {key: float(np.mean([r["time_components"][key] for r in rows])) for key in rows[0]["
 time_components"]}
 return dict(cases=size, total=int(total.sum()), cleared=int(cleared.sum()),
 clearance_rate=float(cleared.sum() / total.sum()),
 clearance_rate_ci=interval(cleared[idx].sum(axis=1) / total[idx].sum(axis=1)),
 full_clear_cases=int(full.sum()), full_clear_rate=float(full.mean()),
 certified_cases=sum(r["certified"] is True for r in rows),
 continuous_certified_cases=sum(r["certified"] is True and r["certification_mode"] == "continuous"
 for r in rows),
 exit_s_ci=interval(exit_times[idx].mean(axis=1)),
 exit_per_cleared_ci=interval(np.array([r["exit_per_cleared"] for r in rows])[idx].mean(axis=1)),
 pooled_exit_per_cleared=float(exit_times.sum() / cleared.sum()),
 failures_total=sum(r["failures"] for r in rows), rejected_total=sum(r["rejected"] for r in rows),
 time_components=time_components, runtime_max_s=max(r["runtime_s"] for r in rows), **means)

def paired(before, after, bootstrap_reps):
 aa = {r["seed"]: r for r in before}
 bb = {r["seed"]: r for r in after}
 assert aa.keys() == bb.keys()
 keys = sorted(aa)
 assert all(aa[k]["source_sha256"] == bb[k]["source_sha256"] for k in keys)
 x = np.array([aa[k]["exit_s"] for k in keys])
 y = np.array([bb[k]["exit_s"] for k in keys])
 delta = x - y
 idx = np.random.default_rng(2026091210).integers(0, len(keys), size=(bootstrap_reps, len(keys)))
 last_delta = np.array([aa[k]["last_per_cleared"] - bb[k]["last_per_cleared"] for k in keys])
 return dict(cases=len(keys), before=before[0]["policy"], after=after[0]["policy"],
 saved_s=float(delta.mean()), saved_s_ci=interval(delta[idx].mean(axis=1)),
 saved_fraction=float(delta.mean() / x.mean()),
 saved_fraction_ci=interval(delta[idx].mean(axis=1) / x[idx].mean(axis=1)),
 last_per_cleared_saved_s=float(last_delta.mean()),
 faster_cases=int((delta > 0).sum()),
 both_full_clear_cases=sum(aa[k]["full_clear"] and bb[k]["full_clear"] for k in keys))

def aggregate(rows, args, manifest):
 groups, comparisons = [], []
 for scenario in SCENARIO_DESCRIPTIONS:
 experiment = "main" if scenario == "mixed_uniform" else "stress"
 subset = {p: [r for r in rows if r["policy"] == p and r["scenario"] == scenario] for p in BASELINES}
 for policy, rr in subset.items():
 groups.append(dict(experiment=experiment, policy=policy, scenario=scenario, **describe(rr, args.
 bootstrap)))
 for before, after in (("legacy", "joint_strict"), ("joint_strict", "compact_strict"), ("legacy", "
 compact_strict")):
 comparisons.append(dict(experiment=experiment, scenario=scenario, **paired(subset[before], subset[
 after], args.bootstrap)))
 ref = [r for r in rows if r["policy"] == "compact_strict" and r["scenario"] == "mixed_uniform" and r["
 case_index"] < args.extra_cases]
 for experiment, variants in (("ablation", ("compact_no_posterior", "compact_no_information")),

```

```

 ("speed", ("compact_fast99", "compact_fast95", "compact_fast90"))):
 groups.append(dict(experiment=experiment, policy="compact_strict", scenario="mixed_uniform", **describe(
ref, args.bootstrap)))
 for policy in variants:
 rr = [r for r in rows if r["policy"] == policy]
 groups.append(dict(experiment=experiment, policy=policy, scenario="mixed_uniform", **describe(rr,
args.bootstrap)))
 before, after = (rr, ref) if experiment == "ablation" else (ref, rr)
 comparisons.append(dict(experiment=experiment, scenario="mixed_uniform", **paired(before, after, args
.bootstrap)))
strict = [r for r in rows if r["policy"] in ("joint_strict", "compact_strict")]
return dict(schema_version=1, generated_utc=datetime.now(timezone.utc).isoformat(),
 run_count=len(rows), unique_scenarios=len({r["scenario"], r["seed"]} for r in rows}),
 total_source_exposures=sum(r["total"] for r in rows),
 rejected_total=sum(r["rejected"] for r in rows),
 ledger_audited_cases=len(rows), strict_cases=len(strict),
 strict_full_clear_cases=sum(r["full_clear"] for r in strict),
 strict_continuous_certified_cases=sum(r["certified"] is True for r in strict),
 groups=groups, comparisons=comparisons,
 representative=dict(scenario="mixed_uniform", seed=args.seed, selection="First predeclared main
case, no outcome-based selection."),
 traces={r["policy"]: r["trace"] for r in rows if r["scenario"] == "
mixed_uniform" and r["seed"] == args.seed and r["policy"] in BASELINES}),
 manifest=manifest)

def main():
 parser = argparse.ArgumentParser(description=__doc__)
 parser.add_argument("--out", type=Path, default=ROOT / "out" / "paper_q4")
 parser.add_argument("--main-cases", type=int, default=30)
 parser.add_argument("--stress-cases", type=int, default=10)
 parser.add_argument("--extra-cases", type=int, default=10)
 parser.add_argument("--seed", type=int, default=2026091201)
 parser.add_argument("--stride", type=int, default=79)
 parser.add_argument("--workers", type=int, default=6)
 parser.add_argument("--bootstrap", type=int, default=10000)
 parser.add_argument("--resume", action="store_true")
 parser.add_argument("--summarize-only", action="store_true")
 args = parser.parse_args()
 if not (0 < args.extra_cases <= args.main_cases and args.stress_cases > 0):
 parser.error("Need 0 < extra-cases <= main-cases and positive stress-cases")
 args.out.mkdir(parents=True, exist_ok=True)
 (args.out / "traces").mkdir(exist_ok=True)
 tasks = build_tasks(args)
 config = {k: str(v) if isinstance(v, Path) else v for k, v in vars(args).items() if k not in ("resume", "
summarize_only")}
 code_hashes = {p.name: sha256(p) for p in sorted(SRC.glob("*.py"))}
 code_hashes["../run_paper_q4.py"] = sha256(__file__)
 manifest_path = args.out / "manifest.json"
 rows_path = args.out / "runs.jsonl"
 started = time.perf_counter()
 if args.resume or args.summarize_only:
 manifest = json.loads(manifest_path.read_text(encoding="utf-8"))
 assert manifest["source_sha256"] == code_hashes, "Source changed since run began"
 assert manifest["arguments"] == config, "Reproduction parameters changed"
 rows = [json.loads(line) for line in rows_path.read_text(encoding="utf-8").splitlines()]
 else:
 if rows_path.exists():
 raise FileExistsError(f"Refusing to overwrite {rows_path}; use --resume or another --out")
 manifest = dict(schema_version=1, started_utc=datetime.now(timezone.utc).isoformat(),
 arguments=config, python=sys.version, executable=sys.executable,
 numpy_version=np.__version__, platform=platform.platform(),

```

```

 source_sha256=code_hashes, tasks=tasks, policies=POLICIES,
 scenario_descriptions=SCENARIO_DESCRIPTIONS,
 bootstrap=dict(repetitions=args.bootstrap, unit="whole paired scenario/seed", ci="2.5 and
97.5 percentile bootstrap", summary_seed=2026091209, paired_seed=2026091210),
 primary_metric="Complete enter-to-exit virtual time, including final unknown-channel
exclusion; exit_per_cleared averaged per run.",
 secondary_metric="Last successful clear time divided by cleared count, averaged per run;
not a complete exit time.",
 simulation="Offline MockArena only; no online arena or HTTP access.")
 json_write(manifest_path, manifest)
 rows = []
 existing = {r["policy"], r["scenario"], r["seed"]} for r in rows}
 assert len(existing) == len(rows)
 pending = [t for t in tasks if (t["policy"], t["scenario"], t["seed"]) not in existing]
 if args.summarize_only and pending:
 raise RuntimeError(f"{len(pending)} unfinished tasks")
 if pending:
 with rows_path.open("a", encoding="utf-8") as stream, ProcessPoolExecutor(max_workers=args.workers) as
pool:
 futures = {pool.submit(run_task, task, args.out): task for task in pending}
 for future in as_completed(futures):
 row = future.result()
 stream.write(json.dumps(row, ensure_ascii=False, allow_nan=False) + "\n")
 stream.flush()
 rows.append(row)
 print(json.dumps(dict(done=len(rows), expected=len(tasks), policy=row["policy"], scenario=row["
scenario"], seed=row["seed"],
 cleared=row["cleared"], total=row["total"], exit_s=round(row["exit_s"], 3),
 runtime_s=round(row["runtime_s"], 3)), ensure_ascii=False), flush=True)
 assert len(rows) == len(tasks)
 assert code_hashes == {**{p.name: sha256(p) for p in sorted(SRC.glob("*.py"))}, "../run_paper_q4.py": sha256(
__file__)}
 # Stable ordering makes cross-policy auditing straightforward.
 rows.sort(key=lambda r: (r["scenario"], r["seed"], r["policy"]))
 rows_path.write_text("".join(json.dumps(r, ensure_ascii=False, allow_nan=False) + "\n" for r in rows),
encoding="utf-8")
 manifest.update(finished_utc=datetime.now(timezone.utc).isoformat(), last_invocation_wall_s=time.perf_counter
()-started,
 runs_sha256=sha256(rows_path), actual_run_count=len(rows))
 json_write(manifest_path, manifest)
 result = aggregate(rows, args, manifest)
 json_write(args.out / "summary.json", result)
 # Convenient uncompressed trace for the fixed first-case paper figure.
 for policy, relative_path in result["representative"]["traces"].items():
 with gzip.open(args.out / relative_path, "rt", encoding="utf-8") as stream:
 json_write(args.out / f"representative_{policy}.json", json.load(stream))
 print(json.dumps({key: result[key] for key in ("run_count", "strict_cases", "strict_full_clear_cases", "
strict_continuous_certified_cases", "rejected_total")}, flush=True)

if __name__ == "__main__":
 main()

```

## 7.23 Q4/scripts/make\_paper\_figures.py

"""Render Q4's five paper figures from frozen evidence and model geometry.

Run ``python B\_locator/Q4/scripts/make\_paper\_figures.py`` from the repository root. The renderer does not run a robot, alter evidence, or consult live APIs. All result panels use ``Q4/out/paper\_q4``; the two geometric illustrations are

```

explicitly constructed examples. Output width is 160 mm, with vector PDF and
330 dpi PNG counterparts in ``paper/figures``. ``--geometry-only`` is useful
before an evidence batch has finished. Mathematical glyphs use STIX and all
Chinese text uses the paper's embedded SimSun font.
"""
from __future__ import annotations

import argparse
import json
import math
from pathlib import Path
import sys

import numpy as np
import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt
from matplotlib import font_manager
from matplotlib.lines import Line2D
from matplotlib.patches import Arc, Circle, Polygon, Rectangle, Wedge

ROOT = Path(__file__).resolve().parents[2]
DATA = ROOT / "Q4" / "out" / "paper_q4"
OUT = ROOT / "paper" / "figures"
sys.path.insert(0, str(ROOT / "Q4" / "src"))
from p4_certificate import strict_layout
from p4_layout_v2 import compact21_layout

MM = 1 / 25.4
FIG_W = 160 * MM
DPI = 330
INK = "#263846"
GRAY = "#87919d"
TEAL = "#118a8c"
BLUE = "#3b82a0"
GOLD = "#b7791f"
RED = "#b33e36"
LIGHT = "#e5f1f0"
POLLS = ["legacy", "joint_strict", "compact_strict"]
LABEL = dict(legacy="grid", joint_strict="joint", compact_strict="compact")
COLOR = dict(legacy=GRAY, joint_strict=BLUE, compact_strict=TEAL)

def setup_style():
 font = ROOT / "paper" / "fonts" / "simsun.ttc"
 font_manager.fontManager.addfont(str(font))
 matplotlib.rcParams.update({
 "font.family": [font_manager.FontProperties(fname=font).get_name(), "Times New Roman"],
 "mathtext.fontset": "stix", "font.size": 9.0,
 "axes.titlesize": 9.3, "axes.labelsize": 9.0,
 "xtick.labelsize": 8.5, "ytick.labelsize": 8.5, "legend.fontsize": 8.5,
 "axes.spines.top": False, "axes.spines.right": False,
 "axes.unicode_minus": False, "pdf.fonttype": 42, "ps.fonttype": 42,
 })

def save(fig, stem):
 OUT.mkdir(parents=True, exist_ok=True)
 fig.savefig(OUT / f"{stem}.pdf")
 fig.savefig(OUT / f"{stem}.png", dpi=DPI)
 plt.close(fig)

```

```

def arrow(ax, a, b, color=INK, **kw):
 ax.annotate("", xy=b, xytext=a,
 zorder=8, arrowprops=dict(arrowstyle="→", lw=.85, color=color,
 shrinkA=1, shrinkB=0, mutation_scale=8, **kw))

def geometrical_axes(ax, xlim, ylim):
 ax.set(xlim=xlim, ylim=ylim, aspect="equal")
 ax.axis("off")

def fig_geometry():
 fig, axes = plt.subplots(1, 3, figsize=(FIG_W, 75*MM))
 fig.subplots_adjust(left=.015, right=.985, bottom=.12, top=.86, wspace=.10)
 ax, bx, cx = axes
 ax.set_title("(a) 定向源的接收半平面", pad=8)
 geometrical_axes(ax, (-1.33, 1.33), (-1.7, 1.45))
 ax.add_patch(Wedge((0, 0), 1.1, -90, 90, fc=LIGHT, ec="none"))
 ax.add_patch(Circle((0, 0), 1.1, fill=False, ec=GRAY, lw=.8, ls="--"))
 ax.plot([0, 0], [-1.1, 1.1], color=TEAL, lw=.9)
 ax.plot(0, 0, "*", color=INK, ms=7)
 ax.text(-.08, -.25, "p", ha="right")
 arrow(ax, (.04, 0), (.69, 0), TEAL)
 ax.text(.40, .11, "d", color=TEAL)
 ax.plot(-.43, .18, "x", color=RED, ms=5)
 ax.text(-.55, .39, "近点仍可无信号", ha="center", fontsize=8.5)
 ax.plot(.59, -.47, "o", color=TEAL, ms=3.8)
 ax.text(.53, -.71, "可接收测点", ha="center", fontsize=8.5)
 ax.text(0, 1.23, "距离条件与朝向条件同时成立", ha="center", fontsize=8.5)
 ax.text(0, -1.34, r"$|q-p|\leq r, \text{quad } (q-p)\cdot d\geq 0$", ha="center")
 ax.text(0, -1.60, "单个负观测不能排除整个圆盘", ha="center", fontsize=8.5)

 bx.set_title("(b) 方格余量与排除朝向", pad=8)
 geometrical_axes(bx, (-1.30, 1.30), (-1.7, 1.45))
 c = np.array([-1.68, .55]); s = np.array([.95, .55]); rho = .32
 bx.add_patch(Rectangle(c-rho/math.sqrt(2), rho*math.sqrt(2), rho*math.sqrt(2),
 fc=LIGHT, ec=TEAL, lw=.8))
 bx.add_patch(Circle(c, rho, fc="none", ec=TEAL, ls=":", lw=.8))
 bx.plot(*c, "o", color=TEAL, ms=3); bx.plot(*s, "s", color=INK, ms=4)
 bx.plot([c[0],s[0]],[c[1],s[1]], color=INK, lw=.7)
 bx.text(c[0]-.04, c[1]-.45, "c", ha="center")
 bx.text(s[0]+.10, s[1], "q", va="center")
 bx.text(.14, .66, r"$D_q=|q-c|$", ha="center")
 bx.text(-.63, 1.06, r"$\rho=h/\sqrt{2}$", ha="center")
 bx.text(0, -.05, r"$D_q+\rho\leq 1000\ \{\rm m\}$", ha="center")
 ac = np.array([-1.10, -.85]); ar = .48
 half = math.degrees(math.acos(rho / np.linalg.norm(s-c)))
 bx.add_patch(Circle(ac, ar, fc="none", ec="#c8ced4", lw=.8))
 bx.add_patch(Arc(ac, 2*ar, 2*ar, theta1=-half, theta2=half, ec=TEAL, lw=3))
 for a in [-half, half]:
 p=ac+ar*np.array([math.cos(math.radians(a)),math.sin(math.radians(a))])
 bx.plot([ac[0],p[0]],[ac[1],p[1]], color=TEAL, lw=.6)
 arrow(bx,ac,ac+np.array([.70,0]),INK)
 bx.text(.17,-.80,r"α",ha="center")
 bx.text(-.73,-.91,"朝向圆",ha="center",fontsize=8.5)
 bx.text(0,-1.56,r"$\alpha=\arccos(\rho/D_q)$",ha="center")

 cx.set_title("(c) 25站压缩为21站", pad=8)
 geometrical_axes(cx, (-2250, 2250), (-2870, 2420))
 p=np.asarray(strict_layout()); q=np.asarray(compact21_layout())
 cx.add_patch(Circle((0,0),1800,fill=False,ec=GRAY,ls="--",lw=.7))

```

```

cx.scatter(p[1:,0],p[1:,1],s=16,facecolors="none",edgecolors=GOLD,lw=.7,label="joint: 25站")
cx.scatter(q[1:,0],q[1:,1],s=11,color=TEAL,label="compact: 21站")
cx.plot(0,0,"s",ms=3.5,color=INK)
cx.text(0,-2250,"未证明方格逐级细分",ha="center",fontsize=8.5)
cx.text(0,-2670,"40 → 20 → 10 → 5 → 2.5 m",ha="center",fontsize=8.5)
cx.legend(loc="upper center",bbox_to_anchor=(.5,1.09),ncol=1,frameon=False,
 handletextpad=.3,labelsacing=.25,fontsize=8.3)
fig.text(.50,.035,"负观测只排除带余量证明的“位置-朝向”状态; 布局认证后仍须逐频道实际检测。",
 ha="center",fontsize=8.5)
save(fig,"q4-geometry-certificate")

def posterior_weights(negatives):
 """Same 150 x 9 x 36 planning quadrature as p4_posterior, at S=(0,0)."""
 ts=np.linspace(5,1500,150)
 candidates=np.column_stack((ts,np.zeros_like(ts)))
 radii=np.linspace(1000,1500,9)
 angles=np.linspace(-math.pi/2+math.pi/72,math.pi/2-math.pi/72,36)
 directions=np.column_stack((np.cos(angles),np.sin(angles)))
 valid=(ts[:,None,None]<=radii[None,:,None])*np.ones((1,1,36),bool)
 omni=ts[:,None]<=radii[None,:]
 for xy in negatives:
 v=candidates-np.asarray(xy)
 within=np.linalg.norm(v,axis=1)[:,None]<=radii[None,:]
 bearing=v@directions.T>=0
 valid &= ~(within[:,None]&bearing[:,None,:]); omni &= ~within
 w=ts*(.5*omni.mean(axis=1)+.5*valid.mean(axis=(1,2)))
 return ts,w/w.sum()

def fig_homing():
 fig,axes=plt.subplots(1,3,figsize=(FIG_W,73*MM))
 fig.subplots_adjust(left=.025,right=.985,bottom=.32,top=.80,wspace=.31)
 ax,bx,cx=axes
 fig.text(.170,.91,"(a) 双侧短基线与双负截断",ha="center",fontsize=9.3)
 geometrical_axes(ax,(-.08,1.08),(-.40,.52))
 # The drawn 10.05 degree wedge is an explicit tenfold angular enlargement.
 eps=math.radians(10.05); t=.45; lat=.18
 ax.add_patch(Polygon([[0,0],[1,math.tan(eps)],[1,-math.tan(eps)]]),
 fc=LIGHT,ec=TEAL,lw=.6))
 ax.add_patch(Polygon([[t,t*math.tan(eps)],[1,math.tan(eps)],[1,-math.tan(eps)],[t,-t*math.tan(eps)]]),
 fc="#f5e7df",ec="none"))
 ax.plot([t,t],[-lat,lat],color=RED,lw=1.2)
 ax.plot([0,1.03],[0,0],ls="--",color=GRAY,lw=.6)
 ax.plot(0,0,"o",color=TEAL,ms=4); ax.text(-.02,-.11,"$$",ha="center")
 for sign in [-1,1]:
 ax.plot(t,sign*lat,"x",color=RED,ms=5)
 ax.text(t+.03,sign*(lat+.06),"Q_+" if sign>0 else "Q_-",ha="left")
 ax.text(.75,.40,"双负时排除远端",ha="center",fontsize=8.4)
 ax.text(.22,.07,"t",ha="center"); ax.text(.40,.13,r"ℓ",ha="right")
 ax.text(.50,-.32,"任一侧正观测 → 新角楔交会",ha="center",fontsize=8.4)

 fig.text(.505,.91,"(b) 单方位后验用于估价",ha="center",fontsize=9.3)
 ts,w0=posterior_weights([]); _,w1=posterior_weights([(550,180)])
 step=ts[1]-ts[0]
 bx.plot(ts,w0/step*1000,color=GRAY,ls="--",lw=1,label="仅正观测")
 bx.plot(ts,w1/step*1000,color=TEAL,lw=1.2,label="再有负观测")
 bx.fill_between(ts,w1/step*1000,color=TEAL,alpha=.10)
 for w,col in [(w0,GRAY),(w1,TEAL)]:
 bx.axvline(float(ts@w),color=col,ls=":",lw=.85)
 bx.set(xlim=(0,1550),ylim=(0,None),xlabel="沿方位距离 / m",ylabel="密度 / 10^{-3}")

```



```

bx.set_xticks([0,750,1500]);bx.set_yticks([0,1,2])
bx.grid(alpha=.15, lw=.4)
bx.legend(loc="upper left", frameon=False, handlelength=1.2, handletextpad=.4, fontsize=8.1)

fig.text(.839,.91, "(c) 清除圆盘覆盖兜底", ha="center", fontsize=9.3)
geometrical_axes(cx, (-30,85), (-42,89))
poly=np.array([[-13,-9],[59,5],[49,45],[-4,34]])
cx.add_patch(Polygon(poly,fc=LIGHT,ec=TEAL,lw=1.1,zorder=3))
for x in (0,28,56):
 for y in (0,28,56):
 cx.add_patch(Circle((x,y),20,fc="none",ec=BLUE,lw=.55,alpha=.72,zorder=4))
 cx.plot(x,y,".",color=BLUE,ms=3,zorder=5)
cx.annotate("x",xy=(28,-25),xytext=(0,-25),arrowprops=dict(arrowstyle="|-|",color=INK,lw=.75))
cx.text(14,-36,"28 m",ha="center",fontsize=8.5)
cx.text(28,82,"清除半径20 m",ha="center",fontsize=8.5)
fig.text(.50,.100,r"(a) 误差角示意放大10倍; 实际 $\epsilon=1.005^\circ$ 。(b) 负测点为(550,180) m, 竖线为均值。",ha="center",fontsize=8.3)
fig.text(.50,.035,"后验均值不缩小保守可行域; 兜底依靠清除半径, 不依赖信号接收方向。",ha="center",fontsize=8.5)
save(fig,"q4-homing-posterior")

def load_summary(data):
 obj=json.loads((data/"summary.json").read_text(encoding="utf-8"))
 assert obj["run_count"]==230, "This paper batch must contain all 230 runs"
 return obj

def group(summary,experiment,policy,scenario="mixed_uniform"):
 rows=[r for r in summary["groups"] if r["experiment"]==experiment and
 r["policy"]==policy and r["scenario"]==scenario]
 assert len(rows)==1,(experiment,policy,scenario)
 return rows[0]

def fig_routes(data):
 traces={p:json.loads((data/f"representative_{p}.json").read_text(encoding="utf-8")) for p in POLS}
 assert len({t["row"] for t in traces.values()})==1
 fig,axes=plt.subplots(1,3,figsize=(FIG_W,76*MM))
 fig.subplots_adjust(left=.095,right=.985,bottom=.30,top=.76,wspace=.11)
 for i,pol in enumerate(POLS):
 ax=axes[i];tr=traces[pol];r=tr["row"]
 aa=[a for a in tr["actions"] if "x" in a and "y" in a]
 pts=np.array([[0,0]]+[a["x"],a["y"]] for a in aa)
 ax.plot(pts[:,0],pts[:,1],lw=.9,color=COLOR[pol],zorder=3)
 meas=np.unique(np.array([a["x"],a["y"]] for a in aa if a["kind"]=="measure")),axis=0
 ax.scatter(meas[:,0],meas[:,1],s=4,color="#d7a06b",alpha=.7,zorder=4)
 for s in tr["sources"]:
 x,y=s["x"],s["y"]
 ax.plot(x,y,"*",color=INK,ms=4.2,zorder=6)
 if s["cone_half"]<180:
 # Mock dir_deg is station -> source; source -> receiving side is reversed.
 a=math.radians(s["dir_deg"]+180)
 arrow(ax,(x,y),(x+310*math.cos(a),y+310*math.sin(a)),INK)
 for ok,marker in [(True,"x"),(False,"^")]:
 pp=np.array([a["x"],a["y"]] for a in aa if a["kind"]=="clear" and (a["clear_result"]=="success")==ok
])
 if len(pp):
 if ok:
 ax.scatter(pp[:,0],pp[:,1],marker=marker,s=19,color=RED,linewidths=.7,zorder=7)
 else:
 ax.scatter(pp[:,0],pp[:,1],marker=marker,s=14,facecolors="none",edgecolors=GOLD,linewidths=.7,zorder=7)

```

```

ax.add_patch(Circle((0,0),1800,fill=False,ec="#aeb5bd",ls="--",lw=.65))
ax.plot(0,0,"s",ms=3.2,color=INK,zorder=9)
ax.set_title(f"({chr(97+i)}) {LABEL[pol]}\nT={r['exit_s']:.0f} s; L={r['travel_m']/1000:.2f} km\n清除 {r['cleared']}/{r['total']}",fontsize=9,linespacing=1.3,pad=7)
ax.set(xlim=(-2100,2100),ylim=(-2100,2100),aspect="equal",xlabel="x / m")
ax.set_xticks([-1500,0,1500]);ax.set_yticks([-1500,0,1500]);ax.grid(alpha=.15,lw=.4)
if i==0:ax.set_ylabel("y / m")
else:ax.tick_params(labelleft=False)
handles=[Line2D([],[],color="#d7a06b",marker="o",ls="",ms=3,label="检测点"),
Line2D([],[],color=RED,marker="x",ls="",ms=4,label="清除成功"),
Line2D([],[],color=GOLD,marker="^",mfc="none",ls="",ms=4,label="清除失败"),
Line2D([],[],color=INK,marker="*",ls="",ms=5,label="源位"),
Line2D([],[],color=INK,marker="s",ls="",ms=3,label="起点")]
fig.legend(handles,loc="lower center",bbox_to_anchor=(.5,.092),ncol=5,
frameon=False,columnspacing=.8,handlelength=1,handletextpad=.35)
fig.text(.5,.020,"同一预定首局; 源位及箭头均为退出后真值, 箭头表示可接收半平面的中心方向。",ha="center",fontsize=8.3)
save(fig,"q4-route-generations")

def fig_results(summary):
fig,(ax,bx)=plt.subplots(1,2,figsize=(FIG_W,74*MM),gridspec_kw={"width_ratios":[1,1.12]})
fig.subplots_adjust(left=.12,right=.975,bottom=.33,top=.84,wspace=.55)
parts=[("move_s",BLUE,"移动"),("measure_s","#d97736","检测"),
("switch_s","#7656a6","切频"),("clear","#55a868","清除")]
values=[]
for p in POLS:
d=group(summary,"main",p)["time_components"]
values.append([d["move_s"],d["measure_s"],d["switch_s"],d["clear_success_s"]+d["clear_failure_s"]])
values=np.asarray(values);left=np.zeros(3)
for j,_,c,l in enumerate(parts):
ax.barh(np.arange(3),values[:,j],left=left,color=c,label=l,height=.5);left+=values[:,j]
for y,v in enumerate(left):ax.text(v+100,y,f"{v:.0f}",va="center",fontsize=8.4)
ax.set_yticks(range(3),[LABEL[p] for p in POLS]);ax.invert_yaxis()
ax.set(xlim=(0,10000),xlabel="平均完整时间 / s")
ax.set_xticks([0,4000,8000]);ax.grid(axis="x",alpha=.16,lw=.4)
ax.set_title("(a) 主集30局: 动作时间分解",pad=10)
fig.legend(*ax.get_legend_handles_labels(),loc="lower center",bbox_to_anchor=(.31,.095),ncol=4,frameon=False,
columnspacing=.65,handlelength=.8,handletextpad=.3,fontsize=8.3)
scenes=["minrange","edge","omni"];names=["最小接收半径","贴边朝外","全向源"]
for j,p in enumerate(POLS):
xx=[];yy=[];lo=[];hi=[]
for i,sc in enumerate(scenes):
r=group(summary,"stress",p,sc);v=r["exit_s"]
xx.append(v);yy.append(i+(j-1)*.22);lo.append(v-r["exit_s_ci"][0]);hi.append(r["exit_s_ci"][1]-v)
bx.errorbar(xx,yy,xerr=[lo,hi],fmt=["s","^","o"][j],ms=4,color=COLOR[p],
lw=.9,capsize=2,label=LABEL[p])
bx.set_yticks(range(3),names);bx.invert_yaxis();bx.set_ylim(2.55,-.5)
bx.set(xlim=(4600,15900),xlabel="平均完整时间 / s")
bx.set_xticks([5000,9000,13000]);bx.grid(axis="x",alpha=.16,lw=.4)
bx.set_title("(b) 压力集: 每场景10局",pad=10)
fig.legend(*bx.get_legend_handles_labels(),loc="lower center",bbox_to_anchor=(.79,.095),ncol=3,frameon=False,
columnspacing=.75,handlelength=1.0,handletextpad=.3,fontsize=8.3)
fig.text(.5,.035,"压力图误差线为95% bootstrap区间: grid 在贴边朝外场景仅清除94.81%, 其余组均全清。",
ha="center",fontsize=8.3)
save(fig,"q4-results-cost")

def fig_ablation(summary):
fig,(ax,bx)=plt.subplots(1,2,figsize=(FIG_W,71*MM),gridspec_kw={"width_ratios":[1.06,1]})
fig.subplots_adjust(left=.17,right=.97,bottom=.27,top=.85,wspace=.49)
variants=["compact_no_posterior","compact_no_information"]

```

```

labels=["取消后验估计","取消信息通路"]
for i,p in enumerate(variants):
 r=next(r for r in summary["comparisons"] if r["experiment"]=="ablation" and r["before"]==p)
 v=r["saved_s"];lo,hi=r["saved_s_ci"]
 ax.errorbar(v,i,xerr=[[v-lo],[hi-v]],fmt="o",color=BLUE,ecolor=INK,ms=5,capsize=3,lw=1)
 ax.annotate(f"{v:+.0f} s",(v,i),xytext=(0,12),textcoords="offset points",ha="center",fontsize=8.6)
ax.axvline(0,lw=.7,color=GRAY);ax.set_yticks([0,1],labels);ax.invert_yaxis()
ax.set(xlim=(-180,850),ylim=(1.6,-.65),xlabel="相对 compact 的耗时增加 / s")
ax.set_xticks([0,400,800]);ax.grid(axis="x",alpha=.16,lw=.4)
ax.set_title("(a) 配对10局: 组件消融",pad=10)
tiers=["compact_fast90","compact_fast95","compact_fast99","compact_strict"]
pts=[]
for i,p in enumerate(tiers):
 r=group(summary,"speed",p);x=r["exit_s"];y=100*r["clearance_rate"]
 lo,hi=np.array(r["clearance_rate_ci"])*100
 c=[GOLD,BLUE,"#7656a6",TEAL][i]
 bx.errorbar(x,y,yerr=[[y-lo],[hi-y]],fmt="o",ms=4.5,color=c,capsize=3,lw=.9)
 label=p.removeprefix("compact_").replace("strict","strict")
 offset=[(0,-17),(0,10),(-5,-17),(1,10)][i]
 bx.annotate(label,(x,y),xytext=offset,textcoords="offset points",ha="center",fontsize=8.5)
 pts.append((x,y))
pts=np.array(pts);bx.plot(pts[:,0],pts[:,1],color=GRAY,lw=.6,ls="--",zorder=0)
bx.set(xlim=(3950,5950),ylim=(91.7,101.6),xlabel="平均完整时间 / s",ylabel="累计清除率 / %")
bx.set_xticks([4000,4800,5600]);bx.set_yticks([92,96,100]);bx.grid(alpha=.16,lw=.4)
bx.set_title("(b) 同一10局: 速度档取舍",pad=10)
fig.text(.5,.090,"误差线: 整局配对/重采样的95%区间。消融区间跨零,表示此样本尚不能确认该组件的独立收益。",
 ha="center",fontsize=8.3)
fig.text(.5,.028,"速度档名称是配置标签: 样本清除率不能替代 strict 的连续完成认证。",ha="center",fontsize=8.5)
save(fig,"q4-ablation-tradeoff")

def main():
 global OUT
 ap=argparse.ArgumentParser(description=__doc__)
 ap.add_argument("--data",type=Path,default=DATA)
 ap.add_argument("--out",type=Path,default=OUT)
 ap.add_argument("--geometry-only",action="store_true")
 args=ap.parse_args();OUT=args.out
 setup_style();fig_geometry();fig_homing()
 if not args.geometry_only:
 summary=load_summary(args.data)
 fig_routes(args.data);fig_results(summary);fig_ablation(summary)
 print(f"Wrote Q4 paper figures to {OUT}; width=160 mm, PNG={DPI} dpi")

if __name__=="__main__":
 main()

```

## 7.24 Q4/src/p4\_run.py

"""B 题问题 4: 运行入口（离线演练 / 真实模拟器 / 几何自检）。

```

1) 几何自检: 三角形格网的覆盖与"定向盲区"（不跑模拟器）
python src/p4_run.py --geometry

2) 离线演练（定向 + 全向混合; 正式测试只有 3 次机会, 调参必须在这里做）
python src/p4_run.py --mode mock --cases 10 --scenario directed --verbose
python src/p4_run.py --mode mock --cases 30 --scenario omni # 全向对照
python src/p4_run.py --mode mock --cases 10 --scenario directed --out out/p4 --figure

```

```

3) 真实模拟器（先在模拟器里登录、开始"问题4 演练测试"，等接口就绪）
python src/p4_run.py --mode live --robot-id <参赛队号> --url http://127.0.0.1:2026

统计口径与题目表 1 一致：
* 被清除干扰源个数的比例 = 被清除个数 / 干扰源总数；
* 平均定位清除时间 = 定位清除总时间 / 被清除干扰源个数
 （总时间含移动、频道切换、检测、光学精确定位与清除，取最后一个源被清除的时刻）；
* 完整退出每源时间 = /exit 时的虚拟时间 / 被清除源数（本轮优化主指标）；
* 程序运行时间 = 程序自身跑完的墙钟时间。
"""
from __future__ import annotations

import argparse
import csv
import json
import math
import os
import statistics as st
import sys
import time

_HERE = os.path.dirname(os.path.abspath(__file__))
if _HERE not in sys.path:
 sys.path.insert(0, _HERE)
_ROOT = os.path.normpath(os.path.join(_HERE, ".."))

for _s in (sys.stdout, sys.stderr):
 try:
 _s.reconfigure(encoding="utf-8", errors="replace")
 except Exception:
 pass

import numpy as np # noqa: E402

from p3_arena import (# noqa: E402
 ArenaError, HttpArena, MockArena, MockSource, R_ARENA, R_CLEAR, R_RECV_MAX,
 R_RECV_MIN, V_ROBOT,
)

from p3_robot import P3Robot # noqa: E402
from p4_arena_ext import SCENARIOS, directed_case, make_arena # noqa: E402
from p4_robot import P4Config, P4GridRobot # noqa: E402

OUT_DEFAULT = os.path.normpath(os.path.join(_ROOT, "out", "p4"))
DEFAULT_VIRTUAL_BUDGET = 360000.0 # 与真实模拟器一致（虚拟世界限时 100 小时）
LIVE_REAL_BUDGET = 1200.0 # 现实时间预算（/enter 返回）

当前最优策略与档位：compact = 21 站紧凑布局 + 后验估计 + 信息排路；
strict = 100% 档（离线配对：混合源 5933 s、全定向 6491 s，360 局 4668 源全部清除并认证）。
速度档 fast99/98/95/90 以牺牲清除率为代价换时间（贴边朝外场景 fast98 及以下清除率 0%），
正式测试**不要**当默认用；见 review/p4-compact-design.md。
DEFAULT_POLICY = 'compact'
DEFAULT_TIER = 'strict'

场景（定义搬到了 p4_arena_ext，自检/诊断共用同一套）

def make_robot(arena, cfg=None, log=None):
 from p4_search import P4SearchConfig, P4SearchRobot
 from p4_compact import P4CompactConfig, P4CompactRobot, compact_config
 if cfg is None:
 cfg=compact_config('strict')

```

```

 if isinstance(cfg,P4CompactConfig):
 return P4CompactRobot(arena,cfg,log=log)
 if isinstance(cfg,P4SearchConfig):
 return P4SearchRobot(arena,cfg,log=log)
 return P4GridRobot(arena, cfg or P4Config(), base=None, log=log)

def write_json(path, obj):
 os.makedirs(os.path.dirname(os.path.abspath(path)), exist_ok=True)
 with open(path, "w", encoding="utf-8") as f:
 json.dump(obj, f, ensure_ascii=False, indent=2, default=_json_default)
 return path

def _json_default(o):
 if isinstance(o, (np.floating, np.integer)):
 return o.item()
 if isinstance(o, np.ndarray):
 return o.tolist()
 return str(o)

def run_mock_case(seed, cfg, scenario="directed", outdir=None, verbose=False,
 figure=False, budget_s=None, n_sources=None,directed_frac=.5):
 arena = make_arena(scenario, seed, n_sources=n_sources,
 budget=(DEFAULT_VIRTUAL_BUDGET if budget_s is None
 else budget_s),directed_frac=directed_frac)
 rb = make_robot(arena, cfg, log=(lambda m: print(" " + m)) if verbose else None)
 rep = rb.run()
 rep['exit_per_cleared_s']=(rep['virtual_time_s']/rep['cleared'] if rep['cleared'] else None)
 rep["seed"] = seed
 rep["scenario"] = scenario
 rep["n_directed"] = sum(1 for s in arena.sources if s.cone_half < 180.0)
 rep["stations_total"] = len(rb.stations)
 rep["stations_visited"] = len(rb.visited_stations)
 rep["stations"] = [[round(x, 1), round(y, 1)] for (x, y) in rb.stations]
 if outdir:
 d = os.path.join(outdir, f"case_{scenario}_{seed}")
 write_json(os.path.join(d, "summary.json"), rep)
 with open(os.path.join(d, "actions.jsonl"), "w", encoding="utf-8") as f:
 for a in arena.actions:
 f.write(json.dumps(a, ensure_ascii=False, default=_json_default) + "\n")
 write_json(os.path.join(d, "robot_events.json"), rb.events)
 if figure:
 try:
 rep["figure"] = plot_case(arena, rb, rep, os.path.join(d, "p4_case.png"))
 except Exception as e:
 rep["figure_error"] = repr(e)
 return rep

def plot_case(arena, robot, rep, path):
 import matplotlib
 matplotlib.use("Agg")
 import matplotlib.pyplot as plt

 fig, ax = plt.subplots(figsize=(8.6, 8.6))
 th = np.linspace(0, 2 * math.pi, 720)
 ax.plot(R_ARENA * np.cos(th), R_ARENA * np.sin(th), "k--", lw=1.0,
 label="target area R=1800 m")
 for s in rep.get("truth", {}).get("sources", []):

```

```

 col = "tab:green" if s["cleared"] else "tab:red"
 ax.plot([s["x_m"], s["y_m"]], "o", color=col, ms=8, zorder=6)
 ax.annotate(f"ch{s['channel']}", (s["x_m"], s["y_m"]), fontsize=8,
 xytext=(4, 4), textcoords="offset points")

格网站位
st = np.asarray(rep.get("stations", []), float).reshape(-1, 2)
if st.size:
 ax.plot(st[:, 0], st[:, 1], "-", color="tab:orange", ms=6, alpha=0.75,
 label="triangle lattice stations")
pts = [(a["x"], a["y"]) for a in getattr(arena, "actions", [])
 if a.get("kind") in ("measure", "clear")]
if pts:
 P = np.array([[0.0, 0.0]] + pts)
 ax.plot(P[:, 0], P[:, 1], "-", color="tab:blue", lw=0.9, alpha=0.75,
 label="robot path")
for ch, rec in robot.recs.items():
 for (x, y), a in zip(rec.pts, rec.svds):
 L = 900.0
 ax.plot([x, x + L * math.cos(math.radians(a))],
 [y, y + L * math.sin(math.radians(a))],
 color="tab:purple", lw=0.5, alpha=0.4)
ax.plot([0], [0], "k*", ms=13, label="start (0,0)")
ax.set_aspect("equal")
ax.set_xlim(-1950, 1950)
ax.set_ylim(-1950, 1950)
ax.grid(alpha=0.25)
ax.legend(loc="upper right", fontsize=8)
ax.set_title(f"Q4 drill seed={rep.get('seed')} scenario={rep.get('scenario')}\n"
 f"cleared {rep['cleared']}/{rep['n_sources']} "
 f"vtime={rep['virtual_time_s']:.0f}s "
 f"avg clear={rep['avg_clear_time_s']:.1f}s "
 f"travel={rep['travel_m']:.0f} m", fontsize=9)
fig.tight_layout()
os.makedirs(os.path.dirname(os.path.abspath(path)), exist_ok=True)
fig.savefig(path, dpi=150)
plt.close(fig)
return path

def aggregate(reps):
 cl = [r["cleared"] for r in reps]
 tot = [r["n_sources"] for r in reps]
 tms = [r["avg_clear_time_s"] for r in reps if r["avg_clear_time_s"]]
 return {
 "cases": len(reps),
 "n_sources_total": int(sum(tot)),
 "n_cleared_total": int(sum(cl)),
 "clear_ratio_total": (sum(cl) / sum(tot)) if sum(tot) else None,
 "full_clear_cases": sum(1 for r in reps if r["cleared"] == r["n_sources"]),
 "certified_complete_cases": sum(r.get('completion_certified') is True for r in reps),
 "cleared_mean": st.mean(cl) if cl else None,
 "avg_clear_time_mean_s": st.mean(tms) if tms else None,
 "avg_clear_time_median_s": st.median(tms) if tms else None,
 "virtual_time_mean_s": st.mean(r["virtual_time_s"] for r in reps),
 "exit_per_cleared_mean_s": st.mean(r["virtual_time_s"]/r["cleared"] for r in reps if r["cleared"]) if any
 (cl) else None,
 "runtime_mean_s": st.mean(r["program_runtime_s"] for r in reps),
 "travel_mean_m": st.mean(r["travel_m"] for r in reps),
 "n_measure_mean": st.mean(r["n_measure"] for r in reps),
 "stations_mean": st.mean(r["stations_visited"] for r in reps),
 "n_rejected": int(sum(r["n_rejected"] for r in reps)),
 }

```

```

 "n_clear_fail": int(sum(r["n_clear_fail"] for r in reps)),
 }

def print_table(rows, cols, title=None):
 if title:
 print(f"\n=== {title} ===")
 w = {c: max([len(c)] + [len(f"{r.get(c, '')}") for r in rows]) for c in cols}
 print(" ".join(c.rjust(w[c]) for c in cols))
 print(" ".join("-" * w[c] for c in cols))
 for r in rows:
 cells = []
 for c in cols:
 v = r.get(c)
 if v is None:
 s = ""
 elif isinstance(v, float):
 s = f"{v:.4f}" if abs(v) < 10 else f"{v:.1f}"
 else:
 s = str(v)
 cells.append(s.rjust(w[c]))
 print(" ".join(cells))

def geometry_report(args):
 """几何自检：三角形格网的覆盖半径 + 定向盲区（不跑模拟器）。"""
 if getattr(args, 'policy', 'compact') == 'compact':
 from p4_compact import compact_config
 from p4_layout_v2 import adaptive_complete
 rb = make_robot(_DummyArena(), compact_config(args.tier))
 points = [(0., 0.)] + rb.stations
 complete, refinement = adaptive_complete(points, detail=True)
 result = {'policy': 'compact', 'tier': args.tier, 'stations': len(points),
 'continuous_arbitrary_orientation_coverage': complete,
 'refinement': refinement}
 print(json.dumps(result, ensure_ascii=False, indent=2))
 return result
 if getattr(args, 'policy', 'grid') == 'joint':
 from p4_search import P4SearchRobot, tier_config
 from p4_certificate import DirectionalCertificate
 cfg = tier_config(args.tier)
 rb = P4SearchRobot(_DummyArena(), cfg)
 cert = DirectionalCertificate(20, 48)
 points = [(0., 0.)] + rb.stations
 complete = cert.continuous_complete(points)
 result = {'policy': 'joint', 'tier': args.tier, 'stations': len(points),
 'continuous_arbitrary_orientation_coverage': complete}
 print(json.dumps(result, ensure_ascii=False, indent=2))
 return result
 from p4_grid import cov_stats, dir_cov_stats, eval_grid
 cfg = P4Config()
 rb = P4GridRobot(_DummyArena(), cfg)
 stations = [(0.0, 0.0)] + list(rb.stations)
 gx, gy = eval_grid(20.0)
 c, cu = cov_stats(stations, gx, gy)
 dw, db, hd = dir_cov_stats(stations, gx, gy, n_dir=16)
 print(f"三角形格网 a={cfg.grid_a:.0f} m + 外环 "
 f"{[f'r{r:.0f}x{n}' for r, n in (cfg.outer_rings or ())]}"
 f": 站位 {len(stations)} 个 (含原点) ")
 print(f" 覆盖半径 (到最近站位) 最大 {c:.0f} m, 未覆盖占比 {cu:.2%}"
 f" [判据 ≤ {R_RECV_MIN:.0f} m ⇒ 全向源必被听到]")

```

```

print(f" 最坏朝向下最近可听站位 {dw:.0f} m, 定向盲区 {db:.2%}, "
 f"半平面硬伤 {hd:.2%} [离散位置/方向采样, 不是连续覆盖证明]")
print("\n站位分布 (按半径): ")
ring = {}
for (x, y) in stations:
 r = math.hypot(x, y)
 k = 0 if r < 1 else int(round(r / cfg.grid_a))
 ring.setdefault(k, []).append(r)
for k in sorted(ring):
 rs = ring[k]
 print(f" 第 {k} 层: {len(rs):>3} 个, 半径 {min(rs):>7.0f}--{max(rs):>7.0f} m"
 + (" (原点)" if k == 0 else ""))
return {"n_stations": len(stations), "cov_max_m": c, "cov_unc": cu,
 "dir_worst_m": dw, "dir_blind": db, "halfplane_hard": hd}

class _DummyArena:
 """只为构造站位列表用的假 arena (不调用任何接口)。"""
 pos = (0.0, 0.0)
 channel = 1
 virtual_time_s = 0.0

 def remaining_budget_s(self):
 return 1e9

 def budget_kind(self):
 return "virtual"

def main():
 ap = argparse.ArgumentParser(description="问题4 联合航路与清除率配置")
 ap.add_argument('--policy', choices=['compact', 'joint', 'grid'], default=DEFAULT_POLICY,
 help=f'compact=紧凑布局与信息排路 (默认, 当前最优) | joint=上一版 | grid=旧策略')
 ap.add_argument('--tier', choices=['strict', 'fast99', 'fast98', 'fast95', 'fast90'], default=DEFAULT_TIER,
 help='strict=全清档 (默认, 360 局全部清除并认证); fast* 用清除率换时间, '
 '速度档名称是离线标定目标, 非逐局清除率保证, 正式测试勿用')
 ap.add_argument("--mode", choices=["mock", "live"], default="mock")
 ap.add_argument("--cases", type=int, default=10)
 ap.add_argument("--seed", type=int, default=20260914)
 ap.add_argument("--scenario", default="directed", choices=list(SCENARIOS))
 ap.add_argument("--directed-frac", type=float, default=0.5)
 ap.add_argument("--n-sources", type=int, default=0, help="0 = 随机 10-16 个")
 ap.add_argument("--out", default=OUT_DEFAULT)
 ap.add_argument("--figure", action="store_true")
 ap.add_argument("--verbose", action="store_true")
 ap.add_argument("--budget", type=float, default=DEFAULT_VIRTUAL_BUDGET)
 ap.add_argument("--grid-a", type=float, default=P4Config.grid_a)
 ap.add_argument("--outer-rings", default=None,
 help="外侧补漏环, 格式 r1:n1,r2:n2: 默认取 P4Config 的值, "
 "留空字符串表示不加外环")
 ap.add_argument("--spiral-w", type=float, default=0.35)
 ap.add_argument("--plan-tour", default="0",
 help="仅 grid: 0 = 逐站贪心走完旧模型判定的有用站位; "
 "1 = 最小覆盖集 + 半径偏置 TSP (快 ~10%, 有 ~0.4% 漏源风险)")
 ap.add_argument("--replan-every", type=int, default=0,
 help="每走这么多站重排一次航线 (0 = 不重排)")
 ap.add_argument("--geometry", action="store_true", help="只做几何自检")
 ap.add_argument("--url", default="http://127.0.0.1:2026")
 ap.add_argument("--robot-id", default="")
 ap.add_argument("--timeout", type=float, default=5.0)
 args = ap.parse_args()

```



```

if args.outer_rings is None:
 rings = tuple(P4Config.outer_rings)
else:
 rings = []
 for spec in (args.outer_rings or "").split(","):
 spec = spec.strip()
 if spec:
 r, n = spec.split(":")
 rings.append((float(r), int(n)))
 rings = tuple(rings)
cfg = P4Config(grid_a=args.grid_a, outer_rings=rings,
 spiral_radius_w=args.spiral_w,
 plan_tour=bool(int(args.plan_tour)),
 replan_every=int(args.replan_every))

if args.policy=='joint':
 from p4_search import tier_config
 cfg=tier_config(args.tier)
elif args.policy=='compact':
 from p4_compact import compact_config
 cfg=compact_config(args.tier)

if args.geometry:
 geometry_report(args)
 return

if args.mode == "live":
 if not args.robot_id:
 raise SystemExit("live 模式必须提供 --robot-id (参赛队号)")
 d = os.path.join(args.out, time.strftime("live_%Y%m%d_%H%M%S"))
 arena = HttpArena(base_url=args.url, robot_id=args.robot_id,
 timeout=args.timeout,
 log_path=os.path.join(d, "protocol.jsonl"))
 rb = make_robot(arena, cfg, log=print)
 print(f"连接模拟器 {args.url} (队号 {args.robot_id}); ")
 f"若接口尚未开放会自动等待重试..."
 try:
 rep = rb.run()
 rep['exit_per_cleared_s']=(rep['virtual_time_s']/rep['cleared'] if rep['cleared'] else None)
 except ArenaError as e:
 print(f"\n[失败] {e}")
 print("请确认: ① 模拟器已在线登录; ② 已开始『问题4 演练测试/正式测试』; ")
 "③ 5 秒倒计时结束、界面提示机器狗接口已就绪; ④ 端口一致。")
 write_json(os.path.join(d, "error.json"), {"error": repr(e), "url": args.url})
 raise SystemExit(2)
 rep["url"] = args.url
 rep["scenario"] = "live"
 write_json(os.path.join(d, "summary.json"), rep)
 print(json.dumps({k: v for k, v in rep.items()
 if k not in ("truth", "mock_stats", "channels", "stations")},
 ensure_ascii=False, indent=2, default=_json_default))
 print(f"\n结果已写入 {d}")
 return

seeds = [args.seed + 100 * i for i in range(args.cases)]
reps = []
t0 = time.time()
for s in seeds:
 rep = run_mock_case(s, cfg, scenario=args.scenario, outdir=args.out,
 verbose=args.verbose, figure=args.figure,
 budget_s=args.budget,
 n_sources=(args.n_sources or None), directed_frac=args.directed_frac)
 reps.append(rep)

```

```

exit_avg=(f"{rep['exit_per_cleared_s']:.1f}" if rep['exit_per_cleared_s'] is not None else '-')
print(f" seed={s} 清除 {rep['cleared']}/{rep['n_sources']}"
 f" (定向 {rep['n_directed']}) | 定位清除 {rep['avg_clear_time_s']:.1f} s"
 f" | 完整退出 {rep['virtual_time_s']:.0f} s"
 f" (每源 {exit_avg} s) | 检测 {rep['n_measure']}"
 f" | 站位 {rep['stations_visited']}/{rep['stations_total']}"
 f" | 行程 {rep['travel_m']:.0f} m", flush=True)

agg = aggregate(reps)
agg["wall_s"] = round(time.time() - t0, 2)
print()
print(json.dumps(agg, ensure_ascii=False, indent=2, default=_json_default))
write_json(os.path.join(args.out, f"benchmark_{args.scenario}.json"),
 {"cfg": cfg.__dict__, "seeds": seeds, "scenario": args.scenario,
 "aggregate": agg,
 "cases": [{k: v for k, v in r.items()
 if k not in ("truth", "channels", "stations")} for r in reps]})
with open(os.path.join(args.out, f"benchmark_{args.scenario}.csv"), "w",
 newline="", encoding="utf-8-sig") as f:
 w = csv.writer(f)
 cols = ["seed", "n_sources", "n_directed", "cleared", "clear_ratio",
 "avg_clear_time_s", "virtual_time_s", "exit_per_cleared_s", "program_runtime_s",
 "n_measure", "stations_visited", "n_locate", "n_clear",
 "n_clear_fail", "travel_m", "n_rejected"]
 w.writerow(cols)
 for r in reps:
 w.writerow([r.get(c) for c in cols])
print(f"\n结果已写入 {args.out}")

if __name__ == "__main__":
 main()

```

## 7.25 Q4/scripts/run\_p4.py

"""B 题问题 4: 运行入口（转发到 ``src/p4\_run.py``）。

本机 ``scripts/`` 目录存在 ACL 限制（子进程无法读取该目录下的文件），  
所以真正可执行的实现放在 ``src/p4\_run.py``；本文件只是保持目录约定的转发入口。

```

python scripts/run_p4.py --mode mock --cases 10
python src/p4_run.py --mode mock --cases 10 # 等价的直接调用
"""
from __future__ import annotations

import os
import sys

_HERE = os.path.dirname(os.path.abspath(__file__))
_SRC = os.path.normpath(os.path.join(_HERE, "..", "src"))
if _SRC not in sys.path:
 sys.path.insert(0, _SRC)

from p4_run import main # noqa: E402

if __name__ == "__main__":
 main()

```